

台灣自編大學教科書

@prof

地圖建構與同步定位

Simultaneous Localization and Mapping

SLAM

李世淵（機器人叫獸） 編著

助教：李天宇、李宇晴

大學智慧機器人學程

高中 AI 機器人社團課程

書籍資訊

書名	地圖建構與同步定位 (Simultaneous Localization and Mapping, SLAM)
編著者	李世淵 (機器人叫獸)
教學助理	李天宇、李宇晴
適用對象	大學智慧機器人學程、高中 AI 機器人社團課程
全書架構	九篇、三十一章
聯絡方式	LINE @prof (https://page.line.me/prof)
版本	台灣自編大學教科書系列

本書為教學用自編教材，內容涵蓋 SLAM 的理論、系統與實作。書中引用之開源專案、公開資料集與學術文獻，其著作權均屬原作者所有；相關延伸閱讀清單列於各章末，供讀者查閱原始出處。

序言

這本書的起點，是一個看似簡單的問題：一台機器人被放進一個陌生的房間，它如何知道自己在哪裡？

對人類而言，這幾乎不需要思考。我們一眼掃過房間，就能建立空間感、認出門的位置、知道自己站在哪個角落。但對機器人來說，這是一個需要感測器、數學、演算法與工程整合才能回答的難題——而且是過去四十年間，全球最頂尖的機器人研究者持續投入的核心課題。這個問題有一個名字：同步定位與建圖，SLAM。

【本書的緣起】

筆者在大學智慧機器人學程講授機器人導航相關課程多年，一直面臨同一個困難：中文的 SLAM 教材，要不是翻譯自英文而術語與台灣學界慣用法不一致，就是偏重數學推導而缺乏系統層面的視野；適合高中 AI 機器人社團課程的入門教材，更是幾乎空白。本書因此而生，目標是寫一本以繁體中文為母語、從觀念到實作、同時服務大學部與高中兩種讀者的完整教材。

【本書的架構】

全書分為九篇、三十一章，依循一條清楚的學習路徑：

第一篇建立觀念，說明 SLAM 要解決什麼問題、國際頂尖大學如何教這門學問。第二篇介紹機器人的六大感官——從里程計、IMU、光達到相機，逐一分析它們能做什麼、又會在什麼情況下失效。第三篇補足數學工具：座標轉換、機率估測與圖優化。

第四篇到第六篇是三套完整的 SLAM 系統：2D 光達 SLAM (含掃地機器人)、3D 光達 SLAM (含自主巡檢)、以及視覺 SLAM (從特徵到 ORB-SLAM3)。第七篇走向極限平台——毫瓦級的微型機器人、無人機與群體協作。

第八篇回到實作：ROS 環境建置、光達建圖導航、視覺 SLAM，以及完整的專題與評量標準。第九篇則展望未來，討論 AI 與視覺語言模型如何讓機器人從「知道形狀」進化到「理解意義」。

【如何使用這本書】

本書刻意設計成可依讀者層次選讀。高中課程可著重第一、二、四篇與各章的實作活動，多數實驗都能以模擬完成，不需昂貴硬體。大學部建議完整修習第一至六篇，並以第八篇的實作章節搭配課程進行。專題生與研究生則可直接進入第五、六、七、九篇，並以第三十章的專題設計作為研究起點。

每一章都採相同結構：學習目標、核心概念、完整課文、公式或流程、案例、實作活動、延伸閱讀與小結。其中「案例」與「常見失敗」的討論，是本書刻意強調的部分——筆者相信，理解一項技術在什麼情況下會失敗，比記住它在理想條件下的表現更有價值。

【貫穿全書的一句話】

如果要用一句話概括這三十一章，那就是：理解限制，才能突破限制。

書中反覆討論每一種感測器的弱點、每一個演算法的失效模式、每一個平台的資源約束。這不是悲觀，而是因為工程的本質就是在限制中求解。而人工智慧的加入並未改

變這個本質——視覺語言模型讓機器人聽得懂、想得通，但走得穩、不撞人，仍然要靠紮實的幾何與控制。

【致謝】

感謝助教李天宇、李宇晴協助教材整理與實驗驗證；感謝歷屆修課同學以他們的提問、錯誤與突破，讓這本書的每一個案例都有真實的教學現場作為依據。也感謝全球 SLAM 社群長年開源分享的傳統——本書所介紹的系統，從 Gmapping、Cartographer 到 ORB-SLAM3，全部公開可取得。正是這個傳統，讓今日台灣的任何一位學生，都能站在世界頂尖研究的肩膀上。

最後，願讀者帶著這本書打下的基礎，去建造屬於自己的機器人——無論它是巡檢校園的機器狗、掃描古蹟的無人機，還是一台只會沿線走的小車。每一台能自己找到路的機器人，背後都是同一套人類花了四十年才建立起來的智慧。

現在，輪到你了。

李世淵（機器人叫獸）

大學智慧機器人學程

目錄

第一篇 SLAM 與自主導航概論

第 1 章	什麼是地圖建構與同步定位	25
1.1	SLAM 的基本定義：Localization + Mapping	
1.2	為什麼「定位」與「建圖」是雞生蛋、蛋生雞問題	
1.3	機器人如何感知世界：里程計、IMU、相機、光達、無線電、地磁	
1.4	SLAM 與自動駕駛、無人機、掃地機器人、AR / VR 的關係	
1.5	從教室到校園：高中與大學場域的 SLAM 教學案例	
1.6	SLAM 系統輸出：軌跡、2D 地圖、3D 點雲、語意地圖、拓樸地圖	
第 2 章	世界知名大學 SLAM 課程導覽	48
2.1	MIT VNAV：視覺導航、自駕車、無人機與最佳化	
2.2	CMU Robot Localization and Mapping：機器人定位、建圖與後端最佳化	
2.3	Stanford Robot Autonomy：自主機器人的定位、感知、SLAM、規劃	
2.4	ETH Zürich / UZH：視覺慣性里程計、VIO、MAV 無人機導航	
2.5	University of Bonn / Freiburg：Robot Mapping 與 Cyrill Stachniss SLAM 課程	
2.6	SLAM 教材的共同架構：感測器 → 數學 → 前端 → 後端 → 閉環 → 專題	
第 3 章	機器人導航系統總覽	67
3.1	Sense–Think–Act：感知、決策、行動	
3.2	地圖的種類：Occupancy Grid、Feature Map、Point Cloud、Semantic Map	
3.3	定位的種類：相對定位、絕對定位、地圖定位、重定位	
3.4	導航系統架構：SLAM、路徑規劃、避障、控制	
3.5	ROS / ROS 2 在 SLAM 教學中的角色	
3.6	從掃地機器人到自駕車：不同平台的導航需求	

第二篇 定位與導航感測器

第 4 章 室內定位技術	87
4.1 為什麼室內不能只靠 GPS	
4.2 Wi-Fi RSSI 指紋定位	
4.3 Bluetooth Beacon / BLE 定位	
4.4 UWB 超寬頻定位	
4.5 RFID / NFC / QR Code 地標定位	
4.6 視覺地標與 AprilTag / ArUco 定位	
4.7 室內定位與 SLAM 的融合	
第 5 章 無線電導航與衛星導航	102
5.1 GPS / GNSS 基本原理	
5.2 三角定位、三邊定位與時間差定位	
5.3 RTK-GNSS 與高精度戶外定位	
5.4 5G / LoRa / Wi-Fi RTT 在機器人定位的應用	
5.5 無線電定位誤差來源：多路徑、遮蔽、干擾	
5.6 無線電導航與 IMU / SLAM 的融合	
5.7 校園巡檢機器人的室內外混合導航	
第 6 章 地磁導航與電子羅盤	117
6.1 地球磁場與磁力計	
6.2 電子羅盤與航向角	
6.3 室內地磁指紋定位	
6.4 鐵磁干擾與硬鐵 / 軟鐵校正	
6.5 地磁導航在手機、無人機、機器狗中的角色	
6.6 地磁、IMU、視覺 SLAM 的互補關係	
第 7 章 IMU 慣性導航	132
7.1 加速度計、陀螺儀、磁力計	
7.2 姿態估測：Roll、Pitch、Yaw	
7.3 積分漂移問題	
7.4 姿態濾波：Complementary Filter、Kalman Filter、EKF	

- 7.5 IMU 預積分概念
- 7.6 視覺慣性 SLAM 中的 IMU 角色
- 7.7 ORB-SLAM3 與 IMU 緊耦合

第 8 章 光達 LiDAR 介紹 150

- 8.1 光達的基本原理：Time of Flight 與掃描
- 8.2 2D 光達、3D 光達、多線雷射
- 8.3 光達資料格式：LaserScan 與 PointCloud
- 8.4 光達解析度、角解析度、量測距離與反射率
- 8.5 光達在室內建圖、倉儲 AGV、自駕車中的應用
- 8.6 光達與相機、IMU 的比較
- 8.7 低成本光達與教學型機器人平台

第 9 章 相機與視覺導航感測器 165

- 9.1 針孔相機模型
- 9.2 鏡頭畸變與相機校正
- 9.3 單目相機、雙目相機、RGB-D 相機
- 9.4 特徵點、邊緣、光流、深度估測
- 9.5 視覺導航的優點與限制
- 9.6 從影像處理到 Visual SLAM
- 9.7 VLM / AI 視覺模型與語意導航初探

第三篇 SLAM 數學基礎

第 10 章 座標系統與剛體運動 180

- 10.1 世界座標、機器人座標、感測器座標
- 10.2 齊次座標
- 10.3 2D 位姿： x 、 y 、 θ
- 10.4 3D 位姿：位置與姿態
- 10.5 旋轉矩陣、歐拉角、四元數
- 10.6 座標轉換與 TF Tree

10.7 ROS 中的 map、odom、base_link、camera_link

第 11 章 機率、雜訊與狀態估測 195

- 11.1 為什麼 SLAM 一定要處理不確定性
- 11.2 高斯分布、均值、變異數、共變異數
- 11.3 感測器雜訊與運動模型
- 11.4 Bayes Filter
- 11.5 Kalman Filter
- 11.6 Extended Kalman Filter, EKF-SLAM
- 11.7 Particle Filter 與 Monte Carlo Localization

第 12 章 最佳化與圖論 SLAM 212

- 12.1 前端與後端的分工
- 12.2 Pose Graph 的概念
- 12.3 節點、邊、限制條件
- 12.4 Least Squares 最小平方
- 12.5 Bundle Adjustment
- 12.6 g2o、Ceres、GTSAM
- 12.7 為什麼現代 SLAM 常用圖優化

第四篇 2D SLAM 與掃地機器人

第 13 章 2D 地圖建構 227

- 13.1 Occupancy Grid Map
- 13.2 佔據機率的更新:對數勝算比
- 13.3 雷射掃描匹配
- 13.4 里程計與光達融合
- 13.5 地圖解析度與更新
- 13.6 走廊、教室、辦公室的 2D 建圖
- 13.7 地圖品質評估:變形、斷裂、漂移

第 14 章 Gmapping : 粒子濾波 SLAM 242

- 14.1 Gmapping 的基本概念
- 14.2 Rao-Blackwellized Particle Filter
- 14.3 粒子數、權重與重採樣
- 14.4 Gmapping 的兩項關鍵改良
- 14.5 ROS Gmapping 實作流程
- 14.6 常見失敗案例:輪子打滑、場景重複、光達遮蔽
- 14.7 教學實作:用小車建立教室地圖

第 15 章 Cartographer : Google 2D / 3D SLAM 258

- 15.1 Cartographer 系統架構
- 15.2 Submap 子地圖
- 15.3 Scan Matching
- 15.4 閉環偵測與全域最佳化
- 15.5 2D Cartographer 實作
- 15.6 3D Cartographer 與背包式空間建模
- 15.7 Cartographer 與 Gmapping 比較

第 16 章 掃地機器人 SLAM 274

- 16.1 掃地機器人為什麼需要 SLAM
- 16.2 隨機碰撞式掃地 vs SLAM 規劃式掃地
- 16.3 光達掃地機器人與 2D 光達建圖
- 16.4 房間分割、禁區設定、路徑覆蓋
- 16.5 掃地機器人的感測器:光達、IMU、碰撞、懸崖感測器
- 16.6 家庭場景中的 SLAM 挑戰:桌椅、鏡面、低矮障礙物
- 16.7 教學活動:從掃地路線理解 Coverage Path Planning

第五篇 3D 光達 SLAM 與自主巡檢

第 17 章 3D 點雲與光達建圖 290

- 17.1 3D 點雲資料
- 17.2 Voxel Grid 降採樣

- 17.3 ICP 點雲配準
- 17.4 NDT Normal Distributions Transform
- 17.5 3D 地圖與室內空間掃描
- 17.6 點雲顏色化與彩色視覺建模
- 17.7 工廠、校園、展場的 3D 空間建模

第 18 章 LOAM 與光達里程計 306

- 18.1 LOAM : LiDAR Odometry and Mapping
- 18.2 邊緣特徵與平面特徵
- 18.3 高頻里程計與低頻地圖最佳化
- 18.4 LeGO-LOAM、LIO-SAM、Fast-LIO 簡介
- 18.5 光達 + IMU 緊耦合
- 18.6 自主巡檢機器人的 3D SLAM
- 18.7 大學場域專題:走廊、電梯、實驗室巡檢

第六篇 Visual SLAM 與視覺導航機器人

第 19 章 Visual SLAM 入門 323

- 19.1 視覺 SLAM 的基本流程
- 19.2 特徵法與直接法
- 19.3 ORB、SIFT、SURF、FAST、BRIEF
- 19.4 Feature Tracking 與 Feature Matching
- 19.5 Monocular SLAM 的尺度不確定性
- 19.6 Stereo SLAM 與 RGB-D SLAM
- 19.7 視覺 SLAM 在 AR / VR 與機器人中的應用

第 20 章 對極幾何與相機運動估測 340

- 20.1 兩張影像如何推估相機移動
- 20.2 Epipolar Geometry
- 20.3 Essential Matrix 與 Fundamental Matrix
- 20.4 PnP 問題

20.5	三角測量	
20.6	RANSAC 排除錯誤匹配	
20.7	從影像處理課程銜接到視覺 SLAM	
第 21 章	ORB-SLAM2 理論與實作	358
21.1	ORB 特徵提取	
21.2	單目、雙目、RGB-D 初始化	
21.3	Tracking 執行緒	
21.4	Local Mapping 執行緒	
21.5	Loop Closing 執行緒	
21.6	KeyFrame、MapPoint、Covisibility Graph	
21.7	ORB-SLAM2 實作與資料集測試	
第 22 章	ORB-SLAM3 與 Visual-Inertial SLAM	375
22.1	ORB-SLAM3 的系統特色	
22.2	四種資料關聯:系統的架構主軸	
22.3	Monocular / Stereo Visual-Inertial SLAM	
22.4	IMU 初始化與預積分	
22.5	Multi-Map 多地圖系統(Atlas)	
22.6	失去追蹤後的重定位與地圖融合	
22.7	GoPro、手機、無人機上的視覺慣性 SLAM	
第 23 章	視覺導航機器人	392
23.1	視覺導航與 SLAM 的差異	
23.2	線追蹤、地標導航、視覺伺服	
23.3	AprilTag / ArUco 導航	
23.4	YOLO 物件辨識與語意導航	
23.5	VLM 視覺語言模型輔助導航	
23.6	機器狗視覺導航	
23.7	校園巡檢與資源回收辨識機器人	

第七篇 微型機器人、無人機與邊緣 AI SLAM

第 24 章	NanoSLAM 與低功耗 SLAM.....	411
24.1	為什麼微型機器人很難跑 SLAM	
24.2	NanoSLAM 的設計目標	
24.3	RISC-V GAP9 與低功耗平行運算	
24.4	毫瓦級 SLAM 的系統設計	
24.5	微型無人機、Swarm Robots、AR 眼鏡應用	
24.6	Edge AI 與 TinyML 對 SLAM 的影響	
24.7	高中專題:微型無人機地圖建構模擬	
第 25 章	無人機 SLAM 與視覺慣性導航	429
25.1	無人機定位的特殊困難	
25.2	GPS-denied 環境導航	
25.3	Optical Flow、VIO、Visual SLAM	
25.4	高速運動、震動與影像模糊	
25.5	無人機室內導航實驗	
25.6	視覺慣性資料集:EuRoC、TUM-VI	
25.7	從 nanoSLAM 到視覺導航無人機	
第 26 章	多機器人 SLAM 與雲端 / 邊緣協作	448
26.1	單機 SLAM 與多機 SLAM	
26.2	Map Fusion 地圖融合	
26.3	Collaborative SLAM 的三種架構	
26.4	Cloud SLAM 與 Edge SLAM	
26.5	多台機器狗校園巡檢	
26.6	SLAM 與 AI Agent 協作	
26.7	隊形、通訊、資料同步與安全	

第八篇 ROS / Python / C++ 實作

第 27 章	SLAM 程式設計環境	466
27.1	Ubuntu 與 ROS / ROS 2	
27.2	Python、C++、CMake	
27.3	ROS Topic、Service、Action	
27.4	rosviz 資料錄製與重播	
27.5	RViz、Gazebo、Foxglove	
27.6	GitHub 開源專案閱讀方法	
27.7	從高中積木式實作到大學原始碼實作	
第 28 章	2D 光達小車建圖實作	483
28.1	硬體:小車底盤、馬達、編碼器、2D 光達	
28.2	TF 設定與 URDF	
28.3	里程計資料發布	
28.4	2D SLAM 建圖	
28.5	AMCL 定位	
28.6	Navigation Stack 導航	
28.7	系統性除錯:當機器人不動的時候	
第 29 章	RGB-D / 雙目相機 SLAM 實作	503
29.1	RealSense / ZED / OAK-D 相機介紹	
29.2	相機校正	
29.3	RGB-D 點雲產生	
29.4	RTAB-Map 實作	
29.5	ORB-SLAM3 實作	
29.6	室內場景測試	
29.7	視覺 SLAM 與光達 SLAM 比較	
第 30 章	SLAM 專題與評量	522
30.1	專題一:教室 2D 地圖建構	
30.2	專題二:智慧機器人教室 3D 點雲掃描	
30.3	專題三:掃地機器人路徑分析	

- 30.4 專題四:機器狗校園巡檢 SLAM
- 30.5 專題五:視覺導航無人機
- 30.6 專題六:地磁 + IMU + 視覺融合定位
- 30.7 評分 Rubric:五個向度

第九篇 AI、VLM 與語意導航 SLAM

第 31 章 訓練 AI 與 VLM 輔助 SLAM 導航..... 542

- 31.1 為什麼傳統 SLAM 還需要 AI
- 31.2 AI 視覺模型在 SLAM 中的角色
- 31.3 從 Visual SLAM 到 Semantic SLAM 到具身 AI 導航
- 31.4 資料蒐集、標註與模型訓練
- 31.5 VLM 語言導航與 RAG 校園知識庫
- 31.6 LLM Agent 與 SLAM 系統整合
- 31.7 邊緣 AI 部署:Jetson、量化與蒸餾
- 31.8 風險、限制與未來發展

索引

本索引依中文術語筆畫與拼音混合排序，數字為該術語出現的章次。同一術語若跨多章出現，表示它是貫穿全書的核心概念。

3D 飛時測距 (3D ToF)	第 16 章
GNSS 拒止環境 (GNSS-denied)	第 5 章
GPS 拒止環境 (GPS-denied)	第 25 章
ICP (Iterative Closest Point)	第 13 章
IMU 初始化 (IMU Initialization)	第 22 章
IMU 預積分 (IMU Preintegration)	第 7 章
KD 樹 (KD-Tree)	第 17 章
PnP 問題 (Perspective-n-Point)	第 20 章
RGB-D 里程計 (RGBD Odometry)	第 29 章
Rao-Blackwell 化 (Rao-Blackwellization)	第 14 章
三角定位 (Triangulation)	第 4 章
三角測量 (Triangulation)	第 20 章
三邊定位 (Trilateration)	第 4 章
世界地磁模型 (WMM)	第 6 章
世界模型 (World Model)	第 31 章
中期資料關聯 (Mid-term)	第 22 章
主動紅外投射 (IR Projector)	第 29 章
互補濾波器 (Complementary Filter)	第 7 章
代價地圖 (Costmap)	第 3、28 章
位姿 (Pose)	第 1、10 章
位姿圖 (Pose Graph)	第 12 章
位姿圖優化 (Pose Graph Optimization)	第 24 章
位置式視覺伺服 (PBVS)	第 23 章
低功耗藍牙信標 (BLE Beacon)	第 4 章
佔據格地圖 (Occupancy Grid Map)	第 1、13 章
佔據機率 (Occupancy Probability)	第 13 章

傾斜補償 (Tilt Compensation)	第 6 章
僅運動 BA (Motion-only BA)	第 21 章
光度校正 (Photometric Calibration)	第 25 章
光束法平差 (Bundle Adjustment, BA)	第 12 章
光流 (Optical Flow)	第 9、25 章
光流定位 (Optical Flow Positioning)	第 25 章
光線投射 (Ray Casting)	第 13 章
光達 (LiDAR)	第 1、8 章
光達慣性里程計 (LIO)	第 18 章
光達里程計 (LiDAR Odometry)	第 18 章
光達里程計與建圖 (LOAM)	第 18 章
內點與外點 (Inlier / Outlier)	第 20 章
全域 BA (Full BA)	第 21 章
全域 SLAM (Global SLAM)	第 15 章
全域快門 (Global Shutter)	第 25、29 章
全域規劃器 (Global Planner)	第 28 章
全球衛星導航系統 (GNSS)	第 5 章
兩步最佳化 (Two-step Optimization)	第 18 章
八叉樹 (Octree)	第 17 章
六自由度位姿 (6-DoF Pose)	第 4 章
共視圖 (Covisibility Graph)	第 21 章
共變異數 (Covariance)	第 11 章
具身智慧 (Embodied AI)	第 31 章
分支定界 (Branch and Bound)	第 15 章
分散式架構 (Distributed)	第 26 章
到達時間差定位 (TDoA)	第 4 章
前端 (Front-End)	第 12 章
剛體運動 (Rigid Body Motion)	第 10 章
動作 (Action)	第 27 章
動作捕捉系統 (Motion Capture)	第 25 章
動態模糊 (Motion Blur)	第 25 章
動態頻率調節 (Dynamic Frequency Scaling)	第 24 章
協作式 SLAM (Collaborative SLAM)	第 26 章

卡爾曼增益 (Kalman Gain)	第 11 章
卡爾曼濾波 (Kalman Filter)	第 11 章
卡爾曼濾波器 (Kalman Filter)	第 7 章
即時動態定位 (RTK)	第 5 章
去中心化 (Decentralized)	第 26 章
反射率 (Reflectivity)	第 8 章
可重現性 (Reproducibility)	第 30 章
同步定位與建圖 (SLAM)	第 1 章
啟動檔 (Launch File)	第 27 章
單應矩陣 (Homography, H)	第 20 章
四元數 (Quaternion)	第 10 章
固態光達 (Solid-State LiDAR)	第 8 章
圖優化函式庫 (g2o / Ceres / GTSAM)	第 12 章
地圖伺服器 (map_server)	第 14 章
地圖合併 (Map Merging)	第 22 章
地圖品質指標 (Map Quality Metric)	第 30 章
地圖定位 (Map-based Localization)	第 3 章
地圖座標 (map frame)	第 10 章
地圖檔 (pgm / yaml)	第 14 章
地圖漂移 (Map Drift)	第 13 章
地圖融合 (Map Fusion / Merging)	第 26 章
地圖解析度 (Map Resolution)	第 13 章
地圖點 (MapPoint)	第 21 章
地標導航 (Landmark Navigation)	第 23 章
地磁場 (Geomagnetic Field)	第 6 章
地磁指紋 (Magnetic Fingerprint)	第 6 章
地面分離 (Ground Segmentation)	第 18 章
地點辨識 (Place Recognition)	第 21 章
均值 (Mean)	第 11 章
基礎矩陣 (Fundamental Matrix, F)	第 20 章
基線 (Baseline)	第 9、20 章
場景圖 (Scene Graph)	第 31 章
外觀式建圖 (Appearance-based Mapping)	第 29 章

多地圖系統 (Atlas)	第 22 章
多地圖資料關聯 (Multi-map)	第 22 章
多普勒效應 (Doppler Effect)	第 8 章
多樓層地圖 (Multi-floor Map)	第 16 章
多解析度預算格 (Precomputed Grids)	第 15 章
多路徑效應 (Multipath)	第 4 章
多階段作業 (Multi-session)	第 22 章
套件 (Package)	第 27 章
姿態角 (Roll / Pitch / Yaw)	第 7 章
子地圖 (Submap)	第 15 章
定位 (Localization)	第 1 章
定位模式 (Localization Mode)	第 29 章
專題導向學習 (Project-Based Learning)	第 30 章
專題里程碑 (Milestone)	第 30 章
對數勝算比 (Log-odds)	第 13 章
對極幾何 (Epipolar Geometry)	第 20 章
對極約束 (Epipolar Constraint)	第 20 章
對極線 (Epipolar Line)	第 20 章
對極點 (Epipole)	第 20 章
尺度不確定性 (Scale Ambiguity)	第 7、9、19 章
尺度不變性 (Scale Invariance)	第 19 章
局部 BA (Local BA)	第 21 章
局部 SLAM (Local SLAM)	第 15 章
局部建圖 (Local Mapping)	第 21 章
工作空間 (Workspace)	第 27 章
差分 GNSS (DGNSS)	第 5 章
差速驅動 (Differential Drive)	第 28 章
常態分布轉換 (NDT)	第 17 章
平行超低功耗 (PULP)	第 24 章
平面特徵 (Planar Feature)	第 18 章
幻覺 (Hallucination)	第 31 章
座標系 (Coordinate Frame)	第 10 章
座標轉換樹 (TF Tree)	第 10 章

建圖 (Mapping)	第 1 章
建置工具 (colcon)	第 27 章
弓字形路徑 (Boustrophedon Path)	第 16 章
彩色點雲 (Colored Point Cloud)	第 29 章
影像式視覺伺服 (IBVS)	第 23 章
後端 (Back-End)	第 12 章
復原行為 (Recovery Behavior)	第 28 章
微型機器學習 (TinyML)	第 24 章
微型無人機 (Nano-UAV / Nano-drone)	第 24 章
微型飛行器 (MAV)	第 25 章
感知—決策—行動 (Sense–Think–Act)	第 3 章
感知混淆 (Perceptual Aliasing)	第 26 章
慣性量測單元 (IMU)	第 1、7 章
懸崖感測器 (Cliff Sensor)	第 16 章
成果證據 (Evidence)	第 30 章
房間分割 (Room Segmentation)	第 16 章
技術債 (Technical Debt)	第 30 章
拓樸地圖 (Topological Map)	第 1 章
指紋定位 (Fingerprinting)	第 4 章
掃描匹配 (Scan Matching)	第 13、14、15、24 章
接收訊號強度指標 (RSSI)	第 4 章
控制器 (Controller)	第 28 章
描述子 (Descriptor)	第 19 章
提議分布 (Proposal Distribution)	第 14 章
擴展卡爾曼濾波 (EKF)	第 7、11 章
整週波未定值 (Integer Ambiguity)	第 5 章
數位分身 (Digital Twin)	第 17 章
旋轉不變性 (Rotation Invariance)	第 19 章
旋轉矩陣 (Rotation Matrix)	第 10 章
星基增強系統 (SBAS)	第 5 章
星曆 (Ephemeris)	第 5 章
曲率 / 粗糙度 (Curvature / Smoothness)	第 18 章
最大後驗估測 (MAP)	第 7 章

最大後驗估測 (MAP Estimation)	第 22 章
最小平方 (Least Squares)	第 12 章
會合 (Rendezvous)	第 26 章
有效粒子數 (Effective Sample Size)	第 14 章
服務 (Service)	第 27 章
未知區域 (Unknown)	第 13 章
本質圖 (Essential Graph)	第 21 章
本質矩陣 (Essential Matrix, E)	第 20 章
模型蒸餾 (Distillation)	第 31 章
模擬器 (Gazebo)	第 27 章
機器人作業系統 (ROS / ROS 2)	第 3 章
機器人描述檔 (URDF)	第 28 章
機率分布 (Probability Distribution)	第 11 章
機率格 (Probability Grid)	第 15 章
機載運算 (Onboard Computation)	第 24 章
檢索增強生成 (RAG)	第 31 章
權重 (Weight)	第 14 章
歐拉角 (Euler Angles)	第 10 章
殘差 (Residual)	第 12 章
比力 (Specific Force)	第 7 章
法向量 (Normal Vector)	第 17 章
活躍地圖 (Active Map)	第 22 章
深度對齊 (Depth Alignment)	第 29 章
深度影像 (Depth Image)	第 29 章
深度睡眠 (Deep Sleep)	第 24 章
滾動快門 (Rolling Shutter)	第 25 章
漢明距離 (Hamming Distance)	第 19 章
焊接視窗 (Welding Window)	第 22 章
焦距 (Focal Length)	第 9 章
物件目標導航 (Object-Goal Navigation)	第 23 章
物件目標導航 (ObjectNav)	第 31 章
特徵匹配 (Feature Matching)	第 19 章
特徵地圖 (Feature Map)	第 3 章

特徵法 (Feature-based / Indirect)	第 19 章
特徵軌跡縮短 (Shortened Feature Tracks)	第 25 章
特徵追蹤 (Feature Tracking)	第 19 章
特徵點 (Feature / Keypoint)	第 9 章
狀態發布器 (robot_state_publisher)	第 28 章
生命週期節點 (Lifecycle Node)	第 28 章
生成樹 (Spanning Tree)	第 21 章
異質團隊 (Heterogeneous Team)	第 26 章
直接法 (Direct Method)	第 19 章
相似變換 (Similarity Transform)	第 21 章
相對位姿誤差 (RPE)	第 30 章
相對定位 (Relative Localization)	第 3 章
相對變換 (Relative Transform)	第 26 章
相機內參 (Intrinsic Parameters)	第 9 章
相機外參 (Extrinsic Parameters)	第 9 章
相機校正 (Camera Calibration)	第 9 章
相機資訊 (CameraInfo)	第 29 章
短期資料關聯 (Short-term)	第 22 章
硬鐵誤差 (Hard-iron Error)	第 6 章
磁偏角 (Declination)	第 6 章
磁傾角 (Inclination / Dip)	第 6 章
磁力計 (Magnetometer)	第 6 章
磁暴 (Geomagnetic Storm)	第 6 章
禁區 (No-go Zone)	第 16 章
科氏力 (Coriolis Force)	第 7 章
稀疏位姿調整 (Sparse Pose Adjustment)	第 15 章
稀疏性 (Sparsity)	第 12 章
積分漂移 (Integration Drift)	第 7 章
積極機動 (Aggressive Maneuver)	第 25 章
穩健核函數 (Robust Kernel)	第 15 章
節點 (Node / Vertex)	第 12 章
節點 (Node)	第 27 章
節點與話題 (Node / Topic)	第 3 章

粒子 (Particle)	第 14 章
粒子濾波 (Particle Filter)	第 11 章
粒子濾波 SLAM (RBPF-SLAM)	第 14 章
粒子退化 (Particle Depletion)	第 14 章
精度稀釋因子 (DOP)	第 5 章
系統單晶片 (SoC)	第 24 章
系統穩定性 (Robustness)	第 30 章
結構光 (Structured Light)	第 16 章
絕對定位 (Absolute Localization)	第 3 章
絕對軌跡誤差 (ATE)	第 30 章
綁架機器人問題 (Kidnapped Robot Problem)	第 3 章
網路 RTK (NTRIP)	第 5 章
緊耦合 (Tightly Coupled)	第 7 章
線數 (Channels / Lines)	第 8 章
線追蹤 (Line Following)	第 23 章
背包式建模 (Backpack Mapping)	第 15 章
脈衝式測距 (Pulsed / dToF)	第 8 章
膨脹層 (Inflation Layer)	第 28 章
自由空間 (Free Space)	第 13 章
航向角 (Heading / Yaw)	第 6 章
萬向鎖 (Gimbal Lock)	第 10 章
蒙地卡羅定位 (MCL)	第 11 章
虛擬牆 (Virtual Wall)	第 16 章
虛擬距離 (Pseudorange)	第 5 章
行為樹 (Behavior Tree)	第 28 章
覆蓋率 (Coverage Rate)	第 16、30 章
覆蓋路徑規劃 (Coverage Path Planning)	第 16 章
視差 (Disparity)	第 9 章
視差角 (Parallax Angle)	第 20 章
視覺 SLAM (Visual SLAM)	第 19 章
視覺伺服 (Visual Servoing)	第 23 章
視覺化工具 (RViz2)	第 27 章
視覺基準標記 (Fiducial Marker)	第 4 章

視覺導航 (Visual Navigation)	第 23 章
視覺慣性 SLAM (Visual-Inertial SLAM)	第 22 章
視覺慣性 SLAM (VI-SLAM)	第 25 章
視覺慣性里程計 (VIO)	第 25 章
視覺語言動作模型 (VLA)	第 23、31 章
視覺語言導航 (VLN)	第 23 章
視覺語言模型 (VLM)	第 9、23、31 章
視覺里程計 (Visual Odometry)	第 19、29 章
視野角 (Field of View, FoV)	第 8 章
觀測模型 (Observation Model)	第 11 章
角解析度 (Angular Resolution)	第 8 章
訊息擷取格式 (MCAP)	第 27 章
記憶體管理 (Memory Management)	第 29 章
評分標準 (Rubric)	第 30 章
詞袋模型 (Bag of Words)	第 19 章
話題 (Topic)	第 27 章
語意 SLAM (Semantic SLAM)	第 31 章
語意地圖 (Semantic Map)	第 1 章
語意導航 (Semantic Navigation)	第 23 章
調頻連續波 (FMCW)	第 8 章
變換矩陣 (Transformation Matrix)	第 10 章
變異數 (Variance)	第 11 章
貝氏濾波 (Bayes Filter)	第 11 章
資料庫檔 (Database File)	第 29 章
資料錄製 (rosbag2)	第 27 章
資訊矩陣 (Information Matrix)	第 12 章
超寬頻 (UWB)	第 4 章
跨機器人閉環 (Inter-robot Loop Closure)	第 26 章
路徑規劃 (Path Planning)	第 3 章
軟鐵誤差 (Soft-iron Error)	第 6 章
載波相位 (Carrier Phase)	第 5 章
近似同步 (Approximate Sync)	第 29 章
迭代擴展卡爾曼濾波 (iEKF)	第 18 章

迭代最近點 (ICP)	第 17 章
追蹤 (Tracking)	第 21 章
逆感測器模型 (Inverse Sensor Model)	第 13 章
速度指令 (cmd_vel)	第 28 章
連桿與關節 (Link / Joint)	第 28 章
運動模型 (Motion Model)	第 11 章
運動變形 (Motion Distortion)	第 18 章
運算卸載 (Computation Offload)	第 24 章
選擇性重採樣 (Selective Resampling)	第 14 章
避障 (Obstacle Avoidance)	第 3 章
邊 (Edge / Constraint)	第 12 章
邊界探索 (Frontier Exploration)	第 23 章
邊緣人工智慧 (Edge AI)	第 24 章
邊緣特徵 (Edge Feature)	第 18 章
邊緣運算 (Edge Computing)	第 26 章
邊緣部署 (Edge Deployment)	第 31 章
都市峽谷效應 (Urban Canyon)	第 5 章
里程計 (Odometry)	第 1、13、28 章
里程計座標 (odom frame)	第 10 章
重定位 (Relocalization)	第 3、21 章
重投影誤差 (Reprojection Error)	第 12 章
重採樣 (Resampling)	第 14 章
重複率 (Redundancy Rate)	第 16、30 章
量化 (Quantization)	第 31 章
針孔相機模型 (Pinhole Camera Model)	第 9 章
鏡頭畸變 (Lens Distortion)	第 9 章
長期支援版 (LTS)	第 27 章
長期資料關聯 (Long-term)	第 22 章
閉環 (Loop Closing)	第 21 章
閉環偵測 (Loop Closure)	第 1 章
閉環約束 (Loop Closure Constraint)	第 15 章
開放指令集架構 (RISC-V)	第 24 章
開放詞彙 (Open-Vocabulary)	第 31 章

關鍵幀 (KeyFrame)	第 21 章
關鍵點 (Keypoint)	第 19 章
隨機取樣一致 (RANSAC)	第 20 章
隨機游走 (Random Walk)	第 7 章
隨機碰撞式 (Random Bounce)	第 16 章
集中式架構 (Centralized)	第 26 章
雙向測距 (TWR)	第 4 章
離群點剔除 (Outlier Rejection)	第 26 章
雲端運算 (Cloud Computing)	第 26 章
零偏 (Bias)	第 7 章
零樣本 (Zero-Shot)	第 23 章
雷射掃描 (LaserScan)	第 8 章
電子羅盤 (Electronic Compass)	第 6 章
非活躍地圖 (Inactive Map)	第 22 章
非線性最小平方 (Nonlinear Least Squares)	第 15 章
非視距 (NLOS)	第 4 章
飛時測距矩陣 (ToF Matrix Sensor)	第 24 章
飛行時間 (Time of Flight, ToF)	第 8 章
體素 (Voxel)	第 17 章
體素降採樣 (Voxel Grid Downsampling)	第 17 章
高斯分布 (Gaussian Distribution)	第 11 章
點到線距離 (Point-to-Edge Distance)	第 18 章
點到面距離 (Point-to-Plane Distance)	第 18 章
點對面 ICP (Point-to-Plane ICP)	第 17 章
點雲 (Point Cloud)	第 1、8、17 章
點雲配準 (Point Cloud Registration)	第 17 章
點雲顏色化 (Colorization)	第 17 章
齊次座標 (Homogeneous Coordinates)	第 10 章

台灣自編大學教科書

@prof

《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第 1 章

什麼是地圖建構與同步定位

編者：李世淵 (機器人叫獸)

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

本章在全書的位置

在正式進入內容之前，先說明這一章在整本書中扮演的角色，以及它與後續各章的連結。這個「地圖」會在每一章開頭出現，幫助同學隨時知道自己走到哪裡、下一步要去哪裡——這正是本書所談的定位問題，只是換成了知識的空間。

圖 1-0 第 1 章與全書的連結地圖

【本章負責】建立 SLAM 的核心問題意識：機器人如何同時知道自己在哪、並畫出環境的樣子。

【往下延伸】

1.3 節的六大感測器 → 第二篇 (第 4–9 章) 逐一深入：室內定位、衛星、地磁、IMU、光達、相機

1.2 節的雞生蛋問題 → 第 11 章以機率濾波求解、第 12 章以圖優化求解

1.4 節的四大應用 → 第四篇掃地機器人、第五篇自主巡檢、第六篇 AR / VR、第七篇無人機

1.5 節的地圖表示 → 第 13 章佔據格、第 17 章點雲與八叉樹、第 31 章語意地圖

1.6 節的前端後端 → 第 12 章正式定義、第 15、18、21 章在真實系統中體現

1.7 節的技術挑戰 → 貫穿全書「常見失敗」的討論主軸

【先備知識】無。本章不需要任何數學或程式基礎，只需要好奇心。

學習目標

本章是全書的起點，目的在於建立同學對「地圖建構與同步定位」(Simultaneous Localization and Mapping，簡稱 SLAM) 的整體影像。學完本章之後，同學應能夠達成下列目標：

1. 說明 SLAM 的基本定義，並能以自己的話解釋「定位」(Localization) 與「建圖」(Mapping) 分別在解決什麼問題。
2. 解釋為什麼「定位」與「建圖」互為前提，形成雞生蛋、蛋生雞的循環依賴問題，以及 SLAM 如何同時估測兩者。
3. 列舉機器人常用的感測器 (里程計、IMU、相機、光達、無線電、地磁)，並說明各自量測的物理量與優缺點。
4. 舉例說明 SLAM 在自動駕駛、無人機、掃地機器人與 AR / VR 中扮演的角色。
5. 分辨 SLAM 系統의各種輸出形式：軌跡、2D 佔據格地圖、3D 點雲、語意地圖與拓撲地圖。
6. 完成本章實作活動，親身體會「感測器看到的世界」與「實際地圖」之間的差異。

核心概念

中文名詞	英文名詞	概念說明
同步定位與建圖	SLAM	機器人在未知環境中，一邊建立環境地圖、一邊估測自身位置的技術。

中文名詞	英文名詞	概念說明
定位	Localization	估測機器人在地圖或世界座標中的位置與姿態（位姿，Pose）。
建圖	Mapping	利用感測器資料建立環境的幾何或語意表示。
位姿	Pose	位置（ x 、 y 、 z ）加上姿態（方向角），2D 常記為（ x 、 y 、 θ ）。
里程計	Odometry	由輪子編碼器或感測器累積推算的相對位移，會隨時間漂移。
慣性量測單元	IMU	由加速度計與陀螺儀組成，量測加速度與角速度。
光達	LiDAR	以雷射光飛行時間量測距離的感測器，可產生掃描線或點雲。
閉環偵測	Loop Closure	機器人回到走過的地點時，辨識出「這裡我來過」，用來修正累積誤差。
佔據格地圖	Occupancy Grid Map	把空間切成小格子，標記每格「有障礙、無障礙、未知」的 2D 地圖。
點雲	Point Cloud	由大量三維點組成的空間表示，常由 3D 光達或深度相機產生。
語意地圖	Semantic Map	除了幾何形狀，還標註「門、桌子、走廊」等意義的地圖。
拓樸地圖	Topological Map	以「節點與連線」描述地點與通路關係的地圖，類似捷運路線圖。

1.1 SLAM 的基本定義：Localization + Mapping

想像一個情境：你被蒙上眼睛，帶到一棟從來沒去過的建築物裡，然後把眼罩拿掉。你手上沒有平面圖，牆上也沒有任何指標。這時你會怎麼辦？大多數人會先環顧四周，記下「左邊有一扇門、前面是長走廊、右邊有樓梯」，然後開始走動。走著走著，你腦中會逐漸浮現這棟建築的樣子，同時你也隨時知道「我現在大概在剛剛那扇門的右前方十公尺」。

這個看似自然的過程，正是機器人領域中最經典、也最困難的問題之一：同步定位與建圖（Simultaneous Localization and Mapping，SLAM）。SLAM 要回答的核心問題只有一句話：

本書核心問題

機器人沒有地圖，也不知道自己在哪裡，如何一邊走、一邊建立地圖、一邊修正自己的位置？

把這個問題拆開來看，SLAM 其實是兩個子問題的結合：

- 定位（Localization）：已知地圖，估測機器人自己在地圖中的位姿。例如：手機打開 Google Maps，GPS 告訴你「你在校門口」，這就是定位。
- 建圖（Mapping）：已知機器人每一刻的位姿，把感測器量到的環境資訊拼接成一張地圖。例如：測量員拿著儀器，在已知的測站位置上量測周圍地形，繪製地形圖。

單獨來看，這兩個問題都已經有成熟的解法。麻煩的是，真實世界的機器人往往「兩者都沒有」：進入新環境時，既沒有現成地圖可以定位，也沒有精準位姿可以建圖。SLAM 的精神，就是讓機器人同時估測這兩個未知量——一邊移動、一邊觀測、

一邊把「我在哪裡」與「世界長什麼樣」這兩件事互相校正、越估越準。

MIT 的視覺導航課程 (VNAV, 16.485) 將 SLAM 定義為：在沒有環境先驗知識的情況下，僅利用感測器資料，同時估測機器人的位姿（或整條軌跡）以及環境表示的問題。CMU 的 16-833 課程則進一步強調，SLAM 必須在「感測不確定性」之下即時完成——感測器有雜訊、輪子會打滑、光線會變化，機器人永遠拿不到完美的量測值，因此 SLAM 本質上是一個「在不確定性中做最佳估測」的問題。這也是為什麼本書第三篇會花整整三章討論機率、濾波與最佳化的數學基礎。

SLAM 這個名字的由來

SLAM 一詞源自 1980 年代末至 1990 年代機器人學界對「隨機建圖」（Stochastic Mapping）問題的研究。Smith、Self 與 Cheeseman 在 1986 年前後提出以機率方法同時估測機器人位姿與地標位置的架構，其後 Durrant-Whyte、Leonard、Dissanayake 等學者以擴展卡爾曼濾波器（EKF）建立了完整的理論基礎，並證明 SLAM 問題是「可收斂」的——只要持續觀測，地圖與定位的不確定性會逐漸下降。2000 年代之後，粒子濾波（如本書第 14 章的 Gmapping）與圖優化（如第 15 章的 Cartographer、第 21 章的 ORB-SLAM）陸續成為主流，SLAM 也從實驗室走進掃地機器人、自駕車與手機 AR，成為現代智慧機器人不可或缺的核心技術。

1.2 為什麼「定位」與「建圖」是雞生蛋、蛋生雞問題

要理解 SLAM 為什麼困難，關鍵在於看清楚定位與建圖之間的循環依賴：

- 想要建圖，必須先知道自己在哪裡。感測器量到「前方三公尺有一面牆」，這個資訊只有在知道「我現在的位姿」時才有意義——同樣一筆量測，機器人站在不同位置、朝向不同方向，牆就會被畫在地圖上完全不同的地方。

- 想要定位，必須先有地圖。定位的本質是「比對」：把眼前看到的景象，跟地圖中的特徵比對，才能推論出自己的位置。沒有地圖，就沒有比對的依據。

這就是典型的雞生蛋、蛋生雞（Chicken-and-Egg）問題：建圖需要定位，定位又需要地圖，兩者互為前提，似乎哪一個都無法先開始。

沒有 SLAM 會發生什麼事：誤差累積

有同學可能會問：機器人不是有里程計嗎？輪子轉了幾圈、走了多遠，累加起來不就知道位置了？理論上是，但實務上里程計有一個致命弱點：漂移（Drift）。輪子會打滑、地毯與磁磚的摩擦力不同、輪徑有製造誤差，每一步的小誤差都會被「累加」進位置估測中，而且永遠不會自己消失。走十公尺可能只差幾公分，但繞行整層樓一圈之後，估測位置可能已經偏差數公尺——如果直接拿這樣的位姿去建圖，畫出來的走廊會彎曲、房間會錯位，同一面牆甚至會在地圖上出現兩次。

SLAM 打破這個僵局的策略，可以用一句話概括：不要把定位與建圖當成兩個先後的步驟，而是當成同一個估測問題，讓兩者互相修正。具體而言，SLAM 系統通常這樣運作：

1. 起步：機器人把出發點定為原點，此時地圖是空的，但位姿是確定的（我就在原點）。
2. 移動與預測：機器人前進一小步，用里程計或 IMU 預測新的位姿——這個預測帶有誤差。
3. 觀測與更新：感測器量測周圍環境，若量到的特徵與地圖中已有的特徵吻合，就用「吻合的程度」反過來修正位姿；同時把新看到的區域加入地圖。
4. 閉環修正：當機器人繞了一圈回到走過的地點，系統辨識出「這裡我來過」（閉環偵測，Loop Closure），此時可以一次性地把整條軌跡與整張地圖的累積誤差攤平修正，讓地圖「閉合」起來。

換句話說，SLAM 並不是奇蹟般地讓誤差消失，而是持續利用「環境本身」作為參考基準：牆壁不會動、柱子不會移位，只要機器人重複觀測到同樣的環境特徵，就能把漂移「拉回來」。這個觀念會貫穿全書——從第 11 章的貝氏濾波、第 12 章的位姿圖優化，到第 15 章 Cartographer 的閉環偵測，都是同一個精神在不同層次的展開。

兩條解法脈絡：本書的主線預告

面對這個互相依賴的難題，四十年來的研究者發展出兩條主要的解法脈絡。它們構成了本書第三篇之後的主線，同學現在不需要理解細節，但先知道有這兩條路，後面讀起來會清楚得多。

- 機率濾波路線：既然無法確定機器人到底在哪，那就同時保留許多種可能性，再用感測器資料逐一淘汰不合理的假設。這條路發展出卡爾曼濾波與粒子濾波，是第 11 章的主題，也是第 14 章 Gmapping 的理論基礎。
- 最佳化路線：把「機器人走過的每個位置」與「感測器量到的每個約束」寫成一個巨大的方程組，再一次求出最符合所有觀測的解。這條路發展出圖優化與束調整，是第 12 章的主題，也是第 15 章 Cartographer 與第 21 章 ORB-SLAM 的核心。

這兩條路線並非互相取代。第 11 章會說明濾波方法的即時性優勢，第 12 章則會說明最佳化方法為何能回頭修正歷史錯誤。第 14 章與第 15 章更會讓同學在同一個場域親手比較兩者的差異——那時再回頭看本節，會對「雞生蛋問題」的解法有更立體的理解。

1.3 機器人如何感知世界：里程計、IMU、相機、光達、無線電、地磁

人類靠眼睛、耳朵、前庭系統（內耳的平衡器官）與肌肉本體感覺來感知「我在哪裡、周圍有什麼」。機器人也需要對應的感官。SLAM 所使用的感測器可以粗分為兩大類：

- 本體感測器（Proprioceptive Sensors）：量測機器人「自己」的運動狀態，例如輪子轉了多少（里程計）、機身加速度與轉動角速度（IMU）。它們不需要看外界，但誤差會累積。
- 外部感測器（Exteroceptive Sensors）：量測機器人「周圍環境」，例如相機拍到的影像、光達量到的距離、收到的無線電訊號強度、地球磁場方向。它們提供絕對或半絕對的參考，但容易受環境干擾。

SLAM 的藝術，就在於把「會漂移但連續平滑」的本體感測器，與「不漂移但有雜訊、可能瞬間出錯」的外部感測器融合起來，截長補短。以下逐一介紹本書後續章節會深入探討的六大類感測器。

一、里程計（Odometry）

輪型機器人最基本的感測器。在馬達軸或輪軸上安裝編碼器（Encoder），計算輪子轉動的圈數，配合輪徑與輪距，就能推算機器人前進的距離與轉彎的角度。里程計的優點是取樣頻率高、成本低、資料連續；缺點是前面提過的漂移——打滑、輪徑誤差、地面不平都會讓誤差不斷累積。在 ROS 系統中，里程計對應 odom 座標框架，是幾乎所有 SLAM 演算法的「運動預測」來源（詳見第 10 章與第 28 章）。

二、慣性量測單元（IMU）

IMU（Inertial Measurement Unit）通常包含三軸加速度計與三軸陀螺儀，有些還加上三軸磁力計。加速度計量測線加速度，陀螺儀量測角速度，把它們對時間積分，理論上可以推算速度、位置與姿態——這就是慣性導航。IMU 的取樣頻率極高（常見 100 Hz 到 1000 Hz），對快速運動非常敏感，是無人機與視覺慣性 SLAM（如第 22

章的 ORB-SLAM3) 不可或缺的元件。但積分會放大雜訊與偏差，造成快速漂移，因此 IMU 幾乎一定要與其他感測器融合使用（詳見第 7 章）。

三、相機 (Camera)

相機是資訊量最豐富、成本又最低的感測器：一張影像包含紋理、顏色、形狀、甚至語意（這是門、那是人）。單目相機 (Monocular) 便宜但無法直接量測深度，會有尺度不確定性；雙目相機 (Stereo) 利用左右影像視差計算深度；RGB-D 相機 (如 RealSense) 直接輸出每個像素的距離。以相機為主的 SLAM 稱為 Visual SLAM，是本書第六篇的主軸。相機的弱點是依賴光線與紋理——黑暗、逆光、白牆、快速運動造成的模糊，都會讓視覺 SLAM 失效（詳見第 9 章、第 19 章）。

四、光達 (LiDAR)

光達 (Light Detection and Ranging) 發射雷射光並量測反射回來的時間 (Time of Flight)，直接得到「這個方向、這個距離有東西」的精確幾何資訊。2D 光達旋轉掃描一個平面，輸出一圈距離值 (LaserScan)，是掃地機器人與教學小車的標準配備；3D 光達 (多線雷射) 同時掃描多個仰角，輸出三維點雲，是自駕車與自主巡檢機器人的核心感測器。光達的優點是量測精確、不受光線影響；缺點是價格較高、對玻璃與鏡面會失效、雨霧會造成雜訊（詳見第 8 章、第五篇）。

五、無線電 (Radio)

GPS / GNSS 衛星定位是最廣為人知的無線電定位，但在室內、隧道、高樓峽谷中訊號會被遮蔽。室內環境則可利用 Wi-Fi 訊號強度指紋、藍牙 Beacon、UWB 超寬頻等技術進行定位。無線電定位的優點是提供「絕對位置」參考、不會漂移；缺點是精度有限 (GPS 約數公尺、Wi-Fi 約數公尺、UWB 約十至三十公分)、易受多路徑反射干擾。實務上常作為 SLAM 的輔助資訊或初始化來源（詳見第 4 章、第 5 章）。

六、地磁 (Geomagnetism)

地球本身就是一個大磁鐵，磁力計（電子羅盤）可以量測地磁方向，提供絕對航向角參考。更有趣的是，建築物內的鋼筋與設備會讓地磁產生獨特的局部扭曲，這些扭曲形成「地磁指紋」，可以用來做室內定位。地磁感測的優點是完全被動、零功耗需求極低；缺點是易受鐵磁物體干擾，必須經過校正（詳見第 6 章）。

感測器	量測內容	主要優點	主要弱點	本書章節
里程計	輪子轉動→相對位移	便宜、連續、高頻	打滑漂移、誤差累積	第 10、28 章
IMU	加速度、角速度	高頻、對快速運動敏感	積分漂移快	第 7、22 章
相機	影像（紋理、顏色、語意）	資訊豐富、成本低	依賴光線與紋理	第 9、19–23 章
光達	雷射測距（2D / 3D）	幾何精確、不受光線影響	價格較高、鏡面失效	第 8、13–18 章
無線電	GPS / Wi-Fi / UWB 訊號	絕對位置、不漂移	精度有限、多路徑干擾	第 4、5 章
地磁	地球磁場方向與強度	被動、提供絕對航向	鐵磁干擾、需校正	第 6 章

表 1-1 SLAM 常用感測器比較

感測器如何在本書中展開

本節提到的六類感測器，每一類在第二篇都有專章討論。這裡先給出對照表，讓同學知道每個感測器的「深入版本」在哪裡，也方便日後查閱。

表 1-0 感測器與專章對照

里程計 → 第 3 章 (航位推算) 、第 13 章 (與光達融合) 、第 28 章 (實作與校正)

IMU 慣性單元 → 第 7 章 (原理、漂移、姿態濾波、預積分)

相機 → 第 9 章 (成像模型) 、第六篇 (視覺 SLAM 全篇)

光達 → 第 8 章 (原理與規格) 、第四篇 (2D) 、第五篇 (3D)

無線電 → 第 4 章 (室內定位) 、第 5 章 (衛星導航)

地磁 → 第 6 章 (電子羅盤與地磁指紋)

每一章的結構都相同：原理 → 規格與限制 → 失效情境 → 與其他感測器的互補 → 實作活動。

這個一致的結構，是為了讓同學能快速比較不同感測器，而不是把它們當成孤立的知識。

這裡先埋下一個貫穿全書的觀念：沒有任何一種感測器是全能的。光達精確卻看不懂玻璃、相機資訊豐富卻怕黑暗、IMU 反應快卻會漂移、衛星訊號準確卻進不了室內。第二篇的每一章都會誠實地討論該感測器「在什麼情況下會失敗」，而第 29 章的實驗更會讓同學親眼看到光達與相機在同一場域各自崩潰的樣子。理解限制，是本書從第一頁到最後一頁的核心信念。

1.4 SLAM 與自動駕駛、無人機、掃地機器人、AR / VR 的關係

SLAM 之所以在過去二十年從冷門學術題目變成兵家必爭的核心技術，是因為它

剛好是四大熱門應用的共同地基。

自動駕駛 (Autonomous Driving)

自駕車需要公分級的定位精度——GPS 的數公尺誤差足以讓車子跨越車道。因此自駕車普遍採用「高精地圖 + 即時定位」的策略：先以測繪車搭載 3D 光達與相機，用 SLAM 技術建立高精度地圖；行駛時再把即時感測資料與高精地圖比對，實現精準定位。在沒有高精地圖的區域（例如新開通的道路、停車場），車輛則必須即時執行 SLAM。本書第 17、18 章介紹的 3D 光達建圖與 LOAM 系列演算法，正是自駕車定位建圖的技術骨幹。

無人機 (Drone / UAV)

無人機在室內、橋樑下方、隧道等 GPS 訊號被遮蔽 (GPS-denied) 的環境飛行時，只能依靠機載感測器定位。由於載重與電力極為有限，無人機偏好輕巧的「相機 + IMU」組合，也就是視覺慣性里程計 (VIO) 與視覺慣性 SLAM——這正是 ETH Zürich 與 UZH 研究群的強項，也是 MIT VNAV 課程以無人機為實驗平台的原因。2015 年後，DJI 等消費級無人機把 VIO 做進產品，讓無人機能在室內穩定懸停；學術端則持續往更小、更省電推進，例如本書第 24 章介紹的 NanoSLAM，讓 44 公克的微型無人機也能完全機載執行 SLAM。

掃地機器人 (Robot Vacuum)

掃地機器人是 SLAM 最成功的大眾化案例。早期的掃地機採用「隨機碰撞」策略——撞到牆就轉彎，清掃路線雜亂無章、重複又遺漏。2010 年 Neato 率先把 2D 光達裝上掃地機，2017 年小米第一代光達掃地機器人以親民價格普及，讓 SLAM 技術首次大量走進一般家庭：機器人先建立房間地圖，再規劃弓字形路徑逐區覆蓋，效率與覆蓋率大幅提升。今天打開掃地機 App 看到的房間地圖、禁區設定、指哪掃哪，背後全是 SLAM (詳見第 16 章)。

AR / VR (擴增實境 / 虛擬實境)

AR 眼鏡或手機 AR 要把虛擬物件「釘」在真實世界的桌面上，前提是裝置必須即時知道自己在空間中的位姿——這其實就是 SLAM，只是業界慣稱「空間定位」或「Inside-Out Tracking」。Apple 的 ARKit、Google 的 ARCore、Meta Quest 的頭盔追蹤，核心都是視覺慣性 SLAM。VR 頭盔用機身相機追蹤環境特徵，取代了早期需要外部基地台的定位方式。可以說，SLAM 讓 AR / VR 從「固定座位的體驗」進化為「自由走動的體驗」。

觀念小結

這四大應用看似天差地遠，但底層都是同一個問題：在未知或部分未知的環境中，即時估測自身位姿並理解周圍空間。

差別只在於感測器組合（光達為主或相機為主）、運算資源（車載電腦或手機晶片）與精度要求（公分級或公寸級）。

這也是為什麼本書強調「一套觀念、多種平台」：學會 SLAM 的核心原理，就能在自駕車、無人機、掃地機器人與 AR 之間觸類旁通。

1.5 從教室到校園：高中與大學場域的 SLAM 教學案例

本書的一大特色，是把 SLAM 的學習「落地」在兩個真實的教學場域：高中的 AI 機器人社團課程，與大學智慧機器人學程。同樣的核心觀念，會依照學習階段設計成不同深度的活動。

高中場景：看得懂、做得出來

高中階段的重點是建立直覺與動手經驗，不急著推導數學。以「教室」為實驗場域，可以設計以下循序漸進的活動：

- 活動一：人肉 SLAM。一位同學蒙眼、由同學口頭指揮在教室行走，行走者邊走邊口述「我覺得我在哪裡」，其他同學在紙上記錄實際位置與估測位置的差距，體會「沒有外部觀測時誤差如何累積」。
- 活動二：手機感測器初體驗。用手機的指南針、加速度計 App 觀察感測器數值，理解 IMU 與地磁感測的原理與雜訊。
- 活動三：掃地機器人觀察。觀察家用掃地機器人的清掃路線與 App 中的地圖，討論「它怎麼知道哪裡掃過了」。
- 活動四：模擬器建圖。在 ROS 模擬環境（如 TurtleBot3 模擬器）中操作虛擬小車建立教室地圖，不需要真實硬體也能完整體驗 SLAM 流程。

大學場景：能推導、能實作、能研究

大學部與專題階段，則以大學智慧機器人教室與校園為場域，把同一套觀念往下扎根、往上延伸：

- 基礎實作：以 2D 光達教學小車在教室與走廊執行 Gmapping 與 Cartographer 建圖（第 14、15、28 章），量化比較兩種演算法的地圖品質。
- 進階實作：以 RGB-D 相機執行 RTAB-Map 與 ORB-SLAM3（第 29 章），比較視覺 SLAM 與光達 SLAM 在同一場域的表現。
- 專題研究：機器狗校園巡檢、緊耦合光達慣性建圖、視覺導航無人機、多機器人協同建圖（第 18、22、24–26 章），銜接研究所等級的題目。

本書第 30 章提供六個完整專題與評分 Rubric，涵蓋從高中到大學專題的完整光譜。建議教師在學期初就讓學生看到這條「學習地圖」：知道現在學的每一個觀念，最終會通往哪一個看得見、摸得著的機器人系統。

1.6 SLAM 系統輸出：軌跡、2D 地圖、3D 點雲、語意地圖、拓樸地圖

SLAM 系統執行完畢後，到底會「產出」什麼？依照應用需求不同，輸出形式可以分為五大類。理解這些輸出形式，有助於在後續章節中快速掌握「這個演算法是在解決哪一種輸出」。

一、軌跡 (Trajectory)

機器人從出發到結束，每一個時間點的位姿序列。軌跡是評估 SLAM 精度最直接的依據：把估測軌跡與真實軌跡 (Ground Truth，通常由動作捕捉系統或高精度 GPS 提供) 比較，計算絕對軌跡誤差 (ATE) 與相對位姿誤差 (RPE)。EuRoC、TUM、KITTI 等公開資料集 (第 25、29 章) 都提供真實軌跡供研究者評測。

二、2D 佔據格地圖 (Occupancy Grid Map)

把地面切成固定大小的格子 (例如每格 5 公分)，每一格記錄「被佔據的機率」：黑色代表障礙物 (牆、桌腳)、白色代表可通行空間、灰色代表尚未觀測的未知區域。這是室內輪型機器人與掃地機器人最常用的地圖形式，也是 ROS Navigation 導航堆疊的標準輸入 (第 13 章)。

三、3D 點雲 (Point Cloud)

由數十萬至數億個三維點構成的空間模型，每個點帶有座標 (x 、 y 、 z)，可再附加顏色或反射強度。3D 點雲能完整呈現立體結構——樓梯、天花板管線、桌面高度，是自主巡檢、工廠數位分身 (Digital Twin) 與自駕車高精地圖的基礎 (第 17 章)。

四、語意地圖 (Semantic Map)

在幾何地圖之上，為每個區域或物件貼上「意義標籤」：這一塊是門、那一塊是桌子、這條是走廊、那裡是充電站。語意地圖讓機器人不只知道「哪裡有障礙」，更

知道「那個障礙是什麼」，因而能執行「去教務處」「避開樓梯」這類人類語言等級的任務。語意地圖是 AI 視覺模型 (YOLO、SAM、VLM) 與 SLAM 的交會點，也是本書最後一章 (第 31 章) 的主題。

五、拓樸地圖 (Topological Map)

拋開精確的幾何座標，只保留「地點」與「地點之間的連通關係」，就像捷運路線圖不在乎車站的真實距離，只在乎「哪一站可以到哪一站」。拓樸地圖記憶體需求極低、規劃路徑極快，適合大範圍導航；實務系統常採「拓樸為骨架、格地圖為血肉」的混合式地圖。

輸出形式	資料結構	典型應用	代表章節
軌跡	位姿的時間序列	精度評估、路徑重播	第 30 章
2D 佔據格地圖	機率格網	室內導航、掃地機器人	第 13–16 章
3D 點雲	三維點集合	空間掃描、自主巡檢、自駕車	第 17、18 章
語意地圖	幾何 + 物件標籤	語意導航、任務型機器人	第 23、31 章
拓樸地圖	節點與邊的圖	大範圍快速規劃	第 3、31 章

表 1-2 SLAM 系統五大輸出形式

公式或流程

一、SLAM 的機率定義

雖然嚴謹的推導要到第 11 章才展開，本章先給出 SLAM 的標準數學表述，讓同學對「SLAM 在估什麼」有一個精確的錨點。定義以下符號：

- $x(t)$ ：機器人在時間 t 的位姿，2D 情況下為 $(x、y、\theta)$ 。
- m ：環境地圖 (可能是格地圖、地標集合或點雲)。

- $z(1:t)$ ：從時間 1 到 t 的所有感測器觀測（雷射掃描、影像特徵等）。
- $u(1:t)$ ：從時間 1 到 t 的所有控制輸入或里程計讀值。

完整 SLAM (Full SLAM) 問題，是在給定所有觀測與控制之下，估測整條軌跡與地圖的聯合機率分布：

$$p(x(1:t), m \mid z(1:t), u(1:t))$$

線上 SLAM (Online SLAM) 問題，則只估測「當下」的位姿與地圖，把過去的軌跡邊緣化掉，適合即時系統：

$$p(x(t), m \mid z(1:t), u(1:t))$$

用白話翻譯：SLAM 要找的答案，是「在我看到的所有東西、走過的所有路之下，最有可能的軌跡與地圖是什麼」。之所以寫成機率分布而不是單一數值，正是因為感測器有雜訊、運動有誤差——SLAM 給的答案永遠附帶「有多確定」的資訊。粒子濾波 SLAM (第 14 章)、EKF-SLAM (第 11 章) 與圖優化 SLAM (第 12 章) 本質上都是求解這個機率問題的不同數值方法。

二、SLAM 系統標準流程

現代 SLAM 系統普遍採用「前端 + 後端」的架構，流程如圖 1-1 所示：

圖 1-1 SLAM 系統標準流程

感測器資料 (光達掃描 / 影像 / IMU / 里程計)

↓

前端 (Front-End)：特徵提取、資料關聯、掃描匹配、里程計估測

↓

後端 (Back-End) : 濾波或圖優化 , 估測位姿與地圖的最佳解

↓

閉環偵測 (Loop Closure) : 辨識重訪地點 , 修正全域累積誤差

↓

輸出 : 軌跡 + 地圖 (2D 格地圖 / 3D 點雲 / 語意地圖)

前端負責「把原始感測資料轉成幾何約束」, 後端負責「在所有約束下求出最一致的答案」, 閉環偵測則是修正長期漂移的關鍵武器。本書第二篇講感測器 (前端的輸入)、第三篇講數學 (後端的工具)、第四篇到第六篇講各種具體系統如何實現這條流程, 第七篇之後則探討微型化、多機協作與 AI 語意化的前沿延伸。

1.7 SLAM 的技術演進與當代前沿

前面幾節介紹的是 SLAM 的基本問題與構成。本節把視野拉高, 看這門學問四十年來如何演變, 以及此刻正在發生什麼——這能幫助同學理解, 本書後續章節所教的技術, 各自處於這條演進線的哪個位置。

四個階段的演進

綜合近年的回顧文獻, SLAM 的發展大致可分為四個階段, 而且每個階段並未取代前一個, 而是不斷疊加、互相補強:

1. 手工設計時期 (1986–2010 年代) : 研究者以人類對幾何的理解, 手工設計特徵偵測器 (如第 19 章的 SIFT、ORB) 與濾波演算法 (第 11 章)。這個時期奠定了 SLAM 的數學骨架, 也是本書第三篇到第六篇的主要內容。
2. 深度學習時期 (2015 年前後起) : 以神經網路取代部分手工模組——學習來

的特徵、學習來的深度估計、學習來的閉環偵測。本書第 9.4 節的單目深度估計、第 19.7 節的深度特徵，都屬於這個階段的成果。

3. 輻射場時期（2021 年起）：以神經輻射場（NeRF）與三維高斯潑濺（3D Gaussian Splatting，3DGS）重新定義「地圖」是什麼——地圖不再只是點雲或格子，而是一個能重新算繪出照片級畫面的場。
4. 基礎模型時期（2023 年起）：視覺語言模型與機器人基礎模型進入 SLAM，讓機器人從「知道形狀」走向「理解意義」。這是本書第 31 章的主題。

輻射場如何改變地圖的定義

第三階段的轉折特別值得說明，因為它改變了一個看似理所當然的前提：地圖應該長什麼樣。

傳統 SLAM 的地圖是佔據格（第 13 章）或點雲（第 17 章）——它們回答「哪裡有東西」，但無法回答「從這個角度看過去，畫面應該長什麼樣」。NeRF 以神經網路隱式地表示整個空間的輻射場，能從任意視角算繪出逼真的畫面；而 3DGS 則改採顯式表示，把場景建模為一群三維高斯基元的集合，每個基元有位置（均值）、共變異數、顏色與不透明度。

3DGS 之所以在機器人領域快速崛起，關鍵在於效率。不同於隱式表示需要對空間中每一個點查詢神經網路，3DGS 的可微分光柵化管線克服了大型神經網路訓練與算繪速度緩慢的瓶頸——而那正是 NeRF 系統難以用於機器人的主因。此外它的顯式結構讓地圖能被輕易編輯或形變，這對地圖修正與環境適應相當有利。

觀念釐清：三種場景表示的取捨

【隱式】以神經網路逼近輻射場。記憶體小，但每次查詢都要跑網路，速度慢。

【顯式】直接在學到的空間特徵（體素、雜湊格、高斯基元）上做算繪，不含神經元件。速度快，但記憶體需求高。

【混合】把學到的空間特徵與神經網路結合。訓練與算繪都能加速，同樣需要較多記憶體。

對機器人而言，速度與記憶體的權衡永遠存在——這與第 24 章「毫瓦級平台」的取捨是同一個問題，

只是換到了地圖表示的層次。

當代前沿的四個方向

綜觀近兩年的研究，SLAM 領域最活躍的方向可歸納為四項，同學若日後要投入研究，這是相當實用的地圖：

- 穩健性：讓輻射場 SLAM 能應付動態環境與運動模糊。近期已有專門處理模糊輸入的稠密神經 SLAM，以及針對演化中場景的高斯建圖方法；也有系統以 IMU 輔助處理模糊、滾動快門與雜訊嚴重的熱像影像——這正好呼應第 25.4 節討論的高速飛行三重打擊。
- 協作化：多台機器人共同建立高斯地圖。值得注意的是，傳統協作式 SLAM 有走向分散式以提升擴展性與穩健度的趨勢，但由於三維高斯訓練的運算成本高昂，目前協作式高斯 SLAM 反而以集中式設計居多——這與第 26.3 節的架構討論恰成對照，也說明技術選型如何受運算成本左右。
- 語言化：把語言能力融入地圖表示，讓機器人能以自然語言查詢空間。這是第 31 章的主題，也是第 23.5 節 VLM 導航的延伸。
- 基礎模型化：以多視圖基礎模型支撐單目高斯 SLAM，或探討「對機器人而言，

究竟什麼才是最好的三維場景表示」這類根本問題。這條線正把幾何 SLAM 與人工智慧推向同一個交會點。

本書的定位

讀到這裡，同學可能會問：既然輻射場與基礎模型如此熱門，為什麼本書大部分篇幅仍在講傳統的幾何方法？

答案有三層。其一，實務性：目前絕大多數真正部署在產品上的系統——掃地機器人、自駕車、巡檢機器狗、AR 眼鏡——骨幹仍是本書所教的幾何方法。它們成熟、可解釋、運算需求明確。

其二，基礎性：輻射場與基礎模型並未取代幾何，而是建立在幾何之上。3DGS SLAM 仍需要位姿估計、仍需要閉環修正、仍需要處理感測器的限制。不理解幾何，就無法判斷這些新方法在什麼時候會失效。

其三，教學性：本書的目標是培養能做出判斷的工程師，而非追逐術語的使用者。第 30 章的專題與第 31 章的討論會反覆強調同一件事——技術會不斷演進，但「理解限制、選擇合適工具」的能力不會過時。

案例

案例 1-1 小米掃地機器人：SLAM 走進千家萬戶

2017 年，小米推出第一代米家掃地機器人，機頂搭載一顆旋轉式 2D 光達，售價卻壓在傳統高階掃地機的一半以下。它的工作流程完整示範了本章介紹的 SLAM 循環：開機後光達每秒掃描周圍數千個距離點，機器人一邊行走一邊以粒子濾波演算法建立房間的佔據格地圖，同時在地圖中定位自己；地圖完成後，路徑規劃模組把房間切成

區塊，以弓字形路徑逐區覆蓋，確保不重複、不遺漏。這台機器讓「SLAM」這個原本只出現在論文中的名詞，第一次大規模進入一般家庭的客廳。對學習者的啟示是：SLAM 不是遙不可及的尖端科技，它的核心版本可以跑在一顆手機等級的處理器上——這也是本書相信高中生也能學會 SLAM 的原因。

案例 1-2 DARPA 地下挑戰賽：SLAM 的極限考場

2018 至 2021 年，美國國防高等研究計畫署（DARPA）舉辦地下挑戰賽（Subterranean Challenge），要求機器人團隊在人造隧道、地下城市與天然洞穴中自主探索、建圖並尋找目標物。地下環境集合了 SLAM 的所有噩夢：完全沒有 GPS、灰塵與煙霧干擾光達、黑暗讓相機失效、狹窄通道與垂直豎井考驗運動能力。參賽隊伍（包括 MIT、CMU、ETH 等名校團隊）普遍採用多感測器融合——光達 + IMU + 相機 + 熱像儀，並讓輪型機器人、四足機器狗與無人機組成異質團隊協同建圖。這場比賽大幅推進了光達慣性 SLAM（第 18 章）與多機器人 SLAM（第 26 章）的實用化，也證明了一件事：單一感測器、單一機器人的 SLAM 已趨成熟，未知嚴苛環境下的強韌性（Robustness）才是下一個十年的主戰場。

案例 1-3 手機 AR 量尺：口袋裡的視覺慣性 SLAM

打開 iPhone 內建的「測距儀」App，把鏡頭對準桌面移動幾秒，就能在空中拉出一條貼合真實世界的虛擬量尺。這背後是 ARKit 的視覺慣性 SLAM：手機相機以每秒 30 至 60 張的速度追蹤影像特徵點，IMU 以每秒數百次的頻率量測運動，兩者融合後即時估測手機的六自由度位姿，並重建出桌面的平面結構。這個案例適合作為課堂的第一個示範——不需要任何機器人硬體，每位同學的手機裡就住著一套工業級 SLAM 系統。教師可以讓同學實驗：在白牆（無紋理）、昏暗房間、快速甩動手機時量尺會如何失準，親身體會第 1.3 節所述相機感測器的限制。

案例 1-4 從 iMAP 到 3DGS：五年之間，地圖變成了什麼

本書的案例通常談的是成熟技術的應用，這一則例外——它談的是正在發生的事，目的是讓同學感受這個領域的變化速度。

2021 年，一項名為 iMAP 的工作首度展示了以神經隱式表示做即時 SLAM 的可能性，被視為輻射場 SLAM 的起點。此後短短數年間，相關研究呈爆炸性成長：有的以體素特徵格改良、有的改用特徵平面、有的結合點雲式的輻射場。到了 2023 年底，三維高斯潑潑出現，因其顯式表示與即時算繪能力，迅速成為新的主流方向。

這個變化的速度值得同學留意：從一個概念驗證，到成為整個社群的研究主軸，中間只隔了大約三年。相較之下，本書第 21 章的 ORB-SLAM 從 2015 年發表至今，仍是視覺 SLAM 的標準比較基準——十年不衰。

兩者的對比帶出一個值得深思的問題：什麼樣的技術能長存，什麼樣的會被快速取代？筆者的觀察是，解決「根本問題」的方法（如對極幾何、圖優化、閉環偵測）生命週期極長，而解決「表示方式」的方法（地圖用什麼格式儲存、畫面如何算繪）則迭代快速。這也是本書把大部分篇幅放在前者的原因。

對同學的建議是：把新技術當成「又一種表示方式」來理解，問它解決了舊方法的什麼限制、又付出了什麼代價（記憶體？運算？穩健度？）。有了這個框架，無論明年出現什麼新名詞，你都能快速定位它的價值。

實作活動

實作 1-1 感測器看到的世界 vs 實際地圖

目的：體會「以第一人稱視角記錄環境」與「俯視地圖」之間的落差，理解 SLAM 要

解決的資料轉換問題。

器材

- 手機一支（錄影用），或教學用輪型機器人一台。
- 方格紙兩張、鉛筆、量尺（或捲尺）。

步驟

1. 分組（每組 3 至 4 人），選定一條室內路線：例如從教室後門出發，沿走廊直行、右轉、經過飲水機、回到前門。
2. 第一人稱記錄：一位同學手持手機以走路高度錄影走完全程，過程中口述看到的地標（「左手邊有滅火器」「前方是玻璃門」）。
3. 憑記錄畫地圖：回到教室後，只准看影片、不准回到現場，全組合作在方格紙上畫出「感測器看到的世界」——路線、轉角、地標的相對位置，並標注估計的距離。
4. 實地測量：帶著捲尺回到現場，實際測量走廊長度、轉角位置與地標間距，在第二張方格紙上畫出「實際地圖」。
5. 比對與討論：把兩張圖疊在一起，量測誤差最大的地方，討論誤差從哪裡來——距離感失準？轉彎角度估錯？地標記錯順序？

討論問題

1. 走得越遠，你們畫的地圖誤差越大還是越小？這與課文中的「漂移」有什麼關係？
2. 如果路線最後回到出發點，你們畫的終點有沒有跟起點重合？如果沒有，該怎麼「修正」整張圖？這個動作對應課文中的哪一個名詞？

3. 影片中哪些地標最容易辨認、哪些最容易搞混？這對「相機式 SLAM 喜歡什麼樣的環境」有什麼啟發？

延伸挑戰 (大學部)

以手機錄製的影片作為輸入，在電腦上執行開源視覺 SLAM 系統（如 ORB-SLAM3 的單目模式），輸出估測軌跡，與你們手繪的地圖及實測地圖三方比較。此挑戰預告了第 21、22、29 章的內容。

延伸閱讀

1. C. Cadena et al., "Past, Present, and Future of Simultaneous Localization and Mapping: Toward the Robust-Perception Age," IEEE Transactions on Robotics, vol. 32, no. 6, 2016. (SLAM 三十年發展的權威回顧論文，全書多章將反覆引用)
2. H. Durrant-Whyte and T. Bailey, "Simultaneous Localization and Mapping: Part I & Part II," IEEE Robotics & Automation Magazine, 2006. (SLAM 經典入門教程，建立理論基礎)
3. S. Thrun, W. Burgard, and D. Fox, Probabilistic Robotics, MIT Press, 2005. (機率機器人學聖經，本書第三篇數學基礎的主要參考)
4. MIT OpenCourseWare, 16.485 Visual Navigation for Autonomous Vehicles (VNAV). (免費公開的視覺導航完整課程，含講義與實驗)
5. CMU 16-833 Robot Localization and Mapping 課程網站. (以狀態估測與最佳化為主軸的 SLAM 研究所課程)
6. Cyrill Stachniss (University of Bonn) Robot Mapping 系列課程影片，YouTube 免費公開. (以直觀動畫講解 SLAM 演算法，適合自學)

7. C. Campos et al., "ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM," IEEE Transactions on Robotics, 2021. (本書第 22 章主角的原始論文)

全書導覽：三十一章的地圖

作為第一章的收尾，這裡提供一份全書的鳥瞰圖。同學不必現在就記住它，但在往後每次感到迷失時，可以回來看看自己在哪一段旅程上。

圖 1-9 全書九篇的學習路徑

【第一篇 觀念建立】第 1–3 章

SLAM 是什麼、國際頂尖大學怎麼教、導航系統的完整架構

【第二篇 機器人的感官】第 4–9 章

室內定位、衛星、地磁、IMU、光達、相機——每一種的能力與限制

【第三篇 數學工具箱】第 10–12 章

座標轉換 (確定性) → 機率估測 (不確定性) → 最佳化 (求最一致解)

【第四篇 第一套完整系統】第 13–16 章

2D 佔據格 → Gmapping (濾波路線) → Cartographer (最佳化路線) → 掃地機器人

【第五篇 升到三維】第 17–18 章

點雲處理與配準 → LOAM 系列與自主巡檢

【第六篇 換成相機】第 19–23 章

特徵 → 對極幾何 → ORB-SLAM2 → ORB-SLAM3 → 視覺導航任務

【第七篇 極限平台】第 24–26 章

毫瓦級微型機器人 → 無人機 → 多機器人與雲端邊緣協作

【第八篇 動手實作】第 27–30 章

環境建置 → 光達建圖導航 → 視覺 SLAM → 專題與評量

【第九篇 通往未來】第 31 章

AI 視覺模型、VLM 語言導航、邊緣部署、風險與展望

三條可選的閱讀路線

- 高中 / 入門路線：第 1–3 章建立觀念 → 第 8、9 章認識光達與相機 → 第 13、16 章理解 2D 建圖與掃地機器人 → 第 27、28 章動手做模擬。全程不需深入數學，各章的實作活動多數可用模擬完成。
- 大學部完整路線：依序讀第一篇到第六篇，搭配第八篇的實作章節同步進行。第三篇的數學是關鍵門檻，建議搭配第 11、12 章的實作活動來建立直覺。
- 專題 / 研究路線：先快速瀏覽第一至三篇建立共同語言，再依研究方向深入—

—做光達選第五篇、做視覺選第六篇、做微型平台選第七篇、做 AI 整合選第九篇，最後以第 30 章的專題規劃收斂。

無論選擇哪條路線，本書建議的學習節奏都相同：先建立直覺、再理解原理、然後動手驗證、最後刻意尋找失效邊界。第 27.7 節會把這個節奏展開成五個具體階段。

小結

1. SLAM (同步定位與建圖) 要解決的核心問題是：機器人在沒有地圖、不知道自己位置的情況下，一邊移動、一邊建立地圖、一邊修正自身位姿。
2. 定位需要地圖、建圖需要定位，兩者形成雞生蛋、蛋生雞的循環依賴；SLAM 的解法是把兩者視為同一個估測問題，利用環境觀測持續互相修正，並以閉環偵測攤平長期累積誤差。
3. 機器人的感測器分為本體感測 (里程計、IMU，連續但漂移) 與外部感測 (相機、光達、無線電、地磁，提供環境參考但各有弱點)；SLAM 的精髓在於感測器融合、截長補短。
4. 自動駕駛、無人機、掃地機器人與 AR / VR 四大應用共用同一套 SLAM 地基，差別只在感測器組合、運算資源與精度要求。
5. SLAM 系統的輸出包含軌跡、2D 佔據格地圖、3D 點雲、語意地圖與拓樸地圖五大形式，各自對應不同的應用需求。
6. 數學上，SLAM 是在給定觀測 z 與控制 u 之下，估測軌跡 x 與地圖 m 的聯合機率分布；現代系統以「前端—後端—閉環」的架構求解。

下一章，我們將走訪 MIT、CMU、Stanford、ETH / UZH 與 Bonn / Freiburg

等世界知名大學的 SLAM 課程，看看國際一流學府如何安排這門學問的學習路徑，並從中提煉出本書「感測器 → 數學 → 前端 → 後端 → 閉環 → 專題」的教學主軸。

台灣自編大學教科書

@prof

《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第 2 章

世界知名大學 SLAM 課程導覽

編者：李世淵 (機器人叫獸)

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

本章在全書的位置

第 1 章建立了 SLAM 的問題意識，本章則回答一個實際的問題：這門學問，世界上最好的學校是怎麼教的？這不只是見聞的補充——本書的整體架構正是從這些課程的共同骨架提煉而來，因此讀懂本章，等於拿到了理解後續二十九章編排邏輯的鑰匙。

圖 2-0 第 2 章與全書的連結地圖

【承接前章】第 1 章的六大感測器、雞生蛋問題、五種地圖表示，在本章各校課程中都能找到對應的單元。

【本章負責】導覽五所頂尖大學的 SLAM 課程，並提煉出它們的共同架構。

【往下延伸】

2.1 MIT VNAV → 第 7 章 IMU 預積分、第 12 章流形最佳化、第 22 章視覺慣性 SLAM

2.2 CMU 16-833 → 第 11 章機率濾波、第 12 章因子圖與 GTSAM

2.3 Stanford → 第 3 章導航系統架構、第 28 章規劃與控制

2.4 ETH / UZH → 第 9 章相機模型、第 20 章對極幾何、第 25 章無人機 VIO

2.5 Bonn / Freiburg → 第 11 章粒子濾波、第 13 章佔據格、第 14 章 Gmapping

2.6 共同架構 → 本書九篇的編排依據

【先備知識】無。本章是導覽性質，不涉及數學推導。

學習目標

工欲善其事，必先利其器；學欲深其道，必先觀其路。本章帶領同學「站在巨人的肩膀上」，導覽五所世界頂尖大學的 SLAM 相關課程，看看國際一流學府如何安排這門學問的學習路徑。學完本章之後，同學應能夠達成下列目標：

7. 說出 MIT、CMU、Stanford、ETH / UZH、Bonn / Freiburg 五個學術重鎮的 SLAM 課程名稱、主授教授與課程定位。
8. 比較各校課程的切入角度：視覺導航、機率推論、系統整合、視覺慣性、機率建圖，並說明其異同。
9. 歸納 SLAM 教材的共同架構：感測器 → 數學 → 前端 → 後端 → 閉環 → 專題，並對應到本書的九篇章節。
10. 知道各校公開的免費學習資源（OpenCourseWare、YouTube 課程影片、公開講義）在哪裡、如何取用。
11. 依照自己的程度與目標，規劃一條適合自己的 SLAM 自學路線。

核心概念

關鍵詞	英文 / 代號	概念說明
視覺導航	Visual Navigation	以相機為主要感測器的機器人導航技術，MIT VNAV 課程的主軸。
狀態估測	State Estimation	從帶雜訊的量測中推算系統狀態（位姿、速度、地圖）的數學方法。

關鍵詞	英文 / 代號	概念說明
機率圖模型	Probabilistic Graphical Model	以節點與邊表示隨機變數關係的模型，是現代 SLAM 後端的理論語言。
因子圖	Factor Graph	一種機率圖模型，把量測表示為「因子」，CMU 16-833 的核心工具。
視覺里程計	Visual Odometry, VO	只用相機影像逐幀估測相機運動的技術，UZH 課程的期末專題。
視覺慣性里程計	VIO	相機加 IMU 融合的里程計，手機 AR 與無人機的核心技術。
流形最佳化	Manifold Optimization	在旋轉群等彎曲空間上做最佳化的數學技術，MIT VNAV 的特色主題。
佔據格建圖	Occupancy Grid Mapping	Freiburg / Bonn 課程系統化整理的經典建圖方法。
開放式課程	OpenCourseWare, OCW	大學免費公開的完整課程教材，MIT OCW 是最著名的代表。
大規模開放線上課程	MOOC	如 edX、Coursera 上的線上課程，ETH 自主移動機器人課程即有 MOOC 版。

2.1 MIT VNAV：視覺導航、自駕車、無人機與最佳化

麻省理工學院 (MIT) 航空航天系開設的 16.485 「自主載具視覺導航」 (Visual Navigation for Autonomous Vehicles，簡稱 VNAV)，由 Luca Carlone 教授主持的 SPARK 實驗室開授，是全世界最具代表性的視覺導航研究所課程之一。Carlone 教授本人是 SLAM 領域的重量級學者——第 1 章延伸閱讀中那篇 2016 年的 SLAM 回顧論文，他正是主要作者之一。

課程主軸與特色

VNAV 的課程設計有一條清晰的邏輯線：從幾何出發、以最佳化收斂、用無人機驗證。課程內容大致分為四大塊：

- 三維幾何基礎：旋轉表示 (旋轉矩陣、四元數)、剛體變換、相機模型與二視圖幾何——對應本書第 10 章與第 20 章。
- 流形上的最佳化 (Optimization on Manifolds)：旋轉不是普通的向量，不能直接做加減，必須在「流形」這種彎曲空間上進行最佳化。這是 VNAV 最具特色的數學主題，也是理解現代 SLAM 後端的鑰匙——本書第 12 章會以較平易的方式介紹其精神。
- 視覺前端：特徵提取與匹配、視覺里程計、地點辨識 (Place Recognition) 與閉環偵測——對應本書第 19 至 21 章。
- SLAM 後端與系統：位姿圖優化、Bundle Adjustment、視覺慣性融合，以及語意理解的前沿延伸——對應本書第 12、22、31 章。

教學方式：無人機團隊專題

VNAV 最令人印象深刻的是它的實作安排：學生分組操作真實無人機，逐週完成從姿態控制、軌跡追蹤、視覺特徵、視覺里程計到完整導航系統的實驗，期末以無人

機競賽式專題收尾。這種「每一週的數學都立刻在飛行器上驗證」的設計，正是本書第八篇實作章節效法的精神。

VNAV 的完整教材（講義、作業、實驗指引）已在 MIT OpenCourseWare 免費公開（課號 16.485，2020 年秋季版），是自學視覺導航最完整的免費資源之一。對本書讀者而言，建議在讀完第三篇數學基礎後，再回頭挑戰 VNAV 的講義，會有事半功倍的效果。

2.2 CMU Robot Localization and Mapping：機器人定位、建圖與後端最佳化

卡內基美隆大學（CMU）機器人研究所（Robotics Institute）是全球歷史最悠久、規模最大的機器人研究機構，其 16-833「機器人定位與建圖」（Robot Localization and Mapping）由 Michael Kaess 教授主講。Kaess 教授是 SLAM 後端演算法的代表性人物——現代圖優化 SLAM 廣泛使用的 iSAM / iSAM2 增量式平滑演算法與 GTSAM 函式庫，正是出自他與 Frank Dellaert 教授的研究團隊。

課程主軸與特色

16-833 的課程描述開宗明義指出：定位與建圖是行動機器人在真實世界運作的基礎能力，而比單獨解決這兩個問題更具挑戰的，是它們的組合——SLAM；穩健的解法必須能處理感測量測中固有的不確定性，同時即時提供定位與地圖估測。課程因此把重心放在「線性代數與機率圖模型交會處的高效機率推論演算法」，內容包含：

- 貝氏濾波家族：卡爾曼濾波、擴展卡爾曼濾波（EKF-SLAM）、粒子濾波與 FastSLAM——對應本書第 11、14 章。
- 平滑與圖優化：因子圖（Factor Graph）、最小平方、iSAM 增量式求解——對應本書第 12 章。

- 資料關聯與閉環：量測到底對應地圖中哪個地標？這是 SLAM 最容易出錯的環節。
- 最先進系統導讀：課程後段帶學生閱讀當代 SLAM 系統論文，理解理論如何落地。

與 MIT VNAV 的對照

如果說 VNAV 是「從視覺與幾何切入」，16-833 就是「從機率與推論切入」：VNAV 花大量篇幅講相機與流形，16-833 則把濾波、因子圖與稀疏線性代數講到透徹。兩門課合起來，恰好覆蓋現代 SLAM 前端與後端的完整知識版圖——這也是本書把第二篇（感測器）與第三篇（數學）分開安排、再於第四至六篇整合成系統的原因。

2.3 Stanford Robot Autonomy：自主機器人的定位、感知、SLAM、規劃

史丹佛大學的「機器人自主原理」（Principles of Robot Autonomy，課號 AA274A / CS237A）由航太系與電腦科學系合開，Marco Pavone 教授等人主持的自主系統實驗室（ASL）負責授課。與前兩門課不同，這門課的定位不是「把 SLAM 一個主題挖深」，而是「把自主機器人整個系統走一遍」。

課程主軸與特色

Stanford 這門課涵蓋自主機器人的完整技術堆疊，SLAM 是其中承上啟下的一環：

- 運動與控制：運動學模型、軌跡追蹤、回授控制——機器人「會動」的基礎。
- 感知：相機模型、影像特徵、物件偵測與機器學習感知——機器人「看得見」。
- 定位與 SLAM：貝氏濾波、蒙地卡羅定位、EKF-SLAM——機器人「知道自己在哪」。

- 規劃與決策：路徑規劃 (A*、RRT)、軌跡最佳化、任務決策——機器人「知道要去哪、怎麼去」。
- 系統整合：全學期以 ROS 為骨幹，期末在模擬與實體機器人上整合出完整的主導航系統。

對本書的啟示

Stanford 的課程設計回答了一個重要問題：SLAM 學了要幹嘛？答案是——SLAM 是自主系統的中樞，往下承接感測器資料，往上支撐路徑規劃與任務執行。本書第 3 章的「導航系統總覽」與第 28 章的「Navigation Stack 導航」正是採用這個系統觀：讓同學永遠知道，眼前這個演算法在整台機器人中扮演什麼角色。這種「以系統帶演算法」的教學法，特別適合高中與大學部的專題導向課程。

2.4 ETH Zürich / UZH：視覺慣性里程計、VIO、MAV 無人機導航

瑞士蘇黎世聯邦理工學院 (ETH Zürich) 與蘇黎世大學 (UZH) 地理相鄰、師資互通，共同構成歐洲行動機器人研究的心臟地帶。這裡有兩條互補的課程脈絡，分別代表「系統派」與「視覺派」。

ETH：自主移動機器人 (Autonomous Mobile Robots)

由 Roland Siegwart 教授領銜的 ETH 自主系統實驗室 (ASL) 長期開設「自主移動機器人」課程，其同名教科書 (MIT Press 出版，Siegwart、Nourbakhsh 與 Scaramuzza 合著) 是行動機器人領域最廣泛使用的教材之一，涵蓋運動學、感知、定位與導航規劃。這門課也推出了 edX 平台上的 MOOC 版本，讓全世界的學習者免費修習。此外，ASL 在視覺慣性估測上的研究成果——ROVIO、OKVIS 等開源系統——已成為無人機與行動裝置定位的重要基礎建設，本書第 22、25 章會再遇到它們。

UZH：行動機器人視覺演算法 (Vision Algorithms for Mobile Robotics)

UZH 機器人與感知研究群 (RPG) 的 Davide Scaramuzza 教授，是視覺里程計與事件相機 (Event Camera) 研究的國際領軍人物。他開設的「行動機器人視覺演算法」課程，內容涵蓋影像形成、濾波、特徵提取、多視圖幾何、稠密重建、追蹤、影像檢索、事件視覺、視覺慣性里程計與 SLAM——課程網站直接點名，這些正是 Apple ARKit、Microsoft HoloLens、Meta Quest 與 NASA 火星直升機背後的演算法。

這門課最精彩的設計是「積木式期末專題」：每一講之後都有對應的實作練習，學生逐週實作視覺里程計的一個模組 (特徵提取、匹配、位姿估測、三角測量.....)，期末把所有積木組裝成一條完整可跑的視覺里程計管線，並在 KITTI 等真實資料集上驗證。所有課程影片與講義都在 RPG 網站與 YouTube 免費公開。本書第九章、第 19 至 20 章的知識架構，很大程度上參考了這門課的鋪陳順序。

2.5 University of Bonn / Freiburg : Robot Mapping 與 Cyrill Stachniss SLAM 課程

德國的弗萊堡大學 (University of Freiburg) 與波昂大學 (University of Bonn) 是機率機器人學 (Probabilistic Robotics) 的發源重鎮。Wolfram Burgard 教授 (《Probabilistic Robotics》作者之一) 在弗萊堡長期開設「行動機器人導論」與「機器人建圖」 (Robot Mapping) 課程；其學生 Cyrill Stachniss 教授其後在波昂大學延續並發揚這條教學路線，開設以 SLAM 與環境建模為主軸的完整課程。

課程主軸與特色

Freiburg / Bonn 路線的最大特色，是把機率方法講得極其系統化、極其循序漸進：

- 機率基礎：貝氏法則、運動模型、觀測模型——對應本書第 11 章。
- 濾波式 SLAM：EKF-SLAM、UKF、粒子濾波與 FastSLAM、Gmapping 的 RBPF 原理——對應本書第 11、14 章。
- 圖優化式 SLAM：最小平方、位姿圖、閉環與強健化——對應本書第 12 章。
- 建圖方法：佔據格建圖、雷射掃描匹配、地圖表示——對應本書第 13 章。

對自學者最友善的資源

Stachniss 教授把歷年課程完整錄影上傳 YouTube 免費公開，從五分鐘的單一觀念短片到九十分鐘的完整講課都有，搭配動畫與板書推導，被全球 SLAM 學習者公認為「最適合自學入門」的影音資源。特別值得一提的是，本書第 14 章的主角 Gmapping 正是出自 Stachniss 與 Grisetti、Burgard 的研究成果——讀完第 14 章後去看原作者親自講解，是最原汁原味的學習體驗。

2.6 各校課程與本書章節的對照

在歸納共同架構之前，先做一件對同學很實用的事：把五所學校的核心單元，逐一對應到本書的章節。這張表有兩個用途——一是當你想深入某個主題時，知道該去看哪一門國際課程；二是當你修習這些線上課程時，知道可以回頭參照本書的哪一章。

表 2-0 五校課程單元與本書章節對照

【MIT VNAV：視覺導航與最佳化】

影像形成與相機模型	→ 第 9 章
流形上的最佳化	→ 第 10 章（旋轉表示）、第 12 章（圖優化）
IMU 預積分	→ 第 7.5 節

視覺慣性里程計 → 第 22 章

【CMU 16-833：定位、建圖與後端】

貝氏濾波與粒子濾波 → 第 11 章

EKF-SLAM → 第 11.6 節

因子圖與 iSAM2 → 第 12.6 節、第 26 章 (多機器人)

閉環偵測 → 第 19.7 節、第 21.5 節

【Stanford Robot Autonomy：系統整合】

機器人系統架構 → 第 3 章

感知與狀態估測 → 第二篇、第 11 章

路徑規劃與控制 → 第 3.4 節、第 28.6 節

【ETH / UZH：視覺與無人機】

特徵偵測與匹配 → 第 19 章

多視圖幾何 → 第 20 章

視覺慣性融合 → 第 22 章

微型飛行器導航 → 第 25 章

【Bonn / Freiburg：機率建圖】

佔據格地圖 → 第 13 章

掃描匹配與 ICP → 第 13.3 節、第 17.3 節

Gmapping 與 RBPF → 第 14 章

圖優化 SLAM → 第 12 章、第 15 章

觀察這張表，同學會發現一個現象：沒有任何一所學校的課程涵蓋全部主題。MIT 偏視覺慣性與最佳化、Bonn 偏機率建圖、Stanford 偏系統整合。這是合理的——一門十四週的課無法窮盡整個領域，各校自然依其研究專長取捨。

本書的編排策略因此是「取各校之長並補其缺口」：以 Bonn 的機率建圖為第三、四篇的骨幹、以 MIT 與 UZH 的視覺慣性為第六篇的主線、以 CMU 的因子圖為第十二章的核心、以 Stanford 的系統觀貫穿第三章與第八篇，再補上台灣課程實際需要卻在國際課程中較少著墨的部分——例如第 4 章的室內定位、第 6 章的地磁導航、第 16 章的掃地機器人，以及第九篇的 AI 語意導航。

2.7 SLAM 教材的共同架構：感測器 → 數學 → 前端 → 後端 → 閉環 → 專題

走訪五校之後，我們把鏡頭拉遠，會發現一個有趣的現象：儘管切入角度各異——MIT 從幾何、CMU 從機率、Stanford 從系統、蘇黎世從視覺、德國從建圖——這些課程的知識骨架卻驚人地一致，都可以歸納為六個階段：

圖 2-1 SLAM 教材的共同架構

① 感測器 (Sensors)：認識相機、光達、IMU 等資料來源與其誤差特性

↓

② 數學 (Math) : 座標變換、機率、濾波、最佳化——處理不確定性的語言

↓

③ 前端 (Front-End) : 特徵提取、資料關聯、掃描匹配、里程計

↓

④ 後端 (Back-End) : 濾波或圖優化，求出最一致的軌跡與地圖

↓

⑤ 閉環 (Loop Closure) : 辨識重訪地點，修正全域累積誤差

↓

⑥ 專題 (Projects) : 在真實或模擬平台上整合驗證，完成完整系統

本書的九篇結構，正是把這六階段展開並在地化的結果：第二篇對應「感測器」、第三篇對應「數學」、第四至六篇把「前端、後端、閉環」落實在 2D 光達、3D 光達與視覺三種系統上、第七至八篇對應「專題」、第九篇則延伸到各校課程尚在快速演進的 AI 語意導航前沿。

學校 / 課程	主授教授	切入角度	代表特色	對應本書
MIT 16.485 VNAV	Luca Carlone	幾何與最佳化	流形最佳化、無人機專題	第 10、12、20 章
CMU 16-833	Michael Kaess	機率推論	因子圖、iSAM / GTSAM	第 11、12 章
Stanford AA274A	Marco Pavone 等	系統整合	ROS 全堆疊、期末整合	第 3、27、28 章

學校 / 課程	主授教授	切入角度	代表特色	對應本書
ETH 自主移動機器人	Roland Siegwart	行動機器人 總論	經典教科書、edX MOOC	第 3、7 章
UZH 視覺演算法	Davide Scaramuzza	視覺與 VIO	積木式 VO 專題、 事件相機	第 9、19、 20 章
Bonn / Freiburg	Stachniss、 Burgard	機率建圖	YouTube 全公開、 Gmapping	第 11、13、 14 章

表 2-1 五校 SLAM 課程總覽與本書章節對應

公式或流程

一、以「估測問題」統一五校課程

五校課程雖然風格迥異，但都圍繞著第 1 章介紹的同一條核心公式。回顧線上 SLAM 的機率定義：

$$p(x(t), m \mid z(1:t), u(1:t))$$

五校課程的差別，其實只是「攻打這條公式的不同兵法」：

- CMU 與 Freiburg / Bonn 專攻「如何求解這個機率分布」：濾波法（第 11 章）沿時間遞迴更新，圖優化法（第 12 章）把它轉成最小平方問題求解。
- MIT 與 UZH 專攻「觀測 z 從哪裡來」：相機影像如何變成幾何約束——特徵、對極幾何、重投影誤差（第 19、20 章）。
- Stanford 專攻「解出來之後怎麼用」：位姿與地圖如何餵給路徑規劃與控制，構成完整自主系統（第 3、28 章）。

二、SLAM 自學路線規劃流程

綜合五校資源，本書建議的自學路線如下，同學可依自己的起點選擇入口：

圖 2-2 SLAM 自學路線建議流程

起點判斷：我有程式與線性代數基礎嗎？

└─ 還沒有 → 先看 ETH 自主移動機器人 MOOC (觀念導向、門檻最低)

└─ 已具備 → 進入下一步

↓

想走視覺路線 → UZH 課程影片 + 逐週實作視覺里程計積木

想走機率路線 → Stachniss YouTube 課程 + 本書第三篇推導

↓

進階深化 → MIT VNAV OCW 講義 (流形最佳化) + CMU 16-833 (因子圖)

↓

系統整合 → 以 ROS 完成本書第八篇實作與第 30 章專題

2.8 開放教育的演進：從 MOOC 到具身智慧課程

本章介紹的課程有一個共同特徵：教材公開。這件事在今天看來理所當然，但它其實是過去十餘年才發生的變化，而且至今仍在演進。理解這個演進，對同學規劃自己的學習路徑很有幫助。

三個階段的變化

知識取得的方式在過去十幾年發生了根本改變。這個歷程大致始於 2011 年前後，當時史丹佛的教授們把人工智慧課程透過線上形式向所有人開放；在那之前，人工智慧一般只能透過數量有限的大學課程、或書籍與論文來學習，而且這些資源主要集中在已開發國家，新興市場的學習者難以取得。大規模開放線上課程的出現，開啟了所

謂「知識民主化」的行程。

對 SLAM 這類需要大量背景知識的領域，這個變化尤其關鍵。在 2010 年代之前，想學 SLAM 幾乎只能進入少數幾個研究室；今天，任何一位台灣的學生只要有網路，就能取得與麻省理工學生相同的講義與作業。本書第 2.2 節案例提到的 Stachniss 教授把整套課程放上影音平台，正是這個趨勢的代表。

第二階段：硬體導向的開放課程

純影片課程有一個限制：學生看得懂理論，卻沒有機器人可以驗證。因應這個缺口，出現了結合實體平台的開放課程。例如 2021 年三月，Duckietown 基金會發布了第一個以硬體為基礎的人工智慧與機器人大規模開放線上課程，名為「以 Duckietown 學自駕車」，在公開平台上免費提供；該課程讓學習者親手操作，使真實的機器人自己做決策並完成廣泛定義的任務，從理論、實作、到在模擬與真實機器人上部署，逐步進行；並為此從零設計了一個專門支援這種學習體驗的機器人平台。

這個模式與本書第八篇的設計理念一致：先在模擬中把流程與參數調通，再搬到便宜的實體硬體上驗證。第 27.1 節建議的容器化統一環境、第 28 章的「先模擬再實體」路徑，都是同一個思路。

第三階段：具身智慧進入課程

近兩年課程內容出現了新的變化。大型語言模型的突破正被應用於機器人領域，具身智慧讓機器人能理解自然語言指令並轉化為實體動作；而 Google DeepMind 與 OpenAI 等組織正在開發能從龐大資料集學習通用操作技能的機器人基礎模型，理解如何為特定應用微調這些模型，將成為必備能力。

與此同時，另一個方向也進入課程：由於雲端連線的限制，裝置端人工智慧推論的需求持續上升，現代課程越來越常涵蓋模型最佳化技術（量化、剪枝、知識蒸餾）

與邊緣人工智慧框架及硬體加速。這正是本書第 24 章（毫瓦級平台）與第 31.7 節（邊緣部署）所討論的內容——本書把這兩個主題納入，正是為了對齊這個教學趨勢。

對學習方法的三點啟示

1. 模擬優先：有研究者指出，模擬專案能以硬體學習約十分之一的成本，教會約八成的必要概念；等基礎穩固後，硬體工作會進行得更有效率，而過早嘗試複雜專案則會導致挫折。這與本書第八篇的建議完全一致。
2. 漸進式挑戰：建議的學習結構是逐級推進——基本運動（讓機器人走出方形、圓形、8 字形）、感測器整合（加入障礙偵測與避障）、環境建圖（以 SLAM 技術建立空間地圖）、物件操作（視覺偵測與抓取）、自主任務（把各項能力整合為完整任務）。對照本書，這五級分別對應第 3 章、第 8 章、第 13 至 15 章、第 23 章與第 30 章。
3. 線上課程與教科書互補：線上課程長於動態演示與最新進度，教科書長於系統性與可反覆查閱。本書建議的用法是：以本書建立架構、以線上課程補充直覺、以開源專案驗證理解——這三者的分工，正是第 27.6 節閱讀開源專案方法的延伸。

案例

案例 2-1 從課堂作業到改變產業：GTSAM 的故事

CMU 16-833 課堂上教的因子圖與增量式平滑，並不只是紙上談兵。Kaess 教授與 Dellaert 教授開發的 GTSAM (Georgia Tech Smoothing and Mapping) 函式庫，把這些理論包裝成高效、開源的 C++ 工具箱，如今已是全球機器人公司與研究室的標準配備——從水下機器人、自駕車到 AR 裝置的後端最佳化，處處可見它的身影。這個案例告訴我們：一門課的核心作業（在 16-833 中，學生要親手實作因子圖求解

器)，可能就是未來產業基礎建設的雛形。本書第 12 章介紹 g2o、Ceres 與 GTSAM 時，同學將親手使用這個「從課堂長出來」的工具。

案例 2-2 YouTube 上的 SLAM 教授：知識公開的力量

Stachniss 教授從 2013 年起把 Freiburg 時期的 Robot Mapping 課程完整錄影公開，其後在波昂持續更新，累積數百支影片、被觀看數百萬次。世界各地無數自學者——包括許多台灣的研究生與工程師——都是靠這些免費影片入門 SLAM 的。同樣地，MIT 把 VNAV 完整放上 OCW、UZH 公開所有講義與影片、ETH 推出 MOOC。這股「頂尖大學知識公開」的浪潮，意味著學習 SLAM 的門檻已從「能否進入名校」變成「有沒有一份好的學習地圖與持續的毅力」。本書正是希望扮演那份用繁體中文寫成的學習地圖。

案例 2-3 台灣學生的視角：從大學教室到國際課程

對台灣的學習者而言，五校課程是很好的「進階疊代」素材，但直接啃原文教材常會遇到兩個障礙：一是數學符號與英文術語的雙重門檻，二是缺乏在地可操作的實驗場域。本書的策略是「在地打底、國際接軌」：先用中文與熟悉的場景（教室建圖、校園巡檢、掃地機器人）把觀念與實作打穩，每章的延伸閱讀再指向對應的國際課程單元。以第 14 章 Gmapping 為例：先在大學教室用小車完成建圖實作，再回頭看 Stachniss 講解 RBPF 的影片，多數同學會發現原本抽象的粒子濾波突然「看得見了」。

案例 2-4 為什麼還需要教科書：MOOC 時代的大學課程

本章介紹了大量免費的線上資源，這自然會引出一個尖銳的問題：既然頂尖大學的課程都能免費看，為什麼還要修課？為什麼還要有這本書？

這個問題在教育界確實被嚴肅討論。有學者指出，隨著教學數位轉型快速推進，大學已開始面臨來自大規模開放線上課程的重大競爭，因此必須讓自己與這些課程有

所區隔，才能向學生說明三到五年學位的附加價值何在。

筆者認為，教科書與課堂在這個時代的價值，至少有四個是線上影片難以取代的：

- 在地化：國際課程以英文授課、以歐美場域為例。本書刻意採用台灣的专业術語（第 1 章的用語討論）、以大學校園與台灣的高中教室為場域、並納入台灣特別關切的主題（如第 6 章地磁導航在鋼構建築中的表現、第 16 章掃地機器人）。
- 系統性：線上課程各自獨立，學生需自行拼湊。本書的九篇架構、每章的「本章在全書的位置」、以及跨章的交叉引用，正是為了提供這種整體視野。
- 可查閱性：影片難以快速回查某個公式或參數。教科書的索引、目錄、章節編號，在實作卡關時遠比重看影片有效率。
- 失敗經驗的傳承：這是本書最刻意經營的部分。國際課程通常展示成功的結果，而本書每一章都討論「常見失敗」——第 14.6 節的四大失敗模式、第 25.5 節的安全原則、第 28.7 節的除錯順序。這些來自真實教學現場的經驗，是任何影片都不會告訴你的。

所以本書的定位很明確：它不與線上課程競爭，而是作為它們的地圖與註解。同學若能同時使用兩者——以本書建立骨架、以國際課程補充血肉——得到的會比任何單一來源都多。這也是第 2.2 節案例強調「知識公開的力量」時，真正想傳達的訊息：資源不是問題，如何組織資源才是。

實作活動

實作 2-1 國際 SLAM 課程資源探索與比較報告

目的：親手找到並瀏覽五校公開資源，建立自己的學習資源地圖，並練習閱讀英文課

程網頁的能力。

步驟

5. 分組（每組 2 至 3 人），每組認領一所學校：MIT VNAV、CMU 16-833、Stanford AA274A、UZH 視覺演算法、Bonn Stachniss 課程。
6. 搜尋並開啟該課程的官方網站或 OCW / YouTube 頁面，找出：課程名稱與課號、主授教授、最近一次開課的主題週次表（Syllabus）。
7. 挑選一週你們「看得懂標題」的主題（例如 Kalman Filter、Feature Extraction），瀏覽該週講義的前五頁或影片的前十分鐘。
8. 製作一頁簡報回答：這門課假設學生已經會什麼？最先教什麼？作業或專題長什麼樣子？哪一部分對應本書的哪一章？
9. 各組上台報告三分鐘，全班合力完成一張「五校課程比較表」，貼在教室或課程網站上，作為整學期的參考地圖。

討論問題

6. 五校課程中，哪一門的入門門檻最低？哪一門最需要數學基礎？你的判斷依據是什麼？
7. 如果只能追蹤一個免費資源搭配本書自學，你會選哪一個？為什麼？
8. （大學部）比較 VNAV 與 16-833 的專題要求，哪一種評量方式更能檢驗「真的學會了 SLAM」？

延伸挑戰（大學部）

從 UZH 課程網站下載第一份實作練習（通常是特徵提取或相機模型），嘗試在自己的電腦上執行，並記錄遇到的環境設定問題與解決過程——這份紀錄將在第 27 章建置 SLAM 開發環境時派上用場。

延伸閱讀

4. MIT OpenCourseWare, 16.485 Visual Navigation for Autonomous Vehicles (VNAV), Fall 2020. (完整講義、作業與無人機實驗指引，免費公開)
5. CMU 16-833 Robot Localization and Mapping 課程網站
(cs.cmu.edu/~kaess/teaching/16833) . (歷年課綱與閱讀清單)
6. Stanford AA274A / CS237A Principles of Robot Autonomy I 課程網站與公開講義. (自主機器人全堆疊課程)
7. R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, Introduction to Autonomous Mobile Robots, 2nd ed., MIT Press, 2011. (ETH 課程指定教科書，另有 edX MOOC 版本)
8. UZH Robotics and Perception Group, Vision Algorithms for Mobile Robotics 課程頁面 (rpg.ifi.uzh.ch/teaching.html) . (全部講義與影片公開)
9. Cyrill Stachniss, Robot Mapping / SLAM 系列課程影片, YouTube (頻道 : Cyrill Stachniss) . (最適合自學的 SLAM 影音資源)
10. S. Thrun, W. Burgard, and D. Fox, Probabilistic Robotics, MIT Press, 2005.
(Freiburg 路線的理論總整合，本書第三篇主要參考)
11. D. Scaramuzza and F. Fraundorfer, "Visual Odometry: Tutorial," IEEE Robotics & Automation Magazine, 2011. (視覺里程計的經典入門教程，分上下兩部)

從課程導覽到系統架構

本章的最後，把視線轉回本書自身的旅程。

第 2.6 節歸納出五校課程的共同架構：感測器 → 數學 → 前端 → 後端 → 閉環 →

專題。這個架構之所以反覆出現，不是因為互相抄襲，而是因為它反映了 SLAM 問題的內在邏輯——你必須先有資料（感測器），才能談如何處理資料（數學）；必須先能估計相鄰時刻的關係（前端），才能談如何全域一致（後端）；而閉環是後端能力的極致展現，專題則是所有能力的整合驗收。

本書的九篇正是這個邏輯的展開，只是把每個環節都放大成完整的篇幅：

圖 2-8 共同架構與本書九篇的對應

感測器 → 第二篇（第 4–9 章） 六種感測器各一章，含失效分析

數 學 → 第三篇（第 10–12 章）座標、機率、最佳化三大工具

前 端 → 第 13、17、19、20 章 掃描匹配、點雲配準、特徵與幾何

後 端 → 第 12、14、15、21 章 濾波與最佳化兩條路線

閉 環 → 第 15.4、21.5、22.6 節 從偵測、驗證到全域修正

專 題 → 第八篇（第 27–30 章） 環境、實作、評量

【本書增補的部分】

第一篇 觀念與導覽 （國際課程通常直接進入技術）

第七篇 極限平台 （微型機器人、無人機、多機協作）

第九篇 AI 語意導航 （對齊具身智慧的教學趨勢）

下一章我們就要進入這個架構的第一站——不是感測器，而是比感測器更上位的東西：機器人導航系統的完整架構。第 3 章會回答「SLAM 在整個導航系統中處於什麼位置」，並介紹反應式、審議式與混合式三種架構、代價地圖與路徑規劃、以及輪

型、履帶、四足、無人機等不同平台的導航需求差異。有了那張系統藍圖，第二篇的六種感測器才知道各自該安裝在什麼位置上。

小結

8. MIT VNAV 以幾何與流形最佳化為主軸，用無人機專題貫穿全學期，完整教材已在 OCW 免費公開。
9. CMU 16-833 把 SLAM 定位為「感測不確定性下的即時估測問題」，專攻濾波、因子圖與圖優化後端，是 iSAM / GTSAM 的發源地。
10. Stanford 機器人自主課程以系統整合為特色，SLAM 被放在「感知—定位—規劃—控制」的完整堆疊中教授，全程以 ROS 實作。
11. ETH 的自主移動機器人課程（含 edX MOOC）提供最平易的總論入口；UZH Scaramuzza 課程則以積木式視覺里程計專題深入視覺與 VIO。
12. Freiburg / Bonn 路線把機率建圖講得最系統化，Stachniss 教授的 YouTube 課程是公認最適合自學的免費影音資源。
13. 五校課程共享同一個知識骨架：感測器 → 數學 → 前端 → 後端 → 閉環 → 專題；本書九篇即依此架構展開，並以台灣校園場域在地化。

下一章，我們將把鏡頭從「課程」轉回「機器人本身」，綜覽機器人導航系統的完整架構——Sense–Think–Act 循環、地圖與定位的分類、以及 SLAM 在整個導航系統中的位置，為第二篇的感測器之旅做好準備。

台灣自編大學教科書

@prof

《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第 3 章

機器人導航系統總覽

編者：李世淵 (機器人叫獸)

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

本章在全書的位置

第 1 章說明了 SLAM 要解決什麼問題，第 2 章看了世界怎麼教這門學問。但這裡有一個容易被忽略的前提：SLAM 從來不是獨立存在的技術，它是導航系統中的一個環節。本章要畫出那張完整的系統藍圖，讓同學知道 SLAM 的上游是什麼、下游又接到哪裡去。

這一章是第一篇的收尾，也是理解全書架構的關鍵：第二篇的感測器要裝在這個系統的哪個位置、第三篇的數學支撐哪個模組、第八篇的實作又要把哪些元件接起來——答案都在本章的架構圖裡。

圖 3-0 第 3 章與全書的連結地圖

【承接前章】第 1 章的五種地圖表示與六大感測器，在本章被放進完整的系統架構中定位。

【本章負責】建立導航系統的整體藍圖：感知、決策、行動如何串接，SLAM 位於何處。

【往下延伸】

- 3.1 感知決策行動 → 貫穿全書的分層思想 (第 15、18、25、31 章反覆出現)
- 3.2 地圖的種類 → 第 13 章佔據格、第 17 章點雲、第 19 章特徵地圖、第 31 章語意地圖
- 3.3 定位的種類 → 第 11 章重定位與綁架問題、第 22 章多地圖、第 28.5 節 AMCL

3.4 導航系統架構 → 第 28.6 節 Nav2 完整實作、第 30 章專題整合

3.5 ROS 的角色 → 第八篇全篇 (第 27–30 章)

3.6 平台差異 → 第 16 章掃地機器人、第 18 章機器狗、第 25 章無人機

【先備知識】第 1 章的基本觀念。本章仍不涉及數學。

學習目標

前兩章我們認識了 SLAM 這個核心問題與世界名校的教學路線；本章把鏡頭拉高，綜覽一台自主導航機器人的完整系統架構，讓同學在深入各項技術之前，先擁有一張「全系統的地圖」。學完本章之後，同學應能夠達成下列目標：

12. 以 Sense–Think–Act 循環說明機器人系統的基本運作原理，並舉出各階段的具體例子。
13. 分辨四種主要地圖表示法（佔據格地圖、特徵地圖、點雲、語意地圖）的資料結構、優缺點與適用場景。
14. 區分相對定位、絕對定位、地圖定位與重定位四種定位方式，並說明各自的誤差特性。
15. 畫出「SLAM、路徑規劃、避障、控制」四大模組組成的導航系統架構圖，並說明模組間的資料流。
16. 說明 ROS / ROS 2 的核心概念（節點、話題、服務）以及它在 SLAM 教學與研究中的角色。
17. 比較掃地機器人、服務機器人、機器狗、無人機與自駕車五種平台的導航需求差異。

核心概念

中文名詞	英文名詞	概念說明
感知—決策—行動	Sense–Think–Act	機器人系統的基本循環：感測環境、做出決策、執行動作。
特徵地圖	Feature Map	只記錄環境中顯著地標（角點、柱子、標誌）位置的地圖。
相對定位	Relative Localization	由運動量測累積推算位姿（航位推算），誤差隨時間累積。
絕對定位	Absolute Localization	直接量測相對於全域參考系的位置（如 GPS、UWB），不漂移。
地圖定位	Map-based Localization	把即時感測資料與既有地圖比對來估測位姿，如 AMCL。
重定位	Relocalization	機器人「迷路」或被搬動後，在地圖中重新找回自己位置的能力。
綁架機器人問題	Kidnapped Robot Problem	機器人被突然移到未知位置的極端測試，檢驗重定位能力。
路徑規劃	Path Planning	在地圖中找出從起點到目標的可行且最佳路徑。
避障	Obstacle Avoidance	偵測並閃避地圖上沒有的動態或臨時障礙物。
代價地圖	Costmap	把障礙物加上安全膨脹距離的導航用地圖，分全域與局部兩層。
機器人作業系統	ROS / ROS 2	機器人軟體開發框架，提供模組間通訊、工具與生態系。
節點與話題	Node / Topic	ROS 的基本單元：節點是功能模組，話題是節點間的資料通道。

3.1 Sense–Think–Act：感知、決策、行動

所有自主機器人——無論是掃地機器人、機器狗還是自駕車——的運作都可以化為一個不斷重複的循環：感知（Sense）、決策（Think）、行動（Act）。這個循環每秒執行數次到數百次，構成機器人「活著」的心跳。

- 感知（Sense）：透過感測器蒐集資訊。包含感知「外在世界」（光達掃描到前方有牆、相機看到行人）與感知「自身狀態」（輪子轉了幾圈、機身傾斜幾度）。第 2 篇介紹的六大感測器都屬於這一層。
- 決策（Think）：把感知資訊轉化為行動指令。這一層又可細分為多個子層：狀態估測（我在哪裡？——SLAM 與定位）、環境理解（周圍有什麼？）、任務規劃（我要去哪裡？）、路徑規劃（怎麼去？）與運動規劃（下一步怎麼動？）。
- 行動（Act）：把指令轉為馬達、舵機、螺旋槳的實際輸出，改變機器人與世界的狀態——而這又會改變下一輪感知到的內容，循環於是延續。

早期機器人學曾流行「感知—建模—規劃—行動」的嚴格序列式架構（Sense–Model–Plan–Act），每一輪都要先建完整世界模型再行動，反應遲鈍；1980 年代 Rodney Brooks 提出的行為式架構（Behavior-based Architecture）則主張跳過建模，讓感知直接驅動行為，反應快但難以完成複雜任務。現代機器人普遍採用混合式架構：底層用快速的反應式迴圈處理避障與平衡（毫秒級），上層用審議式的規劃處理建圖與導航（秒級）——SLAM 正是上層審議式架構的核心成員。

用一個生活比喻：騎腳踏車閃避突然衝出的小狗，靠的是不經大腦的反射（反應式）；規劃「怎麼從家裡騎到學校」，靠的是腦中的地圖與路線（審議式）。人如此，機器人亦然。

3.2 地圖的種類：Occupancy Grid、Feature Map、Point Cloud、Semantic Map

第 1 章我們從「輸出」的角度初識過地圖的樣貌；本節從「資料結構」的角度深入比較，因為地圖表示法的選擇，直接決定了記憶體用量、演算法複雜度與應用能力。

一、佔據格地圖 (Occupancy Grid Map)

把空間切成規則格網，每格儲存一個「被佔據的機率」(0 到 1 之間)。優點是結構簡單、查詢快速、天然支援路徑規劃與避障；缺點是記憶體隨面積線性成長、解析度固定 (格子 5 公分就看不見 3 公分的物體)，且不含任何語意。2D 佔據格是室內輪型機器人的主流選擇；3D 版本則常用八叉樹 (Octree，如 OctoMap) 壓縮儲存，只有在有東西的地方細分格子。

二、特徵地圖 (Feature Map)

反其道而行：不記錄「每一格有沒有東西」，只記錄環境中少數顯著地標的位置與描述——牆角、柱子、AprilTag、視覺特徵點。優點是極度精簡 (一間教室可能只需幾十個地標)、與 EKF-SLAM 等估測理論天作之合；缺點是「地標以外的空間一無所知」，無法直接用來避障，且依賴環境中存在穩定可辨識的特徵。ORB-SLAM 系列建立的地圖點 (Map Point) 本質上就是視覺特徵地圖。

三、點雲地圖 (Point Cloud Map)

直接保存感測器量到的原始三維點，不做格網化也不抽取特徵。優點是保真度最高、能呈現任意複雜的立體結構；缺點是資料量龐大 (動輒數 GB)、缺乏「空與未知」的區別 (沒有點的地方是沒東西？還是沒掃到？)、直接拿來規劃路徑並不方便。實務上點雲常作為「原始素材」，再依需求轉換為格地圖或網格模型。

四、語意地圖 (Semantic Map)

在幾何之上加註意義：這塊區域是「廚房」、那個物體是「椅子」、這條線是「可通行的走廊」。語意地圖讓機器人能執行人類語言等級的任務，是服務機器人與第 31 章 AI 語意導航的基礎。它通常不是獨立存在，而是疊加在前三種幾何地圖之上的「標籤層」。

地圖類型	資料結構	記憶體	優點	限制
佔據格地圖	機率格網	中（隨面積）	規劃避障方便、演算法成熟	解析度固定、無語意
特徵地圖	地標清單	極小	精簡、利於估測理論	不能避障、依賴特徵
點雲地圖	三維點集合	大	保真度最高、立體完整	難直接規劃、無空區概念
語意地圖	幾何 + 標籤	視底層而定	支援語言任務、人機互動	依賴 AI 辨識正確性

表 3-1 四種地圖表示法比較

3.3 定位的種類：相對定位、絕對定位、地圖定位、重定位

一、相對定位（航位推算）

從已知起點出發，靠里程計與 IMU 累加每一步的位移來推算目前位姿，又稱航位推算（Dead Reckoning）。這是大航海時代水手的方法：記錄航向與航速，累加推算船位。優點是自給自足、更新快速；致命傷是第 1 章反覆強調的漂移——誤差只增不減。

二、絕對定位

直接量測相對於全域座標系的位置：GPS 給出經緯度、UWB 基站給出室內座標、地磁羅盤給出絕對航向。優點是誤差有界、不隨時間惡化；缺點是依賴外部基礎設施

(衛星、基站)、精度有限、更新率通常較低。

三、地圖定位

在「已有地圖」的前提下，把即時感測資料與地圖比對來推算位姿。ROS 生態中最經典的是 AMCL (Adaptive Monte Carlo Localization，自適應蒙地卡羅定位)：在地圖上撒出大量「猜測位置」的粒子，看哪些粒子預期看到的雷射掃描與實際掃描最吻合，逐步收斂到真實位姿 (原理詳見第 11 章，實作詳見第 28 章)。掃地機器人第二次打掃時不必重新建圖，就是靠地圖定位。

四、重定位

如果機器人被抱起來搬到另一個房間 (著名的「綁架機器人問題」)，或視覺 SLAM 因快速轉動而跟丟，系統必須在不知道自己在哪的情況下，於整張地圖中重新找回位置——這就是重定位。它比一般定位困難得多，因為沒有「上一刻位置」可以參考，必須做全域搜尋。ORB-SLAM 系列以詞袋模型 (Bag of Words) 快速檢索相似畫面實現重定位 (第 21 章)；這個能力也與閉環偵測共用同一套技術核心。

四種定位並非互斥，而是互補的層次：實務系統以相對定位提供高頻率的短期估測，以地圖定位或絕對定位提供不漂移的長期校正，並隨時準備在迷失時啟動重定位——這正是感測器融合思想在定位層的體現。

3.4 導航系統架構：SLAM、路徑規劃、避障、控制

有了地圖與定位，機器人還不會「走到目的地」。完整的導航系統由四大模組組成，彼此以明確的資料介面串接：

圖 3-1 自主導航系統架構與資料流

感測器 (光達 / 相機 / IMU / 里程計)



SLAM / 定位模組：輸出地圖 + 即時位姿



全域路徑規劃 (Global Planner)：在地圖上算出起點到目標的路徑 (A^* 、Dijkstra)



局部規劃與避障 (Local Planner)：依據局部代價地圖即時修正路徑、閃避動態障礙 (DWA、TEB)



運動控制 (Controller)：把路徑轉為速度指令 (線速度 v 、角速度 ω)



底盤馬達執行 → 環境改變 → 回到感測器 (循環)

- SLAM / 定位模組是整個系統的「眼睛與內心地圖」：沒有正確的位姿，後面所有模組都在錯誤前提下工作——這就是為什麼本書用整本書的篇幅講清楚這一個模組。
- 全域規劃在靜態地圖上運作，回答「大方向怎麼走」；常用 A^* 與 Dijkstra 等圖搜尋演算法。
- 局部規劃在機器人周圍數公尺的「局部代價地圖」上高頻運作，處理地圖上沒有的東西：走動的人、臨時擺放的椅子。代價地圖把障礙物向外膨脹一圈安全距離，確保機器人不會擦撞。

- 控制器把幾何路徑翻譯成馬達聽得懂的語言。差速輪機器人的指令通常就是一組 (v, ω) ：前進多快、轉多快。

這個架構在 ROS 生態中的具體實現，就是家喻戶曉的 Navigation Stack (ROS 1) 與 Nav2 (ROS 2)。第 28 章將帶同學把這整條管線在實體小車上跑起來。

SLAM 在系統中的上下游：一張責任分界圖

前面說明了導航系統的四大模組。這裡特別釐清一件同學常混淆的事：SLAM 的責任邊界到底在哪裡？它不做什麼？

圖 3-5 SLAM 的上下游責任分界

【上游：SLAM 需要什麼】

- 感測器資料 (第二篇) → 光達掃描、影像、IMU、輪速
- 座標系設定 (第 10 章) → 感測器安裝位置的外參
- 時間同步 (第 5、26 章) → 各感測器的時間戳記必須對齊

【SLAM 負責什麼】

- 估計機器人位姿 → 我在哪裡 (第 11、12 章的數學)
- 建立環境地圖 → 環境長什麼樣 (第 13、17 章的表示)
- 消除累積漂移 → 閉環偵測與全域修正 (第 15、21 章)

【SLAM 不負責什麼】

- 決定要去哪裡 → 任務規劃 (第 23、31 章的語意層)

規劃路徑	→ 全域與局部規劃器 (第 28.6 節)
閃避突然出現的障礙	→ 局部避障與反應式行為
控制馬達	→ 底層控制迴路

【下游：誰使用 SLAM 的輸出】

路徑規劃	→ 需要地圖與當前位姿
避障	→ 需要即時位姿以判斷障礙的相對位置
任務系統	→ 需要位姿以判斷是否抵達目標

這張分界圖有一個實用價值：除錯時能快速定位問題。第 28.7 節會提供一套系統性的排查順序，其邏輯正是依循這條上下游鏈——若機器人行為異常，先確認上游（感測器、TF、時間）是否正常，再看 SLAM 本身，最後才檢查下游的規劃與控制。許多同學花數小時調整規劃器參數，問題其實出在外參填錯——這正是不理解責任分界的代價。

3.5 ROS / ROS 2 在 SLAM 教學中的角色

如果說 SLAM 是導航的靈魂，ROS (Robot Operating System，機器人作業系統) 就是承載靈魂的骨架。名字裡雖有「作業系統」，它其實是跑在 Linux 之上的機器人軟體框架，解決一個關鍵問題：讓感測器驅動、SLAM、規劃、控制這些來自不同作者的模組，能像積木一樣拼在一起。

三個核心概念

- 節點 (Node) : 一個獨立的功能程式。光達驅動是一個節點、Gmapping 是一個節點、路徑規劃是一個節點。
- 話題 (Topic) : 節點間的資料通道，採「發布 / 訂閱」模式。光達節點發布 /scan 話題，SLAM 節點訂閱它——雙方不必認識彼此，只要約好資料格式。
- 服務與動作 (Service / Action) : 一問一答式 (查詢地圖) 與長任務式 (導航到目標點、可回報進度可取消) 的通訊模式。

為什麼 SLAM 教學離不開 ROS

第一，標準化介面讓演算法可以「熱插拔」：同一台小車、同一份雷射資料，換一個節點就能比較 Gmapping 與 Cartographer 的建圖品質——這是本書第四篇實驗設計的基礎。第二，rosvbag 錄放工具能把感測資料錄下來反覆重播，沒有機器人的同學也能用公開資料集完整練習 SLAM。第三，RViz 視覺化工具即時顯示地圖、位姿、粒子雲與規劃路徑，把抽象演算法變成看得見的動畫。第四，全球主流 SLAM 開源系統 (Gmapping、Cartographer、RTAB-Map、ORB-SLAM 的社群封裝) 都提供 ROS 介面，站上 ROS 就站上了整個生態系。

從 ROS 1 到 ROS 2

ROS 1 誕生於 2007 年，已於 2025 年 5 月隨最後版本 Noetic 終止官方支援；新專案一律建議採用 ROS 2。ROS 2 重新設計了底層通訊 (採用 DDS)，支援即時性、多機器人與嵌入式平台，每年 5 月 23 日 (世界烏龜日) 發布一個新版本：偶數年為長期支援版 (LTS，支援五年)、奇數年為一般版。截至本書撰寫時，常用版本包括 Humble (2022，LTS)、Jazzy Jalisco (2024，LTS) 與最新的 Lyrical Luth (2026，LTS，搭配 Ubuntu 26.04)。本書實作章節以 LTS 版本為基準，並在第 27 章詳細說明環境安裝。教學上的好消息是：本章介紹的節點、話題等觀念在 ROS 1 與 ROS 2 中一脈相承，學會一套，兩邊通用。

Nav2 的當代地位：從研究工具到產業標準

本節介紹了 ROS 在教學中的角色，這裡補充一個對同學規劃職涯有意義的觀察：
ROS 生態中的導航堆疊，已經從研究工具演變為產業標準。

作為 ROS Navigation Stack 的後繼者，Nav2 建立在十五年的傳承之上，它把自駕車領域的技術降維、重新設計並最佳化，用於行動與地面機器人；它提供感知、規劃、控制、定位、視覺化與行為等完整功能，能從原始或預處理過的感測與語意資料計算環境模型、動態規劃路徑、產生可行的馬達指令、避開障礙，並組織更高層次的機器人行為。

值得注意的是它的產業採用程度：Nav2 是一套生產級的高品質導航框架，受到全球一百多家公司信賴；它讓行動機器人能在複雜環境中完成使用者定義的應用任務，且幾乎適用於任何一類機器人運動學。它的能力也不限於「從 A 點移動到 B 點」——還能設定中間位姿，並表達物件跟隨、完整覆蓋導航等其他類型的任務。

最後一點對本書特別有意義：「完整覆蓋導航」正是第 16 章掃地機器人的核心任務，而「物件跟隨」則呼應第 23 章的視覺導航。這說明 Nav2 不只是一個路徑規劃器，而是一個可組合的任務框架——第 28.6 節會透過行為樹的概念說明它如何做到這件事。

學習建議：ROS 技能的市場價值

對台灣的學生而言，ROS 2 與 Nav2 的實作能力有幾個現實的價值：

- 產業共通語言：無論做 AGV、服務機器人、農業機器人或巡檢機器狗，多數團隊都用 ROS 2。
- 可累積的作品：第 30 章的專題成果能直接放進履歷，且他人可重現驗證。

- 銜接國際：本書第 2 章介紹的各校課程作業，多數也以 ROS 為平台。

本書第八篇（第 27–30 章）因此不只是教學單元，也是職能訓練——

第 27 章的環境建置、第 28 章的完整導航系統、第 30 章的專題評量，

構成一份可展示的實作履歷。

3.6 從掃地機器人到自駕車：不同平台的導航需求

同一套導航架構，落在不同平台上會長出截然不同的樣貌。理解「需求如何塑造系統」，是系統設計能力的起點。

- 掃地機器人：2D 平面、低速、環境半靜態。重點不是「從 A 到 B」而是「覆蓋每一寸地板」（Coverage Path Planning），感測器成本被壓到極致——一顆 2D 光達加碰撞與懸崖感測器就要打天下（第 16 章）。
- 室內服務機器人（送餐、導覽）：與人共處是最大挑戰。需要語意理解（電梯、櫃檯）、社交性避障（不從人群中硬穿）、以及長期運行下的地圖更新能力。
- 機器狗（四足機器人）：能上樓梯、跨障礙，導航從 2D 升級為 2.5D / 3D——地圖必須包含地形高度與可踏性，定位必須在劇烈震動下保持穩定，光達慣性緊耦合（第 18 章）成為標配。
- 無人機：完整 6 自由度、高速、載重與電力極度受限。偏好輕量的視覺慣性方案（第 25 章），且對延遲零容忍——定位慢半秒，飛機已經撞牆。

- 自駕車：速度快、後果重、範圍大。採高精地圖 + 多重冗餘感測（光達、相機、雷達、RTK-GNSS 缺一不可），定位精度要求公分級，且必須通過車規安全驗證。

平台	空間維度	速度	核心難點	典型感測組合
掃地機器人	2D	慢	全覆蓋、成本極限	2D 光達 + 碰撞 + IMU
服務機器人	2D + 語意	慢	人機共處、長期地圖	2D 光達 + 相機 + 超音波
機器狗	2.5D / 3D	中	地形理解、震動	3D 光達 + IMU + 深度相機
無人機	3D (6DOF)	快	重量功耗、零延遲	相機 + IMU (VIO)
自駕車	2D 大範圍	極快	安全冗餘、公分級	3D 光達 + 相機 + 雷達 + RTK

表 3-2 五種平台的導航需求比較

本書的章節安排正是沿著這條光譜前進：第四篇從最親民的掃地機器人與 2D SLAM 出發，第五篇進入機器狗與 3D 光達，第六、七篇走向視覺導航與無人機，讓同學在難度階梯上逐級而上。

公式或流程

一、航位推算的誤差累積

以 2D 差速輪機器人為例，設時間 t 的位姿為 $(x(t), y(t), \theta(t))$ ，一個取樣週期 Δt 內量得線速度 v 與角速度 ω ，則航位推算的更新式為：

$$x(t+1) = x(t) + v \cdot \Delta t \cdot \cos \theta(t)$$

$$y(t+1) = y(t) + v \cdot \Delta t \cdot \sin \theta(t)$$

$$\theta(t+1) = \theta(t) + \omega \cdot \Delta t$$

注意每一步的 v 與 ω 都帶有量測誤差，而且誤差會透過 $\cos$ 與 $\sin$ 「傳染」給位置——特別是角度誤差：航向偏 1 度，直走 10 公尺後位置就偏約 17 公分，而且之後的每一步都建立在這個歪掉的方向上。這條簡單公式，就是「為什麼純里程計不能撐起導航」的數學解釋，也是第 10 章座標運動學與第 11 章誤差傳播理論的預告。

二、導航問題的分解

本章介紹的系統架構，可以整理為導航三問與其對應模組：

圖 3-2 導航三問

問題一：我在哪裡？（Where am I?）

→ 定位 / SLAM（本書第一至六篇）

問題二：我要去哪裡？（Where am I going?）

→ 任務與目標決策（第 30、31 章專題）

問題三：我該怎麼去？（How do I get there?）

→ 路徑規劃 + 避障 + 控制（第 28 章 Nav2 實作）

這三問出自行動機器人學的經典表述（Leonard 與 Durrant-Whyte）：整本教科書的任務，就是把第一問回答到透徹，並讓同學有能力把答案接上第二、三問。

3.7 導航技術的當代演進：從模組化到基礎模型

本章描述的是經典的模組化導航架構——感知、定位、規劃、控制各司其職，模組之

間以明確的介面串接。這個架構穩定、可解釋、易於除錯，是目前絕大多數實際部署系統的骨幹，也是本書第八篇實作的對象。

但近兩年出現了一條截然不同的路線，值得同學知道它的存在與定位。

第二條路線：導航基礎模型

傳統架構的每個模組各自訓練或設計；新路線則試圖用單一的大型模型端到端地完成導航。這類模型被稱為導航基礎模型或視覺語言動作模型。

其能力範圍相當驚人。以一項代表性工作為例，它提供統一的框架來處理多種任務——包含影像問答、影片問答與導航；在物件搜尋、追蹤與自動駕駛等多項基準上驗證，並在多種真實機器人平台上實測，包括人形機器人、四足機器人、無人機與輪型機器人，顯示出通往通用導航的可能性。

這個「同一套模型跨多種平台」的特性，與本書第 3.6 節的討論形成有趣的對照：本節說明不同平台的導航需求差異極大（輪型只需 2D、四足需地形評估、無人機需 6-DoF），而基礎模型路線正試圖用一套模型涵蓋這些差異。第 23.5 節與第 31 章會再回到這個主題。

世界模型：讓機器人「想像」未來

另一個新興概念是世界模型。傳統導航的規劃基於當前地圖；世界模型則讓機器人能預測「如果我往這走，接下來會看到什麼」，再據此選擇行動。近年已有研究把視覺語言世界模型用於階層式規劃與雙時域的導航預測，也有工作探討如何讓世界模型在連續環境中自我演化。

這個思路對本書的讀者並不陌生——第 11 章的貝氏濾波就是「預測—更新」的循環，只是預測的是位姿而非畫面。世界模型可以理解為把預測的對象從「狀態」擴展到「感知」，這是一個自然但強大的延伸。

成長的速度與務實的判斷

這個領域的成長速度值得留意。有整理指出，2025 至 2026 年間釋出的機器人基礎模型已超過九十個，涵蓋視覺語言動作模型、世界模型、人形機器人基礎模型、移動、導航與操作策略等類別；而視覺語言動作模型在頂級研討會的投稿量，從 2025 到 2026 年成長了約十八倍。

面對這樣的變化速度，同學該如何自處？本書的建議是三點：

1. 先掌握模組化架構：因為它是目前實際部署的主流，也是理解新方法的基礎——基礎模型仍然要輸出位姿或路徑點，仍然要面對感測器的限制，仍然要在真實世界中避障。
2. 理解新方法解決了什麼舊問題：基礎模型的價值在於泛化與語言介面，而非取代幾何精度。第 31.8 節會詳細討論它的四大風險，包括幻覺、系統性誤判、延遲與安全性。
3. 注意分層而非替換：目前最務實的整合方式，是讓基礎模型處理高階決策（秒級以上），幾何系統處理即時控制（毫秒級）。這正是第 31.6 節提出的分層架構，也是第 3.1 節「感知—決策—行動」思想的現代版本。

觀念總結：兩條路線的分工

【模組化架構】本書第一至八篇

優點：可解釋、可除錯、運算需求明確、不需訓練資料

限制：每個模組各自最佳化，未必達成全域最優；語意理解能力弱

【基礎模型路線】本書第 31 章

優點：泛化能力強、支援語言介面、可跨平台

限制：運算與記憶體需求高、可能產生幻覺、難以除錯與問責

【務實的整合】上層用基礎模型理解任務，下層用模組化架構保證安全。

這個分工原則在本書第 23、26、31 章會反覆出現。

案例

案例 3-1 送餐機器人的一天：架構總動員

觀察餐廳裡的送餐機器人完成一趟任務，本章所有觀念會一次到齊。開機：載入既有地圖，AMCL 地圖定位確認自己在充電站（地圖定位 + 重定位能力）。接單：任務系統指定 12 號桌（導航第二問）。出發：全域規劃在佔據格地圖上算出路徑（ A^* ），沿途里程計與 IMU 提供高頻位姿（相對定位），雷射持續與地圖比對修正（地圖定位）。半路一位客人起身擋道：局部代價地圖瞬間更新，局部規劃繞出一條弧線（避障），控制器輸出新的（ v 、 ω ）。送達後返航充電。整趟任務中，SLAM 建好的那張地圖是一切的基礎——地圖歪一公尺，機器人就會把餐送進牆裡。

案例 3-2 綁架測試：把掃地機器人抱到另一個房間

許多同學家裡就能做這個經典實驗：讓掃地機器人在客廳工作到一半，把它抱起來放到臥室。早期低價機型會當場「精神錯亂」——以為自己還在客廳，對著臥室的牆重複碰撞；具備重定位能力的機型則會停頓數秒，旋轉一圈重新掃描，比對雷射輪廓與記憶中的全屋地圖，然後在 App 地圖上「瞬移」到正確的臥室位置繼續工作。這數秒鐘裡發生的，正是教科書上的綁架機器人問題求解：拋棄先前位姿假設、全域搜

尋、多假設驗證、收斂。一個抱起來再放下的動作，就是一堂免費的定位理論課。

案例 3-3 從 Willow Garage 到 Nav2：ROS 生態的力量

2010 年前後，美國 Willow Garage 公司把 PR2 機器人與 ROS 一同開源，並整理出第一代 Navigation Stack——AMCL、代價地圖、全域與局部規劃器的完整組合。此後十餘年，全球數以千計的機器人（包括台灣許多大學實驗室的自製小車）不必重寫導航系統，站在同一套開源積木上疊出各自的應用。ROS 2 時代的 Nav2 更進一步引入行為樹（Behavior Tree）任務架構與外掛式規劃器。這段歷史說明了開源生態對機器人教育的意義：學生的心力可以集中在理解原理與創新應用，而不是重造每一顆輪子——這也是本書第八篇直接以 ROS 2 為實作骨幹的原因。

案例 3-4 語意進入導航堆疊：當分割模型接上 Nav2

本章介紹的模組化架構，近年正在發生一個安靜但重要的變化：語意資訊開始進入導航堆疊的核心。

傳統的代價地圖（第 28.6 節）只區分「可通行」與「有障礙」。但真實世界的通行性遠比這細膩——人行道與車道都是平坦的地面，幾何上沒有差別，語意上卻完全不同；草地可以壓過去，水窪最好繞開；玻璃在光達上不存在，在視覺上卻清晰可辨。

近期已有實際展示把分割基礎模型與 Nav2 結合，在戶外環境中分割人行道與路面來進行語意導航，並在一般等級的運算平台上運行。這意味著語意導航已經從論文走向可部署的系統。

對照本書的架構，這個變化發生在兩個層次的交界處：語意層（第 31 章的 AI 視覺模型）產出「這是人行道、那是車道」的判斷，幾何層（第 28 章的代價地圖）把這個判斷轉換為通行成本。兩者的介面很單純——語意結果變成代價地圖的一個圖層。這正是模組化架構的優勢：新能力能以圖層的形式加入，不需重寫整個系統。

對同學的啟示有兩點。其一，第 28.6 節談的「代價地圖分層設計」不只是實作細節，而是一個能容納未來擴充的架構決策。其二，若你要做語意導航的專題（第 23、30 章），最務實的切入點不是重寫導航系統，而是「多加一個語意圖層」——這個做法成本低、風險小，而且能立即看到效果。

實作活動

實作 3-1 導航系統拆解與紙上設計

目的：不寫程式，先用「系統設計師」的眼光完成一次導航系統的完整拆解與規格設計，建立全局觀。

步驟

10. 分組（每組 3 至 4 人），每組抽選一種平台：掃地機器人、送餐機器人、校園巡檢機器狗、室內巡檢無人機、無人載具小巴。
11. 需求分析：列出該平台的運行環境（室內外？有人？地形？）、速度需求、失敗的後果嚴重度。
12. 感測器選型：從第 1 章表 1-1 的六大感測器中挑選組合，說明每一顆的用途與捨棄其他選項的理由（含成本考量）。
13. 地圖與定位選型：從表 3-1 選擇地圖表示法、從 3.3 節選擇定位策略，說明理由。
14. 繪製系統架構圖：仿照圖 3-1，畫出你們平台的完整資料流，標明各模組的更新頻率（哪些要毫秒級？哪些秒級即可？）。
15. 各組上台簡報五分鐘，其他組扮演「客戶」提出至少一個刁鑽問題（例如：玻璃隔間怎麼辦？機器人被小孩推走怎麼辦？）。

討論問題

9. 你們的平台中，哪一個模組失效會造成最嚴重的後果？系統該如何為它設計備援？
10. 如果預算砍半，你會先犧牲哪顆感測器？系統能力會退化成什麼樣子？
11. (大學部) 你們設計的架構中，哪些模組可以直接使用 ROS 2 / Nav2 的現成元件？哪些必須自行開發？

延伸挑戰 (大學部)

安裝 ROS 2 與 TurtleBot3 模擬器 (安裝步驟預告於第 27 章，網路上有充足教學)，在 Gazebo 模擬世界中啟動 Nav2，實際觀察 RViz 中的全域路徑、局部路徑與代價地圖，對照本章圖 3-1 指認每一個元件。

延伸閱讀

12. R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, Introduction to Autonomous Mobile Robots, 2nd ed., MIT Press, 2011. (本章系統觀的主要參考，第 1 章即以「導航三問」開場)
13. J. J. Leonard and H. F. Durrant-Whyte, "Mobile Robot Localization by Tracking Geometric Beacons," IEEE Transactions on Robotics and Automation, 1991. (「我在哪裡」三問的經典出處之一)
14. R. Brooks, "A Robust Layered Control System for a Mobile Robot," IEEE Journal of Robotics and Automation, 1986. (行為式架構的開山之作，理解架構演進必讀)
15. S. Macenski et al., "The Marathon 2: A Navigation System," IROS 2020. (Nav2 導航系統的官方論文)

16. ROS 2 官方文件 (docs.ros.org) 與 Nav2 官方文件 (docs.nav2.org) . (實作章節前建議先瀏覽其觀念介紹頁)
17. S. Macenski et al., "Robot Operating System 2: Design, Architecture, and Uses in the Wild," Science Robotics, 2022. (ROS 2 設計理念與產業應用的權威介紹)
18. A. Hornung et al., "OctoMap: An Efficient Probabilistic 3D Mapping Framework Based on Octrees," Autonomous Robots, 2013. (3D 佔據格地圖的代表作)

從系統藍圖到感測器：第一篇的收束

第一篇到這裡結束。三章的分工是這樣的：第 1 章建立問題意識 (SLAM 要解決什麼)、第 2 章提供學習地圖 (世界怎麼教、本書怎麼編)、本章畫出系統藍圖 (SLAM 在整體中的位置)。

這三章刻意不含任何數學與程式，目的是先建立完整的觀念框架。筆者的教學經驗是：太早進入公式與程式碼，學生容易見樹不見林，做出了能跑的系統卻說不清楚它為什麼這樣設計。有了這個框架，後續的技術細節才有掛靠的地方。

圖 3-9 從第一篇通往第二篇

【第一篇建立的三個框架】

問題框架 → 定位與建圖互相依賴，必須同時解決 (第 1 章)

學習框架 → 感測器 → 數學 → 前端 → 後端 → 閉環 → 專題 (第 2 章)

系統框架 → 感知 → 定位建圖 → 規劃 → 控制 (本章)

【第二篇要回答的問題】

機器人用什麼「看」世界？每一種感測器能做什麼、又會在什麼時候失敗？

第 4 章 室內定位 → Wi-Fi、UWB、藍牙、視覺標記

第 5 章 衛星導航 → GNSS 原理、都市峽谷、與慣性的組合

第 6 章 地磁導航 → 電子羅盤、磁干擾、地磁指紋

第 7 章 IMU → 加速度計、陀螺儀、漂移、姿態濾波

第 8 章 光達 → 飛行時間、2D 與 3D、規格與盲區

第 9 章 相機 → 針孔模型、校正、單目雙目與 RGB-D

【第二篇的共同結構】每章都以「原理 → 規格 → 失效情境 → 互補 → 實作」展開。

特別提醒同學留意每一章的「失效情境」討論。第 4 章會談 Wi-Fi 訊號的波動、第 5 章會談都市峽谷的多路徑、第 6 章會談鋼構建築中的磁場扭曲、第 7 章會談 IMU 的漂移速度、第 8 章會談光達看不見玻璃、第 9 章會談相機的白牆詛咒。

這些不是負面資訊，而是工程判斷的基礎。到了第 29.7 節，同學會在同一個場域親眼看到光達與相機各自崩潰——那時再回頭看第二篇的每一章，會明白為什麼本書從一開始就這麼強調限制。

小結

14. 自主機器人以 Sense–Think–Act 循環運作；現代系統採混合式架構——底層反應式迴圈處理避障與平衡，上層審議式規劃承載 SLAM 與導航。
15. 四種地圖表示法各擅勝場：佔據格利於規劃避障、特徵地圖精簡利於估測、點雲保真度最高、語意地圖支援語言等級任務；實務常混合使用。
16. 定位分為相對定位（高頻但漂移）、絕對定位（不漂移但精度有限）、地圖定位（與既有地圖比對）與重定位（迷失後全域找回）；四者互補構成完整定位能力。
17. 導航系統由 SLAM / 定位、全域規劃、局部避障與運動控制四大模組串成資料流；SLAM 是一切下游模組的前提。
18. ROS / ROS 2 以節點、話題、服務的標準化介面串接模組，配合 rosbag 與 RViz 工具，成為 SLAM 教學與研究的事實標準；ROS 1 已於 2025 年終止支援，本書實作以 ROS 2 LTS 版本為基準。
19. 從掃地機器人到自駕車，導航需求隨空間維度、速度與安全要求逐級升高，感測器組合與系統設計亦隨之改變；本書章節即沿此難度光譜安排。

至此，第一篇「SLAM 與自主導航概論」告一段落——同學已經知道 SLAM 是什麼問題、世界名校怎麼教、以及它在整台機器人中的位置。下一章起進入第二篇，我們將逐一拆解機器人的「感官」：先從一個看似矛盾的問題開始——為什麼在室內，天上三十多顆 GPS 衛星幫不上忙？

台灣自編大學教科書

@prof

《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第二篇 定位與導航感測器

第 4 章

室內定位技術

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

學習目標

從本章起，我們進入第二篇「定位與導航感測器」，逐一拆解機器人的各種「感官」。第一站是室內定位——這是最貼近同學日常生活的定位技術，也是理解「絕對定位」與「SLAM」如何互補的最佳起點。學完本章之後，同學應能夠達成下列目標：

18. 解釋 GPS 訊號為何在室內失效，並說明訊號衰減與多路徑效應的物理成因。
19. 說明 Wi-Fi RSSI 指紋定位的離線建庫與線上比對兩階段流程，並實作簡易版本。
20. 比較 BLE Beacon 的三種定位模式（鄰近偵測、三邊定位、指紋定位）與其精度差異。
21. 說明 UWB 的高精度來源，並區分 TWR 與 TDoA 兩種測距架構。
22. 比較 RFID、NFC、QR Code 與視覺標記（AprilTag / ArUco）作為「地標式定位」的原理與適用場合。
23. 說明室內定位與 SLAM 如何互補融合，並知道各技術在精度、成本、部署難度上的權衡。

核心概念

中文名詞	英文名詞	概念說明
接收訊號強度指標	RSSI	接收端量到的訊號強度，隨距離增加而衰減，是最便宜的測距依據。
指紋定位	Fingerprinting	先建立各位置的訊號特徵資料庫，再比對即時訊號找出最相似位置。
多路徑效應	Multipath	訊號經牆面反射後多條路徑抵達，造成測距偏長或不穩定。

中文名詞	英文名詞	概念說明
非視距	NLOS	發射端與接收端之間被遮蔽，訊號需穿透或繞射，是誤差主因。
三邊定位	Trilateration	由三個以上已知點的距離求交點，推算未知點位置。
三角定位	Triangulation	由角度資訊（而非距離）推算位置。
超寬頻	UWB	使用極短脈衝、極寬頻譜的無線技術，可達公寸級甚至公分級測距。
雙向測距	TWR	標籤與基站來回通訊量測飛行時間，不需基站時間同步。
到達時間差定位	TDMA	比較訊號抵達多個基站的時間差，需基站精確同步，容量大。
低功耗藍牙信標	BLE Beacon	週期性廣播識別碼的低功耗藍牙裝置，常用於室內導引。
視覺基準標記	Fiducial Marker	如 AprilTag、ArUco，可由影像精確解算相對位姿的人工標記。
六自由度位姿	6-DoF Pose	三軸位置加三軸旋轉，視覺標記可一次解出完整位姿。

4.1 為什麼室內不能只靠 GPS

走進百貨公司地下停車場，手機地圖上的藍點會開始亂飄，甚至直接顯示「無法定位」。這個日常經驗背後，是三個疊加的物理限制。

一、訊號實在太微弱

GPS 衛星在距地表約兩萬公里的中軌道上，發射功率僅約 25 至 50 瓦——大約是一顆家用燈泡的等級。訊號跋涉兩萬公里抵達地面時，強度已衰減到約 -130 dBm 這個量級，比環境的背景雜訊還要微弱，接收機必須靠「展頻編碼」的相關運算才能把它從雜訊中撈出來。這樣脆弱的訊號，遇到鋼筋混凝土樓板可能再衰減數十 dB，等同於把僅存的一絲訊號徹底掐熄。

二、多路徑效應讓測距失真

即使部分訊號從窗戶漏進室內，它往往已經在牆面、天花板、金屬櫃上彈跳過。GPS 的測距原理是「訊號飛行時間乘上光速」，反射路徑比直線路徑長，算出來的距離就會偏大。更麻煩的是，室內反射路徑往往比直達路徑更強（直達的被牆擋住了），接收機無從分辨，定位結果因此劇烈飄移——這正是停車場裡藍點亂跳的主因。

三、幾何條件惡化

衛星定位需要至少四顆分布良好的衛星（三顆定位、一顆解算時鐘誤差）。室內能「看見」的衛星往往只剩靠近窗戶方向的少數幾顆，幾何分布極差。定位學上用精度稀釋因子（DOP，Dilution of Precision）描述這個現象：衛星擠在同一方向時 DOP 值飆高，即使每顆的測距都準確，交會出的位置誤差仍會被放大數倍。

觀念釐清：室內定位的三難

① 精度：要公尺級還是公分級？精度每提高一個數量級，成本往往上升一個數量級。

② 成本：需要在環境中安裝多少基礎設施（基站、信標、標記）？

③ 覆蓋：一棟樓？一層樓？一個房間？覆蓋範圍越大，部署與維護成本越高。

本章介紹的每一種技術，本質上都是在這三個維度上取捨的不同答案。

也正因為沒有任何一種室內定位技術能同時滿足三者，SLAM 才顯得如此重要：SLAM 不需要任何預先安裝的基礎設施，機器人靠自己的感測器就能在陌生環境建圖定位。但反過來說，SLAM 有漂移與重定位的弱點——這就是本章最後 4.7 節要談的融合主題。

4.2 Wi-Fi RSSI 指紋定位

Wi-Fi 是室內最普及的無線基礎設施——教室、辦公室、商場處處都有 AP（無線基地台），而且每支手機、每台筆電都內建接收器。既然訊號無所不在，能不能拿來定位？答案是可以，但方法與 GPS 完全不同。

為什麼不能直接用 RSSI 算距離

直覺想法是：訊號強度隨距離衰減，那麼量到強度就能反推距離，再用三邊定位求交點。理論上，自由空間中的訊號強度遵循對數距離路徑損耗模型：

$$RSSI(d) = RSSI(d_0) - 10 \cdot n \cdot \log_{10}(d / d_0)$$

其中 n 是路徑損耗指數（自由空間約為 2，室內因遮蔽常在 2 至 4 之間）。問題在於：室內的 n 值會隨牆壁材質、人流、家具擺設而劇烈變化，同一距離的實測 RSSI 可能相差 10 dB 以上，換算成距離誤差可達數公尺。此外人體本身就是很好的訊號吸

收體——手機拿在手上與放在桌上，讀數就不一樣。因此室內 Wi-Fi 定位主流不走「測距」路線，而走「指紋」路線。

指紋定位：兩階段流程

指紋定位的核心洞見是：不要試圖理解訊號如何傳播，只要記住「每個地點看起來像什麼樣子」。流程分為兩階段：

- 離線建庫階段 (Offline / Training)：把場域劃分為格點 (例如每 2 公尺一個參考點)，在每個點上停留數秒，記錄所有可見 AP 的 MAC 位址與 RSSI 值，形成一組「訊號指紋」。全部參考點的指紋集合構成指紋資料庫 (Radio Map)。
- 線上定位階段 (Online / Positioning)：使用者裝置掃描當下的 AP 與 RSSI，與資料庫中每一筆指紋比對相似度，取最相似者 (或前 K 個的加權平均) 作為估測位置。

比對演算法：K 最近鄰

最常用的比對方法是 KNN (K-Nearest Neighbors)。設當前量到的指紋向量為 S 、資料庫中第 i 個參考點的指紋為 F_i ，兩者的歐氏距離為：

$$D(S, F_i) = \sqrt{\sum_j (S_j - F_{ij})^2}$$

其中 j 掃過所有共同可見的 AP。取 D 值最小的 K 個參考點，把它們的座標做 (加權) 平均，就是估測位置。加權時常用距離倒數作為權重，稱為 WKNN。

優點與限制

- 優點：零硬體成本 (利用既有 AP)、覆蓋範圍廣、手機即可實作，非常適合作為教學入門。

- 限制：精度通常在 3 至 8 公尺；指紋庫建置費工（一層樓可能需數百個參考點）；環境變動（AP 更換位置、隔間改裝、人潮）會使資料庫失效，需要定期重建。

近年改良方向包含：以深度學習取代 KNN 比對、結合手機 IMU 的行人航位推算（PDR）平滑軌跡、以及利用 Wi-Fi RTT（往返時間，IEEE 802.11mc）直接測距——後者可將精度提升至 1 至 2 公尺，但需要 AP 與裝置雙方都支援。

4.3 Bluetooth Beacon / BLE 定位

低功耗藍牙（BLE，Bluetooth Low Energy）信標是一顆鈕扣電池就能運作一到兩年的小裝置，週期性廣播自己的識別碼。2013 年 Apple 推出 iBeacon 規格後大為普及，Google 也提出 Eddystone 規格。相較 Wi-Fi AP，信標的優勢在於「可以自己決定裝在哪裡」——密度與位置都能為定位需求量身設計。

三種使用模式

- 鄰近偵測（Proximity）：最簡單也最可靠。不算座標，只判斷「使用者是否靠近某信標」，並粗分為緊鄰（10 公分內）、近（約 1 公尺）、遠（10 公尺內）三級。博物館展品導覽、賣場櫃位推播多採此模式。對機器人而言，這相當於「通過了某個檢查點」。
- 三邊定位（Trilateration）：由三顆以上信標的 RSSI 換算距離後求交點。理論漂亮，但受制於 4.2 節談過的 RSSI 不穩定，實務精度多在 2 至 5 公尺，且需要良好的信標幾何布局。
- 指紋定位：與 Wi-Fi 指紋法完全相同，只是把 AP 換成信標。由於信標密度可控，精度通常略優於 Wi-Fi 指紋。

進階：藍牙尋向（Direction Finding）

藍牙 5.1 起加入尋向功能，透過天線陣列量測訊號的到達角 (AoA) 或離開角 (AoD)。由於角度資訊比 RSSI 穩定得多，配合適當設計可把精度推進到公分甚至公分級——這也是近年 BLE 定位最重要的技術躍進。代價是需要陣列天線硬體，成本與部署複雜度上升。

工程注意事項

- 廣播頻率與電池壽命是蹺蹺板：每秒廣播 10 次反應快但耗電，每秒 1 次省電但定位延遲明顯。
- BLE 使用 2.4 GHz 頻段，與 Wi-Fi、微波爐共用，人多的場合干擾顯著（人體含水量高，對 2.4 GHz 吸收特別強）。
- 信標需要定期巡檢換電池——一個常被忽略的長期維運成本，部署數百顆時尤其可觀。

4.4 UWB 超寬頻定位

如果說 Wi-Fi 與 BLE 是「用訊號強度猜距離」，超寬頻 (UWB, Ultra-Wideband) 就是「直接量時間算距離」——這是精度躍升的根本原因。

為什麼 UWB 這麼準

UWB 發射的是持續時間僅約數奈秒的極短脈衝，佔用高達 500 MHz 以上的頻寬。時間解析度與頻寬成反比，脈衝越短，接收端就越能精確判定「訊號是哪一刻抵達的」。由於光速約每奈秒 30 公分，能把到達時間解析到次奈秒級，距離就能算到公分級。

更關鍵的是抗多路徑能力：反射訊號比直達訊號晚到，一般窄頻訊號的波形拖得長，直達與反射會疊在一起難以分離；UWB 的脈衝極短，第一個抵達的尖峰（直達波）能與後續反射波清楚分開，接收機只取第一個峰值即可——這正是室內環境中最需要

的特性。

兩種測距架構

- 雙向測距 (TWR , Two-Way Ranging) : 標籤發訊給基站、基站回覆，標籤依據來回總時間扣掉處理時間，算出單程飛行時間與距離。優點是不需基站間時間同步；缺點是每次測距都要來回通訊，標籤數量多時通道會塞車。
- 到達時間差定位 (TDoA , Time Difference of Arrival) : 標籤只單向廣播，多個基站記錄各自的到達時間，由伺服器比較時間差解算位置。優點是標籤只發不收、極省電，且系統容量幾乎不受標籤數限制；缺點是所有基站必須達到奈秒級的時間同步，部署要求較高。

精度的實話

廠商文宣常標榜「10 公分精度」，這在視距良好 (LOS)、基站幾何布局理想的條件下確實可達成。但真實場域中，一旦人體或設備遮擋形成非視距 (NLOS)，誤差就會明顯上升；研究比較顯示，較新的 IEEE 802.15.4z 標準在 NLOS 條件下的平均測距誤差可控制在 0.2 公尺以內，較舊標準有明顯改善。工程上的務實預期是：良好部署下公分級，遮蔽嚴重處誤差可能達數十公分甚至更多。

應用現況

UWB 已從實驗室走入消費與工業場域：手機與追蹤標籤的精確尋物、汽車數位鑰匙 (防中繼攻擊的安全測距)、工廠與倉儲的即時定位系統 (RTLS) 避免堆高機碰撞，乃至於運動賽事中的球體追蹤。對機器人而言，UWB 特別適合作為「不漂移的絕對參考」，與 SLAM 融合使用——這是 4.7 節的主題，也是無人機室內定位的常見方案。

4.5 RFID / NFC / QR Code 地標定位

前三節的技術都試圖「連續地」估測位置。本節介紹的是另一種思路：離散地標

定位——不追求隨時知道座標，只求「經過特定地點時，能百分之百確定自己在那裡」。這種思路成本極低、可靠度極高，在工業場域反而更受歡迎。

RFID (無線射頻識別)

被動式 RFID 標籤本身沒有電池，靠讀取器發出的電磁場感應供電並回傳識別碼，成本可低至數元台幣。依頻段分為低頻 (LF，數公分)、高頻 (HF，約 10 公分) 與超高頻 (UHF，數公尺)。倉儲 AGV 的經典做法是在地板下埋設 RFID 標籤陣列，車輛底部的讀取器經過時讀到編號，即可校正位置——精度取決於標籤密度，鋪得密就準。

NFC (近場通訊)

可視為 HF RFID 的一個特化子集，作用距離僅約 4 公分。距離短反而成為優點：讀到就代表「幾乎貼在標籤上」，位置確定性極高。適合充電站對接、工具櫃取放確認等需要「零歧義」的場合。

QR Code 與資料矩陣碼

QR Code 的最大優勢是零成本——一張印表機列印的貼紙就是一個地標，壞了撕掉重貼即可。這正是亞馬遜倉儲機器人 (Kiva 系統) 採用的方案：地板上規則貼滿 QR 碼，機器人底部相機向下拍攝，讀到編號就知道自己在網格的哪一格，同時由影像中 QR 碼的位置與旋轉角推算細部偏移與朝向。數十萬台機器人在全球倉庫中運行，靠的就是這個看似樸素的方案。

技術	作用距離	成本	提供資訊	典型應用
被動 RFID	公分至數公尺	極低	識別碼	倉儲地板標籤、物流追蹤
NFC	約 4 公分	極低	識別碼	充電對接、工具確認

技術	作用距離	成本	提供資訊	典型應用
QR Code	視相機而定	近乎為零	識別碼 + 粗略位姿	倉儲機器人、教學實驗
AprilTag / ArUco	公分至數公尺	近乎為零	完整 6-DoF 位姿	機器人導航、相機校正

表 4-1 四種地標式定位技術比較

4.6 視覺地標與 AprilTag / ArUco 定位

QR Code 能告訴機器人「我在幾號格子」，但如果我們想知道更精確的資訊——相機相對於標記的完整三維位置與朝向呢？這就是視覺基準標記 (Fiducial Marker) 的舞台。AprilTag (密西根大學開發) 與 ArUco (西班牙 Córdoba 大學開發，已整合進 OpenCV) 是兩套最廣泛使用的系統。

為什麼能解出完整位姿

原理其實很直觀：標記是一個已知邊長的正方形，四個角點的三維座標在標記自身座標系中完全已知（例如邊長 10 公分的標記，四角就是 $(0,0,0)$ 、 $(0.1,0,0)$ 、 $(0.1,0.1,0)$ 、 $(0,0.1,0)$ ）。相機拍到這個標記時，四個角點在影像上的二維像素座標也可以精確偵測出來。已知四組「三維點對應二維點」，加上相機內參（第 9 章會詳細介紹），就能解出相機相對標記的旋轉與平移——這在電腦視覺中稱為 PnP 問題 (Perspective-n-Point)，本書第 20 章會深入探討。

偵測流程

16. 影像二值化：把灰階影像轉為黑白，凸顯標記的高對比邊界。
17. 輪廓偵測與四邊形篩選：找出封閉輪廓，保留近似正方形者。
18. 透視校正：把傾斜拍到的四邊形拉回正方形（單應性變換）。
19. 編碼解讀：讀取內部黑白格陣列的位元編碼，比對字典確認 ID，並利用內建的漢明碼糾錯拒絕誤判。

20. 位姿解算：以四角點與相機內參求解 PnP，輸出 6-DoF 位姿。

AprilTag 與 ArUco 的比較

- AprilTag：糾錯能力與抗誤判率設計得更嚴謹，在遠距離、低解析度、部分遮蔽下仍相對穩健，學術與機器人領域偏好使用。
- ArUco：偵測速度快、字典可自訂大小、與 OpenCV 深度整合（幾行程式碼即可上手），教學與快速原型開發極為方便。

在機器人系統中的角色

- 絕對位姿校正：在充電站、電梯口、實驗室門口貼標記，機器人一看到就能消除累積漂移——這是最常見的用法。
- 精確對接：充電、取放料等需要公釐級對位的動作，靠視覺標記做最後階段的視覺伺服（第 23 章）。
- 無人機降落：降落平台上的標記讓無人機精準著陸。
- SLAM 系統的真值來源：教學實驗中，可用密集布置的標記提供近似真值軌跡，用來評估 SLAM 精度（第 30 章專題會用到）。

值得一提的是，視覺標記本質上是「人造的、保證可靠的特徵點」。理解它，其實就是理解第 19 章視覺 SLAM 中「自然特徵點」的簡化版——差別在於自然特徵不保證穩定、不保證唯一、也沒有已知尺寸，這正是視覺 SLAM 困難之所在。

4.7 室內定位與 SLAM 的融合

讀到這裡，同學應該已經發現一個模式：室內定位技術與 SLAM 的優缺點恰好互補。

特性	室內定位 (Wi-Fi / BLE / UWB / 標記)	SLAM
誤差行為	有界，不隨時間累積	短期精確，長期漂移
基礎設施	需事先部署與維護	不需要，隨到隨用
初始位置	開機即知道自己在哪	需從原點出發或執行重定位
地圖資訊	只有座標，不含環境結構	產出完整地圖，可供避障與規劃
更新頻率	較低 (每秒數次)	較高 (每秒數十次)

表 4-2 室內定位與 SLAM 的互補性

三種融合層次

- 鬆耦合 (Loosely Coupled)：兩套系統各自獨立運作，各自輸出位置估計，再以卡爾曼濾波等方式加權融合。實作最簡單，是教學與快速原型的首選。
- 緊耦合 (Tightly Coupled)：把室內定位的原始量測 (UWB 測距值、標記角點) 直接當作約束項，放進 SLAM 的後端最佳化中一起求解。精度最佳，因為所有資訊在同一個機率架構下被一致地處理——這也是第 12 章位姿圖優化的自然延伸：UWB 測距就是位姿與基站之間的一條「邊」。
- 輔助初始化與重定位：最務實的法。SLAM 平時自行運作，只在開機時由 UWB 或視覺標記提供初始位姿，或在跟丟時提供重定位線索。以極低的整合成本，解決了 SLAM 最頭痛的兩個問題。

典型架構案例

一台校園服務機器人的完整定位系統可能長這樣：以 2D 光達 SLAM 為主力，提供每秒 10 次的位姿與地圖；在建築物各樓層電梯口與大廳貼 AprilTag，機器人每次經過就校正一次絕對位姿；充電站前加裝 NFC 確保對接零誤差；跨樓層時由 BLE 信標

判斷「現在在幾樓」(這是光達 SLAM 完全無法回答的問題，因為每層樓的雷射輪廓可能長得一模一樣)。每個技術各司其職，這正是系統工程的精髓——沒有萬用的感測器，只有恰當的組合。

公式或流程

一、三邊定位的數學

設有三個已知位置的基站 $A_1(x_1, y_1)$ 、 $A_2(x_2, y_2)$ 、 $A_3(x_3, y_3)$ ，量得標籤到各基站的距離分別為 d_1 、 d_2 、 d_3 ，則標籤位置 (x, y) 滿足三個圓方程式：

$$(x - x_i)^2 + (y - y_i)^2 = d_i^2 \quad , \quad i = 1, 2, 3$$

把第一式分別減去第二、三式，平方項相消，得到兩條線性方程式，即可解出 (x, y) 。

在理想無誤差情況下，三個圓恰好交於一點；但真實量測有誤差，三圓會交出一個「三角形區域」。此時改以最小平方方法求解——找一個點使它到三個圓的偏差平方和最小：

$$\min \sum_i (\sqrt{(x - x_i)^2 + (y - y_i)^2} - d_i)^2$$

這個「用多餘量測對抗雜訊」的思路，正是第 12 章 SLAM 後端最佳化的核心精神——同學會發現，從三邊定位到 Bundle Adjustment，數學骨架是一脈相承的。

二、TWR 飛行時間計算

雙向測距中，設標籤發出訊號到收到回覆的總時間為 T_{round} ，基站從收到訊號到發出回覆的處理時間為 T_{reply} ，則單程飛行時間與距離為：

$$T_{\text{flight}} = (T_{\text{round}} - T_{\text{reply}}) / 2$$

$$d = c \times T_{\text{flight}} \quad (c \text{ 為光速，約 } 3 \times 10^8 \text{ m/s})$$

由此可見時間量測精度的重要性：光速下 1 奈秒的誤差就等於 30 公分的距離誤差；若要達到 10 公分精度，時間解析度必須優於 0.33 奈秒。這也解釋了為什麼一般 Wi-Fi 與藍牙晶片做不到——它們的時脈解析度遠不足以支撐這個等級的量測。

三、室內定位技術選型流程

圖 4-1 室內定位技術選型決策流程

問題一：需要什麼精度？

- └─ 公尺級即可 → Wi-Fi / BLE 指紋定位 (成本最低)
- └─ 公寸級 → UWB 或 BLE 尋向 (需部署基站)
- └─ 公分級 → UWB (LOS 良好) 或視覺標記 (近距離)

↓

問題二：可以在環境中安裝設備嗎？

- └─ 完全不行 → 只能用 SLAM (第四至六篇)
- └─ 可貼紙張 → QR Code / AprilTag (零成本)
- └─ 可施工布線 → UWB 基站 / BLE 信標陣列

↓

問題三：需要連續軌跡還是離散校正？

- └─ 連續 → Wi-Fi / BLE / UWB
- └─ 離散校正 → RFID / NFC / 視覺標記 + SLAM 為主力

案例

案例 4-1 亞馬遜倉儲機器人：用貼紙撐起全球物流

亞馬遜 (Amazon Robotics · 前身 Kiva Systems) 的倉儲機器人是「低科技解決高難度問題」的教科書案例。數十萬台橘色搬運機器人在全球倉庫中穿梭，它們的定位方案卻樸素得驚人：地板上以固定間距貼滿 QR 碼，機器人底部的向下相機讀取編號確認格位，再由 QR 碼在畫面中的偏移與旋轉計算細部位姿；格位之間則靠輪子里程計短距離推算。

這個方案的高明之處在於徹底利用了環境的可控性：倉庫是自家蓋的，地板可以隨意貼、光線可以自己打、路線可以規定成網格。既然如此，何必讓機器人辛苦地做視覺 SLAM？相較之下，服務機器人進入的是別人的建築、無法改造環境，才不得不倚賴 SLAM。這個對比給同學一個重要啟示：選擇定位技術之前，先問「我能改造環境到什麼程度」——這往往比演算法本身更關鍵。

案例 4-2 醫院裡的 UWB 即時定位系統

大型醫院常面臨一個昂貴的問題：輸液幫浦、超音波推車等移動式設備經常「不知道被推到哪裡去了」，護理人員每天花大量時間找設備，醫院只好超額採購。UWB 即時定位系統 (RTLS) 的解法是：在天花板部署基站網格，每台設備貼一顆 UWB 標籤，以 TDoA 架構單向廣播，伺服器即時解算所有標籤位置並顯示在樓層平面圖上。

這個案例展示了 TDoA 架構的價值：設備標籤只發不收，一顆電池能撐數年；且系統容量幾乎不受標籤數量限制，一家醫院上萬個標籤也不會塞車。醫院場域的挑戰則在於金屬設備與人流造成的 NLOS——這也是實務精度往往達不到文宣數字的原因。同樣的架構稍作調整，就能用於工廠的堆高機防撞、倉儲的人員安全區域管理，乃至於機器人的絕對位置校正。

案例 4-3 教室裡的 AprilTag 導航實驗

大學智慧機器人教室的一個典型教學實驗：在教室四面牆上貼十張 AprilTag，每張標記的世界座標事先量測記錄。教學小車搭載一顆前向相機，只要視野中出現任一標記，就能解算出自己相對該標記的 6-DoF 位姿，再換算為世界座標——瞬間獲得不漂移的絕對定位。

這個實驗的教學價值極高：第一，它讓同學親手驗證「四個角點加相機內參就能解出位姿」這件看似神奇的事；第二，把標記遮住一半、拉遠距離、關掉燈光，同學能親身體會視覺定位的脆弱點；第三，最精彩的是把它與光達 SLAM 並行執行，在 RViz 中同時顯示兩條軌跡——SLAM 軌跡平滑但慢慢偏離，AprilTag 軌跡跳動但始終正確。這張圖勝過千言萬語，一眼道盡本章 4.7 節的融合動機。

實作活動

實作 4-1 教室 Wi-Fi 訊號定位熱區圖

目的：親手完成一次完整的指紋定位流程——建庫、量測、比對、驗證，理解 RSSI 定位的能力與極限。本實作只需手機，適合高中與大學部共同進行。

器材

- 智慧型手機 (Android 建議安裝 Wi-Fi 分析類 App ; iOS 因系統限制較嚴，可改用 AirPort 工具或以 BLE 信標替代) 。
- 方格紙或試算表軟體、捲尺、標示參考點的膠帶。

步驟

12. 場域規劃：選定一間教室，以捲尺量出長寬，在地面以 2 公尺間隔貼出格點標記 (一間普通教室約 12 至 20 個參考點)，並在方格紙上編號。

13. 離線建庫：在每個參考點站定，面向固定方向（例如一律面向講台，減少人體遮蔽造成的差異），記錄可見的前五強 AP 的名稱與 RSSI 值，每點量測三次取平均，填入試算表。
14. 繪製熱區圖：針對訊號最強的那台 AP，把各參考點的 RSSI 值以色階填入方格紙（強為紅、弱為藍），觀察訊號強度的空間分布是否符合「離 AP 越遠越弱」的預期。
15. 線上定位測試：由一位同學閉眼被帶到某個未告知的參考點，量測當下指紋；其他同學以 KNN 方法（計算與資料庫中每筆指紋的歐氏距離，取最相近的 3 筆平均座標）推算位置。
16. 誤差統計：重複測試至少 8 次，記錄每次的估測位置與真實位置，計算平均誤差與最大誤差。
17. 干擾實驗：請全班同學聚集在測試點周圍，重新量測同一點的指紋，比較 RSSI 變化與定位誤差的變化。

討論問題

19. 你們量到的平均定位誤差是多少？與課文提到的 3 至 8 公尺是否相符？誤差最大的參考點在哪裡？為什麼是那裡？
20. 人群聚集讓誤差變大了多少？這對「商場尖峰時段的室內導航」意味著什麼？
21. 如果把參考點間隔從 2 公尺縮小到 1 公尺，精度會提升多少？建庫工作量增加多少？這個代價值得嗎？
22. (大學部) 若要把這套系統與第 14 章的光達 SLAM 融合，你會採用 4.7 節的哪一種耦合方式？為什麼？

延伸挑戰 (大學部)

以 Python 撰寫 WKNN 定位程式，讀入建好的指紋資料庫與即時量測值，自動輸出估

測座標與誤差統計；進一步比較 $K = 1、3、5$ 時的精度差異，並嘗試加入手機 IMU 的步數偵測做軌跡平滑，觀察誤差是否下降。

延伸閱讀

20. P. Bahl and V. N. Padmanabhan, "RADAR: An In-Building RF-Based User Location and Tracking System," IEEE INFOCOM, 2000. (Wi-Fi 指紋定位的開山之作，必讀經典)
21. H. Liu et al., "Survey of Wireless Indoor Positioning Techniques and Systems," IEEE Transactions on Systems, Man, and Cybernetics, 2007. (室內定位技術的完整綜述)
22. IEEE Std 802.15.4z-2020, Enhanced Ultra Wideband (UWB) Physical Layers and Associated Ranging Techniques. (UWB 測距的現行標準文件)
23. E. Olson, "AprilTag: A Robust and Flexible Visual Fiducial System," ICRA, 2011. (AprilTag 原始論文，說明糾錯設計思路)
24. S. Garrido-Jurado et al., "Automatic Generation and Detection of Highly Reliable Fiducial Markers Under Occlusion," Pattern Recognition, 2014. (ArUco 原始論文)
25. OpenCV 官方文件之 ArUco 模組教學 (docs.opencv.org) . (附完整程式範例，實作首選參考)
26. Bluetooth SIG, Bluetooth Core Specification 5.1 之 Direction Finding 章節. (AoA / AoD 尋向技術規格)

小結

7. GPS 在室內失效有三個原因：訊號極度微弱難以穿透建材、多路徑反射造成測距失真、可見衛星幾何條件惡化使誤差被放大。
8. Wi-Fi RSSI 因室內傳播環境複雜，不適合直接測距，主流採指紋定位：離線建立訊號資料庫、線上以 KNN 比對，精度約 3 至 8 公尺，零硬體成本但需維護資料庫。
9. BLE 信標可依需求布置，有鄰近偵測、三邊定位、指紋定位三種模式；藍牙 5.1 起的尋向功能以 AoA / AoD 大幅提升精度，代價是需要陣列天線。
10. UWB 以奈秒級短脈衝直接量測飛行時間，能分離直達波與反射波，是精度最高的無線定位技術；TWR 免同步但容量有限，TDoA 省電且容量大但需基站精確同步。
11. RFID、NFC、QR Code 提供離散但極可靠的地標定位，成本低廉，適合可改造環境的場域，亞馬遜倉儲機器人即為代表案例。
12. AprilTag / ArUco 等視覺標記能由四個角點與相機內參解出完整 6-DoF 位姿（PnP 問題），是機器人絕對位姿校正、精確對接與 SLAM 精度評估的利器。
13. 室內定位與 SLAM 恰好互補：前者誤差有界但需基礎設施，後者自給自足但會漂移；融合方式分鬆耦合、緊耦合與輔助初始化重定位三種層次。

下一章，我們把視野從室內拉到戶外，探討無線電導航與衛星導航——GPS / GNSS 究竟如何運作？為什麼 RTK 能把公尺級誤差壓到公分級？以及校園巡檢機器人如何在室內外之間無縫切換定位模式。

台灣自編大學教科書

@prof

《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第 5 章

無線電導航與衛星導航

編者：李世淵 (機器人叫獸)

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

學習目標

上一章我們探討了室內定位；本章走出戶外，探討覆蓋全球的無線電與衛星導航系統。這是機器人「絕對定位」能力的主要來源，也是理解「為什麼自駕車與無人機仍然需要 SLAM」的關鍵一課。學完本章之後，同學應能夠達成下列目標：

24. 說明 GNSS 的三大組成（太空、控制、使用者區段）與衛星定位的基本原理，並解釋為何至少需要四顆衛星。
25. 區分三角定位、三邊定位與到達時間差定位三種幾何定位法的差異與適用場合。
26. 說明載波相位量測與 RTK 的原理，理解為何 RTK 能將公尺級誤差壓縮到公分級。
27. 比較 5G、LoRa、Wi-Fi RTT 等地面無線電定位技術在機器人應用上的潛力與限制。
28. 列舉並解釋無線電定位的主要誤差來源：電離層、對流層、多路徑、遮蔽、幾何條件與干擾。
29. 說明 GNSS 如何與 IMU、SLAM 融合，並設計一套校園巡檢機器人的室內外混合導航策略。

核心概念

中文名詞	英文名詞	概念說明
全球衛星導航系統	GNSS	各國衛星定位系統的統稱，含 GPS、GLONASS、Galileo、BeiDou。
虛擬距離	Pseudorange	由訊號傳播時間乘光速得到的距離，因含接收機時鐘誤差故稱「虛擬」。

中文名詞	英文名詞	概念說明
星曆	Ephemeris	衛星軌道參數，用來精確計算衛星在任一時刻的位置。
精度稀釋因子	DOP	描述衛星幾何分布對定位誤差的放大倍率，值越小越好。
載波相位	Carrier Phase	量測載波波形的相位，精度遠高於虛擬距離，是 RTK 的基礎。
整週波未定值	Integer Ambiguity	載波相位量測中未知的整數週期數，解出它才能達成公分級精度。
即時動態定位	RTK	以基準站差分修正達成公分級定位的技術。
差分 GNSS	DGNSS	利用已知座標的基準站計算誤差修正量，播送給移動站使用。
星基增強系統	SBAS	以同步衛星播送修正訊息的廣域增強系統，如 WAAS、EGNOS。
網路 RTK	NTRIP	透過網際網路傳送 RTK 修正資料的通訊協定。
GNSS 拒止環境	GNSS-denied	衛星訊號被遮蔽或干擾而無法使用的環境，如室內、隧道、都市峽谷。
都市峽谷效應	Urban Canyon	高樓夾道遮蔽衛星並造成強烈多路徑反射的現象。

5.1 GPS / GNSS 基本原理

GPS (Global Positioning System) 是美國於 1970 年代開始建置、1990 年代全面運作的衛星定位系統。今日它已不是唯一選擇：俄羅斯的 GLONASS、歐盟的 Galileo、中國的北斗 (BeiDou) 都已達成全球覆蓋，加上日本的 QZSS 與印度的 NavIC 兩套區域系統，統稱為全球衛星導航系統 (GNSS · Global Navigation Satellite System)。截至 2026 年 3 月，四大系統在軌運作的衛星合計約 130 顆，其中 GPS 約 31 顆，分布於 6 個軌道面、約兩萬公里高的中軌道。2020 年後生產的智慧型手機大多同時支援四大系統——這正是近年手機定位明顯變準的原因：可用衛星變多，幾何條件自然變好。

三大區段

- 太空區段 (Space Segment)：在軌衛星群。每顆衛星攜帶高精度原子鐘，持續廣播自己的識別碼、時間戳記與軌道參數 (星曆)。
- 控制區段 (Control Segment)：地面監測站網路與主控站。持續追蹤衛星、修正軌道與時鐘偏差，並把最新星曆上傳給衛星。
- 使用者區段 (User Segment)：接收機。只接收、不發射——這是理解 GNSS 的關鍵：衛星並不知道你在哪裡，定位完全由你的接收機自行完成，因此使用者數量可以無限多。

定位的基本原理：測距交會

GNSS 定位的本質是三維的三邊定位。衛星在訊號中蓋上「我在時刻 t 發出這個訊號」的時間戳記；接收機收到時記下自己的時刻，兩者相減得到傳播時間，乘上光速就是距離。已知衛星位置 (由星曆算出) 與距離，接收機就位於一個以該衛星為球心的球面上。三顆衛星的三個球面相交，理論上就能定出位置。

為什麼需要第四顆衛星

問題出在時鐘。衛星上的原子鐘造價高昂、精度達奈秒級；但接收機裡只是一顆廉價石英振盪器，與衛星時間隨時可能相差數微秒。而光速下 1 微秒就等於 300 公尺的距離誤差——這誤差會讓三個球面根本交不到一起。

解法非常巧妙：把接收機的時鐘誤差當作第四個未知數，一起解出來。未知數變成四個 (x 、 y 、 z 、 Δt)，就需要四條方程式，也就是四顆衛星。這也是為什麼由於接收機時鐘不準，量到的距離不能稱為真距離，而稱為虛擬距離 (Pseudorange)。

這個設計的副作用是：GNSS 接收機在完成定位的同時，也把自己的時鐘校正到了原子鐘等級的精度。全球的電信基地台、電網、金融交易系統都靠 GNSS 授時同步——衛星導航系統對現代社會的重要性，其實遠超過「幫你導航」。

5.2 三角定位、三邊定位與時間差定位

無線電定位的幾何方法有三大類，同學務必分清楚——這三個名詞在中文文獻中常被混用，但原理截然不同。

一、三角定位 (Triangulation)：用角度

量測目標相對於兩個已知基準點的方位角，兩條射線的交點即為目標位置。傳統測量學、航海用的無線電測向 (RDF)、以及第 4 章提到的藍牙 AoA 尋向，都屬於這一類。特點是需要方向性天線或天線陣列，但不需要精確的時間量測。

二、三邊定位 (Trilateration)：用距離

量測目標到各已知點的距離，求球面 (或圓) 的交點。GNSS、UWB 的 TWR 都屬於這一類。特點是需要精確的時間或訊號強度量測，但不需要知道方向。第 4 章的公式已經介紹過二維情況，GNSS 則是三維版本再加上時鐘未知數。

三、到達時間差定位 (TDoA)：用時間差

不量絕對距離，只量「訊號抵達兩個基站的時間差」。時間差固定的點集合，在幾何上構成一條雙曲線（三維中為雙曲面），兩組以上雙曲面的交點即為目標位置——因此又稱雙曲線定位。第 4 章介紹的 UWB TDoA 架構、以及二戰時期的羅蘭 (LORAN) 長波導航系統都採用此法。特點是目標端只需發射或只需接收，極省電且系統容量大，代價是各基站必須精確時間同步。

方法	量測物理量	幾何軌跡	需要條件	典型應用
三角定位	角度	射線	方向性天線	BLE AoA、無線電測向
三邊定位	絕對距離	圓 / 球面	精確計時或已知發射功率	GNSS、UWB TWR
TDoA	距離差	雙曲線 / 雙曲面	基站間精確同步	UWB TDoA、LORAN

表 5-1 三種無線電定位幾何方法比較

精度稀釋因子 (DOP)

無論哪一種方法，基準點（衛星或基站）的幾何分布都會直接影響精度。想像兩顆衛星在天空中幾乎同一方向：它們的球面交角極小，量測誤差稍有變化，交點就會沿著切線方向大幅滑動。反之，衛星均勻分布在四面八方時，交角接近垂直，誤差被侷限在小範圍內。

這個放大效應以精度稀釋因子 (DOP) 量化：實際定位誤差約等於測距誤差乘上 DOP 值。開闊天空下 DOP 常小於 2，都市峽谷或室內則可能飆到 10 以上——這正是第 4 章所說「幾何條件惡化」的量化解釋。

5.3 RTK-GNSS 與高精度戶外定位

一般手機 GNSS 的精度約在 3 至 10 公尺，這對「找路」夠用，但對自駕車、農

機自動駕駛、無人機測繪而言遠遠不足。要把誤差壓到公分級，需要兩項關鍵技術：差分修正與載波相位量測。

一、差分的思想

GNSS 的誤差有一大部分是「區域共通」的：電離層與對流層的延遲、衛星時鐘與軌道的微小偏差，在相距數公里的兩台接收機上幾乎完全相同。

差分 GNSS (DGNSS) 利用這個特性：在一個座標已經精確測定的位置架設基準站 (Base Station)，基準站把「算出來的位置」與「已知的真實位置」相比，就能反推出當下的誤差量，再把修正量播送給附近的移動站 (Rover)。移動站扣掉共通誤差後，精度可從公尺級提升到公分級。

二、載波相位：精度躍升的關鍵

一般 GNSS 使用的是調變在載波上的「測距碼」，其碼片長度約數百公尺，量測解析度自然受限。但載波本身的波長只有約 19 公分 (L1 頻段)，如果能量測載波的相位，解析度可達波長的百分之一，也就是公釐級。

代價是產生了一個新的未知數：接收機能精確量出「不滿一週的相位」，卻不知道訊號總共走了幾個完整波長——這就是整週波未定值 (Integer Ambiguity) 問題。RTK 系統的核心技術，就是快速且可靠地把這個整數解出來 (稱為 Ambiguity Resolution 或「固定解」)。一旦固定成功，定位精度可達公分級；固定失敗時只能維持「浮動解」，精度退回公分級。

三、RTK 系統架構

圖 5-1 RTK-GNSS 系統架構

基準站 (Base) : 座標已知 , 接收衛星訊號 → 計算誤差修正量

↓ (電台、4G / 5G 或網際網路 NTRIP 傳送)

移動站 (Rover) : 接收衛星訊號 + 基準站修正量

↓

雙差分運算 → 消除共通誤差 → 解算整週波未定值

↓

固定解 (Fixed) : 公分級定位 / 浮動解 (Float) : 公寸級定位

四、網路 RTK 與實務考量

自建基準站成本高、覆蓋有限，因此發展出網路 RTK：由國家或業者建置基準站網路，透過 NTRIP 協定以網際網路播送修正資料。台灣的內政部國土測繪中心即建置有 e-GNSS 系統，提供全台的即時定位服務。

- 基線長度限制：移動站與基準站距離越遠，「共通誤差」的假設越不成立，通常建議在數十公里內。
- 初始化時間：從開機到固定解通常需要數十秒，過隧道或樹下失鎖後需要重新固定。
- 仍需開闊天空：RTK 消除的是大氣與衛星誤差，對「遮蔽」與「多路徑」無能為力——樹下、高樓旁的 RTK 一樣不可靠。這點對機器人應用至關重要。

值得一提的是 Galileo 的高精度服務 (HAS) 等新一代技術，可透過衛星直接播送修正資料，達到約 20 公分等級的精度而不需地面基準站，這對大範圍戶外機器人應用是相當實用的折衷方案。

5.4 5G / LoRa / Wi-Fi RTT 在機器人定位的應用

衛星訊號來自兩萬公里外，天生脆弱。地面基地台的訊號則強得多，能不能拿來定位？這是近年通訊與定位融合的熱門方向。

一、5G 定位

5G 從標準制定之初就把定位納入設計，具備三項有利條件：毫米波與大頻寬帶來高時間解析度、大規模天線陣列 (Massive MIMO) 可量測到達角、小基站密集布建縮短了距離。5G 定位綜合使用到達時間差、到達角與往返時間等方法，在理想條件下可達公尺甚至次公尺級。對工廠 AGV、園區自駕車而言，這意味著「通訊與定位共用同一套基礎設施」，極具吸引力。限制在於：定位精度高度依賴基站密度，且需要電信業者開放相關能力。

二、LoRa 定位

LoRa 是長距離、低功耗的物聯網通訊技術，單一閘道器覆蓋可達數公里。它的定位能力來自 TDoA——多個閘道器記錄同一封包的到達時間差。精度僅約數十至數百公尺，遠不及前述技術，但勝在覆蓋廣、耗電極低、成本低廉。適用場景是「大範圍粗定位」：農場設備追蹤、山區資產監控，或第 4 章案例中「UWB 精確測距 + LoRa 長距離回傳」的混合架構。

三、Wi-Fi RTT (往返時間)

IEEE 802.11mc 標準引入的 Fine Timing Measurement (FTM) 機制，讓裝置能與 AP 交換時間戳記，直接量測往返時間換算距離——這與第 4 章的 RSSI 猜距離有本質差異。精度可達 1 至 2 公尺，且不需要建立指紋資料庫。限制是 AP 與裝置雙方都必須支援 FTM，目前普及率仍是主要瓶頸。

技術	涵蓋範圍	典型精度	基礎設施需求	機器人應用定位
GNSS	全球	3–10 公尺	無（衛星已在天上）	戶外主力
RTK-GNSS	基準站數十公里內	公分級	基準站或網路服務	農機、測繪、自駕
5G	基站覆蓋範圍	公尺至次公尺	密集基站	園區、工廠
LoRa	數公里	數十至數百公尺	閘道器	大範圍粗追蹤
Wi-Fi RTT	室內數十公尺	1–2 公尺	支援 FTM 的 AP	室內輔助

表 5-2 無線電定位技術的機器人應用比較

5.5 無線電定位誤差來源：多路徑、遮蔽、干擾

理解誤差，才能判斷什麼時候可以信任 GNSS、什麼時候必須切換到 SLAM。

GNSS 的誤差可分為三大類。

一、傳播路徑誤差（可用差分消除）

- 電離層延遲：距地表約 50 至 1000 公里的電離層含大量自由電子，會延遲訊號。這是單頻接收機最大的誤差源，可達數公尺，且隨太陽活動劇烈變化。雙頻接收機可利用不同頻率的延遲差異加以消除——這也是雙頻 GNSS 較昂貴但精準得多的原因。
- 對流層延遲：大氣層底部的水氣與氣壓造成的延遲，約數十公分至數公尺，可用模型估計修正。
- 衛星時鐘與軌道誤差：控制區段已持續修正，殘餘誤差約在公尺以下。

二、局部環境誤差（差分無法消除）

- 多路徑：訊號經建築、地面、水面反射後抵達，路徑變長使測距偏大。這是都市與校園環境的主要誤差來源，可達數公尺甚至更多。抑制方法包括扼流圈天線、訊號品質篩選演算法，但無法根除。

- 遮蔽：建築、樹冠、隧道直接擋住訊號，導致可用衛星數不足或幾何條件惡化。台灣校園常見的大王椰子與榕樹樹冠，就足以讓 RTK 從固定解掉回浮動解。
- 都市峽谷效應：高樓夾道時，天空只剩狹長一條，可見衛星集中在同一方向使 DOP 惡化，同時牆面反射造成強烈多路徑——遮蔽與多路徑雙重打擊，這是自駕車在市區最頭痛的定位問題。

三、人為干擾

- 干擾 (Jamming)：以強訊號淹沒微弱的衛星訊號，使接收機失鎖。市面上非法的小型干擾器即可癱瘓周圍數十公尺內的 GNSS。
- 欺騙 (Spoofing)：發射偽造的衛星訊號，讓接收機算出錯誤但「看起來正常」的位置。這比干擾更危險——系統不會報錯，只會安靜地把你導向錯誤的地方。對無人機與自駕車而言，這是嚴肅的資安議題。

這三類誤差的存在，正是「機器人不能只靠 GNSS」的根本理由。一台在校園樹蔭下巡檢的機器人，隨時可能遇到訊號中斷、跳點數公尺、或 RTK 掉解——若導航系統完全依賴 GNSS，機器人就會在這些時刻失去方向。解法就是下一節的融合。

5.6 無線電導航與 IMU / SLAM 的融合

第 4 章我們看過室內定位與 SLAM 的互補；本節的 GNSS 與 IMU / SLAM 融合，是同一思想在戶外的展開，也是目前最成熟的感測融合應用之一。

GNSS 與 IMU 的天作之合

這兩者的誤差特性恰好完全相反：

- IMU：更新率極高（數百 Hz）、短時間內極為精確、完全不受遮蔽與干擾影響；但積分漂移使誤差隨時間快速累積（第 7 章詳述）。

- GNSS：更新率低（1 至 10 Hz）、單次量測有雜訊；但誤差有界，長時間不會漂移。

融合的邏輯因此非常自然：以 IMU 提供高頻率的連續軌跡，以 GNSS 定期把累積漂移「拉回來」。這套組合稱為 GNSS / INS 組合導航，是航空、船舶、飛彈與自駕車的標準配備。實作上通常以卡爾曼濾波器（第 11 章）進行，濾波器會持續估計並補償 IMU 的偏差（Bias），使得即使 GNSS 短暫中斷（過隧道、樹下），IMU 仍能維持數秒到數十秒的可用精度——這段時間稱為「慣性導航的推算能力」。

加入 SLAM：三重保險

GNSS / INS 在長隧道或室內仍會失效，此時就需要 SLAM 接手。完整的三重架構如下：

圖 5-2 GNSS / INS / SLAM 三重融合架構

IMU（數百 Hz）：提供高頻連續運動估測，短期精確

+

GNSS（1–10 Hz）：戶外提供絕對位置，消除長期漂移

+

SLAM（10–30 Hz）：以環境特徵提供相對約束，GNSS 失效時的主力

↓

融合濾波器 / 圖優化後端（第 11、12 章）

↓

連續、不漂移、全天候可用的位姿估測

關鍵工程問題：座標系統一

融合前必須先解決一個容易被忽略的問題：GNSS 給的是經緯度（WGS84 地理座標），SLAM 用的是以出發點為原點的區域直角座標。兩者必須先轉換到同一個座標系——通常做法是把經緯度轉為東北天（ENU）局部直角座標，並以第一個 GNSS 定位點作為原點。此外還需要處理時間同步（各感測器時間戳記對齊）與外參校正（GNSS 天線、IMU、光達三者的相對安裝位置）。這些「不起眼」的工程細節，往往才是實作能否成功的決定因素——第 10 章的座標轉換與 TF Tree 會完整處理這個主題。

信任度切換

實務系統還需要一個「裁判」機制，判斷此刻該相信誰。常用依據包括：可用衛星數、DOP 值、RTK 解算狀態（固定解 / 浮動解 / 單點解）、以及 GNSS 與 SLAM 估測之間的差異是否過大。當機器人從戶外進入室內，系統應平滑地把權重從 GNSS 轉移到 SLAM，而不是突然跳變——這正是下一節校園場景的核心挑戰。

5.7 校園巡檢機器人的室內外混合導航

把本章與前一章的技術整合起來，我們可以完整設計一台校園巡檢機器人的導航系統。這個場景之所以經典，正是因為它涵蓋了所有困難情境。

場域分析：一趟巡檢的四種環境

- 開闊操場與停車場：天空開闊，GNSS / RTK 表現最佳，可作為定位主力。
- 樹蔭步道與建築物旁：遮蔽與多路徑同時發生，GNSS 可能跳點數公尺或掉解。此區必須以 SLAM 為主、GNSS 為輔，且要有能力判斷 GNSS 不可信。
- 穿堂與騎樓：半室內，GNSS 時有時無，是最容易出錯的「過渡區」。

- 室內走廊與教室：GNSS 完全失效，純靠光達 / 視覺 SLAM，並以第 4 章的 AprilTag 或 BLE 信標提供樓層資訊與絕對校正。

系統設計策略

21. 地圖分層：戶外使用以 GNSS 座標為基準的全域地圖，室內使用各棟建築的 SLAM 區域地圖，兩者以「門口」作為錨點連接，形成拓樸式的地圖網路（第 3 章的拓樸地圖在此派上用場）。
22. 感測器配置：RTK-GNSS 天線（戶外主力）+ IMU（連續性骨幹）+ 3D 或 2D 光達（SLAM 主力）+ 相機（視覺標記與語意辨識）。
23. 模式切換機制：以 GNSS 品質指標（衛星數、DOP、解算狀態）與 SLAM 匹配品質共同決策，在過渡區採漸進式權重調整而非硬切換。
24. 過渡區強化：在每棟建築出入口貼設 AprilTag，作為「戶外座標系」與「室內座標系」之間的橋樑——機器人一看到標記，就同時知道自己在兩個座標系中的位置，完成無縫接軌。
25. 失效備援：若所有定位手段同時異常（例如強烈干擾加上光達被遮），機器人應停止移動並回報，而非帶著錯誤位姿繼續行駛——安全設計的基本原則。

這套架構將在第 30 章的專題中實際落地。同學此刻只需掌握一個核心觀念：真實世界的機器人導航，從來不是「選一種最好的技術」，而是「讓多種技術在各自擅長的場合接力」。這正是本書第二篇逐一介紹六大感測器的用意——你必須認識每一位隊員的長處與弱點，才能當一個好教練。

公式或流程

一、虛擬距離方程式

設接收機位置為 (x, y, z) 、時鐘偏差為 Δt ，第 i 顆衛星的已知位置為 (x_i, y_i, z_i) ，

量得的虛擬距離為 ρ_i ，則：

$$\rho_i = \sqrt{(x - x_i)^2 + (y - y_i)^2 + (z - z_i)^2} + c \cdot \Delta t$$

其中 c 為光速。式中未知數為 x 、 y 、 z 、 Δt 共四個，因此至少需要四顆衛星建立四條方程式。由於方程式非線性，實務上以泰勒展開線性化後迭代求解，或以最小平方法處理多於四顆衛星時的冗餘量測——當可見衛星有 10 顆時，就有 10 條方程式解 4 個未知數，多出來的資訊正好用來抑制雜訊。

二、定位誤差與 DOP

定位誤差可近似分解為兩個因子的乘積：

$$\text{定位誤差} \approx \text{UERE} \times \text{DOP}$$

UERE (User Equivalent Range Error) 是單一衛星的等效測距誤差 (含電離層、對流層、多路徑等)，DOP 則是幾何放大倍率。這條簡潔的關係式告訴我們兩條改善途徑：降低測距誤差 (用雙頻、用差分修正)，或改善幾何條件 (用多星系、找開闊處)。同學可以用手機的 GNSS 檢視 App 實際觀察：走到大樓旁邊時，可見衛星數下降、DOP 上升，定位誤差圈立刻變大。

三、載波相位觀測式

載波相位觀測量 φ (以週為單位) 與真實距離 r 的關係為：

$$\lambda \cdot \varphi = r + c \cdot \Delta t + \lambda \cdot N + \text{誤差項}$$

其中 λ 為載波波長 (L1 約 19 公分)， N 即整週波未定值——一個未知的整數。RTK 的全部精髓，就在於利用基準站與移動站的雙重差分消去大部分誤差項後，快速可靠地把 N 解算為整數。一旦 N 固定， $\lambda \cdot \varphi$ 就成為公釐級精度的距離量測，這就是公分級定位的來源。

四、GNSS 定位可信度判斷流程

圖 5-3 機器人判斷是否信任 GNSS 的決策流程

① 可見衛星數 ≥ 6 顆？ 否 $\rightarrow$ 不信任，切換 SLAM 主導

↓ 是

② HDOP < 2.0 ？ 否 $\rightarrow$ 降低權重

↓ 是

③ RTK 狀態為固定解？ 浮動解 $\rightarrow$ 中等權重；單點解 $\rightarrow$ 低權重

↓ 固定解

④ 與 IMU / SLAM 推算位置的差異 $<$ 門檻？ 否 $\rightarrow$ 判定為多路徑跳點，拒絕本次量測

↓ 是

⑤ 採納為高權重絕對位置量測，校正累積漂移

案例

案例 5-1 農機自駕：RTK 最成功的商業應用

在自駕車還在測試的年代，RTK 自動駕駛曳引機早已在全球農田中大規模商業運轉。農業場景之所以成為 RTK 的完美舞台，原因很值得同學思考：田地空曠無遮蔽，衛星訊號品質極佳；作業路徑是規則的直線來回，不需複雜決策；而精度需求卻極高——播種行距偏差幾公分，就會造成重疊或漏播，累積成可觀的產量與成本損失。

RTK 讓曳引機能以公分級精度沿著預定路線自動行駛，駕駛只需在田邊轉彎時介

入（新型系統連轉彎也自動化）。這個案例展示了一個重要的工程判斷：當環境條件對某項技術極為有利時，就應該把它用到極致，而不必為了「技術完整性」硬加上不需要的感測器。反過來說，同一台曳引機開進果園樹冠下，RTK 立刻失效——這時才需要光達或視覺 SLAM 接手。技術選型永遠是場域決定的。

案例 5-2 都市峽谷中的自駕車定位

自駕車在市區行駛時面臨 GNSS 最惡劣的考驗：兩側高樓把天空切成一條窄縫，可見衛星從十餘顆掉到三、四顆且集中在同一方向，DOP 惡化數倍；更麻煩的是玻璃帷幕大樓的鏡面反射——接收機可能收到的是從對面大樓彈回來的訊號，算出的位置偏差可達數十公尺，而且系統往往不知道自己錯了。

主流自駕車的解法是徹底降低對 GNSS 的依賴：以事先測繪的高精地圖為基準，用車上的光達與相機即時比對地圖特徵（車道線、路緣、標誌桿、建物輪廓）來定位，GNSS 僅作為初始化與粗略區域判斷之用。這正是第 17、18 章 3D 光達建圖技術的商業價值所在——當天上的訊號不可靠時，地面的環境結構才是最忠實的參考。

案例 5-3 無人機的 GPS 拒止飛行

消費級無人機在開闊處能靠 GNSS 穩定懸停，一旦飛進橋樑下方、室內廠房或森林，就進入所謂的 GNSS 拒止（GNSS-denied）環境。早期無人機在此情境下會失控漂移，甚至觸發返航卻撞上障礙物。

現代解法是視覺慣性里程計（VIO）：以下視或前視相機追蹤地面／環境的紋理特徵，配合 IMU 估測運動，完全不依賴外部訊號（第 25 章詳述）。這也解釋了為什麼無人機的視覺定位研究會集中在 ETH / UZH（第 2 章）——他們的核心命題正是「當衛星幫不上忙時，機器人如何只靠自己的眼睛飛行」。對本書讀者而言，這是一條清楚的學習線索：本章談的是「有訊號時怎麼用好」，第六、七篇談的則是「沒訊

號時怎麼辦」。

實作活動

實作 5-1 校園 GNSS 訊號品質勘測

目的：以手機實測校園各處的衛星訊號品質，親身驗證遮蔽、多路徑與 DOP 對定位精度的影響，並產出一張「GNSS 可信度地圖」，作為第 30 章巡檢機器人專題的路線規劃依據。

器材

- 智慧型手機，安裝可顯示衛星資訊的 App (可查詢「GNSS 檢視」「GPS Test」類工具，需能顯示衛星數、訊號強度與 DOP 或精度估計值) 。
- 校園平面圖、記錄表、捲尺 (選用) 。

步驟

18. 選點：在校園中選出至少 8 個具代表性的測點，涵蓋四種環境——開闊操場、樹蔭步道、建築物北側牆邊、穿堂或騎樓下、以及室內走廊各至少一點。
19. 量測：在每個測點靜置手機 60 秒後記錄：可見衛星總數、實際用於定位的衛星數、各衛星訊號強度分布、HDOP 值 (或 App 顯示的精度估計)、以及顯示的經緯度座標。
20. 重複性測試：在同一測點間隔 5 分鐘再量一次，比較兩次的經緯度差異——這個差異就是定位的重複性誤差。
21. 繪製可信度地圖：把各測點依「衛星數與 DOP」分為三級 (綠：可信賴，可用 GNSS 主導；黃：需降權，SLAM 輔助；紅：不可信任，須切換 SLAM)，標示在校園平面圖上。

22. 多路徑觀察：在高樓玻璃帷幕旁與空曠處各站 2 分鐘，比較定位點的跳動幅度，並記錄各自的衛星訊號強度分布是否異常。
23. 綜合報告：依據可信度地圖，為一台校園巡檢機器人規劃一條巡檢路線，標明沿途各段應以何種定位方式為主導，以及應在哪些位置設置 AprilTag 校正點。

討論問題

23. 哪一個測點的定位重複性誤差最大？該處的衛星數與 DOP 值如何？符合本章 5.5 節的哪一類誤差來源？
24. 樹蔭下與建築物旁，哪一種遮蔽對定位的影響較大？你認為原因是什麼（提示：樹葉是部分穿透，牆壁是完全遮蔽但會反射）？
25. 若這台巡檢機器人只配置 GNSS 而沒有 SLAM，它在你們規劃的路線上會在哪些路段出問題？後果是什麼？
26. (大學部) 依據圖 5-3 的決策流程，為你們的校園場域設定各項門檻值（衛星數、HDOP、位置差異），並說明訂定理由。

延伸挑戰 (大學部)

以 Python 撰寫程式解析手機或 USB GNSS 接收器輸出的 NMEA 語句 (GGA、GSA、GSV)，自動記錄衛星數、HDOP 與定位品質指標，並繪製時間序列圖與軌跡圖。進一步嘗試計算「靜態測點的位置散布標準差」，量化不同環境下的定位精度差異。這份程式將可直接沿用於第 30 章的巡檢專題。

延伸閱讀

27. P. Misra and P. Enge, Global Positioning System: Signals, Measurements, and Performance, 2nd ed., Ganga-Jamuna Press, 2011. (GNSS 原理的標準教科書)

28. B. Hofmann-Wellenhof, H. Lichtenegger, and E. Wasle, GNSS – Global Navigation Satellite Systems, Springer, 2008. (涵蓋四大系統與 RTK 的完整參考書)
29. P. D. Groves, Principles of GNSS, Inertial, and Multisensor Integrated Navigation Systems, 2nd ed., Artech House, 2013. (GNSS / INS 組合導航的權威著作・第 5.6 節主要參考)
30. 內政部國土測繪中心 e-GNSS 即時動態定位系統網站. (台灣網路 RTK 服務的官方資訊與使用說明)
31. RTKLIB 開源專案 (rtklib.com) . (免費的 GNSS / RTK 處理軟體・可用於教學實驗與後處理分析)
32. European Union Agency for the Space Programme, Galileo High Accuracy Service (HAS) 官方說明文件. (新一代衛星直播修正服務)
33. 3GPP TS 38.305, NG-RAN Stage 2 Functional Specification of User Equipment Positioning. (5G 定位的標準規範文件)

小結

14. GNSS 由太空、控制、使用者三大區段組成；接收機只收不發，以虛擬距離進行三維三邊定位，並因需同時解算時鐘偏差而至少需要四顆衛星。截至 2026 年，四大系統在軌衛星合計約 130 顆。
15. 無線電定位有三種幾何方法：三角定位用角度、三邊定位用距離、TDoA 用距離差；基準點的幾何分布以 DOP 量化，定位誤差約等於測距誤差乘上 DOP。
16. RTK 結合差分修正與載波相位量測達成公分級定位，關鍵在於解算整週波未定值；固定解為公分級、浮動解為公寸級，且對遮蔽與多路徑仍無能為力。

17. 地面無線電定位各有定位：5G 適合園區與工廠、LoRa 適合大範圍粗追蹤、Wi-Fi RTT 可提供室內公尺級測距，但都受限於基礎設施普及度。
18. GNSS 誤差分為可差分消除的傳播誤差（電離層、對流層、衛星時鐘軌道）、無法消除的局部誤差（多路徑、遮蔽、都市峽谷）與人為干擾（Jamming、Spoofing）。
19. GNSS 與 IMU 誤差特性互補，構成成熟的組合導航；再加入 SLAM 即成三重架構，可在戶外、過渡區與室內全程維持連續且不漂移的定位。
20. 校園巡檢機器人需依環境分區採用不同定位主力，並以視覺標記作為室內外座標系的橋樑；系統設計的核心觀念是「讓多種技術在各自擅長的場合接力」。

下一章，我們探討一個最古老卻仍在服役的導航手段——地磁。從航海羅盤到手機指南針，從鐵磁干擾校正到室內地磁指紋定位，看看地球本身的磁場如何為機器人提供絕對方向參考。

台灣自編大學教科書

@prof

《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第 6 章

地磁導航與電子羅盤

編者：李世淵 (機器人叫獸)

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

學習目標

指南針是人類最古老的導航工具之一，卻仍活躍在每一支手機、每一台無人機之中。本章探討地球磁場如何為機器人提供絕對方向參考，以及它在室內為何會從「乾淨的方向感測器」變成「獨特的位置指紋」。學完本章之後，同學應能夠達成下列目標：

- 30. 說明地球磁場的成因與三個描述參數：磁偏角、磁傾角與磁場強度。
- 31. 說明磁力計如何量測磁場，並由三軸磁力計配合傾斜補償計算出航向角。
- 32. 解釋室內地磁指紋定位的原理，並說明「干擾」為何反而成為定位的資訊來源。
- 33. 區分硬鐵誤差與軟鐵誤差的成因，並執行八字形校正等實務校正程序。
- 34. 說明地磁感測在手機、無人機與機器狗中扮演的角色與已知限制。
- 35. 分析地磁、IMU 與視覺 SLAM 三者的誤差特性，說明其互補融合的邏輯。

核心概念

中文名詞	英文名詞	概念說明
地磁場	Geomagnetic Field	地球外核液態金屬對流產生的磁場，強度約 25 至 65 微特斯拉。
磁偏角	Declination	磁北與真北之間的水平夾角，隨地點與時間變化。
磁傾角	Inclination / Dip	磁力線與水平面的夾角，赤道近 0 度、兩極近 90 度。
磁力計	Magnetometer	量測磁場強度與方向的感測器，多採霍爾效應或磁阻原理。
電子羅盤	Electronic Compass	結合磁力計與加速度計，輸出航向角的模組。

中文名詞	英文名詞	概念說明
航向角	Heading / Yaw	機器人前進方向與參考北方之間的夾角。
傾斜補償	Tilt Compensation	利用加速度計量得的姿態，把磁力計讀值投影回水平面。
硬鐵誤差	Hard-iron Error	載具上永磁材料造成的固定偏移，使量測球心偏離原點。
軟鐵誤差	Soft-iron Error	鐵磁材料扭曲外部磁場，使量測球面變形為橢球。
地磁指紋	Magnetic Fingerprint	建築物鋼構造成的局部磁場扭曲，可作為位置特徵。
世界地磁模型	WMM	由 NOAA 與 BGS 共同維護的全球磁場模型，每五年更新一次。
磁暴	Geomagnetic Storm	太陽活動引發的磁場劇烈擾動，會暫時降低羅盤精度。

6.1 地球磁場與磁力計

地球是一顆巨大的磁鐵。這個磁場並非來自某塊固態磁石，而是源自地球外核中高溫液態鐵鎳的對流運動——熾熱的導電流體在自轉與熱對流的共同作用下形成環流，如同一台巨型發電機持續產生磁場，這個機制稱為地球發電機（Geodynamo）。

地磁場的三個描述參數

在地表任一點，地磁場是一個三維向量，通常以三個參數描述：

- 磁場總強度（Total Intensity）：約在 25 至 65 微特斯拉（ μT ）之間，赤道附近較弱、高緯度較強。台灣約在 45 μT 上下。作為對照，一顆冰箱磁鐵的表面磁場強度可達數千微特斯拉——地磁其實非常微弱，這正是它容易被干擾的原因。
- 磁偏角（Declination）：磁北與真北的水平夾角。地圖與導航軟體使用的是真北，但羅盤指的是磁北，兩者的差值就是磁偏角。台灣地區的磁偏角約為西偏 4 度上下（隨地點與年份略有變化），也就是說羅盤指的「北」比真北偏西約 4 度。
- 磁傾角（Inclination）：磁力線與水平面的夾角。在磁赤道附近接近 0 度（磁力線平行地面），在磁極附近接近 90 度（磁力線垂直插入地面）。台灣約在 35 度左右——這個參數對機器人很重要，因為它意味著地磁向量有相當大的垂直分量，必須做傾斜補償才能正確算出航向。

磁場會漂移：世界地磁模型

由於外核流體的運動並不穩定，地磁場持續緩慢變化，磁北極更是不斷移動——近數十年來它從加拿大北部往西伯利亞方向漂移，且自 1990 年代末以來速度明顯加快。因此，精確導航必須依賴持續更新的模型。

美國 NOAA 國家環境資訊中心 (NCEI) 與英國地質調查所 (BGS) 共同維護的世界地磁模型 (WMM · World Magnetic Model) 就是這個標準。它每五年更新一次，現行版本 WMM2025 於 2024 年 12 月發布，效期至 2029 年底；同時另有高解析度版本 WMMHR2025。這個模型被北約、美國國防部、航空與航海系統，以及每一支手機的指南針 App 所使用——同學手機上顯示的「真北」，背後就是這個模型在做磁偏角修正。

模型也標示出兩極附近的「磁場失效區」 (Blackout Zone)：該處水平磁場強度過低，羅盤導航不再可靠。此外，太陽活動引發的磁暴會使磁偏角暫時偏離模型預測值——這對高緯度的航空作業影響尤其顯著。

磁力計的量測原理

- 霍爾效應 (Hall Effect)：導體中通以電流，若受到垂直方向的磁場，載子會偏向一側形成橫向電壓，電壓大小正比於磁場強度。原理直觀、成本低，但靈敏度相對較低。
- 磁阻效應 (Magnetoresistance)：某些材料的電阻會隨磁場改變。異向性磁阻 (AMR) 與穿隧磁阻 (TMR) 技術靈敏度高、功耗低，是目前手機與機器人常用的方案。

現代 MEMS 磁力計把三軸感測元件整合在一顆數公釐見方的晶片中，常與三軸加速度計、三軸陀螺儀封裝在一起，構成所謂的九軸 IMU 模組。要注意的是，磁力計輸出的原始資料是「感測器座標系中的三軸磁場分量」，距離「航向角」還有相當的處理距離——這正是下一節的主題。

6.2 電子羅盤與航向角

傳統羅盤的指標是一根可自由轉動的磁針，靠重力保持水平、靠磁力指向磁北。

電子羅盤沒有活動部件，必須用數學完成同樣的工作——而這裡藏著一個許多初學者會踩到的陷阱。

最簡單的情況：完全水平

若磁力計恰好保持水平，航向角只需用水平面上的兩個分量計算：

$$\psi = \text{atan2}(-m_y, m_x)$$

其中 m_x 、 m_y 是磁力計在水平面上的兩軸讀值， ψ 即磁航向角。使用 atan2 而非 atan 是為了正確判斷四個象限。

現實問題：機器人會傾斜

問題在於，機器人幾乎不可能永遠保持水平——小車經過斜坡、無人機前傾加速、機器狗上下樓梯，感測器都會跟著傾斜。由於台灣地區的磁傾角約 35 度，地磁向量有相當大的垂直分量；一旦感測器傾斜，這個垂直分量就會「洩漏」到水平兩軸上，造成航向角嚴重偏差。實測上，傾斜 10 度就可能造成數度到十餘度的航向誤差。

解法：傾斜補償

解法是先用加速度計量出重力方向，推算出滾轉角 φ 與俯仰角 θ ，再把磁力計讀值旋轉回水平面：

$$m_x' = m_x \cdot \cos \theta + m_z \cdot \sin \theta$$

$$m_y' = m_x \cdot \sin \varphi \cdot \sin \theta + m_y \cdot \cos \varphi - m_z \cdot \sin \varphi \cdot \cos \theta$$

$$\psi = \text{atan2}(-m_y', m_x')$$

這就是為什麼電子羅盤模組一定要包含加速度計——單靠磁力計無法在傾斜狀態下正確工作。這也解釋了一個生活現象：手機指南針 App 通常會提示你「請將手機保持水平」，因為傾斜補償雖能修正，但加速度計在載具運動時會同時量到運動加速度與重力，導致姿態估計失準，進而使航向角變得不穩。

從磁航向到真航向

最後一步是加上磁偏角修正：

$$\text{真航向} = \text{磁航向} + \text{磁偏角}$$

在台灣，磁偏角約為西偏 4 度（依慣例記為負值），因此真航向約等於磁航向減去 4 度。精確的磁偏角值可由 WMM 模型依經緯度與日期查詢——這也是 GNSS 與地磁感測合作的一個小例子：必須先知道自己大概在哪裡（GNSS），才能查到正確的磁偏角修正量。

6.3 室內地磁指紋定位

前兩節把「干擾」視為敵人——建築物的鋼筋會扭曲地磁，使羅盤失準。但如果換個角度想：既然每個位置的扭曲模式都不一樣，而且相當穩定，那麼這些扭曲本身不就是絕佳的位置特徵嗎？這個反轉思維，正是室內地磁定位的核心洞見。

原理：把 bug 變成 feature

現代建築的鋼筋、鋼樑、電梯井、大型機具都由鐵磁材料構成，它們會使周圍地磁場產生局部扭曲。這些扭曲有三個對定位極為有利的性質：

- 空間獨特性：不同位置的磁場向量（大小與方向）各不相同，形成類似指紋的特徵。
- 時間穩定性：建築結構不會移動，因此磁場分布數月甚至數年不變——遠比 Wi-Fi 訊號穩定（AP 會被更換位置、人潮會造成起伏）。
- 零基礎設施：不需要安裝任何信標或基站，只要建立資料庫即可。這是它相對 UWB、BLE 的最大優勢。

流程：與 Wi-Fi 指紋法一脈相承

流程與第 4 章的 Wi-Fi 指紋定位幾乎相同：離線階段沿走廊或格點蒐集各位置的

磁場特徵建庫；線上階段以即時量測比對資料庫求出最相似位置。

一個關鍵差異：資訊維度太低

但地磁指紋有一個先天弱點：Wi-Fi 指紋在一個點上可能量到十幾台 AP，形成十幾維的特徵向量，區別力很強；地磁指紋卻只有三個分量（甚至常只用磁場總強度這一維），區別力相對薄弱——走廊上相隔十公尺的兩個點，磁場強度完全相同並不罕見。

因此實務上的地磁定位很少採用「單點比對」，而是採用軌跡比對：讓使用者或機器人邊走邊記錄磁場序列，把這一整段「磁場曲線」與資料庫比對。一段曲線的獨特性遠高於單一數值，就像一段旋律比一個音符更容易辨識。常用方法包括粒子濾波（第 11 章）與動態時間校正（DTW）。

實務精度與限制

- 精度：搭配軌跡比對與行人航位推算（PDR），室內精度可達約 1 至 3 公尺，與 Wi-Fi 指紋法相當甚至更好。
- 建庫成本：與 Wi-Fi 指紋法相同，需要人工走訪蒐集，這是最大的部署障礙。
- 環境變動：大型金屬設備移位、建築改建會使局部資料庫失效；電梯運行、大型馬達啟停也會造成瞬時干擾。
- 磁暴影響：太陽活動雖會影響地磁，但室內定位比對的是「局部相對模式」而非絕對值，因此影響相對輕微。

6.4 鐵磁干擾與硬鐵 / 軟鐵校正

磁力計是最需要校正的感測器之一。原因很簡單：地磁只有約 $45 \mu\text{T}$ ，而機器人自己身上的馬達、電池、鋼製螺絲、金屬底盤所產生的磁場，往往遠大於此。若不校正，羅盤讀出來的幾乎是「機器人自己的磁場」而不是地球的。

理想情況：一顆完美的球

先建立一個關鍵的幾何直覺：把磁力計朝各個方向緩慢轉動一圈，蒐集所有三軸讀值並畫在三維空間中。由於地磁場的大小固定、只有方向改變，這些點理論上應該落在一個以原點為球心、半徑等於地磁強度的球面上。校正的全部工作，就是把實際量到的變形點雲「矯正回」這顆理想的球。

硬鐵誤差：球心偏移

硬鐵誤差來自載具上會產生固定磁場的物體——永久磁鐵、被磁化的鋼件、馬達。它們在感測器座標系中疊加一個固定的磁場向量，效果是把整顆球平移離開原點。

校正方法相對簡單：找出量測點雲在各軸的最大值與最小值，取中點即為偏移量：

$$b = (m_{max} + m_{min}) / 2$$

$$m_{corrected} = m_{raw} - b$$

軟鐵誤差：球變橢球

軟鐵誤差來自不會自己產生磁場、但會「扭曲」外部磁場的鐵磁材料——鐵板、鋼樑、機殼。它們讓磁力線在某些方向被聚集、某些方向被排開，效果是把球壓扁或拉長成橢球，甚至傾斜旋轉。

校正需要一個 3×3 的變換矩陣把橢球拉回正球：

$$m_{corrected} = A \cdot (m_{raw} - b)$$

矩陣 A 通常以橢球擬合 (Ellipsoid Fitting) 演算法從量測點雲估算。這也是為什麼完整的磁力計校正需要蒐集「各個方向」的資料——只在水平面轉圈，無法約束垂直方向的形變。

實務校正程序

26. 八字形校正：手持裝置在空中畫「8」字並同時翻轉，讓感測器涵蓋盡可能多的三維姿態。手機 App 提示「請揮動手機」時，做的就是這件事。
27. 在最終安裝狀態下校正：必須把磁力計裝到機器人上、接好所有線路、電池就位之後再校正——因為硬鐵誤差是「整台機器人」造成的，拆下來單獨校正毫無意義。
28. 遠離環境干擾源：校正時應遠離鋼筋牆面、電腦、大型金屬桌，否則會把環境的軟鐵效應誤當成載具本身的誤差記錄下來。
29. 驗證：校正後再轉一圈，檢查磁場總強度是否維持穩定（應接近當地地磁強度且變化小），這是判斷校正是否成功最簡單的指標。

動態干擾：校正也救不了的部分

還有一類干擾無法靠事前校正解決：馬達電流變化產生的磁場。機器人加速時電流增大，磁場隨之改變，這是與運動狀態相關的動態誤差。緩解方法包括把磁力計安裝在遠離馬達與電源線的位置（無人機常把羅盤裝在延伸支架或 GPS 天線頂端就是這個原因）、以電流值建立補償模型，或在濾波器中降低磁力計在高電流時段的權重。

6.5 地磁導航在手機、無人機、機器狗中的角色

手機

手機中的磁力計肩負三項任務：指南針 App 的方向指示、地圖 App 的「視角朝向」（讓地圖隨你轉身而旋轉）、以及 AR 應用的初始方位設定。此外，磁力計也在感測器融合中提供「不漂移的航向參考」——單靠陀螺儀，手機的方向估計會在數分鐘內明顯偏離。

無人機

無人機是最依賴、也最容易被羅盤問題困擾的平台。飛控系統需要航向資訊才能

正確解讀 GNSS 的位置變化並執行方向控制；羅盤一旦錯誤，會出現典型的「圓周飛行」(Toilet Bowl Effect) ——無人機以為自己在往前飛，實際上繞著圈子越飛越偏。

- 起飛前校正是標準程序，且必須在實際起飛地點附近進行。
- 切勿在鋼筋結構物、鐵製圍籬、地下停車場、大型金屬屋頂附近校正或起飛。
- 專業機種常配置雙羅盤互相驗證，並在偵測到讀值異常時自動改用 GNSS 航向（由運動方向推算）。

機器狗與地面機器人

四足機器人與輪型機器人的姿態變化劇烈、機身金屬件多、馬達功率大，磁力計的工作環境相當惡劣。因此在室內場域，多數系統會刻意降低磁力計的權重，甚至完全不用——航向改由光達或視覺 SLAM 的掃描匹配結果提供，因為在有結構的室內環境中，「牆的方向」比「地磁」可靠得多。

反過來說，在戶外開闊、少金屬的環境（農田、草原、大型戶外場域），地磁仍是最經濟可靠的航向來源。這再次印證本書反覆強調的觀念：沒有絕對優劣的感測器，只有適合與不適合的場合。

6.6 地磁、IMU、視覺 SLAM 的互補關係

把地磁放進整個定位系統來看，它的角色可以用一句話概括：唯一能提供「絕對航向」的低成本被動感測器。

感測器	提供資訊	誤差特性	主要弱點
陀螺儀	角速度（積分得角度變化）	短期極準，長期漂移	零偏漂移累積
加速度計	重力方向（滾轉、俯仰）	長期無漂移	運動加速度污染

感測器	提供資訊	誤差特性	主要弱點
磁力計	磁北方向 (航向)	長期無漂移	鐵磁干擾、需校正
視覺 / 光達 SLAM	相對位姿與環境結構	短期精確，長期漂移	依賴環境特徵
GNSS	絕對位置 (可推算航向)	誤差有界	室內失效、更新率低

表 6-1 各感測器在姿態與航向估測中的角色

三軸互補：AHRS 的邏輯

姿態與航向參考系統 (AHRS, Attitude and Heading Reference System) 正是把前三者組合起來的經典設計：

- 陀螺儀提供高頻率、平滑的角度變化，是姿態估計的骨幹。
- 加速度計以重力方向校正滾轉與俯仰的漂移——但它對航向 (繞垂直軸旋轉) 完全沒有約束力，因為繞著重力軸轉動，重力方向不會改變。
- 磁力計正好補上這個缺口：以磁北方向校正航向漂移。

這三者的分工可以這樣記憶：陀螺儀管「短期一切」，加速度計管「長期的上下」，磁力計管「長期的東西南北」。互補濾波器與擴展卡爾曼濾波 (第 7、11 章) 就是實現這個分工的數學工具。

加入 SLAM 之後

視覺或光達 SLAM 也能提供航向，而且在室內結構化環境中往往比磁力計可靠。因此現代室內機器人常見的策略是：

30. 系統啟動時，以磁力計提供粗略的初始航向，讓 SLAM 有一個合理的起點 (避免地圖建在完全隨機的方向上)。

31. 運行期間，以 SLAM 的掃描匹配結果為主要航向來源，磁力計權重調低或僅作為異常監測。
32. 進入戶外或無結構空曠區域（SLAM 失效風險高）時，重新提高磁力計權重。
33. 若磁力計讀值與 SLAM 估計持續大幅不一致，觸發警示——這往往意味著機器人附近有大型金屬物體，或 SLAM 已經跟丟。

這個「依情境調整信任度」的設計思路，與第 5 章圖 5-3 的 GNSS 可信度判斷完全一致。同學至此應該能感受到，第二篇各章看似在講不同的感測器，實則反覆在談同一件事：認識每個感測器的誤差特性，然後讓它們在對的時機貢獻對的資訊。

公式或流程

一、地磁向量的分量表示

地磁場向量 B 在當地水平座標系（北 N 、東 E 、下 D ）中可分解為：

$$B = (B_N, B_E, B_D)$$

由此可導出三個常用參數：

$$\text{水平分量 } H = \sqrt{B_N^2 + B_E^2}$$

$$\text{磁偏角 } D = \text{atan2}(B_E, B_N)$$

$$\text{磁傾角 } I = \text{atan2}(B_D, H)$$

$$\text{總強度 } F = \sqrt{B_N^2 + B_E^2 + B_D^2}$$

以台灣為例，總強度 F 約 $45 \mu\text{T}$ 、磁傾角 I 約 35° ，代入可得水平分量 $H \approx F \cdot \cos I \approx 37 \mu\text{T}$ 、垂直分量約 $26 \mu\text{T}$ 。垂直分量佔比可觀，這正是 6.2 節強調傾斜補償不可省略的量化理由。

二、磁力計校正模型

完整的校正模型同時處理硬鐵與軟鐵誤差：

$$m_{corrected} = A \cdot (m_{raw} - b)$$

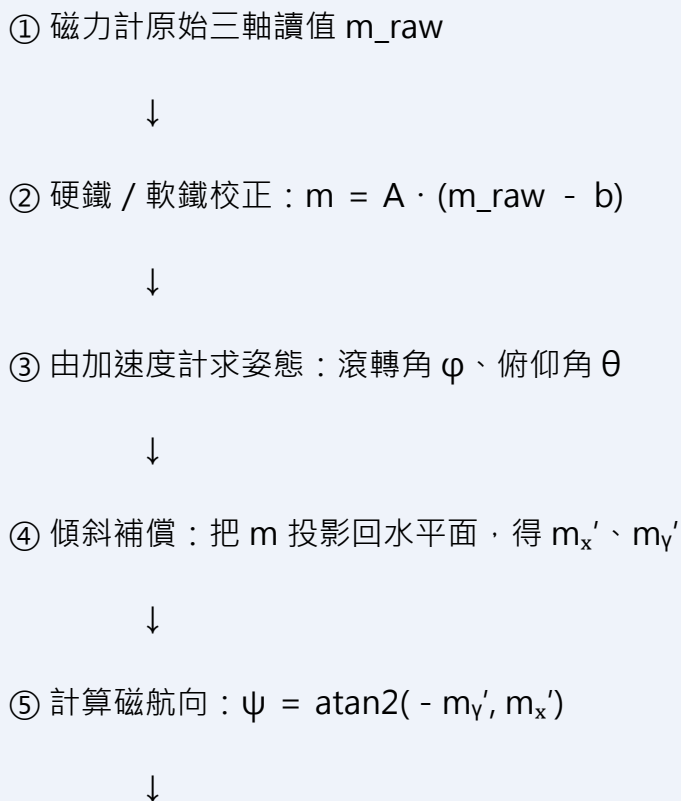
其中 b 為 3×1 的硬鐵偏移向量（球心平移）， A 為 3×3 的軟鐵校正矩陣（橢球還原為正球）。校正品質的驗證指標是：校正後在任意姿態下，磁場總強度應保持穩定：

$$\|m_{corrected}\| \approx F \text{ (當地地磁總強度, 恆定)}$$

實務上可計算一段旋轉資料中 $\|m\|$ 的標準差，數值越小表示校正越成功。這是一個同學在實驗中可以立即計算、立即判斷的實用指標。

三、電子羅盤完整處理流程

圖 6-1 電子羅盤從原始讀值到真航向的處理流程



⑥ 加磁偏角修正 (由 WMM 依經緯度查得) → 真航向

↓

⑦ 與陀螺儀融合 (互補濾波 / EKF) → 平滑且不漂移的航向輸出

案例

案例 6-1 無人機的「馬桶效應」：羅盤錯誤的教科書案例

無人機飛手圈中流傳一個生動的名詞：Toilet Bowl Effect (馬桶效應) ——無人機在定點懸停模式下，不但停不住，反而像水沖進馬桶一樣繞著圈子越飛越大。這幾乎必然是羅盤問題。

原因可以這樣理解：飛控以為機頭朝北，實際上朝東北。當 GNSS 顯示飛機往東漂移，飛控便下令「往西修正」——但它認知中的西，實際上是西北。修正動作因此永遠帶著一個固定的角度偏差，位置誤差不但沒被消除，反而被推著沿切線方向旋轉，形成圓周軌跡。

常見肇因包括：在鋼筋陽台或金屬車頂上校正、起飛點下方有鋼筋或地下管線、機上加裝了含磁性的配件 (某些相機雲台或喇叭含永磁)。這個案例的教學價值在於：它清楚展示了「航向誤差」與「位置誤差」如何耦合——航向錯了，所有基於位置的控制決策都會系統性地偏向同一側。這個觀念在第 10 章的座標轉換與第 28 章的導航實作中會再次出現。

案例 6-2 從干擾到資訊：室內地磁定位的商業化嘗試

2010 年代中期，芬蘭的 IndoorAtlas 等公司提出以地磁指紋做室內定位的商業方案，一度被視為「不需要任何硬體」的理想解——不必安裝信標、不必依賴 Wi-Fi，

只要手機內建的磁力計即可。這個構想在購物中心、機場、展館的導覽應用中確實展示了公尺級的定位能力。

但商業化過程也暴露了現實限制：指紋庫必須人工走訪建立，一座大型商場需要數十小時的勘測工作；商場改裝櫃位、更換大型金屬展示架後，局部資料庫就得重測；且單靠地磁的區別力不足，實務上仍須融合 Wi-Fi、藍牙與手機 PDR 才能穩定運作。

這個案例給同學兩個啟示：第一，「零硬體成本」不等於「零部署成本」，資料蒐集與維護往往才是真正的門檻——這與第 4 章 Wi-Fi 指紋定位的結論完全一致；第二，在真實產品中，很少有單一技術能獨力撐起需求，融合幾乎是必然的答案。

案例 6-3 教室裡的磁場地圖：把鋼筋畫出來

一個在大學與高中課堂都極受歡迎的示範實驗：讓同學拿著手機的磁力計 App，緩慢走過教室的地板與牆邊，觀察磁場強度讀值的變化。

結果往往令同學印象深刻：在教室中央的空曠處，讀值穩定在約 45 μT 附近；一走近柱子或承重牆，數值可能瞬間跳到 60 μT 以上；把手機貼近電腦桌的金屬腳架或黑板的鐵框，讀值甚至可能翻倍或急遽下降。若把測得的數值依位置畫成色階圖，就會浮現出一張「隱形的鋼筋分布圖」——建築的骨架被磁場顯影了出來。

這個實驗一次達成三個教學目的：讓同學直觀感受地磁的微弱與易受干擾（6.1 節）、理解為何室內羅盤不可信（6.4 節），同時也親眼見證這些「干擾」的空間穩定性與獨特性，正是地磁指紋定位的基礎（6.3 節）。同一組資料，同時說明了「為什麼不能用」與「為什麼可以用」——這正是本章最核心的觀念轉折。

實作活動

實作 6-1 磁力計校正與教室地磁指紋地圖

目的：完成一次完整的磁力計校正，並實測教室的地磁分布，同時體會地磁作為「航向感測器」與「位置指紋」的雙重身分。本實作以手機即可進行，大學部可延伸至 Arduino / ESP32 硬體。

器材

- 智慧型手機，安裝可顯示三軸磁力計原始數值的感測器 App (可搜尋「magnetometer」「sensor logger」類工具)。
- 教室平面圖、記錄表、色鉛筆或試算表軟體。
- (大學部選用) ESP32 或 Arduino 搭配 HMC5883L / QMC5883L / MPU-9250 等九軸模組。

第一部分：校正與航向驗證

24. 未校正基準測試：在教室中央手持手機，緩慢原地旋轉一整圈，記錄磁場總強度 $\|m\|$ 的最大值與最小值。理論上旋轉時總強度應保持不變，實測的變化幅度即反映未校正誤差的大小。
25. 八字形校正：依 App 或系統指示，手持手機在空中畫「8」字並翻轉各個角度約 20 秒，讓感測器涵蓋盡可能多的姿態。
26. 校正後驗證：重複步驟 1，比較校正前後總強度的變化幅度 (可計算標準差)，驗證校正成效。
27. 傾斜實驗：把手機水平朝向教室黑板，記錄航向角；再把手機前傾約 30 度、維持同一朝向，觀察航向讀值變化了多少。討論這與 6.2 節傾斜補償的關係。
28. 干擾實驗：把手機分別靠近筆電、金屬椅腳、消防栓、電源插座，記錄磁場總強度與航向角的變化幅度。

第二部分：地磁指紋地圖

29. 在教室地面以 1.5 至 2 公尺間隔標出格點（約 15 至 20 個點），並在平面圖上編號。
30. 在每個格點以固定姿態（手機水平、面向同一方向）停留 5 秒，記錄磁場總強度與三軸分量，每點量測三次取平均。
31. 繪製色階圖：以磁場總強度為色階值（低為藍、高為紅）填入平面圖，觀察是否浮現柱子、承重牆或大型金屬設備的輪廓。
32. 盲測驗證：由一位同學閉眼被帶至某格點，量測磁場後由其他同學比對資料庫推測位置，重複 6 次並統計正確率與平均誤差。
33. 軌跡比對進階：讓一位同學沿教室邊緣走一圈並連續記錄磁場序列，觀察這條「磁場曲線」是否比單點數值更容易辨識出走的是哪一段路線。

討論問題

27. 校正前後，旋轉一圈時磁場總強度的變化幅度各是多少？這個差異說明了什麼？
28. 傾斜 30 度造成的航向誤差有多大？如果一台機器人爬上 15 度的斜坡而沒有做傾斜補償，導航會出現什麼問題？
29. 你們的盲測正確率如何？失敗的案例多半發生在哪些格點？這印證了 6.3 節提到的哪一項限制？
30. 比較單點比對與軌跡比對的辨識效果，說明為何實務系統偏好後者。
31. （大學部）根據你們的教室磁場地圖，判斷這間教室適合用磁力計提供機器人航向嗎？若不適合，你會改用什麼方法（提示：參考 6.6 節）？

延伸挑戰（大學部）

以 ESP32 搭配九軸模組，撰寫程式完成下列功能：讀取三軸磁力計與加速度計原始值、實作硬鐵校正（最大最小值法）與橢球擬合軟鐵校正、實作傾斜補償並輸出真航向角

(含台灣磁偏角修正) 、最後以互補濾波器融合陀螺儀資料，比較融合前後航向輸出的平滑度與長期穩定性。這份程式可直接作為第 7 章 IMU 姿態估測的基礎。

延伸閱讀

34. NOAA NCEI, World Magnetic Model 2025 (WMM2025) 官方網站與技術報告. (含線上磁偏角計算工具，可查詢台灣各地數值)
35. British Geological Survey, Geomagnetism 教學資源 (geomag.bgs.ac.uk). (地磁基礎知識與模型說明)
36. M. Kok, J. D. Hol, and T. B. Schön, "Using Inertial Sensors for Position and Orientation Estimation," Foundations and Trends in Signal Processing, 2017. (慣性與磁感測融合的完整教程，可免費取得)
37. B. Li et al., "How Feasible Is the Use of Magnetic Field Alone for Indoor Positioning?," IPIN, 2012. (地磁室內定位可行性的代表性研究)
38. J. Haverinen and A. Kemppainen, "Global Indoor Self-localization Based on the Ambient Magnetic Field," Robotics and Autonomous Systems, 2009. (地磁指紋定位的早期經典論文)
39. J. F. Vasconcelos et al., "Geometric Approach to Strapdown Magnetometer Calibration in Sensor Frame," IEEE Transactions on Aerospace and Electronic Systems, 2011. (磁力計校正的嚴謹數學處理)
40. PX4 / ArduPilot 開源飛控之 Compass Calibration 官方文件. (無人機羅盤校正的實務指引與常見問題排除)

小結

21. 地磁場源自地球外核液態金屬對流，強度僅約 25 至 65 μT ，遠弱於日常金屬物體產生的磁場，因此極易受干擾；以磁偏角、磁傾角與總強度三個參數描述。
22. 磁北極持續朝西伯利亞方向漂移，精確導航須依賴世界地磁模型（WMM）；現行版本 WMM2025 於 2024 年 12 月發布，效期至 2029 年底，並另有高解析度版本 WMMHR2025。
23. 電子羅盤必須結合加速度計進行傾斜補償——台灣磁傾角約 35 度，地磁垂直分量可觀，未補償的傾斜會造成顯著航向誤差；最後再加磁偏角修正才得真航向。
24. 室內鋼構造成的磁場扭曲，在導航上是干擾，在定位上卻是穩定且獨特的位置指紋；因單點資訊維度低，實務多採軌跡比對配合粒子濾波，精度約 1 至 3 公尺。
25. 磁力計校正的幾何本質是把變形的量測點雲還原成正球：硬鐵誤差造成球心平移（以偏移向量 $\mathbf{b}$ 修正），軟鐵誤差造成球變橢球（以矩陣 $\mathbf{A}$ 修正）；校正須在最終安裝狀態下、遠離環境干擾處進行。
26. 無人機最依賴羅盤也最易受其害，羅盤錯誤會導致「馬桶效應」；室內機器人則常降低磁力計權重，改以 SLAM 掃描匹配提供航向。
27. 在 AHRS 架構中，陀螺儀管短期一切、加速度計管長期的上下、磁力計管長期的東西南北；加入 SLAM 後，磁力計常退居初始化與異常監測的角色。

下一章，我們深入第二篇最核心的感測器——IMU 慣性導航。加速度計與陀螺儀如何量測運動？為什麼積分會產生漂移？姿態濾波、IMU 預積分與視覺慣性緊耦合又是什麼？這些將是後續視覺 SLAM 與無人機導航章節的重要基礎。

台灣自編大學教科書

@prof

《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第 7 章

IMU 慣性導航

編者：李世淵 (機器人叫獸)

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

學習目標

IMU 是機器人身上最忙碌的感測器——它以每秒數百次的頻率不斷回報運動狀態，卻也是最容易「說謊」的一個。本章深入慣性導航的原理、限制與現代解法，這些內容將直接支撐第六篇的視覺慣性 SLAM 與第七篇的無人機導航。學完本章之後，同學應能夠達成下列目標：

36. 說明加速度計與陀螺儀的 MEMS 量測原理，並解釋加速度計為何在靜止時仍讀到約 9.8 的數值。
37. 定義滾轉、俯仰、偏航三個姿態角，並由加速度計計算滾轉與俯仰角。
38. 解釋積分漂移的成因，並說明位置誤差為何隨時間以平方甚至三次方成長。
39. 說明互補濾波器與卡爾曼濾波器的基本思想，並比較兩者的適用場合。
40. 說明 IMU 預積分要解決什麼問題，以及它為何是現代視覺慣性 SLAM 的關鍵技術。
41. 說明 IMU 在視覺慣性 SLAM 中的四大貢獻，並解釋鬆耦合與緊耦合的差異。
42. 說明 ORB-SLAM3 如何以最大後驗估測完成 IMU 初始化，以及初始化需要「運動激勵」的原因。

核心概念

中文名詞	英文名詞	概念說明
慣性量測單元	IMU	整合加速度計與陀螺儀（有時含磁力計）的感測器模組。
比力	Specific Force	加速度計實際量到的量：運動加速度減去重力加速度。

中文名詞	英文名詞	概念說明
科氏力	Coriolis Force	MEMS 陀螺儀藉以量測角速度的物理效應。
姿態角	Roll / Pitch / Yaw	滾轉、俯仰、偏航，描述剛體在三維空間中的朝向。
零偏	Bias	感測器在零輸入時仍有的固定輸出誤差，會隨溫度與時間變化。
隨機游走	Random Walk	白雜訊經積分後產生的隨機漂移，誤差隨時間平方根成長。
積分漂移	Integration Drift	微小誤差經積分不斷累積，使估測值逐漸偏離真值。
互補濾波器	Complementary Filter	以高通與低通互補的方式融合陀螺儀與加速度計。
卡爾曼濾波器	Kalman Filter	在高斯雜訊假設下的最佳線性遞迴估測器。
擴展卡爾曼濾波	EKF	將非線性系統線性化後套用卡爾曼濾波的方法。
IMU 預積分	IMU Preintegration	把兩關鍵幀之間的 IMU 量測預先積分為單一約束，避免重複計算。
緊耦合	Tightly Coupled	將 IMU 原始量測與視覺量測放進同一個最佳化問題求解。
最大後驗估測	MAP	在已知觀測下尋找機率最大的狀態估計，ORB-SLAM3 初始化的理論基礎。
尺度不確定性	Scale Ambiguity	單目相機無法確定真實尺寸，需由 IMU 或已知物體提供尺度。

7.1 加速度計、陀螺儀、磁力計

IMU (Inertial Measurement Unit，慣性量測單元) 的核心是兩種感測器：加速度計量測線加速度、陀螺儀量測角速度。第 6 章介紹的磁力計常一併封裝，構成所謂的「九軸」模組。這三者的分工，正是第 6.6 節提過的：陀螺儀管短期一切、加速度計管長期的上下、磁力計管長期的東西南北。

一、加速度計：一顆彈簧上的質量塊

MEMS 加速度計的原理可以用一個極簡模型理解：一個微小的質量塊 (Proof Mass) 以彈性樑懸掛在晶片中。當晶片加速時，質量塊因慣性而相對位移，這個位移改變了電容極板間距，由電路量出電容變化即可換算加速度。整個結構刻在數平方公釐的矽晶片上，成本可低至數十元台幣。

這裡有一個關鍵觀念，初學者幾乎必定會困惑：把加速度計靜置在桌上，它讀到的不是 0，而是垂直方向約 9.8 m/s^2 。為什麼靜止的東西會有加速度？

觀念釐清：加速度計量的是「比力」而非「加速度」

加速度計實際量測的物理量是比力 (Specific Force)，定義為：

$$a_{\text{measured}} = a_{\text{true}} - g$$

其中 a_{true} 是載體相對慣性系的真實加速度、 g 是重力加速度向量。

直觀理解：加速度計量的是「質量塊被彈簧推了多用力」。靜止時，彈簧必須施力抵抗重力把質量塊撐住，因此讀到約 9.8；

而自由落體時，質量塊與外殼一起下墜、彈簧完全不受力，加速度計反而讀到 0——

這正是太空站「失重」的物理本質。

結論：使用加速度計前，必須先知道重力方向並扣除它，才能得到真正的運動加速度。而重力方向又必須由姿態決定——

這個「姿態與加速度互相依賴」的循環，正是慣性導航困難的根源之一。

二、陀螺儀：振動與科氏力

MEMS 陀螺儀量測的是角速度。它的原理利用科氏力 (Coriolis Force)：晶片內的質量塊被驅動做高頻往復振動；當整體發生旋轉時，振動中的質量塊會受到垂直於振動方向的科氏力作用，產生可偵測的側向位移，位移量正比於角速度。

要特別注意：陀螺儀輸出的是角速度（每秒轉幾度），不是角度。要得到角度，必須對時間積分——而這正是漂移問題的來源（7.3 節）。

三、等級與規格

IMU 依精度分為數個等級，價格可以差上數千倍：

等級	典型零偏穩定度	價格級距	典型應用
消費級 MEMS	每小時數十度	數十至數百元	手機、玩具、教學模組
工業級 MEMS	每小時數度	數千至數萬元	無人機、機器人、AGV
戰術級	每小時 0.1 至 1 度	數十萬元	導引、測繪、自駕車
導航級	每小時 0.001 度以下	數百萬元以上	航空、潛艦、太空

表 7-1 IMU 精度等級與應用

本書教學實驗使用的 MPU-6050、MPU-9250、ICM-20948、BNO055 等模組

屬於消費級到工業級之間；無人機飛控（Pixhawk 系列）則多採工業級並常配置多顆互相驗證。理解等級差異很重要：同一套演算法在戰術級 IMU 上可以獨立推算數分鐘，在消費級模組上可能數秒就失準——這決定了系統設計時能對 IMU 寄予多大的信任。

7.2 姿態估測：Roll、Pitch、Yaw

姿態（Attitude）描述剛體在空間中的朝向。航空領域慣用的歐拉角表示法有三個角度，同學可以用飛機來記憶：

- 滾轉角（Roll， φ ）：繞機身前後軸旋轉——飛機左右翼上下擺動，機身「側傾」。
- 俯仰角（Pitch， θ ）：繞機身左右軸旋轉——機頭上仰或下俯。
- 偏航角（Yaw， ψ ）：繞垂直軸旋轉——機頭左右轉向，也就是第 6 章談的航向角。

由加速度計求滾轉與俯仰

在載體靜止或等速運動時，加速度計量到的就是重力在三個軸上的分量。既然知道重力必定指向地心（正下方），比對三軸讀值就能反推出感測器傾斜了多少：

$$\varphi = \text{atan2}(a_y, a_z)$$

$$\theta = \text{atan2}(-a_x, \sqrt{a_y^2 + a_z^2})$$

關鍵限制：加速度計管不到偏航角

這裡有一個必須牢記的限制：加速度計無法量測偏航角。原因很直觀——把裝置水平放在桌上原地旋轉，重力方向始終不變，三軸讀值也就不變，加速度計對此完全「無感」。這正是第 6 章磁力計不可或缺的原因，也是純視覺 SLAM 需要其他資訊提供初始航向的原因。

第二個限制：加速度污染

上述公式有一個前提：加速度計量到的「只有重力」。但機器人一旦加速、轉彎、震動，讀值就是重力與運動加速度的疊加，此時算出的姿態角會嚴重失真。一台急加速的小車，用這個公式會誤判成「車頭上仰」——因為向前的加速度在感測器看來與「傾斜後重力的分量」無法區分。

這個限制帶來一個重要結論：加速度計適合提供長期平均的姿態參考，但不能用於瞬時姿態估測。而陀螺儀恰好相反——短期精確、長期漂移。兩者的互補性，就是 7.4 節姿態濾波的全部動機。

歐拉角的萬向鎖問題

歐拉角直觀好懂，但有一個著名缺陷：當俯仰角接近 $\pm 90^\circ$ （機頭垂直朝上或朝下）時，滾轉與偏航兩個旋轉軸會重合，喪失一個自由度，稱為萬向鎖（Gimbal Lock）。這對無人機特技飛行或機械手臂是致命的。因此實務系統內部多改用四元數（Quaternion）表示姿態——它沒有奇異點、運算效率高，只是不易直觀理解。四元數的完整介紹留待第 10 章，本章先以歐拉角建立直覺即可。

7.3 積分漂移問題

慣性導航的美好願景是：只要有 IMU，機器人就能完全自給自足地知道自己的位姿——不需要衛星、不需要地標、不需要任何外部訊號。這個願景在理論上完全成立，在實務上卻幾乎不可能單獨實現。原因只有一個字：積分。

積分的雙重放大

從 IMU 讀值推算位姿，需要做積分：

$$\text{角速度 } \omega \text{ —— (積分一次) } \rightarrow \text{姿態角}$$

加速度 a —— (積分一次) $\rightarrow$ 速度 —— (再積分一次) $\rightarrow$ 位置

積分是一個「只進不出」的運算：每一刻的微小誤差都會被永久累加進結果，而且永遠不會自己抵消。更糟的是位置需要積分兩次，誤差被放大兩輪。

誤差成長的數學

假設加速度計有一個固定的零偏誤差 b (例如 0.01 m/s^2 ，這在消費級模組上很常見)，則經過時間 t 後：

$$\text{速度誤差} = b \cdot t$$

$$\text{位置誤差} = (1/2) \cdot b \cdot t^2$$

代入數字感受一下： $b = 0.01 \text{ m/s}^2$ 時，10 秒後位置誤差約 0.5 公尺，60 秒後約 18 公尺，300 秒後高達 450 公尺。這還只是加速度計的零偏；陀螺儀的零偏會造成姿態誤差隨時間線性成長，而姿態一旦錯了，重力就扣不乾淨，殘餘的重力分量又會被當成運動加速度積分兩次——這條路徑造成的位置誤差甚至隨 t^3 成長，是實務上最主要的誤差來源。

直觀理解：姿態誤差如何毒害位置估測

假設姿態估計只偏了 1 度。此時系統認為的「垂直方向」與真正的垂直方向差 1 度，

扣除重力後會殘留一個水平分量： $9.8 \times \sin(1^\circ) \approx 0.17 \text{ m/s}^2$ 。

這個「不存在的加速度」經過兩次積分，10 秒後就造成約 8.5 公尺的位置誤差。

結論：在慣性導航中，姿態精度比加速度精度更關鍵。這正是 7.4 節姿態濾波如此重

要的原因，

也是為什麼所有實用系統都必須引入外部資訊（磁力計、GNSS、視覺）來約束姿態。

誤差的三種來源

- 零偏（Bias）：零輸入時的固定輸出偏移。它並非常數，會隨溫度變化（開機暖機期間尤其明顯）與時間緩慢遊走。這是最主要的誤差源，也是各種濾波器與最佳化演算法都要「線上估計」的對象。
- 雜訊（Noise）：隨機的高頻擾動。積分後形成隨機游走，誤差隨時間的平方根成長——相較零偏的線性或平方成長，這一項反而不是主要問題。
- 尺度因子與軸不正交誤差：實際靈敏度與標稱值的偏差、三軸未完全垂直。可透過事前校正大幅減輕。

務實的結論

IMU 的正確用法不是「拿來獨立導航」，而是「拿來填補其他感測器的空隙」。消費級 IMU 的可用推算時間約在數秒等級，工業級可達數十秒，戰術級以上才能撐到分鐘等級。因此系統設計的核心問題永遠是：在 IMU 漂移到不可接受之前，下一次外部校正會在什麼時候到來？無論那個校正來自 GNSS（第 5 章）、光達掃描匹配（第 13 章）還是視覺特徵（第 19 章），道理都是一樣的。

7.4 姿態濾波：Complementary Filter、Kalman Filter、EKF

既然陀螺儀短期準、長期飄，加速度計長期準、短期受污染，答案已經呼之欲出：把兩者融合，各取所長。這就是姿態濾波要做的事。

一、互補濾波器：最簡單而有效

互補濾波器的想法極為直觀：對陀螺儀的積分結果取「高通」（相信它的快速變化）、對加速度計算出的角度取「低通」（相信它的長期平均），兩者加權相加：

$$\theta = \alpha \cdot (\theta_{prev} + \omega \cdot \Delta t) + (1 - \alpha) \cdot \theta_{accel}$$

其中 α 通常取 0.95 至 0.99 之間。 α 越大表示越信任陀螺儀，輸出越平滑但抗漂移能力較弱； α 越小則越快跟隨加速度計，但也越容易被震動干擾。

- 優點：只需幾行程式碼、幾乎不佔運算資源、參數只有一個且物理意義清晰，非常適合 Arduino 等級的微控制器與教學實驗。
- 限制：沒有誤差的統計模型，無法估計零偏，也無法提供「估計有多不確定」的資訊。

二、卡爾曼濾波器：最佳線性估測

卡爾曼濾波器把問題提升到機率的層次。它不只估計「姿態是多少」，還同時追蹤「這個估計有多不確定」（以共變異數矩陣表示），並在每一步做兩件事：

34. 預測（Predict）：用陀螺儀讀值推算下一刻的姿態，同時讓不確定性增大——因為積分會累積誤差。
35. 更新（Update）：用加速度計的觀測修正預測值。修正的力道由「卡爾曼增益」決定：若預測的不確定性大、觀測的雜訊小，就多相信觀測；反之則多相信預測。

這個「預測—更新」的循環，正是第 1 章 SLAM 標準流程的縮影，也是第 11 章要深入展開的核心數學。卡爾曼濾波器的一大優勢是可以把零偏納入狀態向量一起估計——系統會自動學習「這顆陀螺儀今天偏了多少」，這是互補濾波器做不到的。

三、擴展卡爾曼濾波（EKF）：處理非線性

標準卡爾曼濾波要求系統是線性的，但姿態運動學充滿三角函數與四元數乘法，本質上是非線性的。EKF 的解法是在當前估計點附近做一階泰勒展開（求雅可比矩陣）將系統線性化，再套用卡爾曼濾波的公式。

EKF 是機器人領域最廣泛使用的姿態估測方法：無人機飛控（PX4、ArduPilot 的 EKF2 / EKF3）、ROS 的 robot_localization 套件、手機的感測器融合，背後幾乎都是 EKF。它的限制是：線性化在誤差很大時可能失準，且需要合理的初始估計才能收斂。

方法	運算需求	可估零偏	提供不確定度	適用場合
互補濾波器	極低	否	否	教學、微控制器、簡易平台
卡爾曼濾波器	中	是	是	線性系統、簡化姿態模型
EKF	中高	是	是	無人機飛控、機器人主流方案
Mahony / Madgwick	低	部分	否	開源飛控、AHRS 模組常用

表 7-2 常見姿態濾波方法比較

補充說明：Mahony 與 Madgwick 濾波器是介於互補濾波與卡爾曼之間的實用方案，以四元數表示姿態並採用梯度下降或比例積分修正，運算量小卻表現不俗，在開源飛控與 AHRS 模組中極為常見，適合作為實作進階的第一站。

7.5 IMU 預積分概念

本節進入現代視覺慣性 SLAM 的關鍵技術。內容較為進階，高中課程可只掌握「要解決什麼問題」的直覺，大學部則應理解其設計邏輯。

問題的起點：頻率不匹配

視覺 SLAM 的相機大約每秒 20 至 30 幀，IMU 卻高達每秒 200 至 1000 次。這意味著相鄰兩張影像之間，夾著數十到上百筆 IMU 量測。如何把這一大堆高頻資料，變成後端最佳化能用的「一條約束」？

樸素做法的致命問題

最直覺的做法是：把這段時間內的 IMU 量測直接積分，得到「從第 i 幀到第 j 幀的位姿變化」，當作一條約束加入最佳化。

但這裡有一個嚴重問題。積分加速度時，必須先扣除重力，而扣除重力需要知道姿態；姿態又與第 i 幀的位姿估計有關。最佳化過程中，第 i 幀的位姿估計會被反覆更新——每更新一次，這段 IMU 積分就必須全部重算一遍。以上百筆量測、數十次迭代計算，運算量將完全無法即時處理。

預積分的核心洞見

預積分 (Preintegration) 的巧妙之處在於：把積分結果表示成「與初始位姿無關」的形式。

具體做法是，不積分「世界座標系中的位姿變化」，而是積分「第 i 幀機體座標系中的相對運動量」。這些相對量只依賴 IMU 的原始讀值與零偏估計，與第 i 幀在世界中的位置、朝向完全無關。於是最佳化過程中無論第 i 幀的位姿如何被更新，預積分結果都不必重算——只需在最外層做一次簡單的座標變換即可。

圖 7-1 預積分解決的問題

【樸素做法】

第 i 幀位姿更新 $\rightarrow$ 重力方向改變 $\rightarrow$ 整段 IMU 必須重新積分 $\rightarrow$ 運算量爆炸

【預積分做法】

預先積分「相對運動量」（與初始位姿無關） $\rightarrow$ 只算一次

↓

最佳化時，第 i 幀位姿如何變動，都只需外層做一次座標變換

↓

運算量大幅下降，得以即時執行

預積分產出什麼

兩個關鍵幀之間的預積分，會產出三個相對量與一個不確定度：

- 相對旋轉 ΔR ：這段時間機體轉了多少。
- 相對速度變化 Δv ：速度改變了多少。
- 相對位置變化 Δp ：在初始機體座標系中移動了多少。
- 共變異數矩陣：這條約束有多可信，用來決定它在最佳化中的權重。

零偏的處理

還有一個細節：積分時必須先扣掉零偏，但零偏本身也是待估計的未知數。若最佳化過程中零偏估計改變了，預積分不就又要重算？預積分理論的解法是同時計算「預積分結果對零偏的一階偏微分（雅可比）」，零偏微幅改變時只需用線性修正快速更新，不必重新積分。

預積分理論由 Forster 等人於 2015 年提出並嚴謹處理了旋轉群的流形結構，如今

已成為所有主流視覺慣性系統的標準元件——VINS-Mono、OKVIS、ORB-SLAM3 無一例外。這也呼應第 2 章 MIT VNAV 課程為何要花大量篇幅講「流形上的最佳化」：旋轉不是普通的向量，處理它需要專門的數學工具。

7.6 視覺慣性 SLAM 中的 IMU 角色

為什麼視覺 SLAM 一定要加上 IMU？答案不是「多一個感測器多一份保險」這麼簡單——IMU 補上的是純視覺方法在原理上就辦不到的幾件事。

IMU 的四大貢獻

36. 提供絕對尺度：單目相機有先天的尺度不確定性——同一段影像序列，可能是「大房間中的大運動」，也可能是「模型屋中的小運動」，純視覺無法分辨。IMU 量測的是有物理單位的加速度，積分後即可提供真實尺度。這是單目視覺慣性系統最重要的價值。
37. 提供重力方向：加速度計長期平均指向重力，使系統能確定「哪邊是下」。這讓滾轉與俯仰角成為可觀測量，地圖也能被擺正——對無人機控制與 AR 應用至關重要。
38. 填補視覺失效的空隙：快速轉動造成影像模糊、鏡頭被短暫遮擋、進入白牆或黑暗區域時，視覺前端會跟丟。IMU 在數秒內仍能維持可用的推算，讓系統撐過這段空白並重新接上——這大幅提升了系統的強韌性。
39. 提供高頻輸出：相機每秒 30 幀對飛行控制而言太慢。IMU 讓系統能以數百 Hz 輸出位姿，滿足即時控制迴路的需求。

視覺對 IMU 的回報

這個合作是雙向的。視覺提供的是 IMU 最缺的東西：不漂移的位置與姿態觀測。透過視覺約束，系統能持續估計並補償 IMU 的零偏，讓 7.3 節的漂移噩夢受到控制。

兩者的關係可以這樣總結：IMU 給視覺尺度與重力，視覺給 IMU 校正與約束。

鬆耦合與緊耦合

- 鬆耦合 (Loosely Coupled)：視覺系統與慣性系統各自獨立計算出位姿，再把兩個結果加權融合。優點是模組化、易實作、可直接沿用現成的視覺 SLAM；缺點是資訊在各自處理階段就已經被壓縮，融合時無法再回頭利用原始量測，精度受限。
- 緊耦合 (Tightly Coupled)：把視覺的重投影誤差與 IMU 的預積分約束放進同一個最佳化問題，一起求解位姿、速度、零偏與地圖點。優點是所有資訊在同一個機率架構下被一致處理，精度顯著較佳；缺點是實作複雜、運算量大。

現代高精度系統一律採用緊耦合。這也再次呼應第 4 章 4.7 節與第 5 章的融合討論——同學應該已經發現，「緊耦合優於鬆耦合」是貫穿整本書的一個普遍原則：越晚壓縮資訊，越能求得最佳解。

7.7 ORB-SLAM3 與 IMU 緊耦合

ORB-SLAM3 是目前最完整的開源視覺慣性 SLAM 系統，由西班牙薩拉戈薩大學團隊開發。它是第一套能同時支援純視覺、視覺慣性與多地圖 SLAM，並相容單目、雙目、RGB-D 相機以及針孔與魚眼鏡頭模型的系統。本節先建立整體概念，完整的系統剖析留待第 22 章。

核心創新：全程以最大後驗估測為基礎

ORB-SLAM3 最重要的貢獻，是一套完全建立在最大後驗估測 (MAP , Maximum-a-Posteriori) 之上的緊耦合視覺慣性系統——連 IMU 初始化階段也不例外。這使得系統能在室內外、大小場景中穩健即時運作，精度較先前方法提升約二至十倍。

為什麼 IMU 初始化這麼困難

系統剛啟動時，有一堆未知數必須同時確定：地圖的真實尺度、重力方向、加速度計與陀螺儀的零偏、以及初始速度。這些量彼此耦合，而且不是全部都能立刻觀測到。

早期系統（如 ORB-SLAM-VI）的初始化程序需要約 15 秒才能完成，這段期間系統精度不佳，嚴重影響穩健性與使用體驗。ORB-SLAM3 把初始化重新表述為一個最佳化問題——在 MAP 的框架下，同時求解尺度、重力方向、零偏與速度的最可能值，使初始化大幅加速且更為可靠。

運動激勵：初始化的必要條件

這裡有一個實作上必須知道的關鍵：IMU 初始化需要足夠的運動激勵（Motion Excitation）。原因在於，如果載體只是等速直線移動或完全靜止，加速度計讀到的幾乎只有重力，系統無法把「尺度」與「零偏」區分開來——數學上稱這些量不可觀測（Unobservable）。

因此實務操作時，啟動視覺慣性 SLAM 後應先做一段帶有加減速與旋轉的運動（拿著相機小幅度前後左右移動並轉動數秒），系統才能順利完成初始化。這是同學在第 29 章實作 ORB-SLAM3 時最常遇到的問題之一：「為什麼一直初始化不成功？」答案往往是動得不夠。

三執行緒架構中的 IMU

- 追蹤執行緒（Tracking）：對每一幀提取 ORB 特徵，最小化重投影誤差與慣性誤差以估計相機位姿；在視覺慣性模式下，還會一併估計速度與 IMU 零偏。

- 局部建圖執行緒 (Local Mapping)：插入關鍵幀、剔除冗餘資訊，並執行局部 Bundle Adjustment——在慣性模式下即為視覺慣性 BA，同時以 MAP 估測完成並持續精修 IMU 參數。
- 迴路與地圖合併執行緒：偵測閉環與地圖重合，執行全域修正。

多地圖系統：跟丟不再是災難

ORB-SLAM3 的第二項創新是多地圖架構 (Atlas)。當系統因長時間視覺資訊不良而跟丟時，它不會崩潰，而是開啟一張新地圖繼續運作；日後重新回到曾經走過的區域時，再把新舊地圖無縫合併。這對長時間運行的機器人極為實用——第 22 章會完整討論這個機制。

精度表現

在公開資料集上，ORB-SLAM3 的雙目慣性模式於 EuRoC 無人機資料集達到平均約 3.5 公分的精度；在代表 AR / VR 情境、以手持快速運動錄製的 TUM-VI 房間序列中，精度可達約 9 公釐。這個數量級足以說明：一顆便宜的相機加上一顆便宜的 IMU，在正確的演算法下，能達到令人驚訝的精度——而這正是本書希望同學親手驗證的事。

公式或流程

一、慣性導航基本運動學方程

設世界座標系中的位置為 p 、速度為 v 、姿態旋轉矩陣為 R ；IMU 量得角速度 ω 與比力 a ，重力向量為 g 。連續時間的更新方程為：

$$\dot{R} = R \cdot [\omega]_x$$

$$\dot{v} = R \cdot a + g$$

$$\dot{p} = v$$

其中 $[\omega]_x$ 表示角速度的反對稱矩陣。這三條式子就是慣性導航的全部——但也正

因為要對它們積分兩次，才有了 7.3 節的漂移問題。離散化後的實作形式為：

$$R(k+1) = R(k) \cdot \text{Exp}(\omega \cdot \Delta t)$$

$$v(k+1) = v(k) + (R(k) \cdot a + g) \cdot \Delta t$$

$$p(k+1) = p(k) + v(k) \cdot \Delta t + (1/2) \cdot (R(k) \cdot a + g) \cdot \Delta t^2$$

二、零偏造成的誤差成長

若加速度計零偏為 b_a 、陀螺儀零偏為 b_g ，主要誤差項的成長行為為：

$$\text{姿態誤差} \approx b_g \cdot t \quad (\text{線性})$$

$$\text{速度誤差} \approx b_a \cdot t + g \cdot b_g \cdot t^2 / 2$$

$$\text{位置誤差} \approx b_a \cdot t^2 / 2 + g \cdot b_g \cdot t^3 / 6 \quad (\text{三次方項主導})$$

注意第三式中含重力 g 的 t^3 項：陀螺儀零偏透過「姿態錯誤 → 重力扣不乾淨 → 假加速度」的途徑，最終成為長時間慣性導航的主要誤差來源。這條式子解釋了為什麼高階 IMU 的價格幾乎全花在陀螺儀上。

三、互補濾波器完整實作流程

圖 7-2 互補濾波器姿態估測流程

① 讀取 IMU：三軸加速度 a 、三軸角速度 ω

↓

② 由加速度計計算姿態： $\varphi_{\text{acc}} = \text{atan2}(a_y, a_z)$ 、 $\theta_{\text{acc}} = \text{atan2}(-a_x, \sqrt{a_y^2 + a_z^2})$

↓

③ 由陀螺儀積分： $\varphi_{\text{gyro}} = \varphi_{\text{prev}} + \omega_x \cdot \Delta t$ (俯仰同理)

↓

④ 互補融合： $\varphi = \alpha \cdot \varphi_{\text{gyro}} + (1 - \alpha) \cdot \varphi_{\text{acc}}$ (α 約 0.95 ~ 0.99)

↓

⑤ 偏航角 ψ ：由磁力計提供 (第 6 章)，或由陀螺儀積分但接受其漂移

↓

⑥ 輸出姿態 (φ, θ, ψ)，並更新 φ_{prev} 、 θ_{prev} 供下一輪使用

四、視覺慣性緊耦合的代價函數

緊耦合系統把所有資訊放進同一個最佳化問題，其代價函數的結構為：

$$\min \Sigma // \text{視覺重投影誤差} //^2 + \Sigma // \text{IMU 預積分誤差} //^2 + // \text{先驗誤差} //^2$$

第一項要求「地圖點投影到影像上的位置」與「實際觀測到的特徵點位置」盡量接近；第二項要求「相鄰關鍵幀之間的位姿變化」與「IMU 預積分推算的結果」盡量一致；第三項則保留先前最佳化的資訊。待求解的變數包含所有關鍵幀的位姿、速度、IMU 零偏與地圖點座標。

這正是第 12 章圖優化的完整形式——同學此刻只需理解一件事：所謂「緊耦合」，就是讓不同感測器的誤差項在同一條式子裡互相角力，最後求出一個對所有觀測都最合理的答案。

案例

案例 7-1 潛艦與飛彈：慣性導航的原初戰場

慣性導航最早的大規模應用出現在冷戰時期的潛艦與洲際飛彈上。潛艦下潛後完全收不到任何外部訊號，必須靠純慣性導航維持數週的航行；飛彈飛行途中不能依賴

可能被干擾的外部訊號，同樣只能自主推算。

這些系統使用的是機械式或雷射環形陀螺儀等導航級元件，體積如公事包、重達數十公斤、造價數百萬美元，零偏穩定度比手機 IMU 好上百萬倍。即便如此，潛艦仍必須定期上浮或以其他方式校正——因為 7.3 節的數學是無情的：再小的誤差，乘上足夠長的時間都會變成大誤差。

這個歷史對比極具教學意義：同學手上數十元的 MPU-6050，物理原理與那些天價系統完全相同，差別只在精度等級。也因此，本書強調的重點不是「買更好的 IMU」，而是「用更聰明的演算法與感測器融合，讓便宜的 IMU 發揮最大價值」——這正是過去二十年 SLAM 研究最大的貢獻。

案例 7-2 手機的計步器與螢幕自動旋轉

同學口袋裡的手機，每天都在示範慣性感測的應用與限制。螢幕自動旋轉靠的是加速度計判斷重力方向——所以躺在床上平放手機時，它常常轉錯方向，因為此時重力垂直於螢幕，滾轉角在數學上不可觀測（7.2 節的限制在生活中現形）。

計步器則展示了另一種思路：它不積分加速度求位置（那會如 7.3 節所述迅速失準），而是偵測加速度訊號中的週期性峰值來「數步數」，再乘上估計的步長得到距離。這個方法巧妙地繞開了積分漂移——與其積分，不如辨識模式。這正是行人航位推算（PDR）的核心思想，也常與第 4 章的 Wi-Fi 指紋定位、第 6 章的地磁指紋定位搭配使用。

這給同學一個重要啟示：面對感測器的先天限制，除了「用更好的濾波器」之外，還有一條路是「換一種用法」——不要求它做它做不到的事。

案例 7-3 EuRoC 資料集：視覺慣性 SLAM 的共同考場

EuRoC MAV 資料集由 ETH Zürich 團隊發布，以微型飛行器搭載雙目相機與 IMU，在機房與工業廠房中錄製多段序列，並以動作捕捉系統與雷射掃描提供高精度真值。它已成為視覺慣性 SLAM 領域的標準評測基準——幾乎所有相關論文都會報告在 EuRoC 上的成績。

資料集刻意包含由易到難的序列：從緩慢平穩的飛行，到快速激烈、影像明顯模糊的機動。這種設計讓研究者能清楚看出演算法在什麼條件下開始失效。ORB-SLAM3 在此資料集上的雙目慣性模式達到平均約 3.5 公分精度，即為在此標準下的成績。

對同學而言，公開資料集的價值在於：不需要無人機、不需要動作捕捉系統，只要一台電腦，就能執行與世界頂尖研究相同的實驗並比較結果。第 29 章的實作將直接使用 EuRoC 資料集——這是「站在巨人肩膀上」最具體的實踐。

實作活動

實作 7-1 親手見證積分漂移

目的：透過實際量測與計算，親身體驗慣性導航的漂移現象，並實作互補濾波器比較融合前後的差異。這是本書第二篇最重要的一個實驗——它會讓同學對「為什麼需要 SLAM」產生深刻的體感。

器材

- 智慧型手機（安裝可記錄並匯出感測器原始資料的 App），或 ESP32 / Arduino 搭配 MPU-6050 / MPU-9250 模組。
- 電腦（Python 環境，需 numpy 與 matplotlib）、捲尺、平坦桌面。

第一部分：靜止漂移測試

34. 把裝置完全靜止平放在桌上，記錄 60 秒的三軸加速度與角速度原始資料並匯出。
35. 計算零偏：取前 10 秒資料的平均值。理論上靜止時角速度應為 0、加速度應只有重力，實測偏差即為零偏。記錄三軸陀螺儀零偏（單位轉換為度 / 秒）。
36. 積分實驗（不扣零偏）：把 60 秒的角速度直接積分求角度。裝置明明完全沒動，畫出的角度曲線偏離了多少度？
37. 積分實驗（扣除零偏）：扣掉步驟 2 算出的零偏後重新積分，比較兩條曲線。誤差減少了多少？還剩下多少？剩下的是什麼造成的（提示：7.3 節的三種誤差來源）？
38. 位置漂移：扣除重力後對加速度做兩次積分求位置。裝置靜止 60 秒，算出的「位置」跑了多遠？與 7.3 節的理論估算是否吻合？

第二部分：運動測試與濾波

39. 沿桌面把裝置以直線推動 1 公尺後停下，記錄全程資料。以兩次積分計算移動距離，與捲尺實測的 1 公尺比較，計算誤差百分比。
40. 把裝置緩慢傾斜至約 30 度後放回水平，重複數次，記錄資料。
41. 以 Python 分別計算三條姿態曲線：純陀螺儀積分、純加速度計計算、互補濾波融合（ $\alpha = 0.98$ ），畫在同一張圖上比較。
42. 震動測試：輕敲桌面製造震動，觀察三條曲線的反應差異——哪一條會劇烈跳動？哪一條最平滑？哪一條長期最可靠？
43. 參數實驗：把 α 依序設為 0.90、0.95、0.98、0.999，比較輸出曲線的平滑度與跟隨速度，找出你認為最適合的值並說明理由。

討論問題

32. 靜止 60 秒後，你們算出的「位置」漂移了多少公尺？如果一台機器人靠這個結果導航，會發生什麼事？
33. 扣除零偏後誤差顯著下降，這說明零偏是主要誤差來源。但零偏會隨溫度與時間變化——這對「開機後立刻校正一次就好」的做法有什麼啟示？
34. 直線推動 1 公尺的實驗中，你們的誤差百分比是多少？誤差主要來自加速階段還是減速階段？為什麼？
35. 比較三條姿態曲線，說明陀螺儀與加速度計各自的優缺點如何在圖上具體呈現。
36. (大學部) 依據你們量到的零偏值，套用 7.3 節的公式估算「位置誤差達到 1 公尺」需要多久。這個時間對機器人系統設計意味著什麼？

延伸挑戰 (大學部)

其一：實作 Madgwick 或 Mahony 濾波器（兩者皆有公開的參考程式碼），以四元數表示姿態，與互補濾波器比較長時間運行的穩定度。其二：下載 EuRoC 資料集的一段序列，讀取其中的 IMU 資料與真值軌跡，實作純慣性推算並與真值比較，繪製誤差隨時間成長的曲線，驗證 7.3 節關於 t^2 、 t^3 成長的理論。這份實驗將直接銜接第 29 章的 ORB-SLAM3 實作。

延伸閱讀

41. P. D. Groves, Principles of GNSS, Inertial, and Multisensor Integrated Navigation Systems, 2nd ed., Artech House, 2013. (慣性導航的權威教科書，第 4 至 5 章為 IMU 誤差模型的標準參考)

42. M. Kok, J. D. Hol, and T. B. Schön, "Using Inertial Sensors for Position and Orientation Estimation," Foundations and Trends in Signal Processing, 2017. (可免費取得的完整教程，含程式範例，強烈推薦大學部閱讀)
43. C. Forster et al., "On-Manifold Preintegration for Real-Time Visual-Inertial Odometry," IEEE Transactions on Robotics, 2017. (IMU 預積分理論的代表論文，第 7.5 節主要依據)
44. C. Campos et al., "ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM," IEEE Transactions on Robotics, 2021. (第 7.7 節主要依據，第 22 章將完整剖析)
45. S. Madgwick, "An Efficient Orientation Filter for Inertial and Inertial/Magnetic Sensor Arrays," Technical Report, University of Bristol, 2010. (Madgwick 濾波器原始文件，附完整程式碼)
46. M. Burri et al., "The EuRoC Micro Aerial Vehicle Datasets," International Journal of Robotics Research, 2016. (EuRoC 資料集說明論文)
47. T. Qin, P. Li, and S. Shen, "VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator," IEEE Transactions on Robotics, 2018. (另一套經典的緊耦合視覺慣性系統，可與 ORB-SLAM3 對照閱讀)

小結

28. 加速度計量測的是比力（運動加速度減重力），因此靜止時讀到約 9.8、自由落體時讀到 0；陀螺儀利用科氏力量測角速度，輸出必須積分才能得到角度。
29. 加速度計可由重力方向解算滾轉與俯仰角，但對偏航角完全無感，且在載體加速時會受運動加速度污染；歐拉角在俯仰角接近 ± 90 度時會發生萬向鎖，實務多改用四元數。

30. 積分漂移是慣性導航的根本限制：加速度計零偏造成位置誤差隨 t^2 成長，陀螺儀零偏透過「姿態錯誤使重力扣不乾淨」的途徑造成 t^3 成長；姿態精度比加速度精度更關鍵。
31. 姿態濾波融合陀螺儀（短期準）與加速度計（長期準）：互補濾波器簡單高效但無法估零偏；卡爾曼與 EKF 能同時估計零偏與不確定度，是無人機飛控與機器人的主流方案。
32. IMU 預積分把兩關鍵幀之間的高頻量測積分為「與初始位姿無關」的相對量，使最佳化過程中不必反覆重算，是現代視覺慣性 SLAM 得以即時運行的關鍵技術。
33. IMU 在視覺慣性 SLAM 中提供四大貢獻：絕對尺度、重力方向、視覺失效時的空隙填補、高頻輸出；視覺則回報以不漂移的觀測來校正 IMU 零偏。緊耦合精度優於鬆耦合。
34. ORB-SLAM3 全程以最大後驗估測為基礎完成緊耦合視覺慣性 SLAM，連 IMU 初始化亦然，精度較先前方法提升約二至十倍；初始化需要足夠的運動激勵，否則尺度與零偏不可觀測。

下一章，我們轉向另一種截然不同的感測器——光達。它不像 IMU 那樣猜測自己動了多少，而是直接以雷射量出「那面牆在三公尺外」。從飛行時間原理、2D 與 3D 光達的差異，到它在掃地機器人、倉儲 AGV 與自駕車中的角色，第 8 章將為第四篇的 2D SLAM 實作揭開序幕。

台灣自編大學教科書

@prof

《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第 8 章

光達 LiDAR 介紹

編者：李世淵 (機器人叫獸)

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

學習目標

前幾章的感測器都在「猜測」——里程計猜位移、IMU 猜運動、Wi-Fi 猜距離。本章介紹的光達則不同：它以雷射光直接「量」出「那個方向、那個距離有東西」。這種精確的幾何量測，使光達成為 2D 與 3D SLAM 最重要的感測器之一。學完本章之後，同學應能夠達成下列目標：

43. 說明飛行時間 (ToF) 測距原理，並解釋為何光達能提供精確的距離量測。
44. 區分 2D 光達、3D 多線光達的結構差異，以及機械式與固態式掃描機構的優缺點。
45. 說明 LaserScan 與 PointCloud 兩種資料格式的內容與差異。
46. 解釋角解析度、量測距離、精度與反射率如何影響光達的實際表現。
47. 舉例說明光達在室內建圖、倉儲 AGV 與自駕車中的角色。
48. 比較光達與相機、IMU 的優缺點，說明為何實務系統常將三者融合。
49. 認識適合教學的低成本光達平台，為第四篇的 2D SLAM 實作做好準備。

核心概念

中文名詞	英文名詞	概念說明
光達	LiDAR	以雷射光量測距離的感測器，全名為光偵測與測距。
飛行時間	Time of Flight, ToF	由光發射到反射返回的時間乘光速，計算距離。
脈衝式測距	Pulsed / dToF	發射短脈衝並直接量測往返時間，適合長距離。
調頻連續波	FMCW	連續調變雷射頻率，可同時量測距離與速度。
雷射掃描	LaserScan	2D 光達輸出的一圈距離值資料格式。

中文名詞	英文名詞	概念說明
點雲	Point Cloud	3D 光達輸出的三維點集合。
線數	Channels / Lines	3D 光達垂直方向的雷射束數量，如 16、32、64、128 線。
視野角	Field of View, FoV	光達可掃描的角度範圍，分水平與垂直。
角解析度	Angular Resolution	相鄰兩個量測點之間的角度間隔，越小越密。
反射率	Reflectivity	物體表面反射雷射的比例，影響可偵測距離與訊號強度。
固態光達	Solid-State LiDAR	無大型旋轉機構的光達，含 MEMS、Flash、OPA 等架構。
多普勒效應	Doppler Effect	FMCW 光達藉以直接量測目標徑向速度的物理現象。

8.1 光達的基本原理：Time of Flight 與掃描

光達 (LiDAR) 的全名是 Light Detection and Ranging——光偵測與測距。它的核心概念與第 5 章的 GNSS、第 4 章的 UWB 一脈相承：發射訊號、量測往返時間、乘上訊號速度得到距離。差別只在於光達用的是雷射光，而且距離短得多、精度高得多。

一、脈衝式飛行時間 (dToF)

最直觀的測距法是直接飛行時間 (direct ToF, dToF)：發射一個極短的雷射脈衝，計時它從發射到反射返回所經過的時間 t ，距離即為：

$$d = (c \times t) / 2$$

其中 c 為光速，除以 2 是因為光走了「去程加回程」的來回距離。這個公式簡單，但對計時精度的要求極高——光速下 1 奈秒等於 30 公分，要達到公分級精度，計時必須精準到約 0.07 奈秒。這正是光達晶片的技術門檻所在，也是它比超音波、紅外線測距昂貴的原因。dToF 的優點是能量集中在短脈衝中、可測遠距離，是自駕車長距離光達的主流。

二、調頻連續波 (FMCW)

另一種日益重要的方法是調頻連續波 (FMCW, Frequency-Modulated Continuous Wave)：不發射脈衝，而是持續發射一道頻率隨時間線性變化 (Chirp) 的雷射，再把返回訊號與當下發射訊號混合，量測兩者的頻率差 (拍頻)。頻率差同時編碼了距離與速度——這是 FMCW 最迷人的特性。

FMCW 的最大優勢是能利用多普勒效應直接量出每一點的徑向速度，不需要像脈衝式那樣比較連續多次量測才能推算速度；而且它採用同調偵測 (Coherent Detection)，能幾乎完全排除環境光與其他光達的干擾。代價是需要窄線寬雷射光源，

製造難度高、成本較貴，目前主要用於高階車用光達與研究領域。近年矽光子技術的進展，已能在微米級平台上實現約 120 公尺偵測距離與 0.3 公尺精度的 FMCW 系統，被視為下一世代的重要方向。

三、為什麼需要掃描

單束雷射一次只能量測一個方向的距離——那只是一個「點」。要建立環境的輪廓或立體形狀，就必須讓雷射束掃過不同方向，這就是掃描（Scanning）機構的任務。掃描方式決定了光達的形態與價格，也是 8.2 節的主題。

觀念釐清：光達、雷達、超音波的差異

三者原理相同（發射訊號、量往返時間），差別在使用的波：

- 光達（LiDAR）：雷射光，波長短、方向性極佳，可達公分至公釐級精度與高解析度；弱點是玻璃、鏡面、雨霧。
- 雷達（Radar）：無線電波，穿透力強、不受雨霧影響、可測速；弱點是解析度低、難分辨物體形狀。
- 超音波（Ultrasonic）：聲波，極便宜、近距離可靠；弱點是範圍小、精度低、易受風與吸音材料影響。

自駕車常三者並用，正是取其互補——這與本書反覆強調的感測器融合思想完全一致。

8.2 2D 光達、3D 光達、多線雷射

依照掃描維度，光達可分為 2D 與 3D 兩大類；依照掃描機構，又可分為機械式與固態式。這兩個分類軸決定了光達的能力、形態與價格。

一、2D 光達：掃描一個平面

2D 光達讓單束雷射在一個水平面上旋轉一圈（常見 360 度或 270 度視野），量出這個平面上四面八方的距離，輸出一圈距離值。它就像一把在固定高度水平劃過空間的「雷射尺」。

2D 光達的優點是資料量小、運算輕、價格親民（教學型號約數千至上萬元台幣），是掃地機器人、教學小車與室內 AGV 的標準配備，也是本書第四篇 Gmapping、Cartographer 實作的主角。它的先天限制是「只看得到一個高度切面」——放在 20 公分高處掃描，就看不到 30 公分高的桌面，也偵測不到懸空的橫桿或低矮的門檻。這個限制在 8.5 節與第 16 章掃地機器人的討論中會反覆出現。

二、3D 光達：多線與掃描

要獲得立體資訊，就需要在垂直方向也能量測。3D 光達主要靠兩種方式達成：

- 多線（Multi-beam）：在垂直方向堆疊多組雷射發射與接收器，每一組負責一個仰角，同時旋轉掃描。常見規格有 16 線、32 線、64 線、128 線——線數越高，垂直方向越密、立體重建越精細，價格也越貴。這是自駕車與自主巡檢機器人的主流。
- 單束垂直掃描：以震盪鏡面或旋轉稜鏡讓單束（或少數幾束）雷射在垂直方向也擺動，用較少的雷射組件覆蓋垂直視野，藉此降低成本。

三、機械式 vs 固態式

傳統光達靠馬達帶動整個光學組件或鏡面旋轉，稱為機械式。它視野大、技術成熟，但有明顯缺點：體積大、有旋轉部件易磨損、在震動環境中壽命受限、成本高。

固態光達（Solid-State LiDAR）則試圖減少或消除大型旋轉機構，主要有三種架構：

- MEMS：以晶片上的微振鏡偏轉雷射束逐線掃描。嚴格說是「半固態」，因為仍有微小的振鏡運動；目前是出貨量最大的固態方案。
- Flash：像相機閃光燈一樣，一次照亮整個場景，由 SPAD / APD 感測器陣列接收全部回波，完全無移動部件；優點是無掃描延遲，限制是照明能量分散導致距離較短。
- OPA（光學相位陣列）：透過調整多個波導發射器的相位，以純電子方式在微秒內偏轉光束、無任何機械運動；技術最先進但量產成熟度仍在推進中。

固態光達的整體趨勢是體積更小、更耐震、成本更低，正逐步從高階車用走向機器人與消費領域。對本書讀者而言，理解「機械式為主的今天」與「固態化的明天」這個演進方向，有助於判讀市場上琳琅滿目的產品。

8.3 光達資料格式：LaserScan 與 PointCloud

感測器量到的資料，最終要以特定格式交給 SLAM 演算法處理。光達在 ROS 生態中有兩種標準格式，分別對應 2D 與 3D。

一、LaserScan：2D 光達的資料格式

2D 光達輸出的是一圈距離值，ROS 以 sensor_msgs/LaserScan 訊息表示。它的核心是一個距離陣列，配合幾個描述掃描範圍的參數：

- angle_min、angle_max：掃描的起始與結束角度。
- angle_increment：相鄰兩個量測點的角度間隔（即角解析度）。
- range_min、range_max：可信的最近與最遠量測距離。
- ranges[]：距離值陣列——這是資料主體，每個元素就是對應角度上量到的距離。
- intensities[]：反射強度陣列（選用）。

由於角度是規律遞增的，LaserScan 不需要為每個點單獨儲存角度，只要距離陣列加上起始角與角度間隔，就能還原每個點的方向。這種緊湊表示讓 2D 資料非常輕量——一圈掃描通常只有數百到上千個距離值。要在座標系中畫出某個點，只需簡單的極座標轉直角座標（8.5 節公式）。

二、PointCloud : 3D 光達的資料格式

3D 光達的資料是三維空間中的大量點，ROS 以 `sensor_msgs/PointCloud2` 訊息表示。與 LaserScan 不同，每個點都直接儲存自己的三維座標（ x 、 y 、 z ），可再附加反射強度、線號、時間戳記等欄位。

點雲的資料量遠大於 LaserScan：一台 64 線光達每秒可產生上百萬個點。這帶來兩個工程課題——龐大的傳輸與儲存需求，以及後續處理前通常需要的降採樣（第 17 章的 Voxel Grid）。點雲的處理有專門的開源函式庫 PCL（Point Cloud Library），第五篇會深入使用。

特性	LaserScan (2D)	PointCloud2 (3D)
資料維度	一圈距離值 (平面)	三維點集合 (立體)
每筆資料量	數百至上千點	數萬至數百萬點
點的表示	距離 + 規律角度	直接存 x 、 y 、 z
運算負擔	輕	重，常需降採樣
典型來源	2D 旋轉光達	多線或固態 3D 光達
本書應用	第 13 至 16 章 2D SLAM	第 17 至 18 章 3D SLAM

表 8-1 LaserScan 與 PointCloud2 資料格式比較

8.4 光達解析度、角解析度、量測距離與反射率

選用或評估一台光達，必須看懂它的規格表。本節說明四個最關鍵的參數，它們

直接決定 SLAM 建圖的品質。

一、量測距離與精度

- 量測範圍：最近與最遠可量的距離。教學型 2D 光達常見 0.15 至 12 公尺，自駕車長距離光達可達 200 公尺以上。
- 精度 (Accuracy)：量測值與真實距離的接近程度，常見規格為 ± 2 至 ± 3 公分。
- 精密度 (Precision)：對同一目標重複量測的一致性。精度與精密度是兩回事——一把總是量多 5 公分的尺不精確但可能很精密，且系統性偏差可校正。

二、角解析度：地圖的細膩程度

角解析度是相鄰兩個量測點之間的角度間隔，它決定了地圖能呈現多細的結構。這裡有一個關鍵的幾何事實：同樣的角解析度，距離越遠，相鄰兩點的實際間距越大。

$$\text{相鄰點間距} \approx \text{距離} \times \text{角解析度 (弧度)}$$

舉例：角解析度 1 度、距離 5 公尺時，相鄰兩點間距約 8.7 公分——這表示 5 公尺外一根 5 公分寬的椅腳，很可能整個落在兩束雷射之間而完全偵測不到。這個現象解釋了為什麼遠處的細小物體容易被光達漏掉，也是 8.5 節與第 16 章討論「掃地機器人為何撞到桌腳」的關鍵。

三、掃描頻率

掃描頻率 (Scan Rate，如 10 Hz 表示每秒轉 10 圈) 影響機器人移動時的建圖品質。頻率太低而機器人移動快，同一圈掃描的起點與終點之間機器人已經移動了一段距離，造成掃描變形 (Motion Distortion) ——這在第 18 章的 3D 光達 SLAM 中是必須專門處理的問題。

四、反射率：被忽略卻關鍵的變數

光達依賴目標把雷射反射回來，因此目標的反射率至關重要。白牆反射率高、容易偵測；黑色物體吸收大部分雷射，反射微弱，可偵測距離大幅縮短——一台標稱 12 公尺的光達，面對黑色沙發可能只剩幾公尺。更棘手的是：

- 鏡面與玻璃：雷射被鏡面反射到別處而非返回，光達會「看穿」玻璃或量到鏡中的虛像，在地圖上造成錯誤——這是室內光達 SLAM 最常見的失敗來源之一。
- 高反光物體：金屬、反光標誌可能造成過強回波或多重反射。
- 液體表面：水面、光滑地面可能產生鏡面反射造成漏測。

因此光達規格表上的「最大距離」通常標註在特定反射率（如 10% 或 90%）條件下——同學看規格時務必注意這個前提，否則實際表現會與預期落差很大。

8.5 光達在室內建圖、倉儲 AGV、自駕車中的應用

一、室內建圖與服務機器人

2D 光達是室內建圖的主力。掃地機器人、送餐機器人、導覽機器人多以一顆水平 2D 光達搭配 Gmapping 或 Cartographer 建立佔據格地圖（第四篇）。室內環境結構分明（牆、門、走廊），光達的精確幾何量測能建出清晰穩定的地圖，這是視覺方法在白牆、昏暗環境下難以匹敵的。但前述的玻璃隔間、鏡面與單一高度切面問題，也讓純 2D 光達在現代玻璃帷幕建築中屢屢受挫，實務上常需搭配超音波或紅外線補足。

二、倉儲 AGV 與工廠自動化

倉儲自動導引車（AGV）是光達最成熟的工業應用之一。除了用於 SLAM 導航，光達還兼任安全防護——許多工業光達具備安全認證，能在人員進入危險區域時即時觸發減速或停車。工廠環境相對受控（地面平整、光線穩定、佈局已知），使光達得

以發揮最大可靠度。這也呼應第 4 章亞馬遜案例的觀念：環境越可控，技術選型越能簡化。

三、自駕車

3D 光達是多數自駕車的核心感測器（特斯拉的純視覺路線是著名的例外）。它提供精確的立體環境結構，用於障礙物偵測、可行駛區域判定，以及與高精地圖比對來定位（第 5 章都市峽谷案例）。自駕車對光達的要求最為嚴苛：長距離（看清高速前方）、高解析度（分辨行人與電線桿）、高更新率、以及嚴格的車規可靠度。這也是車用光達單價仍高（每顆約數百美元）、且積極往固態化與 FMCW 演進的原因。

四、其他新興應用

- 測繪與空間掃描：背包式或手持光達掃描建築、古蹟、工地，產生高精度 3D 模型（第 15、17 章）。
- 農林業：無人機載光達穿透樹冠量測林木高度與地形。
- 機器狗與巡檢：3D 光達配合 IMU 緊耦合，在無 GNSS 的室內外環境自主巡檢（第 18 章）。

8.6 光達與相機、IMU 的比較

把光達放進第二篇的感測器全景中比較，能更清楚看出它的定位。第 1 章的表 1-1 已做過總覽，此處針對建圖與定位任務做更深入的對照。

比較項目	光達	相機	IMU
直接量測	距離（幾何）	亮度 / 顏色	加速度 / 角速度
深度資訊	直接、精確	需推算（單目無尺度）	無
受光線影響	幾乎不受（主動光	嚴重（依賴環境光）	不受

比較項目	光達	相機	IMU
	源)		
語意資訊	無 (只有形狀)	豐富 (可辨識物體)	無
資料量	中至大	大	小
成本	中至高	低	低至中
主要弱點	玻璃鏡面、雨霧、無語意	光線、無尺度、無深度	漂移

表 8-2 光達、相機、IMU 在建圖定位任務上的比較

三者的互補邏輯

這張表清楚顯示三者為何天生互補：

- 光達給精確的幾何與深度，但看不懂「那是什麼」，也怕玻璃鏡面。
- 相機給豐富的語意與顏色，但沒有可靠的深度與尺度，還怕黑暗逆光。
- IMU 給高頻的運動資訊填補空隙，但會漂移。

於是最強健的系統往往三者並用：光達 SLAM 建立精確幾何骨架、相機提供語意標註與閉環辨識、IMU 在光達掃描變形與快速運動時提供補償。第 18 章的光達慣性緊耦合 (LIO) 就是光達加 IMU 的代表，第 31 章的語意 SLAM 則是光達 (或相機) 加 AI 的代表。

光達 vs 視覺：一場沒有標準答案的辯論

在自駕車領域，「該不該用光達」曾是激烈爭論的話題。主張純視覺者認為：人類只用眼睛就能開車，且相機便宜、含語意、可辨識號誌；主張用光達者認為：光達提供的可靠深度是安全冗餘的關鍵，不該為了省成本而犧牲。這場辯論至今沒有定論，但它給同學一個重要啟示：感測器選型從來不是純技術問題，還牽涉成本、安全、法

規與商業策略。作為工程師，理解每種感測器的能力邊界，才能在具體情境中做出有依據的判斷——這正是第二篇七章的共同目的。

8.7 低成本光達與教學型機器人平台

好消息是：光達已經便宜到能進入教室。本節介紹適合教學的光達與平台，為第四篇的實作預做準備。

一、入門級 2D 光達

近年低成本 2D 光達的普及，是 SLAM 教育得以下放到大學部甚至高中的關鍵。這類光達多採三角測距或低成本 ToF 原理，360 度掃描，量測範圍約 8 至 12 公尺，價格從數千元台幣起跳，並提供完整的 ROS 驅動——插上就能發布 LaserScan，直接餵給 Gmapping 或 Cartographer。它們的精度與穩定度雖不及工業型號，但對教學建圖已綽綽有餘。

二、教學型機器人平台

- TurtleBot 系列：ROS 官方的標準教學平台，搭載 2D 光達與完整軟體堆疊，是全球 SLAM 教學使用最廣的平台，網路教學資源極其豐富。本書第 28 章的實作以它為主要參考。
- 自組 ROS 小車：以差速輪底盤、馬達編碼器、微控制器搭配低成本 2D 光達與單板電腦（如 Raspberry Pi 或 Jetson Nano）自行組裝。成本較低、學習深度更高，適合大學部專題——同學能親手處理第 10 章的 TF 座標、第 28 章的里程計發布等每一個環節。
- 模擬平台：完全不需硬體。在 Gazebo / Foxglove 等模擬環境中，可放置虛擬光達小車進行完整 SLAM 實驗（第 27 章）。這讓高中課程與大班教學也能人人動手，是本書強烈推薦的入門途徑。

三、進階：3D 光達教學

低線數（如 16 線）的 3D 光達近年價格也大幅下降，配合 Jetson 等邊緣運算平台，已可用於大學部的 3D SLAM 專題（第 17、18 章）與大學的自主巡檢研究。對高中與大學部入門而言，仍建議從 2D 光達或模擬起步，把 SLAM 的核心觀念建立穩固後，再進入 3D 的世界。

無論選擇哪一種平台，本書的建議都一致：先在模擬中理解流程、再用便宜硬體驗證、最後才追求精度與規模。第四篇的每一個實作，都會標明模擬版與硬體版兩條路徑，讓不同資源條件的同學都能參與。下一章我們暫別「主動量測距離」的光達，轉向資訊最豐富、卻也最需要數學處理的感測器——相機，為第六篇的視覺 SLAM 鋪路。

公式或流程

一、飛行時間測距

脈衝式光達的基本測距公式：

$$d = (c \times t) / 2$$

其中 $c \approx 3 \times 10^8 \text{ m/s}$ 為光速、 t 為脈衝往返時間、除以 2 因光走了來回雙程。由此可推得計時精度需求：

$$\text{距離誤差 } \Delta d = (c \times \Delta t) / 2$$

要達到 1 公分（0.01 m）精度，需要 $\Delta t \approx 0.067$ 奈秒——這說明光達計時電路的技術挑戰，也解釋了為何精確光達價格不菲。

二、LaserScan 點的座標轉換

2D 光達的第 i 個量測點以極座標（距離 r_i 、角度 θ_i ）表示，其中角度可由掃描參

數算出：

$$\theta = angle_min + i \times angle_increment$$

轉換到光達座標系的直角座標（用於建圖與顯示）：

$$x_i = r_i \times \cos \theta$$

$$y_i = r_i \times \sin \theta$$

這個簡單的極座標轉直角座標，是所有 2D SLAM 把「一圈距離值」變成「地圖上一圈點」的第一步。第 13 章的掃描匹配、第 14 章的 Gmapping，都建立在這個轉換之上。

三、角解析度與可偵測物體尺寸

相鄰兩束雷射在距離 d 處の間距，決定了能偵測的最小物體尺寸：

$$\text{點間距 } s \approx d \times \Delta\varphi \quad (\Delta\varphi \text{ 為角解析度，以弧度表示})$$

一個寬度小於 s 的物體，可能整個落在兩束雷射之間而被漏測。例如 $\Delta\varphi = 1^\circ$ (≈ 0.0175 弧度)、 $d = 5 \text{ m}$ 時， $s \approx 8.7 \text{ cm}$ 。這條式子是理解光達「看得多細」的關鍵，也量化了「遠處小物體易漏測」的現象。

四、光達資料處理流程

圖 8-1 光達從發射到 SLAM 輸入的資料流

① 雷射發射脈衝 / 調頻波



② 目標反射，感測器接收回波



③ 計算飛行時間 → 距離 d (dToF) 或拍頻 → 距離 + 速度 (FMCW)

↓

④ 掃描機構提供角度資訊 → 每個點的方向

↓

⑤ 組成資料：2D → LaserScan (距離 + 角度) / 3D → PointCloud (x,y,z)

↓

⑥ 極座標轉直角座標 → 交給 SLAM 前端 (掃描匹配、特徵提取)

案例

案例 8-1 掃地機器人的「隱形陷阱」：2D 光達的高度盲區

許多同學家裡的掃地機器人都上演過同樣的戲碼：它能靈巧地繞開沙發與牆壁，卻總是一頭撞上餐桌的椅腳，或卡在踢腳線邊緣。這不是機器人「笨」，而是 2D 光達的先天限制在現形。

2D 光達裝在機器人頂部某個固定高度（例如 8 公分處）水平掃描，它只「看得見」這個高度切面上的障礙。細瘦的椅腳在這個高度可能只佔一兩束雷射，若又距離較遠（8.4 節的角解析度問題），就可能整個落在雷射束的間隙中被漏掉。更低矮的障礙（電線、門檻、寵物碗）根本不在掃描平面上，光達完全無感。

這個案例生動說明了為何現代掃地機器人紛紛加裝其他感測器——前方的紅外線或超音波障礙偵測、底部的懸崖感測器、甚至改用 3D ToF 相機（第 16 章）。它也預告了一個重要觀念：SLAM 建的地圖只包含感測器「看得到」的世界，感測器的物理限制，直接決定了地圖的完整性與導航的安全性。

案例 8-2 自駕車的光達之爭：Waymo 與 Tesla 的路線分歧

自駕車產業存在一場著名的技術路線分歧。以 Waymo 為代表的主流陣營，車頂與車身佈滿多顆 3D 光達，主張光達提供的可靠深度是安全的基石；以 Tesla 為代表的另一陣營，則走純視覺路線，主張用足夠多的相機加上強大的 AI，就能像人類一樣只靠「看」來開車，且成本大幅降低、可大規模量產。

雙方各有理據：光達派強調惡劣光線與安全冗餘，視覺派強調成本、語意理解與可擴展性。隨著車用光達持續降價與固態化、以及視覺 AI 持續進步，這場辯論仍在演變。對本書讀者，重點不在於「誰對誰錯」，而在於看懂背後的權衡：這正是第 8.6 節「感測器選型是技術、成本、安全、商業的綜合判斷」的最佳教材。

案例 8-3 背包式 3D 掃描：讓建築「數位分身」

把一台多線 3D 光達、IMU 與運算單元裝進背包，操作員背著它走過建築物，就能即時建立整棟樓的三維點雲模型——這就是背包式移動掃描（Backpack Mobile Mapping），Google 的 Cartographer 系統最初就展示過這類應用（第 15 章）。

走過走廊、樓梯、房間的過程中，系統靠 SLAM 即時拼接每一時刻的光達掃描，最終產出可用於建築資訊模型（BIM）、室內導航、古蹟保存的高精度數位分身。這個案例把本書的多個主題串在一起：ToF 測距（本章）、IMU 補償掃描變形（第 7 章）、3D 點雲處理（第 17 章）、閉環修正（第 15 章）。它也展示了 SLAM 的價值不僅在於「讓機器人動起來」，更在於「讓人類以前所未有的效率把真實空間數位化」——這是大學場域專題極具潛力的方向。

實作活動

實作 8-1 光達資料視覺化與規格解讀

目的：透過真實或模擬的光達資料，理解 LaserScan 的結構、親手完成極座標轉直角座標把「一圈距離值」畫成「環境輪廓」，並學會解讀光達規格表。本實作提供模擬與硬體兩條路徑，適合不同資源的班級。

路徑 A：模擬（無需硬體，適合全班）

40. 在電腦上安裝 ROS 2 與 TurtleBot3 模擬環境（詳細步驟見第 27 章，或依官方教學），啟動 Gazebo 世界與虛擬小車。
41. 用 RViz 顯示 /scan 話題，觀察光達掃描的紅點如何隨小車移動而貼合牆面。
42. 用指令查看 LaserScan 訊息內容，找出 angle_min、angle_increment、range_max 等參數，記錄下來並計算這台虛擬光達的視野角與總點數。
43. 錄製一段 rosbag，匯出一圈 ranges 陣列。

路徑 B：硬體（有低成本 2D 光達的班級）

44. 將 2D 光達接上電腦，啟動其 ROS 驅動，確認 /scan 話題正常發布。
45. 把光達放在教室中央靜置，用 RViz 觀察掃描輪廓是否貼合四面牆與桌椅。
46. 分別把一塊白紙板、黑色書包、玻璃板（或鏡子）放在光達前方 2 公尺處，比較三者 RViz 上的顯示差異，記錄玻璃與鏡面造成的異常。
47. 錄製一圈掃描資料匯出，供下一步分析。

共同部分：資料分析（Python）

48. 讀入匯出的 ranges 陣列，依 8.5 節公式把每個（距離、角度）轉為（x、y）直角座標。
49. 用 matplotlib 把所有點畫出來，觀察是否重現了環境的俯視輪廓（牆、桌、門口）。

50. 計算並標示：這台光達在 3 公尺與 6 公尺處，相鄰兩點的實際間距各是多少？

討論這對偵測細小物體的影響。

51. 規格解讀：找一份真實光達的規格表（教學型或車用型皆可），指出其量測範圍、角解析度、掃描頻率、以及「最大距離」標註的反射率前提。

討論問題

44. 你畫出的點雲輪廓，哪些地方最清晰？哪些地方有缺口或雜點？缺口最可能對應環境中的什麼（玻璃？黑色物體？太遠？）？

45. （路徑 B）玻璃與鏡子在光達資料上造成了什麼異常？如果一台掃地機器人用這台光達建圖，遇到落地窗會發生什麼事？

46. 3 公尺與 6 公尺處的點間距差多少？若要可靠偵測 5 公分寬的椅腳，這台光達在多遠的距離內才辦得到？

47. 根據本章的比較（表 8-2），如果要為一台校園服務機器人選主要感測器，你會選光達還是相機？為什麼？需要補上什麼感測器？

延伸挑戰（大學部）

其一：撰寫一個 ROS 2 節點，訂閱 /scan、把 LaserScan 即時轉為點雲並發布，在 RViz 中同時顯示原始掃描與轉換結果。其二：實作一個簡單的「最近障礙物警示」——找出每一圈掃描中的最小距離與其方向，當小於安全門檻時輸出警告，這是第 28 章避障功能的雛形。

延伸閱讀

37. R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, Introduction to Autonomous Mobile Robots, 2nd ed., MIT Press, 2011. （第 4 章感測器部分含測距原理的系統介紹）

- 38.T. Raj et al., "A Survey on LiDAR Scanning Mechanisms," Electronics, 2020.
(各種掃描機構——機械、MEMS、Flash、OPA——的完整比較)
- 39.You Li and Javier Ibanez-Guzman, "Lidar for Autonomous Driving," IEEE Signal Processing Magazine, 2020. (車用光達原理與系統的權威綜述)
- 40.ROS 官方文件 sensor_msgs/LaserScan 與 sensor_msgs/PointCloud2 訊息定義. (資料格式的第一手規格)
- 41.Point Cloud Library (PCL) 官方教學 (pointclouds.org) . (3D 點雲處理的標準工具，第五篇實作將使用)
- 42.SLAMTEC RPLIDAR 系列與 Livox 系列產品技術文件. (入門級 2D 與低成本 3D 光達的規格與 ROS 驅動範例)
- 43.ROBOTIS TurtleBot3 e-Manual (emanual.robotis.com) . (教學平台的完整實作指引，含 SLAM 範例)

小結

- 48.光達以飛行時間量測距離：脈衝式 (dToF) 直接量往返時間、能量集中適合長距離；FMCW 連續調頻，可同時量測距離與速度且抗干擾，但成本較高，是下一世代的重要方向。
- 49.2D 光達掃描單一水平面、資料輕量價格低，是室內建圖主力；3D 光達以多線或垂直掃描獲得立體資訊；掃描機構從機械式走向 MEMS、Flash、OPA 等固態方案，趨勢是更小、更耐震、更便宜。
- 50.2D 光達輸出 LaserScan (距離陣列 + 規律角度，緊湊輕量)，3D 光達輸出 PointCloud (直接存三維座標，資料量大需降採樣)；兩者分別對應本書第四篇與第五篇。

51. 關鍵規格包含量測範圍、精度與精密度、角解析度（決定可偵測的最小物體，且同一角解析度下距離越遠點越疏）、掃描頻率（過低會造成運動變形）與反射率（黑色物體、玻璃鏡面是主要失效來源）。
52. 光達廣泛用於室內建圖、倉儲 AGV（兼安全防護）、自駕車（立體感知與地圖定位），以及背包式測繪、農林量測與機器狗巡檢等新興領域。
53. 光達給精確幾何但無語意且怕玻璃鏡面、相機給豐富語意但無可靠深度與尺度、IMU 給高頻運動但漂移；三者天生互補，最強健的系統往往並用。
54. 低成本 2D 光達與 TurtleBot 等教學平台，加上完全免費的模擬環境，讓 SLAM 教育得以下放到大學部與高中；本書建議「先模擬、再便宜硬體、後追求精度」的學習路徑。

下一章，我們轉向資訊最豐富的感測器——相機。從針孔相機模型、鏡頭畸變與校正，到單目、雙目、RGB-D 的差異，第 9 章將說明影像如何從一堆像素變成幾何資訊，為第六篇的視覺 SLAM 奠定基礎。

台灣自編大學教科書

@prof

《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第 9 章

相機與視覺導航感測器

編者：李世淵 (機器人叫獸)

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

學習目標

相機是資訊最豐富、成本最低廉的感測器——一張影像同時包含形狀、紋理、顏色與語意。但影像的本質是「三維世界被壓扁成二維」，如何把遺失的深度找回來，正是視覺 SLAM 的核心挑戰。本章是第二篇的最後一章，也是第六篇視覺 SLAM 的入口。學完本章之後，同學應能夠達成下列目標：

50. 說明針孔相機模型，並寫出三維點如何投影到二維影像的數學關係。
51. 解釋相機內參與外參的意義，說明鏡頭畸變的成因與相機校正的目的。
52. 比較單目、雙目、RGB-D 三種相機取得深度的方式與各自的優缺點。
53. 說明特徵點、邊緣、光流三種視覺資訊的差異與用途。
54. 分析視覺導航相對於光達的優勢與限制，特別是尺度不確定性問題。
55. 說明從影像處理銜接到視覺 SLAM 的知識脈絡。
56. 初步認識 VLM 視覺語言模型與語意導航的概念，為第 31 章預做準備。

核心概念

中文名詞	英文名詞	概念說明
針孔相機模型	Pinhole Camera Model	描述三維點透過光心投影到成像平面的幾何模型。
焦距	Focal Length	光心到成像平面的距離，決定放大倍率與視野。
相機內參	Intrinsic Parameters	焦距、主點等相機自身的成像參數，構成內參矩陣 K 。
相機外參	Extrinsic Parameters	相機在世界座標中的位置與姿態（旋轉 R 與平移 t ）。

中文名詞	英文名詞	概念說明
鏡頭畸變	Lens Distortion	鏡頭造成的影像變形，分徑向畸變與切向畸變。
相機校正	Camera Calibration	量測並估計內參與畸變係數的程序。
視差	Disparity	同一點在左右兩張影像中的水平位移，用於雙目測深。
基線	Baseline	雙目相機兩鏡頭之間的距離，影響測深範圍與精度。
特徵點	Feature / Keypoint	影像中易辨識、可重複偵測的顯著點，如角點。
光流	Optical Flow	影像中像素亮度隨時間的移動，反映運動資訊。
尺度不確定性	Scale Ambiguity	單目相機無法確定真實尺寸的先天限制。
視覺語言模型	VLM	能同時理解影像與語言的 AI 模型，用於語意導航。

9.1 針孔相機模型

要理解相機如何「看見」世界，必須先理解一個古老而優雅的模型——針孔相機 (Pinhole Camera)。它的原型就是暗箱：一個密閉盒子的一面開一個小孔，外界的光線穿過小孔，在對面的內壁上投出一個上下顛倒的倒影。這個看似簡單的裝置，蘊含了所有相機成像的核心幾何。

投影的幾何

把針孔 (光心, Optical Center) 設為座標原點，光軸沿 Z 軸指向前方。空間中一個三維點 $P = (X, Y, Z)$ 發出的光線穿過光心，投射在焦距 f 處的成像平面上。由相似三角形，可得投影點的座標：

$$x = f \cdot X / Z$$

$$y = f \cdot Y / Z$$

這兩條式子雖然簡單，卻道破了視覺 SLAM 全部困難的根源——注意分母都是 Z (深度)。這意味著：

- 三維被壓成二維：三個座標 (X, Y, Z) 投影後只剩兩個 (x, y)，深度 Z 這個維度在投影過程中被「除掉」了。
- 深度資訊遺失：距離 2 公尺的小物體，與距離 4 公尺的大物體，可能在影像上佔完全相同的大小。單憑一張影像，無法區分「近的小東西」與「遠的大東西」。

這就是 9.5 節要深入討論的尺度不確定性的幾何本質。找回這個被除掉的 Z ，正是雙目、RGB-D 相機以及視覺 SLAM 各種技術的共同目標。

從物理座標到像素座標

上面算出的 (x, y) 是成像平面上的物理座標 (單位為公釐)。但數位影像用的

是像素座標——以左上角為原點、以像素為單位。兩者的轉換需要三個參數：把公釐換算為像素的比例（ f_x 、 f_y ，把焦距用像素表示），以及影像中心（主點）的位置（ c_x 、 c_y ）。完整的投影關係可寫成矩陣形式：

$$[u \ v \ 1]^T = (1/Z) \cdot K \cdot [X \ Y \ Z]^T$$

其中 K 稱為相機內參矩陣（Intrinsic Matrix）：

$$K = [f_x \ 0 \ c_x; 0 \ f_y \ c_y; 0 \ 0 \ 1]$$

這個 K 矩陣是視覺 SLAM 中最基礎、最常出現的物件。第 4 章的 AprilTag 解算、第 20 章的對極幾何、第 21 章的 ORB-SLAM,都要用到它。而 K 的數值必須透過相機校正（9.2 節）量測得到。

9.2 鏡頭畸變與相機校正

針孔模型假設光線走完美的直線，但真實相機用的是玻璃或塑膠鏡頭。鏡頭雖然能收集更多光線、讓影像更明亮清晰，卻也引入了畸變（Distortion）——影像的幾何變形。

兩種畸變

- 徑向畸變（Radial Distortion）：由鏡頭形狀造成，離影像中心越遠變形越嚴重。桶狀畸變讓直線向外凸（廣角鏡與魚眼鏡明顯）、枕狀畸變讓直線向內凹。用手機廣角鏡拍一面磚牆，邊緣的磚縫明顯彎曲，就是徑向畸變。
- 切向畸變（Tangential Distortion）：由鏡頭與感測器未完全平行安裝造成，使影像某些區域看起來被「傾斜拉扯」。通常比徑向畸變輕微。

畸變對視覺 SLAM 是致命的:SLAM 的幾何計算全都假設針孔模型（直線投影），若不先校正畸變，一條真實的直線在影像中是彎的，所有基於直線與角度的幾何推算都會系統性出錯,建出的地圖會扭曲變形。

相機校正:找出 K 與畸變係數

相機校正 (Camera Calibration) 的目的,就是量測出這台相機的內參矩陣 K 與畸變係數。最經典的方法是張正友 (Zhang) 於 2000 年提出的棋盤格校正法,其原理與第 4 章 AprilTag 的 PnP 一脈相承:

52. 準備一塊尺寸已知的棋盤格標定板 (每個方格邊長已知) 。
53. 用待校正的相機,從不同角度、距離拍攝棋盤格十至二十張。
54. 演算法自動偵測每張影像中棋盤格角點的像素座標。
55. 由於棋盤格的真實三維座標已知(平面上規律排列),而它們的影像座標也測到了,就能反推出使這些對應關係最吻合的 K 與畸變係數(這又是一個最小平方最佳化問題)。
56. 用求得的畸變係數對影像做「去畸變」(Undistortion),把彎曲的影像矯正回符合針孔模型的樣子。

OpenCV 內建完整的校正工具 (`calibrateCamera` 等函式) ,ROS 也有 `camera_calibration` 套件,只要對著棋盤格晃動相機即可自動完成。這是所有視覺 SLAM 實作的第一步,第 29 章實作 RGB-D SLAM 時必定會用到。

實務提醒:校正不是一勞永逸

- 換鏡頭、調焦距、甚至鏡頭受碰撞位移,都會使原校正失效,必須重新校正。
- 校正影像應涵蓋畫面各個區域與多種傾斜角度,尤其別忘了邊角——徑向畸變在邊角最嚴重。
- 校正誤差(重投影誤差)應小於 1 像素才算良好;若偏大,多半是棋盤格拍得不夠多樣或偵測到錯誤角點。

- 工業相機出廠常附校正參數,但精度要求高的應用仍建議自行校正驗證。

9.3 單目相機、雙目相機、RGB-D 相機

9.1 節說明了單張影像會遺失深度。要找回深度,主流有三條路,對應三種相機。

一、單目相機(Monocular):最簡單也最困難

一顆相機、一張影像。硬體最簡單、最便宜、最省電、最普及(每支手機都有)。

但如前所述,單張影像無法直接得到深度——這帶來著名的尺度不確定性(9.5 節)。

那單目相機如何做 SLAM?答案是利用運動:相機移動時,從不同位置拍攝同一場景,就像人閉上一隻眼、左右晃頭也能感知遠近。透過連續多幀影像的視差變化,可以「三角測量」出深度——但這個深度只有相對比例,沒有絕對尺度(不知道是真實的一公尺還是模型屋的一公分),除非引入其他資訊(IMU、已知物體大小、或輪子里程計)。這正是第 7 章 IMU 為單目視覺提供絕對尺度的價值所在。

二、雙目相機(Stereo):模仿雙眼

兩顆相機以固定的已知距離(基線,Baseline)並排安裝,同時拍攝。同一個空間點在左右兩張影像中的水平位置會有落差,稱為視差(Disparity)。近的物體視差大、遠的物體視差小——把手指放在眼前,輪流閉左右眼,手指相對背景跳動很大;看遠山則幾乎不動,就是這個道理。

由視差配合已知的基線與焦距,即可用三角幾何直接算出深度(9.6 節公式)。雙目的優點是:單張(左右對)影像就能得到深度、有絕對尺度、室內外皆可用。限制是:需要精確校正兩相機的相對位置、對無紋理表面(白牆)因為找不到對應點而失效、且測深精度隨距離增加而快速下降(基線越短衰減越快)。

三、RGB-D 相機:主動測深

RGB-D 相機同時輸出彩色影像(RGB)與每個像素的深度(D)。它以主動方式取得深度,主流有兩種技術:

- 結構光(Structured Light):投射一組已知的紅外線圖案(如密集光點)到場景,由圖案的變形程度反推深度。初代微軟 Kinect 即採此法,近距離精度高,但強光下失效、範圍較短。
- 主動紅外雙目(Active IR Stereo):本質是雙目測距,但額外投射紅外線紋理圖案,解決白牆等無紋理表面找不到對應點的問題。Intel RealSense D400 系列即採此法,室內外皆可運作。

RGB-D 相機的最大優點是「開箱即得稠密深度」,不需自己做複雜的深度估計,是室內 RGB-D SLAM(第 29 章 RTAB-Map、ORB-SLAM3)的理想輸入。限制是:多數依賴紅外線,強烈日光下戶外表現不佳(陽光的紅外線會淹沒投射圖案)、有效範圍通常在 0.2 至 10 公尺、且對玻璃與鏡面同樣束手無策(與第 8 章光達的弱點相同)。

類型	取得深度方式	絕對尺度	主要優點	主要限制
單目	靠運動 + 三角測量	無(需外部提供)	最便宜、最普及、最省電	尺度不確定、需運動
雙目	左右視差三角幾何	有	單幀得深度、室內外可用	需校正、怕無紋理、遠處衰減
RGB-D	結構光 / 主動紅外雙目	有	直接輸出稠密深度	怕強光、範圍有限、怕玻璃

表 9-1 三種相機取得深度的方式比較

9.4 特徵點、邊緣、光流、深度估測

相機給的是一整張由數十萬像素組成的影像。但 SLAM 不可能、也不需要處理每

一個像素,而是從影像中提取有用的資訊。本節介紹三種最重要的視覺資訊。

一、特徵點(Feature / Keypoint):視覺 SLAM 的路標

特徵點是影像中「容易被認出來、換個角度也還認得」的顯著點——通常是角點(兩條邊的交會處),例如窗框的角、磁磚的交界、海報的文字邊角。

為什麼偏愛角點?想像在一面白牆上選一個點,下一幀要在新影像中找回它——白牆上到處長得一樣,根本找不到;選在一條直線的邊上,雖能判斷垂直方向卻無法確定沿線位置(這稱為孔徑問題);唯有角點,在兩個方向上都獨一無二,才能被準確地重新定位。這就是特徵點法視覺 SLAM(第 19、21 章)的基石:把角點當作環境中的「路標」,追蹤它們在連續影像中的移動來推算相機運動。ORB、SIFT、SURF、FAST 等都是知名的特徵偵測與描述方法,第 19 章會詳述。

二、邊緣(Edge):結構的輪廓

邊緣是影像中亮度急遽變化的地方,通常對應物體的輪廓或不同表面的交界。邊緣資訊在某些直接法 SLAM 與線特徵 SLAM 中扮演重要角色,尤其在特徵點稀少的人造環境(如空曠走廊、只有牆線的場景)中,線條反而是穩定的線索。

三、光流(Optical Flow):像素的運動

光流描述影像中亮度模式隨時間的移動——當相機或物體移動,畫面中的紋理會「流動」。想像雨滴打在車窗上向後流的景象,那就是光流的視覺化。光流可用來估計運動、追蹤特徵、偵測動態物體。它是直接法視覺里程計的核心,也是無人機定點懸停(光流定位,第 25 章)的關鍵技術。

四、深度估測(Depth Estimation)

除了前述的雙目視差與 RGB-D 主動測深,近年深度學習帶來了單目深度估測的突破:訓練神經網路從單張影像「猜」出每個像素的相對深度。這類模型(如 Depth

Anything,第 31 章會提到)利用了大量影像學到的先驗知識——它「知道」天空很遠、地板向遠方延伸、人比螞蟻大。雖然是「猜」的、且多半只有相對深度,但在許多應用中已足夠實用,是傳統幾何方法之外的重要補充。

這四種資訊並非互斥:現代視覺 SLAM 常混合使用——以特徵點做穩健的定位骨架、以光流做快速追蹤、以深度(不論來自雙目、RGB-D 或神經網路)重建稠密地圖。

9.5 視覺導航的優點與限制

把相機放進第二篇的感測器全景中(第 8 章表 8-2 已初步比較),本節深入視覺方法特有的優勢與軟肋。

視覺導航的獨特優勢

- 資訊最豐富:一張影像同時含形狀、紋理、顏色與語意。這是光達與 IMU 完全給不了的——光達只知道「三公尺外有東西」,相機卻能知道「那是一扇紅色的門」。這正是第 31 章語意導航建立在視覺之上的原因。
- 成本最低、最普及:相機比光達便宜一至兩個數量級,且無所不在,量產成熟。
- 被動感測、低功耗:不像光達需主動發射雷射,相機只接收環境光,省電且無互相干擾問題,特別適合無人機與微型機器人(第 24、25 章)。
- 輕巧:對載重與電力極度受限的無人機而言,「相機 + IMU」是最理想的組合。

視覺導航的先天限制

- 尺度不確定性:單目相機最根本的軟肋。如 9.1 節所述,投影除掉了深度,單目影像無法得知絕對尺寸,必須由 IMU、雙目基線或已知物體提供尺度。
- 依賴光線與紋理:這是視覺方法最致命的弱點。黑暗、逆光、過曝會讓相機「看不見」;白牆、純色地板等無紋理表面則找不到特徵點;快速運動造成的影像模糊也會使追蹤失敗。相較之下,光達自帶光源、不怕黑暗。

- 運算負擔:影像資料量大,特徵提取與匹配需要相當運算力,這也是邊緣運算平台(第 31 章 Jetson)的用武之地。
- 怕動態場景:畫面中走動的人、車會干擾以「環境靜止」為假設的視覺 SLAM。

觀念總結:視覺與光達的互補

光達:精確幾何、自帶光源不怕黑、但無語意、怕玻璃鏡面、較貴。

相機:豐富語意、便宜輕巧、但無可靠尺度深度、怕黑暗與無紋理。

兩者的弱點幾乎不重疊——這正是「光達建幾何骨架、相機貼語意標籤」的融合邏輯(第 31 章),

也再次印證第二篇的核心信念:沒有全能的感測器,只有恰當的組合。

9.6 從影像處理到 Visual SLAM

許多同學在修這門課之前,已經學過影像處理(Image Processing)——濾波、邊緣偵測、二值化、形態學運算等。本節說明這些基礎如何一步步通往視覺 SLAM,把兩門課的知識接起來。

知識的階梯

57. 影像處理(Image Processing):處理單張影像的像素——去雜訊、增強對比、偵測邊緣。這是「讓影像更清楚」的層次。
58. 特徵工程(Feature Extraction):從影像中抽出角點、描述子,把「一堆像素」濃縮成「少數可辨識的點」。這是第 19 章的內容。
59. 多視圖幾何(Multiple-View Geometry):分析同一場景的多張影像之間的幾何關係,從特徵點的移動反推相機運動與場景結構。這是第 20 章對極幾何的內容。

60. 視覺里程計(Visual Odometry, VO):把上一步逐幀串起來,持續估計相機軌跡——但會累積漂移。

61. 視覺 SLAM(Visual SLAM):在 VO 基礎上加入建圖、閉環偵測與全域最佳化,消除長期漂移,得到一致的軌跡與地圖。這是第六篇的主題。

這個階梯呼應了第 2 章 UZH 課程的鋪陳順序:影像形成 → 特徵 → 多視圖幾何 → 視覺里程計 → SLAM。每一階都是下一階的基礎,缺一不可。本書把影像處理視為先備知識,從第 19 章的特徵開始正式進入視覺 SLAM;但本章打下的相機模型與校正基礎,是理解後續一切的前提。

兩大流派:特徵法與直接法

視覺 SLAM 有兩條主要技術路線,此處先建立概念,第 19 章詳述:

- 特徵法(Feature-based):提取並匹配特徵點,以特徵點的幾何關係求解運動。穩健、成熟,ORB-SLAM 系列為代表。
- 直接法(Direct):不提取特徵,直接用像素亮度變化(光流概念)估計運動。在弱紋理場景有優勢,但對光線變化敏感,LSD-SLAM、DSO 為代表。

9.7 VLM / AI 視覺模型與語意導航初探

本節是全書最後一章(第 31 章 AI 語意導航)的預告。傳統視覺 SLAM 回答的是幾何問題:「相機在哪、環境長什麼形狀」;而 AI 視覺模型能進一步回答語意問題:「畫面裡有什麼、代表什麼意義」。這正是相機相對於光達最珍貴的價值——它承載了語意。

從幾何到語意

一台只做傳統 SLAM 的機器人,建出的地圖是「這裡有牆、那裡有障礙」;加上 AI 視覺辨識後,地圖能升級為「這是門、那是樓梯、那邊是滅火器」——這就是語意地圖(第 1 章、第 3 章提過)。這個升級讓機器人能執行人類語言等級的任務。

關鍵 AI 視覺模型(第 31 章詳述)

- YOLO:即時物件偵測,在影像中框出「這是人、那是椅子」,是校園場景辨識的主力(附件教材與大學課程的重點)。
- SAM(Segment Anything):把影像精確分割成一塊塊物件區域。
- Depth Anything:從單張影像估計深度(9.4 節提過)。
- CLIP / DINO:理解影像的語意內容,連結影像與文字。

VLM:讓機器人「看懂並聽懂」

視覺語言模型(VLM,Vision-Language Model)能同時理解影像與自然語言。這讓機器人可以接受口語任務:「請走到白板前」「避開地上的書包」「幫我找電梯」——VLM 分析相機畫面,理解語言指令,再把任務轉換成導航目標。

這是機器人導航正在發生的典範轉移:從「給定座標、走過去」進化為「聽懂人話、理解場景、自主決定怎麼走」。這也是為什麼本章要放在第二篇的最後——相機不只是又一個測距的感測器,它是機器人通往「理解世界」的窗口。第 31 章會完整展開這個激動人心的方向,包括如何蒐集與標註訓練資料、如何訓練校園場景辨識模型、以及如何整合 VLM 與 ROS 2。

至此,第二篇「定位與導航感測器」全部完成。同學已經認識了機器人的六大感官:里程計與 IMU 感知自身運動(第 7 章),無線電與衛星、地磁提供絕對參考(第 5、6 章),室內定位技術補足室內空缺(第 4 章),光達以雷射精確量測幾何(第 8 章),相機則帶來最豐富的視覺與語意(本章)。下一篇,我們將從「感測器給了什麼」轉向「如何用數學處理這些帶著雜訊的資料」——進入 SLAM 的數學基礎。

公式或流程

一、針孔投影完整方程

三維點 P_w 在世界座標中的位置,經外參(旋轉 R 、平移 t)轉到相機座標,再經內參 K 投影到像素座標,完整關係為:

$$s \cdot [u \ v \ 1]^T = K \cdot [R/t] \cdot [X_w \ Y_w \ Z_w \ 1]^T$$

其中 s 為尺度因子(等於該點在相機座標系中的深度 Z)。這條式子把本章的三個核心概念串在一起:內參 K (9.1)、外參 $[R|t]$ (相機在世界中的位姿,正是 SLAM 要求解的量)、以及尺度 s (9.5 的尺度不確定性來源)。整個視覺 SLAM,本質上就是在大量這樣的投影方程中,同時求解相機位姿 $[R|t]$ 與三維點座標。

二、雙目測深公式

雙目相機的深度計算是視覺測距最直觀的公式。設兩相機基線為 b 、焦距為 f (像素單位)、某點在左右影像的視差為 d (disparity,單位像素),則深度 Z 為:

$$Z = f \cdot b / d$$

這條式子透露三個重要事實:

- 視差 d 越大(近物),深度 Z 越小;視差趨近 0(遠物),深度趨近無窮——所以雙目對遠物測不準。
- 基線 b 越大,同樣距離的視差越大、測深越準、可測範圍越遠(這是 RealSense D455 加大基線以提升遠距精度的原因)。
- 深度誤差與距離平方成正比:距離加倍,誤差變四倍。這是所有被動視覺測深的共同宿命。

三、深度誤差隨距離成長

對視差量測誤差 Δd ,深度誤差近似為:

$$\Delta Z \approx (Z^2 / (f \cdot b)) \cdot \Delta d$$

式中 Z^2 說明了為何雙目與 RGB-D 相機都標榜「近距離精確、遠距離快速衰減」。
例如某相機在 1 公尺處誤差 1 公分,到 4 公尺處誤差就可能達 16 公分。這條式子是為
機器人選相機、定工作範圍的重要依據。

四、相機用於視覺 SLAM 的處理流程

圖 9-1 從相機到視覺 SLAM 的資料流

① 相機擷取影像(單目 / 雙目 / RGB-D)



② 去畸變:用校正得到的畸變係數矯正影像(9.2)



③ 特徵提取:偵測角點並計算描述子(9.4,第 19 章)



④ 特徵匹配 / 光流追蹤:找出相鄰幀的對應點



⑤ 幾何求解:由對應點與內參 K 推算相機運動(第 20 章)



⑥ 三角測量 / 深度:重建三維點(單目需運動、雙目用視差、RGB-D 直接給)



⑦ 後端最佳化 + 閉環:消除漂移,輸出軌跡與地圖(第 21、22 章)

案例

案例 9-1 手機人像模式:雙鏡頭的日常魔法

同學口袋裡的雙鏡頭手機,每天都在示範本章的原理。開啟人像模式拍照,主體清晰、背景柔美虛化——這個效果的背後,正是雙目測深:兩顆鏡頭以微小基線同時拍攝,手機計算每個像素的視差,估出深度圖,判斷「哪些像素是近的主體、哪些是遠的背景」,再對背景做模擬的散景虛化。

這個案例也暴露了雙目的限制。人像模式常在髮絲邊緣、鏤空物體、或純色背景「穿幫」——因為這些地方視差難以準確估計(無紋理、細節過小)。這正是 9.3 節所說「雙目怕無紋理表面」的生活寫照。近年手機改用「雙鏡頭視差 + AI 語意分割 + 神經網路深度估計」的混合方案來改善,恰恰對應 9.4 節提到的「多種視覺資訊混合使用」的趨勢。同學下次用人像模式時,不妨觀察它在哪些場景會出錯,這是最貼身的視覺深度實驗。

案例 9-2 火星直升機「機智號」:純視覺的星際導航

NASA 的火星直升機機智號(Ingeniuty)在火星上完成了數十次飛行,創下地外行星動力飛行的歷史。火星上沒有 GPS、沒有磁場可用的羅盤(火星幾乎沒有全球磁場),它如何知道自己在哪、飛得多穩?

答案是視覺慣性導航:機腹一顆朝下的單目相機以每秒數十幀拍攝地面,追蹤地表岩石與紋理的特徵點(9.4 節),配合 IMU 估計運動——這正是第 2 章提過 UZH 研究群擅長、也是第 25 章無人機導航的核心技術。火星稀薄的大氣(密度僅地球約 1%)讓飛行極不穩定,對導航的即時性與穩健性要求極高。機智號的成功證明了一件事:在完全沒有外部訊號的極端環境,一顆便宜的相機加上 IMU,就能撐起可靠的自主導航。這也是為什麼本書如此重視視覺慣性方法——它是機器人「離開基礎設施、真正自主」的關鍵。

案例 9-3 白牆的詛咒:視覺 SLAM 的經典失敗

一個在實驗室反覆上演的場景:同學興沖沖地讓搭載相機的機器人在教室跑視覺 SLAM,一切順利——直到機器人轉向一面乾淨的白牆,系統立刻「跟丟」,地圖崩壞。

原因就是 9.4 與 9.5 節反覆強調的:白牆上沒有任何特徵點,相機看到的每一處都長得一樣,系統無從判斷自己是否移動、移動了多少。這與第 7 章 IMU 初始化需要「運動激勵」、第 8 章光達怕玻璃,並列為 SLAM 三大經典「環境陷阱」。

解法呼應了全書的融合思想:加一顆 IMU,在視覺跟丟的數秒內靠慣性推算撐過去(第 7 章);或改用光達,牆面對光達反而是清晰的平面;或投射紋理(RGB-D 主動紅外的做法)。這個案例的教學價值在於,它讓同學親身體會「感測器的限制會直接變成系統的失敗」——理解限制,比讚嘆能力更重要。

實作活動

實作 9-1 相機校正與視覺深度初體驗

目的:親手完成相機校正、觀察畸變矯正效果、並體驗特徵點偵測與雙目測深。本實作以一般網路攝影機或手機即可進行,是第 29 章視覺 SLAM 實作的暖身。

器材

- 網路攝影機或手機相機、電腦(Python + OpenCV 環境)。
- 列印的棋盤格標定板(OpenCV 官網可下載,貼在硬板上保持平整)。
- (選用)雙目相機或 RGB-D 相機(如 RealSense),供進階測深。

第一部分:相機校正與去畸變

48. 用相機從不同角度、距離拍攝棋盤格 15 至 20 張,盡量涵蓋畫面各區域與各種傾斜。

49. 用 OpenCV 的 findChessboardCorners 偵測角點、calibrateCamera 求出內參矩陣 K 與畸變係數,記錄下來。
50. 觀察 K 矩陣:fx、fy 是否接近?cx、cy 是否約為影像寬高的一半?思考這些數字的物理意義。
51. 用求得的參數對一張含直線的影像(如磚牆、書架)做去畸變,把矯正前後並排比較,觀察邊緣彎曲的直線是否被拉直。
52. 檢查重投影誤差是否小於 1 像素;若偏大,增加校正影像數量或重拍再試。

第二部分:特徵點偵測

53. 用 OpenCV 的 ORB 偵測器,對一張教室或桌面的影像提取特徵點並畫出,觀察特徵點集中在哪裡(角點、紋理豐富處)、又缺席於哪裡(白牆、純色桌面)。
54. 對同一場景拍兩張(相機小幅移動),用特徵匹配找出對應點並連線,觀察匹配的正確與錯誤情形。
55. 故意拍一張大面積白牆的影像,觀察 ORB 幾乎偵測不到特徵點——親身驗證「白牆的詛咒」(案例 9-3)。

第三部分:深度體驗(依器材選做)

56. 若有 RGB-D 相機:讀取並顯示深度圖(以色階呈現遠近),把相機對準桌面與遠處,觀察深度值變化;再對準玻璃或鏡面,觀察深度資料的缺失或錯誤。
57. 若只有單目:把一個已知大小的物體(如 A4 紙)放在不同距離拍攝,記錄它在影像中的像素大小,驗證「距離加倍、像素尺寸減半」的針孔投影關係(9.1)。

討論問題

44. 去畸變前後,影像邊緣的直線變化有多明顯?若不做去畸變就拿去跑視覺 SLAM,你預期地圖會出現什麼問題?

45. ORB 特徵點在你的教室裡集中在哪些地方? 哪些區域幾乎沒有? 這對「機器人該往哪看才不會跟丟」有什麼啟示?
46. (有 RGB-D 者) 玻璃與鏡面的深度資料出了什麼問題? 這與第 8 章光達的弱點是否相同? 說明背後的原因。
47. 根據本章表 9-1, 若要為一台室內服務機器人選相機, 你會選單目、雙目還是 RGB-D? 搭配 IMU 嗎? 說明理由。

延伸挑戰(大學部)

其一: 用校正後的相機, 實作一個以 ArUco 標記估計相機位姿的程式(呼應第 4 章), 把相機相對標記的 6-DoF 位姿即時顯示出來——這串起了第 4 章與本章。其二: 若有雙目相機, 實作視差計算(OpenCV 的 StereoBM / StereoSGBM), 產生深度圖, 並用 9.6 節公式驗證某已知距離物體的測深精度, 觀察誤差是否隨距離平方成長。這份程式將直接銜接第 29 章。

延伸閱讀

55. R. Hartley and A. Zisserman, Multiple View Geometry in Computer Vision, 2nd ed., Cambridge University Press, 2004. (多視圖幾何的權威聖經, 第 20 章的主要參考)
56. R. Szeliski, Computer Vision: Algorithms and Applications, 2nd ed., Springer, 2022. (電腦視覺的標準教科書, 可免費取得, 含相機模型與校正完整章節)
57. Z. Zhang, "A Flexible New Technique for Camera Calibration," IEEE TPAMI, 2000. (張氏棋盤格校正法原始論文, 9.2 節依據)

58. OpenCV 官方文件之 Camera Calibration and 3D Reconstruction 教學
(docs.opencv.org).(附完整程式範例,實作首選)
59. L. Keselman et al., "Intel RealSense Stereoscopic Depth Cameras," CVPR
Workshops, 2017.(RealSense 系列深度相機的原理與規格,9.3 節依據)
60. 高翔、張濤,《視覺 SLAM 十四講》,電子工業出版社.(華文世界最受歡迎的視覺
SLAM 入門書,含相機模型到 SLAM 的完整脈絡)
61. UZH Robotics and Perception Group, Vision Algorithms for Mobile
Robotics 課程(第 2 章介紹).(影像形成到視覺 SLAM 的完整免費課程)

小結

35. 針孔相機模型描述三維點如何投影為二維影像;投影公式的分母是深度 Z ,意味著
深度在成像時被「除掉」,這是視覺 SLAM 尺度不確定性的幾何根源。內參矩陣
 K 是視覺 SLAM 最基礎的物件。
36. 真實鏡頭有徑向與切向畸變,會使直線變彎,破壞針孔模型假設;相機校正(如張氏
棋盤格法)量測內參 K 與畸變係數,去畸變是所有視覺 SLAM 的第一步。
37. 三種相機各以不同方式取得深度:單目靠運動三角測量但無絕對尺度、雙目靠左
右視差有絕對尺度但怕無紋理、RGB-D 主動投射取得稠密深度但怕強光與玻
璃。
38. 視覺 SLAM 從影像提取三種資訊:特徵點(角點,當作路標)、邊緣(輪廓)、光流(像
素運動);深度學習的單目深度估測是新興補充。現代系統常混合使用。
39. 相機資訊最豐富、成本最低、被動省電輕巧;但有尺度不確定性、嚴重依賴光線
與紋理(黑暗、白牆、模糊即失效)、運算負擔大、怕動態場景。其弱點與光達
幾乎互補。

40. 知識階梯為:影像處理 → 特徵提取 → 多視圖幾何 → 視覺里程計 → 視覺 SLAM;
視覺 SLAM 分特徵法(ORB-SLAM)與直接法(DSO)兩大流派。

41. 相機承載語意,是通往語意導航的窗口:YOLO、SAM、Depth Anything、CLIP
等 AI 模型與 VLM,能讓機器人從「知道形狀」進化到「理解場景、聽懂人話」,
這是第 31 章的主題。

至此,第二篇「定位與導航感測器」全部完成——我們認識了機器人的所有感官。
但感測器給的都是帶著雜訊的原始資料:里程計會漂移、光達有雜點、相機會誤匹配。
下一篇「SLAM 數學基礎」將回答一個根本問題:面對這些不完美的量測,機器人如何用
數學做出最合理的估測?我們將從座標系與剛體運動開始,一步步建立處理不確定性的
數學工具。

台灣自編大學教科書

@prof

《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第三篇 SLAM 數學基礎

第 10 章

座標系統與剛體運動

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

學習目標

第二篇認識了機器人的所有感官，但每個感測器都在「自己的座標系」中回報資料——相機說「前方兩公尺有東西」、光達說「我的左邊三公尺有牆」。要把這些資訊拼成一張一致的地圖，必須先學會在不同座標系之間精確轉換。本章是第三篇的第一站，也是整個 SLAM 數學的地基。學完本章之後，同學應能夠達成下列目標：

57. 說明世界、機器人、感測器三種座標系的關係，並解釋為何 SLAM 需要多個座標系。
58. 使用齊次座標把旋轉與平移統一表示為單一矩陣運算。
59. 寫出 2D 位姿 (x 、 y 、 θ) 的表示法與座標轉換公式。
60. 說明 3D 旋轉的三種表示法——旋轉矩陣、歐拉角、四元數——及其優缺點與適用場合。
61. 說明座標轉換的組合與反轉，理解 TF 座標樹的概念。
62. 說明 ROS 中 map、odom、base_link、camera_link 等標準座標框架的意義與關係。

核心概念

中文名詞	英文名詞	概念說明
座標系	Coordinate Frame	描述位置與方向的參考基準，SLAM 中同時存在多個。
位姿	Pose	位置加姿態，2D 為 (x 、 y 、 θ)，3D 為位置加三軸旋轉。
剛體運動	Rigid Body Motion	不改變物體形狀與大小的運動，只含旋轉與平移。

中文名詞	英文名詞	概念說明
齊次座標	Homogeneous Coordinates	多加一維的座標表示法，使旋轉與平移能以單一矩陣表示。
旋轉矩陣	Rotation Matrix	描述旋轉的正交矩陣，屬於特殊正交群 $SO(3)$ 。
歐拉角	Euler Angles	以滾轉、俯仰、偏航三個角度表示旋轉，直觀但有萬向鎖。
四元數	Quaternion	以四個數表示旋轉，無奇異點、運算高效，SLAM 的標準表示。
變換矩陣	Transformation Matrix	4×4 矩陣，同時包含旋轉 R 與平移 t ，屬於 $SE(3)$ 。
萬向鎖	Gimbal Lock	歐拉角在特定姿態下喪失一個自由度的奇異現象。
座標轉換樹	TF Tree	ROS 中管理各座標框架關係的樹狀結構。
里程計座標	odom frame	連續平滑但長期漂移的參考框架。
地圖座標	map frame	不漂移但可能跳變的全域參考框架。

10.1 世界座標、機器人座標、感測器座標

想像一台機器人在教室裡,一顆相機裝在它的頭部,拍到「正前方兩公尺處有一張椅子」。這句話裡藏著一個問題:「正前方」是相對於誰?相對於相機、相對於機器人的身體、還是相對於教室?這三者的答案都不一樣——機器人一轉身,相對相機的「前方」變了,但椅子相對教室的位置沒變。這就是為什麼 SLAM 必須同時處理多個座標系。

三個層次的座標系

- 世界座標系(World Frame):固定不動的全域參考,通常以機器人開機的起點或地圖的某個角落為原點。地圖上所有東西的「絕對位置」都以它為基準。它是 SLAM 最終想把一切統一過去的座標系。
- 機器人座標系(Robot / Body Frame):固定在機器人身上、隨機器人一起移動的座標系,原點通常設在底盤中心。在 ROS 中稱為 `base_link`。機器人的運動,本質上就是這個座標系相對世界座標系的移動。
- 感測器座標系(Sensor Frame):固定在各感測器上。相機有相機座標系、光達有光達座標系,原點在各感測器的量測中心。感測器直接量到的資料,都在自己的座標系中。

為什麼需要分這麼多層

關鍵在於:感測器裝在機器人上的位置是固定的(相機永遠在頭部前方 20 公分、光達永遠在底盤上方 15 公分),但機器人相對世界的位置卻時刻在變。把「固定的安裝關係」與「變動的機器人位姿」分開處理,數學才會清晰。

於是一筆感測器資料要進地圖,需要走兩步轉換:先從感測器座標系轉到機器人座標系(用固定的安裝關係,稱為外參,第 5 章提過),再從機器人座標系轉到世界座標系(用機器人當前位姿,這正是 SLAM 要估計的)。理解這個「感測器 → 機器人 → 世界」的轉換

鏈,是讀懂所有 SLAM 系統的前提。本章接下來就是要把「轉換」這件事的數學工具備齊。

10.2 齊次座標

座標轉換包含兩種動作:旋轉(轉個方向)與平移(移個位置)。麻煩的是,旋轉是「乘法」(矩陣乘向量)、平移是「加法」(向量加向量),兩者形式不同,串接多次轉換時會變得很囉嗦。齊次座標(Homogeneous Coordinates)是一個優雅的技巧,能把兩者統一成單一的矩陣乘法。

多加一維的魔法

做法很簡單:在原本的座標後面多補一個 1。一個 2D 點(x、y)寫成(x、y、1),一個 3D 點(x、y、z)寫成(x、y、z、1)。

這樣做的好處是,平移也能寫進矩陣裡了。以 2D 為例,「旋轉角度 θ 再平移(tx、ty)」可以合併成單一的 3×3 矩陣乘法:

$$[x'; y'; 1] = [\cos\theta \quad -\sin\theta \quad tx; \sin\theta \quad \cos\theta \quad ty; 0 \quad 0 \quad 1] \cdot [x; y; 1]$$

這個 3×3 矩陣稱為 2D 的變換矩陣(或齊次變換矩陣)。左上角 2×2 是旋轉、右上角是平移、底下補一列(0 0 1)。原本要「先乘旋轉、再加平移」兩個步驟,現在一次矩陣乘法就完成。

為什麼這件事很重要

- 串接變換變成連乘:機器人經過多段移動,或資料經過「感測器→機器人→世界」多層轉換,只要把各段的變換矩陣依序相乘即可,不必分別處理旋轉與平移。
- 反向變換就是反矩陣:要把世界座標轉回機器人座標,只需求變換矩陣的反矩陣。

- 程式實作統一:所有座標處理都是矩陣乘法,程式碼簡潔、不易出錯,也便於 GPU 加速。

3D 的情況完全類似,只是變成 4×4 矩陣:左上角 3×3 是旋轉矩陣 R 、右上角 3×1 是平移向量 t 、底下補一行 $(0 \ 0 \ 0 \ 1)$ 。這個 4×4 矩陣是 SLAM 中最核心的數學物件之一,它同時裝著「機器人朝哪、在哪」的完整位姿資訊。

觀念釐清:變換矩陣的兩種讀法

同一個變換矩陣 T ,可以有兩種解讀,初學者常混淆:

- ① 座標轉換:把「同一個點」在 A 座標系的座標,換算成在 B 座標系的座標。
- ② 剛體運動:把「一個點」實際移動、旋轉到新位置(在同一座標系中)。

兩者數學形式相同,但意義相反(把座標系轉一個方向,等於把點反向轉)。

SLAM 中大多是第①種——我們不是真的移動世界,而是換個座標系描述同一件事。

10.3 2D 位姿 : x 、 y 、 θ

掃地機器人、教學小車都在平面上運動,只需 2D 位姿就足以描述。這是理解 3D 的最佳起點,因為 2D 只有一個旋轉自由度,一切都很直觀。

三個數描述一切

平面上一台機器人的位姿,用三個數就能完整描述:

- x 、 y :機器人在世界座標系中的位置。
- θ :機器人的朝向角(航向),即機器人的「正前方」與世界座標 X 軸的夾角。

這正是第 3 章航位推算、第 14 章 Gmapping 粒子所估計的量。每一個粒子、每

一個位姿節點,在 2D SLAM 中就是這樣一組 $(x、y、\theta)$ 。

2D 座標轉換的實例

假設光達在機器人座標系中量到一個點 $p_{\text{robot}} = (a、b)$ (機器人前方 a 、左方 b),而機器人在世界座標系中的位姿是 $(x、y、\theta)$,那麼這個點在世界座標系中的位置為:

$$x_{\text{world}} = x + a \cdot \cos\theta - b \cdot \sin\theta$$

$$y_{\text{world}} = y + a \cdot \sin\theta + b \cdot \cos\theta$$

這正是第 8 章「把一圈光達距離值畫進地圖」時,每個點都要做的計算。用齊次座標寫,就是前一節那個 3×3 變換矩陣乘上 $(a、b、1)$ 。同學可以驗證:當 $\theta = 0$ (機器人朝正東)、位姿在原點時,公式退化為 $x_{\text{world}} = a、y_{\text{world}} = b$,符合直覺。

角度的兩個實務陷阱

- 角度環繞(Angle Wrapping): θ 是週期性的,+180 度與 -180 度是同一個方向。計算兩個角度的差時,必須把結果規範回 $(-\pi, \pi]$ 區間,否則會出現「轉了 350 度」這種本該是「反向轉 10 度」的荒謬結果。這是 2D SLAM 程式最常見的 bug 之一。
- 弧度與角度:數學運算一律用弧度(radian),但人類閱讀習慣用度(degree)。程式中務必統一單位,混用會造成難以察覺的錯誤。

10.4 3D 位姿：位置與姿態

無人機、機器狗、手持相機在三維空間中運動,需要完整的 3D 位姿。3D 位置很簡單,就是 $(x、y、z)$ 三個數;真正複雜的是 3D 姿態——描述「朝哪」需要三個旋轉自由度,而表示這三個自由度的方法,竟有好幾種,各有優劣。這是本章的數學重點。

六個自由度

剛體在三維空間中共有六個自由度(6-DoF):三個平移(沿 $X、Y、Z$ 移動)加三個旋

轉(繞 X、Y、Z 轉動)。第 7 章、第 22 章反覆出現的「6-DoF 位姿」,指的就是這六個數。位置的三個自由度沒有爭議,以下三節專注於旋轉的三種表示法。

為什麼旋轉這麼麻煩?因為旋轉不是普通的量。兩個平移可以隨意相加、順序無關;但兩個旋轉的合成與順序有關——先繞 X 轉 90 度再繞 Y 轉 90 度,和先繞 Y 再繞 X,結果完全不同(同學可拿手機實際轉轉看)。這種「不可交換性」使得旋轉的數學遠比平移微妙,也是第 2 章 MIT VNAV 課程強調「流形上的最佳化」的原因——旋轉構成的空間是彎曲的,不是平坦的向量空間。

10.5 旋轉矩陣、歐拉角、四元數

這是本章最重要的一節。3D 旋轉有三種主要表示法,每一種都在 SLAM 系統中出現,同學必須理解它們各自的角色。

一、旋轉矩陣(Rotation Matrix):理論的基石

用一個 3×3 的矩陣 R 表示旋轉。它把一個向量旋轉到新方向,運算就是矩陣乘向量。旋轉矩陣有嚴格的數學性質:它是正交矩陣(各列互相垂直且長度為 1)、行列式為 +1,這類矩陣構成的集合稱為特殊正交群 $SO(3)$ 。

- 優點:數學性質完美、直接可用於座標轉換、無奇異點、旋轉合成就是矩陣相乘。
- 缺點:用 9 個數表示只有 3 個自由度的旋轉,冗餘度高;儲存與最佳化時,還必須維持「正交」這個約束,不能自由地對 9 個數做加減(否則就不再是合法旋轉)。

二、歐拉角(Euler Angles):最直觀但有陷阱

用三個角度——滾轉 φ 、俯仰 θ 、偏航 ψ (第 7 章介紹過)——表示旋轉。這是最貼近人類直覺的方式,「機頭上仰 10 度、右傾 5 度、朝向東方」一聽就懂。

- 優點:直觀、只用 3 個數(無冗餘)、便於人類理解與介面顯示。
- 缺點:有萬向鎖(Gimbal Lock)——當俯仰角接近 ± 90 度時,滾轉與偏航的旋轉軸重合,喪失一個自由度(第 7 章詳述);此外,旋轉合成不能直接相加,且同一旋轉可能有多組角度表示(週期性),數值最佳化時容易出問題。

因此歐拉角適合「給人看」(介面顯示、除錯),但不適合「給機器算」(內部計算)。

這是一個重要的工程分工。

三、四元數(Quaternion):SLAM 的實戰標準

四元數用四個數(w 、 x 、 y 、 z)表示旋轉,可以理解為「繞某個軸轉某個角度」的一種緊湊編碼。它初看不直觀,卻是現代 SLAM、無人機、遊戲、動畫內部一致採用的旋轉表示。

- 優點:無萬向鎖(沒有奇異點)、只用 4 個數(冗餘極小)、旋轉合成只需四元數乘法(運算高效)、數值穩定、便於插值(平滑地從一個姿態轉到另一個)。
- 缺點:不直觀,人難以直接讀懂(w 、 x 、 y 、 z)代表什麼方向;需要維持「單位長度」的約束(但只有一個約束,遠比旋轉矩陣的正交約束簡單)。

實務上的分工非常清楚:內部計算與儲存用四元數、介面顯示與人工判讀轉成歐拉角、座標轉換運算轉成旋轉矩陣。三者可以互相轉換,ROS 的 tf2 與各種數學函式庫都內建轉換函式。同學不必背誦四元數的乘法規則,但必須理解「為什麼實務都用它」——答案就是:它同時避開了旋轉矩陣的冗餘與歐拉角的萬向鎖。

表示法	數字個數	奇異點	主要優點	主要用途
旋轉矩陣	9(3×3)	無	座標轉換直接、理論完美	座標變換運算
歐拉角	3	有(萬向鎖)	最直觀、便於人讀	介面顯示、除錯

表示法	數字個數	奇異點	主要優點	主要用途
四元數	4	無	緊湊、高效、便於插值	內部計算與儲存

表 10-1 三種 3D 旋轉表示法比較

10.6 座標轉換與 TF Tree

有了變換矩陣這個工具,現在來看真實機器人系統如何管理眾多座標系之間的關係。

答案是一個樹狀結構——TF 座標樹。

變換的組合與反轉

先複習兩個關鍵運算,它們是 TF 樹運作的基礎:

- 組合(Composition):若已知「A 到 B 的變換」與「B 到 C 的變換」,把兩個變換矩陣相乘,就得到「A 到 C 的變換」。這讓我們能把感測器資料一路轉換到世界座標。
- 反轉(Inverse):若已知「A 到 B 的變換」,求它的反矩陣,就得到「B 到 A 的變換」。

有了這兩個運算,只要知道任意兩個相鄰座標系之間的變換,就能算出任意兩個座標系之間的關係——這正是 TF 樹的核心思想。

TF Tree:座標系的家譜

在一台真實機器人上,座標系可能有十幾個:世界、地圖、里程計、底盤、每個輪子、光達、相機、機械臂的每一節.....ROS 用一個樹狀結構(TF Tree,座標轉換樹)來管理它們的關係。

- 每個座標系是樹上的一個節點。
- 每個父子節點之間,記錄著一個變換(父到子)。

- 查詢任意兩個座標系的關係時,系統自動沿著樹路徑把沿途的變換組合(相乘)起來。

這個設計的巧妙之處在於:每個變換只需由「最了解它的那個模組」負責發布。光達驅動發布「底盤到光達」的固定變換(來自安裝量測)、SLAM 系統發布「地圖到底盤」的變換(來自定位估計)、里程計發布「里程計到底盤」的變換。各模組各司其職,TF 樹把它們自動整合成完整的座標關係網。第 28 章實作時,同學會親手設定這棵樹。

時間戳記:一個關鍵細節

TF 不只管「空間關係」,還管「時間」。因為機器人在動,每個變換都是隨時間變化的,TF 系統為每個變換都標上時間戳記,查詢時可以問「在 0.1 秒前那一刻,相機相對地圖在哪」。這對感測器融合至關重要——相機與光達的資料到達時間不同,必須用正確時刻的變換來對齊,否則高速運動時會產生明顯誤差。這呼應了第 5 章融合中「時間同步」的工程課題。

10.7 ROS 中的 map、odom、base_link、camera_link

ROS 為行動機器人定義了一套標準座標框架命名與階層,幾乎所有 ROS 導航系統都遵循它。理解這套慣例,是讀懂與實作 SLAM 系統的基本功。

標準框架階層

從上到下的標準鏈是:map → odom → base_link → 各感測器框架。

圖 10-1 ROS 標準座標框架階層

map(地圖座標系):全域固定、不漂移,但會跳變

↓ (由 SLAM / 定位系統發布)

odom(里程計座標系):連續平滑,但長期漂移

↓ (由里程計發布)

base_link(機器人本體):固定在機器人底盤中心

↓ (由固定安裝關係發布,即外參)

laser / camera_link / imu_link(各感測器座標系)

為什麼要 map 和 odom 兩層

這是整套設計最精妙、也最容易困惑的地方。為什麼不直接用一個座標系,要分成 map 與 odom 兩層?因為它們各自解決不同的需求,而且是互補的:

- odom(里程計座標系):由里程計與 IMU 積分得到。它的特性是連續、平滑、不跳變——非常適合短期的運動控制與避障。但如第 3、7 章所述,它會長期漂移,越走越不準。
- map(地圖座標系):由 SLAM 或定位系統(如 AMCL)得到。它的特性是長期正確、不漂移——因為它靠比對地圖來校正。但它會跳變:當 SLAM 突然發現「我其實在這裡」而修正位置時,機器人在 map 中的座標會瞬間跳動一下。

兩層並存的智慧在於:讓需要「平滑」的模組(如避障控制)用 odom,讓需要「準確」的模組(如全域路徑規劃)用 map。SLAM 系統實際發布的,是「map 到 odom」的變換——它不斷微調這個變換來補償里程計的漂移,而 base_link 相對 odom 的運動仍然平滑。這個設計讓「平滑」與「準確」兩個看似衝突的需求,優雅地共存。

感測器框架與外參

base_link 之下的每個感測器框架(laser、camera_link、imu_link 等),相對 base_link 的變換是固定的,由感測器在機器人上的安裝位置決定,也就是第 5 章提過的

外參。這些變換通常寫在機器人的 URDF 描述檔中(第 28 章),或由靜態變換發布器發布。準確量測這些外參,是系統精度的基礎——安裝位置差幾公分或幾度,建出的地圖就會模糊或錯位。

至此,同學已經備齊了 SLAM 的第一組數學工具:如何在座標系之間精確地轉換位姿。但真實世界的量測都帶著雜訊——里程計會滑、光達有誤差、相機會誤匹配。下一章「機率、雜訊與狀態估測」,將教我們如何用機率的語言描述這些不確定性,並在其中做出最合理的估計。這是 SLAM 從「幾何」邁向「估測」的關鍵轉折。

公式或流程

一、3D 齊次變換矩陣

3D 剛體運動以 4×4 齊次變換矩陣 T 表示,同時包含旋轉 $R(3 \times 3)$ 與平移 $t(3 \times 1)$:

$$T = [R \ t; 0 \ 1] \quad (\text{左上 } 3 \times 3 \text{ 為 } R, \text{ 右上 } 3 \times 1 \text{ 為 } t, \text{ 左下為 } 0, \text{ 右下為 } 1)$$

把 A 座標系中的點 p_A (以齊次座標表示)轉到 B 座標系:

$$p_B = T_{BA} \cdot p_A$$

這裡 T_{BA} 讀作「從 A 到 B 的變換」。下標的書寫順序是 SLAM 中最容易出錯的地方,建議固定一種慣例並貫徹到底。

二、變換的組合與反轉

組合:把 $A \rightarrow B$ 與 $B \rightarrow C$ 的變換串接成 $A \rightarrow C$:

$$T_{CA} = T_{CB} \cdot T_{BA}$$

注意相乘順序:相鄰的下標要對齊(B 與 B 相鄰)。反轉:

$$T_{AB} = (T_{BA})^{-1}$$

齊次變換矩陣的反矩陣有簡便公式,不需做一般的 4×4 反矩陣運算:

$$T^{-1} = [R^T \quad -R^T t; 0 \quad 1]$$

利用旋轉矩陣的正交性(反矩陣等於轉置),反轉變得非常高效。

三、2D 位姿複合(里程計串接)

機器人從位姿 $p_1 = (x_1, y_1, \theta_1)$ 經一段相對運動 $(\Delta x, \Delta y, \Delta \theta)$ 在 p_1 的機體座標系中) 後,新位姿為:

$$x_2 = x_1 + \Delta x \cdot \cos \theta_1 - \Delta y \cdot \sin \theta_1$$

$$y_2 = y_1 + \Delta x \cdot \sin \theta_1 + \Delta y \cdot \cos \theta_1$$

$$\theta_2 = \text{normalize}(\theta_1 + \Delta \theta) \quad (\text{normalize 表示規範回 } (-\pi, \pi])$$

這是第 3 章航位推算、第 14 章 Gmapping 運動更新的具體實現。normalize 步驟即 10.3 節強調的角度環繞處理,不可省略。

四、感測器資料進地圖的完整流程

圖 10-2 一筆感測器量測進入世界地圖的轉換鏈

① 感測器量測:點 p 在感測器座標系(如 camera_link)

↓ × $T(\text{base_link} \leftarrow \text{camera_link})$ 外參,固定

② 點 p 在機器人座標系(base_link)

↓ × $T(\text{odom} \leftarrow \text{base_link})$ 里程計

③ 點 p 在里程計座標系(odom)

↓ × $T(\text{map} \leftarrow \text{odom})$ SLAM 修正

④ 點 p 在世界地圖座標系(map)→ 寫入地圖

整條鏈就是多個變換矩陣的連乘,TF 樹自動完成這個計算。

案例

案例 10-1 阿波羅 11 號的萬向鎖驚魂

萬向鎖不是抽象的數學名詞,它曾在人類登月任務中造成真實危機。阿波羅太空船的慣性導航平台用三個機械環(萬向架)支撐,以歐拉角的方式追蹤太空船姿態。當三個環接近共面時,就會發生萬向鎖——喪失一個旋轉自由度,導航平台可能失去姿態基準。

工程師深知這個風險,在儀表板上設計了警示,並規劃飛行姿態盡量避開危險區域。太空人麥可·柯林斯曾半開玩笑地抱怨這個限制。事後來看,這正是「用歐拉角表示姿態」的先天缺陷在極端任務中的體現。現代太空船、無人機、機器人早已改用四元數,正是為了根除這個問題。

這個案例給同學的啟示是:表示法的選擇不是純粹的數學品味,它有真實的工程後果。10.5 節看似枯燥的三種旋轉表示法比較,背後是無數工程師用教訓換來的智慧——這也是為什麼本書堅持在數學章節中講清楚「為什麼這樣選」,而不只是「怎麼算」。

案例 10-2 RViz 裡的座標樹:讓抽象變具體

第一次接觸 TF 樹的同學,常覺得它抽象難懂。但在 ROS 的視覺化工具 RViz 中,座標樹會變成看得見的東西:每個座標系顯示為一組紅綠藍的小軸(對應 X、Y、Z),當機器人移動,base_link 的軸就跟著動;當光達安裝在機器人前方,laser 的軸就固定顯示在 base_link 前方一小段距離。

更有教育意義的是「觀察錯誤」。如果外參設錯了(例如把光達的安裝方向填反),RViz 中光達掃描的點雲就會與實際環境「錯位」——牆明明在前方,點雲卻畫在後

方或側面。同學透過調整外參、即時觀察點雲是否貼合環境,能非常直觀地理解座標轉換的意義。這是把 10.1 到 10.7 節所有抽象概念一次串起來的最佳實驗,第 28 章實作會大量用到。

大學課堂上一個常見的除錯口訣是:「地圖歪了先看 TF」。許多 SLAM 建圖失敗,根源不在演算法,而在座標框架設定錯誤——外參填錯、時間戳記沒對齊、TF 樹斷裂。理解本章,能幫同學省下無數除錯的冤枉路。

案例 10-3 機械手臂的正向運動學:同一套數學的另一個舞台

本章的座標變換數學,不只用於行動機器人,也是機械手臂的核心。一支六軸機械手臂,從基座到夾爪之間有六個關節,每個關節的轉動都是一個旋轉變換。把基座到第一節、第一節到第二節……一路到夾爪的變換矩陣依序相乘,就能算出「給定六個關節角度,夾爪在空間中的位姿」——這稱為正向運動學(Forward Kinematics)。

這正是 10.6 節「變換組合就是矩陣連乘」的直接應用。同學若在大學同時修習機械手臂相關課程,會發現兩門課用的是同一套齊次變換數學——這並非巧合,而是因為「剛體在空間中如何運動」是機器人學共通的語言。學好本章,等於同時打下了行動機器人與機械手臂的數學基礎。這也呼應第 2 章各校課程「感測器→數學→系統」的共同架構:數學是所有機器人技術的公分母。

實作活動

實作 10-1 座標轉換動手算與 TF 樹觀察

目的:透過紙筆計算與程式驗證,掌握 2D 座標轉換;並在 ROS 中觀察真實的 TF 樹,把抽象概念變具體。本實作分紙筆、程式、ROS 三部分,適合不同深度的教學。

第一部分:紙筆計算(全體)

62. 一台機器人在世界座標系中位姿為($x=3, y=2, \theta=90^\circ$)。它的光達在機體座標系中量到一個點(前方 $a=2$, 左方 $b=0$)。用 10.3 節公式手算這個點的世界座標,並在方格紙上畫圖驗證答案合理。
63. 同一台機器人,光達量到另一點($a=1, b=1$)。手算其世界座標。畫圖確認。
64. 角度環繞練習:機器人朝向 $\theta_1=170^\circ$,右轉 $\Delta\theta=30^\circ$ 。直接相加得 200° ,把它規範回 $(-180^\circ, 180^\circ]$ 應是多少?說明為何不處理會出問題。

第二部分:Python 驗證(全體)

65. 用 numpy 建立 10.2 節的 3×3 齊次變換矩陣,把第一部分的計算改用矩陣乘法完成,驗證結果一致。
66. 實作變換的組合:給定「世界→機器人」與「機器人→光達」兩個變換,相乘得到「世界→光達」,並驗證。
67. 實作反轉:求「世界→機器人」變換的反矩陣,驗證它等於「機器人→世界」變換(用 10. 公式的簡便法與 numpy 直接求逆兩種方式比較)。
68. (大學部)實作歐拉角、旋轉矩陣、四元數三者的互相轉換(可用 `scipy.spatial.transform.Rotation`),取一個俯仰角接近 90° 的姿態,觀察歐拉角表示如何出現異常,親身驗證萬向鎖。

第三部分:ROS TF 樹觀察(有 ROS 環境者)

69. 啟動 TurtleBot3 模擬(第 27 章)或任一含 TF 的 ROS 系統,用 `view_frames` 或 `rqt_tf_tree` 工具產生 TF 樹的圖,找出 `map`、`odom`、`base_link` 與各感測器框架的階層關係。
70. 在 RViz 中顯示 TF,移動機器人,觀察哪些座標軸跟著動(`base_link` 及其子框架)、哪些不動(`map`)。

71. 用 tf2 的查詢工具,查詢「camera_link 相對 map」的變換,並在機器人移動前後各查一次,觀察數值變化。

討論問題

58. 第一部分中,機器人朝向 90° (正北)時,它機體的「前方」對應世界座標的哪個方向?你的計算結果符合直覺嗎?
59. 為什麼 SLAM 系統發布的是「map→odom」變換,而不是直接發布「map→base_link」?這樣設計對「平滑」與「準確」的兼顧有何幫助?
60. (大學部)你在萬向鎖實驗中觀察到什麼異常?如果無人機的飛控用歐拉角而非四元數,在做翻滾特技時可能發生什麼事?
61. 若一台機器人建出的地圖總是「歪斜」或「牆的點雲對不上」,依本章知識,你會優先檢查哪些地方?

延伸挑戰(大學部)

撰寫一個 ROS 2 節點,發布一個自訂感測器相對 base_link 的靜態變換(故意設一個已知的旋轉與平移),在 RViz 中顯示該感測器的模擬點雲,調整外參數值並即時觀察點雲位置變化,直到點雲貼合預期位置。這個練習直接對應第 28 章設定機器人 TF 的實務,能為 2D SLAM 小車實作打下座標基礎。

延伸閱讀

48. 高翔、張濤,《視覺 SLAM 十四講》第 3 講(三維空間剛體運動)、第 4 講(李群與李代數),電子工業出版社.(華文世界最清楚的 SLAM 旋轉數學入門)
49. K. M. Lynch and F. C. Park, Modern Robotics: Mechanics, Planning, and Control, Cambridge University Press, 2017.(剛體運動與座標變換的權威教科書,有免費電子版與線上課程)

- 50.J. Solà, J. Deray, and D. Atchuthan, "A Micro Lie Theory for State Estimation in Robotics," arXiv, 2018.(旋轉群與流形最佳化的友善入門,銜接第 12 章)
- 51.ROS 官方文件之 tf2 教學(docs.ros.org).(TF 座標樹的實作指引,附完整範例)
- 52.REP 105: Coordinate Frames for Mobile Platforms(ros.org/reps).(ROS 標準座標框架 map / odom / base_link 的官方定義文件)
- 53.3Blue1Brown, "Quaternions and 3D rotation" 系列影片(YouTube).(以絕佳動畫直觀解釋四元數,強烈推薦)
- 54.R. Siegwart et al., Introduction to Autonomous Mobile Robots, 2nd ed., MIT Press, 2011.(第 2、3 章含座標系與運動學的機器人學脈絡)

小結

- 62.SLAM 同時處理世界、機器人、感測器三層座標系;感測器安裝關係固定(外參)、機器人位姿變動(SLAM 要估的量),一筆資料進地圖需經「感測器→機器人→世界」的轉換鏈。
- 63.齊次座標多補一維,把旋轉(乘法)與平移(加法)統一為單一矩陣乘法;2D 用 3×3 、3D 用 4×4 變換矩陣,使串接變換成為連乘、反轉成為求反矩陣。
- 64.2D 位姿以 $(x、y、\theta)$ 三個數描述,座標轉換用含 $\cos\theta$ 、 $\sin\theta$ 的公式;實務上務必處理角度環繞(規範回 $-\pi$ 到 π)並統一弧度與角度單位。
- 65.3D 剛體有六個自由度(三平移三旋轉);旋轉的合成不可交換,構成的空間是彎曲的,這是旋轉數學複雜與「流形最佳化」的根源。
- 66.3D 旋轉三種表示法各有分工:旋轉矩陣(9 數、無奇異、用於座標轉換)、歐拉角(3 數、直觀但有萬向鎖、用於人機介面)、四元數(4 數、無奇異、高效、用於內部計算與儲存)。

67. TF 樹以樹狀結構管理眾多座標系,每個變換由最了解它的模組發布,查詢時自動沿路徑連乘;變換附時間戳記以支援感測器融合的時間對齊。

68. ROS 標準框架 $\text{map} \rightarrow \text{odom} \rightarrow \text{base_link} \rightarrow \text{感測器:odom}$ 連續平滑但漂移、 map 準確但跳變;SLAM 發布「 $\text{map} \rightarrow \text{odom}$ 」變換來補償漂移,讓「平滑」與「準確」兩需求優雅共存。

本章備齊了 SLAM 的第一組數學工具——確定性的座標轉換。但真實世界充滿不確定性:里程計會滑、光達有雜點、相機會誤匹配。下一章「機率、雜訊與狀態估測」將引入機率的語言,教機器人如何描述「我大概在這裡,但不太確定」,並在雜訊中做出最合理的估計。這是 SLAM 從幾何邁向估測的關鍵一步,也是貝氏濾波、卡爾曼濾波與粒子濾波的共同起點。

台灣自編大學教科書

@prof

《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第 11 章

機率、雜訊與狀態估測

編者：李世淵 (機器人叫獸)

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

學習目標

上一章我們學會了在座標系之間精確轉換——那是「確定性」的數學。但真實世界沒有這麼乾淨:里程計會打滑、光達有雜點、相機會誤匹配。本章引入機率的語言,教機器人如何說出「我大概在這裡,但不太確定」,並在雜訊中做出最合理的估計。這是 SLAM 從幾何邁向估測的關鍵轉折,也是全書數學的核心。學完本章之後,同學應能夠達成下列目標:

- 63. 說明 SLAM 為何必須以機率方式處理不確定性,而非追求單一精確答案。
- 64. 解釋高斯分布的均值與變異數的意義,並理解共變異數如何描述二維以上的不確定性。
- 65. 說明運動模型與觀測模型如何以機率描述機器人的預測與感知。
- 66. 說明貝氏濾波「預測—更新」兩步驟的遞迴架構,理解它是所有濾波器的共同骨架。
- 67. 說明卡爾曼濾波器的運作,並解釋卡爾曼增益如何權衡預測與觀測。
- 68. 說明 EKF 如何處理非線性,以及 EKF-SLAM 的基本概念與限制。
- 69. 說明粒子濾波與蒙地卡羅定位的原理,並比較它與卡爾曼家族的差異。

核心概念

中文名詞	英文名詞	概念說明
機率分布	Probability Distribution	描述隨機變數各種可能取值的機率。
高斯分布	Gaussian Distribution	鐘形的常態分布,以均值與變異數完整描述。
均值	Mean	分布的中心,代表最可能的估計值。

中文名詞	英文名詞	概念說明
變異數	Variance	分布的離散程度,代表不確定性的尺寸。
共變異數	Covariance	描述多維隨機變數之間的相關性與聯合不確定性。
運動模型	Motion Model	描述機器人執行控制後,狀態如何變化(含雜訊)。
觀測模型	Observation Model	描述在某狀態下,感測器會量到什麼(含雜訊)。
貝氏濾波	Bayes Filter	以預測與更新兩步驟遞迴估計狀態的通用架構。
卡爾曼濾波	Kalman Filter	線性高斯系統下的最佳遞迴估測器。
卡爾曼增益	Kalman Gain	決定相信預測還是觀測的權重。
擴展卡爾曼濾波	EKF	將非線性系統線性化後套用卡爾曼濾波。
粒子濾波	Particle Filter	以大量樣本近似任意分布的非參數濾波器。
蒙地卡羅定位	MCL	以粒子濾波在已知地圖中定位的方法,即 AMCL。

11.1 為什麼 SLAM 一定要處理不確定性

回想第 7 章那個令人印象深刻的實驗:一台完全靜止的裝置,靠 IMU 積分算出的「位置」在 60 秒內漂移了數公尺。再想第 4 章的 Wi-Fi 定位、第 5 章的 GPS 跳點、第 9 章相機的誤匹配——沒有任何一個感測器給的是「真值」,它們給的都是「帶著雜訊的猜測」。

面對這個現實,有兩種態度。天真的做法是:假裝感測器是準的,直接用它的讀值。這正是航位推算失敗的原因——把每個帶誤差的量測當成真值累加,誤差就永無止境地累積。

成熟的做法是:承認每個量測都不確定,並量化這個不確定性,然後在多個不確定的資訊之間做最佳的權衡。這就是機率機器人學(Probabilistic Robotics)的核心思想,也是現代 SLAM 的數學骨幹。

關鍵觀念:估計的是「分布」而非「數值」

這是全章最重要的觀念轉變。傳統思維認為「機器人在(3.5, 2.1)」;機率思維則認為「機器人最可能在(3.5, 2.1)附近,但也可能在周圍一個橢圓範圍內,越靠近中心可能性越高」。

換句話說,SLAM 估計的不是一個點,而是一整個機率分布——它同時告訴我們「最佳猜測」與「這個猜測有多可信」。這個「有多可信」的資訊至關重要,因為它決定了下一步該多相信這個估計。第 1 章給出的 SLAM 機率定義 $p(x, m \mid z, u)$,正是在說「給定所有觀測與控制,狀態的機率分布長什麼樣」。

直觀比喻:蒙眼走路的人

想像你蒙著眼在熟悉的房間走動。每走一步,你「大概」知道自己前進了多少(運動),但不太確定——可能多一點少一點(運動雜訊)。這時你對自己位置的把握越來越模糊。

突然你的手摸到了牆角(觀測)。這個明確的地標,讓你對位置的把握瞬間變清晰。

SLAM 就是在做這件事:走路時不確定性擴大(預測),摸到地標時不確定性收縮(更新)。本章要學的,就是如何用數學精確地描述「模糊」與「變清晰」這兩個動作。

11.2 高斯分布、均值、變異數、共變異數

要用數學描述不確定性,首先需要一種「分布」。SLAM 中最常用的是高斯分布(Gaussian Distribution,又稱常態分布),因為它有兩個無可取代的優點:只需兩個參數就能完整描述、而且在各種運算下仍保持高斯形式(數學上極為友善)。

一維高斯:鐘形曲線

一維高斯分布就是熟悉的鐘形曲線,由兩個參數完整決定:

- 均值 μ (mu):曲線的中心,代表「最可能的值」,也就是我們的最佳估計。
- 變異數 σ^2 (sigma 平方):曲線的胖瘦,代表不確定性的大小。變異數小,曲線又高又窄,表示很有把握;變異數大,曲線又矮又寬,表示很不確定。

標準差 σ 是變異數的平方根,與量測值同單位,更直觀。「機器人在 3.5 公尺處,標準差 0.2 公尺」意思是:最佳估計是 3.5,而真值有約 68% 的機率落在 3.3 到 3.7 之間(高斯分布的「一個標準差」範圍)。

多維高斯:從點到橢圓

機器人的位置是二維的(x 、 y),不確定性也就變成二維。此時均值是一個點(最可能的位置),而不確定性用一個共變異數矩陣(Covariance Matrix)描述。

共變異數矩陣的幾何意義非常直觀:它對應一個不確定性橢圓。橢圓的中心是均值、大小反映不確定程度、形狀與方向反映「哪個方向比較不確定」。

- 對角線元素:各軸自己的變異數(x 方向多不確定、 y 方向多不確定)。
- 非對角線元素(共變異數): x 與 y 的相關性。若為正,表示「 x 偏大時 y 也傾向偏大」,橢圓會傾斜。

舉個 SLAM 中的實例:一台機器人沿走廊直行,它對「走了多遠」(沿走廊方向)很不確定(誤差累積),但對「有沒有偏離走廊中線」(垂直方向)相當確定(牆壁一直在校正)。它的不確定性橢圓因此是一個「沿走廊方向拉長」的細長橢圓。這個橢圓會隨機器人運動與觀測不斷變形——SLAM 的每一步,本質上都在更新這個橢圓。

為什麼偏愛高斯

- 兩個參數搞定:均值與共變異數就完整描述了不確定性,儲存與計算都輕量。
- 運算封閉性:兩個高斯相乘仍是高斯、高斯經線性變換仍是高斯。這讓濾波器的每一步都能保持高斯形式,不會越算越複雜。這正是卡爾曼濾波如此優雅的數學根源。
- 中央極限定理:許多獨立的小誤差疊加,結果趨近高斯。感測器雜訊往往正是眾多微小因素的總和,因此高斯假設在實務上常常成立。

但要提醒:高斯假設並非萬能。當不確定性是「多峰」的(例如機器人可能在兩個長得一樣的房間之一,但不知是哪個),單一高斯就無法描述——這正是 11.7 節粒子濾波登場的原因。

11.3 感測器雜訊與運動模型

要把機器人放進機率框架,需要兩個「模型」,它們是所有濾波器的輸入。這兩個模型分別描述機器人如何「動」與如何「看」。

一、運動模型(Motion Model)

運動模型回答:「機器人執行了某個控制(如前進一公尺、左轉 30 度)後,新的位姿會是什麼?」關鍵在於——答案不是一個確定的位姿,而是一個機率分布,因為執行控制時會有雜訊(輪子打滑、地面不平)。

數學上寫成 $p(x_t | x_{t-1}, u_t)$:給定前一刻位姿 x_{t-1} 與控制 u_t ,新位姿 x_t 的機率分布。運動模型的效果是讓不確定性擴大——每動一步,不確定性橢圓就膨脹一點,因為又累積了新的運動雜訊。這正是第 10 章 2D 位姿複合公式,加上一個雜訊項。

二、觀測模型(Observation Model)

觀測模型回答:「如果機器人真的在某個位姿,感測器應該量到什麼?」同樣地,答案是機率分布,因為感測器有雜訊。

數學上寫成 $p(z_t | x_t)$:給定位姿 x_t ,觀測 z_t 的機率分布。例如「若機器人在此處,光達應該量到前方 2 公尺有牆」,但實際量測會在 2 公尺附近有些抖動。觀測模型的效果是讓不確定性收縮——一個明確的觀測(摸到牆角)能大幅縮小可能位置的範圍。

兩個模型的分工

這兩個模型恰好對應 11.1 節「蒙眼走路」比喻的兩個動作:運動模型讓不確定性擴大(走路時越來越模糊),觀測模型讓不確定性收縮(摸到地標時變清晰)。SLAM 的整個過程,就是這兩個模型交替作用——動一步、看一眼、動一步、看一眼。下一節的貝氏濾波,就是把這個「交替」用數學精確地表達出來。

	運動模型	觀測模型
數學形式	$p(x_t x_{t-1}, u_t)$	$p(z_t x_t)$
回答的問題	動了之後會到哪	在這裡會看到什麼
輸入	前一位姿 + 控制指令	當前位姿
雜訊來源	打滑、地面、輪徑誤差	感測器量測誤差
對不確定性	擴大(膨脹)	收縮(縮小)
對應比喻	蒙眼走路	摸到牆角

表 11-1 運動模型與觀測模型的對照

11.4 Bayes Filter

貝氏濾波(Bayes Filter)是本章的核心——它是卡爾曼濾波、EKF、粒子濾波等所有濾波器的共同骨架。理解了它,後面幾節都只是它的不同實作版本。

信念:對狀態的機率描述

貝氏濾波追蹤的核心物件稱為信念(Belief),記為 $bel(x_t)$ ——它是「綜合目前所有資訊後,對當前狀態的機率分布」。信念就是 11.1 節說的那個「不確定性橢圓」的數學本體。貝氏濾波的任務,就是每來一筆新資料,就更新這個信念。

兩個步驟的遞迴

貝氏濾波以兩個步驟不斷循環,這兩步驟正對應前一節的兩個模型:

72. 預測(Prediction):機器人執行控制 u_t ,用運動模型把信念往前推。這一步讓不確定性擴大——「我動了,所以更不確定自己在哪了」。數學上是把前一刻信念與運動模型做積分(對所有可能的前一位姿加總)。

73.更新(Update / Correction):感測器給出觀測 z_t ,用觀測模型修正信念。這一步讓不確定性收縮——「我看到了地標,所以更確定自己在哪了」。數學上是把預測信念與觀測模型相乘,再正規化。

這個「預測—更新」的循環,正是第 1 章圖 1-1 SLAM 標準流程、第 7 章卡爾曼濾波「預測—更新」的同一件事,在此得到了精確的機率表述。同學應牢牢記住這個雙步驟結構:它是理解一切遞迴估測的鑰匙。

圖 11-1 貝氏濾波的遞迴循環

前一刻信念 $bel(x_{t-1})$

↓ ① 預測:套用運動模型 $p(x_t | x_{t-1}, u_t)$

→ 信念變寬(不確定性擴大)

預測信念(先驗)

↓ ② 更新:乘上觀測模型 $p(z_t | x_t)$,再正規化

→ 信念變窄(不確定性收縮)

新信念 $bel(x_t)$

↓

(回到頂端,處理下一時刻)

貝氏法則:更新步驟的根據

更新步驟背後的數學,就是機率論中著名的貝氏法則(Bayes' Rule)。它的精神是:把「先驗信念」(預測步驟得到的、看到觀測之前的信念)與「觀測提供的證據」結合,得到「後驗信念」(看到觀測之後、更精確的信念)。

$$\text{後驗} \propto \text{觀測似然} \times \text{先驗} \quad \text{bel}(x_t) \propto p(z_t | x_t) \cdot \text{預測信念}$$

這條式子讀作:更新後的信念,正比於「觀測模型」乘上「預測信念」。它是所有濾波器更新步驟的理論根源。貝氏濾波本身是一個抽象框架,實際計算時無法直接處理任意分布——不同的濾波器,就是對「信念用什麼形式表示、積分與相乘怎麼算」給出不同的答案。接下來三節就是三種答案。

11.5 Kalman Filter

卡爾曼濾波器(Kalman Filter, KF)是貝氏濾波在「線性系統 + 高斯雜訊」假設下的精確解。在這個假設下,信念永遠是高斯分布,於是整個濾波只需追蹤兩個量:均值(最佳估計)與共變異數(不確定性)。這讓計算極其高效,也讓卡爾曼濾波成為工程史上最成功的演算法之一——從阿波羅登月導航到今天的手機,無處不在。

卡爾曼濾波的兩步驟

它完全遵循貝氏濾波的架構,只是所有運算都在高斯的世界裡進行:

74. 預測:用運動模型把均值往前推(機器人動了,估計位置跟著移動),同時把共變異數加大(加入運動雜訊,不確定性擴大)。
75. 更新:用觀測修正均值,並把共變異數縮小。修正的力道由卡爾曼增益決定。

卡爾曼增益:全章最精妙的一個概念

卡爾曼增益(Kalman Gain, K)是整個濾波器的靈魂。它回答一個關鍵問題:預測和觀測不一致時,該相信誰、相信多少?

它的邏輯完全符合直覺:

- 若預測很不確定(共變異數大)、觀測很準(雜訊小):增益大,多相信觀測——「我本來就沒把握,既然感測器說得這麼肯定,那就聽它的」。

- 若預測很有把握、觀測很雜:增益小,多相信預測——「我很確定自己的推算,這個雜訊很大的量測參考就好」。

換句話說,卡爾曼濾波不是死板地取平均,而是根據雙方的可信度做加權——誰更可信就更聽誰的。這個「按可信度加權」的思想,是本書反覆出現的融合精神(第 4、5、7 章)的數學化身。第 7 章的姿態濾波、第 5 章的 GNSS/INS 組合導航,底層都是這個增益在運作。

觀念總結:卡爾曼濾波在做什麼

一句話:用「上一刻的估計 + 運動」預測現在,再用「感測器觀測」修正,兩者按各自的可信度(共變異數)加權融合,得到比單獨任一方都更準的估計。

關鍵洞見:融合兩個帶雜訊的資訊,結果會比其中任何一個單獨都更準——

這就是為什麼「多感測器融合」永遠優於「只用最好的那一個」。

11.6 Extended Kalman Filter, EKF-SLAM

卡爾曼濾波很美,但有一個嚴格的前提:系統必須是線性的。可惜機器人的世界充滿非線性——第 10 章看過,位姿更新含 $\cos\theta$ 、 $\sin\theta$;觀測模型「若在此處,光達會量到多遠」也是非線性的。標準卡爾曼濾波無法直接處理。EKF(擴展卡爾曼濾波)就是為此而生。

EKF 的核心技巧:局部線性化

EKF 的想法很務實:既然系統是非線性的,那就在當前估計點附近,用切線去近似曲線。數學上這叫一階泰勒展開,得到的斜率矩陣稱為雅可比矩陣(Jacobian)。線性化之後,就

能套用標準卡爾曼濾波的公式。

這個近似在「非線性不太嚴重、不確定性不太大」時效果良好,是機器人領域最廣泛使用的估測方法——第 7 章提過的無人機飛控 EKF、ROS 的 `robot_localization`,底層都是它。但它的近似本質也帶來限制:當非線性很強或不確定性很大時,切線偏離曲線太多,估計可能失準甚至發散。

EKF-SLAM:把地圖也放進狀態

EKF 如何用來做 SLAM?關鍵一步是:把機器人位姿和所有地標的位置,通通放進同一個狀態向量,用同一個高斯分布(同一個大共變異數矩陣)一起估計。

這個設計精妙地抓住了 SLAM 的本質——第 1 章強調的「定位與建圖互相耦合」。當機器人觀測到一個地標並修正自己的位置時,共變異數矩陣中的相關性會把這個修正「傳播」到所有相關的地標與位姿。這正是閉環修正能一次攤平整張地圖誤差的數學機制:回到熟悉地點時,一個強觀測透過共變異數的相關性,同時校正了機器人與沿途所有地標。

EKF-SLAM 的歷史地位與限制

EKF-SLAM 是 SLAM 發展史上第一個完整可行的解法(1990 年代),奠定了整個領域的理論基礎,地位崇高。但它有兩個致命限制,使它逐漸退出主流:

- 計算複雜度:共變異數矩陣的大小隨地標數量平方成長。地標一多(數千個),矩陣運算就慢到無法即時——這對動輒數萬地標的大場景是硬傷。
- 線性化誤差累積:EKF 每步都做線性近似,誤差會累積,長期或大場景下可能導致估計不一致。

這兩個限制,催生了後續的兩條主流路線:一是第 14 章的粒子濾波(用另一種方式處理),二是第 12 章的圖優化(用批次最佳化取代遞迴濾波)。可以說,理解 EKF-SLAM 的限

制,就理解了現代 SLAM 為何走向圖優化——這是承先啟後的關鍵一章。

11.7 Particle Filter 與 Monte Carlo Localization

卡爾曼家族都假設信念是「單一高斯」。但 11.2 節末尾提過:有些情況信念是多峰的——機器人可能在兩個一模一樣的房间之一,信念應該是「兩個高峰」。單一高斯無法描述這種情況。粒子濾波(Particle Filter)提供了截然不同的思路。

用一群樣本代表信念

粒子濾波的核心洞見是:不用數學公式描述分布,而是用一大群樣本(粒子)來「近似」它。每個粒子代表「機器人可能在這裡」的一個假設。粒子密集的地方,就是可能性高的地方。

這個表示法的最大優點是:它能描述任意形狀的分布——多峰、不規則、非高斯都沒問題。這是卡爾曼家族做不到的。

粒子濾波的三步驟

它同樣遵循貝氏濾波架構,但以粒子的方式實現:

76. 預測(移動粒子):每個粒子依運動模型移動,並各自加上隨機雜訊。於是原本聚集的粒子雲會散開——對應不確定性擴大。

77. 更新(秤重):用觀測模型給每個粒子打分數(權重)。與觀測吻合的粒子(例如「這個位置預期看到的牆,和實際量到的相符」)得高分,不吻合的得低分。

78. 重採樣(Resampling):依權重重新抽選粒子——高分粒子被複製多份,低分粒子被淘汰。於是粒子雲逐漸聚集到最可能的位置。這是粒子濾波的精髓:適者生存。

蒙地卡羅定位(MCL)與 AMCL

把粒子濾波用於「在已知地圖中定位」,就是蒙地卡羅定位(MCL, Monte Carlo Localization), ROS 中的實作稱為 AMCL(第 3 章提過)。它的運作極為直觀:一開始不知道機器人在哪,就在地圖各處撒滿粒子;機器人一邊移動一邊掃描,與觀測不符的粒子被淘汰,粒子雲逐漸收斂到真實位置。

MCL 天生就能解決第 3 章的「綁架機器人問題」:即使機器人被搬走,只要在地圖各處保留一些粒子,它們就能在新位置重新「勝出」,讓機器人找回自己。這是單一高斯的卡爾曼濾波難以做到的——這也是掃地機器人被抱到別的房間後能重新定位的數學原因。

粒子濾波與 SLAM:預告 Gmapping

把粒子濾波用於 SLAM(而非只是定位),需要一個巧妙的技巧,稱為 Rao-Blackwellized 粒子濾波(RBPF)——每個粒子不只代表一個位姿,還各自攜帶一整張地圖。這正是第 14 章 Gmapping 的核心,也是本書第四篇 2D SLAM 的主角。本章先把粒子濾波的原理講清楚,第 14 章就能專注於它如何建圖。

特性	卡爾曼 / EKF 家族	粒子濾波
信念表示	單一高斯(均值 + 共變異數)	一群粒子樣本
可處理分布	限高斯(單峰)	任意形狀(可多峰)
非線性	EKF 靠線性化近似	天然支援,不需線性化
綁架問題	難處理	天生可解
計算成本	低(維度低時)	高(需大量粒子)
代表應用	GNSS/INS、EKF-SLAM	AMCL、Gmapping

表 11-2 卡爾曼家族與粒子濾波的比較

至此,同學已經掌握了 SLAM 的核心估測工具:用機率描述不確定性,用貝氏濾波的

「預測—更新」處理它,並認識了卡爾曼、EKF、粒子濾波三種具體實作。這些是遞迴濾波的路線——每來一筆資料就更新一次。但現代 SLAM 還有另一條更強大的路線:不急著逐筆處理,而是累積一批資料後一次性最佳化。下一章「最佳化與圖論 SLAM」,將介紹這條主宰當代 SLAM 的路線——Pose Graph、Bundle Adjustment 與 g2o、Ceres、GTSAM。

公式或流程

一、一維高斯分布

一維高斯分布的機率密度函數:

$$p(x) = (1 / \sqrt{2\pi \sigma^2}) \cdot \exp(- (x - \mu)^2 / (2\sigma^2))$$

其中 μ 為均值(分布中心、最佳估計)、 σ^2 為變異數(不確定性大小)。約 68% 的機率落在 $\mu \pm \sigma$ 、95% 落在 $\mu \pm 2\sigma$ 、99.7% 落在 $\mu \pm 3\sigma$ ——這組「68-95-99.7 法則」是判斷不確定性最實用的直覺。

二、貝氏法則

狀態估測的理論核心:

$$p(x/z) = p(z/x) \cdot p(x) / p(z)$$

讀作:後驗 = (似然 $\times$ 先驗) / 證據。在濾波中, $p(x)$ 是預測步驟得到的先驗信念、 $p(z|x)$ 是觀測模型、 $p(x|z)$ 是更新後的後驗信念。分母 $p(z)$ 只是正規化常數(確保機率總和為 1),因此常寫成正比形式:

$$p(x/z) \propto p(z/x) \cdot p(x)$$

三、貝氏濾波兩步驟

預測步驟(套用運動模型,對前一狀態積分):

$$bel^-(x_t) = \int p(x_t / x_{t-1}, u_t) \cdot bel(x_{t-1}) dx_{t-1}$$

更新步驟(乘上觀測模型並正規化):

$$bel(x_t) = \eta \cdot p(z_t / x_t) \cdot bel^-(x_t)$$

其中 bel^- 是預測信念(先驗)、 bel 是更新後信念(後驗)、 η 是正規化常數。卡爾曼濾波、EKF、粒子濾波都是這兩條式子在不同表示法下的具體實現。

四、卡爾曼濾波核心方程(概念版)

預測:

$$\text{均值: } \hat{x}^- = F \cdot \hat{x} \quad \text{共變異數: } P^- = F \cdot P \cdot F^T + Q$$

更新:

$$\text{卡爾曼增益: } K = P^- \cdot H^T \cdot (H \cdot P^- \cdot H^T + R)^{-1}$$

$$\text{均值: } \hat{x} = \hat{x}^- + K \cdot (z - H \cdot \hat{x}^-) \quad \text{共變異數: } P = (I - K \cdot H) \cdot P^-$$

其中 F 是運動模型、 H 是觀測模型、 Q 是運動雜訊、 R 是觀測雜訊。細看增益 K 的式子:當觀測雜訊 R 很小(觀測很準), K 變大,更新時就更相信觀測($z - H \cdot \hat{x}^-$ 這個「觀測與預測的差」被乘上大的 K);當 R 很大, K 變小,更相信預測。這條式子把 11.5 節「按可信度加權」的直覺精確地數學化了。

五、三種濾波器選擇流程

圖 11-2 濾波器選擇決策流程

問題一:系統是線性的嗎?

└ 是 → 卡爾曼濾波(KF):最佳、最快

└ 否 → 進入下一問

↓

問題二:非線性溫和、信念單峰、有好的初始估計嗎?

└ 是 → 擴展卡爾曼濾波(EKF):線性化近似,高效

└ 否(強非線性 / 多峰 / 綁架問題)→ 進入下一問

↓

問題三:狀態維度低、可負擔大量粒子嗎?

└ 是 → 粒子濾波(PF / MCL):能處理任意分布

└ 否(高維、大場景)→ 考慮第 12 章的圖優化

案例

案例 11-1 阿波羅登月:卡爾曼濾波的成名之戰

卡爾曼濾波器由 Rudolf Kálmán 於 1960 年提出,而它第一個震撼世界的應用,就是阿波羅登月計畫。太空船在地月之間航行,導航系統必須融合慣性量測與地面追蹤等多種帶雜訊的資料,即時估出太空船的位置與速度——這正是卡爾曼濾波「按可信度融合多源資訊」的完美舞台。

當時太空船的運算能力極其有限(遠不如今天一支計算機),但卡爾曼濾波只需追蹤均值與共變異數兩個量、且是遞迴計算(不必儲存全部歷史資料),恰好能在如此拮据的硬體上即時運行。可以說,沒有卡爾曼濾波,阿波羅的導航幾乎無法實現。

這個案例呼應了第 10 章的阿波羅萬向鎖——同一個登月任務,同時推動了旋轉表示法與狀態估測兩大數學的發展。它也給同學一個啟示:本章這些看似抽象的機率工具,是人類踏上月球的實際基石。六十多年後的今天,你手機裡的定位、掃地機器人的導航,

用的仍是同一套數學。

案例 11-2 掃地機器人的「粒子雲」:AMCL 的視覺化

許多支援地圖顯示的掃地機器人 App,若進入除錯或進階模式,能看到一個生動的畫面:機器人開機時,地圖上散布著大量代表「可能位置」的小點(粒子);隨著機器人移動幾步、掃描幾次,這些小點迅速從「散布全圖」收斂成「緊密的一小團」——機器人「想通了」自己在哪。

這正是 11.7 節 AMCL 的即時視覺化。散開的粒子代表「我還不確定在哪」,收斂的粒子團代表「我很確定了」。如果把機器人抱到別的房間,會看到粒子團先是「困惑」(與觀測不符),然後在正確的新位置重新聚集——綁架問題的解決過程活生生地展現在眼前。

這個案例的教學價值在於:它把本章最抽象的「機率分布」變成看得見的粒子雲。大學課堂上,教師常讓同學在 RViz 中觀察 AMCL 的粒子演化,配合本章理論,抽象的「信念收縮」瞬間變得具體。這也是本書強調「先理解原理、再動手觀察」的又一個例子。

案例 11-3 自駕車的感測器融合:增益在真實道路上的權衡

一台自駕車同時擁有 GPS、IMU、輪速計、光達、相機多種感測器,它們對「車現在在哪」各有各的說法,而且各有各的可靠時機。卡爾曼濾波家族(通常是 EKF 或其變體)正是融合這些資訊的主力。

本章的卡爾曼增益概念,在此有極為真實的體現:當車輛開進隧道,GPS 失鎖(觀測雜訊 R 飆升),濾波器自動調低對 GPS 的信任、改倚重 IMU 與輪速計的推算(第 5 章的組合導航);出隧道後 GPS 恢復,濾波器又逐漸把信任轉回來。這個「依情境動態調整信任」的過程,不需要工程師寫死規則,而是共變異數與增益的數學自然導出的結果。

這正是機率方法相對「寫死規則」的根本優勢:它讓系統依據每個資訊源當下的可信度自動權衡,優雅地適應各種情境。這個思想貫穿全書——從第 4 章的室內定位融合、第 5 章的 GNSS/INS、第 7 章的姿態濾波,到本章的統一數學表述,同學應該能感受到:整個第二篇與第三篇,其實都在講同一件事的不同層面。

實作活動

實作 11-1 一維卡爾曼濾波與粒子濾波動手做

目的:用最簡單的一維例子,親手實作卡爾曼濾波與粒子濾波,親眼看見「不確定性擴大與收縮」以及「粒子雲收斂」。本實作以 Python 為主,是理解本章所有理論的關鍵一步。

第一部分:一維卡爾曼濾波(全體)

62. 設定情境:一台機器人在一維軌道上,真實位置未知。它有一個會漂移的「速度感測器」(運動模型,雜訊大)與一個偶爾回報的「位置感測器」(觀測模型,雜訊中等)。用程式產生一段帶雜訊的模擬資料。
63. 實作卡爾曼濾波的預測步驟:每步用速度更新位置均值,並把變異數加大。畫出「估計位置」與「不確定範圍(均值 $\pm$ 標準差)」隨時間的變化,觀察只有預測時不確定範圍如何持續擴大。
64. 加入更新步驟:每當位置感測器回報,就用觀測修正均值、縮小變異數。觀察不確定範圍在每次觀測時如何「收縮」。
65. 把「純預測(航位推算)」與「完整卡爾曼濾波」的估計軌跡畫在一起,與真值比較,量化卡爾曼濾波減少多少誤差。
66. 實驗卡爾曼增益:把觀測雜訊 R 分別設得很大與很小,觀察增益如何變化、估計軌跡如何更偏向預測或更偏向觀測。

第二部分:一維粒子濾波(全體或大學部)

67. 在一維軌道上撒 500 個粒子(均勻分布),代表「一開始不知道機器人在哪」。
68. 實作三步驟:每步讓所有粒子依運動模型移動(加雜訊)、依觀測給每個粒子打權重、再依權重重採樣。
69. 每幾步畫一次粒子分布的直方圖,觀察粒子雲如何從「散布全軌道」收斂到「真實位置附近」。
70. 綁架測試:在中途突然把真實位置換到別處,觀察粒子雲如何先「困惑」、再重新聚集到新位置。

第三部分:多峰情境(大學部)

71. 設計一個「對稱走廊」情境:兩個地點的觀測完全相同(對稱環境)。用粒子濾波處理,觀察粒子雲如何維持「兩個高峰」——親身驗證 11.7 節「粒子濾波能處理多峰、卡爾曼不能」的論述。
72. 思考:若用卡爾曼濾波處理這個對稱情境,會發生什麼?(提示:單一高斯會把均值放在兩峰中間——一個機器人根本不可能在的位置。)

討論問題

55. 在第一部分,完整卡爾曼濾波比純航位推算減少多少誤差?這個改善來自哪裡?
56. 當你把觀測雜訊 R 調得很小,估計軌跡發生什麼變化?這符合 11.5 節對卡爾曼增益的描述嗎?
57. 粒子濾波的綁架測試中,粒子雲花了多久重新聚集?若粒子數只有 50 個(而非 500),結果會如何?這說明粒子數的什麼取舍?
58. (大學部)在對稱走廊情境,粒子濾波維持了兩個高峰,而卡爾曼濾波會失敗。這對「選擇哪種濾波器」有什麼實務啟示?

延伸挑戰(大學部)

把一維卡爾曼濾波擴展到二維(x 、 y),追蹤共變異數矩陣,並用 matplotlib 畫出每一步的「不確定性橢圓」,觀察機器人沿直線運動時橢圓如何沿運動方向拉長、觀測後如何收縮。這個二維視覺化直接對應 11.2 節,也為第 14 章 Gmapping 與第 28 章 AMCL 實作打下直覺基礎。

延伸閱讀

69. S. Thrun, W. Burgard, and D. Fox, Probabilistic Robotics, MIT Press, 2005.(機率機器人學聖經,本章全部主題的權威來源,第 2 至 8 章尤其相關)
70. Cyrill Stachniss, SLAM 課程之 Bayes Filter、Kalman Filter、Particle Filter 系列影片(YouTube).(以動畫直觀講解本章三種濾波器,自學首選)
71. R. Faragher, "Understanding the Basis of the Kalman Filter via a Simple and Intuitive Derivation," IEEE Signal Processing Magazine, 2012.(卡爾曼濾波最友善的直觀推導)
72. R. Labbe, "Kalman and Bayesian Filters in Python"(GitHub 免費電子書).(附完整可執行程式,實作本章的最佳伴讀)
73. D. Fox et al., "Monte Carlo Localization for Mobile Robots," ICRA, 1999.(蒙地卡羅定位 MCL 的原始論文)
74. CMU 16-833 Robot Localization and Mapping 課程(第 2 章介紹)之濾波單元.(EKF-SLAM 與粒子濾波的研究所級教材)
75. 高翔、張濤,《視覺 SLAM 十四講》第 6 講(非線性最佳化)前段.(銜接本章濾波與第 12 章圖優化的橋樑)

小結

42. SLAM 必須以機率處理不確定性:每個感測器給的都是帶雜訊的猜測,成熟的做法是量化不確定性再做最佳權衡。關鍵觀念是——估計的是「機率分布」(最佳猜測 + 可信度),而非單一數值。
43. 高斯分布以均值(最佳估計)與變異數(不確定性)兩參數完整描述;多維時共變異數矩陣對應「不確定性橢圓」。高斯因兩參數搞定、運算封閉、中央極限定理而成為 SLAM 首選,但無法描述多峰分布。
44. 運動模型 $p(x_t|x_{t-1},u_t)$ 描述「動了會到哪」使不確定性擴大;觀測模型 $p(z_t|x_t)$ 描述「在這裡會看到什麼」使不確定性收縮。兩者是所有濾波器的輸入。
45. 貝氏濾波是所有濾波器的共同骨架,以「預測(套運動模型、信念變寬)—更新(套觀測模型、信念變窄)」兩步驟遞迴,更新步驟的根據是貝氏法則:後驗 $\propto$ 似然 $\times$ 先驗。
46. 卡爾曼濾波是線性高斯下的最佳解,只需追蹤均值與共變異數;卡爾曼增益依雙方可信度權衡預測與觀測——這是「多感測器融合優於單一感測器」的數學核心。
47. EKF 以雅可比矩陣局部線性化處理非線性,是機器人最廣用的估測法;EKF-SLAM 把位姿與地標放進同一狀態聯合估計,體現定位與建圖的耦合,但共變異數隨地標平方成長且線性化誤差累積,催生了圖優化路線。
48. 粒子濾波以一群樣本近似任意分布,經預測—秤重—重採樣三步驟,能處理多峰與非線性、天生可解綁架問題;蒙地卡羅定位(AMCL)是其定位應用,Rao-Blackwellized 版本(Gmapping)則是其 SLAM 應用。

本章的濾波器都是「遞迴」的——每來一筆資料就更新一次,處理完就丟掉。這高效,但也意味著過去的資訊一旦被壓縮進當前信念,就無法再回頭修正。下一章「最佳化

與圖論 SLAM」介紹另一條路線:把一批位姿與觀測全部保留,建成一張「位姿圖」,一次性最佳化求出最一致的解。這條路線精度更高、能更好地處理閉環,正是 ORB-SLAM、Cartographer 等當代主流系統的數學基礎。

台灣自編大學教科書

@prof

《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第 12 章

最佳化與圖論 SLAM

編者：李世淵 (機器人叫獸)

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

學習目標

上一章的濾波器是「遞迴」的——每來一筆資料就更新一次,處理完就丟掉。本章介紹另一條路線:把一批位姿與觀測全部保留,建成一張「圖」,一次性最佳化求出最一致的解。這條圖優化路線精度更高、更能處理閉環,是 ORB-SLAM、Cartographer 等當代主流系統的數學基礎,也是第三篇的收尾。學完本章之後,同學應能夠達成下列目標:

70. 說明 SLAM 前端與後端的分工,理解圖優化屬於後端。
71. 說明位姿圖(Pose Graph)的節點與邊代表什麼,並看懂它如何描述一個 SLAM 問題。
72. 解釋為什麼 SLAM 可以化為一個最小平方問題。
73. 說明 Bundle Adjustment 的概念與它在視覺 SLAM 中的角色。
74. 認識 g2o、Ceres、GTSAM 三大最佳化函式庫的定位與用途。
75. 說明為什麼現代 SLAM 多採圖優化,而非上一章的濾波器。

核心概念

中文名詞	英文名詞	概念說明
前端	Front-End	把原始感測資料轉成幾何約束(特徵、匹配、里程計)。
後端	Back-End	在所有約束下求出最一致的位姿與地圖。
位姿圖	Pose Graph	以節點表示位姿、邊表示約束的圖結構。
節點	Node / Vertex	待估計的量,如機器人位姿或地標位置。
邊	Edge / Constraint	兩節點之間的相對量測約束。
最小平方	Least Squares	尋找使所有誤差平方和最小的解的方法。

中文名詞	英文名詞	概念說明
殘差	Residual	量測值與預測值之間的差異。
光束法平差	Bundle Adjustment, BA	同時最佳化相機位姿與三維點的視覺最佳化。
重投影誤差	Reprojection Error	三維點投影到影像與實際觀測像素的差距。
資訊矩陣	Information Matrix	共變異數的反矩陣,代表約束的可信度權重。
稀疏性	Sparsity	每個約束只牽涉少數變數,使大問題可高效求解。
圖優化函式庫	g2o / Ceres / GTSAM	求解圖優化問題的開源工具。

12.1 前端與後端的分工

現代 SLAM 系統普遍採用「前端 + 後端」的兩段式架構(第 2 章各校課程的共同骨架、第 1 章圖 1-1 的核心)。理解這個分工,是讀懂本章的第一步。

前端:把感測資料變成幾何約束

前端(Front-End)直接面對感測器,它的工作是把原始資料轉成「幾何約束」——也就是「A 位置到 B 位置之間,相對移動了多少」或「在 A 位置看到了某個地標」這類幾何關係。

- 對光達 SLAM:前端做掃描匹配(第 13 章),算出「這一幀相對上一幀移動了多少」。
- 對視覺 SLAM:前端做特徵提取與匹配(第 9、19 章),算出相機運動與看到的地標。
- 前端還負責閉環偵測:辨識「這個地方我來過」,產生一條關鍵的閉環約束。

後端:在所有約束下求最一致解

後端(Back-End)不直接碰感測器,它接收前端產生的一堆幾何約束,任務是找出一組位姿與地圖,使它們「盡可能同時滿足所有約束」。

問題來了:這些約束往往互相矛盾。里程計說「我從 A 走到 B 移動了 10 公尺」,但閉環偵測說「B 其實就在 A 旁邊」——兩個約束打架。後端的工作,就是在這些矛盾的約束之間找一個「最佳妥協」,讓總體誤差最小。這正是本章的主題。

兩種後端路線

如何求這個「最佳妥協」,歷史上有兩條路線:

- 濾波路線(第 11 章):遞迴地逐筆處理,以 EKF、粒子濾波為代表。優點是即時、記憶體固定;缺點是過去資訊被壓縮後無法回頭修正。
- 最佳化路線(本章):保留一批約束,一次性求解整個問題,以圖優化為代表。優點是精度高、能反覆修正、能漂亮地處理閉環;代價是計算量較大。

過去十餘年,最佳化路線因為精度與閉環處理的優勢,已成為主流。本章就是要說清楚這條路線的數學與工具。

12.2 Pose Graph 的概念

圖優化 SLAM 的核心資料結構,是一張「位姿圖」(Pose Graph)。它把抽象的 SLAM 問題,變成一張直觀的「圖」——就是資料結構課學過的、由節點與邊構成的圖。

節點與邊

- 節點(Node / Vertex):代表「待估計的量」。在最單純的位姿圖中,每個節點是機器人在某個時刻的位姿(第 10 章的 x 、 y 、 θ 或 6-DoF)。在含地標的版本中,地標位置也是節點。這些就是我們想求出來的未知數。
- 邊(Edge / Constraint):代表兩個節點之間的「相對量測約束」。它記錄「根據某個感測器,這兩個節點之間應該是什麼關係」。

兩種關鍵的邊

位姿圖中有兩種來源不同、但同等重要的邊:

- 里程計邊(相鄰邊):連接時間上相鄰的兩個位姿,來自里程計或掃描匹配——「我從第 t 幀走到第 $t+1$ 幀,移動了這麼多」。這些邊像一條鏈子,把位姿串起來。
- 閉環邊(迴路邊):連接時間上不相鄰、但空間上重逢的兩個位姿,來自閉環偵測——「我現在所在的位置,和很久以前的第 5 幀是同一個地方」。這種邊在圖上形成「迴路」,是消除累積漂移的關鍵。

為什麼閉環邊這麼重要

想像機器人繞行大樓一圈回到起點。里程計邊像一條鏈子,一路累積誤差,走完一圈後終點與起點之間出現明顯落差(第 1 章的漂移)。這時閉環偵測產生一條邊,宣告「終點就是起點」——這條邊與整條里程計鏈子產生矛盾。

後端最佳化的魔法,就是把這個矛盾「均攤」到整條軌跡上:它微調沿途每一個位姿,讓終點回到起點,同時盡量不破壞里程計約束。結果是——整條軌跡被「拉正」,累積的漂移一次性被消除。這就是為什麼閉環對 SLAM 如此關鍵,也是圖優化相對濾波器的第一大優勢:它能回頭修正所有歷史位姿。

圖 12-1 位姿圖的結構

節點: ①—②—③—④—⑤—⑥—⑦—⑧ (每個是一個位姿)

└——里程計邊(相鄰,像鏈子)——┐

① ←——閉環邊——→ ⑧

(偵測到第⑧幀回到了第①幀的位置)

最佳化前:里程計鏈累積漂移,⑧ 與 ① 對不上

最佳化後:閉環邊把矛盾均攤到 ①~⑧,整條軌跡被拉正

12.3 節點、邊、限制條件

本節更仔細地看邊所代表的「限制條件」,以及如何量化「約束被滿足的程度」——

——這是把位姿圖變成可求解問題的關鍵。

殘差:量測與預測的差

每一條邊都帶有一個量測值(感測器說兩節點之間應該是什麼關係)。而根據當前對兩節點位姿的估計,我們也能算出一個預測值(按目前估計,兩節點之間實際是什麼關係)。兩者的差,稱為殘差(Residual):

$$\text{殘差} = \text{量測值} - \text{預測值}$$

殘差為零,表示這條邊的約束被完美滿足;殘差越大,表示當前的位姿估計越違背這條約束。後端的目標,就是調整所有節點,讓所有邊的殘差盡量小。

不是所有約束都一樣可信:資訊矩陣

這裡有一個關鍵細節:不同的邊可信度不同。一條來自高精度光達掃描匹配的邊,遠比一條來自打滑里程計的邊可信。最佳化時,應該更「尊重」可信的邊。

這個可信度用資訊矩陣(Information Matrix)描述——它正是第 11 章共變異數矩陣的反矩陣。共變異數大(不確定)對應資訊矩陣小(權重低);共變異數小(確定)對應資訊矩陣大(權重高)。於是可信的約束在最佳化中「說話比較大聲」。這讓圖優化與第 11 章的機率框架完美接軌:兩者其實是同一個機率問題的不同解法。

限制條件的一致性

位姿圖的美妙之處在於:它把各種來源的約束——里程計、掃描匹配、閉環、甚至第 4 章的 UWB 測距、第 5 章的 GNSS、第 7 章的 IMU 預積分——通通統一成「節點之間的邊」。無論約束來自哪個感測器,在圖裡都是一條帶殘差與資訊矩陣的邊。這種統一性,正是圖優化能優雅融合多感測器的原因,也呼應了前面各章反覆出現的「緊耦合」思想:把所有原始約束放進同一個最佳化問題一起求解。

12.4 Least Squares 最小平方

現在把前面的概念收攏成一個明確的數學問題。既然要讓所有邊的殘差都盡量小,自然的做法是:找一組節點值,使「所有殘差的平方和」最小。這就是最小平方(Least Squares)。

為什麼是「平方」和

為什麼不是殘差的絕對值和,而是平方和?有兩個理由:

- 數學理由:平方函數處處可微、形式漂亮,便於用微積分求極小值(令導數為零)。絕對值在零點不可微,處理起來麻煩。
- 機率理由:當雜訊是高斯分布時(第 11 章),「最小化殘差平方和」在數學上恰好等價於「最大化後驗機率」(找最可能的解)。換句話說,最小平方不是隨便選的,它是高斯假設下的最佳估計。這再次顯示圖優化與第 11 章機率框架是同一件事。

加上資訊矩陣作為權重後,更精確的說法是「最小化加權殘差平方和」——可信的邊權重大,對總和的影響也大。

如何求解:反覆逼近

這個最小平方問題通常是非線性的(因為殘差含 $\cos\theta$ 、 $\sin\theta$ 等,第 10 章),無法一步解出。實務上用迭代法反覆逼近:

79. 從一個初始猜測開始(通常用里程計串起來的粗略軌跡)。
80. 在當前估計附近把問題線性化(求雅可比矩陣,第 11 章 EKF 用過同樣的技巧)。
81. 解一個線性方程組,得到一個「往誤差更小的方向修正」的步伐。
82. 更新估計,回到步驟 2,反覆進行直到殘差不再明顯下降(收斂)。

常用的迭代方法有高斯—牛頓法(Gauss-Newton)與列文伯格—馬夸特法

(Levenberg-Marquardt); 後者更穩健, 是許多 SLAM 系統的預設選擇。同學不需手算這些方法——這正是 g2o、Ceres、GTSAM 這些函式庫要替我們做的事(12.6 節)。

稀疏性: 大問題能求解的祕密

一個大場景的位姿圖可能有數萬個節點, 對應數萬個未知數。直接解這麼大的線性方程組, 計算量會大到不可行。幸好 SLAM 問題有一個救命的性質: 稀疏性(Sparsity)。

稀疏性的意思是: 每一條邊只牽涉兩個節點, 而不是全部節點。因此線性化後的大矩陣裡, 絕大多數元素是零。利用這個結構, 有專門的稀疏矩陣演算法能極快地求解——這正是 EKF-SLAM(第 11 章)辦不到而圖優化能辦到的關鍵: EKF 的共變異數矩陣會變稠密(每個地標都與其他相關), 而圖優化保持稀疏。這是圖優化能處理大場景的數學根源。

12.5 Bundle Adjustment

前面的位姿圖主要處理「位姿之間」的約束。在視覺 SLAM 中, 還有一個更精細的最佳化問題, 同時處理「相機位姿」與「三維地圖點」——這就是光束法平差(Bundle Adjustment, BA)。它是視覺 SLAM 精度的來源, 也是第 21、22 章 ORB-SLAM 的核心。

名字的由來

「Bundle」(光束)指的是: 從每個三維點, 發出許多道光線, 分別投射到看見它的各個相機。BA 的任務, 就是調整所有相機位姿與所有三維點的位置, 讓這些光束盡可能「對齊」——也就是讓每個三維點投影到影像上的位置, 盡量接近實際觀測到的像素位置。這個「投影與觀測的差」, 稱為重投影誤差(Reprojection Error)。

重投影誤差: 視覺 SLAM 的殘差

重投影誤差是 BA 的核心殘差, 它把第 9 章的相機投影模型用上了:

83. 取一個三維地圖點,用當前估計的相機位姿與內參 K (第 9 章),把它投影到影像上,得到一個「預測像素位置」。

84. 這個地圖點在該影像中,其實有一個「實際觀測到的像素位置」(前端特徵提取得到)。

85. 兩者的差(以像素為單位),就是重投影誤差。

BA 同時調整所有相機位姿與所有三維點,最小化所有重投影誤差的平方和。這是一個巨大的最小平方問題——一個場景可能有數百個相機位姿與數萬個三維點——但同樣靠稀疏性(每個點只被少數相機看到)得以高效求解。

BA 與 Pose Graph 的關係

- Bundle Adjustment: 同時最佳化相機位姿與三維點,精度最高,但變數多、計算量大。適合局部(第 21 章的 Local Mapping 執行緒)或關鍵時刻。
- Pose Graph: 只最佳化位姿(把三維點「邊緣化」掉),變數少、速度快。適合大範圍的閉環修正(第 21 章的 Loop Closing)。

現代視覺 SLAM 巧妙地結合兩者:平時用局部 BA 維持高精度,閉環時用位姿圖快速拉正大範圍軌跡。這種「局部 BA + 全域位姿圖」的架構,是 ORB-SLAM 系列的經典設計,第 21、22 章會完整展開。BA 的理論也正是第 2 章 MIT VNAV、CMU 16-833 課程強調的核心——同學現在應能理解,為什麼那些頂尖課程要花大量篇幅在最佳化上。

12.6 g2o、Ceres、GTSAM

前面的數學看似複雜,但好消息是:同學幾乎不需要自己實作這些迭代求解與稀疏矩陣運算。有三個成熟的開源函式庫替我們把重活做好了。認識它們的定位,是進入 SLAM 實作的必備知識。

三大函式庫

- **g2o(General Graph Optimization)**:專為圖優化設計的 C++ 函式庫,由 SLAM 社群開發。使用方式非常貼近本章的概念——你定義節點(位姿、地標)與邊(約束),它替你求解。ORB-SLAM2 即使用 g2o。對學習者而言,g2o 的架構最能對應本章的位姿圖概念,是理解圖優化的好起點。
- **Ceres Solver**:由 Google 開發的通用非線性最小平方函式庫,不限於 SLAM。它功能強大、文件完善、自動微分方便(不必手算雅可比),被廣泛用於各種最佳化問題。Cartographer(第 15 章)即使用 Ceres。
- **GTSAM(Georgia Tech Smoothing and Mapping)**:由喬治亞理工學院開發(第 2 章 CMU 16-833 的 Kaess 教授即為核心貢獻者),以因子圖(Factor Graph)為核心概念,並提供 iSAM2 等增量式求解演算法——能在新資料到來時高效更新,而不必每次從頭重算。

因子圖:另一種等價觀點

GTSAM 使用的「因子圖」是位姿圖的一個更一般化的表述。在因子圖中,待估計的變數是一種節點,而每個約束(稱為「因子」)是另一種節點,連接它所牽涉的變數。它與位姿圖在數學上等價,但因子圖的表述更靈活,能自然地容納各種感測器約束。第 2 章提過,因子圖是 CMU 16-833 課程的核心工具——這也顯示本章內容與國際頂尖課程完全接軌。

函式庫	開發者	核心概念	代表使用者
g2o	SLAM 社群	通用圖優化	ORB-SLAM2
Ceres	Google	通用非線性最小平方	Cartographer
GTSAM	喬治亞理工	因子圖 + 增量式求解	LIO-SAM 等

表 12-1 三大圖優化函式庫比較

對本書讀者的建議是:高中與大學部入門階段,理解「位姿圖—殘差—最小平方」的概念即可,不必深入函式庫細節;大學部專題與研究階段,則可從 g2o 入手(概念最貼近本章),再依專案需求選用 Ceres 或 GTSAM。第 21、29 章的實作會實際接觸這些工具。

12.7 為什麼現代 SLAM 常用圖優化

本章最後,回答一個貫穿第 11、12 兩章的問題:既然第 11 章的濾波器已經能做 SLAM,為什麼現代主流系統幾乎都改用圖優化?這個對比,是理解當代 SLAM 走向的關鍵。

圖優化的四大優勢

- 86. 能回頭修正歷史:濾波器把過去壓縮進當前信念,無法回頭;圖優化保留整張圖,閉環時能一次性修正所有歷史位姿,把漂移漂亮地攤平。這是精度差異的最大來源。
- 87. 反覆線性化,精度更高:EKF 對每個量測只線性化一次(且無法重來);圖優化在迭代中反覆重新線性化,逐步逼近真解,大幅減少第 11 章提到的線性化誤差。
- 88. 善用稀疏性,能處理大場景:圖優化的矩陣保持稀疏,配合稀疏演算法可處理數萬節點;EKF-SLAM 的共變異數矩陣會變稠密,隨地標平方成長而卡死。
- 89. 統一融合多感測器:各種約束都是圖上的邊,里程計、視覺、光達、IMU、GNSS 能自然地緊耦合在同一個問題裡(第 7 章的緊耦合思想)。

圖優化的代價

圖優化並非全無代價。它需要保留較多歷史資料(記憶體較大)、批次求解的單次計算量較大、且實作較複雜。早期受限於運算能力,濾波器才是主流;是計算能力的提升與稀疏求解演算法的成熟,才讓圖優化成為可能並最終勝出。這也提醒同學:技術的主流會隨硬體與演算法演進而變化,理解「為什麼」比記住「現在用什麼」更重要。

比較項目	濾波器(第 11 章)	圖優化(本章)
處理方式	遞迴,逐筆更新	批次,一次性最佳化
歷史修正	無法回頭	可修正所有歷史
線性化	每量測一次	迭代中反覆進行
閉環處理	較弱	強項
大場景	EKF 受限(稠密)	善用稀疏,可擴展
計算與記憶體	固定、輕量	較大
代表系統	AMCL、早期 EKF-SLAM	ORB-SLAM、Cartographer

表 12-2 濾波器與圖優化的總比較

至此,第三篇「SLAM 數學基礎」全部完成。同學已經備齊了 SLAM 的完整數學工具箱:第 10 章的座標轉換讓我們能在不同座標系間精確換算、第 11 章的機率與濾波讓我們能處理不確定性、本章的最佳化與圖優化則讓我們能求出最一致的全域解。這三章是抽象的,但它們是後面所有具體系統的共同地基。從下一篇起,我們將把這些數學「用出來」——第四篇從最親民的 2D SLAM 與掃地機器人開始,看 Gmapping 如何用第 11 章的粒子濾波建圖、Cartographer 如何用本章的圖優化做閉環。理論與實作,即將在真實的機器人上會合。

公式或流程

一、SLAM 的最小平方目標函數

圖優化 SLAM 的核心,是求一組狀態 X (所有位姿與地標),使加權殘差平方和最小:

$$X^* = \operatorname{argmin} \sum_k e_k(X)^T \cdot \Omega_k \cdot e_k(X)$$

其中 e_k 是第 k 條邊的殘差(量測與預測的差)、 Ω_k 是該邊的資訊矩陣(可信度權重,即共變異數的反矩陣)、 X^* 是最佳解。這條式子把整個 SLAM 濃縮成一句話:找一組位

姿與地圖,使所有約束的加權矛盾最小。第 11 章的機率最大化,在高斯假設下正等價於這個最小平方。

二、殘差的一般形式

位姿圖中,一條連接節點 i 與 j 的邊,其殘差為:

$$e_{ij} = \text{量測的相對位姿}(z_{ij}) \ominus \text{預測的相對位姿}(x_i, x_j)$$

符號 $\ominus$ 表示在位姿空間(第 10 章)中做「相減」——由於位姿含旋轉,這個相減不是普通減法,而需在旋轉群上進行(第 2 章 MIT VNAV 的流形運算)。殘差為零表示約束完美滿足。

三、重投影誤差(Bundle Adjustment)

視覺 SLAM 中,三維點 P 在第 i 個相機影像上的重投影誤差為:

$$e = u_{\text{observed}} - \pi(K, R_i, t_i, P)$$

其中 u_{observed} 是實際觀測到的像素座標、 $\pi(\cdot)$ 是第 9 章的投影函數(用相機位姿 R_i 、 t_i 與內參 K 把三維點 P 投影到影像)。BA 同時對所有相機位姿與三維點最小化所有重投影誤差平方和。

四、迭代求解流程(高斯—牛頓 / LM)

圖 12-2 圖優化的迭代求解流程

① 初始化:用里程計串起粗略的位姿(初始猜測 X_0)

↓

② 線性化:在當前 X 附近計算所有殘差與雅可比矩陣

↓

③ 建立線性系統: $H \cdot \Delta x = -b$ (H 為稀疏資訊矩陣)

↓

④ 稀疏求解: 利用稀疏性快速解出修正量 Δx

↓

⑤ 更新: $X \leftarrow X \oplus \Delta x$ (在位姿空間上更新)

↓

⑥ 收斂? 否 $\rightarrow$ 回到 ② / 是 $\rightarrow$ 輸出最佳解 X^*

這整個流程,就是 g2o、Ceres、GTSAM 在背後替我們執行的。同學理解流程即可,不需手動實作——正如開車不必自己造引擎。

案例

案例 12-1 閉環的威力:一張歪掉的地圖如何被拉正

這是圖優化最戲劇性的展示,幾乎每個 SLAM 課堂都會演示。一台機器人繞行一棟大樓一圈,全程只靠里程計與掃描匹配建圖。走到一半,地圖看起來還算正常;但快繞完一圈時,由於漂移累積,地圖開始「歪斜」——本該閉合的走廊,終點與起點錯開了好幾公尺,同一面牆甚至被畫成兩道分岔的線。

然後,閉環偵測啟動:系統認出「我現在看到的,就是一開始的大廳」,產生一條閉環邊。圖優化隨即執行——在螢幕上,同學會看到整張歪掉的地圖「唰」地一下被拉正:分岔的牆合而為一、錯開的走廊接合、整個軌跡變得規整。這一瞬間,本章 12.2 節「把矛盾均攤到整條軌跡」的抽象描述,變成了眼前活生生的畫面。

這個案例的震撼力在於:它讓同學親眼看到「一條邊如何修正整張圖」。這是濾波器做不到的——濾波器沒有保留歷史位姿,無法回頭。圖優化的全部價值,在這一個「拉正」的瞬間展露無遺。第 15 章 Cartographer 實作時,同學將親手觸發這個時刻。

案例 12-2 GTSAM 的血脈:從課堂到全球產業

第 2 章介紹 CMU 16-833 課程時提過:Michael Kaess 教授與 Frank Dellaert 教授開發的 GTSAM 函式庫,把因子圖與增量式平滑的理論包裝成開源工具,如今已是全球機器人公司與研究室的標準配備。本章的內容,正是理解 GTSAM 在做什麼的鑰匙。

GTSAM 的一項關鍵創新是 iSAM2(增量式平滑與建圖)。傳統圖優化每次有新資料都要重解整個問題;iSAM2 則聰明地只更新「受新資料影響的那部分」,大幅提升效率,讓大規模、長時間的 SLAM 得以即時運行。這解決了本章 12.7 節提到的「圖優化計算量大」的痛點。

這個案例串起了本書的多條線索:第 2 章的名校課程、本章的圖優化數學、以及真實世界的產業應用。它告訴同學:你正在學的這些數學,不是象牙塔裡的玩具,而是驅動水下機器人、自駕車、AR 裝置的實際引擎。理解本章,你就站在了與那些頂尖工程師相同的知識基礎上。

案例 12-3 從測量學到 SLAM:Bundle Adjustment 的百年血統

Bundle Adjustment 常被誤以為是電腦視覺的新發明,其實它的血統可追溯到十九世紀的攝影測量學(Photogrammetry)。當年的測量學家用多張空拍照片重建地形,面對的正是同一個問題:如何調整相機位置與地面點,讓所有照片的投影一致?他們發展出的平差理論,正是今天 BA 的前身。

這段歷史給同學兩個啟示。其一,最小平方與平差的思想歷久彌新——從十九世紀的測繪、二十世紀的阿波羅導航(第 10、11 章),到二十一世紀的 SLAM,人類處理「多

個帶誤差量測」的核心方法一脈相承。其二,跨領域的知識常常相通:測量學、攝影測量、電腦視覺、機器人學,在 BA 這個點上交會。這也呼應第 10 章「機械手臂與行動機器人共用同一套數學」的觀察——數學是所有工程領域的共同語言,學好基礎,處處相通。

實作活動

實作 12-1 位姿圖優化動手做

目的:透過紙上推演與程式實作,親身理解位姿圖如何被「拉正」,體會閉環約束的威力。
本實作分紙上、程式兩部分,適合不同深度的教學。

第一部分:紙上位姿圖(全體)

73. 在方格紙上畫一個「正方形巡行」:機器人從原點出發,依里程計「向右 4 格、向上 4 格、向左 4 格、向下 4 格」應回到原點。先照里程計畫出四個轉角位姿。
74. 加入漂移:假設每段的轉角都少轉了一點(例如每次只轉 85 度而非 90 度),重新畫出軌跡,觀察終點如何無法回到原點(出現漂移缺口)。
75. 加入閉環邊:宣告「終點就是起點」。用直覺思考:要讓終點回到起點,又盡量不破壞四條邊的長度,四個轉角應該如何微調?動手把「拉正後」的軌跡畫出來。
76. 討論:閉環邊把誤差「均攤」到四個轉角,而不是集中修正某一個。為什麼均攤比集中好?

第二部分:Python 位姿圖優化(大學部)

77. 用 Python 建立一個簡單的 2D 位姿圖:數個位姿節點、連接它們的里程計邊(含模擬漂移)、以及一條閉環邊。
78. 實作殘差計算:對每條邊,算出「量測的相對位姿」與「當前估計的相對位姿」之差。

79. 用現成的最佳化工具(如 `scipy.optimize.least_squares`, 或安裝 `g2o` / `GTSAM` 的 Python 介面)求解, 最小化所有殘差平方和。
80. 把「最佳化前」與「最佳化後」的軌跡畫在一起, 觀察漂移的軌跡如何被閉環邊拉正回閉合。
81. 實驗資訊矩陣: 把閉環邊的權重(資訊矩陣)分別調大與調小, 觀察拉正的力道如何變化。權重大時, 系統更「相信」閉環, 拉得更用力。

第三部分: 觀察真實 SLAM 的閉環(有 ROS 環境者)

82. 用 `Cartographer`(第 15 章)或其他支援閉環的 SLAM, 在模擬環境中讓機器人繞一圈, 在 `RViz` 中即時觀察: 繞行時地圖逐漸漂移, 閉環觸發瞬間地圖被拉正。
83. 錄下這個「拉正瞬間」, 對照案例 12-1 與本章 12.2 節, 指認哪些是里程計邊、哪條是閉環邊。

討論問題

59. 第一部分中, 加入閉環邊後, 你如何微調四個轉角? 最佳化把誤差均攤, 和「把所有誤差算在最後一段」相比, 哪個結果的地圖更合理? 為什麼?
60. 第二部分中, 調大閉環邊的權重, 拉正的力道有何變化? 若閉環偵測其實是「誤判」(錯誤的閉環), 高權重會造成什麼後果?
61. 本章說圖優化「能回頭修正歷史」, 而第 11 章的濾波器不能。從你的實驗結果, 說明這個差異如何影響最終地圖品質。
62. (大學部) 為什麼稀疏性讓大型位姿圖得以快速求解? 若每條邊都牽涉所有節點(不稀疏), 會發生什麼?

延伸挑戰(大學部)

安裝 `g2o` 或 `GTSAM` 的 Python 綁定, 載入一個公開的標準位姿圖資料集(如 `g2o` 格式的 `intel`、`manhattan` 資料集), 執行最佳化並用 `matplotlib` 視覺化最佳化前後的差異。

進一步嘗試「加入一條錯誤的閉環邊」,觀察它如何破壞整張地圖——這將幫助你理解為何第 19、21 章要花心力在「穩健的閉環偵測」上,以及穩健核函數(Robust Kernel)為何重要。

延伸閱讀

76. G. Grisetti, R. Kümmerle, C. Stachniss, and W. Burgard, "A Tutorial on Graph-Based SLAM," IEEE Intelligent Transportation Systems Magazine, 2010.(圖優化 SLAM 最經典的入門教程,本章主要依據)
77. 高翔、張濤,《視覺 SLAM 十四講》第 6 講(非線性最佳化)、第 9-10 講(後端與 BA、位姿圖).(華文世界最清楚的圖優化實作教材)
78. R. Kümmerle et al., "g2o: A General Framework for Graph Optimization," ICRA, 2011.(g2o 函式庫原始論文)
79. F. Dellaert and M. Kaess, "Factor Graphs for Robot Perception," Foundations and Trends in Robotics, 2017.(GTSAM 與因子圖的權威著作,可免費取得)
80. B. Triggs et al., "Bundle Adjustment — A Modern Synthesis," Vision Algorithms, 2000.(Bundle Adjustment 的權威綜述)
81. Ceres Solver 官方文件(ceres-solver.org)與 GTSAM 官方教學(gtsam.org).(三大函式庫的第一手實作指引)
82. CMU 16-833 Robot Localization and Mapping 課程(第 2 章介紹)之因子圖與圖優化單元.(研究所級的圖優化教材)

小結

49. 現代 SLAM 採前端 + 後端架構:前端把感測資料轉成幾何約束(掃描匹配、特徵匹配、閉環偵測),後端在所有(常互相矛盾的)約束下求最一致解;圖優化屬於後端。
50. 位姿圖以節點(待估的位姿與地標)與邊(相對量測約束)描述 SLAM;里程計邊像鏈子串起位姿、閉環邊形成迴路,是消除累積漂移的關鍵。
51. 每條邊有殘差(量測減預測)與資訊矩陣(可信度權重,即共變異數的反矩陣);圖優化把各種感測器約束統一成帶權重的邊,實現多感測器緊耦合。
52. SLAM 化為最小平方問題:找一組狀態使加權殘差平方和最小;用平方兼具數學(可微)與機率(高斯下等價於最大後驗)理由,以高斯—牛頓 / LM 迭代求解,並靠稀疏性處理大場景。
53. Bundle Adjustment 同時最佳化相機位姿與三維點,最小化重投影誤差,是視覺 SLAM 精度的來源;現代系統結合「局部 BA(高精度) + 全域位姿圖(快速閉環)」。
54. g2o(通用圖優化,ORB-SLAM2 使用)、Ceres(Google 通用最小平方,Cartographer 使用)、GTSAM(因子圖 + 增量式 iSAM2)三大函式庫替我們完成求解;因子圖是位姿圖的一般化表述。
55. 現代 SLAM 多採圖優化,因它能回頭修正歷史、反覆線性化提升精度、善用稀疏性處理大場景、統一融合多感測器;代價是記憶體與計算量較大,是硬體與演算法進步後才成為主流。

第三篇「SLAM 數學基礎」到此完成。我們已備齊完整的數學工具箱:座標轉換(第 10 章)、機率與濾波(第 11 章)、最佳化與圖優化(本章)。這三章抽象卻是一切的地基。從下一篇起,理論將在真實機器人上落地——第四篇從最親民的 2D SLAM 與掃地機器人開始,你會看到 Gmapping 如何運用第 11 章的粒子濾波、Cartographer 如何運用

本章的圖優化。數學與實作,即將會合。

台灣自編大學教科書

@prof

《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第四篇 2D SLAM 與掃地機器人

第 13 章

2D 地圖建構

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

學習目標

前三篇打好了觀念與數學地基,從本篇開始,理論要在真實機器人上落地了。第四篇聚焦最親民、也最貼近生活的 2D SLAM——掃地機器人用的正是這套技術。本章先講清楚「2D 地圖是什麼、怎麼建」,為後兩章的 Gmapping 與 Cartographer 鋪路。學完本章之後,同學應能夠達成下列目標:

76. 說明佔據格地圖的結構,以及每一格「佔據機率」的意義與更新方式。
77. 說明雷射掃描匹配如何估計機器人的相對移動。
78. 解釋為什麼要融合里程計與光達,兩者如何互補。
79. 說明地圖解析度的選擇如何影響地圖品質、記憶體與運算。
80. 分析走廊、教室、辦公室等典型室內場景的 2D 建圖挑戰。
81. 說明如何評估一張 2D 地圖的品質,辨識變形、斷裂與漂移等常見問題。

核心概念

中文名詞	英文名詞	概念說明
佔據格地圖	Occupancy Grid Map	把空間切成規則格子,每格記錄被佔據的機率。
佔據機率	Occupancy Probability	某格子被障礙物佔據的機率,介於 0 與 1 之間。
對數勝算比	Log-odds	以對數形式表示機率,使地圖更新變成簡單的加減法。
掃描匹配	Scan Matching	比對前後兩次雷射掃描,估計機器人的相對移動。
ICP	Iterative Closest Point	反覆找對應點並對齊的掃描匹配經典演算法。

中文名詞	英文名詞	概念說明
里程計	Odometry	由輪速估計的相對位移,連續但會漂移。
光線投射	Ray Casting	沿雷射光路徑更新途經格子的方法。
地圖解析度	Map Resolution	每個格子代表的實際邊長,如 5 公分。
自由空間	Free Space	雷射穿過、確認無障礙的區域。
未知區域	Unknown	感測器尚未觀測到的區域。
地圖漂移	Map Drift	因累積誤差導致地圖逐漸偏移或變形。
逆感測器模型	Inverse Sensor Model	由一次量測推算沿途各格佔據機率的模型。

13.1 Occupancy Grid Map

第 1 章與第 3 章我們初識過佔據格地圖,本節深入它的運作機制——這是 2D SLAM 最核心的地圖表示法,掃地機器人 App 上那張房間地圖,底層就是它。

把空間切成棋盤

佔據格地圖(Occupancy Grid Map)的想法極其樸素:把機器人活動的平面切成一格一格的方格(像棋盤或方格紙),每一格記錄一個數字——這一格「被障礙物佔據的機率」。

- 機率接近 1(習慣畫成黑色):幾乎確定有障礙,例如牆、桌腳。
- 機率接近 0(畫成白色):幾乎確定是空的,可以通行。
- 機率為 0.5(畫成灰色):不知道——感測器還沒看過這一格。

這三種狀態(佔據、自由、未知)的區分至關重要。特別是「未知」——它不等於「自由」。地圖邊緣那一大片灰色,代表機器人還沒探索的地方,不能當成可通行區域,否則規劃路徑時會「以為那裡是空的」而出錯。這是佔據格地圖比單純「有/無障礙」二元地圖高明的地方。

每一格獨立處理

佔據格地圖有一個關鍵假設:每一格的佔據狀態互相獨立。這在數學上並不完全成立(相鄰格子顯然相關——牆是連續的),但這個簡化讓計算變得極其高效:每來一筆雷射量測,只需更新它掃到的那幾格,不必考慮格子之間的關聯。這是工程上「用合理的簡化換取可行的計算」的又一個範例,與第 11 章佔據格地圖採高斯假設的精神一致。

前提:必須先知道位姿

這裡要強調一個第 1 章反覆出現的觀念:建佔據格地圖的前提,是已知機器人每一刻的位姿。因為一筆雷射量測「前方三公尺有牆」,只有知道機器人在哪、朝哪,才能決定

這面牆該畫在地圖的哪一格。

於是問題來了——位姿從哪來?這正是 SLAM 的雞生蛋問題。本章先假設「位姿已由某種方式提供」(例如里程計加掃描匹配),專注於「有了位姿,如何建圖」;至於「如何同時估好位姿」,是第 14、15 章 Gmapping 與 Cartographer 的核心。把「建圖」與「定位」暫時拆開理解,是學習 SLAM 的有效策略。

13.2 佔據機率的更新:對數勝算比

機器人不會只看一眼就下結論——它會反覆掃描同一個地方,每次掃描都提供新證據,地圖應該把這些證據累積起來。本節說明這個累積是怎麼做的,以及一個讓計算變簡單的巧妙技巧。

累積證據:貝氏更新

每一格的佔據機率,隨著一次次觀測而更新。若某格連續多次被雷射打到(每次都說「這裡有東西」),它的佔據機率就逐漸逼近 1;若連續多次被雷射穿過(每次都說「這裡是空的」),機率就逼近 0。這正是第 11 章貝氏更新的應用——用新證據修正既有信念。

但直接對機率做貝氏更新,需要反覆做乘法與除法(還要正規化),計算較繁瑣,而且機率被限制在 0 到 1 之間,接近邊界時數值容易不穩定。於是實務上採用一個漂亮的技巧。

對數勝算比:把乘除變成加減

這個技巧是把機率改寫成對數勝算比(Log-odds)形式。勝算(odds)是「佔據機率 ÷ 空置機率」,再取對數就是對數勝算比。它的好處立竿見影:

- 更新變成加法:每來一筆觀測,只需把對應的「對數勝算比增量」加上去即可——乘除變成了加減,計算極快。

- 數值範圍無界:對數勝算比可以是任意實數(正無窮對應必定佔據、負無窮對應必定自由、0 對應五五波),不再卡在 0 到 1 的邊界,數值穩定。
- 需要時再轉回機率:平時都用對數勝算比累積,只在要顯示或決策時,才轉換回 0 到 1 的機率。

這個「取對數把乘除變加減」的手法,在工程與科學中隨處可見(例如分貝、pH 值),佔據格地圖只是其中一例。同學不必記憶公式,但要理解這個設計動機:讓「累積大量觀測」這件事變得又快又穩。

光線投射:一筆量測更新一整條線

這裡還有一個容易忽略但很重要的細節。當雷射量到「前方三公尺有牆」,它其實更新了一整條線上的格子,而不只是牆那一格:

- 雷射終點那一格(牆):增加佔據機率(這裡有東西)。
- 雷射沿途穿過的所有格子(三公尺內):降低佔據機率(雷射能穿過,表示這些地方是空的)。

這個「沿雷射路徑更新沿途格子」的過程稱為光線投射(Ray Casting),背後的模型稱為逆感測器模型(Inverse Sensor Model)——由一次量測反推沿途各格的佔據狀態。正是這個機制,讓機器人不只知道「哪裡有牆」,也知道「哪裡是確定可走的空間」,這對第 28 章的路徑規劃至關重要。

13.3 雷射掃描匹配

上一節假設「位姿已知」。但位姿到底怎麼估?除了里程計,光達本身就能提供強大的位姿線索——這就是掃描匹配(Scan Matching),2D SLAM 前端的核心技術。

核心想法:讓兩次掃描對齊

掃描匹配的想法非常直觀。機器人在位置 A 掃描一次,前進一小步到位置 B 再掃描一次。如果環境沒變(牆還是那面牆),那麼「B 的掃描」相對「A 的掃描」的位移與旋轉,就反映了機器人從 A 到 B 的移動。

換句話說:找出一個旋轉與平移,把 B 的掃描疊到 A 的掃描上,使兩者「最吻合」——這個旋轉與平移,就是機器人的相對移動。這是一個最佳化問題(第 12 章的精神):調整位姿,讓兩次掃描的差異最小。

ICP:最經典的掃描匹配

最著名的掃描匹配演算法是 ICP(Iterative Closest Point,迭代最近點)。它的流程是一個反覆逼近的循環:

90. 對應(Correspondence):對 B 掃描中的每個點,在 A 掃描中找最近的點,配成一對。
91. 求解(Alignment):計算一個旋轉與平移,使所有配對點的距離總和最小。
92. 更新與迭代:套用這個變換,然後回到步驟 1 重新找對應點,反覆進行直到不再改善(收斂)。

ICP 的名字精準描述了它的運作:反覆(Iterative)地找最近點(Closest Point)並對齊。它簡單有效,是點雲配準的基石,第 17 章的 3D 建圖還會再遇到它。

掃描匹配的優點與陷阱

- 優點:光達精確,掃描匹配估出的相對移動,通常比里程計準得多,尤其在轉彎時不受輪子打滑影響。
- 陷阱——環境重複:走廊、空曠大廳等「處處長得一樣」的場景,掃描匹配會找不到唯一的對齊方式(沿走廊滑動,掃描看起來都差不多),導致估計失準。這與第 9 章「白牆的詛咒」是同一類問題:缺乏獨特特徵。

- 陷阱二——初始猜測:ICP 需要一個好的初始猜測才容易收斂到正確解,否則可能陷入錯誤的局部最佳。這正是里程計派上用場的地方——下一節說明。

13.4 里程計與光達融合

第二篇反覆強調「感測器融合」,2D SLAM 是這個思想最乾淨的示範:里程計與光達的優缺點恰好互補,合起來遠勝任何一方單獨使用。

兩者的互補

特性	里程計(輪速)	光達掃描匹配
資料連續性	連續、高頻、平滑	受掃描頻率限制
短期精度	尚可	高(尤其轉彎)
長期行為	漂移累積	相對穩定
失效場景	打滑、地面不平	環境重複、特徵不足
需要初始猜測	不需要	需要(否則易發散)

表 13-1 里程計與光達掃描匹配的互補

融合的邏輯:里程計給起點,光達來校正

看出關鍵了嗎?里程計不需要初始猜測、且連續平滑,恰好能為「需要初始猜測」的掃描匹配提供起點;而掃描匹配精確、不漂移,恰好能校正里程計的漂移。兩者一拍即合:

93. 機器人移動一步,里程計先給出一個粗略的相對移動估計(初始猜測)。
94. 以這個猜測為起點,執行掃描匹配(ICP),精修出更準確的相對移動。
95. 用精修後的位姿更新地圖(13.2 節),並作為下一步的基礎。

這個「里程計預測、光達校正」的循環,正是第 11 章貝氏濾波「預測—更新」的具體化身:里程計負責預測(運動模型)、光達負責更新(觀測模型)。同學應該能感受到,第三篇的抽象數學,在此變成了可操作的建圖流程。第 14 章的 Gmapping 與第 15 章的 Cartographer,都是這個基本循環的精緻化。

打滑時怎麼辦

當輪子打滑,里程計會突然給出錯誤的大位移。此時掃描匹配的價值更加凸顯:即使里程計說「我前進了半公尺」,只要光達掃描顯示「環境幾乎沒變」,掃描匹配就會把估計拉回「其實幾乎沒動」。這種「以環境觀測否決錯誤運動估計」的能力,正是 SLAM 比純里程計航位推算強韌的根本原因。

13.5 地圖解析度與更新

建 2D 地圖時,一個必須做的關鍵決定是:每個格子代表多大的實際範圍?這就是地圖解析度(Map Resolution),它牽動地圖品質、記憶體與運算的三方權衡。

解析度的取捨

地圖解析度指每個格子的實際邊長,室內常用 5 公分。這個選擇是一個典型的工程權衡:

- 解析度高(格子小,如 1 公分):地圖細膩,能呈現細小結構(桌腳、門框),但記憶體用量暴增(格子數與邊長平方成反比——邊長減半,格子數變四倍),運算也變慢。
- 解析度低(格子大,如 20 公分):記憶體省、運算快,但地圖粗糙,細小障礙可能被整格「抹平」,牆的位置也不精確。

如何選?取決於應用需求。掃地機器人不需分辨一公分的細節,5 公分綽綽有餘;而需要精密對接的機器人,可能需要更高解析度。這呼應第 8 章光達角解析度的討論——地圖解析度再高,也不可能超越感測器本身的解析能力,兩者要匹配。

記憶體的現實

讓同學對記憶體用量有具體概念:一個 20 公尺 $\times$ 20 公尺的房間,用 5 公分解析度,就是 $400 \times 400 = 16$ 萬個格子。若解析度提高到 1 公分,則變成 $2000 \times 2000 = 400$ 萬個格子——整整 25 倍。對大範圍場景(整棟大樓),這個成長會很可觀,這也是第 3 章提到 3D 地圖要用八叉樹(OctoMap)壓縮的原因。2D 佔據格因為只有平面,記憶體壓力相對小得多,這也是它適合教學與掃地機器人的原因之一。

動態更新:地圖不是拍一張照就好

地圖需要持續更新,原因有二:

- 探索新區域:機器人走到沒去過的地方,把「未知」的灰色格子填成「自由」或「佔據」。
- 修正舊估計:反覆觀測同一區域,累積證據修正之前不確定的格子(13.2 節的貝氏更新)。

但這帶來一個微妙問題:如果環境本身變了怎麼辦?例如有人搬動了椅子。若地圖對舊觀測「記憶」太深(對數勝算比累積太高),就很難更新成新狀態;若記憶太淺,又容易被雜訊干擾。實務上會對對數勝算比設上下限(clamping),讓地圖保持一定的「可塑性」,能反映環境變化。這是第 16 章掃地機器人「動態環境」討論的伏筆。

13.6 走廊、教室、辦公室的 2D 建圖

理論講完,來看真實室內場景。不同的空間結構,對 2D 建圖有截然不同的挑戰。理解這些,同學才能預判「哪裡容易出問題」,這是實作前的重要準備。

教室:2D SLAM 的友善場景

一般教室是 2D SLAM 最理想的場景:四面牆界線分明、有桌椅提供豐富的幾何特徵、空間封閉不大。掃描匹配在這裡能穩定運作,建出的地圖清晰。這也是為什麼本書

的實作活動多以教室為場域(第 14、28 章)——它讓同學在最順利的條件下先建立成功經驗。

走廊:掃描匹配的惡夢

長直走廊是 2D SLAM 的經典難題,正是 13.3 節「環境重複」陷阱的典型。走廊兩側是平行的長牆,沿走廊方向移動時,雷射掃描看起來幾乎不變——掃描匹配無法判斷「沿走廊走了多遠」(這個方向缺乏約束),只能確定「有沒有偏離走廊中線」(垂直方向有牆約束)。

這正是第 11 章 11.2 節那個「沿走廊方向拉長的不確定性橢圓」的來源!此時里程計的價值凸顯——它能提供沿走廊方向的移動估計,補上掃描匹配缺失的約束。這是里程計與光達融合(13.4 節)不可或缺的又一個實例。長走廊建圖若只靠光達,幾乎必定失敗。

辦公室:動態與玻璃的挑戰

開放式辦公室綜合了多種挑戰:隔間玻璃讓光達「看穿」或產生錯誤反射(第 8 章的玻璃問題)、人員走動造成動態干擾、相似的格位排列容易混淆。這類環境往往需要更精細的參數調校,或搭配其他感測器。

大廳與中庭:空曠的困境

大型空曠空間(如挑高大廳)的問題是:光達的量測範圍有限(教學型約 8 至 12 公尺,第 8 章),當空間大於量測範圍,機器人走到中央時可能「四面都掃不到牆」,完全失去參考。此時掃描匹配無從進行,只能倚賴里程計硬撐,誤差快速累積。

場景	主要挑戰	關鍵對策
教室	幾乎無(理想場景)	直接建圖即可

場景	主要挑戰	關鍵對策
長走廊	環境重複,沿走廊方向無約束	倚賴里程計補足縱向估計
辦公室	玻璃、動態人員、格位相似	參數調校、多感測器
空曠大廳	超出光達量程,四周掃不到	里程計硬撐、加大量程或加地標

表 13-2 典型室內場景的 2D 建圖挑戰

13.7 地圖品質評估:變形、斷裂、漂移

建完地圖,怎麼判斷它「建得好不好」?這是實作中極重要卻常被忽略的一環。本節教同學辨識三種常見的地圖病徵,以及如何評估品質——這也是第 30 章專題評分 Rubric 的基礎。

三種常見的地圖病徵

- 漂移(Drift):最常見的問題。累積誤差使地圖逐漸偏移,最典型的症狀是「繞一圈回不到原點」——本該閉合的迴路,終點與起點錯開。這正是第 12 章閉環要解決的問題,也是判斷「需不需要閉環偵測」的依據。
- 變形(Distortion):地圖的幾何被扭曲,直牆變成彎的、直角變成歪的、房間比例失真。多半來自掃描匹配的系統性誤差或里程計的尺度誤差(輪徑校正不準)。
- 斷裂/重影(Ghosting):同一面牆在地圖上出現兩道(重影),或牆線斷開。通常是位姿估計在某處突然跳錯,導致同一結構被畫在兩個位置。走廊、空曠處最容易發生。

如何評估地圖品質

評估方法分主觀與客觀兩類:

- 目視檢查(定性):牆是否筆直?直角是否方正?迴路是否閉合?有無重影與斷裂?這是最快的第一步,有經驗的人一眼就能看出地圖好壞。

- 對照實測(定量):量測地圖上幾個關鍵距離(走廊長度、房間寬度),與捲尺實測值比較,計算誤差百分比。這是客觀量化地圖精度的方法。
- 閉環一致性:讓機器人繞迴路回到起點,量測地圖上「起點」與「終點」的落差。落差越小,表示漂移控制越好。
- 與真值比對(研究級):在有動作捕捉系統或已知精確地圖的環境,把估計地圖與真值疊合比對(第 30 章專題會用到)。

養成「建完圖就評估品質」的習慣至關重要。一張看似完整、實則變形的地圖,會讓後續的路徑規劃(第 28 章)全盤出錯——機器人會照著一張錯誤的地圖撞牆。這呼應第 8 章「感測器限制直接變成系統失敗」的觀念:地圖品質,直接決定整個導航系統的成敗。

至此,同學已經理解 2D 地圖「是什麼、怎麼建、怎麼評估」。但本章刻意把「建圖」與「定位」分開,假設位姿已知。真實的 SLAM 必須同時解這兩者——下一章 Gmapping 登場,它用第 11 章的粒子濾波,讓每個粒子各自攜帶一張地圖,一邊定位一邊建圖,漂亮地解開這個雞生蛋問題。理論與演算法,終於要合而為一了。

公式或流程

一、對數勝算比更新

某格子的佔據機率 p ,其對數勝算比定義為:

$$l = \log(p / (1 - p))$$

每來一筆觀測,新的對數勝算比只需做加法:

$$l_{new} = l_{old} + l_{measurement}$$

其中 $l_{measurement}$ 是本次觀測帶來的證據(佔據觀測為正、自由觀測為負)。這就是 13.2 節「把乘除變加減」的數學核心。需要顯示或決策時,再轉回機率:

$$p = 1 - 1 / (1 + \exp(l))$$

l 為正表示傾向佔據、為負表示傾向自由、為零表示五五波。實務上對 l 設上下限, 保持地圖對環境變化的可塑性。

二、掃描匹配的最佳化目標

掃描匹配求一個相對位姿 T (旋轉 + 平移), 使當前掃描 S 對齊參考掃描 R 後的總誤差最小:

$$T^* = \operatorname{argmin} \sum_i \| T \cdot s_i - r_j(i) \|^2$$

其中 s_i 是當前掃描的點、 $r_j(i)$ 是參考掃描中與之對應的最近點。這是一個最小平方問題(第 12 章), ICP 以「找對應—求解—迭代」反覆逼近其解。

三、里程計與光達融合建圖流程

圖 13-1 2D SLAM 單步建圖流程(本章版本, 假設閉環另行處理)

① 機器人移動一步

↓

② 里程計估出粗略相對移動(初始猜測)

↓

③ 以初始猜測為起點, 執行掃描匹配(ICP)精修相對移動

↓

④ 更新機器人位姿(累加精修後的相對移動)

↓

⑤ 光線投射: 沿雷射路徑, 以對數勝算比更新沿途格子



⑥ 回到 ①,處理下一步(週期性檢查閉環 → 第 14、15 章)

四、地圖記憶體估算

一張佔據格地圖的格子總數為:

$$\text{格子數} = (\text{寬} / \text{解析度}) \times (\text{高} / \text{解析度})$$

例如 20 m × 20 m 房間、解析度 0.05 m: 格子數 = $400 \times 400 = 160,000$ 格。解析度減半(0.025 m)則格子數變四倍(640,000 格)——這條式子量化了 13.5 節「解析度與記憶體」的平方關係。

案例

案例 13-1 掃地機器人 App 上那張地圖:佔據格的日常

打開任何一款支援地圖功能的掃地機器人 App,那張你用來設定禁區、指定房間的房間地圖,底層就是本章的佔據格地圖。黑色的牆線是高佔據機率的格子、白色的可清掃區是低佔據機率的自由空間、而機器人還沒去過的地方則是空白(未知)。

觀察這張地圖的「成長過程」很有教育意義:機器人第一次清掃時,地圖是一點一點「長」出來的——走到哪、掃到哪、畫到哪,未知的區域逐漸被填滿。這正是 13.5 節「動態更新—探索新區域」的視覺化。若機器人在某處打滑或撞到黑色家具(光達看不到,第 8 章),你可能會在地圖上看到牆線歪斜或缺口——這就是 13.7 節的漂移與斷裂,活生生出現在你家客廳。

更進階的觀察:有些掃地機器人第二次清掃同一個家時,地圖會比第一次更精確、更平整。這是因為它累積了更多觀測,對每一格的估計更有把握(13.2 節的貝氏累積),甚至

可能執行了閉環修正(第 12 章)。一台掃地機器人,其實是本篇所有理論的隨身教材。

案例 13-2 長走廊翻車記:一個真實的教學失敗

大學課堂上,一個幾乎年年上演的場景:同學在教室裡把 2D SLAM 建圖跑得很順,信心滿滿地把機器人開進長廊,結果地圖瞬間「爆掉」——走廊被畫得忽長忽短、牆線扭曲重疊,機器人徹底迷失。

原因就是 13.3 與 13.6 節反覆強調的「環境重複」:長走廊沿縱向缺乏獨特特徵,掃描匹配無法確定「沿走廊走了多遠」。若此時里程計又因地面材質改變(磁磚換成地毯)而不準,兩個資訊都失效,建圖必然失敗。

這個「失敗」其實是最好的一堂課。它讓同學親身體會:SLAM 不是萬能的,環境結構會直接決定成敗。解法呼應全書:融合可靠的里程計(13.4 節)、加大光達量程看到走廊盡頭、或在走廊貼 AprilTag(第 4 章)提供獨特地標。理解為什麼失敗,比在教室裡順利建圖學到的更多——這也是本書為何在每一章都誠實討論「限制與失敗」的原因。

案例 13-3 從 2D 到智慧:掃地機器人的演進

回顧掃地機器人的演進,是一部 2D SLAM 的普及史。最早的機型(2002 年前後)根本沒有地圖,採「隨機碰撞」——撞到就轉彎,清掃路線雜亂、重複又遺漏(第 3、16 章)。2010 年 Neato 首度把 2D 光達裝上掃地機,讓它能建佔據格地圖、規劃系統性路徑。2017 年小米以親民價格普及光達掃地機(第 1 章案例),讓佔據格地圖進入千家萬戶。

近年的機型更進一步:加入 3D ToF 相機辨識並繞開電線、鞋子、寵物排泄物(彌補 2D 光達的高度盲區,第 8 章);用 AI 辨識房間類型自動分區;甚至記住多層樓的地圖。這條演進線清楚顯示:2D 佔據格地圖是基礎,但真實產品會不斷疊加感測器與 AI 來彌補單一技術的限制——這正是全書貫穿的融合思想,也預告了第九篇 AI 語意導航的方向。同學今天學的 2D 建圖,是理解這一切的起點。

實作活動

實作 13-1 親手建一張佔據格地圖

目的:透過程式從零實作佔據格地圖的更新,親身理解對數勝算比、光線投射與掃描匹配。

本實作以 Python 為主,分模擬與資料兩條路徑,是第 14 章 Gmapping 實作的直接暖身。

第一部分:對數勝算比更新(全體)

84. 用 numpy 建立一個二維陣列當作地圖(例如 200×200 格,每格初始對數勝算比為 0,代表未知)。
85. 模擬單一雷射光束:給定機器人位置、光束方向與量到的距離,實作光線投射——把沿途格子的對數勝算比減少(自由)、終點格子增加(佔據)。
86. 把對數勝算比轉回機率,用 matplotlib 以灰階顯示地圖(黑=佔據、白=自由、灰=未知),確認單束雷射畫出一條「白線到黑點」。
87. 模擬一整圈掃描(360 束):讓機器人在房間中央,對四周的模擬牆面各發一束,觀察是否畫出房間輪廓。
88. 多次觀測:讓機器人在同一位置重複掃描數次(加入隨機雜訊),觀察牆線如何隨觀測次數增加而變得更「確定」(對數勝算比累積,顏色更黑)。

第二部分:移動建圖(全體或大學部)

89. 設計一個簡單的模擬房間(用程式定義幾道牆),讓一台虛擬機器人沿預定路線移動,每步用已知位姿與模擬光達更新地圖,觀察完整房間地圖如何逐步「長」出來。
90. 加入里程計雜訊:故意讓機器人的位姿估計帶有累積誤差(漂移),觀察建出的地圖如何變形、牆線如何重影——親身重現 13.7 節的病徵。

91. 解析度實驗:把同一場景分別用 2 公分、5 公分、20 公分解析度建圖,比較地圖細膩度、記憶體用量(格子數)與運算時間。

第三部分:ROS 真實/模擬建圖(有 ROS 環境者)

92. 在 TurtleBot3 模擬(第 27 章)中,啟動內建的 2D SLAM(如 slam_toolbox 或 gmapping),遙控機器人在模擬教室走一圈,在 RViz 中即時觀察佔據格地圖的成長。
93. 把機器人開進模擬長走廊,觀察本章 13.6 節描述的「走廊困境」是否出現(地圖沿走廊方向變形)。
94. 建完圖後,用 13.7 節的方法評估品質:目視檢查牆是否筆直、量測地圖上的房間尺寸與模擬設定值比較。

討論問題

63. 第一部分中,同一面牆被觀測 1 次與觀測 10 次,地圖上的顏色(佔據機率)有何差異?這說明對數勝算比累積的什麼作用?
64. 第二部分加入里程計漂移後,地圖出現了哪一種病徵(變形/斷裂/漂移)?若要修正它,你會用第 12 章的什麼技術?
65. 解析度從 5 公分改為 2 公分,格子數增加了幾倍?記憶體與運算時間的變化符合 13.5 節的平方關係嗎?
66. (有 ROS 者)長走廊的地圖出了什麼問題?對照 13.6 節,你認為加入里程計、加大光達量程、貼 AprilTag,哪個對策最有效?為什麼?

延伸挑戰(大學部)

實作一個簡化版的 ICP 掃描匹配:給定兩次帶有已知相對位移的模擬掃描,用「找最近對應點—求最佳旋轉平移—迭代」的流程,估計出這個相對位移,並與真值比較。進一步在「長走廊」情境(縱向特徵不足)測試你的 ICP,觀察它為何沿走廊方向估計失準——

這將讓你深刻理解為何 Gmapping(第 14 章)要用粒子濾波,而非單純倚賴掃描匹配。

延伸閱讀

83. S. Thrun, W. Burgard, and D. Fox, Probabilistic Robotics, MIT Press, 2005.(第 9 章 Occupancy Grid Mapping 為本章的權威依據)
84. Cyrill Stachniss, SLAM 課程之 Occupancy Grid Maps、Scan Matching、ICP 系列影片(YouTube).(以動畫講解本章核心,自學首選)
85. P. Besl and N. McKay, "A Method for Registration of 3-D Shapes," IEEE TPAMI, 1992.(ICP 掃描匹配的原始論文)
86. A. Elfes, "Using Occupancy Grids for Mobile Robot Perception and Navigation," Computer, 1989.(佔據格地圖的開創性論文)
87. slam_toolbox 開源專案(GitHub)與 ROS 官方文件.(現代 ROS 2 主流的 2D SLAM 工具,取代舊 gmapping,實作首選)
88. ROBOTIS TurtleBot3 e-Manual 之 SLAM 章節(emanual.robotis.com).(2D 建圖的完整實作步驟)
89. 高翔、張濤,《視覺 SLAM 十四講》之地圖相關章節.(雖以視覺為主,對地圖表示法的討論可與本章互補)

小結

56. 佔據格地圖把平面切成方格,每格記錄佔據機率(佔據/自由/未知三態);「未知」不等於「自由」,這是它優於二元地圖之處。建圖的前提是已知位姿,本章暫將建圖與定位分開,專注於「有位姿如何建圖」。

57. 估據機率隨觀測以貝氏方式累積更新;實務用對數勝算比,把乘除變成加減、數值範圍無界又穩定;一筆雷射量測透過光線投射,更新終點格(估據)與沿途格(自由)一整條線。
58. 掃描匹配比對前後兩次雷射掃描估計相對移動,經典演算法為 ICP(找對應—求解—迭代);它精確但怕環境重複(走廊)且需要好的初始猜測。
59. 里程計與光達恰好互補:里程計連續、免初始猜測但漂移,光達精確、不漂移但需初始猜測且怕重複環境;融合邏輯是「里程計預測、光達校正」,正是貝氏濾波預測—更新的化身,並能以環境觀測否決打滑造成的錯誤運動。
60. 地圖解析度是品質、記憶體、運算的三方權衡,格子數與解析度成平方反比;地圖需動態更新以探索新區並修正舊估計,並對對數勝算比設限以保持對環境變化的可塑性。
61. 不同場景挑戰各異:教室理想、長走廊因縱向無約束而最難(需里程計補足)、辦公室有玻璃與動態、空曠大廳超出光達量程;預判挑戰是實作成功的關鍵。
62. 地圖三大病徵為漂移(繞圈回不到原點)、變形(幾何扭曲)、斷裂重影(同牆兩線);評估方法有目視檢查、對照實測、閉環一致性與真值比對。地圖品質直接決定導航成敗。

本章刻意把建圖與定位分開,假設位姿已知。但真正的 SLAM 必須同時解開這兩者——這正是第 1 章的雞生蛋問題。下一章 Gmapping 登場:它運用第 11 章的粒子濾波,讓每一個粒子各自攜帶一條軌跡與一張地圖,在眾多假設中「適者生存」,漂亮地同時完成定位與建圖。第三篇的粒子濾波理論,即將在第一個完整的 SLAM 系統中大顯身手。

台灣自編大學教科書

[@prof](#)

《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第 14 章

Gmapping : 粒子濾波 SLAM

編者：李世淵 (機器人叫獸)

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

學習目標

上一章我們假設「位姿已知」來建圖,但真正的 SLAM 必須同時解開定位與建圖。本章介紹第一個完整的 SLAM 系統——Gmapping。它運用第 11 章的粒子濾波,讓每個粒子各自攜帶一條軌跡與一張地圖,在眾多假設中「適者生存」,漂亮地解開第 1 章的雞生蛋問題。學完本章之後,同學應能夠達成下列目標:

82. 說明 Gmapping 的基本概念與它在 SLAM 發展史上的地位。
83. 解釋 Rao-Blackwellized 粒子濾波的核心思想:為何「每個粒子帶一張地圖」能解開雞生蛋問題。
84. 說明粒子數、權重與重採樣的作用,以及粒子退化與有效粒子數的概念。
85. 說明 Gmapping 兩項關鍵改良:改良提議分布與選擇性重採樣。
86. 說明在 ROS 中執行 Gmapping 建圖的完整流程與必要前置條件。
87. 辨識常見的建圖失敗案例——輪子打滑、場景重複、光達遮蔽——並提出對策。
88. 完成以教學小車或模擬器建立教室地圖的實作。

核心概念

中文名詞	英文名詞	概念說明
粒子濾波 SLAM	RBPF-SLAM	以粒子濾波同時估計軌跡與地圖的 SLAM 方法。
Rao-Blackwell 化	Rao-Blackwellization	把問題拆解,一部分用粒子取樣、另一部分用解析法求解。
粒子	Particle	一個完整的假設:包含一條軌跡與其對應的一張地圖。

中文名詞	英文名詞	概念說明
權重	Weight	該粒子的假設與感測器觀測吻合的程度。
重採樣	Resampling	依權重複製高分粒子、淘汰低分粒子的過程。
粒子退化	Particle Depletion	重採樣過度導致粒子多樣性喪失的現象。
有效粒子數	Effective Sample Size	衡量粒子多樣性的指標,用於決定是否重採樣。
提議分布	Proposal Distribution	產生下一步粒子候選位置的機率分布。
掃描匹配	Scan Matching	比對雷射掃描以精修位姿,Gmapping 用它改良提議分布。
選擇性重採樣	Selective Resampling	只在有效粒子數過低時才重採樣,避免退化。
地圖伺服器	map_server	ROS 中儲存與載入地圖檔的工具。
地圖檔	pgm / yaml	ROS 地圖的影像檔與描述檔格式。

14.1 Gmapping 的基本概念

Gmapping 是 2D 雷射 SLAM 領域最著名、也最廣泛使用的開源系統之一。它由 Grisetti、Stachniss 與 Burgard 於 2007 年提出——正是第 2 章介紹的德國 Freiburg / Bonn 機率機器人學研究群。同學讀完本章後,若去看 Stachniss 教授在 YouTube 上親自講解粒子濾波 SLAM 的影片,會有「原作者現身說法」的難得體驗。

Gmapping 解決什麼問題

第 13 章我們刻意假設「位姿已知」,專注於建圖。但真實機器人的位姿正是未知的——而且位姿錯了,地圖就跟著錯。Gmapping 要同時回答兩個互相依賴的問題:機器人走過的軌跡是什麼?環境地圖長什麼樣?

它的答案極具巧思:不要試圖找出「唯一正確」的軌跡,而是同時維持數十到數百個「可能的軌跡假設」,每一個假設都各自建一張地圖;然後看哪些假設與感測器觀測最吻合,讓它們存活繁衍,不吻合的則被淘汰。這正是第 11 章粒子濾波「適者生存」思想的直接應用。

輸入與輸出

- 輸入:2D 光達掃描(LaserScan,第 8 章)、里程計(第 3、13 章)、以及機器人的座標框架設定(TF,第 10 章)。
- 輸出:一張佔據格地圖(第 13 章)、以及機器人在地圖中的位姿(即 map→odom 的座標變換,第 10.7 節)。

注意這個輸入輸出清單:它幾乎把前面所有章節都用上了。這正是本篇的意義——理論在此匯流成一個可以真正跑起來的系統。

歷史地位

Gmapping 長年是 ROS 生態中 2D SLAM 的預設選擇,無數大學實驗室與教學平台

以它入門。近年在 ROS 2 環境中,slam_toolbox 等基於圖優化(第 12 章)的方案逐漸成為主流,但 Gmapping 的教學價值絲毫未減:它是理解「粒子濾波如何做 SLAM」最清晰的範例,也是連接第 11 章理論與真實系統的最佳橋樑。

14.2 Rao-Blackwellized Particle Filter

這是本章最核心的觀念。Gmapping 的全名是「Rao-Blackwellized 粒子濾波 SLAM」,這個拗口的名詞背後,是一個非常聰明的拆解。

直接用粒子濾波做 SLAM 的困境

回想第 11 章:粒子濾波用一群樣本近似機率分布。若直接套用到 SLAM,每個粒子必須同時代表「機器人位姿」與「整張地圖」的一個假設。問題是——地圖有數萬個格子,等於數萬個未知數。要用粒子在如此高維的空間中取樣,所需粒子數會是天文數字,完全不可行。這稱為維度詛咒(Curse of Dimensionality)。

Rao-Blackwell 化:聰明的拆解

Rao-Blackwell 化的核心洞見是:把問題拆成兩部分,一部分用粒子取樣,另一部分用解析方法直接算。

關鍵觀察在於第 13 章的結論:只要知道機器人的完整軌跡,建地圖就是一件確定性的簡單事(用對數勝算比更新即可,不需取樣)。換句話說,地圖並不需要「猜」,它可以由軌跡直接算出來。

於是拆解就很自然了:

- 用粒子取樣的部分:機器人的軌跡(只有每步三個數: x 、 y 、 θ)。這是低維的,粒子濾波應付得來。

- 用解析方法算的部分:地圖。每個粒子拿著自己那條軌跡,用第 13 章的方法直接算出對應的地圖。

觀念核心:每個粒子是一個「平行世界」

想像 100 個平行世界,每個世界裡的機器人走了略微不同的路線(軌跡假設),因此各自畫出略微不同的地圖。

新的雷射掃描到來時,每個世界都問自己:「以我的地圖與位置,這筆掃描合理嗎?」

合理的世界得高分(權重高)、不合理的得低分。

重採樣時,高分世界被複製、低分世界消失。存活下來的,就是與觀測最一致的假設。

這就是 Gmapping:用平行世界的競爭,同時找出正確的軌跡與地圖。

為什麼這樣就解開了雞生蛋問題

第 1 章的雞生蛋問題是:建圖需要位姿、定位需要地圖。RBPF 的解法是——不必先有誰。每個粒子先「假設」一條軌跡(不需要地圖就能假設),依此軌跡建圖(建圖只需軌跡),再用觀測評估這個假設好不好(評估需要地圖,而地圖剛剛建好了)。整個循環自洽,不需要任何一方先存在。這個優雅的拆解,正是 Gmapping 在 SLAM 史上的重要貢獻。

14.3 粒子數、權重與重採樣

本節深入 RBPF 的三個實務關鍵:要用幾個粒子?怎麼給分?什麼時候淘汰?

一、粒子數:精度與運算的權衡

粒子數是 Gmapping 最重要的參數。它的取舍很直觀：

- 粒子多(如 100 個):能涵蓋更多假設,對複雜環境與較大不確定性更穩健,但每個粒子都要維護一張地圖,記憶體與運算成本高。
- 粒子少(如 5 至 30 個):快速輕量,但假設涵蓋不足,遇到大幅漂移或環境重複時,可能所有粒子都「猜錯」而無法挽回。

教學實務上,一般教室與走廊環境用 30 個粒子多半足夠;若環境大或里程計品質差,可增加到 50 至 100 個。這裡有一個重要提醒:粒子數與地圖大小相乘決定記憶體用量——100 個粒子各帶一張 400×400 的地圖,就是 1600 萬個格子。這是 Gmapping 難以用於超大場景的根本原因,也是圖優化方法(第 15 章 Cartographer)後來居上的動機之一。

二、權重:用觀測給假設打分數

每個粒子的權重,代表「這個假設與最新觀測的吻合程度」。計算方式是:用該粒子的位姿與地圖,預測「應該會量到什麼」,再與實際雷射掃描比較。

若某粒子的地圖顯示「前方 3 公尺有牆」,而實際掃描也量到 3 公尺,兩者吻合,權重高;若某粒子的地圖說「前方是空的」,實際卻量到牆,顯然這個粒子的位姿假設偏了,權重低。這正是第 11 章觀測模型的具體實現。

三、重採樣:適者生存,以及它的陷阱

重採樣依權重複製高分粒子、淘汰低分粒子,讓粒子群逐漸聚集到正確的假設。但這裡藏著一個危險的陷阱:粒子退化(Particle Depletion)。

若每一步都重採樣,粒子的多樣性會迅速流失——最後所有粒子可能都是同一個祖先的複製品,彼此幾乎一樣。這時粒子濾波就失去了「維持多個假設」的意義。更糟的是:如果那個被大量複製的祖先其實猜錯了,系統就再也沒有其他假設可以挽救,SLAM

徹底失敗。

這個問題在 SLAM 中特別嚴重,因為錯誤的軌跡假設一旦獨佔,對應的錯誤地圖也就定案,無法回頭修正(這也是第 12 章指出濾波方法相對圖優化的弱點)。Gmapping 的第二項關鍵改良,正是為了對付它——見下一節。

14.4 Gmapping 的兩項關鍵改良

如果只是「粒子濾波加上每粒子一張地圖」,系統會因為需要大量粒子而不實用。Gmapping 之所以成功,關鍵在於兩項改良,它們讓所需粒子數大幅降低。理解這兩項改良,就理解了 Gmapping 的精髓。

改良一:用掃描匹配改良提議分布

提議分布(Proposal Distribution)是指「下一步要把粒子撒在哪裡」的機率分布。最樸素的做法是只依里程計:粒子依里程計移動,再加上隨機雜訊散開。

問題是里程計雜訊往往很大,粒子會撒得很開,而真正正確的位姿只落在其中一小塊區域。大部分粒子被浪費在明顯錯誤的位置上,因此需要大量粒子才能「碰巧」有幾個落在正確處。

Gmapping 的改良是:先用第 13 章的掃描匹配精修位姿,再以精修後的結果為中心,在一個小得多的範圍內撒粒子。

圖 14-1 提議分布改良的效果

【樸素做法：只用里程計】

粒子撒得很開(里程計雜訊大)→ 大多落在錯誤位置 → 需要大量粒子

【Gmapping：里程計 + 掃描匹配】

- ① 里程計給粗略位置(初始猜測)
- ② 掃描匹配精修 → 找到雷射掃描最吻合的位置
- ③ 在精修結果附近的小範圍撒粒子 → 幾乎每個粒子都在合理位置

↓

結果：所需粒子數從數千降到數十,運算量大幅下降

這個改良的效果驚人:它讓 Gmapping 用數十個粒子就能達到樸素方法需要數千個粒子的效果。同學應該能看出,這正是第 13.4 節「里程計預測、光達校正」思想的延伸應用。

改良二:選擇性重採樣

針對 14.3 節的粒子退化問題,Gmapping 採用選擇性重採樣 (Selective Resampling):不是每步都重採樣,而是先評估粒子的多樣性,只在多樣性太低時才重採樣。

衡量多樣性的指標稱為有效粒子數(Effective Sample Size,ESS)。若所有粒子權重相近(表示大家都差不多合理),ESS 高、多樣性好,不需重採樣;若少數粒子權重極高而其餘極低(表示已經分出勝負),ESS 低,此時才重採樣以集中資源。

這個策略在「保持多樣性」與「集中到正確假設」之間取得平衡,大幅減緩了粒子退化。這兩項改良——改良提議分布降低所需粒子數、選擇性重採樣避免退化——正是 Gmapping 論文的核心貢獻,也是它能成為實用系統的原因。

14.5 ROS Gmapping 實作流程

理論講完,本節說明在 ROS 中實際執行 Gmapping 建圖的完整流程。這也是第 28 章實作章節的預習。

前置條件:三樣東西缺一不可

許多同學第一次跑 Gmapping 失敗,原因幾乎都出在前置條件沒備齊:

96. 雷射掃描資料:光達驅動正常發布 /scan 話題(第 8 章的 LaserScan 格式)。

97. 里程計資料:機器人發布 odom 話題與 odom→base_link 的座標變換(第 10.7 節)。這需要輪速編碼器與正確的輪徑、輪距參數。

98. TF 座標樹完整:特別是 base_link→laser 的固定變換(光達安裝位置的外參,第 10 章)。設錯的話,掃描點會與機器人錯位,地圖必定歪掉——這正是第 10 章「地圖歪了先看 TF」口訣的由來。

建圖流程

99. 啟動機器人底層驅動(馬達控制、編碼器里程計、光達)。

100. 確認前置條件:用工具檢查 /scan 有資料、TF 樹完整無斷裂、里程計會隨移動而變化。

101. 啟動 Gmapping 節點,設定關鍵參數(粒子數、地圖解析度、更新距離門檻等)。

102. 開啟 RViz,顯示地圖、雷射掃描、TF 與粒子雲,以便即時觀察建圖狀況。

103. 遙控機器人在環境中緩慢移動,覆蓋所有想建圖的區域。速度要慢——這是成功的關鍵之一。

104. 觀察地圖成長:牆線是否清晰?有無重影?繞迴路後是否閉合?

105. 建圖完成後,用 map_saver 儲存地圖,產生 .pgm 影像檔與 .yaml 描述檔。

106. 日後導航時,以 map_server 載入這張地圖,配合 AMCL(第 3、11 章)進行定位。

關鍵參數

參數	意義	調校建議
粒子數	維持幾個軌跡假設	教室 30 個;環境大或里程計差可增至 50–100
地圖解析度	每格代表的實際邊長	室內常用 0.05 m(第 13.5 節)
更新距離 / 角度 門檻	移動多少才更新一次	設太大會漏掉細節,太小則運算浪費
雷射最大可用距離	超過此距離的量測捨棄	略小於光達標稱量程較穩健
里程計雜訊參數	描述里程計有多不準	打滑嚴重的平台應調大

表 14-1 Gmapping 主要參數與調校建議

14.6 常見失敗案例:輪子打滑、場景重複、光達遮蔽

這一節可能是全章對實作最有用的部分。SLAM 失敗是常態而非例外,能辨識失敗模式並對症下藥,是工程能力的核心。

失敗一:輪子打滑與里程計失準

症狀:地圖出現明顯錯位或重影,尤其在轉彎處;機器人「以為」自己走了很遠,地圖卻被拉長。

成因:輪子在光滑地面打滑、地毯與磁磚交界處摩擦力突變、輪徑或輪距參數設定錯誤(這一項最常被忽略,而且會造成系統性的尺度誤差,使整張地圖等比例放大或縮小)。

- 對策:確實校正輪徑與輪距(讓機器人直走 5 公尺、原地轉 10 圈,比對里程計讀值與實測);放慢速度減少打滑;調大 Gmapping 的里程計雜訊參數,讓系統少信任里程計、多倚賴掃描匹配。

失敗二:場景重複與特徵不足

症狀:長走廊被畫得忽長忽短、牆線重疊;空曠大廳中機器人徹底迷失。這正是第 13.6 節的走廊困境。

成因:沿走廊方向缺乏獨特幾何特徵,掃描匹配無法確定縱向移動量;空曠處則四周都超出光達量程,無從匹配。

- 對策:確保里程計品質良好(此時它是縱向資訊的唯一來源);增加粒子數以維持更多假設;在走廊沿途放置紙箱等臨時障礙物提供特徵(教學情境常用的技巧);或改用量程更大的光達。

失敗三:光達遮蔽與看不見的障礙

症狀:地圖上出現不該有的缺口;機器人撞到地圖上沒畫出來的東西。

成因:玻璃與鏡面讓雷射穿透或反射(第 8 章);黑色吸光物體反射微弱;障礙物不在光達的掃描平面高度上(第 8 章的高度盲區,案例 8-1 的椅腳問題);或有人站在光達前方形成移動遮蔽。

- 對策:建圖時清空場地、避免人員在機器人周圍走動;玻璃隔間可在建圖期間暫時貼上紙張或放置紙箱;認知到 2D 地圖只反映單一高度切面,導航時需搭配其他感測器補足。

失敗四:移動太快

症狀:掃描匹配跟不上,地圖突然錯位甚至「爆掉」。

成因:機器人移動或轉彎過快,前後兩次掃描差異太大,超出掃描匹配的收斂範圍(第

13.3 節 ICP 需要好的初始猜測)。

- 對策:放慢速度,尤其轉彎時。這是最簡單也最有效的一招——教學經驗中,多數建圖失敗只要「開慢一點」就解決了。

失敗模式	典型症狀	主要對策
輪子打滑	地圖錯位、重影、比例失真	校正輪徑輪距、放慢、調大里程計雜訊
場景重複	走廊變形、機器人迷失	確保里程計品質、增加粒子、加臨時特徵
光達遮蔽	地圖有缺口、撞到未畫出的障礙	清場、處理玻璃、認知高度盲區
移動太快	地圖突然錯位或爆掉	放慢速度,尤其轉彎時

表 14-2 Gmapping 常見失敗模式與對策

14.7 教學實作:用小車建立教室地圖

本節說明如何把 Gmapping 落實為一堂可操作的課,提供高中與大學部兩種深度的建議。詳細步驟見本章實作活動。

高中版:先看懂,再動手

高中課程建議以「觀察與體驗」為主,不必深究數學:

- 先用模擬器(TurtleBot3 Gazebo,第 27 章)讓每位同學都能操作,不受硬體數量限制。
- 重點觀察三件事:地圖如何隨機器人移動而「長」出來、粒子雲如何隨移動散開又收斂、以及開太快時地圖如何出錯。
- 刻意示範失敗:讓同學故意把機器人開很快,親眼看到地圖爆掉——失敗的震撼比成功的順利更能建立直覺。
- 連結生活:對照家中掃地機器人 App 的地圖(案例 13-1),理解兩者是同一回事。

大學部版:動手調參,理解機制

大學部應進一步操作實體平台並理解參數影響:

- 親手完成 TF 設定與里程計校正,體會前置條件的重要性(這往往佔掉一半的除錯時間)。
- 系統性地實驗參數:改變粒子數、地圖解析度、里程計雜訊,量化比較地圖品質(用第 13.7 節的評估方法)。
- 在教室、走廊兩種場域各建一次圖,對照第 13.6 節的預測,驗證走廊確實較難。
- 建完地圖後,接續執行 AMCL 定位與導航(第 28 章),完成「建圖→定位→導航」的完整鏈路。

教學心法

依多年教學經驗,建議教師掌握三個要點。其一,先講清楚前置條件與失敗模式,再讓同學動手——否則課堂會淹沒在除錯中。其二,強調「慢」:速度是新手成功率最大的變數。其三,務必安排「評估地圖品質」的環節(第 13.7 節),讓同學養成不只建圖、還要判斷好壞的習慣。這三點能讓一堂 SLAM 實作課的成功率大幅提升。

至此,同學已經完整走過第一個 SLAM 系統。Gmapping 用粒子濾波優雅地解開了雞生蛋問題,但它也有明確的限制:粒子數與地圖相乘的記憶體壓力使它難以應付大場景,且濾波方法無法回頭修正歷史(第 12 章)。下一章的 Cartographer 走另一條路——用第 12 章的圖優化與閉環偵測,讓大範圍建圖與「一鍵拉正」成為可能。兩相對照,同學將更深刻理解濾波與最佳化兩條路線的取捨。

公式或流程

一、Rao-Blackwell 化的機率分解

SLAM 要估計「軌跡與地圖的聯合分布」(第 1 章)。RBPF 的關鍵是把它拆成兩項:

$$p(x(1:t), m \mid z(1:t), u(1:t)) = p(m \mid x(1:t), z(1:t)) \cdot p(x(1:t) \mid z(1:t), u(1:t))$$

右側第二項是「軌跡的分布」——低維,用粒子取樣;右側第一項是「已知軌跡下的地圖分布」——在第 13 章我們已知它可以用對數勝算比直接算出,不需取樣。這條式子就是 14.2 節「拆解」的數學表述,也是 Gmapping 全部巧思的濃縮。

二、粒子權重

第 i 個粒子的權重更新為:

$$w^{(i)} \propto p(z_t \mid m^{(i)}, x_t^{(i)})$$

讀作:權重正比於「以該粒子的地圖 $m^{(i)}$ 與位姿 $x_t^{(i)}$,觀測到 z_t 的可能性」。掃描與地圖越吻合,權重越高。這正是第 11 章觀測模型在 SLAM 中的具體形式。

三、有效粒子數與選擇性重採樣

衡量粒子多樣性的指標:

$$N_{eff} = 1 / \sum (w^{(i)})^2 \quad (\text{權重已正規化})$$

若所有粒子權重相等(多樣性最佳), N_{eff} 等於粒子總數 N ;若權重集中在單一粒子, N_{eff} 趨近 1。Gmapping 的選擇性重採樣規則為:

$$\text{若 } N_{eff} < N / 2, \text{才執行重採樣}$$

這條簡單的判斷式,正是 14.4 節避免粒子退化的關鍵機制。

四、Gmapping 單步完整流程

圖 14-2 Gmapping 每一步的處理流程

- ① 取得新的里程計讀值與雷射掃描

↓

② 對每個粒子:以里程計為初始猜測,執行掃描匹配精修位姿

↓

③ 在精修結果附近的小範圍取樣,得到該粒子的新位姿(改良提議分布)

↓

④ 計算權重:比較「該粒子地圖預期的掃描」與「實際掃描」的吻合度

↓

⑤ 每個粒子用自己的新位姿,更新自己的地圖(第 13 章對數勝算比)

↓

⑥ 計算 N_{eff} ;若低於門檻則重採樣(選擇性重採樣)

↓

⑦ 輸出權重最高粒子的地圖與位姿 → 回到 ①

五、記憶體估算

Gmapping 的記憶體需求為粒子數乘上單張地圖大小:

$$\text{總格子數} = \text{粒子數} \times (\text{寬} / \text{解析度}) \times (\text{高} / \text{解析度})$$

例如 50 個粒子、20 m × 20 m 房間、0.05 m 解析度: $50 \times 400 \times 400 = 800$ 萬格。這條式子量化了 14.3 節指出的限制——這正是 Gmapping 難以擴展到大場景的根本原因。

案例

案例 14-1 原作者的課堂:從論文到 YouTube

Gmapping 的三位作者 Grisetti、Stachniss 與 Burgard,正是第 2 章介紹的 Freiburg / Bonn 機率機器人學研究群核心成員。有趣的是,Stachniss 教授後來把粒子濾波 SLAM 的完整推導錄成影片免費公開在 YouTube 上,成為全球 SLAM 學習者的共同教材。

這意味著同學讀完本章後,可以直接去看演算法的原創者親自講解——他會用動畫展示粒子如何散開、如何被觀測秤重、如何重採樣收斂。這種「讀教科書理解架構、看原作者影片補足直覺」的組合,是本書第 2 章推薦學習路線的最佳實踐。

這個案例也呼應第 2 章「知識公開的力量」:一個 2007 年發表在頂級期刊的演算法,今天任何一位台灣的高中生或大學生,都能免費取得原始論文、開源程式碼與作者親授影片。學習 SLAM 的門檻,已從「能否進入名校」變成「有沒有一份好的學習地圖與持續的毅力」——這正是本書想扮演的角色。

案例 14-2 三十個粒子的教室:為何 Gmapping 夠用又不夠用

在大學的教學實驗中,一個典型的觀察是:用 30 個粒子在一般教室建圖,結果相當漂亮——牆線筆直、迴路閉合、細節清晰。但同樣的設定拿到整層樓(數十公尺見方、多個房間與長走廊),地圖品質就明顯下降,甚至失敗。

原因是兩重的。其一是記憶體與運算:場景變大,每張地圖的格子數暴增,乘上粒子數後負擔沉重(公式五)。其二更根本:大場景意味著更長的軌跡、更多的累積漂移,需要更多粒子才能涵蓋可能的假設;但粒子一多,運算又跟不上,形成兩難。

更關鍵的是:當機器人繞完一大圈回到起點,即使閉環被「偵測」到,粒子濾波也無法像圖優化那樣回頭修正整條歷史軌跡——每個粒子的地圖早已依錯誤軌跡畫定,無法重來(第 12 章的核心論點)。這正是第 15 章 Cartographer 要解決的問題。同學若能親

手做這個「教室成功、整層樓失敗」的對照實驗,對兩章的差異會有極深刻的體會。

案例 14-3 掃地機器人裡的粒子:理論就在你家

第 11 章案例提過,許多掃地機器人的除錯模式能看到粒子雲收斂的過程。本章讓我們更進一步:掃地機器人的「首次建圖」與「後續清掃」,其實對應兩種不同的技術。

首次進入陌生環境時,機器人必須同時定位與建圖——這是本章的 SLAM。而地圖建好之後,日後每次清掃只需在已知地圖中定位,這就是第 3 章與第 11 章的蒙地卡羅定位(AMCL)。兩者共用粒子濾波的骨架,差別在於「地圖是否已知」:SLAM 的粒子各帶一張成長中的地圖,AMCL 的粒子則共用同一張固定地圖。

理解這個區分很重要,因為它對應 ROS 中兩套不同的工具與流程(14.5 節):建圖階段跑 Gmapping、儲存地圖;導航階段用 map_server 載入地圖、跑 AMCL 定位。第 28 章的實作會完整走過這條鏈路。下次看家中掃地機器人第一次清掃時「小心翼翼地探索」、第二次「熟門熟路地直接開工」,你就知道它切換了兩套演算法。

實作活動

實作 14-1 用 Gmapping 建立教室地圖

目的:完整執行一次粒子濾波 SLAM 建圖,觀察粒子雲的演化,並透過參數實驗與失敗重現,理解 Gmapping 的機制與限制。本實作提供模擬與實體兩條路徑。

路徑 A:模擬建圖(全班可做,不需硬體)

95. 啟動 TurtleBot3 Gazebo 模擬環境與 SLAM 節點(第 27 章有完整安裝步驟)。
96. 開啟 RViz,同時顯示:佔據格地圖、雷射掃描、TF 座標軸,以及粒子雲(若工具支援)。

97. 用鍵盤遙控機器人在模擬房間中緩慢移動,一邊移動一邊觀察地圖如何逐格「長」出來、未知的灰色區域如何被填滿。
98. 觀察粒子雲:機器人直線前進時粒子雲如何散開?轉彎或看到明顯特徵時如何收斂?對照第 11 章與 14.3 節的理論。
99. 速度實驗:先用很慢的速度建一次圖,再用最快速度建一次,比較兩張地圖的品質差異,重現 14.6 節「移動太快」的失敗模式。
100. 儲存地圖,用第 13.7 節的方法評估品質:牆是否筆直?迴路是否閉合?量測地圖上的房間尺寸與模擬設定值比較。

路徑 B:實體小車建圖(有硬體的班級)

101. 前置檢查:確認 /scan 有資料、TF 樹完整(特別是 base_link→laser)、里程計會隨移動變化。這一步不可跳過。
102. 里程計校正:讓小車直走 3 公尺,比對里程計讀值與捲尺實測;原地轉 10 圈,比對角度讀值。若誤差大,修正輪徑與輪距參數。
103. 清場:請同學退到教室邊緣,移開可能遮蔽光達的物品,若有玻璃隔間可暫時貼紙。
104. 緩慢遙控小車繞行教室一圈(含四個角落),回到起點,觀察迴路是否閉合。
105. 儲存地圖檔(.pgm 與 .yaml),並在地圖上量測教室尺寸,與捲尺實測值比較,計算誤差百分比。

第二部分:參數實驗(大學部)

106. 粒子數實驗:分別用 5、30、100 個粒子在同一場域建圖,比較地圖品質與運算負擔(可觀察 CPU 使用率與更新頻率)。
107. 解析度實驗:用 0.02、0.05、0.10 m 三種解析度建圖,比較細節呈現與記憶體用量,驗證公式五的估算。

108. 走廊挑戰:把機器人開進長走廊建圖,對照第 13.6 節與 14.6 節的預測,觀察是否出現變形。嘗試在走廊沿途放置紙箱提供特徵,再建一次,比較改善程度。
109. 打滑實驗:在地面鋪一塊光滑板材或讓一側輪子短暫懸空,製造打滑,觀察地圖如何出錯。

討論問題

67. 快速移動與緩慢移動建出的地圖差多少?從 14.6 節與第 13.3 節的角度,解釋為什麼「開慢一點」如此有效。
68. 粒子數從 5 增加到 100,地圖品質改善了多少?運算負擔增加多少?你認為在你的場域,最合適的粒子數是多少?
69. 走廊建圖出現了什麼問題?放置紙箱後改善了嗎?這印證了 14.6 節的哪一項論述?
70. 若機器人繞了一大圈回到起點但地圖沒有閉合,Gmapping 有辦法「事後修正」整張地圖嗎?為什麼?(提示:第 12 章與案例 14-2)

延伸挑戰(大學部)

其一:用 Python 實作一個極簡的 RBPF 模擬——在程式中定義一個虛擬房間,建立 20 個粒子(各自有位姿與一張小地圖),實作「移動加雜訊→秤重→選擇性重採樣→各自更新地圖」的循環,把每個粒子的地圖並排畫出,觀察正確的假設如何勝出。其二:建完地圖後,接續啟動 map_server 與 AMCL,讓機器人在自己建的地圖中定位並執行導航(第 28 章),完成「建圖→定位→導航」的完整鏈路。

延伸閱讀

90. G. Grisetti, C. Stachniss, and W. Burgard, "Improved Techniques for Grid Mapping with Rao-Blackwellized Particle Filters," IEEE Transactions on Robotics, 2007.(Gmapping 的原始論文,本章主要依據)
91. M. Montemerlo et al., "FastSLAM: A Factored Solution to the Simultaneous Localization and Mapping Problem," AAAI, 2002.(RBPF-SLAM 的開創性工作,與 Gmapping 互為對照)
92. S. Thrun, W. Burgard, and D. Fox, Probabilistic Robotics, MIT Press, 2005.(第 13 章 FastSLAM 與粒子濾波 SLAM 的完整理論)
93. Cyrill Stachniss, Particle Filter 與 FastSLAM 系列課程影片(YouTube).(原作者親自講解,搭配本章閱讀效果最佳)
94. OpenSLAM Gmapping 開源專案與 ROS slam_gmapping 套件文件.(原始程式碼與參數說明)
95. slam_toolbox 開源專案(GitHub).(ROS 2 環境中基於圖優化的現代 2D SLAM,可與 Gmapping 對照實驗)
96. ROBOTIS TurtleBot3 e-Manual 之 SLAM 章節.(模擬與實體建圖的完整操作指引)

小結

63. Gmapping 由 Grisetti、Stachniss 與 Burgard 於 2007 年提出,是最著名的 2D 雷射 SLAM 系統之一;輸入為光達掃描、里程計與 TF,輸出為佔據格地圖與 map→odom 變換。
64. Rao-Blackwell 化把 SLAM 拆成兩部分:軌跡(低維,用粒子取樣)與地圖(已知軌跡即可解析求出);每個粒子是一個「平行世界」,各帶一條軌跡與一張地圖,以觀測評分競爭,漂亮地解開雞生蛋問題。

65. 粒子數是精度與運算的權衡(教室約 30 個);權重代表假設與觀測的吻合度;重採樣讓好假設存活,但過度重採樣會造成粒子退化,使錯誤假設獨佔而無法挽回。
66. Gmapping 的兩項關鍵改良:以掃描匹配改良提議分布(讓粒子撒在合理位置,所需粒子數從數千降至數十)、以及選擇性重採樣(只在有效粒子數低於門檻時才重採樣,避免退化)。
67. ROS 實作的三個前置條件缺一不可:/scan 雷射資料、里程計與 odom→base_link 變換、完整正確的 TF 樹(尤其 base_link→laser 外參);建圖後以 map_saver 存成 .pgm 與 .yaml。
68. 四大失敗模式:輪子打滑(校正輪徑輪距、調大里程計雜訊)、場景重複(走廊,靠里程計與增加特徵)、光達遮蔽(玻璃、黑色物體、高度盲區)、移動太快(放慢是最有效的一招)。
69. Gmapping 的根本限制在於記憶體隨「粒子數×地圖大小」成長,難以應付大場景;且身為濾波方法,無法回頭修正歷史軌跡與地圖——這正是下一章圖優化路線要解決的問題。

本章完成了第一個完整的 SLAM 系統。Gmapping 以粒子濾波優雅地解開雞生蛋問題,但也讓我們清楚看到濾波路線的天花板:大場景吃不消、歷史無法修正。下一章 Cartographer 登場——它由 Google 開發,以子地圖(Submap)組織空間、以第 12 章的圖優化執行閉環修正,能在大範圍場景中即時建圖,並在偵測到閉環的瞬間把整張歪掉的地圖「一鍵拉正」。第 12 章那個令人震撼的畫面,即將在真實系統中重現。

台灣自編大學教科書

[@prof](#)

《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第 15 章

Cartographer : Google 2D / 3D SLAM

編者：李世淵 (機器人叫獸)

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

學習目標

上一章的 Gmapping 以粒子濾波解開了雞生蛋問題,但也暴露了濾波路線的天花板:大場景吃不消、歷史無法修正。本章介紹 Google 開發的 Cartographer——它以子地圖組織空間、以第 12 章的圖優化執行閉環修正,能在大範圍場景中即時建圖,並在偵測到閉環的瞬間把整張歪掉的地圖「一鍵拉正」。學完本章之後,同學應能夠達成下列目標:

89. 說明 Cartographer 的雙子系統架構:局部 SLAM 與全域 SLAM 的分工。
90. 解釋子地圖(Submap)的概念,以及為何要把掃描分組成子地圖。
91. 說明局部與全域兩種掃描匹配的目的與差異。
92. 說明分支定界法如何讓即時閉環偵測成為可能。
93. 說明在 ROS 中執行 2D Cartographer 建圖的流程與關鍵設定。
94. 說明 3D Cartographer 與背包式空間建模的應用。
95. 比較 Cartographer 與 Gmapping 的架構、優缺點與適用場合。

核心概念

中文名詞	英文名詞	概念說明
子地圖	Submap	由數十筆掃描組成的小塊機率格地圖,是 Cartographer 的基本單元。
局部 SLAM	Local SLAM	前端:把新掃描插入當前子地圖,局部一致但會漂移。
全域 SLAM	Global SLAM	後端:在背景尋找閉環約束並最佳化整個位姿圖。
機率格	Probability Grid	子地圖的內部表示,每格記錄佔據機率。
掃描匹配	Scan Matching	把新掃描對齊到子地圖或既有子地圖的過

中文名詞	英文名詞	概念說明
		程。
閉環約束	Loop Closure Constraint	連接重訪位置的邊,用來消除累積誤差。
分支定界	Branch and Bound	有效率地搜尋最佳匹配、剔除不可能解的演算法。
多解析度預算格	Precomputed Grids	事先算好的不同解析度地圖,加速分支定界搜尋。
稀疏位姿調整	Sparse Pose Adjustment	對子地圖與掃描位姿構成的圖執行的最佳化。
非線性最小平方	Nonlinear Least Squares	Cartographer 以 Ceres 求解的最佳化形式。
背包式建模	Backpack Mapping	人員背負感測器組行走以掃描建築空間。
穩健核函數	Robust Kernel	降低錯誤閉環影響的最佳化技巧。

15.1 Cartographer 系統架構

Cartographer 由 Google 開發並於 2016 年開源,其核心論文標題直接點出了它的主打特色:「2D 光達 SLAM 中的即時閉環」。這個名稱本身就宣示了它與 Gmapping 的根本差異——它把閉環偵測與圖優化做進了系統核心,而且能即時執行。

兩個子系統:局部與全域

理解 Cartographer,最重要的是掌握它的雙子系統架構。它可以被視為兩個獨立但相關的子系統:局部 SLAM(也稱為前端或局部軌跡建構器)負責建立一連串的子地圖,每個子地圖在局部上是一致的,但我們接受局部 SLAM 會隨時間漂移;全域 SLAM 則在獨立的執行緒中運行,尋找子地圖之間的閉環約束。

- 局部 SLAM(前端):把每一筆新掃描匹配並插入「當前子地圖」。它追求即時與局部精確,但明知會隨時間累積漂移——這是刻意的設計取捨。
- 全域 SLAM(後端):在背景執行緒中,把掃描與附近的子地圖做匹配以建立閉環約束,並持續最佳化由子地圖與掃描位姿構成的約束圖。

關鍵在於:兩者同時運行。前景快速建圖讓操作者立即看到結果,背景默默尋找閉環並修正全域一致性。這正是它能「即時」的原因,也是與 Gmapping 最大的架構差異。

圖 15-1 Cartographer 雙子系統架構

【局部 SLAM(前端)——前景執行】

新掃描 → 掃描匹配到當前子地圖 → 插入子地圖 → 子地圖完成後封存

特性:即時、局部一致、會漂移

【全域 SLAM(後端)——背景執行緒】

掃描 ↔ 附近既有子地圖做匹配 → 找到好匹配則建立閉環約束

↓

把子地圖位姿與掃描位姿組成位姿圖(第 12 章)

↓

每隔數秒以 Ceres 求解非線性最小平方 → 整張地圖被拉正

兩者同時運行:前景給即時預覽,背景保證全域一致

與第 12 章的連結

同學應該立刻認出來:全域 SLAM 做的正是第 12 章的圖優化。它結合了局部掃描匹配形成的子地圖(以 Ceres 為基礎的非線性最小平方最佳化器精修的機率格),與帶有分支定界閉環偵測的稀疏位姿圖,在一般等級的硬體上達成精確的即時 2D SLAM。第 12 章提到 Cartographer 使用 Ceres 函式庫,在此得到印證。理論與系統,在這一章正式接軌。

15.2 Submap 子地圖

子地圖是 Cartographer 最具識別度的設計。理解它為何存在,就理解了 Cartographer 為何能處理大場景。

什麼是子地圖

子地圖是一小塊機率格地圖(第 13 章的佔據格),由連續數十筆雷射掃描累積而成。機器人移動時,新掃描不斷插入當前子地圖;當累積到設定的掃描數,這個子地圖就「完

成」並被封存,系統改建下一個子地圖。整個環境因此被切成一系列部分重疊的小塊地圖。

為什麼要分子地圖:三個理由

107. 限制誤差累積:由於掃描只與包含少數近期掃描的子地圖做匹配,這個做法會緩慢累積誤差;對於僅僅數十筆連續掃描而言,累積的誤差很小。換句話說,在一個子地圖的短暫生命週期內,漂移小到可以忽略——每個子地圖內部是「乾淨」的。

108. 提供最佳化的單元:全域最佳化時,不需要調整每一筆掃描,只需調整「子地圖的位姿」。一個子地圖代表數十筆掃描,這讓位姿圖的節點數大幅減少,最佳化因此能即時完成。這是效率的關鍵。

109. 成為閉環比對的對象:閉環偵測時,是拿「當前掃描」去比對「過去的子地圖」。子地圖比單筆掃描包含更多資訊、更穩定,比對結果也更可靠。

與 Gmapping 的對比

這裡可以清楚看出兩套系統的哲學差異。Gmapping 讓每個粒子維護一整張完整地圖(第 14 章),記憶體隨粒子數乘上地圖大小成長;Cartographer 則只維護一張由眾多子地圖拼成的地圖,靠調整子地圖位姿來修正全域一致性。

這個差異的後果很直接:Cartographer 的記憶體需求與場景大小成正比(而非乘上粒子數),因此能處理整棟建築;而且因為保留了子地圖與位姿圖,它能「回頭修正」——這正是第 12 章指出的圖優化相對濾波的核心優勢。

15.3 Scan Matching

Cartographer 中有兩種掃描匹配,目的截然不同。分清楚它們,是理解系統運作的關鍵。

一、局部掃描匹配:插入當前子地圖

這是前端的日常工作。每來一筆新掃描,系統要決定「它該貼在當前子地圖的哪個位置」。做法是以里程計或 IMU 提供初始猜測(第 13.4 節的思想),再用最佳化方法精修,使掃描點與子地圖的機率格盡可能吻合。

Cartographer 把這個匹配表述為非線性最小平方問題,以 Ceres 求解(第 12 章)。這個做法與傳統 ICP(第 13.3 節)不同——它不是找點對點的對應,而是直接最大化「掃描點落在高佔據機率格」的程度,這對雜訊較不敏感。

二、全域掃描匹配:尋找閉環

這是後端的工作,難度高得多。問題是:當前掃描,是否與某個很久以前建立的子地圖吻合?若吻合,就找到了閉環。

困難之處在於搜尋空間。閉環候選可能在任何位置、任何角度,而且由於已經累積了漂移,初始猜測可能偏差很大——不能只在小範圍內找。若用暴力搜尋,把每個可能的位置與角度都試一遍,運算量將大到無法即時執行。

這正是 Cartographer 最精妙的技術貢獻要解決的問題,也是下一節的主題。

限制搜尋範圍的實務考量

即使有高效演算法,系統仍需控制計算量。為了限制約束(與計算量)的數量,Cartographer 只考慮所有鄰近節點的一個子取樣集合來建立約束,這由一個取樣比例常數控制;取樣太少可能導致錯過約束、閉環失效,取樣太多則會拖慢全域 SLAM,使即時閉環無法達成。這是實作調參時的重要權衡。

15.4 閉環偵測與全域最佳化

本節是全章的技術核心:Cartographer 如何做到「即時」閉環——這正是它論文

標題所宣稱的成就。

分支定界:剪掉不可能的分支

面對龐大的搜尋空間, Cartographer 採用分支定界法(Branch and Bound)。它以分支定界機制在不同的格地圖解析度上運作,有效率地剔除不正確的匹配。

分支定界的核心思想,可以用一個生活比喻理解:假設你要在一整棟大樓裡找一個人。笨方法是逐間房間搜遍;聰明的做法是先在每一層樓快速掃視(低解析度),若某一層明顯不可能有人(例如整層在裝修),就跳過整層不必逐間找——這就是「剪枝」。

Cartographer 的做法正是如此:它把既有子地圖降尺度成幾種不同解析度(例如四分之一、二分之一大小)作為掃描匹配的輸入,匹配過程從低解析度開始做初步的低精度計算,得到最佳分數的初步結果後,用它跳過高解析度中大量不可能的平移旋轉組合,如此反覆直到抵達完整解析度。系統會預先算好尺寸為 1、4、16、64 的多解析度格網來支援這個搜尋。

為什麼這樣就能即時

關鍵在於「上界」的概念。在低解析度地圖上算出的分數,是該區域在高解析度下所有可能匹配分數的上界(不會低估)。因此若某區域在低解析度下的分數,已經低於目前找到的最佳答案,那麼它在高解析度下也絕不可能更好——整個區域可以安全地跳過,不必展開。

這讓搜尋量從「所有可能組合」降到「少數有希望的分支」,即時閉環因此成為可能。這個機制在 Cartographer 論文中有詳盡的說明,而其專用的快速相關掃描匹配器正是為了讓即時閉環的掃描匹配成為可能而設計的。

全域最佳化:把矛盾攤平

找到閉環約束後,就進入第 12 章的圖優化。閉環最佳化與掃描匹配一樣,也被表述

為非線性最小平方問題,這讓系統能輕易加入殘差以納入額外資料;每隔幾秒,系統就用 Ceres 計算一次解。

這個最佳化調整的是「子地圖位姿」與「掃描位姿」,稱為稀疏位姿調整(Sparse Pose Adjustment)。當閉環約束與里程計約束矛盾時,最佳化把矛盾均攤到整條軌跡上——第 12 章那個「整張歪掉的地圖唰地被拉正」的畫面,在此真實上演。

穩健化:防止錯誤閉環的災難

閉環偵測有可能誤判(例如兩個長得很像的房間)。一條錯誤的閉環約束,會把地圖硬拉到錯誤的位置,造成災難性後果(第 12 章實作活動提過)。因此 Cartographer 在最佳化中採用穩健核函數,降低離群約束的權重,讓少數錯誤匹配不至於毀掉整張地圖。這呼應第 12 章延伸挑戰中「加入錯誤閉環邊」的實驗——現在同學知道實務系統是怎麼防範的了。

15.5 2D Cartographer 實作

本節說明在 ROS 中執行 2D Cartographer 建圖的實務要點。與第 14.5 節的 Gmapping 流程對照閱讀,能看出兩者的異同。

前置條件

與 Gmapping 相同的三要件仍然適用:雷射掃描(/scan)、里程計、完整的 TF 座標樹(第 10 章)。但 Cartographer 有兩點不同:

- IMU 的角色更重要:Cartographer 能有效利用 IMU 提供的姿態資訊(第 7 章),尤其在 3D 模式中幾乎是必要的。2D 模式若有 IMU,建圖穩定度會明顯提升。
- 里程計並非絕對必要:由於局部掃描匹配的能力較強,Cartographer 在特徵豐富的環境中即使沒有里程計也能運作。但在走廊等特徵稀少處(第 13.6 節),里程計仍然關鍵。

設定檔:Lua 組態

Cartographer 的一大特色(也是初學者的門檻)是它使用 Lua 語言的設定檔,參數數量遠多於 Gmapping。大部分局部 SLAM 的選項可以在 2D 的 trajectory_builder_2d.lua 與 3D 的 trajectory_builder_3d.lua 設定檔中找到。

關鍵參數群組包含:

參數群組	控制什麼	調校方向
子地圖掃描數	每個子地圖累積幾筆掃描	太少則子地圖不穩、太多則漂移進子地圖內
解析度	機率格的格子邊長	室內常用 0.05 m(第 13.5 節)
局部匹配權重	掃描匹配時各項殘差的相對重要性	里程計可靠則提高其權重
閉環取樣比例	考慮多少節點來建立約束	太低錯過閉環、太高拖慢即時性(15.3 節)
閉環分數門檻	匹配分數多高才接受為閉環	太低易誤判、太高則閉環偵測不到
最佳化週期	多久執行一次全域最佳化	通常數秒一次

表 15-1 2D Cartographer 主要參數群組

建圖流程

110. 確認感測器資料與 TF 樹正常(同第 14.5 節)。
111. 準備 Lua 設定檔,依平台調整感測器組態(有無 IMU、有無里程計、光達數量)。
112. 啟動 Cartographer 節點與 RViz,顯示子地圖、軌跡與雷射掃描。
113. 緩慢遙控機器人建圖。特別留意:走完迴路回到起點時,觀察是否觸發閉環——這是本章最值得觀察的一刻。

114. 建圖完成後,執行「結束軌跡」與最終最佳化,再匯出地圖檔(.pbstream 格式,可再轉為 ROS 標準的 .pgm 與 .yaml)。

觀察重點:子地圖與閉環

在 RViz 中,Cartographer 會把各個子地圖以不同方式顯示,同學可以清楚看到地圖是由一塊塊子地圖拼成的。而最精彩的時刻是閉環觸發的瞬間:原本錯開的軌跡與地圖會突然「跳」到正確位置。務必安排課堂時間讓同學親眼見證這一刻——這是第 12 章抽象理論最有力的具象化。

15.6 3D Cartographer 與背包式空間建模

Cartographer 不只做 2D。它同樣支援 3D SLAM,而 Google 開發它的原始動機之一,正是室內空間的三維測繪。

3D 模式的差異

- 感測器:需要 3D 光達(多線,第 8 章)。Google 的系統最初搭載兩具 Hokuyo 雷射掃描儀與一個慣性量測單元,後來的版本則改用兩具 Velodyne PUCK 掃描儀。
- IMU 的必要性:3D 模式下,姿態(滾轉與俯仰,第 7 章)的估計至關重要,IMU 幾乎是必備的。
- 子地圖為 3D:機率格從平面變成立體的體素(Voxel)格,記憶體需求大幅上升。
- 運算負擔:3D 掃描匹配與最佳化的計算量遠高於 2D,對硬體要求較高。

背包式空間建模

第 8 章案例 8-3 提過背包式移動掃描,Cartographer 正是這類系統的代表。操作員背負搭載光達與 IMU 的背包系統,一邊行走一邊即時建立整棟建築的三維模型。

這個應用之所以重要,是因為它解決了傳統測繪的痛點:傳統靜態雷射掃描儀必須在每個定點架設、掃描、再搬到下一點,耗時費力;背包式系統則「走到哪掃到哪」,效率提升數十倍。

- 應用場景:建築資訊模型(BIM)建置與驗證、古蹟與文化資產數位保存、廠房設施管理、室內導航地圖製作、災後結構評估。
- 對大學場域的意義:這是極具潛力的專題方向——用 3D SLAM 掃描校園建築、實驗室或工廠,產生可用於教學與研究的數位分身,把第五篇(第 17、18 章)的 3D 點雲技術實際應用出來。

連續時間 SLAM 的延伸研究

值得一提的是,學術界持續在 Cartographer 的基礎上改良。例如有研究團隊指出,把掃描視為「連續時間軌跡」而非離散時刻的快照,能進一步改善 3D 建圖品質——這類研究正是建立在對 Cartographer 架構的深入理解之上。對有志研究的同學而言,理解本章內容,就是進入這個領域的門票。

15.7 Cartographer 與 Gmapping 比較

本篇的兩大主角至此都已介紹完畢。本節做系統性的對照,這也是第四篇最重要的一張表。

比較項目	Gmapping(第 14 章)	Cartographer(本章)
理論基礎	粒子濾波(第 11 章)	圖優化(第 12 章)
地圖組織	每個粒子一張完整地圖	一系列子地圖拼接
閉環處理	無專門機制,難以事後修正	核心功能,可即時拉正全圖
記憶體	粒子數 × 地圖大小	與場景大小成正比

比較項目	Gmapping(第 14 章)	Cartographer(本章)
大場景	受限	擅長,可建整棟建築
3D 支援	無(僅 2D)	支援 2D 與 3D
IMU 利用	不使用	可有效整合,3D 幾乎必備
參數複雜度	較少,易上手	較多(Lua 設定檔),門檻較高
教學價值	理解粒子濾波的最佳範例	理解圖優化與閉環的最佳範例

表 15-2 Cartographer 與 Gmapping 的系統比較

如何選擇

- 小場域教學、理解粒子濾波:Gmapping。它參數少、概念直觀,是入門的好選擇。
- 大場域、需要閉環、要建整棟樓:Cartographer。它的架構就是為此而生。
- 需要 3D 建模、背包式掃描:Cartographer 3D。
- ROS 2 環境的現代 2D SLAM:除了 Cartographer,也可考慮 slam_toolbox——它同樣基於圖優化,參數較簡潔,近年在 ROS 2 社群中相當流行。

兩章的教學意義

把第 14 與 15 章並列閱讀,同學會發現:它們正是第 11 章(濾波)與第 12 章(最佳化)兩條理論路線的實體化身。Gmapping 展示了粒子濾波的巧思與天花板,Cartographer 展示了圖優化如何突破那個天花板。這不只是兩個工具的比較,而是 SLAM 十餘年技術演進的縮影。

至此,同學已經掌握 2D SLAM 的兩大系統。下一章我們把鏡頭轉向最貼近生活的應用——掃地機器人。從隨機碰撞到 SLAM 規劃、從房間分割到覆蓋路徑規劃,看這些理論如何在你家客廳裡運作,並理解家庭場景中桌椅、鏡面與低矮障礙物帶來的真實挑

戰。

公式或流程

一、局部掃描匹配的最佳化

把新掃描插入子地圖時,求一個位姿 ξ 使掃描點盡量落在子地圖的高佔據機率處:

$$\xi^* = \operatorname{argmin} \sum_k (1 - M(T_\xi \cdot h_k))^2$$

其中 h_k 是掃描中的第 k 個點、 T_ξ 是把它變換到子地圖座標的位姿變換(第 10 章)、 $M(\cdot)$ 是該位置在機率格中的佔據機率。式子的意義是:讓所有掃描點都落在「地圖認為有障礙」的地方。這是一個非線性最小平方問題(第 12 章),以 Ceres 求解。

二、全域最佳化(稀疏位姿調整)

全域 SLAM 最佳化子地圖位姿 ξ^m 與掃描位姿 ξ^s ,使所有約束的加權殘差最小:

$$\operatorname{argmin} \frac{1}{2} \sum_{ij} \rho(E^2(\xi^m, \xi^s; \Sigma_{ij}, \xi_{ij}))$$

其中 E 是殘差(第 j 筆掃描相對第 i 個子地圖的實際關係與量測關係之差)、 Σ_{ij} 是該約束的共變異數(可信度,第 12.3 節)、 ρ 是穩健核函數——它讓誤判的閉環約束不至於毀掉整張地圖(15.4 節)。這條式子與第 12 章的最小平方目標函數形式完全一致,只是把節點換成了子地圖與掃描。

三、分支定界搜尋

全域掃描匹配的搜尋空間定義為一個三維視窗(平移 x 、平移 y 、旋轉 θ):

$$W = \{ -w_x, \dots, w_x \} \times \{ -w_y, \dots, w_y \} \times \{ -w_\theta, \dots, w_\theta \}$$

分支定界以「分支—定界—剪枝」反覆進行:

圖 15-2 分支定界閉環搜尋流程

① 預先計算多解析度格網(尺寸 1、4、16、64)



② 從最低解析度開始,把搜尋空間切成數個大區塊(分支)



③ 對每個區塊,在低解析度格網上計算「分數上界」(定界)



④ 若某區塊的上界 < 目前最佳分數 → 整個區塊剪除,不再展開(剪枝)



⑤ 否則把該區塊再切細,到更高解析度重複 ②-④



⑥ 抵達完整解析度且分數超過門檻 → 接受為閉環約束

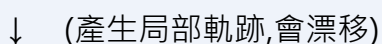
四、Cartographer 完整資料流

圖 15-3 Cartographer 端到端流程

感測器:雷射掃描 + IMU + (可選)里程計



【局部 SLAM】掃描匹配 → 插入當前子地圖 → 子地圖滿了則封存,開新的



【全域 SLAM,背景執行緒】

取樣節點 → 對既有子地圖做分支定界匹配 → 找到閉環約束

↓

組成位姿圖(子地圖位姿 + 掃描位姿)

↓

每隔數秒以 Ceres 執行稀疏位姿調整(含穩健核函數)

↓

輸出:全域一致的地圖與軌跡(閉環時整張地圖被拉正)

案例

案例 15-1 閉環的那一刻:課堂上最震撼的三秒鐘

第 12 章案例 12-1 描述過閉環拉正地圖的畫面,而 Cartographer 讓這個畫面成為課堂上可以重現的實驗。

典型的教學場景是這樣的:同學遙控機器人繞行系館一圈。前半程一切正常,地圖工整;繞到後半段時,牆線開始與最初建的部分錯開,走廊出現明顯的重影,同學臉上開始出現疑惑。就在機器人回到起點附近的瞬間——背景執行緒找到了閉環約束,全域最佳化執行——整張地圖在螢幕上「唰」地一下重組,重影合而為一,錯開的走廊接合,軌跡形成一個漂亮的封閉迴路。

這三秒鐘,把第 12 章的圖優化、第 15.4 節的分支定界與稀疏位姿調整,全部濃縮成一個看得見的瞬間。教學經驗顯示,親眼看過這一幕的同學,對「為什麼要用圖優化」的理解遠勝於任何講述。建議教師務必安排足夠的空間讓機器人繞完一個完整迴路——這是本章最值得投資的課堂時間。

案例 15-2 Google 為何要做 Cartographer:室內地圖的野心

Google 開發 Cartographer 並非純學術興趣。當時 Google 已經用街景車繪製了全世界的道路,但室內空間——商場、機場、車站、博物館——仍是地圖的空白區。要繪製室內地圖,傳統測繪太慢太貴,必須有能「邊走邊建圖」的技術。

這解釋了 Cartographer 的幾個設計取向:為什麼要能即時(操作員需要立即知道哪裡還沒掃到)、為什麼要能處理大場景(整座商場)、為什麼要做背包式系統(人能走的地方都要能掃)、以及為什麼閉環如此重要(大範圍行走必然累積漂移)。

這個案例給同學一個重要啟示:技術的設計取向,往往由應用需求決定。Gmapping 出自學術實驗室,追求演算法的優雅;Cartographer 出自產業需求,追求大規模場域的實用性。兩者沒有優劣,只有適不適合——這正是第 8 章「感測器選型是綜合判斷」觀念的再次印證。

案例 15-3 感測器組態的實驗:什麼配置最適合什麼場域

有研究團隊系統性地測試了不同感測器組合對 Cartographer 效能的影響,結果頗具啟發性:在雜亂的房間中,三具 2D 光達加上 IMU 與里程計的組合誤差最低;而在長走廊場景中,單具 3D Velodyne 光達加 IMU 則表現最佳。

這個結果與本書前面的討論完全呼應。雜亂房間中,多具 2D 光達從不同角度掃描,能涵蓋更多幾何特徵、減少遮蔽(第 8 章的高度盲區);而長走廊的困境在於縱向缺乏約束(第 13.6 節),3D 光達能看到天花板與地面的結構變化,正好補上這個缺口。

對大學專題與大學部實作的啟示是:不要盲目追求「最貴的感測器」,而要依場域特性選擇組態。這也是一個絕佳的專題題目——在校園不同場域(教室、走廊、中庭)實測不同感測器組合的建圖品質,用第 13.7 節的方法量化比較。這樣的研究既有教學價值,也有實用意義。

實作活動

實作 15-1 Cartographer 建圖與閉環觀察

目的:執行 2D Cartographer 建圖,親眼觀察子地圖的形成與閉環拉正的瞬間,並與第 14 章的 Gmapping 做對照實驗。這是第四篇最重要的一個實作。

第一部分:模擬建圖與閉環觀察(全班可做)

110. 在 TurtleBot3 Gazebo 模擬環境(第 27 章)中安裝並啟動 Cartographer 的 2D 設定。
111. 在 RViz 中顯示子地圖與軌跡,先在小範圍移動,觀察子地圖如何一塊塊形成、如何部分重疊。
112. 遙控機器人繞行模擬世界一個完整迴路,全程注意觀察:前半段地圖工整、後半段開始出現錯位。
113. 關鍵時刻:回到起點附近時,留意閉環是否觸發、地圖是否被拉正。錄下這個瞬間並在課堂上重播討論。
114. 若閉環未觸發,嘗試調整閉環分數門檻與取樣比例(表 15-1),再試一次。

第二部分:對照實驗——Cartographer vs Gmapping(核心)

115. 在同一個模擬世界、同一條行走路線下,分別用 Gmapping(第 14 章)與 Cartographer 各建一次圖。
116. 用第 13.7 節的方法評估兩張地圖:牆是否筆直?迴路是否閉合?有無重影?量測地圖上的關鍵尺寸與模擬設定值比較。
117. 把兩張地圖並排,標示出差異最明顯的區域,並解釋原因(提示:表 15-2)。
118. 觀察資源用量:比較兩者的 CPU 與記憶體使用情形,對照第 14 章公式五與 15.2 節的討論。

第三部分:實體平台建圖(有硬體的班級)

119. 準備 Lua 設定檔,依你的平台調整(有無 IMU、有無里程計、光達型號與安裝位置)。
120. 先在教室建圖驗證設定正確,再挑戰較大場域——理想的是能繞一個完整迴路的走廊或系館一層樓。
121. 緩慢行走,務必走完整個迴路回到起點,觀察閉環。
122. 匯出地圖並評估品質,與同一場域的 Gmapping 結果比較。

討論問題

71. 你觀察到閉環拉正的瞬間了嗎?拉正前後,地圖上哪些區域變化最大?為什麼是那些區域?
72. 對照實驗中,Cartographer 與 Gmapping 的地圖在哪些方面差異最明顯?這印證了表 15-2 的哪幾項?
73. 如果場域沒有迴路(例如只是一條走到底的走廊),Cartographer 的閉環機制還能發揮作用嗎?此時它相對 Gmapping 還有優勢嗎?
74. (大學部)15.3 節提到閉環取樣比例「太低錯過閉環、太高拖慢即時性」。請實際調整這個參數,記錄閉環偵測成功率與系統延遲的變化,找出你場域的合適值。

延伸挑戰(大學部)

其一:設計一個「錯誤閉環」實驗——在模擬世界中建造兩個外觀幾乎相同的房間,觀察 Cartographer 是否會誤判閉環;若誤判,觀察穩健核函數是否減輕了災難程度。這將深化你對 15.4 節穩健化的理解。其二:若學校有 3D 光達,嘗試 Cartographer 3D 模式,掃描一間實驗室或一段樓梯,產生三維點雲模型,體驗背包式建模的概念(15.6 節),並為第五篇(第 17、18 章)的 3D SLAM 預作準備。

延伸閱讀

97. W. Hess, D. Kohler, H. Rapp, and D. Andor, "Real-Time Loop Closure in 2D LIDAR SLAM," ICRA, 2016.(Cartographer 的原始論文,本章主要依據)
98. Cartographer 官方文件(google-cartographer.readthedocs.io)與 Cartographer ROS 文件.(含 Algorithm walkthrough for tuning 調參指南,實作必讀)
99. Google Open Source Blog, "Cartographer" 發布公告.(說明開發動機與室內測繪的應用背景)
100. J. Elseberg and A. Nüchter, "Improving Google's Cartographer 3D Mapping by Continuous-Time SLAM," ISPRS Archives, 2017.(3D 模式的改良研究,含背包式系統的實測)
101. slam_toolbox 開源專案(GitHub).(ROS 2 環境中另一套基於圖優化的 2D SLAM,可與 Cartographer 對照)
102. Ceres Solver 官方文件(ceres-solver.org).(Cartographer 使用的最佳化函式庫,呼應第 12 章)
103. Cyrill Stachniss, Graph-Based SLAM 系列課程影片(YouTube).(圖優化 SLAM 的理論基礎,搭配本章閱讀效果佳)

小結

70. Cartographer 由 Google 開發、2016 年開源,核心特色是「即時閉環」;它分為兩個同時運行的子系統:局部 SLAM(前端,建立子地圖,局部一致但會漂移)與全域 SLAM(後端,在背景尋找閉環約束並最佳化位姿圖)。

71. 子地圖是由數十筆掃描組成的機率格小地圖;分子地圖的三個理由是限制誤差累積、提供最佳化的單元(節點數大減故能即時)、以及成為穩定的閉環比對對象。記憶體與場景大小成正比,故能處理整棟建築。
72. 系統中有兩種掃描匹配:局部匹配把新掃描插入當前子地圖(以 Ceres 求解非線性最小平方);全域匹配則搜尋閉環,難度高得多,需限制取樣節點數以兼顧閉環成功率與即時性。
73. 分支定界法是即時閉環的關鍵:在多解析度預算格上,由低解析度算出分數上界,將不可能勝出的區塊整片剪除,使搜尋量大幅下降;找到約束後以稀疏位姿調整拉正全圖,並用穩健核函數防止錯誤閉環造成災難。
74. 2D 實作需備妥雷射、TF 與(建議)IMU;Cartographer 以 Lua 設定檔組態,參數較多,關鍵在子地圖掃描數、解析度、閉環取樣比例與分數門檻;務必走完整迴路以觀察閉環。
75. 3D 模式需 3D 光達與 IMU,子地圖為體素格,運算量大;背包式空間建模讓操作員邊走邊建立整棟建築的三維模型,應用於 BIM、古蹟保存、設施管理,是大學場域極具潛力的專題方向。
76. 與 Gmapping 相比:Cartographer 基於圖優化而非粒子濾波、以子地圖而非每粒子一張地圖、閉環為核心功能、記憶體與場景成正比、支援 3D 與 IMU,但參數較複雜。兩者正是第 11、12 章兩條理論路線的實體化身。

第四篇的兩大技術主角至此完備。下一章,我們把這些技術帶回最貼近生活的場景——掃地機器人。從隨機碰撞到 SLAM 規劃式清掃、從房間分割與禁區設定到覆蓋路徑規劃,並誠實面對家庭場景的真實挑戰:桌椅腳、鏡面、低矮障礙物與寵物。你會發現,本篇所有理論都在你家客廳裡默默運作著。

台灣自編大學教科書

[@prof](#)

《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第 16 章

掃地機器人 SLAM

編者：李世淵 (機器人叫獸)

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

學習目標

第四篇的理論走到這裡,終於要回到最貼近生活的場景——你家客廳。掃地機器人是 SLAM 技術最成功的大眾化案例,前面十五章的所有觀念,幾乎都能在這台小小的圓盤機器上找到對應。本章是第四篇的收尾,也是一堂「把理論帶回家」的課。學完本章之後,同學應能夠達成下列目標:

96. 說明掃地機器人為什麼需要 SLAM,以及沒有 SLAM 會有什麼後果。
97. 比較隨機碰撞式與 SLAM 規劃式清掃的差異,並理解覆蓋率與效率的量化意義。
98. 說明光達掃地機器人如何建立房間地圖,並連結第 13 至 15 章的技術。
99. 說明房間分割、禁區設定與覆蓋路徑規劃的原理。
100. 列舉掃地機器人的感測器組合,說明各自的角色與互補關係。
101. 分析家庭場景的真實挑戰:桌椅腳、鏡面、低矮障礙物與動態環境。
102. 從掃地路線理解覆蓋路徑規劃的基本策略與評估指標。

核心概念

中文名詞	英文名詞	概念說明
覆蓋路徑規劃	Coverage Path Planning	規劃路徑走遍區域中每一個可通行位置。
弓字形路徑	Boustrophedon Path	像犁田般來回折返的清掃路線,是覆蓋規劃的基本形式。
隨機碰撞式	Random Bounce	早期掃地機的策略:撞到障礙就轉向,無地圖無規劃。
覆蓋率	Coverage Rate	實際清掃到的面積佔可清掃面積的比例。
重複率	Redundancy Rate	重複清掃同一位置的比例,越低效率越高。

中文名詞	英文名詞	概念說明
房間分割	Room Segmentation	把整張地圖自動切分成有意義的房間區塊。
禁區	No-go Zone	使用者在地圖上劃定、機器人不得進入的區域。
虛擬牆	Virtual Wall	地圖上的虛擬界線,功能與禁區類似。
懸崖感測器	Cliff Sensor	偵測前方地面突然消失(樓梯)的紅外線感測器。
3D 飛時測距	3D ToF	以立體方式量測前方障礙形狀的感測器。
結構光	Structured Light	投射光線圖案以判讀障礙物形狀與距離。
多樓層地圖	Multi-floor Map	記憶多個樓層地圖並自動辨識當前所在樓層。

16.1 掃地機器人為什麼需要 SLAM

這個問題乍看簡單:不就是把地板掃乾淨嗎?但仔細想想,「把整個家的地板都掃過一遍、而且不重複太多次」這件事,其實需要回答第 3 章的導航三問:我在哪裡?我要去哪裡?我該怎麼去?

沒有 SLAM 的三個困境

- 不知道哪裡掃過了:沒有地圖與定位,機器人無法記錄「我已經走過這一塊」。結果是有些地方掃了五次、有些地方一次都沒到。
- 不知道何時算結束:沒有地圖就不知道房子有多大、還剩多少沒掃。早期機型只能靠計時器——掃滿三十分鐘就收工,完全不管掃完了沒。
- 不知道怎麼回家:電量將盡時要回充電座,沒有地圖就只能亂走碰運氣,常常卡在半路沒電。

SLAM 帶來的四項能力

- 115. 系統性清掃:有了地圖,才能規劃弓字形路線逐區覆蓋,大幅提升覆蓋率、降低重複率(16.2 節)。
- 116. 斷點續掃:電量不足時回充,充飽後能回到「上次掃到的地方」繼續——這需要準確的定位與記憶的地圖。
- 117. 使用者互動:App 上顯示房間地圖,讓使用者能指定「只掃廚房」、設定禁區、指哪掃哪。這是地圖帶來的產品價值。
- 118. 多次清掃的一致性:第二次清掃不必重新探索,直接載入既有地圖定位即可(第 14 章案例 14-3 的 AMCL)。

把前面的章節串起來

掃地機器人幾乎是本書前五章的實體總複習:它用第 8 章的 2D 光達感知環境、

第 13 章的佔據格地圖記錄空間、第 14 或 15 章的 SLAM 演算法同時定位與建圖、第 10 章的座標轉換整合感測器、第 11 章的粒子濾波做重定位(綁架問題,第 3 章)。理解掃地機器人,就是理解整個第四篇。

16.2 隨機碰撞式掃地 vs SLAM 規劃式掃地

掃地機器人的演進史,是一部從「反應式」走向「審議式」的縮影(第 3 章的架構討論)。

隨機碰撞式:反應式架構的極致

2002 年 iRobot 推出第一代 Roomba 時,採用的是隨機碰撞策略:直線前進、撞到東西就轉一個隨機角度、繼續前進。搭配螺旋、沿牆等幾種簡單行為模式交替使用。

這個設計的靈感其實來自第 3 章提過的行為式架構——不建模、不規劃,讓感知直接驅動行為。它的優點是極其便宜、程式簡單、不會因為地圖錯誤而失效;缺點也很明顯:路線雜亂、覆蓋不均、耗時極長。

有趣的是,這個策略在數學上並非全無道理:只要時間夠久,隨機行走理論上能覆蓋整個連通區域。問題是「夠久」可能意味著一小時——而且永遠無法保證真的掃完了。

SLAM 規劃式:審議式架構的勝利

2010 年 Neato 首度把 2D 光達裝上掃地機,讓機器人能建立佔據格地圖並規劃系統性路線。現代機型不再只是靠隨機碰撞轉向的旋轉刷具,而是執行裝置端 AI 來建立地圖、避開障礙、並適應真實的居家生活方式。

規劃式清掃的核心是弓字形路徑(Boustrophedon,原意為「像牛犁田般來回」):機器人沿著一個方向直線清掃到底,轉 180 度移動一個刷寬,再反向掃回來,如此往復覆蓋整個區域。

比較項目	隨機碰撞式	SLAM 規劃式
是否建圖	否	是(佔據格地圖)
清掃路線	隨機、雜亂	弓字形、系統性
覆蓋率	不保證,依賴時間	高,可接近完全覆蓋
重複率	高	低
所需時間	長	短
能否斷點續掃	否	是
成本	極低	較高(需光達或相機)
對地圖錯誤的容忍	不受影響	地圖錯則清掃錯

表 16-1 兩種清掃策略的比較

注意最後一列:規劃式並非全無缺點。若 SLAM 建圖失敗(第 13.7 節的地圖病徵),機器人會照著一張錯誤的地圖清掃,反而可能漏掉整個房間。這呼應第 13 章的結論:地圖品質直接決定系統成敗。

16.3 光達掃地機器人與 2D 光達建圖

第 1 章案例 1-1 提過,2017 年小米以親民價格推出光達掃地機器人,讓 SLAM 技術首次大量進入一般家庭。本節深入看它如何運作。

硬體配置

典型的光達掃地機器人在機頂有一個凸起的圓塔,裡面是一顆旋轉的 2D 光達(第 8 章),掃描高度約在機身頂部(離地 8 至 10 公分)。它每秒旋轉數圈,量測周圍數百個方向的距離。

值得注意的是成本控制:家用掃地機的光達多採低成本的三角測距原理,量測範圍約 6 至 8 公尺、精度數公分——遠不如工業型號,但對室內建圖已經足夠。這正是第 8 章「解析度要與應用需求匹配」的實例。

建圖流程:第 13 至 15 章的實踐

- 119. 首次進入陌生環境:機器人執行 SLAM,一邊移動一邊建立佔據格地圖並估計自己的位姿(第 13 章)。多數機型會先沿牆走一圈建立房屋輪廓,再處理內部。
- 120. 演算法選擇:早期機型多用粒子濾波類方法(第 14 章),近年因運算晶片進步,越來越多採用圖優化與閉環偵測(第 15 章),使大坪數住宅的地圖更一致。
- 121. 地圖儲存:建好的地圖存在機器內部與雲端,日後清掃直接載入。
- 122. 後續清掃:載入地圖後執行定位(AMCL 類方法,第 11 章),不需重新建圖,可直接規劃路線。

近年趨勢:光達與視覺並用

現代高階機型已不只倚賴光達。業界的做法是雷射光達負責建立地圖與追蹤機器人位置,AI 相機則負責視覺 SLAM 的部分——辨識地板上的物體並決定如何處理。這正是第 9 章「光達給幾何、相機給語意」的商業實踐。

這個組合直接呼應第 8 章表 8-2 的互補分析:光達不怕黑暗、幾何精確,但不知道「那是什麼」;相機能辨識物體,卻在昏暗與無紋理處失效。因此有廠商在相機旁加裝 LED 燈以維持在黑暗或低矮空間的視覺能力。

16.4 房間分割、禁區設定、路徑覆蓋

有了地圖只是第一步。要讓地圖真正好用,還需要三項功能,它們共同構成了使用者眼中的「智慧」。

一、房間分割

房間分割是把一整張佔據格地圖,自動切分成「客廳、臥室、廚房」等有意義的區塊。這對使用者體驗至關重要——沒有分割,使用者只能說「掃全部」;有了分割,才能說「只掃廚房」。

分割的基本思路是找出空間的「瓶頸」:門口與狹窄通道處,自由空間的寬度會突然變窄,這些位置就是天然的分界。常用方法包括依據距離變換找出房間的「核心區」再向外擴張、或是偵測門框的幾何特徵。

現代機型建好地圖後,能為房間貼標籤、排除禁區,甚至記住哪些區域需要更頻繁的清掃。這已經觸及第 3 章提過的語意地圖概念——機器人不只知道「這裡可通行」,還知道「這裡是廚房」。第 31 章會完整討論如何用 AI 做語意標註。

二、禁區與虛擬牆

禁區(No-go Zone)讓使用者在 App 地圖上劃出機器人不得進入的區域——寵物碗、小孩的積木區、易被纏繞的電線區。虛擬牆則是劃一條線禁止跨越。

技術上,這是在代價地圖(第 3 章)上人為加入「障礙」:規劃路徑時把禁區視為不可通行。這是一個聰明的設計——不需要改動 SLAM 或感測器,只需在規劃層加一層使用者定義的約束。同學可以想想:這其實就是把「使用者的知識」注入機器人系統的一種方式。

三、覆蓋路徑規劃

覆蓋路徑規劃(Coverage Path Planning, CPP)與一般路徑規劃有本質差異:

- 一般路徑規劃(第 3、28 章):求「從 A 到 B 的最短路徑」,目標是抵達。
- 覆蓋路徑規劃:求「走遍每一個可通行位置的路徑」,目標是不遺漏。

掃地機器人是後者的典型應用。基本策略是把地圖分割成無障礙的長條區域,在每

個區域內走弓字形,再依序連接各區域。機器人利用機載感測器與(若有的話)預先載入的樓層平面圖輪廓,規劃一條能抵達每個可進入房間、走廊與空間的覆蓋路徑。

評估指標

- 覆蓋率:實際掃到的面積 $\div$ 可清掃面積。理想值接近 100%,但家具下方、角落往往難以完全覆蓋。
- 重複率:總行走距離 $\div$ 理論最短覆蓋距離。越接近 1 越有效率。
- 清掃時間:完成全屋所需時間,是使用者最有感的指標。

這三個指標互相拉扯:提高覆蓋率往往增加重複率與時間。工程上的取捨,正是產品設計的核心。這也是第 30 章專題評分 Rubric 可以參考的量化思路。

16.5 掃地機器人的感測器:光達、IMU、碰撞、懸崖感測器

一台掃地機器人身上的感測器,幾乎是本書第二篇的濃縮版。本節逐一說明它們的角色與互補關係。

感測器	角色	對應章節
2D 光達	建圖與定位的主力,量測周圍距離	第 8 章
IMU	偵測轉向與傾斜,輔助位姿估計	第 7 章
輪速編碼器	提供里程計,SLAM 的運動預測來源	第 3、13 章
碰撞感測器	光達沒看到時的最後防線	本章
懸崖感測器	向下紅外線,防止從樓梯跌落	本章
紅外 / 超音波	近距離障礙偵測,補足光達盲區	第 8 章
AI 相機	辨識物體與語意,避開電線與排泄物	第 9、31 章
3D ToF / 結構	量測前方障礙的立體形狀	第 8、9 章

感測器	角色	對應章節
光		

表 16-2 掃地機器人的感測器組合

為什麼需要這麼多感測器

答案是第 8 章案例 8-1 的「高度盲區」:2D 光達只掃描單一水平高度切面,看不到低於或高於它的東西。因此:

- 低矮障礙(電線、拖鞋、寵物玩具):在光達掃描平面之下,完全看不到,必須靠前方的紅外線、3D ToF 或相機。
- 懸空障礙(懸掛的窗簾、低矮家具邊緣):可能剛好在掃描平面上下,偵測不穩定。
- 樓梯與落差:光達水平掃描完全無法偵測「地面突然消失」,必須靠向下的懸崖感測器——這是掃地機器人不可或缺的安全裝置。
- 玻璃與鏡面:第 8 章的老問題,雷射穿透或反射,需靠碰撞感測器兜底。

現代高階配置

近年高階機型的感測器組合越來越豐富。有機型在單一 AI 相機之外搭配 3D 結構光,構成能辨識大量常見障礙物類型的進階障礙偵測系統。不同感測器各司其職:LED 燈維持相機在黑暗或低矮空間的視力、3D 結構光讀取正前方障礙的形狀、3D ToF 則同時量測高度與寬度並能在強光下維持效能。

這個演進趨勢再次印證本書的核心信念:沒有全能的感測器,只有恰當的組合。掃地機器人正是感測器融合(第二篇)最貼近生活的教材。

16.6 家庭場景中的 SLAM 挑戰:桌椅、鏡面、低矮障礙物

實驗室的教室是 SLAM 的友善場景(第 13.6 節),但家庭是另一回事。本節誠實面對

真實家庭的挑戰——這也是同學觀察自家掃地機器人時最能印證的部分。

一、細瘦的桌椅腳

第 8 章案例 8-1 詳細討論過這個問題:餐椅腳細瘦,在遠處可能整個落在兩束雷射之間而被漏測(第 8 章的角解析度公式)。結果是地圖上沒有這根椅腳,機器人一頭撞上去。

- 對策:近距離時角解析度足夠,加上碰撞感測器兜底;現代機型用 3D ToF 或相機在近距離補足偵測。

二、鏡面與落地窗

鏡子會讓光達「看到」鏡中的虛像,在地圖上畫出一個根本不存在的房間;落地窗則讓雷射直接穿透,機器人以為那裡可以通行。這是第 8 章反覆強調的光達弱點,在家庭中極為常見。

- 對策:使用者手動在 App 中設定禁區(16.4 節);部分機型用超音波偵測玻璃(聲波會被玻璃反射);建圖時暫時遮蔽鏡面。

三、低矮與動態障礙

地上的電線、襪子、寵物玩具、寵物排泄物,是掃地機器人最著名的災難來源。它們低於光達掃描平面,且位置每天都不同——地圖上不會有它們。

- 對策:這正是 AI 相機的價值所在。感測系統融合光達、相機、有時還有結構光感測器的資料,以偵測牆壁、家具腳、玩具甚至電線,而且這個過程是持續進行的。

四、動態環境:家會變

椅子被拉開、行李箱放在客廳、門開了又關——家庭環境每天都在變。這對 SLAM 是嚴峻考驗,因為地圖需要既「記得」永久結構(牆),又能「忘記」臨時物件(第 13.5 節的地圖可塑性)。

- 對策:對佔據機率設上下限讓地圖保持可塑性(第 13.5 節);區分「永久地圖」(牆與家具)與「臨時障礙層」(當次清掃偵測到的物體);部分機型會在多次清掃後統計哪些障礙是常駐的。

五、地面材質變化

磁磚換地毯時,輪子的摩擦力與滑動特性改變,里程計品質下降(第 14.6 節的打滑問題);地毯還會讓機器人「陷入」而改變機身高度,影響光達的掃描平面。

- 對策:調高 SLAM 對里程計的雜訊假設、多倚賴掃描匹配;部分機型會偵測地毯並調整行為(如提高吸力、抬高拖布)。

家庭挑戰	根本原因	主要對策
細瘦桌椅腳	角解析度不足,遠處漏測	近距離偵測、碰撞感測器、3D ToF
鏡面與落地窗	雷射穿透或反射,產生虛像	使用者設禁區、超音波、暫時遮蔽
低矮動態障礙	在光達掃描平面之下	AI 相機辨識、3D 結構光
環境每天在變	地圖需兼顧記憶與遺忘	機率設限、分層地圖、多次統計
地毯與地面變化	里程計品質下降、機身高度改變	調高里程計雜訊、倚賴掃描匹配

表 16-3 家庭場景的 SLAM 挑戰與對策

16.7 教學活動:從掃地路線理解 Coverage Path Planning

本節說明如何把覆蓋路徑規劃設計成一堂不需硬體、卻極有啟發性的課。這是本書認為最適合高中課堂的活動之一,大學部也能延伸為程式實作。

為什麼從掃地路線切入

覆蓋路徑規劃是一個「人人都有直覺、但一動手就發現不簡單」的問題。每個人

都掃過地、拖過地,直覺上知道「要一排一排來、不要亂跑」——但要精確定義「怎麼排最有效率」、「遇到障礙怎麼繞」、「如何確保不遺漏」,就進入了演算法的領域。

紙上活動設計(高中適用)

123. 在方格紙上畫一個「房間」:外框是牆,內部放幾個障礙物(代表沙發、桌子),留一個門口。
124. 讓同學用筆規劃一條路線,要求「經過每一格至少一次、從門口出發回到門口」,並記錄總步數。
125. 比較全班的路線:誰的步數最少?最短路線與最長路線差多少?那些「多繞的步數」就是重複率的具體化。
126. 引入弓字形策略:示範如何把房間切成數個長條區域,每區走弓字形再連接,比較它與直覺路線的效率。
127. 加入變數:如果房間中央多一根柱子,弓字形路線要怎麼調整?如果門口在角落而非中間呢?

觀察家中掃地機器人(延伸作業)

128. 觀察並記錄自家(或影片中)掃地機器人的清掃路線:它是隨機式還是弓字形?它如何處理家具?
129. 拍下 App 上的地圖,標示出:哪些區域被分割成一個房間?有沒有明顯的地圖錯誤(第 13.7 節的病徵)?
130. 刻意製造挑戰:在地上放一雙拖鞋或一條(安全的)繩子,觀察機器人是否偵測到、如何反應。
131. 把它抱到另一個房間放下,觀察重定位過程(第 3 章綁架問題、第 11 章案例 11-2)。

大學部程式實作方向

- 用 Python 實作弓字形覆蓋規劃:輸入一張含障礙的佔據格地圖,輸出覆蓋路徑,並計算覆蓋率與重複率。
- 實作房間分割:用形態學運算或距離變換找出門口瓶頸,把地圖切成區塊。
- 在 ROS 模擬中,把第 14 或 15 章建好的地圖接上自製的覆蓋規劃器,讓虛擬機器人執行完整清掃。

至此,第四篇「2D SLAM 與掃地機器人」全部完成。我們從第 13 章的佔據格地圖出發,經過第 14 章 Gmapping 的粒子濾波、第 15 章 Cartographer 的圖優化與閉環,最後在本章看到這些技術如何在家庭中落地。同學應該能感受到:SLAM 不是遙不可及的尖端科技,它的核心版本正在你家客廳裡默默運轉。

下一篇,我們把維度從平面提升到立體。第五篇「3D 光達 SLAM 與自主巡檢」將介紹三維點雲、ICP 與 NDT 配準、LOAM 系列演算法,以及機器狗與巡檢機器人如何在複雜的三維空間中定位建圖。難度會提升,但同學已經備妥了所有必要的基礎。

公式或流程

一、覆蓋率與重複率

評估清掃效率的兩個核心指標:

$$\text{覆蓋率} = \text{實際清掃到的面積} / \text{可清掃總面積} \times 100\%$$

$$\text{重複率} = \text{實際行走距離} / \text{理論最短覆蓋距離}$$

理論最短覆蓋距離約等於「可清掃面積 ÷ 刷寬」——這是完美弓字形不重複走完全區的距離。重複率為 1 代表完美無重複,實務上因轉彎、繞障礙與返回充電,通常落在 1.2 至 2.0 之間;隨機碰撞式則可能高達 5 以上。

二、弓字形路徑的行數

在一個寬 W 、長 L 的無障礙矩形區域中,以刷寬 w 走弓字形所需的行數與距離為:

$$\text{行數 } n = \lceil W / w \rceil$$

$$\text{總距離} \approx n \times L + (n - 1) \times w$$

第二項是每次轉行時橫向移動的距離。這條式子說明了為何刷寬越大效率越高——行數減少,轉彎次數也減少。這也是產品設計上「加寬刷頭」的動機。

三、掃地機器人的完整運作流程

圖 16-1 掃地機器人一次清掃的完整流程

① 從充電座出發,載入既有地圖(首次則進入 SLAM 建圖模式)

↓

② 定位:以 AMCL 類方法確認自己在地圖中的位置(第 11 章)

↓

③ 房間分割與任務規劃:決定清掃順序,排除禁區(16.4 節)

↓

④ 覆蓋路徑規劃:在當前房間內產生弓字形路線

↓

⑤ 執行清掃:沿路徑移動,同時

- 持續定位與地圖更新(SLAM)
- 即時避障(相機 / 3D ToF / 碰撞 / 懸崖感測器)
- 記錄已清掃區域

↓

⑥ 電量不足? → 規劃返回充電座, 充飽後回到中斷點續掃

↓

⑦ 全部房間完成 → 返回充電座, 上傳地圖與清掃紀錄

四、房間分割的基本思路

圖 16-2 房間分割的距離變換法

① 對佔據格地圖執行距離變換: 每個自由格記錄它到最近障礙的距離

↓

② 房間中央距離值大(空曠)、門口與走道距離值小(狹窄)

↓

③ 取高於門檻的區域作為各房間的「核心種子」

↓

④ 由各核心向外擴張, 直到相遇為止 → 邊界即為房間分界

↓

⑤ 輸出分割後的房間區塊, 供使用者命名與個別指定清掃

案例

案例 16-1 二十年演進: 從 Roomba 到 Physical AI

掃地機器人的二十年, 是一部機器人技術大眾化的縮影。2002 年第一代 Roomba

以隨機碰撞策略問世——沒有地圖、沒有定位,純粹是第 3 章「行為式架構」的商業化。2010 年 Neato 首度導入 2D 光達與 SLAM。2017 年小米以親民價格普及光達機型(第 1 章案例 1-1)。

而到了 2026 年,情況又有了質變。這一代的家用機器人被歸類為「實體 AI(Physical AI)」——它們不再只是靠隨機碰撞轉向的旋轉刷具,而是執行裝置端 AI,能建立地圖、避開障礙,並適應真實居家生活的樣態。它們用光達與相機 SLAM 掃描房間、建立牆面家具與常用動線的 2D 或 3D 地圖;地圖建好後能為房間貼標籤、排除禁區,甚至記住哪些區域需要更頻繁清掃。

這條演進線清楚顯示:SLAM 是基礎,但真實產品會不斷疊加感測器與 AI 來突破單一技術的限制。從第 13 章的佔據格,到本章的語意標籤與行為適應,同學看到的正是本書從第四篇通往第九篇(AI 語意導航)的完整脈絡。

案例 16-2 為什麼你家的掃地機器人總是撞到那張椅子

這是一個幾乎每位使用者都遇過、也最能印證本書理論的日常現象。機器人能靈巧繞開沙發與牆壁,卻總是一頭撞上餐桌的椅腳。

原因是三重的。其一是第 8 章的角解析度問題:細瘦的椅腳在數公尺外可能整個落在兩束雷射之間被漏測。其二是掃描平面高度:有些椅腳上寬下窄,光達掃到的高度剛好是最細的部分。其三是動態性:椅子每天被拉到不同位置,地圖上記錄的舊位置早已失效。

對策也對應三個層次:近距離重掃(角解析度隨距離改善)、加裝 3D ToF 或相機看到完整形狀、以及讓地圖保持可塑性以反映椅子的移動(第 13.5 節)。下次看到機器人撞椅腳,不妨想想這三個原因分別對應本書的哪一章——這是最貼身的 SLAM 複習。

案例 16-3 掃地機器人平台在研究與教學中的角色

有趣的是,掃地機器人不只是消費產品,也是研究與教學的實用平台。有研究團隊以可程式化的掃地機器人底盤作為無人地面載具,搭配低成本 360 度 2D 光達,在 ROS 框架下實作 SLAM 與覆蓋路徑規劃研究。

這類平台的優勢很實際:底盤成熟穩定、有現成的馬達與里程計介面、體積小適合室內、價格遠低於研究級機器人,而且結構上容易加裝額外的感測器支架。對大學的專題或高中的進階社團而言,這是一條可行的低成本路徑——用消費級底盤加上教學型光達與單板電腦,就能組出一台能跑第 14、15 章演算法的研究平台。

這個案例呼應第 8 章 8.7 節的教學平台討論,也給同學一個啟示:做機器人研究不一定要昂貴的設備,善用成熟的消費級硬體作為基礎,把心力放在演算法與應用創新上,往往是更聰明的策略。

實作活動

實作 16-1 覆蓋路徑規劃:從紙上到程式

目的:透過紙上推演、實地觀察與程式實作三個層次,理解覆蓋路徑規劃的策略與評估指標,並把第四篇的所有理論連結到生活經驗。

第一部分:紙上覆蓋規劃(全體,不需器材)

123. 在方格紙上畫一個 12×10 格的房間,內部放兩個 2×3 格的障礙物(沙發、桌子),在一側邊緣留一格作為充電座位置。
124. 每人獨立規劃一條路線,要求經過每一個自由格至少一次,並回到充電座。記錄總步數。
125. 全班統計:最少步數是多少?最多是多少?計算各自的重複率(總步數 $\div$ 自由格數)。

126. 示範弓字形策略:把房間依障礙切成數個長條區,每區走弓字形再連接。比較它與直覺路線的步數差異。

127. 挑戰題:如果把兩個障礙物移到房間正中央並排,弓字形要如何調整?畫出你的解法。

第二部分:觀察真實掃地機器人(家庭作業)

128. 觀察家中(或網路影片中)掃地機器人的清掃路線,判斷它是隨機式還是規劃式,並描述它處理家具的方式。

129. 截圖 App 上的房間地圖,標示:房間分割是否合理?有無第 13.7 節的地圖病徵(變形、斷裂、漂移)?

130. 低矮障礙測試:在地上放一雙拖鞋(安全物品),觀察機器人是繞開、推開還是卡住,並用 16.6 節的知識解釋。

131. 綁架測試:清掃中途把機器人移到另一個房間,計時它花多久重新定位成功(第 3 章綁架問題、第 11 章案例 11-2)。

第三部分:程式實作(大學部)

132. 用 Python 讀入一張佔據格地圖(可用第 13 章實作產生的,或自行定義),實作弓字形覆蓋路徑規劃:把地圖切成無障礙長條區,在每區產生來回路徑,再連接各區。

133. 計算並輸出覆蓋率與重複率,用 matplotlib 把路徑畫在地圖上視覺化。

134. 實作房間分割:依圖 16-2 的距離變換法,把地圖自動切分成房間,以不同顏色顯示。

135. 加入禁區功能:讓使用者指定一個矩形區域,規劃時自動排除,驗證路徑確實繞開。

136. (進階)在 ROS 模擬中,把第 14 或 15 章建好的地圖接上你的覆蓋規劃器,讓虛擬機器人實際執行完整清掃。

討論問題

75. 第一部分中,弓字形路線比直覺路線省了多少步數?這個差距在真實的一百坪住宅中,會轉換成多少清掃時間?
76. 觀察真實機器人時,你發現了哪些地圖病徵?依第 13.7 節判斷,它們最可能的成因是什麼?
77. 綁架測試花了多久重新定位?若機器人被放在一個與原位置外觀相似的房間(例如兩間格局相同的臥室),你預期會發生什麼?(提示:第 11 章的多峰分布)
78. 依表 16-3,如果要為「養寵物的家庭」設計一台掃地機器人,你會優先加強哪些感測器?為什麼?

延伸挑戰(大學部)

其一:比較兩種覆蓋策略——弓字形與螺旋式(從外往內繞)——在同一張地圖上的覆蓋率、重複率與轉彎次數,分析各自適合什麼形狀的房間。其二:實作「斷點續掃」功能——在路徑執行到一半時中斷,記錄已完成的區域,重新啟動後只清掃剩餘部分,並驗證覆蓋率仍達 100%。這個功能需要同時用到定位、地圖與任務狀態管理,是整合第四篇所有知識的好題目。

延伸閱讀

104. H. Choset, "Coverage for Robotics – A Survey of Recent Results," *Annals of Mathematics and Artificial Intelligence*, 2001.(覆蓋路徑規劃的經典綜述,含 Boustrophedon 分解法)

105. E. Galceran and M. Carreras, "A Survey on Coverage Path Planning for Robotics," Robotics and Autonomous Systems, 2013.(覆蓋規劃的完整回顧, 適合專題參考)
106. R. Bormann et al., "Room Segmentation: Survey, Implementation, and Analysis," ICRA, 2016.(房間分割方法的比較研究,含開源實作)
107. S. Thrun, W. Burgard, and D. Fox, Probabilistic Robotics, MIT Press, 2005.(佔據格與蒙地卡羅定位的理論基礎,呼應本章的建圖與重定位)
108. R. Chuchro, "Simulating Coverage Path Planning with Roomba," arXiv, 2022.(以強化學習探索覆蓋規劃,並與掃地機器人內建演算法比較)
109. ROS Navigation2 官方文件之 Coverage Server 說明(docs.nav2.org).(ROS 2 生態中的覆蓋路徑規劃工具)
110. OpenCV 官方文件之 Distance Transform 與形態學運算教學.(房間分割實作的技術基礎)

小結

77. 掃地機器人需要 SLAM 才能回答「哪裡掃過了、何時算結束、怎麼回家」;SLAM 帶來系統性清掃、斷點續掃、使用者互動與多次清掃一致性四項能力,是本書前五章的實體總複習。
78. 隨機碰撞式是反應式架構的商業化(2002 年 Roomba),路線雜亂、覆蓋不保證;SLAM 規劃式建圖後走弓字形路徑,覆蓋率高、重複率低,但地圖錯誤會直接導致清掃錯誤。
79. 光達掃地機以低成本 2D 光達建佔據格地圖,首次清掃執行 SLAM、後續清掃載入地圖定位;現代高階機型光達與 AI 相機並用——光達建圖定位、相機辨識地板物體,正是「光達給幾何、相機給語意」的商業實踐。

80. 房間分割靠找出門口等空間瓶頸把地圖切塊,已觸及語意地圖概念;禁區是在代價地圖上注入使用者知識;覆蓋路徑規劃求「走遍每一格」而非「抵達某點」,以覆蓋率、重複率、清掃時間三指標評估。
81. 感測器組合包含 2D 光達、IMU、輪速編碼器、碰撞、懸崖、紅外 / 超音波、AI 相機與 3D ToF ; 如此多元的原因是 2D 光達的高度盲區——低矮障礙、樓梯落差、玻璃鏡面都需要其他感測器補足。
82. 家庭五大挑戰:細瘦桌椅腳(角解析度不足)、鏡面與落地窗(雷射穿透反射)、低矮動態障礙(在掃描平面之下)、環境每天在變(地圖需兼顧記憶與遺忘)、地面材質變化(里程計品質下降)。
83. 覆蓋路徑規劃是「人人有直覺、一動手就不簡單」的好教材,可從紙上規劃、觀察真實機器人到程式實作分三層次教學,是連結理論與生活的最佳橋樑。

第四篇「2D SLAM 與掃地機器人」到此完成。我們從佔據格地圖出發,經過 Gmapping 的粒子濾波與 Cartographer 的圖優化閉環,最後看到這些技術如何在家庭中落地——SLAM 不是遙不可及的尖端科技,它正在你家客廳裡默默運轉。下一篇,我們把維度從平面提升到立體:第五篇「3D 光達 SLAM 與自主巡檢」將介紹三維點雲處理、ICP 與 NDT 配準、LOAM 系列演算法,以及機器狗與巡檢機器人如何在複雜的三維空間中定位建圖。

台灣自編大學教科書

@prof

《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第五篇 3D 光達 SLAM 與自主巡檢

第 17 章

3D 點雲與光達建圖

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

學習目標

第四篇的 2D SLAM 讓機器人在平面上定位建圖,但真實世界是立體的:樓梯有高度、天花板有管線、機器狗要跨越障礙、無人機要在三維空間穿梭。本章把維度從平面提升到立體,介紹三維點雲的處理與配準技術,為第五篇的 3D SLAM 與自主巡檢揭開序幕。學完本章之後,同學應能夠達成下列目標:

103. 說明 3D 點雲的資料結構、來源與特性,並理解它與 2D 佔據格地圖的根本差異。
104. 說明體素降採樣的原理與必要性,並能估算降採樣的效果。
105. 說明 ICP 點雲配準的流程、變體與限制。
106. 說明 NDT 的核心思想,並比較它與 ICP 的差異與適用場合。
107. 說明 3D 地圖的表示方式與室內空間掃描的流程。
108. 說明點雲顏色化的原理與彩色三維視覺建模的應用。
109. 舉例說明 3D 空間建模在工廠、校園與展場的實際應用價值。

核心概念

中文名詞	英文名詞	概念說明
點雲	Point Cloud	由大量三維點組成的空間資料,每點含 x 、 y 、 z 座標。
體素	Voxel	三維空間中的立方格,相當於 2D 的像素在立體中的對應。
體素降採樣	Voxel Grid Downsampling	把空間切成體素,每格只保留一個代表點以減量。
點雲配準	Point Cloud	求兩片點雲之間的旋轉與平移,使其對齊。

中文名詞	英文名詞	概念說明
	Registration	
迭代最近點	ICP	反覆找對應點並求最佳變換的經典配準演算法。
點對面 ICP	Point-to-Plane ICP	ICP 變體,以點到平面的距離為誤差,收斂較快。
常態分布轉換	NDT	把點雲轉為體素內的高斯分布再匹配的配準方法。
法向量	Normal Vector	描述點所在表面朝向的向量,用於點對面配準與著色。
KD 樹	KD-Tree	加速最近鄰搜尋的空間資料結構,ICP 的效率關鍵。
八叉樹	Octree	以遞迴細分表示三維空間的樹狀結構,用於壓縮地圖。
點雲顏色化	Colorization	把相機影像的顏色投影到點雲上,產生彩色模型。
數位分身	Digital Twin	真實空間或設備的高精度三維數位模型。

17.1 3D 點雲資料

第 8 章介紹過點雲是 3D 光達的輸出格式,本節深入它的結構與特性,這是整個第五篇的資料基礎。

點雲是什麼

點雲(Point Cloud)是由大量三維點構成的集合。最基本的形式中,每個點只有三個數字—— x 、 y 、 z 座標,代表「空間中的這個位置有一個表面」。把數十萬個這樣的點畫出來,物體與環境的立體形狀就浮現了。

每個點還可以附帶額外欄位:

- 反射強度(Intensity):雷射回波的強弱,反映表面材質與顏色深淺。金屬與反光標誌強度高、黑色物體低。
- 顏色(RGB):若融合了相機影像,每點可帶顏色,形成彩色點雲(17.6 節)。
- 線號(Ring):多線光達中,該點來自第幾條雷射線。這個資訊在第 18 章的 LOAM 特徵提取中非常重要。
- 時間戳記:該點被量測的精確時刻,用於運動補償(第 18 章)。

點雲的三大特性

- 無序性:點雲是一堆點的集合,沒有固定的排列順序。這與影像的規則像素格不同——影像可以直接套用卷積,點雲不行。這也是深度學習處理點雲時的一大挑戰。
- 稀疏且不均勻:近處的點密集、遠處的點稀疏(第 8 章的角解析度公式);而且只有「表面」有點,物體內部與沒掃到的地方都是空的。
- 資料量龐大:一台 64 線光達每秒可產生上百萬個點。一趟十分鐘的掃描就是數億個點——這是 17.2 節必須降採樣的原因。

與 2D 佔據格地圖的根本差異

特性	2D 佔據格地圖(第 13 章)	3D 點雲
資料結構	規則格網,每格一個機率值	無序的點集合
空間表示	佔據 / 自由 / 未知三態	只有「表面在這裡」
未知區域	明確標示(灰色)	無法區分「沒東西」與「沒掃到」
記憶體	與面積成正比	與點數成正比,通常大得多
直接用於規劃	可以	不方便,需轉換
細節保真度	受解析度限制	極高,保留原始量測

表 17-1 2D 佔據格地圖與 3D 點雲的比較

請特別注意「未知區域」這一系列。第 13 章強調過佔據格地圖能區分「確定空」與「未知」,這對路徑規劃至關重要;但純點雲沒有這個能力——沒有點的地方,可能是真的空,也可能只是雷射沒掃到。這正是實務上常把點雲轉換成 3D 佔據格(如八叉樹)的原因(17.5 節)。

17.2 Voxel Grid 降採樣

面對動輒數千萬點的資料,第一件要做的事幾乎總是降採樣(Downsampling)——減少點數,同時盡量保留形狀。體素降採樣是最常用、也最直觀的方法。

原理:一格只留一個代表

體素(Voxel)是三維版的像素——把空間切成邊長固定的小立方格。體素降採樣的作法極其簡單:

132. 把整個點雲所在的空間,切成邊長為 d 的體素格網。

133. 把每個點分配到它所在的體素。

134. 每個體素只輸出一個代表點——通常取該體素內所有點的重心(平均座標)。

135. 空的體素直接忽略。

結果是:原本密集區域的大量點被合併成少數幾點,稀疏區域則幾乎不受影響。點雲整體變得均勻而輕量,形狀卻大致保留。

取重心還是取中心

這裡有一個看似細微、實則重要的實作差異。取體素內所有點的重心(質心),能較忠實地反映真實表面位置;若圖快而直接取體素中心,速度較快但精度較差。研究比較顯示,精確的體素降採樣結果幾乎與標準實作一致,而近似版本可能產生虛假的點,最多可能多出兩倍的點數。對精度要求高的建圖,建議使用取重心的精確版本。

降採樣的效果與代價

- 效果:體素邊長 d 越大,保留的點越少。點數的減少大致與 d 的立方成反比——邊長加倍,點數約降為八分之一。這是三維空間的必然結果。
- 代價:細節流失。若體素邊長大於物體特徵尺寸(例如 20 公分的體素配上 5 公分的管線),該特徵就會消失。
- 取舍: d 應略小於「你關心的最小特徵尺寸」。室內建圖常用 5 至 10 公分,大範圍戶外可用 20 至 50 公分。

其他降採樣策略

除了體素法,還有幾種常見策略,各有適用場合:

- 隨機降採樣:隨機丟棄一定比例的點。最快,但可能造成分布不均。
- 均勻降採樣:每隔固定間隔取一點,簡單但不考慮密度。
- 距離門檻法:確保輸出點之間至少相距某個距離。

- 最遠點取樣:反覆挑選離已選點最遠的點,分布最均勻但計算較慢,常用於深度學習前處理。
- 保邊降採樣:近年的改良方向,依點雲特徵動態調整降採樣密度,並特意保留邊緣資訊,避免關鍵特徵流失。

本書實作以體素降採樣為主——它在效果、速度與實作難度之間取得最好的平衡,也是 PCL 與 Open3D 等主流函式庫的預設選擇。

17.3 ICP 點雲配準

點雲配準(Registration)是 3D SLAM 的核心運算:給定兩片部分重疊的點雲,求出把其中一片對齊到另一片所需的旋轉與平移。這正是第 13.3 節掃描匹配的三維版本,而 ICP 依然是最經典的方法。

ICP 的三步驟循環

迭代最近點(Iterative Closest Point,ICP)的流程與第 13 章介紹的 2D 版本完全相同,只是在三維空間中進行:

136. 找對應:對來源點雲中的每個點,在目標點雲中找最近的點,配成一對。
137. 求變換:計算一個旋轉 R 與平移 t (第 10 章的剛體變換),使所有配對點之間的距離平方和最小。
138. 更新並迭代:把變換套用到來源點雲,回到步驟 1 重新找對應,反覆進行直到變換量小於門檻(收斂)。

效率關鍵:KD 樹

步驟 1 的「找最近點」是 ICP 最耗時的部分。若用暴力法,每個點都要與目標點雲的所有點比較,對數十萬點的點雲完全不可行。

解法是使用 KD 樹(KD-Tree)——一種把空間遞迴切分的資料結構,能把最近鄰搜尋從線性時間降到對數時間。實務上,建立 KD 樹與搜尋的效率,往往決定了整個 ICP 的速度。這也是為什麼現代配準函式庫會特別強調平行化的 KD 樹實作。

ICP 的重要變體

- 點對點 ICP(Point-to-Point):最原始的形式,誤差是兩點之間的直線距離。實作簡單,但在平面滑動時收斂慢。
- 點對面 ICP(Point-to-Plane):誤差改為「來源點到目標點所在平面的垂直距離」。這個改動讓點能沿著平面自由滑動而不受懲罰,大幅加快收斂——這正是實務上最常用的版本。代價是需要先估計每個點的法向量。
- 廣義 ICP(GICP):同時考慮來源與目標的局部平面結構,穩健度更高。
- 體素化加速版本:先降採樣再配準,或以體素結構加速對應搜尋,在保持精度的同時提升速度。

ICP 的三個限制

- 需要好的初始猜測:ICP 是局部最佳化,若兩片點雲的初始位置差太遠,會收斂到錯誤的局部最佳解。這是第 13.3 節就強調過的老問題——實務上靠里程計或 IMU 提供初始值。
- 怕結構退化:在長廊、隧道、平坦大面等缺乏幾何約束的場景,點雲可以沿某個方向自由滑動而誤差不變,ICP 無法確定該方向的位移。這是第 13.6 節走廊困境的三維版本。
- 對離群點敏感:動態物體(走動的人)、雜訊點會拉扯配準結果。實務上需搭配離群點剔除或穩健核函數(第 12、15 章)。

17.4 NDT Normal Distributions Transform

ICP 有一個根本的效率瓶頸:每次迭代都要找最近點對應。常態分布轉換(NDT)提出了一個完全不同的思路——不找對應點,而是把點雲轉換成連續的機率分布。

核心思想:把點雲變成機率場

NDT 的做法是:

- 139. 把目標點雲所在空間切成體素(格子)。
- 140. 對每個體素,計算內部所有點的均值與共變異數——也就是用一個三維高斯分布來描述「這一格的點大致長什麼樣」(第 11 章的高斯分布)。
- 141. 於是整片點雲變成一個由許多高斯分布拼成的連續機率場:空間中任一位置,都能算出「這裡有表面的機率密度」。
- 142. 配準時,求一個變換使來源點雲的所有點,落在這個機率場中的總機率最大。

這個轉換帶來一個關鍵優勢:目標函數變成平滑可微的連續函數,可以直接用牛頓法等最佳化方法求解(第 12 章),而且完全不需要在迭代中反覆搜尋最近點。

NDT 與 ICP 的比較

比較項目	ICP	NDT
核心運算	找最近點對應	體素內的高斯分布
是否需搜尋對應	需要(KD 樹加速)	不需要
大量點雲速度	較慢	較快
精度	通常較高	略低於 ICP
對初始值敏感度	較敏感	較不敏感,收斂範圍較大
關鍵參數	對應距離門檻	體素解析度
適合場景	精細配準、點數較少	大範圍、大量點雲、初值較差

表 17-2 ICP 與 NDT 的比較

研究比較指出,NDT 在配準過程中不使用對應點的特徵來計算,對於大量點雲資料,配準速度比 ICP 快,但配準精度不如 ICP。這個結論相當一致地出現在多項實驗中。

NDT 的關鍵參數:體素解析度

NDT 最重要的參數是體素(cell)的邊長。太小則每格點數不足,高斯分布估計不可靠;太大則空間描述過於粗糙,精度下降。有研究針對隧道環境的點雲實測發現,當體素解析度設為 0.5 公尺時能滿足解析度需求,且配準時間最短。這提醒同學:最佳參數與場景尺度密切相關,室內、隧道、戶外各不相同,必須實測調校。

NDT 與 ICP 的合作

既然 NDT 快而穩健、ICP 精確,自然的想法就是兩者結合:先用 NDT 做粗配準快速拉近,再用 ICP 做精配準提升精度。這種「NDT 粗配準 + ICP 精配準」的組合,在實務系統中相當常見,也再次體現本書反覆強調的思想:沒有萬能的演算法,只有恰當的組合。

17.5 3D 地圖與室內空間掃描

有了降採樣與配準工具,就能把一連串的光達掃描拼接成完整的三維地圖。本節說明 3D 地圖的表示方式與室內掃描的實務流程。

三種 3D 地圖表示

- 原始點雲地圖:直接保存所有(降採樣後的)點。保真度最高、視覺效果最好,但資料量大、無法區分「空」與「未知」、也不方便直接規劃路徑。適合測繪與展示。
- 3D 佔據格(八叉樹):把空間切成體素並記錄佔據機率,以八叉樹(Octree)遞迴細分——只有在有東西的地方細分,空曠處用大格子表示,大幅節省記憶體。這種表示

能區分佔據、自由、未知三態,適合機器人導航與避障。第 3 章提過的 OctoMap 即為代表。

- 網格模型(Mesh):把點雲轉成三角面片構成的表面模型。檔案小、可貼材質、適合建築與遊戲應用,但重建過程較複雜且可能失真。

室內空間掃描流程

143. 規劃掃描路線:確保覆蓋所有想掃描的空間,並在關鍵處(轉角、房間之間)保留足夠的重疊,重疊不足會導致配準失敗。
144. 資料採集:以背包式、手持式或機器人載具緩慢移動掃描(第 15.6 節)。速度要慢,並避免劇烈晃動造成運動變形。
145. 前處理:去除離群點、依需求裁切範圍、體素降採樣(17.2 節)。
146. 配準拼接:以 NDT 或 ICP 把相鄰掃描對齊(17.3、17.4 節),配合 SLAM 的位姿估計。
147. 全域最佳化:偵測閉環並執行圖優化(第 12、15 章),消除累積漂移,確保整體一致。
148. 後處理與輸出:視需求轉為八叉樹、網格或加上顏色(17.6 節),輸出為標準格式(如 PLY、PCD、LAS)。

品質評估

與第 13.7 節的 2D 地圖評估相呼應,3D 地圖也需檢查:牆面是否「厚」了(表示配準有誤差,同一面牆被畫成兩層)、直線結構是否筆直、閉環處是否對齊、以及量測關鍵尺寸與實測值比較。「牆變厚」是 3D 建圖最典型的品質指標——理想的牆面應該是薄一層點。

17.6 點雲顏色化與彩色視覺建模

純幾何的點雲雖然精確,但看起來像灰白色的骨架,不易辨識也不好看。加上顏色,點雲才真正成為「數位分身」。

顏色從哪來:投影原理

顏色來自相機影像。原理是第 9 章的針孔投影模型反過來用:

149. 已知相機的內參 K (第 9 章)與相機相對光達的外參(第 10 章的座標變換)。

150. 對每個三維點,先用外參把它從光達座標轉到相機座標。

151. 再用內參把它投影到影像平面,得到對應的像素座標。

152. 讀取該像素的 RGB 值,賦予這個三維點。

這個流程再次印證第 10 章的核心價值:精確的座標轉換與外參校正,是多感測器融合的基礎。外參若差幾度或幾公分,顏色就會「貼歪」——牆的顏色跑到窗戶上。

實務挑戰

- 時間同步:光達與相機必須對齊時間戳記。移動中若差幾十毫秒,顏色就會錯位(第 5、10 章的時間同步問題)。
- 遮蔽處理:相機看不到的點(被前方物體擋住)不應被賦予前方物體的顏色。需要做可見性判斷。
- 曝光與色差:不同時刻拍攝的影像亮度不同,拼接後點雲會出現色塊。需做色彩均衡。
- 視野不匹配:光達常為 360 度,相機視野有限。需多台相機或接受部分點沒有顏色。

彩色點雲的價值

- 可讀性:彩色模型讓非技術人員也能一眼看懂——這對向客戶或校方展示專案成果極為重要。

- 語意線索:顏色提供材質與物體的線索,是後續 AI 語意分割的重要輸入(第 31 章)。
- 文化資產保存:古蹟與文物的數位典藏,顏色與紋理是不可或缺的資訊。
- 缺陷檢測:牆面裂縫、鏽蝕、水漬等問題,在幾何上不明顯但在顏色上一目了然。

17.7 工廠、校園、展場的 3D 空間建模

本節說明 3D 建圖的實際應用價值,特別聚焦於大學場域可以實作的方向。

工廠:設施管理與數位分身

工廠的三維模型能用於產線布局規劃(新設備放得下嗎?搬運路徑會不會撞到?)、管線與消防設施記錄、以及設備維護的空間查詢。相關應用會結合預先載入的樓層平面圖輪廓,規劃能抵達每個可進入房間、走廊與機械空間的覆蓋路徑,並在建築物營運時間內確保機器人在人員與設備周圍安全避障。

光達相對相機的關鍵優勢在此格外明顯:不像以相機為基礎的方法在低光或無紋理區域(例如機房與停車場結構)表現不佳,光達不論照明條件如何都能可靠運作,因此成為設施維護應用的明確選擇。

校園:大學場域的實作方向

- 建築測繪:掃描系館內部,產生可用於教學、空間管理與無障礙規劃的三維模型。
- 室內導航地圖:為第九篇的語意導航建立空間基礎,讓機器人知道「電梯在哪、樓梯在哪」。
- 實驗室數位分身:掃描智慧機器人教室,作為模擬環境的基礎——這讓同學能在虛擬的教室中測試演算法,再搬到實體驗證。

- 教學素材:掃描成果本身就是絕佳的教材,讓後續學生看到「我們學校長這樣」的三維模型。

展場與文化資產

展場的三維模型能用於布展規劃、虛擬看展與動線分析;而古蹟與歷史建築的數位保存,更是 3D 掃描最有社會價值的應用之一——一旦建築因天災或歲月損毀,精確的三維紀錄就成為修復的唯一依據。台灣有豐富的歷史建築資源,這是一個兼具技術與人文意義的專題方向。

技術基礎:邊緣運算

值得注意的是,現代 3D 建圖越來越倚賴邊緣運算平台。SLAM 演算法在 NVIDIA Jetson 等邊緣處理器上運行,把個別掃描幀拼接起來,並透過閉環——機器人辨識出先前造訪過的區域,並用這個辨識修正軌跡估計中累積誤差的技術——來修正位置漂移。近年更出現 GPU 加速的體素重建函式庫,能即時建立符號距離場與佔據格,讓機器人在導航的同時重建設施幾何,同時支援建圖與避障。

這意味著 3D SLAM 已經不再是「必須用工作站」的技術,一台掌上型的 Jetson 就能勝任——這對教學與專題實作是極大的好消息,也是第七篇(邊緣 AI SLAM)與第 31 章要深入的方向。

公式或流程

一、體素降採樣的點數估算

把邊長為 d 的體素套用到點雲上,輸出點數約等於「被佔據的體素數」。由於三維空間的關係:

$$\text{輸出點數} \propto 1 / d^2 \quad (\text{表面型點雲})$$

注意這裡是平方而非立方——因為光達掃到的是「表面」而非實心體積,點主要分布在二維曲面上。實務經驗:體素邊長加倍,點數約降為四分之一。若點雲填滿三維體積(如醫學影像),才會是立方關係。

二、ICP 的目標函數

點對點 ICP 求旋轉 R 與平移 t ,使配對點的距離平方和最小:

$$(R^*, t^*) = \operatorname{argmin} \sum_i \| (R \cdot p_i + t) - q_i \|^2$$

其中 p_i 是來源點雲的點、 q_i 是它在目標點雲中的最近點。點對面 ICP 則把誤差改為沿目標點法向量 n_i 的投影:

$$(R^*, t^*) = \operatorname{argmin} \sum_i ((R \cdot p_i + t) - q_i) \cdot n_i)^2$$

差別在於:點對點懲罰「所有方向的偏離」,點對面只懲罰「垂直於表面的偏離」,讓點能沿平面滑動——這正是它收斂較快的原因。

三、NDT 的機率目標

NDT 把目標點雲的每個體素以高斯分布 (μ, Σ) 描述。來源點 p 經變換 T 後落在某體素中,其得分為:

$$\operatorname{score}(p) \propto \exp(-((T \cdot p - \mu)^T \Sigma^{-1} (T \cdot p - \mu)) / 2)$$

配準目標是最大化所有點的總得分:

$$T^* = \operatorname{argmax} \sum_i \operatorname{score}(T \cdot p_i)$$

式中的 $(x - \mu)^T \Sigma^{-1} (x - \mu)$ 正是第 11 章多維高斯分布的核心項(馬氏距離)。同學可以看出:第三篇的機率數學,在此直接成為 3D 配準的工具。

四、3D 建圖完整流程

圖 17-1 從光達掃描到 3D 地圖的完整流程

① 3D 光達輸出原始點雲(每秒數十萬至上百萬點)



② 前處理:離群點剔除、範圍裁切、運動補償



③ 體素降採樣:減少點數並使分布均勻(17.2 節)



④ 特徵/法向量估計:為點對面 ICP 或後續處理準備



⑤ 配準:NDT 粗配準 → ICP 精配準(17.3、17.4 節)

配合里程計/IMU 提供初始猜測



⑥ 累積拼接:把新掃描併入全域地圖



⑦ 閉環偵測 + 圖優化:消除累積漂移(第 12、15 章)



⑧ 後處理:轉八叉樹/網格、顏色化(17.6 節)、輸出

案例

案例 17-1 牆為什麼會變厚:3D 建圖的品質指標

第一次做 3D 建圖的同學,常會遇到一個令人困惑的現象:掃描完一個房間,從上方俯

視點雲,原本應該是薄薄一層的牆面,卻變成厚達十幾公分的「牆板」;甚至有時會看到兩層平行的牆面,中間空著。

這個現象的成因很直接:配準有誤差。機器人在不同時刻從不同位置掃到同一面牆,若位姿估計準確,這些掃描應該完美重疊成一層;若位姿有偏差,同一面牆就被畫在略微不同的位置,疊起來就「變厚」了。偏差大到一定程度,就會出現明顯的雙層牆(這是三維版的「重影」,對應第 13.7 節的 2D 病徵)。

因此「牆的厚度」成為 3D 建圖最實用的品質指標:量測一段牆面點雲在垂直牆面方向的分布厚度,理想值應接近光達本身的量測精度(數公分)。厚度明顯超出,就表示配準或 SLAM 有問題。這個檢查方法簡單、直觀、不需真值,強烈建議同學在每次建圖後都做一次——它能立刻告訴你「這張圖能不能用」。

案例 17-2 從隧道到礦坑:退化場景的三維挑戰

第 13.6 節談過長走廊讓 2D 掃描匹配失效,這個問題在三維世界只會更嚴重。隧道、地下坑道、長廊等場景有一個共同特徵:沿著長軸方向,環境的幾何形狀幾乎不變。點雲可以沿著這個方向自由滑動而不增加配準誤差——這稱為結構退化(Degeneracy)。

針對這類場景,研究者發展出多種對策。有研究團隊在隧道環境中採用「先體素降採樣保持整體結構並減少點數,再以 NDT 求解座標變換並依環境特徵最佳化體素解析度,最後在 ICP 中引入 KD 樹加速點對搜尋並以高斯—牛頓法求解」的組合流程,實測顯示體素解析度設為 0.5 公尺時效果最佳。

這個案例給同學三個啟示:其一,退化場景是 3D SLAM 的共同難題,不是你做錯了;其二,NDT 與 ICP 各有所長,組合使用往往優於單獨使用;其三,參數必須依場景調校,沒有放諸四海皆準的設定。這也預告了第 18 章 LOAM 系列如何用「特徵提取」的方式來對付這個問題。

案例 17-3 工具的選擇:PCL 與 Open3D

做 3D 點雲處理,幾乎一定會用到兩個開源函式庫:PCL(Point Cloud Library)與 Open3D。前者是 C++ 為主的老牌工具,功能極為完整,是 ROS 生態的標準配備;後者由 Intel 開發,以 Python 介面友善、視覺化方便著稱,近年在研究與教學中相當流行。

Open3D 提供多種先進表面配準方法的實作,包括成對的全域配準、成對的局部精修,以及使用位姿圖優化的多路配準——同學應該立刻認出最後一項正是第 12 章的圖優化,只是用在點雲拼接上。

效能方面,兩者各有勝場。有基準測試比較了不同函式庫的配準速度,結果顯示點對面 ICP 通常比點對點 ICP 快、而 NDT 的速度表現也相當有競爭力;近年更出現專為平行化設計的輕量配準函式庫,把降採樣、KD 樹建構、法向量估計等前處理步驟全面平行化,多執行緒版本可比單執行緒快數倍。

對本書讀者的建議:教學與快速原型用 Open3D(Python 上手快、視覺化好),整合進 ROS 系統則用 PCL,追求效能的專題可考慮新一代的平行化函式庫。這與第 8 章「先模擬、再便宜硬體、後追求精度」的學習路徑一脈相承。

實作活動

實作 17-1 點雲處理與配準實作

目的:親手完成點雲的載入、降採樣、視覺化與配準,理解 ICP 與 NDT 的行為差異與參數影響。本實作以 Python 搭配 Open3D 為主,不需 3D 光達硬體即可完成。

第一部分:點雲基礎與降採樣(全體)

137. 安裝 Open3D,載入內建的範例點雲(或從公開資料集下載一片室內掃描),用互動視窗旋轉觀察,感受三維點雲的樣貌。

- 138. 印出點雲的基本資訊:點數、座標範圍(邊界框)、是否含顏色與法向量。
- 139. 體素降採樣實驗:分別用邊長 0.01、0.05、0.2 公尺降採樣,記錄每次的輸出點數,並驗證是否符合公式一的平方關係。
- 140. 視覺比較:把原始點雲與三種降採樣結果並排顯示,觀察在什麼體素大小下,牆角、桌腳等細節開始消失。
- 141. 估計法向量並以顏色顯示(法向量方向對應顏色),觀察牆面、地面、天花板如何被清楚區分——這是點對面 ICP 的前置作業。

第二部分:ICP 配準(全體)

- 142. 取一片點雲,複製一份並套用一個已知的旋轉與平移(例如繞 Z 軸轉 20 度、平移 0.3 公尺),作為「來源點雲」。
- 143. 用點對點 ICP 配準,比較估出的變換矩陣與真值,計算誤差,並記錄迭代次數與耗時。
- 144. 改用點對面 ICP(需先估法向量),比較收斂速度與精度的差異,驗證 17.3 節的論述。
- 145. 初始值實驗:把初始偏移逐步加大(5 度、20 度、60 度、120 度),找出 ICP 開始失敗的臨界點,親身驗證「ICP 需要好的初始猜測」。

第三部分:NDT 與組合配準(大學部)

- 146. 用 NDT 做同一組配準,比較它與 ICP 在速度、精度與初始值容忍度上的差異,填入你自己的表 17-2。
- 147. 體素解析度實驗:調整 NDT 的體素大小(0.2、0.5、1.0 公尺),觀察對成功率與精度的影響,對照案例 17-2 的討論。
- 148. 實作「NDT 粗配準 + ICP 精配準」的組合流程,比較它與單獨使用任一方法的結果。

149. 退化場景測試:自行合成一段「長廊」點雲(兩面平行長牆 + 地板),測試 ICP 沿長廊方向的配準是否失準,重現結構退化現象。

討論問題

79. 體素邊長從 0.01 增加到 0.2 公尺,點數減少了幾倍?符合平方關係還是立方關係?為什麼?
80. 點對面 ICP 比點對點快多少?從 17.3 節的原理解釋這個差異。
81. ICP 在初始偏移多大時開始失敗?這對真實機器人系統意味著什麼(提示:里程計與 IMU 的角色)?
82. 長廊測試中,配準在哪個方向失準?這與第 13.6 節的 2D 走廊困境是同一個問題嗎?
83. (大學部)若要為一台校園巡檢機器狗選擇配準方法,你會選 ICP、NDT 還是兩者組合?說明你的理由與場域考量。

延伸挑戰(大學部)

其一:下載一組公開的 3D 光達資料集(如 KITTI 的一段序列),實作一個簡易的光達里程計——連續對相鄰兩幀做配準並累加位姿,把估出的軌跡與真值比較,觀察漂移如何隨距離累積(這正是第 18 章 LOAM 要解決的問題)。其二:若學校有 3D 光達,實際掃描一間實驗室,完成圖 17-1 的完整流程並輸出彩色點雲模型,再用案例 17-1 的「牆厚度」方法評估品質。這份成果可直接作為第 30 章專題二的基礎。

延伸閱讀

111. P. Besl and N. McKay, "A Method for Registration of 3-D Shapes," IEEE TPAMI, 1992.(ICP 的原始論文,第 13、17 章共同的理論源頭)

112. P. Biber and W. Straßer, "The Normal Distributions Transform: A New Approach to Laser Scan Matching," IROS, 2003.(NDT 的原始論文)
113. Q.-Y. Zhou, J. Park, and V. Koltun, "Open3D: A Modern Library for 3D Data Processing," arXiv, 2018.(Open3D 的介紹論文,含配準工作流程範例)
114. Point Cloud Library (PCL) 官方教學(pointclouds.org).(降採樣、法向量估計、配準的完整範例)
115. Open3D 官方文件與 Python 教學(open3d.org).(本章實作的主要工具,教學友善)
116. A. Hornung et al., "OctoMap: An Efficient Probabilistic 3D Mapping Framework Based on Octrees," Autonomous Robots, 2013.(3D 佔據格地圖的代表作,呼應 17.5 節)
117. X. Li et al., "A Tutorial Review on Point Cloud Registrations: Principle, Classification, Comparison, and Technology Challenges," Mathematical Problems in Engineering, 2021.(點雲配準的完整綜述,含各種降採樣策略比較)

小結

84. 3D 點雲是由大量三維點構成的集合,可附帶反射強度、顏色、線號與時間戳記;它具有無序、稀疏不均、資料量龐大三大特性,且無法像佔據格地圖那樣區分「確定空」與「未知」。
85. 體素降採樣把空間切成立方格,每格以重心作為代表點,能大幅減量並使分布均勻;取重心比取中心精確,近似版本可能產生虛假點。體素邊長應略小於關心的最小特徵尺寸。

86. ICP 以「找對應—求變換—迭代」配準點雲,靠 KD 樹加速最近鄰搜尋,點對面 ICP 因只懲罰垂直表面的偏離而收斂較快,是實務首選。三大限制為需要好初值、怕結構退化、對離群點敏感。
87. NDT 把目標點雲轉成體素內的高斯分布,形成連續可微的機率場,不需搜尋對應點;它比 ICP 快、對初值較不敏感,但精度略低,關鍵參數是體素解析度,須依場景尺度調校。
88. 3D 地圖有三種表示:原始點雲(保真度高)、八叉樹佔據格(能區分三態、適合導航)、網格模型(檔案小、適合展示);室內掃描流程為規劃路線、採集、前處理、配準、全域最佳化、後處理輸出。
89. 點雲顏色化以第 9 章的相機投影模型,把影像 RGB 賦予三維點,關鍵在精確的內外參與時間同步;彩色點雲提升可讀性、提供語意線索,並用於文化資產保存與缺陷檢測。
90. 3D 建模應用於工廠數位分身、校園測繪與導航地圖、展場與古蹟保存;光達不受照明影響的特性使其在低光與無紋理場域勝過相機,而邊緣運算平台的進步已讓 3D SLAM 走出工作站。

本章備齊了三維點雲的處理工具:降採樣、配準、地圖表示與顏色化。但把相鄰掃描逐幀配準累加,誤差仍會隨距離累積——這正是第 7 章漂移問題的三維版本。下一章的 LOAM 給出了漂亮的解答:它不對所有點做配準,而是聰明地提取「邊緣特徵」與「平面特徵」,並用「高頻里程計 + 低頻建圖」的雙層架構,在即時性與精度之間取得絕佳平衡。這是自主巡檢機器人與自駕車 3D 定位的技術骨幹。

台灣自編大學教科書

@prof

《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第 18 章

LOAM 與光達里程計

編者：李世淵 (機器人叫獸)

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

學習目標

上一章我們學會了點雲的降採樣與配準,但把相鄰掃描逐幀 ICP 累加,不僅慢、誤差也持續累積。LOAM 給出了漂亮的解答——不對所有點做配準,而是聰明地提取邊緣與平面特徵,並用「高頻里程計 + 低頻建圖」的雙層架構取得即時性與精度的平衡。這是自主巡檢機器人與自駕車 3D 定位的技術骨幹。學完本章之後,同學應能夠達成下列目標:

- 110. 說明 LOAM 的核心問題與它在 3D 光達 SLAM 發展史上的地位。
- 111. 說明如何以曲率(粗糙度)判別邊緣特徵與平面特徵,以及為何只用特徵點就能配準。
- 112. 解釋高頻里程計與低頻建圖的雙層架構,以及它為何能兼顧即時性與精度。
- 113. 比較 LeGO-LOAM、LIO-SAM、Fast-LIO 等後續系統的改良重點。
- 114. 說明光達與 IMU 緊耦合的動機、優勢與實作要點。
- 115. 說明 3D SLAM 如何應用於自主巡檢機器人。
- 116. 規劃一個以大學場域為對象的巡檢專題(走廊、電梯、實驗室)。

核心概念

中文名詞	英文名詞	概念說明
光達里程計與建圖	LOAM	以邊緣與平面特徵進行即時光達定位建圖的經典方法。
光達里程計	LiDAR Odometry	僅由連續光達掃描推算機器人運動的技術。
曲率 / 粗糙度	Curvature / Smoothness	衡量一點周圍局部起伏程度的指標,用來分類特徵。
邊緣特徵	Edge Feature	曲率高的點,位於角落、柱子、樹幹等尖銳處。

中文名詞	英文名詞	概念說明
平面特徵	Planar Feature	曲率低的點,位於牆面、地面等平坦處。
點到線距離	Point-to-Edge Distance	邊緣特徵配準時使用的誤差量。
點到面距離	Point-to-Plane Distance	平面特徵配準時使用的誤差量。
運動變形	Motion Distortion	光達旋轉期間載具移動造成同一幀點雲扭曲。
地面分離	Ground Segmentation	把地面點與非地面點分開處理,LeGO-LOAM 的關鍵。
兩步最佳化	Two-step Optimization	先由地面平面解部分自由度,再由邊緣解其餘自由度。
光達慣性里程計	LIO	光達與 IMU 緊耦合的里程計方法。
迭代擴展卡爾曼濾波	iEKF	Fast-LIO 採用的高效緊耦合估測架構。

18.1 LOAM : LiDAR Odometry and Mapping

LOAM 由 Ji Zhang 與 Sanjiv Singh 於 2014 年在 CMU 機器人研究所提出,論文標題為「LOAM: Lidar Odometry and Mapping in Real-time」。它是 3D 光達 SLAM 領域影響最深遠的一篇工作——後續幾乎所有主流系統都建立在它的思想之上。

它要解決的問題

第 17 章末尾留下一個難題:把相鄰的 3D 掃描逐幀用 ICP 配準再累加,會遇到兩個瓶頸。其一是速度——每幀數十萬點,ICP 的最近鄰搜尋負擔沉重,難以即時。當掃描包含大量點時,ICP 可能承受過高的計算成本。其二是精度——逐幀累加,誤差持續漂移。

LOAM 的核心洞見是:不需要用上所有的點。環境中真正提供幾何約束的,是那些「有特色」的點——牆角、柱子、樹幹(邊緣),以及牆面、地面(平面)。只要抓住這些特徵點來配準,就能用不到原始點數百分之一的資料量,達到相當甚至更好的精度。

LOAM 的兩大貢獻

- 特徵提取:以曲率(粗糙度)把點分為邊緣特徵與平面特徵,大幅減少參與運算的點數(18.2 節)。
- 雙層架構:把問題拆成「高頻但粗略的里程計」與「低頻但精確的建圖」兩個並行任務,兼顧即時性與精度(18.3 節)。

LOAM 把 SLAM 問題拆成兩個平行任務——里程計與建圖——並藉由從光達掃描中提取邊緣與平面特徵、在長期最佳化中處理特徵對應,在結構化環境中以低運算成本達成高精度。這個「特徵提取 + KD 樹對應搜尋」的架構,已被後續大量光達里程計研究採用為核心流程。

歷史地位

自 Zhang 與 Singh 提出 LOAM 以來,以光達為基礎的 SLAM 研究快速推進;LOAM

以曲率提取邊緣與平面特徵,並透過與 KD 樹的掃描對地圖匹配估計 6 自由度位姿,藉由把里程計與建圖分離到不同的執行頻率,同時達成運算效率與精度。

對本書讀者而言,理解 LOAM 就等於掌握了整個第五篇的鑰匙:第 18.4 節的 LeGO-LOAM、LIO-SAM、Fast-LIO 全都是它的後代,而大學自主巡檢機器人的相關研究,也以這條技術脈絡為基礎。

18.2 邊緣特徵與平面特徵

本節是 LOAM 最具識別度的設計:如何從數十萬點中,挑出少數真正有用的特徵點。

曲率:判別特徵的量尺

LOAM 用一個簡單而巧妙的指標來分類每個點——局部粗糙度(常稱為曲率)。做法是:取某個點與它在同一條掃描線上的鄰居們,計算「這個點偏離鄰居平均位置的程度」。

- 粗糙度高:表示這個點與周圍鄰居的排列不共線,位於尖銳處——這是邊緣特徵。典型例子:牆角、柱子邊、樹幹、桿件。
- 粗糙度低:表示這個點與鄰居近乎排成一直線,位於平坦處——這是平面特徵。典型例子:牆面、地面、天花板、大型平板。

每一幀點雲中,只需挑出粗糙度最高的少數點作為邊緣特徵、最低的少數點作為平面特徵。LeGO-LOAM 論文中的圖示清楚展示了這個過程:原始點雲經分割後,以藍色與黃色分別標示邊緣與平面特徵。

為什麼兩種特徵都需要

這是本節最重要的觀念。兩種特徵提供的幾何約束互補:

- 平面特徵約束「垂直於平面的移動」。一面牆能告訴你「你離牆多遠」,但無法告訴你「你沿著牆走了多遠」——沿牆滑動時,平面約束完全不變。
- 邊緣特徵約束「垂直於邊緣的移動」。一根柱子能鎖定你在水平兩個方向的位置,但沿柱子上下移動它管不著。

把兩者結合,才能鎖定完整的六個自由度。這也解釋了第 17.3 節提到的結構退化:在長廊中,牆面提供的都是「同方向的平面約束」,而缺乏沿走廊方向的邊緣特徵,沿廊方向的位移因而無法確定。

均勻分布:避免特徵集中

LOAM 還有一個實務細節:把每條掃描線分成數個子區段,在每個子區段中各挑選一定數量的特徵點。這確保特徵均勻分布於整個視野,而不會全部集中在某個角落——特徵若集中在一側,配準的幾何條件會很差(這與第 5 章 GNSS 的 DOP 概念異曲同工)。

配準的誤差定義

特徵提取後,配準的方式也隨特徵類型而不同:

- 邊緣特徵:在上一幀(或地圖)中找出對應的邊緣(由兩個鄰近的邊緣點決定),誤差定義為「點到這條線的垂直距離」。
- 平面特徵:在上一幀(或地圖)中找出對應的平面(由三個鄰近的平面點決定),誤差定義為「點到這個平面的垂直距離」。

把所有特徵點的誤差平方和最小化,就得到最佳的位姿變換——這又是第 12 章的非線性最小平方問題。同學應該能感受到:第三篇的數學工具,在每一個具體系統中反覆現身。

18.3 高頻里程計與低頻地圖優化

LOAM 的第二項核心創新,是把 SLAM 拆成兩個並行運行、頻率不同的任務。這個

架構設計極為經典,值得同學仔細理解。

兩個任務的分工

- 光達里程計(高頻):比較相鄰兩幀掃描的特徵,快速估出「這 0.1 秒我移動了多少」。它只用兩幀資料,運算輕、速度快,但精度較低、會漂移。
- 光達建圖(低頻):把當前掃描與「累積的全域地圖」比對,做更精細的配準。地圖包含大量歷史資訊,配準更準,但運算重、頻率低。

在 LOAM 的原始設計中,建圖模組以 1 Hz 的頻率把點雲投影到地圖上;最後,兩個演算法發布的位姿變換被整合起來,產生約 10 Hz 的輸出,代表光達相對於地圖的位姿。

圖 18-1 LOAM 的雙層架構

【光達里程計】高頻(約 10 Hz)

相鄰兩幀特徵匹配 → 快速估出相對運動

特性:即時、輕量、精度較低、會漂移

↓ 提供運動估計與初始猜測

【光達建圖】低頻(約 1 Hz)

當前掃描 ↔ 全域地圖特徵匹配 → 精細配準

特性:精確、運算重、頻率低

↓

【位姿整合】

把兩者的變換整合 → 輸出約 10 Hz 的高頻且精確位姿

為什麼這樣設計是聰明的

這個架構的精妙之處,在於它讓「即時性」與「精確性」不再互相衝突:

153. 高頻里程計滿足即時控制的需求。機器人的運動控制迴路需要高頻位姿,不能等 1 秒才更新一次。

154. 低頻建圖負責消除漂移。它有充裕的時間做精細配準,把高頻里程計累積的誤差拉回來。

155. 高頻里程計為低頻建圖提供良好的初始猜測。這正是第 17.3 節強調的——ICP 類方法需要好的初值才能收斂,而里程計恰好提供了它。

同學應該注意到:這個「快速但粗略的前端 + 緩慢但精確的後端」架構,與第 15 章 Cartographer 的「局部 SLAM + 全域 SLAM」雙子系統如出一轍。這不是巧合,而是機器人系統設計的一個普遍模式:用不同的時間尺度處理不同性質的問題。

運動變形補償

還有一個 3D 光達特有的問題必須處理。機械式光達旋轉一圈需要約 0.1 秒(10 Hz),在這段時間內,載具本身也在移動。結果是:同一「幀」點雲中的各個點,其實是在不同位置量到的——點雲因此被扭曲,稱為運動變形(Motion Distortion,第 8.4 節提過)。

LOAM 的處理方式是:利用里程計估出的運動,把每個點依其量測時刻反推回「幀起始時刻」的座標系,消除變形。這正是第 17.1 節提到「點雲需要時間戳記」的原因。後續系統如 LIO-SAM 也明確要求:當光達以 10 Hz 旋轉時,每個點的時間戳記應在 0 到 0.1 秒之間變化。

18.4 LeGO-LOAM、LIO-SAM、Fast-LIO 簡介

LOAM 開創了道路,後續十餘年間出現了大量改良系統。本節介紹三個最具代表性的後代,它們各自解決了 LOAM 的不同弱點。

一、LeGO-LOAM:輕量化與地面最佳化

LeGO-LOAM 全名為「輕量化且地面最佳化的光達里程計與建圖」,針對地面載具的即時 6 自由度位姿估計而設計。它輕量,能在低功耗嵌入式系統上達成即時位姿估計;它地面最佳化,因為它在分割與最佳化步驟中善用了地平面的存在。

它的兩項關鍵改良是:

- 點雲分割與地面分離:先進行點雲分割以濾除雜訊,再做特徵提取以取得具鑑別度的平面與邊緣特徵。分割能有效去除植被、雜訊等不穩定的點——這對戶外環境特別重要。
- 兩步最佳化:採用兩步驟的 Levenberg-Marquardt 最佳化,用平面與邊緣特徵分別求解 6 自由度變換的不同分量。具體而言,先用地面點提取的平面特徵求出垂直位移與滾轉、俯仰角,再用分割後點雲的邊緣特徵估計水平位移與偏航角,並以前一步的結果作為約束。

實測顯示,LeGO-LOAM 在多變地形的地面載具資料集上,能以更低的運算成本達到與 LOAM 相當或更好的精度。它特別針對水平放置於地面載具上的 16 線光達最佳化,並假設掃描中總是存在地平面——這個假設限制了它的適用範圍,但也正是它高效的原因。

二、LIO-SAM:光達慣性緊耦合與因子圖

LIO-SAM 的創新在於以因子圖搭配貝氏樹進行 IMU 緊耦合,實現具運動感知的子地圖匹配,並整合考量漂移的閉環偵測。

- 緊耦合 IMU:把 IMU 預積分(第 7 章)作為因子加入圖中,大幅提升在快速運動下的穩健度。

- 因子圖架構:採用第 12 章介紹的 GTSAM 與 iSAM2 增量式求解,能高效地融合光達、IMU、GPS 與閉環等多種約束。
- 閉環偵測:其閉環功能改編自 LeGO-LOAM,是基於 ICP 的方法;由於 ICP 執行較慢,官方建議測試時把播放速度設為原速。

值得注意的實作限制:與原始 LOAM 實作相同,LIO-SAM 只能搭配九軸 IMU(能提供滾轉、俯仰與偏航估計),且目前僅支援機械式光達。這類細節在專題實作時非常關鍵,同學選用硬體前務必確認。

三、Fast-LIO 系列:效率的極致

Fast-LIO 走的是另一條路線:不做特徵提取,直接使用原始點雲,並以迭代擴展卡爾曼濾波(iEKF)進行緊耦合估測。它的關鍵創新是一種高效的卡爾曼增益計算公式,使運算複雜度與狀態維度而非量測數量相關,大幅提升速度。

Fast-LIO2 進一步引入增量式 KD 樹(ikd-Tree)資料結構,能高效地增量更新地圖,達成極高的處理頻率,特別適合運算資源受限的平台與快速運動的載具(如無人機)。這條路線證明了:在 IMU 提供良好運動先驗的前提下,不做特徵提取也能又快又準。

系統	年份	關鍵改良	適用場景
LOAM	2014	特徵提取 + 雙層架構	通用,奠基之作
LeGO-LOAM	2018	地面分離、兩步最佳化、輕量化	地面載具、嵌入式平台
LIO-SAM	2020	IMU 緊耦合、因子圖、多源融合	需高穩健度與閉環的場域
Fast-LIO2	2021	免特徵、iEKF、增量式 KD 樹	運算受限平台、快速運動

表 18-1 LOAM 系列主要系統比較

18.5 光達 + IMU 緊耦合

第 18.4 節的三個系統中,有兩個都在強調 IMU。本節說明為什麼光達 SLAM 越來越離不開 IMU,以及緊耦合的意義。

光達單獨使用的三個弱點

- 快速運動下失效:純光達方法在快速動態運動下表現會劣化;掃描之間差異太大,配準難以收斂(第 13.3 節 ICP 需要好初值的老問題)。
- 幾何退化環境:在長走廊或開闊空間等幾何退化的環境中,不足的幾何約束會導致位姿估計失敗。這是第 17.3 節結構退化的再次出現。
- 運動變形:如 18.3 節所述,旋轉期間的載具運動會扭曲點雲,需要運動資訊才能補償。

IMU 恰好補上這三個缺口

回顧第 7 章:IMU 更新率極高(數百 Hz)、短時間內極為精確、完全不受環境結構影響。它恰好能:

156. 在兩幀光達之間提供高頻運動估計,作為配準的優良初值,解決快速運動問題。
157. 在幾何退化的方向上提供獨立的約束——走廊裡光達說不清「走了多遠」,IMU 的加速度積分卻能給出估計。
158. 提供每個點量測時刻的精確姿態,用於運動變形補償。

反過來,光達也回報 IMU 它最需要的東西:不漂移的位姿觀測,用來持續估計並補償 IMU 的零偏(第 7.3 節)。光達慣性里程計(LIO)因而受到高度重視,它緊密融合光達量測與 IMU 資料,結合了光達對光線變化的穩健性與捕捉稠密幾何資訊的能力。

鬆耦合 vs 緊耦合

第 7.6 節在視覺慣性 SLAM 中討論過這組概念,在光達領域完全適用:

- 鬆耦合:光達 SLAM 與 IMU 各自算出位姿,再加權融合。實作簡單,但資訊已被壓縮,精度受限。
- 緊耦合:把 IMU 預積分約束與光達特徵配準誤差,放進同一個最佳化問題(或同一個濾波器)一起求解。精度顯著較佳,是現代系統的一致選擇。

LIO-SAM 以因子圖實現緊耦合(最佳化路線,第 12 章)、Fast-LIO 以 iEKF 實現緊耦合(濾波路線,第 11 章)——兩者殊途同歸,再次印證第 11、12 兩章正是 SLAM 的兩條主幹。

實作要點

- 時間同步:光達與 IMU 的時間戳記必須精確對齊,誤差幾毫秒就會顯著影響精度(第 5、10 章反覆強調的工程細節)。
- 外參校正:IMU 與光達的相對安裝位置與姿態必須準確量測(第 10 章的外參),否則融合結果會系統性偏差。
- IMU 規格:如前所述,部分系統要求九軸 IMU。此外 IMU 的品質(第 7 章的等級表)直接影響在退化場景中能撐多久。

18.6 自主巡檢機器人的 3D SLAM

3D 光達 SLAM 最重要的實際應用之一,就是自主巡檢機器人。本節說明這類系統的需求與設計考量。

為什麼巡檢需要 3D 而非 2D

- 地形起伏:巡檢機器人(尤其是機器狗)需要上下樓梯、跨越門檻、走過斜坡。2D 地圖無法描述高度變化,也無法判斷「這個台階跨得過去嗎」。
- 立體障礙:工廠的懸空管線、機房的機櫃上緣、樹枝,都不在 2D 掃描平面上。第 8 章的高度盲區在巡檢場域會直接造成碰撞。

- 巡檢目標的三維位置:要記錄「三樓東側牆面第二根管線有鏽蝕」,需要三維座標而非平面位置。
- 資產建模:巡檢的副產品往往是設施的三維模型(第 17.7 節的數位分身),本身就有價值。

典型系統架構

圖 18-2 自主巡檢機器人的 3D SLAM 系統架構

【感測層】3D 光達 + IMU + (可選)相機、GNSS、輪速計



【前端】特徵提取(邊緣 / 平面)或直接點雲配準



【緊耦合估測】LIO:光達約束 + IMU 預積分 → 高頻 6-DoF 位姿



【後端】因子圖優化 + 閉環偵測(第 12 章)



【地圖】3D 點雲地圖 → 轉八叉樹供導航使用(第 17.5 節)



【應用層】路徑規劃、避障、巡檢點導航、影像/熱像資料採集

平台選擇:輪型、履帶、四足

- 輪型:效率最高、續航最好,但受限於平坦地面,無法上樓梯。適合工廠平面與走廊巡檢。

- 履帶型:可跨越較大障礙與斜坡,常見於災害現場與戶外巡檢。
- 四足(機器狗):地形適應力最強,能上樓梯、跨障礙,是近年巡檢的熱門選擇。代價是續航較短、成本高,且劇烈的步態震動對 SLAM 是嚴峻考驗——這正是必須用 IMU 緊耦合的原因。

巡檢任務的特殊需求

- 重複性:巡檢是週期性任務,同一條路線每天走。因此「首次建圖、後續定位」的模式特別適用(第 14 章案例 14-3),且能比較不同時間的資料找出變化(例如新出現的裂縫)。
- 精確定位到巡檢點:機器人需要停在特定位置、以特定角度拍攝儀表或設備。定位誤差需控制在數公分,這是 3D SLAM 精度要求的來源。
- 長時間運行:巡檢任務可能持續數小時,對漂移控制與系統穩定性要求極高——閉環偵測因此不可或缺。
- 資料關聯:巡檢採集的影像、熱像、聲音資料,都需要標上精確的三維位置,才能在數位分身中定位問題。

18.7 大學場域專題:走廊、電梯、實驗室巡檢

本節把前面的技術落實成一個可執行的專題規劃,以大學校園為場域。這也是第 30 章專題四的技術基礎。

場域分析:三種環境的不同挑戰

場域	主要挑戰	技術對策
長走廊	幾何退化,沿廊方向無約束	IMU 緊耦合、善用天花板與地面平面特徵

場域	主要挑戰	技術對策
電梯與樓梯	高度變化、封閉空間、跨樓層	3D 地圖分層、視覺標記輔助樓層識別
實驗室	雜亂、動態(人員走動)、玻璃隔間	動態物體剔除、多感測器互補

表 18-2 大學三種巡檢場域的挑戰與對策

專題規劃建議

159. 階段一——資料採集:以手推車或機器人載具搭載 3D 光達與 IMU,分別在走廊、樓梯間、實驗室錄製資料集(rosbag)。這一步不需演算法能力,適合作為專題起點。
160. 階段二——離線建圖:以錄好的資料,分別執行 LeGO-LOAM、LIO-SAM 與 Fast-LIO,比較三者在各場域的軌跡精度、地圖品質與運算負擔。用第 17 章案例 17-1 的「牆厚度」方法量化評估。
161. 階段三——退化場景實驗:專門針對長走廊,比較「純光達」與「光達 + IMU 緊耦合」的差異,量化 IMU 帶來的改善。這是最有研究價值的一段。
162. 階段四——即時運行與巡檢:把最合適的系統部署到機器人上,設定數個巡檢點,執行自主導航與資料採集,產出巡檢報告。
163. 階段五——跨樓層與數位分身:嘗試建立多樓層地圖,並把成果轉為可展示的三維模型(第 17.6 節的彩色點雲)。

教學與研究價值

這個專題的優點是層次分明:階段一、二適合大學部,階段三、四適合專題生,階段五可延伸為研究所題目。而且成果具體可展示——一份校園的三維數位模型,對系所招生、空間管理與後續教學都有實用價值。

值得一提的是,這條技術路線正是大學相關研究的方向之一:緊耦合的三維光達慣性定位建圖導航,應用於自主巡檢機器人。同學若對此有興趣,本章的內容正是進入該研究領域的入門地圖。

至此,第五篇的核心技術已備妥。下一篇我們將轉向另一條主線——視覺 SLAM。從特徵提取、對極幾何,到 ORB-SLAM2 與 ORB-SLAM3 的完整系統剖析,同學會發現:雖然感測器換成了相機,但特徵、配準、最佳化、閉環這些核心思想完全相通。

公式或流程

一、局部粗糙度(曲率)

對第 i 個點 p_i ,取同一掃描線上鄰近的點集合 S ,其粗糙度定義為:

$$c = (1 / (|S| \cdot \|p_i\|)) \cdot \left\| \sum_{j \in S} (p_j - p_i) \right\|$$

式子的意義是:把 p_i 與所有鄰居的位置差向量加總。若 p_i 位於平坦處,左右鄰居的差向量會互相抵消, c 值很小;若位於尖角,差向量同向疊加, c 值很大。分母的 $\|p_i\|$ 用來正規化距離的影響(遠處點間距大,不應因此被誤判為邊緣)。

- c 大於門檻 $\rightarrow$ 邊緣特徵
- c 小於門檻 $\rightarrow$ 平面特徵

二、點到線與點到面的距離

邊緣特徵的誤差為「點到對應邊緣的垂直距離」。設當前邊緣點為 p ,對應邊緣由地圖中兩點 a 、 b 決定:

$$d_{edge} = \| (p - a) \times (p - b) \| / \| a - b \|$$

分子是三點構成的平行四邊形面積(以外積計算),除以底邊長即得高——也就是點到直線的距離。

平面特徵的誤差為「點到對應平面的垂直距離」。設平面由地圖中三點 a 、 b 、 c 決定,其法向量 $n = (b - a) \times (c - a)$:

$$d_{plane} = |(p - a) \cdot n| / \|n\|$$

三、LOAM 的最佳化目標

把所有特徵點的誤差組合起來,求位姿變換 T 使總誤差最小:

$$T^* = \operatorname{argmin} (\sum d_{edge}(T \cdot p_i)^2 + \sum d_{plane}(T \cdot p_j)^2)$$

這是一個非線性最小平方問題,以 Levenberg-Marquardt 法迭代求解(第 12 章)。

同學可以看出:從第 12 章的抽象理論、第 13 章的 2D 掃描匹配、第 17 章的 ICP,到本章的 LOAM,都是同一套數學在不同資料上的應用。

四、LOAM 完整處理流程

圖 18-3 LOAM 每一幀的處理流程

① 接收一幀 3D 點雲(含每點的時間戳記與線號)



② 運動變形補償:依運動估計把各點反推回幀起始時刻



③ 計算每點的局部粗糙度 c



④ 分區選取特徵:每個子區段挑出粗糙度最高(邊緣)與最低(平面)的點



⑤ 【里程計,10 Hz】與上一幀特徵匹配 → 快速估出相對運動

↓

⑥ 【建圖, 1 Hz】與全域地圖特徵匹配 → 精細位姿, 並更新地圖

↓

⑦ 整合兩者輸出 → 約 10 Hz 的精確位姿 + 全域 3D 地圖

案例

案例 18-1 為什麼只用百分之一的點, 反而更準

初學者聽到 LOAM 「只用少數特徵點配準」時, 直覺反應往往是: 丟掉這麼多資料, 精度不會下降嗎? 答案出乎意料——不但會, 往往還更準。

原因有三。其一, 大部分點是「冗餘」的: 一面平坦的牆上有上萬個點, 但它們提供的幾何約束其實只有一個——「你離這面牆多遠」。用一萬個點或一百個點來表達這個約束, 結果幾乎相同, 但運算量差了一百倍。

其二, 有些點是「有害」的: 植被的葉子、飄動的塑膠袋、走動的人, 這些點在下一幀就不在原位了, 強行配準只會把誤差引入。LeGO-LOAM 的點雲分割步驟正是為了濾除這類不穩定的點。

其三, 特徵點是「精選」的: 曲率極高或極低的點, 恰好是最容易在下一幀被正確辨識出來的點——這與第 9 章視覺 SLAM 偏愛角點的道理完全相同(「白牆上找不回任何一個點」)。

這個案例給同學一個超越技術的啟示: 更多資料不等於更好的結果, 關鍵在於資料的品質與代表性。這個原則在機器學習、統計、工程判斷上處處適用。

案例 18-2 地面的價值:LeGO-LOAM 的巧思

LeGO-LOAM 的名字裡有「Ground-Optimized(地面最佳化)」,這個看似樸素的設計其實非常聰明。

思考一下:對一台在地面行駛的機器人而言,地面是什麼?它是一個永遠存在、極為穩定、面積巨大的平面特徵。而且它提供的約束非常明確——地面能精準地告訴機器人「你離地多高、機身有沒有前後傾或左右傾」,也就是垂直位移與滾轉、俯仰這三個自由度。

LeGO-LOAM 因此把 6 個自由度拆成兩組分別求解:先用地面點的平面特徵求出垂直位移與滾轉、俯仰角,再用其他邊緣特徵估計水平位移與偏航角,並把前一步的結果當作約束。這樣做的好處是:每一步要解的未知數變少,問題更好收斂、運算更快、也更穩定。

這個案例展示了一種重要的工程思維:善用場域的先驗知識。地面載具「一定在地上」是一個免費的、可靠的假設,不利用它就浪費了。當然代價是通用性——這個假設在無人機上就完全不成立。這再次呼應第 15 章的觀察:技術的設計取向,由應用需求決定。

案例 18-3 實作的魔鬼細節:硬體規格的坑

許多同學在專題中興沖沖地下載 LIO-SAM 或 LeGO-LOAM,卻卡在「跑不起來」。原因往往不是演算法,而是硬體規格不符。以下是幾個真實的坑:

- IMU 軸數不足:與原始 LOAM 實作相同,LIO-SAM 只能搭配九軸 IMU(能提供滾轉、俯仰與偏航估計)。買了六軸 IMU 的同學會直接卡關。
- 光達類型不支援:目前僅支援機械式光達。固態光達(第 8 章)的資料格式不同,需要額外適配。

- 缺少必要欄位:系統需要每點的時間戳記與線號(ring)。官方文件明確說明:當光達以 10 Hz 旋轉時,每點的時間戳記應在 0 到 0.1 秒之間;線號則指出該點屬於感測器的哪一個通道,用來正確地組織點雲。若驅動程式沒輸出這些欄位,就必須自行改寫。
- 特定光達最佳化:LeGO-LOAM 特別針對水平放置於地面載具上的 16 線光達最佳化,並假設掃描中總是存在地平面。用在其他配置上效果會打折。

這個案例的教學意義在於:開源系統不是「下載就能用」。閱讀官方文件的硬體需求章節,是專題開始前必做的功課。這個習慣能省下數週的挫折——這也是本書每章都詳述「前置條件」與「常見失敗」的原因。

實作活動

實作 18-1 光達里程計與特徵提取

目的:親手實作曲率特徵提取、觀察邊緣與平面特徵的分布,並執行完整的光達里程計系統,理解漂移與閉環的作用。本實作以公開資料集為主,不需 3D 光達硬體。

第一部分:特徵提取實作(全體)

150. 下載一段公開的 3D 光達資料(如 KITTI 或 LeGO-LOAM 官方提供的校園資料集),取出單一幀點雲。
151. 用 Python 實作公式一的粗糙度計算:對每個點,取同一線號上左右各數個鄰居,計算 c 值。
152. 依 c 值分類:取最高的 2% 為邊緣特徵、最低的 20% 為平面特徵,用不同顏色顯示(可用 Open3D),觀察它們分別落在環境的什麼位置。
153. 驗證直覺:邊緣特徵是否集中在柱子、樹幹、牆角?平面特徵是否集中在地面與牆面?若不是,調整鄰居數量與門檻再試。

154. 統計特徵點數佔原始點數的比例,體會 LOAM 的減量效果。

第二部分:執行光達里程計(全體或大學部)

155. 在 ROS 環境中安裝 LeGO-LOAM 或 A-LOAM(注意檢查案例 18-3 的硬體與資料格式需求)。

156. 播放資料集,在 RViz 中即時觀察:特徵點雲、估計軌跡、逐步累積的 3D 地圖。

157. 觀察漂移:讓資料播放完一個大迴路,比較起點與終點的位置差距,量化累積漂移。

158. 開啟閉環偵測後重跑一次,觀察閉環觸發時整條軌跡與地圖如何被拉正(第 12、15 章的震撼時刻)。

第三部分:系統比較與退化實驗(大學部)

159. 在同一段資料上分別執行 LeGO-LOAM、LIO-SAM(若有 IMU 資料)與 Fast-LIO,比較三者的軌跡精度、運算負擔與地圖品質。

160. 用第 17 章案例 17-1 的「牆厚度」方法,量化比較三張地圖的品質。

161. 退化場景實驗:找一段以長走廊為主的資料(或自行採集),比較純光達與光達加 IMU 的表現差異,驗證 18.5 節的論述。

162. 若有 IMU 資料,刻意關閉 IMU 融合再跑一次,量化 IMU 帶來的改善幅度。

討論問題

84. 你的特徵點佔原始點數的百分之幾?若把這個比例提高一倍,配準精度會提升嗎?運算時間增加多少?

85. 觀察你提取的特徵:在植被茂密或雜亂的區域,特徵分布合理嗎?這印證了 LeGO-LOAM 為何要先做點雲分割?

86. 大迴路的累積漂移是多少公尺?佔總行走距離的百分之幾?閉環修正後改善了多少?
87. (大學部)在長走廊場景,哪一個方向的誤差最大?這與第 13.6 節的 2D 走廊困境、第 17.3 節的結構退化是否為同一問題?
88. (大學部)依表 18-1,若要為一台大學的巡檢機器狗選系統,你會選哪一個?考量因素有哪些?

延伸挑戰(大學部)

其一:實作一個最簡化的光達里程計——只用平面特徵,以點到面 ICP(第 17 章)連續配準相鄰兩幀並累加位姿,把軌跡與資料集真值比較,繪製誤差隨距離成長的曲線。接著加入邊緣特徵,觀察精度改善多少,親身驗證 18.2 節「兩種特徵互補」的論述。其二:採集一段大學走廊的實測資料(依 18.7 節階段一的做法),在其上執行本章的系統並評估品質,作為第 30 章專題四的前導研究。

延伸閱讀

118. J. Zhang and S. Singh, "LOAM: Lidar Odometry and Mapping in Real-time," Robotics: Science and Systems, 2014.(LOAM 原始論文,本章主要依據,CMU 機器人研究所出版)
119. T. Shan and B. Englot, "LeGO-LOAM: Lightweight and Ground-Optimized Lidar Odometry and Mapping on Variable Terrain," IROS, 2018.(地面最佳化與兩步最佳化的原始論文)
120. T. Shan et al., "LIO-SAM: Tightly-coupled Lidar Inertial Odometry via Smoothing and Mapping," IROS, 2020.(因子圖緊耦合的代表作)

121. W. Xu and F. Zhang, "FAST-LIO: A Fast, Robust LiDAR-inertial Odometry Package by Tightly-Coupled Iterated Kalman Filter," IEEE RA-L, 2021.(以及 FAST-LIO2, IEEE T-RO, 2022)
122. LeGO-LOAM 與 LIO-SAM 開源專案(GitHub).(含官方資料集、硬體需求說明與參數設定,實作必讀)
123. A-LOAM 開源專案(GitHub, HKUST Aerial Robotics).(LOAM 的簡潔重實作,程式碼易讀,適合教學研讀)
124. KITTI Vision Benchmark Suite 官方網站.(自駕車 3D 光達的標準評測資料集,含真值軌跡)

小結

91. LOAM 由 Zhang 與 Singh 於 2014 年提出,核心洞見是「不需要用上所有的點」;它把 SLAM 拆成里程計與建圖兩個平行任務,藉由提取邊緣與平面特徵,在結構化環境中以低運算成本達成高精度,成為後續光達 SLAM 研究的共同基礎。
92. 特徵以局部粗糙度(曲率)分類:高曲率為邊緣特徵(柱子、牆角)、低曲率為平面特徵(牆面、地面);兩者提供互補的幾何約束——平面約束垂直於面的移動、邊緣約束垂直於線的移動,缺一則自由度無法完全鎖定。
93. 雙層架構是 LOAM 的第二項創新:高頻里程計(約 10 Hz)提供即時位姿與良好初值,低頻建圖(約 1 Hz)與全域地圖精細配準以消除漂移,兩者整合後輸出高頻且精確的位姿。此外必須做運動變形補償。
94. LeGO-LOAM 以點雲分割濾除雜訊、善用地平面做兩步 Levenberg-Marquardt 最佳化,在低功耗嵌入式系統上即時運行,以更低運算成本達到相當或更好的精度,但假設掃描中總是存在地平面。

95. LIO-SAM 以因子圖搭配貝氏樹實現 IMU 緊耦合與漂移感知的閉環整合;Fast-LIO 系列則走免特徵提取、以迭代擴展卡爾曼濾波緊耦合的路線,適合運算受限平台與快速運動。
96. 純光達在快速運動與幾何退化環境(長廊、開闊地)會失效,IMU 恰好補上這三個缺口(提供初值、退化方向的獨立約束、運動變形補償),而光達回報以不漂移的觀測校正 IMU 零偏;緊耦合優於鬆耦合。
97. 自主巡檢需要 3D 而非 2D,因為要處理地形起伏、立體障礙、三維巡檢目標與資產建模;平台可選輪型、履帶或四足,而四足的步態震動使 IMU 緊耦合成為必要。

本章完成了 3D 光達 SLAM 的核心技術。從第 17 章的點雲處理到本章的 LOAM 系列,同學已經具備在三維世界中定位建圖的完整知識。下一篇我們轉向另一條主線——視覺 SLAM。感測器換成了相機,但同學會發現:特徵提取、配準、最佳化、閉環這些核心思想完全相通,只是從「幾何點雲」換成了「影像像素」。第 19 章將從視覺 SLAM 的基本流程與特徵法談起,正式進入第六篇。

台灣自編大學教科書

@prof

《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第六篇 Visual SLAM 與視覺導航機器人

第 19 章

Visual SLAM 入門

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

學習目標

第五篇我們用雷射「量」出了三維世界;從本篇起,改用相機「看」。感測器換了,但同學會發現核心思想完全相通——特徵、配準、最佳化、閉環,只是從幾何點雲換成了影像像素。本章是第六篇的入口,建立視覺 SLAM 的整體框架與特徵技術基礎。學完本章之後,同學應能夠達成下列目標:

- 117. 說明視覺 SLAM 的基本流程,並指出各階段對應本書的哪些章節。
- 118. 比較特徵法與直接法兩大流派的原理、優缺點與適用場合。
- 119. 說明 ORB、SIFT、SURF、FAST、BRIEF 的角色與特性,並理解為何 ORB 成為 SLAM 的主流選擇。
- 120. 區分特徵追蹤與特徵匹配的差異,並說明常見的誤匹配剔除策略。
- 121. 解釋單目 SLAM 的尺度不確定性,以及三種解決途徑。
- 122. 比較雙目 SLAM 與 RGB-D SLAM 的差異與適用場景。
- 123. 舉例說明視覺 SLAM 在 AR / VR 與機器人中的應用。

核心概念

中文名詞	英文名詞	概念說明
視覺 SLAM	Visual SLAM	以相機為主要感測器的同步定位與建圖。
視覺里程計	Visual Odometry	由連續影像估計相機運動,會累積漂移。
特徵法	Feature-based / Indirect	提取特徵點並匹配,再由幾何關係求解運動。
直接法	Direct Method	不提特徵,直接以像素亮度差估計運動。
關鍵點	Keypoint	影像中顯著且可重複偵測的位置,如角點。

中文名詞	英文名詞	概念說明
描述子	Descriptor	描述關鍵點周圍外觀的向量,用於比對。
尺度不變性	Scale Invariance	物體遠近改變時仍能偵測到同一特徵的能力。
旋轉不變性	Rotation Invariance	影像旋轉後仍能正確匹配的能力。
漢明距離	Hamming Distance	兩個二元描述子相異位元的個數,比對極快。
特徵匹配	Feature Matching	在兩張影像間找出對應的同一特徵點。
特徵追蹤	Feature Tracking	在連續影像中持續追蹤同一特徵的位置。
尺度不確定性	Scale Ambiguity	單目相機無法確定地圖真實大小的先天限制。
詞袋模型	Bag of Words	把影像表示為視覺單詞的統計,用於地點辨識。

19.1 視覺 SLAM 的基本流程

第 9.6 節我們畫過一條知識階梯:影像處理 → 特徵提取 → 多視圖幾何 → 視覺里程計 → 視覺 SLAM。本節把這條階梯展開成完整的系統流程。

五個階段

- 感測器資料讀取與預處理:讀取影像、去畸變(第 9.2 節的相機校正)。若是雙目或 RGB-D,還需做立體匹配或深度對齊。
- 視覺里程計(前端):估計相鄰影像之間的相機運動,並三角測量出局部的三維點。這一階段快速但會累積漂移(第 18 章的光達里程計是它的雷射版本)。
- 後端最佳化:對一批位姿與地圖點做非線性最佳化(第 12 章的 Bundle Adjustment),消除累積誤差。
- 閉環偵測:辨識「這個地方我來過」,產生閉環約束以修正全域漂移(第 12、15 章)。視覺 SLAM 常用詞袋模型來做這件事。
- 建圖:依應用需求輸出地圖——稀疏的特徵點地圖、稠密的點雲、或語意地圖(第 1、3 章)。

圖 19-1 視覺 SLAM 的標準流程

影像輸入(單目 / 雙目 / RGB-D)

↓

① 預處理:去畸變、立體匹配(第 9 章)

↓

② 視覺里程計(前端):特徵提取與匹配 → 估計相機運動(第 20 章)

↓

③ 後端最佳化:Bundle Adjustment 精修位姿與地圖點(第 12 章)

↓

④ 閉環偵測:詞袋模型辨識重訪地點 → 全域修正(第 21 章)

↓

⑤ 建圖輸出:稀疏點雲 / 稠密地圖 / 語意地圖(第 31 章)

與光達 SLAM 的對照

把這個流程與第 18 章的 LOAM 並排,會發現架構驚人地一致:兩者都是「特徵提取 → 前端快速估計 → 後端精細最佳化 → 閉環修正」。差別只在於特徵的性質:

164. 光達的特徵是幾何的(邊緣、平面),直接帶有三維座標與距離。

165. 視覺的特徵是外觀的(角點與其周圍紋理),只有二維像素座標,深度需要另外求得。

這個差別導致了視覺 SLAM 的核心難題:如何從「沒有深度的二維觀測」中,恢復出三維的相機運動與場景結構。這正是第 20 章對極幾何要回答的問題。

19.2 特徵法與直接法

視覺 SLAM 有兩條主要技術路線,理解它們的分歧,就理解了這個領域的地形。

特徵法(Feature-based,又稱間接法)

特徵法的流程是三個步驟:

- 偵測:在影像中找出角點等關鍵點。

- 描述:為每個關鍵點計算描述子,記錄它周圍長什麼樣。
- 匹配:依描述子的相似度,在不同影像間找出對應的點。

特徵法在視覺里程計中被廣泛採用,因為它對相機運動、光線變化與動態物體具有穩健性。它之所以稱為「間接」法,是因為它不直接使用像素值,而是先把影像抽象成「一組特徵點」這個中間表示,再用幾何方法求解運動。

166. 優點:對光線變化穩健(描述子多有一定的亮度不變性)、可處理較大的視角變化、易於做閉環偵測(用描述子做詞袋比對)、成熟穩定。

167. 缺點:計算特徵與描述子耗時、丟棄了大部分像素資訊、在紋理稀少處(白牆)找不到特徵而失效(第 9 章的「白牆詛咒」)。

直接法(Direct Method)

直接法完全跳過特徵提取,直接使用像素亮度。它的核心假設是灰度不變性:同一個空間點在不同影像中的亮度應該相同。於是估計運動就變成:找一組相機運動,使得「把上一幀的像素依這個運動投影到當前幀後,亮度差最小」。

168. 優點:不需計算特徵與描述子,速度可以很快;能利用影像中所有有梯度的像素(不只角點),因此在紋理稀少但有微弱梯度的場景仍可運作;能自然地產生半稠密或稠密地圖。

169. 缺點:灰度不變性假設脆弱——光線變化、自動曝光、非朗伯表面都會破壞它;對大幅度運動收斂困難(需要好的初值);較難做閉環偵測。

代表系統包括 LSD-SLAM 與 DSO(Direct Sparse Odometry)。

比較項目	特徵法(間接法)	直接法
使用的資訊	特徵點與描述子	像素亮度

比較項目	特徵法(間接法)	直接法
核心假設	特徵可重複偵測與匹配	灰度不變性
對光線變化	較穩健	敏感
弱紋理場景	易失效	仍可利用微弱梯度
大幅運動	較能處理	易發散,需好初值
閉環偵測	容易(詞袋模型)	較困難
地圖密度	稀疏	半稠密或稠密
代表系統	ORB-SLAM 系列	LSD-SLAM、DSO

表 19-1 特徵法與直接法的比較

本書第六篇以特徵法為主線(第 20 至 22 章的 ORB-SLAM 系列),因為它成熟穩定、資源豐富、也最適合教學。但同學應知道直接法的存在——近年也出現結合兩者優點的混合方法。

19.3 ORB、SIFT、SURF、FAST、BRIEF

這五個名詞常一起出現,但它們的角色其實不同:有的只負責「偵測」、有的只負責「描述」、有的兩者都做。釐清這一點是本節的第一個重點。

偵測器與描述子:分工不同

170. FAST:只是偵測器。它極快地找出角點,但不產生描述子。

171. BRIEF:只是描述子。它為給定的關鍵點產生二元描述向量,但不負責找點。

172. SIFT、SURF、ORB:兩者兼具,既偵測也描述。

實務上這些方法可以獨立、分開地作為偵測器或描述子使用,不同組合會有不同的效能表現。這解釋了為何文獻中會出現「FAST-BRIEF」「SIFT-SURF」這類組合寫法。

五大演算法簡介

173. SIFT(尺度不變特徵轉換):經典中的經典。它以高斯差分找出不同尺度下的極值點,產生 128 維的描述子,具有優異的尺度與旋轉不變性。SIFT 本質上把特徵偵測與描述向量產生合併為單一的關鍵推導步驟。缺點是計算量大。
174. SURF(加速穩健特徵):SIFT 的加速版,以積分影像與盒式濾波器近似高斯運算。整體而言 SURF 的計算需求低於 SIFT。
175. FAST(加速分段測試特徵):它簡單而快速,把一個像素與其周圍 Bresenham 圓上的像素比較,先檢查該像素是否比圓上連續 N 個像素更亮或更暗;若是,它就可能是一個角點。演算法接著採用非極大值抑制來辨識並保留最具鑑別度的關鍵點。
176. BRIEF(二元穩健獨立基本特徵):以「隨機挑選點對比較亮度」產生二元字串描述子。優點是產生極快、比對只需算漢明距離(位元運算),極為高效。缺點是原始版本不具旋轉與尺度不變性。
177. ORB(定向 FAST 與旋轉 BRIEF):把 FAST 與 BRIEF 結合並各自改良——為 FAST 角點加上方向資訊(定向 FAST),並讓 BRIEF 描述子隨方向旋轉(旋轉 BRIEF),因而獲得旋轉不變性。

為什麼 ORB 成為 SLAM 的主流

答案是速度。ORB 的開發者報告它比 SURF 快 14 倍,而 SURF 又比 SIFT 快 24 倍;這個較新的演算法在表現上也大致相當,某些情況下甚至好得多。另有文獻指出 ORB 的提取速度約比 SURF 快 15 倍,同時保持對旋轉與尺度變化的穩健性,這個效率使它特別適合即時應用。

在視覺 SLAM 的各種特徵提取演算法中,ORB 特徵的優勢在於提取與匹配快速,而相較 SIFT 與 SURF 並未損失太多精度。ORB 因此成為 SLAM 系統中主流的特徵點偵

測方法——它以部分精度與穩健度換取運算速度的提升,這個取捨對即時系統極為划算。

ORB 描述子的巧思

值得一提 ORB 描述子的設計細節。在 ORB 中,BRIEF 描述子是 256 位元的向量,每個位元代表同一個 31×31 像素區塊中兩個 5×5 像素視窗平均強度比較的結果;一個區塊中有 676 個這樣的視窗,因此有超過二十萬種可能的成對比較。為了最大化描述子的鑑別力,ORB 從這二十萬種組合中,挑選出「離平均最遠且彼此相關性最低」的 256 個比較結果。

這個設計展現了一種重要的工程思維:不是隨機選,而是用資料驅動的方式挑出最有資訊量的比較。同學若日後設計自己的特徵或指標,這個「最大化鑑別力、最小化冗餘」的原則值得參考。

演算法	角色	描述子形式	速度	不變性
SIFT	偵測 + 描述	128 維浮點	最慢	尺度、旋轉
SURF	偵測 + 描述	64 / 128 維浮點	中	尺度、旋轉
FAST	僅偵測	無	極快	無(原始版)
BRIEF	僅描述	二元字串	極快	無(原始版)
ORB	偵測 + 描述	256 位元二元	快	旋轉,部分尺度

表 19-2 五種特徵演算法的比較

19.4 Feature Tracking 與 Feature Matching

找到特徵之後,下一步是在不同影像間建立對應關係。這裡有兩種思路,同學常混淆,本節釐清它們。

特徵匹配(Matching):比對描述子

匹配的做法是:在兩張影像中各自獨立提取特徵與描述子,然後對第一張的每個特徵,在第二張中找描述子最相似的那一個。

178. 相似度度量:浮點描述子(SIFT、SURF)用歐氏距離;二元描述子(ORB、BRIEF)用漢明距離——只需計算兩個位元串有幾位不同,用 XOR 加位元計數即可完成,速度極快。這是二元描述子的最大優勢。

179. 適用場景:兩張影像差異較大時(視角改變、時間間隔長、閉環偵測),匹配是唯一可行的方法。

特徵追蹤(Tracking):在鄰近位置找

追蹤的做法是:在第一張影像提取特徵後,不在第二張重新提取,而是假設特徵只移動了一小段距離,直接在原位置附近搜尋最相似的影像區塊。典型方法是光流(第 9.4 節)。

180. 優點:不需計算描述子,速度更快;在連續影像中效果好。

181. 限制:只適用於影像變化小的情況(高幀率、緩慢運動);追蹤久了會累積漂移,特徵可能「跟丟」或漂到別的地方。

誤匹配:必須面對的現實

無論匹配或追蹤,都必然產生錯誤的對應。原因包括:重複紋理(磁磚、欄杆、樹葉,處處長得一樣)、視角變化太大、光線改變、動態物體。而在幾何求解中,少數幾個嚴重的誤匹配就可能把結果拉偏——這是視覺 SLAM 最常見的失敗原因之一。

三種誤匹配剔除策略

- 比值檢驗:對每個特徵,找出最相似與次相似的兩個候選。若兩者的距離很接近,表示「難以區分」,這個匹配不可靠,予以捨棄。這是 SIFT 論文提出的經典做法,簡單而有效。

- 交叉檢驗:A 影像的特徵 a 匹配到 B 的特徵 b,同時 b 反過來也匹配到 a,才認可。單向的匹配視為不可靠。
- 幾何驗證(RANSAC):最重要的一招。利用「所有正確匹配必須符合同一個相機運動」這個幾何約束,反覆隨機取樣求解並統計符合者,把不符合的判為離群點剔除。實務上以 RANSAC 估計本質矩陣與內點集合,是從含有離群點的特徵對應中估計相機相對位姿的標準做法。這個方法將在第 20 章詳細展開。

19.5 Monocular SLAM 的尺度不確定性

這是單目視覺 SLAM 最根本、也最無法迴避的限制。第 9.1 節從針孔投影公式指出了它的幾何根源,本節說明它的後果與解法。

問題的本質

回顧第 9.1 節的投影公式: $x = f \cdot X/Z$ 。注意這個式子中,若把場景中所有點的三維座標同時放大 k 倍,同時把相機的所有平移也放大 k 倍,投影到影像上的結果完全相同。

換句話說,單目相機拍到的一段影像序列,可能對應「真實房間中的真實運動」,也可能對應「一個模型屋中的微小運動」——兩者在影像上完全無法區分。單目 SLAM 因此只能恢復出「差一個未知比例因子」的軌跡與地圖。

後果

182. 地圖沒有真實尺寸:建出的地圖可以說「這面牆到那面牆是 3.7 個單位」,但不知道一個單位是幾公尺。
183. 無法直接用於導航:機器人不知道「前方障礙物到底在 0.5 公尺還是 5 公尺外」,自然無法安全避障。
184. 尺度漂移:更麻煩的是,這個未知的比例因子在長時間運行中還會緩慢改變(稱為尺度漂移),使地圖逐漸變形。

三種解決途徑

- 引入 IMU(視覺慣性):加速度計量測的是有物理單位的加速度,積分後即可提供真實尺度。這是最通用的解法,也是第 7 章與第 22 章的核心主題。代價是需要足夠的運動激勵才能觀測到尺度(第 7.7 節)。
- 使用雙目或 RGB-D 相機:雙目的基線是已知的固定距離,RGB-D 直接輸出以公尺為單位的深度。硬體上直接解決問題,是最省事的做法(19.6 節)。
- 已知尺寸的參考物:若場景中有已知大小的物體(如第 4 章的 AprilTag、棋盤格),或已知相機離地高度,就能反推尺度。這在特定應用中很實用。

值得補充的是,近年也有以深度學習從單張影像估計深度(第 9.4 節的 Depth Anything 類模型)來提供尺度先驗的做法,但其精度與可靠度仍不及前三種物理方法。

19.6 Stereo SLAM 與 RGB-D SLAM

第 9.3 節介紹過三種相機取得深度的方式,本節聚焦它們在 SLAM 系統中的差異。

雙目 SLAM

雙目相機以已知基線的兩個鏡頭同時拍攝,由左右影像的視差直接算出深度(第 9.6 節的公式 $Z = f \cdot b / d$)。

185. 優點:單幀即可得深度、有絕對尺度、室內外皆可運作(不依賴主動投射)、量測範圍可透過加大基線延伸。
186. 限制:需精確校正兩相機的相對位置(外參)、在無紋理表面找不到對應而無深度、深度誤差隨距離平方成長(第 9.6 節)、運算量較大(需做立體匹配)。
187. 適用:戶外機器人、無人機、自駕車——凡是需要陽光下運作、且距離範圍較大的場景。

RGB-D SLAM

RGB-D 相機主動投射紅外線圖案取得稠密深度(第 9.3 節的結構光或主動紅外雙目)。

188. 優點:直接輸出每個像素的深度,不需自己做立體匹配;在無紋理的白牆上也有深度(因為是主動投射);有絕對尺度;能輕易建出稠密的三維地圖。

189. 限制:多依賴紅外線,強烈日光下失效;有效範圍通常在 0.2 至 10 公尺;對玻璃鏡面同樣束手無策(與第 8 章光達的弱點相同);功耗較高。

190. 適用:室內服務機器人、桌面級三維重建、室內測繪——這也是第 29 章實作選擇 RGB-D 的原因。

比較項目	單目 SLAM	雙目 SLAM	RGB-D SLAM
絕對尺度	無(需外部提供)	有	有
深度來源	運動三角測量	左右視差	主動投射
無紋理表面	失效	無深度	仍有深度
戶外強光	可用	可用	多半失效
有效距離	不限(但無尺度)	較遠	約 0.2–10 m
地圖密度	稀疏	可半稠密	稠密
硬體成本	最低	中	中

表 19-3 三種視覺 SLAM 配置的比較

ORB-SLAM2 與 ORB-SLAM3(第 21、22 章)的一大優點,就是同時支援這三種模式——同樣一套系統,換個輸入模式即可。這讓同學能在同一個框架下比較三者的差異,是絕佳的教學素材。

19.7 視覺 SLAM 在 AR / VR 與機器人中的應用

第 1 章 1.4 節提過 AR / VR 是 SLAM 四大應用之一,本節深入視覺 SLAM 在其中的角色,並延伸到機器人應用。

AR / VR:口袋裡的視覺 SLAM

AR 眼鏡或手機 AR 要把虛擬物件穩定地「釘」在真實桌面上,前提是裝置必須即時知道自己在空間中的六自由度位姿。業界慣稱這個能力為「空間定位」或「由內而外追蹤」(Inside-Out Tracking),而它的技術本質就是視覺慣性 SLAM。

191. 為什麼是視覺而非光達:AR 裝置對重量、功耗、成本極度敏感,相機加 IMU 是唯一可行的組合(第 9.5 節的視覺優勢)。

192. 特殊需求:延遲必須極低(否則虛擬物件會「漂移」造成不適)、且必須在使用者快速轉頭時仍維持追蹤——這對視覺 SLAM 是嚴苛考驗,也是 IMU 不可或缺的原因(第 7.6 節)。

193. VR 頭盔的演進:早期 VR 需要外部基地台定位,現代頭盔改用機身相機做視覺 SLAM,使用者因此能自由走動——這是 SLAM 直接改變產品形態的例子。

機器人應用

194. 室內服務機器人:視覺 SLAM 提供的不只是幾何,還有語意(第 9.7 節)——機器人能辨識「那是門、那是電梯」,這是光達給不了的(第 31 章的主題)。

195. 無人機:載重與電力極度受限,相機加 IMU 的組合是最佳選擇(第 25 章的主題)。

196. 低速自駕與園區載具:視覺 SLAM 在低速自駕情境中的應用是活躍的研究方向,特徵法因其對相機運動、光線變化與動態物體的穩健性而被廣泛採用。

197. 地點辨識與重定位:視覺特徵天生適合做「這裡我來過嗎」的判斷。詞袋技術是最普遍的做法之一,把局部特徵描述子當作視覺單詞使用;而 DBoW2 引入高效的 FAST 偵測器搭配 BRIEF 描述子,以二元單詞袋的形式,相較傳統使用 SURF 與

SIFT 的做法大幅減少特徵提取所需時間,系統更有效率也更穩健。這正是 ORB-SLAM 閉環偵測的技術基礎(第 21 章)。

趨勢:深度學習的加入

傳統手工設計的特徵方法有一個問題:它們無法有效適應各種複雜且無法預期的情況;因此許多研究者採用卷積神經網路來提取影像的深層特徵,應用於視覺里程計、位姿估計與閉環偵測等各個階段。

這是視覺 SLAM 目前最活躍的方向之一:用學習來的特徵取代手工特徵、用神經網路做地點辨識、乃至於端到端的位姿估計。不過在實務系統中,傳統幾何方法因其可解釋性與穩定性,目前仍是主流;學習方法多作為輔助或用於特定困難場景。第 31 章會回到這個主題,討論 AI 如何與 SLAM 結合。

至此,同學已建立視覺 SLAM 的整體框架。下一章我們深入前端的數學核心——對極幾何:兩張影像之間到底藏著什麼幾何關係?本質矩陣與基礎矩陣是什麼?PnP 問題如何求解?三角測量如何恢復三維點?這些是把「二維像素對應」變成「三維相機運動」的關鍵橋樑。

公式或流程

一、漢明距離:二元描述子的比對

兩個二元描述子 d_i 、 d_j 的相似度以漢明距離衡量——即相異位元的個數:

$$Hamming(d_i, d_j) = popcount(d_i XOR d_j)$$

XOR 後計算 1 的個數(popcount)。對 256 位元的 ORB 描述子而言,這只需數個位元運算指令,遠快於浮點描述子的歐氏距離計算。這正是 ORB 能達到即時效能的關鍵之一。距離越小表示越相似。

二、比值檢驗

對某個特徵,設它與最相似候選的距離為 d_1 、與次相似候選的距離為 d_2 ,則接受此匹配的條件為:

$$d_1 / d_2 < \tau \quad (\tau \text{ 常取 } 0.7 \text{ 至 } 0.8)$$

直觀理解:若最佳與次佳的差距不夠明顯(比值接近 1),表示這個特徵在影像中有多個相似的候選,難以區分,應予捨棄。這條簡單的判準能剔除大量誤匹配。

三、直接法的光度誤差

直接法不比對描述子,而是最小化光度誤差——把上一幀的像素依相機運動 T 投影到當前幀後的亮度差:

$$T^* = \operatorname{argmin}_T \sum_i \| I_2(\pi(T \cdot P_i)) - I_1(u_i) \|^2$$

其中 I_1 、 I_2 是兩幀影像的亮度、 u_i 是第一幀中的像素、 P_i 是它對應的三維點、 $\pi(\cdot)$ 是第 9 章的投影函數。與特徵法比較:特徵法最小化的是「重投影誤差」(幾何距離,以像素為單位),直接法最小化的是「光度誤差」(亮度差)。兩者都是最小平方問題(第 12 章),只是誤差的定義不同。

四、視覺 SLAM 前端處理流程

圖 19-2 特徵法前端的每幀處理流程

① 讀取影像 → 去畸變(第 9.2 節)

↓

② 偵測關鍵點:以 FAST 找角點並做非極大值抑制

↓

③ 計算方向與描述子:定向 FAST + 旋轉 BRIEF → 256 位元描述子

↓

④ 與上一幀(或地圖)匹配:漢明距離比對

↓

⑤ 誤匹配剔除:比值檢驗 → 交叉檢驗 → RANSAC 幾何驗證

↓

⑥ 位姿求解:由通過驗證的匹配估計相機運動(第 20 章)

↓

⑦ 三角測量新的三維點 → 交給後端最佳化(第 12、21 章)

案例

案例 19-1 速度的代價與回報:ORB 為何贏得 SLAM 戰場

如果單看特徵的品質,SIFT 幾乎在各項指標上都優於 ORB——它的尺度不變性更完整、匹配更準確、對視角變化更耐受。那為什麼絕大多數即時 SLAM 系統選擇 ORB?

答案是即時性的殘酷要求。一個以 30 幀每秒運行的 SLAM 系統,每一幀只有約 33 毫秒的預算,而這段時間還得完成特徵提取、匹配、位姿求解、地圖更新等所有工作。ORB 開發者報告它比 SURF 快 14 倍,而 SURF 又比 SIFT 快 24 倍——換算下來,ORB 大約比 SIFT 快兩個數量級。這個差距不是「稍微快一點」,而是「能不能即時運行」的分野。

更關鍵的是,ORB 犧牲的精度在 SLAM 系統中可以被彌補。SLAM 有後端最佳化與閉環修正(第 12 章),即使單次匹配略遜一籌,大量的冗餘觀測與全域最佳化能把誤差壓下

來。換句話說:系統層級的設計,補償了元件層級的不足。

這個案例給同學一個重要的工程觀:選擇技術元件時,不能只看該元件的孤立指標,而要看它在整個系統中的貢獻與代價。這與第 8 章「感測器選型是綜合判斷」、第 15 章「設計取向由應用需求決定」一脈相承。

案例 19-2 重複紋理的陷阱:磁磚地板與欄杆

一個在大學課堂上容易重現的失敗實驗:讓機器人在鋪滿相同磁磚的地板上,或沿著整排相同的欄杆移動,執行視覺 SLAM。結果往往是位姿估計突然跳錯,地圖出現嚴重錯位。

原因是重複紋理造成的系統性誤匹配。每一塊磁磚的角點看起來都一模一樣,描述子幾乎相同。演算法在比對時,很可能把「這一塊磁磚的角」匹配到「隔壁那一塊磁磚的角」——而且它「看起來」是一個完美的匹配,描述子距離極小。

這正是比值檢驗存在的意義:當最佳與次佳候選的距離非常接近時(磁磚場景必然如此),比值檢驗會判定「無法區分」而捨棄這個匹配,寧可少一些匹配,也不要錯誤的匹配。若比值門檻設得太寬鬆,這類場景就會出問題。

對照第 13.6 節的長走廊、第 17.3 節的結構退化、第 9 章的白牆詛咒,同學應該看出一個貫穿全書的模式:環境的重複性與特徵匱乏,是 SLAM 的共同天敵,不分光達或視覺。理解這一點,你就能在專題規劃階段預判哪些場域會出問題。

案例 19-3 從手工特徵到深度學習:一個正在發生的轉變

本章介紹的 SIFT、SURF、ORB 都是「手工設計」的特徵——人類研究者根據對影像的理解,設計出偵測與描述的規則。這些方法主導了電腦視覺與 SLAM 二十餘年。

但它們有一個共同弱點:無法有效適應各種複雜且無法預期的情況;因此許多研究者

轉而採用卷積神經網路來提取影像的深層特徵,應用於視覺里程計、位姿估計與閉環偵測等階段。學習來的特徵能適應手工規則難以描述的情境——季節變化、日夜差異、天氣影響。

不過同學要注意:這個轉變尚未完成,而且未必會全面取代。傳統幾何方法有幾個學習方法目前難以匹敵的優點:可解釋(出錯時知道為什麼)、不需訓練資料、在未見過的場景中行為可預期、運算資源需求明確。目前主流實務系統仍以幾何方法為骨幹,學習方法作為特定環節的增強。

對本書讀者的建議是:先把傳統方法學紮實。理解 ORB 為何這樣設計、RANSAC 為何有效、Bundle Adjustment 在解什麼——這些原理是不會過時的思考框架。有了這個基礎,再去看深度學習的方法,你會知道它們在取代什麼、又保留了什麼。第 31 章會回到這個主題。

實作活動

實作 19-1 特徵提取、匹配與誤匹配剔除

目的:親手實作視覺 SLAM 前端的完整流程,比較不同特徵演算法的效能,並見證誤匹配剔除的威力。本實作以 Python 搭配 OpenCV,只需一般相機或現成影像即可完成,是第 20 至 22 章的直接暖身。

第一部分:特徵演算法比較(全體)

163. 準備兩張同一場景但視角略有不同的照片(可用手機拍攝教室,前後移動一小步)。

164. 分別用 SIFT、ORB、FAST 偵測關鍵點,把結果畫在影像上並排比較:各偵測到幾個點?分布在哪裡?

165. 計時比較:記錄三種方法在同一張影像上的提取時間,驗證本章提到的速度差異 (ORB 遠快於 SIFT)。
166. 對一張大面積白牆的照片重複實驗,觀察三種方法都幾乎偵測不到特徵——再次驗證第 9 章的「白牆詛咒」。
167. 旋轉測試:把第二張影像旋轉 45 度,比較 ORB 與原始 BRIEF 的匹配成功率,驗證「旋轉不變性」的價值。

第二部分:匹配與誤匹配剔除(全體)

168. 用 ORB 對兩張影像做暴力匹配(以漢明距離),畫出所有匹配連線,目視觀察有多少條線明顯是錯的。
169. 套用比值檢驗(門檻 0.75),重新畫出匹配結果,統計剔除了多少匹配、剩下的品質如何。
170. 加入交叉檢驗,再次觀察改善程度。
171. 用 RANSAC 估計兩張影像之間的幾何關係(可先用單應矩陣),把內點與外點以不同顏色顯示,觀察 RANSAC 剔除了哪些匹配。
172. 比值門檻實驗:把門檻設為 0.6、0.75、0.9,比較保留的匹配數量與正確率,找出你的場景最合適的值。

第三部分:重複紋理與追蹤(大學部)

173. 重複紋理實驗:拍攝磁磚地板、欄杆或格子狀的窗戶,執行匹配並觀察誤匹配現象,重現案例 19-2。比較比值檢驗在此類場景的重要性。
174. 光流追蹤:錄製一段緩慢移動的影片,用 OpenCV 的 Lucas-Kanade 光流追蹤特徵點,把軌跡畫出來,觀察哪些點被穩定追蹤、哪些跟丟。
175. 比較追蹤與匹配:對同一段影片,分別用「每幀重新提取並匹配」與「光流追蹤」,比較兩者的耗時與穩定度,對照 19.4 節的討論。

討論問題

89. ORB 與 SIFT 的提取時間相差多少倍?與本章引述的資料是否相符?若你的 SLAM 系統要跑 30 fps,哪一種可行?
90. 比值檢驗剔除了百分之幾的匹配?剩下的匹配品質有明顯提升嗎?若把門檻放寬到 0.9,會發生什麼?
91. 在重複紋理場景中,誤匹配呈現什麼模式(是隨機的,還是系統性地偏向某個方向)?為什麼系統性誤匹配比隨機誤匹配更危險?
92. 光流追蹤在什麼情況下跟丟?這對「相機移動速度」與「幀率」的關係有什麼啟示?
93. (大學部)依表 19-3,若要為大學的室內導覽機器人選視覺配置,你會選單目 + IMU、雙目還是 RGB-D?說明理由。

延伸挑戰(大學部)

其一:實作一個最簡單的視覺里程計——連續讀取影片幀,用 ORB 匹配相鄰幀、以 RANSAC 剔除誤匹配、估計相對運動並累加,把軌跡畫出來。觀察漂移如何隨時間累積(這正是第 20 至 21 章要解決的問題)。其二:實作一個簡易的地點辨識——把一段影片的所有幀提取 ORB 描述子,建立簡單的詞袋表示,測試能否正確找出「同一地點的不同時刻影像」。這是第 21 章閉環偵測的雛形。

延伸閱讀

125. E. Rublee et al., "ORB: An Efficient Alternative to SIFT or SURF," ICCV, 2011.(ORB 原始論文,本章 19.3 節主要依據)
126. D. G. Lowe, "Distinctive Image Features from Scale-Invariant Keypoints," IJCV, 2004.(SIFT 經典論文,含比值檢驗的提出)

127. E. Rosten and T. Drummond, "Machine Learning for High-Speed Corner Detection," ECCV, 2006.(FAST 偵測器原始論文)
128. M. Calonder et al., "BRIEF: Binary Robust Independent Elementary Features," ECCV, 2010.(二元描述子的開創性工作)
129. D. Gálvez-López and J. D. Tardós, "Bags of Binary Words for Fast Place Recognition in Image Sequences," IEEE T-RO, 2012.(DBoW2,ORB-SLAM 閉環偵測的基礎)
130. 高翔、張濤,《視覺 SLAM 十四講》第 7 講(視覺里程計:特徵點法)、第 8 講(直接法).(華文世界最清楚的特徵法與直接法教材,含完整程式碼)
131. OpenCV 官方文件之 Feature Detection and Description 教學(docs.opencv.org).(本章實作的主要工具,附完整範例)

小結

98. 視覺 SLAM 的標準流程為:預處理去畸變 → 視覺里程計(前端) → 後端最佳化 → 閉環偵測 → 建圖;其架構與第 18 章的光達 SLAM 高度一致,差別在於視覺特徵只有二維像素座標,深度需另外求得。
99. 特徵法先提取關鍵點與描述子再以幾何求解,對光線變化與動態物體穩健但在弱紋理處失效;直接法以像素亮度最小化光度誤差,能利用微弱梯度並產生稠密地圖,但依賴脆弱的灰度不變性假設。
100. FAST 只偵測、BRIEF 只描述、SIFT / SURF / ORB 兩者兼具;ORB 結合定向 FAST 與旋轉 BRIEF,以 256 位元二元描述子與漢明距離達成極高效率——比 SURF 快約 14 倍、SURF 又比 SIFT 快約 24 倍,因而成為 SLAM 的主流選擇。

101. 匹配比對描述子,適用於視角差異大與閉環偵測;追蹤在鄰近位置搜尋,適用於連續影像且更快。誤匹配必然發生,三種剔除策略為比值檢驗、交叉檢驗與 RANSAC 幾何驗證。
102. 單目 SLAM 有先天的尺度不確定性:場景與運動同時放大 k 倍,影像完全相同;三種解法為引入 IMU、使用雙目或 RGB-D、或利用已知尺寸的參考物。
103. 雙目以已知基線的視差測深,有絕對尺度且室內外皆可用,但怕無紋理且誤差隨距離平方成長;RGB-D 主動投射取得稠密深度,在白牆上仍有效,但怕強光與玻璃、範圍受限。
104. 視覺 SLAM 是 AR / VR 由內而外追蹤的技術本質,也廣泛用於服務機器人、無人機與低速自駕;詞袋模型(如 DBoW2 以二元單詞袋)是地點辨識與閉環偵測的關鍵技術,而深度學習特徵正逐步進入這個領域。

本章建立了視覺 SLAM 的整體框架與特徵基礎。但我們還沒回答最核心的問題:有了兩張影像之間的特徵對應,究竟如何算出相機移動了多少?下一章「對極幾何與相機運動估測」將補上這塊拼圖——本質矩陣與基礎矩陣描述了兩視圖之間的幾何約束、PnP 問題處理三維點到二維影像的對應、三角測量恢復三維座標、RANSAC 則讓這一切在有誤匹配的現實中依然可行。這是視覺 SLAM 最數學、也最優美的一章。

台灣自編大學教科書

@prof

《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第 20 章

對極幾何與相機運動估測

編者：李世淵 (機器人叫獸)

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

學習目標

上一章我們學會了如何在兩張影像間找出對應的特徵點,但還沒回答最核心的問題:有了這些對應,究竟如何算出相機移動了多少?本章補上這塊拼圖。這是視覺 SLAM 最數學、也最優美的一章——一旦理解對極幾何,你會發現兩張看似平凡的照片裡,竟藏著如此嚴謹的三維約束。學完本章之後,同學應能夠達成下列目標:

- 124. 說明從兩張影像推估相機運動的整體思路與適用條件。
- 125. 說明對極幾何的核心元素:基線、對極點、對極線與對極約束。
- 126. 區分本質矩陣與基礎矩陣的差異,並說明如何由它們恢復相機運動。
- 127. 說明 PnP 問題的定義、求解方法與它在 SLAM 中的角色。
- 128. 說明三角測量的原理與其精度受什麼因素影響。
- 129. 說明 RANSAC 的運作流程,並理解它為何能對抗大量誤匹配。
- 130. 把影像處理課程的知識銜接到視覺 SLAM 的幾何求解。

核心概念

中文名詞	英文名詞	概念說明
對極幾何	Epipolar Geometry	描述同一場景兩個視角之間內在幾何關係的理論。
基線	Baseline	連接兩個相機光心的直線。
對極點	Epipole	基線與各影像平面的交點。
對極線	Epipolar Line	一影像中的點,在另一影像中對應的一條直線。
對極約束	Epipolar Constraint	對應點必落在對極線上的幾何限制。

中文名詞	英文名詞	概念說明
本質矩陣	Essential Matrix, E	已知內參時描述兩視圖關係的 3×3 矩陣。
基礎矩陣	Fundamental Matrix, F	未知內參時描述兩視圖關係的 3×3 矩陣。
單應矩陣	Homography, H	描述平面場景或純旋轉下兩視圖關係的矩陣。
PnP 問題	Perspective-n-Point	由已知三維點與其影像投影求相機位姿。
三角測量	Triangulation	由兩個視角的對應點恢復三維座標。
視差角	Parallax Angle	同一點對兩相機光心的夾角,影響三角測量精度。
隨機取樣一致	RANSAC	以隨機取樣與投票剔除離群點的穩健估計方法。
內點與外點	Inlier / Outlier	符合與不符合估計模型的資料點。

20.1 兩張影像如何推估相機移動

先建立本章的整體影像。假設相機在位置 A 拍了一張照片,移動到位置 B 又拍了一張。第 19 章已經幫我們找出了兩張影像中的對應特徵點。現在的問題是:從這些對應點,能不能算出「相機從 A 到 B 的旋轉與平移」?

直覺:對應點的移動洩漏了運動

直覺上答案是肯定的。想像你站在教室前方拍一張照,往右走兩步再拍一張:畫面中所有物體都往左移了。若你是往前走,則畫面中的物體會向四周放射狀擴散(近的物體移動大、遠的移動小)。不同的運動,會在影像上產生不同的位移模式——這個模式就是我們要解讀的線索。

三種求解情境

視覺 SLAM 中,依「手上有什麼資訊」不同,會用到三種不同的求解方法。這個分類是本章的骨架,同學務必記牢:

情境	已知資訊	求解方法	本章節次
2D-2D	兩張影像的對應像素	對極幾何(本質 / 基礎矩陣)	20.2-20.3
3D-2D	已知三維點 + 其影像投影	PnP 問題	20.4
3D-3D	兩組對應的三維點	ICP(第 17 章)	第 17 章

表 20-1 三種相機運動估測情境

- 2D-2D:系統剛啟動時,還沒有任何三維地圖點,只能靠兩張影像的像素對應來初始化。這是單目 SLAM 的起手式。

- 3D-2D:一旦地圖建立起來,後續每一幀都能用「地圖中已知的三維點」與「當前影像看到的二維投影」求解位姿。這是 SLAM 運行期間最常用的方法,效率高且穩定。
- 3D-3D:當兩個視角都有深度資訊(雙目、RGB-D)時,可直接對兩組三維點做配準——這正是第 17 章的 ICP,同學已經學過。

本章聚焦前兩者。理解這個分類,同學就能看懂 ORB-SLAM(第 21 章)的原始碼在什麼時候呼叫哪一種求解器。

20.2 Epipolar Geometry

對極幾何是本章的理論核心。它回答一個看似簡單卻極深刻的問題:第一張影像中的某個點,在第二張影像中可能出現在哪裡?

搜尋範圍從二維降到一維

直覺上,答案似乎是「任何地方」——畢竟我們不知道那個點有多遠。但對極幾何告訴我們一個驚人的結論:它只可能出現在一條特定的直線上。

為什麼?考慮第一張影像中的一個像素 p_1 。它對應的三維點 P ,必定落在「從第一個相機光心穿過 p_1 的那條射線」上——只是不知道在射線上的哪個位置(這正是第 9 章的深度遺失)。現在把這整條射線投影到第二張影像上,會得到一條直線。無論 P 實際在射線上的哪個位置,它在第二張影像中的投影,必定落在這條直線上。

這條直線就是對極線(Epipolar Line),而這個約束就是對極約束(Epipolar Constraint)。它的實用價值極大:原本要在整張影像(二維)中搜尋對應點,現在只需沿著一條線(一維)搜尋。這既加快了搜尋,也大幅減少了誤匹配的機會。

對極幾何的四個元素

- 基線(Baseline):連接兩個相機光心 O_1 、 O_2 的直線。它是整個幾何的骨幹。
- 對極平面(Epipolar Plane):由兩個光心與空間點 P 三者決定的平面。不同的 P 會產生不同的對極平面,但它們都包含基線——所有對極平面繞著基線形成一束。
- 對極點(Epipole):基線與各影像平面的交點,記為 e_1 、 e_2 。幾何意義是「另一個相機的光心在本影像中的投影位置」。有趣的是:同一張影像中所有的對極線,都會通過該影像的對極點。
- 對極線(Epipolar Line):對極平面與影像平面的交線。 p_1 在第二張影像中的對極線記為 l_2 。

圖 20-1 對極幾何的結構

空間點 P

└—— 投影到影像 1 → 像素 p_1

└—— 投影到影像 2 → 像素 p_2

O_1 ——— 基線 ———→ O_2 (兩相機光心)

| |

└→ 對極點 e_1 └→ 對極點 e_2

對極平面 = O_1 、 O_2 、 P 三點決定的平面

對極線 l_2 = 對極平面與影像 2 的交線

【核心約束】 p_2 必定落在對極線 l_2 上

→ 搜尋範圍從二維(整張影像)降到一維(一條線)

一個生活化的驗證

同學可以親手驗證這個現象:閉上左眼用右眼看前方一枝筆,記住它在視野中的位置;再閉右眼用左眼看,筆會「跳」到另一個位置。若你把筆前後移動再重複,會發現它在視野中跳動的軌跡,始終沿著一條固定的直線——那就是對極線。人的雙眼,就是一組天然的雙目系統。

20.3 Essential Matrix 與 Fundamental Matrix

對極約束若只是文字描述,無法用來計算。本節把它寫成矩陣形式——這就是本質矩陣與基礎矩陣。

對極約束的矩陣表示

對極約束的數學形式極為簡潔優美。設 p_1 、 p_2 是兩張影像中的一組對應點(以齊次座標表示),則:

$$p_2^T \cdot F \cdot p_1 = 0 \quad (\text{基礎矩陣形式,使用像素座標})$$

$$x_2^T \cdot E \cdot x_1 = 0 \quad (\text{本質矩陣形式,使用正規化座標})$$

這兩條式子是本章最重要的公式。它們說:對應點與矩陣相乘的結果必須為零——換句話說,任何一組正確的對應點,都必須「滿足」這個約束。反過來,若我們蒐集足夠多的對應點,就能反解出這個矩陣。

兩者的差異

- 基礎矩陣 F : 作用於原始的像素座標, 不需要知道相機內參 K (第 9 章)。它同時混合了「相機的內參」與「兩相機的相對運動」兩種資訊。自由度為 7。
- 本質矩陣 E : 作用於正規化座標 (已用內參 K 把像素座標轉換過), 因此只包含「相對運動」的資訊, 更為純粹。自由度為 5 (三個旋轉、三個平移, 但平移只知方向不知大小, 故減一)。

兩者的關係為:

$$E = K_2^T \cdot F \cdot K_1$$

實務上, SLAM 系統的相機都經過校正 (第 9.2 節), 內參已知, 因此多直接使用本質矩陣。基礎矩陣則在內參未知的情況下使用 (例如處理網路上來源不明的照片)。

從 E 恢復相機運動

求出本質矩陣後, 可以透過奇異值分解 (SVD) 把它分解為旋轉 R 與平移 t 。但這裡有兩個重要的性質必須注意:

- 平移只知方向, 不知大小: E 的定義中, 把 t 乘上任意非零常數, 約束仍然成立。這正是第 19.5 節單目尺度不確定性在數學上的體現——對極幾何本身無法告訴我們「相機移動了幾公尺」, 只能告訴我們「往哪個方向移動」。
- 分解有四組解: 數學上, 分解 E 會得到四組可能的 (R, t) 組合。要從中挑出正確的那一組, 需要用一個物理判準: 把對應點三角測量出三維座標後, 只有正確的那組解會讓所有點都落在兩個相機的前方 (深度為正)。這稱為正深度檢驗 (Cheirality Check)。

特殊情況: 單應矩陣

對極幾何有一個重要的例外情形。當場景中所有點都落在同一個平面上 (例如相機對著一面牆、一張桌面、地板), 或者相機只做純旋轉沒有平移時, 對極約束會發生退化——本質矩陣的求解變得不穩定甚至無解。

此時應改用單應矩陣(Homography, H),它直接描述「一張影像的點如何映射到另一張影像」:

$$p_2 = H \cdot p_1$$

ORB-SLAM 在初始化時的一個聰明設計,就是同時計算基礎矩陣與單應矩陣,依據兩者的擬合分數自動判斷「這個場景比較像平面還是立體」,再選用合適的模型。這個細節在第 21 章會再提到。

20.4 PnP 問題

對極幾何處理的是「兩張影像」的情形。但 SLAM 運行起來之後,我們手上已經有一張三維地圖了——此時求位姿有更好的方法。

問題定義

PnP(Perspective-n-Point,透視 n 點)問題的定義是:已知 n 個三維空間點的座標,以及它們在影像上的二維投影位置,求相機的位姿(旋轉 R 與平移 t)。

同學應該覺得眼熟——第 4.6 節介紹 AprilTag 時就用過這個概念:標記的四個角點三維座標已知(正方形邊長已知)、它們在影像上的像素座標也偵測得到,於是能解出相機相對標記的完整 6-DoF 位姿。那正是一個 $n = 4$ 的 PnP 問題。

為什麼 PnP 比對極幾何好用

- 有絕對尺度:三維點的座標是有單位的(公尺),因此解出的平移也有真實大小——不像對極幾何只能給方向。
- 不需退化判斷:不會有平面場景退化的問題。
- 所需點數少:最少三個點就有解(P3P),遠少於對極幾何所需的點數。

- 效率高:在 SLAM 中,每一幀只需把「地圖中已知的點」投影比對即可,是追蹤階段的主力方法。

常見求解方法

- P3P:用三個點求解,會得到最多四組解,需第四個點來排除。是許多方法的基礎。
- EPNP:一種高效的線性解法,能處理任意數量的點,速度快且精度良好,是 OpenCV 與許多 SLAM 系統的預設選擇。
- 直接線性變換(DLT):把問題轉為線性方程組求解,概念簡單但精度較差,常作為初值。
- 非線性最佳化(BA):以上述方法求得初值後,再最小化重投影誤差(第 12 章)進行精修。這是實務上必做的最後一步。

重投影誤差:PnP 的最佳化目標

PnP 的精確解法,本質上就是最小化重投影誤差——第 12.5 節 Bundle Adjustment 的核心概念:

198. 用當前估計的位姿,把每個三維點投影到影像上,得到「預測的像素位置」。
199. 與「實際偵測到的像素位置」比較,兩者的差就是重投影誤差。
200. 調整位姿,使所有點的重投影誤差平方和最小。

同學可以看出:PnP 其實是「只最佳化相機位姿、固定三維點」的簡化版 BA。而完整的 BA 則同時最佳化位姿與三維點。兩者是同一套數學的不同應用範圍——這也是第 12 章數學工具的又一次現身。

20.5 三角測量

前面兩節都是「由對應點求相機運動」。本節是反過來:已知相機運動,由對應點求

三維座標。這就是三角測量(Triangulation),它讓單目 SLAM 能夠建立地圖。

原理:兩條射線的交點

原理極其直觀。已知相機在兩個位置的位姿後:

201. 從第一個相機光心,穿過像素 p_1 ,畫一條射線。三維點 P 必在這條射線上。
202. 從第二個相機光心,穿過像素 p_2 ,再畫一條射線。 P 也必在這條射線上。
203. 兩條射線的交點,就是 P 的三維位置。

理論上完美,實務上卻有個問題:由於像素座標有量測誤差,兩條射線通常不會精確相交,而是「擦身而過」。因此實際做法是求「離兩條射線都最近的點」,或以最小化重投影誤差的方式求解。

精度的關鍵:視差角

這是本節最重要的觀念。三角測量的精度,取決於兩條射線的夾角——稱為視差角(Parallax Angle)。

- 視差角大(相機移動距離相對於物體距離較大):兩條射線接近垂直相交,交點位置明確,深度估計精準。
- 視差角小(相機幾乎沒動,或物體非常遠):兩條射線近乎平行,像素的微小誤差就會讓交點沿射線方向大幅滑動,深度極不準確。

這解釋了幾個重要的實務現象:

204. 遠處的點測不準:同樣的相機移動,遠處物體的視差角小得多,因此天空、遠山的深度幾乎無法估計。這與第 9.6 節「雙目深度誤差隨距離平方成長」是同一件事。

205. 純旋轉無法建圖:若相機只轉頭不移動,兩個光心重合,視差角為零,完全無法三角測量。這是單目 SLAM 初始化失敗的常見原因——同學在第 29 章實作時,若一直初始化不成功,多半是「只轉不移」。

206. 需要足夠的基線:因此 SLAM 系統會等到相機移動足夠距離(產生足夠視差)後,才三角測量新的地圖點。這是關鍵幀選擇策略的考量之一(第 21 章)。

矛盾與解法

這裡存在一個微妙的矛盾:視差角要大,就需要相機移動較遠;但移動越遠,兩張影像的外觀差異越大,特徵匹配就越困難(第 19 章)。

實務系統的解法是:先用小基線的相鄰幀做穩定追蹤,再用相隔較遠的關鍵幀做三角測量。這種「近幀追蹤、遠幀建圖」的策略,正是 ORB-SLAM 區分「普通幀」與「關鍵幀」的原因之一。

20.6 RANSAC 排除錯誤匹配

前面幾節的數學都很優美,但它們有一個共同的前提:輸入的對應點是正確的。而第 19.4 節告訴我們,誤匹配必然存在。本節介紹對抗誤匹配的關鍵武器。

為什麼誤匹配如此致命

傳統的最小平方法(第 12 章)會把所有資料點的誤差平方和最小化。問題在於:平方會放大的誤差。一個嚴重的誤匹配,可能產生數十甚至上百像素的誤差,平方之後主導整個目標函數,把估計結果硬生生拉偏。

打個比方:全班九十九位同學的身高都在 160 至 180 公分,但資料中誤植了一筆「1800 公分」。用平均值來估計班級身高,結果會被這一筆離譜的資料嚴重扭曲。最小平方法面對誤匹配,就是這個處境。

RANSAC 的核心思想

RANSAC(RANdom SAmple Consensus,隨機取樣一致)採用完全不同的策略:與其讓所有資料都參與,不如「隨機挑幾個、看看有多少人同意」。

207. 隨機取樣:從所有對應點中,隨機挑出剛好足夠求解模型的最少數量(例如求本質矩陣需要 5 或 8 點,求 PnP 需要 3 或 4 點)。
208. 求解模型:用這幾個點算出一個候選的相機運動(或矩陣)。
209. 投票統計:用這個候選模型檢驗「所有」對應點,計算每個點的誤差。誤差小於門檻的稱為內點(Inlier),記錄內點數量。
210. 反覆迭代:回到步驟 1,重複數十到數百次。
211. 選出最佳:保留內點數最多的那個模型,並用它的所有內點重新做一次精確的最小平方求解。

為什麼這樣就有效

關鍵洞見在於:如果隨機挑的幾個點「碰巧全是正確的匹配」,那麼算出的模型就是正確的,而其他正確的匹配也都會同意它(成為內點),因此內點數會很高。反之,若挑到了誤匹配,算出的模型是錯的,幾乎沒有其他點會同意,內點數很低。

於是「內點數最多」就成為判斷模型正確與否的有效準則。RANSAC 不試圖修正誤匹配,而是直接把它們排除在外——這種「多數決」的精神,讓它能容忍極高比例的離群點(有時超過一半)。

實務要點

- 迭代次數:需要多少次才能有高機率至少抽中一次「全是內點」的樣本?這可由機率公式估算(見公式或流程)。內點比例越低、所需樣本點數越多,就需要越多迭代。

- 門檻設定:誤差門檻決定了「多接近才算內點」。太嚴則正確的匹配也被排除、太鬆則誤匹配混入。通常依像素雜訊水準設定,例如 1 至 3 像素。
- 與穩健核函數的關係:RANSAC 是「先剔除再最佳化」的硬性做法;第 12、15 章提過的穩健核函數則是「降低離群點權重」的軟性做法。兩者常搭配使用——先用 RANSAC 剔除明顯的誤匹配,再在最佳化中用穩健核函數處理殘餘的離群值。
- 通用性:RANSAC 不限於視覺。第 17 章的點雲配準、第 13 章的平面擬合、甚至任何「有離群點的模型擬合」問題都能用它。它是工程師工具箱中最實用的方法之一。

20.7 從影像處理課程銜接到視覺 SLAM

本章的數學密度較高,本節退一步,幫同學把整條知識脈絡連起來,並說明如何從已學過的影像處理課程,一步步走到視覺 SLAM。

知識的接續

階段	核心問題	關鍵技術	本書章節
影像處理	如何讓影像更清楚	濾波、邊緣偵測、形態學	先備知識
相機模型	三維如何變成二維	針孔模型、內參、畸變校正	第 9 章
特徵工程	如何找出可重複辨識的點	ORB、描述子、匹配	第 19 章
多視圖幾何	兩張影像藏著什麼幾何關係	對極約束、E / F、PnP、三角測量	本章
視覺里程計	如何連續估計相機軌跡	逐幀求解與累加	第 19、21 章
視覺 SLAM	如何消除漂移、建一致地圖	BA、閉環、關鍵幀	第 21、22 章

表 20-2 從影像處理到視覺 SLAM 的知識階梯

三個關鍵的觀念轉折

212. 從「處理影像」到「理解幾何」:影像處理把影像當作二維訊號來處理(濾波、增強);視覺 SLAM 則把影像視為「三維世界的投影」,關心的是背後的幾何結構。這是最根本的視角轉變。
213. 從「單張影像」到「多視圖關係」:本章的對極幾何是這個轉折的核心——單張影像遺失了深度,但兩張影像之間的關係,足以恢復運動與結構。
214. 從「精確假設」到「穩健估計」:影像處理的演算法多假設輸入是乾淨的;真實的 SLAM 必須面對誤匹配、雜訊與動態物體,因此 RANSAC 與穩健最佳化成為必備工具。這也呼應第 11 章「以機率處理不確定性」的精神。

學習建議

- 先建立幾何直覺,再看公式:對極幾何的公式很簡潔,但直覺更重要。務必動手做本章的實作,把對極線畫出來看。
- 善用視覺化:OpenCV 能畫出對極線、內外點、重投影結果。看到圖,抽象的矩陣就有了意義。
- 與光達 SLAM 對照:回頭比較第 17 章的 ICP(3D–3D)與本章的 PnP(3D–2D)、對極幾何(2D–2D),你會發現它們是同一個問題在不同資訊條件下的變形。
- 不必手推所有公式:本質矩陣的 SVD 分解、EPnP 的推導等細節,OpenCV 都已實作。理解「它在解什麼、什麼時候用、會失敗在哪」比背推導更重要。有志研究者可再深入 Hartley 與 Zisserman 的經典教科書。

至此,視覺 SLAM 的數學基礎已備妥。下一章我們把這些零件組裝成一個完整的系統——ORB-SLAM2。它如何用三個執行緒協同工作?關鍵幀與地圖點如何管理?共視

圖是什麼?閉環如何偵測與修正?一個真正能跑的視覺 SLAM 系統,即將完整呈現在同學眼前。

公式或流程

一、對極約束

設 p_1 、 p_2 為兩張影像中的一組對應點(齊次像素座標), K_1 、 K_2 為相機內參(第 9 章),則對極約束為:

$$p_2^T \cdot F \cdot p_1 = 0$$

把像素座標用內參轉為正規化座標 $x = K^{-1} \cdot p$,則約束改寫為本質矩陣形式:

$$x_2^T \cdot E \cdot x_1 = 0$$

兩矩陣的關係與本質矩陣的組成為:

$$E = K_2^T \cdot F \cdot K_1, \quad E = [t]_x \cdot R$$

其中 $[t]_x$ 是平移向量的反對稱矩陣(第 10 章)、 R 是旋轉矩陣。這條式子清楚顯示:本質矩陣正是「平移與旋轉」的緊湊編碼。由 E 分解出 R 與 t 即得相機運動,但 t 只有方向沒有大小(尺度不確定性)。

二、對極線的計算

第一張影像中的點 p_1 ,在第二張影像中的對極線為:

$$l_2 = F \cdot p_1 \quad (\text{直線方程式 } a \cdot u + b \cdot v + c = 0 \text{ 的係數})$$

反之, p_2 在第一張影像中的對極線為 $l_1 = F^T \cdot p_2$ 。實作時把 l_2 畫在第二張影像上,就能親眼看見「對應點必落在這條線上」的約束——這是本章實作最有啟發性的一步。

三、重投影誤差(PnP 與 BA 的目標)

設三維點 P 、相機位姿 (R, t) 、內參 K 、實際觀測到的像素 u_{obs} , 則重投影誤差為:

$$e = u_{obs} - \pi(K, R, t, P)$$

PnP 求解即最小化所有點的重投影誤差平方和:

$$(R^*, t^*) = \operatorname{argmin} \sum_i \|u_{obs,i} - \pi(K, R, t, P_i)\|^2$$

注意:PnP 只最佳化 (R, t) , 三維點固定; 完整的 Bundle Adjustment(第 12.5 節)則同時最佳化位姿與三維點。兩者是同一數學框架的不同應用。

四、RANSAC 的迭代次數

設內點比例為 w 、每次取樣需要 s 個點、希望以機率 p 至少抽中一次全是內點的樣本, 則所需迭代次數為:

$$N = \log(1 - p) / \log(1 - w^s)$$

舉例: 內點比例 $w = 0.5$ 、求本質矩陣需 $s = 8$ 點、希望成功機率 $p = 0.99$, 則 $N \approx 1177$ 次。若改用只需 $s = 5$ 點的方法, N 降至約 145 次——這說明了為何「所需點數少的求解器」在 RANSAC 中如此珍貴, 也是五點法比八點法更受青睞的原因。

五、視覺前端的完整求解流程

圖 20-2 從特徵對應到相機位姿的完整流程

① 特徵匹配(第 19 章) → 得到一組可能含誤匹配的對應點

↓

② 判斷情境:

- 系統初始化, 無地圖點 → 走 2D-2D(對極幾何)
- 已有地圖點 → 走 3D-2D(PnP)

↓

③ RANSAC 迭代:隨機取樣 → 求解模型 → 統計內點

↓

④ 取內點數最多的模型,剔除外點

↓

⑤ 用全部內點做非線性最佳化(最小化重投影誤差)精修位姿

↓

⑥ 三角測量:對視差角足夠的新對應點,恢復三維座標並加入地圖

↓

⑦ 輸出相機位姿 + 新增地圖點 → 交給後端(第 21 章)

案例

案例 20-1 為什麼站著原地轉圈,單目 SLAM 就掛了

這是同學實作單目視覺 SLAM 時最常遇到的挫折。拿著手機或相機,系統一直顯示「初始化中」,怎麼樣都成功不了。檢查了相機校正、特徵數量、光線,全都正常——問題到底出在哪?

答案往往極其簡單:你只在原地轉,沒有走動。

依 20.5 節的原理:純旋轉時,兩次拍攝的光心位置完全重合,基線長度為零,視差角為零。此時從兩個光心穿過對應像素的射線完全重疊,根本沒有交點可言——三角測量在數學上無解,系統當然無法建立初始地圖。

更微妙的是,純旋轉時對極幾何也會退化:平移向量 $\mathbf{t}$ 為零,本質矩陣 $\mathbf{E} = [\mathbf{t}]_x \cdot \mathbf{R}$ 也變

成零矩陣,求解自然失敗。此時應該用的是單應矩陣(20.3 節)——這也正是 ORB-SLAM 初始化時要同時計算兩種模型並自動選擇的原因。

解法很簡單:平移移動。拿著相機小幅度左右或前後平移數十公分,同時保持特徵在視野中,通常幾秒內就能成功初始化。這個案例呼應第 7.7 節「IMU 初始化需要運動激勵」——兩者的共同教訓是:視覺與慣性系統都需要「有內容的運動」才能觀測到某些量。

案例 20-2 對極線的視覺化:一眼看懂三維約束

對極幾何的公式看起來抽象,但把它畫出來的瞬間,一切都清晰了。這是大學課堂上最受歡迎的示範之一。

做法是:對同一場景拍兩張視角略不同的照片,用 ORB 找出對應點,以 RANSAC 估計基礎矩陣 F ,然後對第一張影像中的每個特徵點,用公式二算出它在第二張影像中的對極線並畫出來。

結果會呈現一組壯觀的畫面:第二張影像上出現許多條直線,而每一個對應的特徵點,都精準地落在它自己那條線上。更神奇的是,若相機是往前移動,這些線會全部匯聚到畫面中的某一點——那就是對極點,也就是「另一個相機光心的投影位置」,幾何上正對應著你行進的方向。

這個實驗的教學價值極高:它讓同學親眼看到「兩張平凡的照片裡,竟然藏著如此嚴謹的三維約束」。抽象的矩陣 F ,忽然變成了看得見的線。強烈建議教師把這個示範排進課堂,並讓同學自己拍照實作——本章的實作活動就以此為核心。

案例 20-3 RANSAC 的普世價值:超越視覺的方法論

RANSAC 是 1981 年由 Fischler 與 Bolles 提出的方法,原本就是為了電腦視覺中的

位姿估計問題(有趣的是,他們提出時處理的正是 PnP 類問題)。但四十餘年來,它的影響力遠遠超出了視覺領域。

它被用在:點雲的平面與圓柱擬合(第 17 章)、光達的地面分離(第 18 章的 LeGO-LOAM)、機器人的感測器校正、生物資訊的序列比對、金融資料的趨勢擬合——任何「資料中混有離群點、需要找出主流模型」的場合。

RANSAC 之所以如此通用,是因為它體現了一個深刻的方法論:當你無法信任每一筆資料時,不要試圖用全部資料求平均,而要尋找「最多資料同意的解釋」。這是一種從「相信資料」轉向「相信共識」的思維轉變。

對同學的啟發是:學習演算法時,除了記住它怎麼做,更要理解它背後的思維模式。RANSAC 的「隨機取樣 + 多數決」、卡爾曼濾波的「按可信度加權」(第 11 章)、圖優化的「把矛盾均攤」(第 12 章)——這些思維框架,遠比具體的公式更能長久受用。

實作活動

實作 20-1 對極幾何與相機運動估測

目的:親手計算基礎矩陣、把對極線畫出來、執行 PnP 與三角測量,並見證 RANSAC 剔除誤匹配的威力。本實作以 Python 搭配 OpenCV,只需一支手機拍照即可完成,是全書最能建立幾何直覺的實驗之一。

第一部分:對極線視覺化(全體,本章核心)

176. 拍攝素材:對教室或走廊拍兩張照片,第二張相對第一張向右平移約 30 公分(注意是平移,不是原地轉)。確保畫面有豐富紋理。
177. 用 ORB 提取特徵並匹配(第 19 章的成果),套用比值檢驗初步剔除誤匹配。

178. 用 OpenCV 的 findFundamentalMat 搭配 RANSAC 估計基礎矩陣 F ,同時取得內外點的標記。
179. 依公式二,對第一張影像中的內點,計算它們在第二張影像中的對極線並畫出來;同時把對應的特徵點也畫上。
180. 觀察與驗證:每個對應點是否都落在自己的對極線上?誤差有多大(以像素計)?所有對極線是否匯聚到同一點(對極點)?
181. 對照實驗:改拍一組「原地旋轉」的照片,重複上述流程,觀察估計是否變得不穩定——親身驗證案例 20-1 的退化現象。

第二部分:RANSAC 的威力(全體)

182. 不用 RANSAC(改用最小平方法)估計 F ,把對極線畫出來,觀察是否明顯偏移。
183. 比較內點數:RANSAC 判定的內點佔全部匹配的百分之幾?被剔除的外點在影像上是什麼樣的匹配(把它們單獨畫出來看)?
184. 人為加入誤匹配:故意在匹配清單中加入若干隨機的錯誤對應,重跑 RANSAC,觀察它是否仍能正確剔除。加到多少比例才會失敗?
185. 依公式四計算:在你的內點比例下,理論上需要多少次迭代才能有 99% 的成功機率?與 OpenCV 的預設值比較。

第三部分:PnP 與三角測量(大學部)

186. PnP 實作:用第 9 章校正好的相機拍攝一張含棋盤格(或 ArUco 標記)的照片,已知格點的三維座標,用 solvePnP 求相機位姿,並把座標軸投影回影像驗證。
187. 三角測量實作:用第一部分求得的相機運動,對匹配的內點做三角測量(OpenCV 的 triangulatePoints),把恢復的三維點用 matplotlib 或 Open3D 顯示。

188. 視差角實驗:分別用「平移 10 公分」與「平移 50 公分」的兩組照片做三角測量,比較恢復的三維點雲品質(散布程度、深度合理性),驗證 20.5 節視差角對精度的影響。

189. 尺度驗證:量測場景中某兩點的真實距離,與三角測量結果比較——你會發現比例對但絕對值不對,親身體驗單目的尺度不確定性(第 19.5 節)。

討論問題

94. 對極線視覺化的結果中,對應點偏離對極線多少像素?這個誤差來自哪裡(特徵定位誤差?相機校正誤差?)

95. 原地旋轉的那組照片,基礎矩陣的估計為何不穩定?依 20.3 節,此時應該改用什麼模型?

96. RANSAC 剔除的外點,在影像上呈現什麼特徵(是隨機分布,還是集中在某些區域如重複紋理處)?

97. 平移 10 公分與 50 公分的三角測量結果差多少?這對「SLAM 系統何時該三角測量新地圖點」有什麼啟示?

98. (大學部)三角測量恢復的三維點,比例正確但絕對值不對。若要修正,依第 19.5 節你有哪三種選擇?各自的成本如何?

延伸挑戰(大學部)

其一:把本章的零件串成一個完整的兩幀視覺里程計——特徵匹配 → RANSAC 估計本質矩陣 → 分解出 R 與 t → 正深度檢驗選出正確解 → 三角測量建立初始地圖點。這正是 ORB-SLAM 初始化模組的簡化版,完成後你會對第 21 章的系統架構有極深刻的理解。

其二:實作 ORB-SLAM 的模型選擇策略——同時計算基礎矩陣與單應矩陣,依擬合分數自動判斷場景是平面還是立體,並用合適的模型初始化。用一面白牆與一個立體場景分別測試,驗證選擇是否正確。

延伸閱讀

132. R. Hartley and A. Zisserman, Multiple View Geometry in Computer Vision, 2nd ed., Cambridge University Press, 2004.(多視圖幾何的權威聖經, 本章所有主題的完整推導來源)
133. M. A. Fischler and R. C. Bolles, "Random Sample Consensus," Communications of the ACM, 1981.(RANSAC 原始論文,提出時即用於位姿估計問題)
134. D. Nistér, "An Efficient Solution to the Five-Point Relative Pose Problem," IEEE TPAMI, 2004.(五點法求本質矩陣,大幅降低 RANSAC 所需迭代次數)
135. V. Lepetit, F. Moreno-Noguer, and P. Fua, "EPnP: An Accurate $O(n)$ Solution to the PnP Problem," IJCV, 2009.(EPnP 原始論文,OpenCV 的預設 PnP 解法)
136. 高翔、張濤,《視覺 SLAM 十四講》第 7 講(視覺里程計:特徵點法).(對極幾何、PnP、三角測量的華文最佳教材,含完整程式碼)
137. OpenCV 官方文件之 Camera Calibration and 3D Reconstruction 模組 (docs.opencv.org).(findFundamentalMat、solvePnP、triangulatePoints 等函式的使用說明)
138. UZH Vision Algorithms for Mobile Robotics 課程之 Multiple View Geometry 單元(第 2 章介紹).(免費公開的課程影片與講義,可與本章對照學習)

小結

105. 相機運動估測依已知資訊分三種情境:2D–2D 用對極幾何(系統初始化)、3D–2D 用 PnP(SLAM 運行期間的主力)、3D–3D 用 ICP(第 17 章)。理解這個分類,就能看懂 SLAM 系統何時呼叫哪一種求解器。
106. 對極幾何的核心是對極約束:第一張影像中的一個點,在第二張影像中必落在一條對極線上,使搜尋範圍從二維降到一維。四個元素為基線、對極平面、對極點與對極線,且同一影像中所有對極線都通過對極點。
107. 對極約束的矩陣形式為 $p_2^T \cdot F \cdot p_1 = 0$ 與 $x_2^T \cdot E \cdot x_1 = 0$;基礎矩陣用像素座標且不需內參,本質矩陣用正規化座標且更純粹。由 $E = [t]_x \cdot R$ 分解可得運動,但平移只知方向(尺度不確定),且有四組解需以正深度檢驗篩選。平面場景或純旋轉時應改用單應矩陣。
108. PnP 由已知三維點與其影像投影求相機位姿,優點是有絕對尺度、不會退化、所需點數少、效率高;常用解法包含 P3P、EPnP、DLT,並以最小化重投影誤差的非線性最佳化精修——它本質上是固定三維點的簡化版 Bundle Adjustment。
109. 三角測量由兩條射線的交點恢復三維座標,精度取決於視差角:視差角小(相機幾乎沒動或物體很遠)時深度極不準;純旋轉時視差角為零,完全無法三角測量——這是單目 SLAM 初始化失敗最常見的原因。
110. RANSAC 以「隨機取樣、求解模型、統計內點、反覆迭代、取內點最多者」對抗誤匹配;它不修正離群點而是直接排除,能容忍極高的離群比例。所需迭代次數由內點比例與取樣點數決定,這也是五點法優於八點法的原因。
111. 從影像處理到視覺 SLAM 有三個觀念轉折:從「處理影像」到「理解幾何」、從「單張影像」到「多視圖關係」、從「精確假設」到「穩健估計」。理解演算法在解什麼、何時用、會失敗在哪,比背推導更重要。

本章備齊了視覺 SLAM 的幾何工具。但這些都還是零件——特徵、對極幾何、PnP、三角測量、RANSAC,如何組裝成一個能長時間穩定運行、能建立一致地圖、能從跟丟中恢復的完整系統?下一章我們剖析 ORB-SLAM2:它如何以三個執行緒協同工作、關鍵幀與地圖點如何管理、共視圖如何組織空間關係、閉環如何偵測與修正。一個真正能跑的視覺 SLAM 系統,即將完整呈現。

台灣自編大學教科書

@prof

《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第 21 章

ORB-SLAM2 理論與實作

編者：李世淵 (機器人叫獸)

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

學習目標

第 19、20 章我們備齊了視覺 SLAM 的所有零件:特徵、對極幾何、PnP、三角測量、RANSAC。但零件不等於系統——如何讓它們協同運作、長時間穩定、跟丟能恢復、閉環能修正?本章剖析 ORB-SLAM2 這個公認最成熟的開源視覺 SLAM 系統,把理論組裝成真正能跑的架構。學完本章之後,同學應能夠達成下列目標:

131. 說明 ORB 特徵在 ORB-SLAM2 中的角色,以及「同一套特徵貫穿三大任務」的設計優點。
132. 說明單目、雙目、RGB-D 三種模式的初始化差異。
133. 說明追蹤執行緒的工作流程與三種追蹤策略。
134. 說明局部建圖執行緒如何新增地圖點、執行局部 BA 與剔除冗餘。
135. 說明閉環執行緒如何偵測閉環、驗證相似變換並修正全域漂移。
136. 說明關鍵幀、地圖點、共視圖與生成樹的資料結構與作用。
137. 規劃並執行 ORB-SLAM2 在公開資料集上的實作與評測。

核心概念

中文名詞	英文名詞	概念說明
關鍵幀	KeyFrame	被保留下來代表某個視角的影像幀,含位姿與特徵。
地圖點	MapPoint	地圖中的三維特徵點,記錄座標、描述子與觀測資訊。
共視圖	Covisibility Graph	連接觀測到共同地圖點的關鍵幀所形成的圖。
生成樹	Spanning Tree	連接所有關鍵幀的最小生成樹,結構輕量。

中文名詞	英文名詞	概念說明
本質圖	Essential Graph	共視強度高的邊加上生成樹與閉環邊,用於位姿圖優化。
追蹤	Tracking	對每一幀估計相機位姿的前端執行緒。
局部建圖	Local Mapping	管理與最佳化局部地圖的執行緒。
閉環	Loop Closing	偵測重訪並修正全域漂移的執行緒。
僅運動 BA	Motion-only BA	固定地圖點、只最佳化相機位姿的束調整。
局部 BA	Local BA	最佳化局部關鍵幀與其所見地圖點的束調整。
全域 BA	Full BA	最佳化所有關鍵幀與地圖點的束調整。
地點辨識	Place Recognition	以詞袋比對判斷是否曾經來過的模組。
重定位	Relocalization	追蹤失敗後在地圖中重新找回位置。
相似變換	Similarity Transform	含尺度的變換,單目閉環修正時使用。

21.1 ORB 特徵提取

ORB-SLAM2 由西班牙薩拉戈薩大學的 Mur-Artal 與 Tardós 開發,是視覺 SLAM 領域最具影響力的開源系統。它的名字直接點出核心:整套系統建立在 ORB 特徵之上。

為什麼選 ORB:三個理由

第 19 章已經談過 ORB 的速度優勢,但在 ORB-SLAM2 中,選擇 ORB 還有更深的系統層面考量。這些特徵對旋轉與尺度具有穩健性,對相機自動增益、自動曝光與光線變化也有良好的不變性;此外它們提取與匹配快速,能支援即時運作,並在詞袋地點辨識中展現良好的精確率與召回率表現。

- 速度:即時系統的硬性需求(第 19.3 節)。
- 對曝光與光線變化的不變性:這一點常被忽略卻極為關鍵——相機的自動增益與自動曝光會不斷改變影像亮度,若特徵不具此種不變性,系統會頻繁跟丟。這也正是直接法(第 19.2 節)的軟肋。
- 適合詞袋:二元描述子能高效地建立詞彙樹,使地點辨識又快又準。

關鍵設計:一套特徵、三種用途

這是 ORB-SLAM2 架構上最優雅的決定之一。系統對追蹤、建圖與地點辨識三項任務,使用的是同一套 ORB 特徵。

這個設計的好處是巨大的:

215. 省下重複計算:影像只需提取一次特徵,三個模組共用。若追蹤用 ORB、閉環用 SIFT,則每幀要算兩套特徵,運算量倍增。
216. 資料結構統一:地圖點只需儲存一種描述子,關鍵幀只需一種詞袋表示。
217. 行為一致:追蹤能匹配上的特徵,閉環也能匹配上,不會出現「追蹤看得見但閉環認不出」的矛盾。

實作細節:金字塔與均勻分布

- 影像金字塔:為了獲得尺度不變性,ORB-SLAM2 把影像縮放成多層金字塔,在每一層都提取 FAST 角點。這讓系統能匹配遠近不同的相同物體。
- 均勻分布:直接提取 FAST 角點會造成特徵集中在紋理豐富處(如一幅畫上密集出現),其他區域則空無一物。ORB-SLAM2 用四元樹把影像遞迴切分,確保特徵均勻散布全畫面——這與第 18.2 節 LOAM 分區選取特徵的用意完全相同(避免幾何條件退化)。

21.2 單目、雙目、RGB-D 初始化

ORB-SLAM2 的一大特色是同一套系統支援三種相機。但三者的初始化差異極大,這是理解系統的關鍵之一。

單目:最困難的初始化

單目沒有深度資訊,必須靠兩張影像的視差來建立初始地圖(第 20 章)。而如同第 20 章案例 20-1 所述,若場景是平面或相機純旋轉,對極幾何會退化。

ORB-SLAM2 的解法極為聰明:它平行計算兩個幾何模型——針對平面場景的單應矩陣,以及針對非平面場景的基礎矩陣——然後依據兩者的相對分數選擇其中之一。

- 若單應矩陣的分數明顯較高,判定場景為平面,以單應矩陣分解出運動。
- 否則以基礎矩陣(本質矩陣)求解。
- 若兩者都不夠好(例如視差不足),則放棄本次初始化,等待下一幀重試。

這正是同學實作時「一直初始化不成功」的機制原因:系統其實在保護你,不讓它用不可靠的模型建出錯誤的地圖。解法一如第 20 章所述——平移移動,製造足夠視差。

雙目與 RGB-D:立刻就能開始

雙目與 RGB-D 相機的初始化簡單得多:第一幀就有深度資訊,可以直接把有深度的特徵點變成地圖點,建立初始地圖,系統立即進入正常追蹤。

- 雙目:由左右影像視差計算深度(第 9.6 節)。
- RGB-D:直接讀取深度圖。
- 共同優點:有絕對尺度、不需等待視差累積、不會因純旋轉而失敗。

近點與遠點的區分

ORB-SLAM2 在雙目與 RGB-D 模式中有一個重要的設計:依深度把特徵點分為近點與遠點。近點(深度小於某個倍數基線)的深度量測可靠,可直接三角化為地圖點;遠點的深度不可靠(第 9.6 節深度誤差隨距離平方成長),則像單目一樣,等到多視角觀測累積足夠視差後再三角化。

這個設計展現了工程上的務實:不是所有量測都同等可信,系統應該依可信度分別處理——這與第 11 章卡爾曼增益「按可信度加權」的精神一脈相承。

21.3 Tracking 執行緒

ORB-SLAM2 以三個平行執行緒運作。系統有三個主要的平行執行緒:追蹤,對每一幀透過與局部地圖的特徵匹配並套用僅運動 BA 最小化重投影誤差來定位相機;局部建圖,管理並最佳化局部地圖,執行局部 BA;閉環,偵測大迴路並以位姿圖優化修正累積漂移。本節先看第一個。

追蹤的任務

追蹤執行緒是前端,它的唯一任務是:對每一張進來的影像,盡快算出「相機現在在哪裡」。它必須即時(與相機幀率同步),因此運算必須輕量。

三種追蹤策略

ORB-SLAM2 依情況採用三種策略,由易到難依序嘗試:

218. 依運動模型追蹤:假設相機以等速運動,用上一幀的速度預測當前位姿,再與上一幀的地圖點匹配並精修。最快,也是正常情況下的主要路徑。
219. 依參考關鍵幀追蹤:若運動模型失敗(例如相機突然改變運動),改用詞袋加速與最近的參考關鍵幀做特徵匹配,再求解位姿。
220. 重定位:若前兩者都失敗(追蹤丟失),啟動地點辨識模組,在整個地圖的關鍵幀資料庫中搜尋相似畫面,以 PnP 加 RANSAC 求解位姿,把自己「找回來」。

追蹤局部地圖

求出初步位姿後,追蹤執行緒還會做一件關鍵的事:把「局部地圖」中的更多地圖點投影到當前幀,尋找額外的匹配,再做一次最佳化。這讓位姿估計使用更多觀測,精度顯著提升。

局部地圖的取得,靠的是共視圖(21.6 節):找出與當前幀共視程度高的關鍵幀,以及它們所見的地圖點。這些圖結構讓系統能取得關鍵幀的局部視窗,使追蹤與局部建圖能在局部範圍內運作,因而能在大型環境中工作。

關鍵幀的選擇

追蹤執行緒的最後一項工作,是決定「這一幀該不該成為關鍵幀」。判準大致包括:距離上次插入關鍵幀已過足夠幀數、局部建圖執行緒處於閒置、當前幀追蹤到的地圖點數量足夠但與參考關鍵幀的共視比例已下降(表示視角有明顯變化)。

ORB-SLAM2 的策略是「寬鬆插入、嚴格剔除»:先盡量插入關鍵幀以保證追蹤穩健,再由局部建圖執行緒剔除冗餘者(21.4 節)。這個設計讓系統在快速運動時不易跟丟,同時避免關鍵幀無限膨脹。

21.4 Local Mapping 執行緒

局部建圖執行緒是中端,它不需即時,但負責維護地圖的品質。局部建圖執行緒把新的點插入局部地圖,並透過執行局部束調整來最佳化局部地圖。

五個步驟

221. 插入關鍵幀:新的關鍵幀被插入共視圖、連接關鍵幀與擁有最多共同點的關鍵幀的生成樹,並建立該關鍵幀的詞袋表示(用於三角化新點時的資料關聯)。
222. 剔除劣質地圖點:新建立的地圖點必須通過嚴格檢驗才能留下——必須被足夠多的關鍵幀觀測到、追蹤成功率必須夠高。不合格者立即刪除,避免污染地圖。
223. 建立新地圖點:新地圖點由共視圖中相連關鍵幀的 ORB 特徵三角化產生;某關鍵幀中未匹配的 ORB 與其他關鍵幀中未匹配的 ORB 比較,配對必須滿足對極約束才算有效;成對的 ORB 被三角化後,還要檢查在兩幀中的深度是否為正,並檢驗視差、重投影誤差與尺度一致性。
224. 局部束調整:對當前關鍵幀、共視圖中與它相連的關鍵幀,以及這些關鍵幀所見的所有地圖點,執行局部 BA(第 12.5 節)。這是精度的主要來源。
225. 剔除冗餘關鍵幀:若某關鍵幀所見的地圖點中,有九成以上也被其他至少三個關鍵幀看到,則它是冗餘的,予以刪除。這讓地圖規模不會隨時間無限成長。

嚴格的品質控管

同學應該注意到:步驟 2、3、5 都在「刪東西」。這正是 ORB-SLAM2 精度的祕密——它對地圖品質極為嚴苛,寧可少一些地圖點與關鍵幀,也不要不可靠的資料。

三角化新點時的多重檢驗尤其嚴謹:對極約束、正深度、視差角(第 20.5 節)、重投影誤差、尺度一致性——五道關卡缺一不可。這反映了一個工程原則:錯誤的地圖點比沒有地圖點更危險,因為它會持續誤導後續的位姿估計。

基線的考量

實作上還有一個細節值得一提:三角化新點時,系統會從共視圖中選取鄰近的關鍵幀(單目模式取較多、非單目取較少),並且只使用其中基線足夠大的那些。這正是第 20.5 節「視差角決定三角測量精度」的直接應用——基線太小的關鍵幀對,三角化出來的點深度不可靠。

21.5 Loop Closing 執行緒

閉環執行緒是後端,它處理 SLAM 最根本的問題:累積漂移。閉環執行緒負責偵測重訪區域,並使用 g2o 框架以位姿圖優化修正累積誤差;在位姿圖優化之後,它會啟動第四個執行緒執行完整束調整,最佳化整個地圖與關鍵幀之間的軌跡估計。

第一步:偵測候選閉環

ORB-SLAM2 基於 DBoW2 函式庫辨識重訪的地點;它把當前影像的特徵以二元格式,與階層式 DBoW2 資料庫中儲存的二元特徵比較,若當前幀與資料庫中某個表示的相似度達到一定分數,即偵測到閉環。

這裡的關鍵是效率:資料庫可能有數千個關鍵幀,不可能逐一做特徵匹配。詞袋模型把每張影像壓縮成一個「視覺單詞的統計向量」,比對只需計算向量相似度,速度極快——這正是第 19.7 節介紹 DBoW2 的價值所在。

第二步:幾何驗證

詞袋相似只是「看起來像」,不足以確認。系統接著做嚴格的幾何驗證:對候選閉環幀與當前幀做特徵匹配,並求解兩者之間的變換。若相似度得到足夠多內點的支持,閉環才被接受。

這一步至關重要。第 12 章與第 15.4 節都強調過:一條錯誤的閉環會把地圖硬拉到錯誤位置,造成災難性後果。因此寧可漏掉閉環,也不能接受錯誤的閉環。

第三步:相似變換與尺度修正

這裡有一個單目 SLAM 特有的細節。由於單目有尺度漂移(第 19.5 節),閉環時當前的尺度可能與初始尺度不同。因此求解的不是一般的剛體變換(旋轉加平移,第 10 章),而是相似變換——多加一個尺度因子。

修正時,系統會把這個尺度差異也一併攤平到整條軌跡上,同時修正位置與尺度漂移。雙目與 RGB-D 模式因為有絕對尺度,則只需一般的剛體變換。

第四步:位姿圖優化與全域 BA

- 位姿圖優化:在本質圖(21.6 節)上執行位姿圖優化(第 12 章),快速把整條軌跡拉正。因為只最佳化位姿(不含地圖點),速度快,能即時完成——這正是第 15 章 Cartographer 的相同策略。
- 全域 BA:位姿圖優化後,系統啟動第四個執行緒執行完整的束調整,同時最佳化所有關鍵幀位姿與所有地圖點,得到最佳的整體結構與運動解。這一步較慢,因此放在背景執行,不阻塞前端追蹤。

這個「先快速拉正、再精細最佳化」的兩段式設計,同學應該覺得熟悉——它與第 18 章 LOAM 的雙層架構、第 15 章 Cartographer 的雙子系統,是同一個系統設計模式在不同系統中的體現。

21.6 KeyFrame、MapPoint、Covisibility Graph

一個系統的資料結構,決定了它能做什麼。本節說明 ORB-SLAM2 的四個核心資料結構。

關鍵幀(KeyFrame)

關鍵幀是被保留下來的影像幀,代表一個具代表性的視角。它儲存:相機位姿(第 10 章的變換矩陣)、所有 ORB 特徵點與描述子、該幀的詞袋向量、以及與其他關鍵幀的

連接關係。

為什麼不保留所有幀?因為相鄰幀的資訊高度重複,全部保留會使 BA 的規模爆炸。

關鍵幀是「以最少的幀涵蓋最多視角」的取舍結果。

地圖點(MapPoint)

地圖點是三維空間中的特徵點。它儲存:三維座標、代表性描述子(從所有觀測中選出最具代表性的那個)、平均觀測方向、以及「哪些關鍵幀看到了我」的觀測清單。

最後一項至關重要:它建立了「地圖點↔關鍵幀」的雙向連結,使系統能快速查詢「這個關鍵幀看到哪些點」與「這個點被誰看到」。共視圖正是由這個關係推導出來的。

共視圖(Covisibility Graph)

系統維護一個共視圖,連接任何兩個觀測到共同地圖點的關鍵幀,以及一個連接所有關鍵幀的最小生成樹。

- 節點:每個關鍵幀。
- 邊:兩個關鍵幀若共同觀測到足夠多的地圖點,就連一條邊;邊的權重是共同觀測的地圖點數量。

共視圖的作用是定義「鄰居」:與當前幀共視程度高的關鍵幀,就是空間上鄰近、視角相似的幀。追蹤與局部建圖都只在這個局部範圍內運作,因此能在大型環境中維持即時性(21.3 節引述的原理)。

生成樹與本質圖

- 生成樹(Spanning Tree):連接所有關鍵幀的最小生成樹,每個關鍵幀連到「與它共視最多的那個關鍵幀」。它極為輕量,保證所有關鍵幀連通。

- 本質圖(Essential Graph):閉環時做位姿圖優化的結構。它包含生成樹的所有邊、共視權重高於門檻的邊,以及閉環邊。相較於完整共視圖,本質圖的邊少得多,使位姿圖優化能快速完成。

這個「完整共視圖用於局部操作、精簡本質圖用於全域最佳化」的分層設計,是 ORB-SLAM2 能兼顧精度與效率的關鍵。這些圖結構在閉環時,也作為位姿圖優化所使用的結構。

資料結構	內容	主要用途
KeyFrame	位姿、ORB 特徵、詞袋向量、連接關係	地圖的骨架、BA 的變數
MapPoint	三維座標、描述子、觀測清單	追蹤的參考、BA 的變數
共視圖	關鍵幀節點 + 共視權重邊	取得局部地圖、定義鄰居
生成樹	最小生成樹的邊	保證連通、輕量結構
本質圖	生成樹 + 強共視邊 + 閉環邊	閉環時的位姿圖優化

表 21-1 ORB-SLAM2 的核心資料結構

21.7 ORB-SLAM2 實作與資料集測試

本節說明實作要點,為第 29 章的完整實作與第 30 章專題預作準備。

編譯與相依套件

ORB-SLAM2 是 C++ 專案,編譯前需備妥幾個相依套件:

- OpenCV:影像處理與特徵提取(第 9、19 章)。
- Eigen3:線性代數運算。
- Pangolin:三維視覺化,用於顯示地圖與軌跡。

- g2o 與 DBoW2:已包含在專案的 Thirdparty 資料夾中,分別負責圖優化(第 12 章)與地點辨識(第 19.7 節)。

實作提醒:ORB-SLAM2 發布於數年前,在較新的作業系統與 OpenCV 版本上可能需要修改少量程式碼才能編譯成功。建議參考社群維護的分支版本,或使用容器化環境以避免相依性問題——這是同學實作時最常卡關的地方。

組態檔:相機參數

執行前必須準備一個 YAML 組態檔,內容包括:

- 相機內參:fx、fy、cx、cy(第 9.1 節的 K 矩陣)。
- 畸變係數:k1、k2、p1、p2(第 9.2 節)。
- 影像尺寸與幀率。
- 雙目的基線乘焦距、RGB-D 的深度縮放因子。
- ORB 參數:每幀特徵數(常用 1000 至 2000)、金字塔層數與縮放比例、FAST 門檻。

這裡再次強調第 9 章的重要性:相機校正不準,整個系統的精度就無從談起。務必用自己的相機做過完整校正,不要直接套用範例參數。

公開資料集

資料集	相機類型	場景	特色
TUM RGB-D	RGB-D	室內辦公室	有動作捕捉真值,適合精度評測
KITTI	雙目	戶外道路	自駕車場景,大範圍長軌跡
EuRoC MAV	雙目 + IMU	室內機房與廠房	無人機視角,含快速運動序列

表 21-2 常用的視覺 SLAM 公開資料集

建議的學習順序是:先跑 TUM RGB-D 的簡單序列(緩慢運動、室內、有深度)建立成功經驗,再挑戰 EuRoC 的快速運動序列,最後嘗試 KITTI 的大範圍戶外場景。

評測方法

- 絕對軌跡誤差(ATE):把估計軌跡與真值軌跡對齊後,計算對應時刻位置差的均方根。這是最常用的整體精度指標。
- 相對位姿誤差(RPE):計算固定時間間隔內相對運動的誤差,反映局部漂移程度。
- 工具:evo 是目前最方便的軌跡評測工具,支援多種資料集格式,能自動對齊、計算指標並繪圖。
- 單目特別注意:單目軌跡沒有絕對尺度(第 19.5 節),評測時需先與真值做尺度對齊(Sim3 對齊),否則誤差會毫無意義。

觀察重點

執行時,建議同學在 Pangolin 視窗中留意幾件事:

- 226. 初始化過程:單目模式下,觀察系統需要多少幀、多大的運動才能成功初始化(對照 21.2 節)。
- 227. 關鍵幀的分布:觀察關鍵幀在軌跡上如何分布——是否在轉彎與視角變化大處較密集?
- 228. 共視關係:視覺化中關鍵幀之間的連線就是共視圖,觀察它的結構。
- 229. 閉環時刻:繞回起點時,觀察軌跡與地圖被拉正的瞬間(第 12、15 章的震撼畫面,在此再現)。

230. 追蹤丟失與重定位:刻意遮住相機數秒再放開,觀察系統如何丟失又如何重定位回來——這是 ORB-SLAM2 最令人印象深刻的能力之一。

至此,一個完整的視覺 SLAM 系統已經完整呈現。但 ORB-SLAM2 仍有兩個限制:它是純視覺的(沒有 IMU,因此單目有尺度問題、快速運動易跟丟),而且一旦嚴重跟丟且無法重定位,系統就無法繼續。下一章的 ORB-SLAM3 正是為解決這兩點而生——緊耦合的視覺慣性融合,以及能在跟丟時開啟新地圖、日後再合併的多地圖系統。

公式或流程

一、三種 BA 的比較

ORB-SLAM2 在不同階段使用三種束調整(第 12.5 節),差別在於「哪些變數被最佳化」:

$$\text{僅運動 BA: } \operatorname{argmin}_{\{R, t\}} \sum_i \rho(\|u_i - \pi(K, R, t, P_i)\|^2)$$

固定地圖點 P_i , 只求相機位姿 (R, t) 。用於追蹤執行緒的每一幀,速度最快。

$$\text{局部 BA: } \operatorname{argmin}_{\{R, t, P\}} \sum \text{局部關鍵幀與其地圖點的重投影誤差}$$

同時最佳化局部關鍵幀的位姿與它們所見的地圖點。用於局部建圖執行緒,是精度的主要來源。

$$\text{全域 BA: } \operatorname{argmin}_{\{全部 R, t, P\}} \sum \text{所有關鍵幀與地圖點的重投影誤差}$$

最佳化整張地圖,運算量最大,只在閉環後的第四個執行緒中執行。式中 ρ 為穩健核函數(第 12、15 章),用來降低離群觀測的影響。

二、位姿圖優化(閉環修正)

閉環時在本質圖上執行位姿圖優化,只最佳化關鍵幀位姿(不含地圖點):

$$\operatorname{argmin} \sum_{\{(i,j) \in \text{本質圖}\}} \log(\Delta T_{ij}^{-1} \cdot T_i^{-1} \cdot T_j) \|^2$$

其中 ΔT_{ij} 是邊所記錄的相對位姿量測、 T_i 與 T_j 是待最佳化的關鍵幀位姿。單目模式下 T 為含尺度的相似變換(Sim3), 雙目與 RGB-D 則為剛體變換(SE3)。這正是第 12 章位姿圖優化的具體實現。

三、三執行緒協同流程

圖 21-1 ORB-SLAM2 的三執行緒架構

【追蹤執行緒】即時, 每幀執行

提取 ORB → 依運動模型 / 參考幀 / 重定位求初始位姿

→ 追蹤局部地圖 → 僅運動 BA 精修 → 判斷是否插入關鍵幀

↓ (新關鍵幀)

【局部建圖執行緒】非即時, 關鍵幀觸發

插入共視圖與生成樹 → 剔除劣質地圖點 → 三角化新地圖點

→ 局部 BA → 剔除冗餘關鍵幀

↓ (新關鍵幀)

【閉環執行緒】非即時, 背景執行

DBoW2 偵測候選 → 幾何驗證 → 求解相似變換(Sim3)

→ 本質圖位姿圖優化(拉正軌跡)

↓

【第四執行緒】全域束調整, 最佳化整張地圖與軌跡

四、地圖點三角化的五道檢驗

圖 21-2 新地圖點必須通過的檢驗

① 對極約束:配對的 ORB 必須滿足對極幾何(第 20 章)



② 正深度檢驗:三角化後在兩幀中的深度都必須為正



③ 視差角檢驗:視差必須足夠大,否則深度不可靠(第 20.5 節)



④ 重投影誤差檢驗:投影回兩張影像的誤差必須夠小



⑤ 尺度一致性檢驗:觀測到的尺度與距離必須合理



五道全過 → 成為地圖點;任一失敗 → 捨棄

案例

案例 21-1 遮住相機再放開:重定位的魔術時刻

這是展示 ORB-SLAM2 能力最有戲劇效果的示範,幾乎每個課堂都值得做一次。

讓系統正常運行、地圖已建立一段之後,用手完全遮住相機鏡頭。畫面全黑,特徵瞬間歸零,系統立刻顯示追蹤丟失(TRACK LOST),軌跡停止更新。此時把手移開,讓相機重新看到之前走過的區域——不到一秒,系統就「跳」回正確的位置,軌跡無縫接續。

這一秒鐘裡發生了什麼?系統啟動了地點辨識模組:把當前影像的 ORB 特徵轉成詞

袋向量,在關鍵幀資料庫中快速搜尋最相似的候選,對候選做特徵匹配,再以 PnP 加 RANSAC(第 20 章)求解位姿,通過驗證後恢復追蹤。

更有趣的變化是:遮住相機後,把相機搬到房間的另一個角落再放開。系統同樣能找回自己——這正是第 3 章「綁架機器人問題」的視覺版本。相較之下,若是純視覺里程計(沒有地圖與地點辨識),遮擋後就永遠回不來了。

這個示範讓同學親身體會「SLAM 與 VO 的差別」:VO 只能一路往前推算,SLAM 卻擁有地圖與記憶,能從失敗中恢復。這也是第 19.1 節流程圖中「閉環偵測」模組價值的最佳註腳。

案例 21-2 寬鬆插入、嚴格剔除:一個工程哲學

ORB-SLAM2 對關鍵幀的處理策略,乍看有些矛盾:插入時判準寬鬆(常常插入),但局部建圖又會積極剔除冗餘的關鍵幀。為什麼不乾脆一開始就嚴格一點?

原因在於兩者服務的目標不同。插入時的目標是追蹤穩健:若關鍵幀太稀疏,快速運動或大幅轉向時,當前幀與最近關鍵幀的視角差異過大,匹配就會失敗導致跟丟。寬鬆插入等於為系統買了保險。

剔除時的目標則是地圖精簡:關鍵幀無限增加會使 BA 規模膨脹、記憶體耗盡、系統逐漸變慢。所以在確認某個關鍵幀的資訊已被其他關鍵幀充分涵蓋(九成地圖點被至少三個其他關鍵幀觀測到)之後,就把它移除。

這個「先求穩、後求精」的策略,是一個值得記住的工程哲學:在不確定的當下寧可多做(保留選項),等資訊充足後再回頭精簡。同學會發現這與第 12 章「圖優化能回頭修正歷史」的優勢相呼應——正因為系統能事後修正,當下才敢寬鬆。

案例 21-3 從 ORB-SLAM2 長出的家族

ORB-SLAM2 開源之後,成為無數研究的基礎。觀察這個「技術家族」的開枝散葉,能看出視覺 SLAM 的研究地形。

- 加入其他特徵:有研究在點特徵之外加入線段特徵。PL-SLAM 提出的系統同樣基於三個執行緒:視覺里程計、局部建圖與閉環;其地圖由關鍵幀、偵測到的三維地標(同時包含關鍵點與線段)、共視圖與生成樹組成。線特徵在人造環境(走廊、建築)中特別有價值,因為那裡點特徵可能不足但線條豐富。
- 加速與輕量化:有研究提出免描述子的變體。FastORB-SLAM 的結構基於 ORB-SLAM2 並實作三個執行緒,但改以灰階匹配而非描述子匹配來建立相鄰幀之間的關鍵點對應,再以兩階段、與描述子無關的關鍵點匹配方法處理離群點問題,最後透過最小化重投影誤差恢復相機運動。
- 應用延伸:有研究把 ORB-SLAM2 用於視障者的穿戴式導航裝置——由明眼人配戴裝置行走以建立「虛擬導盲路徑」,再供視障者依循。該研究說明追蹤執行緒能透過與局部地圖的特徵匹配定位相機,局部建圖執行緒把新點插入並以局部 BA 最佳化,閉環執行緒偵測大迴路並最佳化位姿圖。
- 持久化地圖:也有研究為自駕車的視覺定位加上地圖儲存功能,讓建好的地圖能存檔重用,系統在定位模式下關閉局部建圖與閉環執行緒,僅執行追蹤。

這些延伸的共同點是:它們都保留了 ORB-SLAM2 的三執行緒骨架,只替換或增添其中的模組。這說明了一個好架構的價值——它不只解決當下的問題,更成為後續創新的平台。同學若要做視覺 SLAM 專題,以 ORB-SLAM2 為基礎進行改良,是一條務實且有前人可循的路徑。

實作活動

實作 21-1 ORB-SLAM2 系統實作與評測

目的:完整編譯並執行 ORB-SLAM2,在公開資料集上評測精度,觀察三執行緒的協同運作,並實驗系統的極限。這是第六篇最重要的實作,也是第 29、30 章的基礎。

第一部分:環境建置與資料集執行(全體)

190. 安裝相依套件(OpenCV、Eigen3、Pangolin),編譯 ORB-SLAM2。若遇到編譯錯誤,記錄問題與解法——這是實務工程的必經訓練。
191. 下載 TUM RGB-D 資料集的一個簡單序列(建議從 freiburg1_xyz 或 freiburg2_desk 開始),以 RGB-D 模式執行,觀察 Pangolin 視窗中的地圖與軌跡。
192. 同一段序列改用單目模式執行,比較兩者的差異:初始化耗時、軌跡完整度、地圖點密度。
193. 觀察並記錄:系統建立了多少關鍵幀?多少地圖點?關鍵幀在軌跡上的分布有什麼規律?

第二部分:三執行緒行為觀察(全體)

194. 初始化觀察(單目):記錄系統從啟動到成功初始化所需的幀數與相機移動距離,對照 21.2 節的原理。
195. 共視圖觀察:在視覺化中觀察關鍵幀之間的連線,判斷哪些關鍵幀共視程度高,並與相機軌跡對照。
196. 閉環觀察:選一段含迴路的序列(如 KITTI 序列 00),觀察閉環觸發瞬間軌跡與地圖被拉正的過程,錄影記錄。
197. 重定位實驗:執行時人為中斷影像輸入數秒(或選用含遮擋的序列),觀察追蹤丟失與重定位的完整過程,重現案例 21-1。

第三部分:精度評測(大學部)

198. 安裝 evo 工具,把 ORB-SLAM2 輸出的軌跡與資料集真值比較,計算絕對軌跡誤差(ATE)與相對位姿誤差(RPE)。
199. 繪製軌跡對照圖與誤差隨時間變化圖,找出誤差最大的區段,並回頭觀察該區段的影像,推測原因(快速運動?弱紋理?動態物體?)
200. 單目評測注意:記得先做尺度對齊(Sim3),否則誤差無意義。比較「有無尺度對齊」的誤差數值差異,親身理解尺度不確定性。
201. 三模式比較:若資料集支援,分別以單目、雙目、RGB-D 執行同一場景,比較三者的 ATE 與運算負擔,填入你自己的比較表。

第四部分:參數實驗與極限測試(大學部)

202. 特徵數實驗:把每幀提取的 ORB 特徵數分別設為 500、1000、2000,比較精度、幀率與地圖點數量的變化,找出你場景的合適值。
203. 弱紋理挑戰:自行錄製一段包含大面積白牆的影片,觀察系統在哪裡跟丟,驗證第 9、19 章的「白牆詛咒」。
204. 快速運動挑戰:錄製一段快速轉向的影片,觀察追蹤是否失敗,並思考 IMU 能如何幫助(預告第 22 章)。

討論問題

99. 單目模式的初始化需要多少幀?若相機只做純旋轉,結果如何?這印證了 21.2 節與第 20.5 節的哪些論述?
100. 系統剔除了多少關鍵幀?若關閉冗餘關鍵幀剔除功能,長時間運行會發生什麼?
101. 閉環拉正的瞬間,軌跡上哪些區段位移最大?為什麼是那些區段?(提示:第 12 章「把矛盾均攤」)
102. 三種模式的 ATE 差多少?RGB-D 是否一定最準?在什麼情況下雙目會勝出?

103. (大學部)特徵數從 500 增至 2000,精度提升多少?幀率下降多少?這個取舍對即時系統意味著什麼?

延伸挑戰(大學部)

其一:閱讀 ORB-SLAM2 的 Tracking.cc 原始碼,找出三種追蹤策略(運動模型、參考關鍵幀、重定位)的實作位置,並在關鍵處加上輸出訊息,執行時觀察系統在什麼情況下切換策略、切換頻率如何。這是把本章理論與真實程式碼對應起來的最佳練習。其二:嘗試用自己的相機(手機或網路攝影機)執行 ORB-SLAM2——先完成第 9 章的相機校正、撰寫 YAML 組態檔、再錄製影片或即時輸入,把大學的教室或走廊建成地圖。這份成果可直接作為第 30 章專題的素材。

延伸閱讀

139. R. Mur-Artal and J. D. Tardós, "ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo and RGB-D Cameras," IEEE T-RO, 2017.(本章主要依據的原始論文)
140. R. Mur-Artal, J. M. M. Montiel, and J. D. Tardós, "ORB-SLAM: A Versatile and Accurate Monocular SLAM System," IEEE T-RO, 2015.(前身之作,單目版本的完整設計)
141. ORB-SLAM2 開源專案(GitHub: raulmur/ORB_SLAM2).(原始程式碼、編譯說明與資料集執行範例)
142. D. Gálvez-López and J. D. Tardós, "Bags of Binary Words for Fast Place Recognition in Image Sequences," IEEE T-RO, 2012.(DBoW2,閉環與重定位的技術基礎)
143. H. Strasdat, J. M. M. Montiel, and A. J. Davison, "Scale Drift-Aware Large Scale Monocular SLAM," RSS, 2010.(共視圖與 Sim3 閉環修正的來源)

144. 高翔、張濤,《視覺 SLAM 十四講》第 9 講(後端)與實踐章節.(華文最佳的 ORB-SLAM 系統剖析輔助教材)
145. evo 軌跡評測工具(GitHub: MichaelGrupp/evo).(ATE 與 RPE 計算、軌跡繪圖,實作評測必備)

小結

112. ORB-SLAM2 對追蹤、建圖與地點辨識三項任務使用同一套 ORB 特徵;ORB 對旋轉、尺度、相機自動增益與曝光及光線變化具良好不變性,提取匹配快速,並在詞袋地點辨識中有良好的精確率與召回率。
113. 單目初始化平行計算單應矩陣(平面場景)與基礎矩陣(非平面場景),依相對分數擇一;雙目與 RGB-D 第一幀即有深度,可立即建圖,並依深度把特徵分為可靠的近點與需累積視差的遠點。
114. 追蹤執行緒以僅運動 BA 最小化重投影誤差來定位相機,依序嘗試運動模型、參考關鍵幀與重定位三種策略,並藉共視圖取得局部地圖做進一步精修;關鍵幀採「寬鬆插入」策略。
115. 局部建圖執行緒插入關鍵幀至共視圖與生成樹並建立詞袋、剔除劣質地圖點、由相連關鍵幀三角化新點(須通過對極約束、正深度、視差、重投影誤差與尺度一致性五道檢驗)、執行局部 BA,並剔除冗餘關鍵幀。
116. 閉環執行緒以 DBoW2 比對二元特徵偵測重訪,經幾何驗證取得足夠內點後接受閉環,求解相似變換(單目含尺度修正),在本質圖上做位姿圖優化,並啟動第四執行緒執行全域 BA。
117. 核心資料結構為關鍵幀、地圖點、共視圖(連接觀測共同地圖點的關鍵幀)、最小生成樹與本質圖;共視圖用於局部操作以支援大型環境,精簡的本質圖則用於閉環的位姿圖優化。

118. 實作需備妥 OpenCV、Eigen3、Pangolin,並以 YAML 提供精確的相機內參與畸變係數;常用資料集為 TUM RGB-D、KITTI 與 EuRoC,評測以 ATE 與 RPE 為指標,單目須先做尺度對齊。

ORB-SLAM2 是視覺 SLAM 成熟度的標竿,但它仍有兩個明確限制:純視覺架構使單目有尺度問題、快速運動易跟丟;而且嚴重跟丟且無法重定位時,系統便無以為繼。下一章的 ORB-SLAM3 正是為此而生——以最大後驗估測為基礎的緊耦合視覺慣性融合(第 7 章的 IMU 預積分在此登場),以及跟丟時開新地圖、日後再無縫合併的多地圖系統。第六篇的最高峰,即將登場。

台灣自編大學教科書

@prof

《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第 22 章

ORB-SLAM3 與 Visual-Inertial SLAM

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

學習目標

上一章的 ORB-SLAM2 已是視覺 SLAM 成熟度的標竿,但它仍有兩個限制:純視覺架構使單目有尺度問題、快速運動易跟丟;而嚴重跟丟且無法重定位時,系統便無以為繼。ORB-SLAM3 正是為此而生。本章是第六篇的最高峰,也是全書視覺路線的集大成。學完本章之後,同學應能夠達成下列目標:

138. 說明 ORB-SLAM3 相對 ORB-SLAM2 的三項核心突破。
139. 說明四種資料關聯(短期、中期、長期、多地圖)的意義與價值。
140. 說明單目慣性與雙目慣性 SLAM 的架構與 IMU 帶來的貢獻。
141. 說明以最大後驗估測為基礎的 IMU 初始化三階段流程。
142. 說明 Atlas 多地圖系統的組成與地圖合併的兩階段策略。
143. 說明追蹤失敗後的重定位與地圖合併機制。
144. 舉例說明視覺慣性 SLAM 在手機、無人機與穿戴裝置上的應用。

核心概念

中文名詞	英文名詞	概念說明
視覺慣性 SLAM	Visual-Inertial SLAM	緊耦合相機與 IMU 的同步定位與建圖。
多地圖系統	Atlas	同時維護多張地圖的架構,含一張活躍地圖與多張非活躍地圖。
活躍地圖	Active Map	追蹤執行緒當前用來定位的地圖。
非活躍地圖	Inactive Map	Atlas 中暫時未使用、但保留待日後合併的地圖。
短期資料關聯	Short-term	匹配最近數秒內的地圖元素。

中文名詞	英文名詞	概念說明
中期資料關聯	Mid-term	匹配相機附近、帶有輕微累積漂移的地圖元素。
長期資料關聯	Long-term	以地點辨識匹配先前造訪區域,不受累積漂移影響。
多地圖資料關聯	Multi-map	匹配並使用先前建圖階段的地圖元素。
最大後驗估測	MAP Estimation	在已知觀測下尋找機率最大的狀態估計。
IMU 初始化	IMU Initialization	估出尺度、重力方向、零偏與速度的啟動程序。
地圖合併	Map Merging	把兩張地圖無縫融合為一的操作。
焊接視窗	Welding Window	地圖合併時先行融合的局部關鍵幀範圍。
多階段作業	Multi-session	跨多次執行、重複使用先前地圖的運作模式。

22.1 ORB-SLAM3 的系統特色

ORB-SLAM3 由 Campos、Elvira、Rodríguez、Montiel 與 Tardós 於 2021 年發表,是第 7.7 節已初步介紹過的系統。它是特徵法 SLAM 這條血脈的集大成之作,從 PTAM、ORB-SLAM、ORB-SLAM2 一路演進而來,發布時是當時最完整、最精確的開源 SLAM 函式庫。

三項核心突破

- 緊耦合的視覺慣性 SLAM:第一個以最大後驗估測(MAP)為基礎、連 IMU 初始化階段也建立在 MAP 之上的緊耦合系統(第 7.7 節)。這解決了單目的尺度問題與快速運動下的穩健度。
- 多地圖系統(Atlas):跟丟時不再崩潰,而是開啟新地圖繼續運作,日後重回舊區域時再無縫合併。這解決了 ORB-SLAM2「跟丟即結束」的致命弱點。
- 四種資料關聯的全面運用:這是最根本的架構創新。ORB-SLAM3 是第一個在所有演算法階段都運用短期、中期、長期與多地圖資料關聯的系統。

支援的感測器組態

第 7.7 節提過,ORB-SLAM3 支援極為廣泛的組態,這是它實用價值的重要來源:

231. 相機類型:單目、雙目、RGB-D。

232. 鏡頭模型:針孔與魚眼(第 9 章)。魚眼支援對室內大視角應用特別有價值。

233. 是否含 IMU:每一種相機模式都有對應的視覺慣性版本。

把這些組合起來,系統支援單目、單目慣性、雙目、雙目慣性、RGB-D、RGB-D 慣性等多種模式——同一套程式碼,換個參數即可。這讓它成為研究與教學的絕佳平台:同學能在完全相同的框架下,比較不同感測器組態的表現。

精度表現

第 7.7 節已引述過關鍵資料,此處補充多階段作業的表現。在 EuRoC 多階段實驗中,Atlas 的地圖合併使精度較 CCM-SLAM 與 VINS-Mono 提升一倍以上;相對 VINS-Mono 的優勢從單階段的 2.6 倍成長到多階段的 3.2 倍。整體而言,論文摘要宣稱其精度較先前方法提升二至十倍;開源釋出後,它成為數百篇後續論文的標準比較基準。

這個「單階段已勝出、多階段優勢更大」的現象,清楚說明了多地圖架構的價值:能重複使用先前的地圖,精度自然更高。這對實際部署的機器人(每天在同一場域運行)意義重大。

22.2 四種資料關聯:系統的架構主軸

本節是理解 ORB-SLAM3 設計哲學的關鍵。作者把「資料關聯」——也就是「判斷不同時刻的觀測是否對應同一個地圖元素」——分為四個層次,並主張先前的系統往往只用了其中一兩種。

類型	匹配對象	技術手段	解決什麼
短期	最近數秒內的地圖元素	投影匹配	基本的逐幀追蹤
中期	相機附近、有輕微漂移的元素	局部地圖投影匹配	減少漂移累積,提升精度
長期	先前造訪區域(不論漂移多大)	地點辨識(DBoW2)	閉環修正、重定位
多地圖	先前建圖階段的地圖元素	地點辨識 + 地圖合併	跨階段重用、跟丟後恢復

表 22-1 四種資料關聯的比較

短期與中期:日常運作的基礎

幀與活躍地圖之間的短期與中期資料關聯,是由追蹤與建圖執行緒例行找出的——把地圖點投影到估計的相機位姿,並在影像中僅數個像素的視窗內搜尋匹配。

中期關聯是 ORB-SLAM3 精度優勢的重要來源。實驗顯示,在 EuRoC 資料集的單目慣性單階段運作中,ORB-SLAM3 的精度是 VINS-Mono 的 2.6 倍,這歸功於它使用中期資料關聯的能力。

為什麼中期關聯如此重要?因為許多視覺里程計系統只做短期關聯(只與前幾幀比對),一旦某個地圖點暫時離開視野再回來,系統就把它當成新的點——這浪費了資訊,也讓漂移持續累積。ORB-SLAM3 則能重新匹配上這些「還在附近但有些漂移」的舊點,等於不斷地做小規模修正。

長期與多地圖:恢復與重用

為了達成重定位與閉環偵測所需的長期資料關聯,ORB-SLAM 使用 DBoW2 詞袋地點辨識系統,這個方法也被多數近期的 VO 與 SLAM 系統所採用。

多地圖關聯則是 ORB-SLAM3 獨有的第四層。它讓系統能匹配「不同建圖階段」的地圖元素——無論是同一次執行中跟丟後開的新地圖,還是昨天建好存下來的地圖。這開啟了多階段作業的可能。

觀念總結

這四層關聯可以用一個記憶的比喻來理解:短期是「剛剛看到的」、中期是「這附近熟悉的」、長期是「以前來過的」、多地圖是「別次經歷的」。一個好的 SLAM 系統,應該像人一樣善用所有層次的記憶——這正是 ORB-SLAM3 的設計哲學。

22.3 Monocular / Stereo Visual-Inertial SLAM

第 7.6 節已經說明過 IMU 在視覺慣性 SLAM 中的四大貢獻(絕對尺度、重力方向、

填補視覺失效空隙、高頻輸出),本節聚焦 ORB-SLAM3 如何實現緊耦合。

為什麼需要緊耦合

IMU 量測角速度與線加速度,但存在顯著漂移,使得單靠 IMU 資料的二次積分無法用於長期位姿估計。這正是第 7.3 節的核心結論。

反過來說,純視覺在快速運動、弱紋理、光線劇變時會失效。因此兩者必須融合——而如第 7.6 節所述,緊耦合(把 IMU 預積分約束與視覺重投影誤差放進同一個最佳化問題)的精度顯著優於鬆耦合。

ORB-SLAM3 的緊耦合實現

系統的三個執行緒都因 IMU 而擴充:

- 234. 追蹤執行緒:除了最小化重投影誤差,還加入 IMU 預積分的慣性誤差項(第 7.5 節),同時估計位姿、速度與 IMU 零偏。
- 235. 局部建圖執行緒:局部建圖模組把關鍵幀與地圖點加入當前活躍地圖、移除冗餘的幀,並在當前幀的鄰近關鍵幀上,以視覺束調整或視覺慣性束調整最佳化地圖。
- 236. 閉環與地圖合併執行緒:在慣性模式下,閉環修正還需處理重力方向與尺度的一致性。

狀態向量的擴充

與純視覺相比,視覺慣性 SLAM 每個關鍵幀要估計的量變多了:

- 237. 純視覺:位姿(旋轉 R 、平移 t),共 6 個自由度。
- 238. 視覺慣性:位姿 + 速度 + 加速度計零偏 + 陀螺儀零偏,共 15 個自由度。

狀態變多雖然增加了運算量,但換來的是:絕對尺度(單目不再有尺度不確定性)、重力方向已知(地圖自動擺正,滾轉與俯仰角可觀測)、以及在視覺短暫失效時仍能維持追

蹤數秒的能力。這個交換在絕大多數應用中都極為划算。

雙目慣性:最強健的組態

若不計成本與體積,雙目慣性是 ORB-SLAM3 最強健的組態:雙目提供絕對尺度與單幀深度(不需等待視差累積)、IMU 提供高頻運動與快速運動下的穩健度。第 7.7 節引述的 EuRoC 3.5 公分精度,正是雙目慣性模式的成績。對大學的巡檢機器人或無人機專題,若條件允許,這是首選組態。

22.4 IMU 初始化與預積分

第 7.7 節已介紹過 IMU 初始化的困難與運動激勵的必要性,本節深入其三階段流程——這是 ORB-SLAM3 最重要的技術貢獻之一。

初始化要解出什麼

系統啟動時,有一堆彼此耦合的未知數必須同時確定:

- 239. 尺度:單目模式下地圖的真實大小(雙目與 RGB-D 已知,故較單純)。
- 240. 重力方向:哪個方向是「下」,用來從加速度計讀值中扣除重力(第 7.1 節的比力觀念)。
- 241. IMU 零偏:加速度計與陀螺儀的偏移(第 7.3 節)。
- 242. 初始速度:每個關鍵幀的速度。

三階段流程

ORB-SLAM3 把初始化拆成三個階段,逐步逼近:

- 純視覺 MAP 估測:先只用視覺,建立一段短時間的地圖與軌跡(單目模式下尺度未知)。這提供了後續慣性估計所需的運動基準。

- 純慣性 MAP 估測:固定視覺軌跡不動,只用 IMU 量測來估計尺度、重力方向、零偏與速度。因為視覺軌跡已定,這個子問題變得相對簡單。
- 聯合視覺慣性 BA:最後把所有變數(位姿、速度、零偏、尺度、重力、地圖點)放進同一個最佳化問題聯合精修。

這個「先分開解、再聯合精修」的策略,大幅提升了初始化的速度與可靠度。

初始化的表現

IMU 初始化在 2 秒內達到 5% 的尺度誤差,經精修後約為 1%。這相較第 7.7 節提到的早期系統(ORB-SLAM-VI 需約 15 秒)是巨大的進步——2 秒對使用者體驗而言幾乎無感,15 秒則明顯令人不耐。

實作提醒:運動激勵

再次強調第 7.7 節的關鍵提醒:IMU 初始化需要足夠的運動激勵。若載體只是等速直線移動或完全靜止,加速度計讀到的幾乎只有重力,系統無法把「尺度」與「零偏」區分開來(數學上不可觀測)。

實務操作:啟動後拿著裝置做數秒帶有加減速與旋轉的運動(小幅度前後左右移動並轉動),系統才能順利完成初始化。這與第 20 章「純旋轉無法三角測量」並列為視覺慣性 SLAM 兩大實作陷阱——同學在第 29 章實作時務必記得。

22.5 Multi-Map 多地圖系統(Atlas)

這是 ORB-SLAM3 的第二項核心創新,也是最能體現「工程智慧」的設計。

Atlas 的組成

Atlas 是由一組離散地圖構成的多子地圖系統。它維護一張活躍地圖,供追蹤執行緒定位當前幀,而局部建圖執行緒持續以新的關鍵幀最佳化並更新該地圖;Atlas 中的其

他地圖則為非活躍地圖。系統並建立一個以詞袋模型為基礎的關鍵幀資訊資料庫,用於重定位、閉環偵測與地圖合併。

243. 活躍地圖(Active Map):正在使用的那一張,追蹤與局部建圖都在它上面運作。

244. 非活躍地圖(Inactive Map):曾經建立、目前未使用,但完整保留在記憶體中等待日後合併。

245. 共用的詞袋資料庫:所有地圖的關鍵幀都登錄在同一個資料庫中——這是能跨地圖比對的關鍵。

跟丟時發生什麼

這正是 Atlas 存在的理由。回顧 ORB-SLAM2 的處境:嚴重跟丟且重定位失敗時,系統就停在那裡,之後的資料全部浪費。

ORB-SLAM3 的做法是:跟丟後若短時間內無法重定位,就把當前地圖設為非活躍、開啟一張全新的地圖,從零開始繼續建圖與追蹤。使用者幾乎不會察覺中斷。而當機器人日後走回舊地圖涵蓋的區域,地點辨識會偵測到跨地圖的匹配,系統隨即把兩張地圖合併,恢復成一張完整一致的地圖。

與其他多地圖方案的差異

值得注意的是,也有其他系統提出 ORB-SLAM2 的多地圖擴充,但它們把子地圖保留為分離的實體;ORB-SLAM3 則執行無縫的地圖合併,建立更精確的全域地圖。

這個差異很關鍵。若只是把地圖各自存著不合併,雖然資料沒丟,但你會得到好幾張互不相干的地圖,無法規劃跨區域的路徑,精度也無法互相提升。無縫合併才能真正發揮多地圖的價值。

多階段作業的價值

Atlas 的另一個重要應用是跨執行階段的重用:今天建好的地圖存起來,明天啟動時

載入,系統能直接在舊地圖中重定位並繼續擴充。這對每天在同一場域運行的服務機器人、巡檢機器人極為實用,也解釋了 22.1 節提到「多階段精度優勢更大」的現象。

22.6 失去追蹤後的重定位與地圖融合

本節說明 Atlas 運作的技術細節:如何偵測跨地圖的關聯,以及如何合併。

地點辨識:高召回率的設計

每當建圖執行緒建立一個新的關鍵幀,地點辨識就會啟動,嘗試偵測與 Atlas 中任何既有關鍵幀的匹配。若找到的匹配關鍵幀屬於活躍地圖,就執行閉環;否則就是多地圖資料關聯,此時活躍地圖與匹配地圖會被合併。

這是一個優雅的統一設計:閉環與地圖合併,其實是同一個機制的兩種結果——差別只在於匹配到的關鍵幀屬於同一張地圖還是不同地圖。

精確率與召回率的取舍

地點辨識面臨一個經典的兩難:

246. 召回率要高:必須盡量找到所有真實的匹配,否則會錯過閉環與合併的機會。

247. 精確率必須是百分之百:一個錯誤的匹配會把地圖硬拉到錯誤位置,造成災難(第 12、15、21 章反覆強調)。

ORB-SLAM3 的做法是:為了達到高召回率,對每個新的活躍關鍵幀,向 DBoW2 資料庫查詢 Atlas 中數個相似的關鍵幀;而為了達到百分之百的精確率,每一個候選都要通過數個步驟的幾何驗證。

幾何驗證透過直接的 $\text{Sim}(3)$ 或 $\text{SE}(3)$ 對齊執行(使用 RANSAC 與 Horn 方法)。這裡再次看到第 20 章 RANSAC 的身影,以及第 21 章單目需用相似變換(含尺度)的道理。

地圖合併的兩階段策略

確認跨地圖關聯後,合併就要開始了。但這裡有一個工程難題:非活躍地圖可能很大,若一次全部合併,運算會阻塞系統。

ORB-SLAM3 的解法是把合併拆成兩個步驟:首先,在由匹配關鍵幀與其鄰居所定義的焊接視窗中執行合併;第二階段,再透過位姿圖優化把修正傳播到合併後地圖的其餘部分。

圖 22-1 地圖合併的兩階段策略

偵測到跨地圖關聯:活躍地圖 M_a 的關鍵幀 $K_a \leftrightarrow$ 非活躍地圖 M_m 的關鍵幀 K_m

↓ 估出對齊變換 T_{am}

【第一階段:焊接視窗】

在 K_a 與 K_m 的鄰居所構成的局部視窗中執行合併

在此視窗中密集搜尋中期資料關聯,提升精度

→ 局部立即可用,追蹤執行緒不中斷

↓

【第二階段:傳播修正】

以位姿圖優化把修正傳播到合併後地圖的其餘部分

→ 全域一致性恢復

這個「先局部焊接、再全域傳播」的策略,讓合併能即時進行而不阻塞追蹤。同學應該注意到,這與第 15 章 Cartographer「局部即時、全域背景」、第 18 章 LOAM「高頻里程計、低頻建圖」的架構思想完全一致——這是機器人系統設計的普遍模式。

一個細節:中期關聯的強化

作者提出的第二項創新是:一旦估出新關鍵幀與匹配地圖之間的相對位姿,便以匹配關鍵幀及其在共視圖中的鄰居定義一個局部視窗,並在此視窗中密集搜尋中期資料關聯,以提升精度。

這再次呼應 22.2 節的主軸:ORB-SLAM3 的精度優勢,很大程度來自它在每一個環節都盡可能運用中期資料關聯。

22.7 GoPro、手機、無人機上的視覺慣性 SLAM

視覺慣性 SLAM 已不只是研究題目,它就在你我的日常裝置中運作。本節說明幾個代表性應用。

手機與 AR / VR

第 19.7 節提過 AR 的「由內而外追蹤」,其技術本質正是視覺慣性 SLAM。手機的相機加上內建 IMU(第 7 章),構成完美的視覺慣性組合:相機提供不漂移的環境參考、IMU 提供高頻運動與快速轉頭時的穩健度。

ORB-SLAM3 論文中一個特別有說服力的資料,正是針對這類應用。在魚眼的 TUM-VI 基準測試上,它整體表現優於 VINS-Mono 與 Basalt;在類似 AR / VR 的房間序列中,雙目慣性模式的平均絕對軌跡誤差為 0.009 公尺,單目慣性模式為 0.011 公尺。

毫米級的精度——這正是把虛擬物件穩定「釘」在真實桌面上所需要的水準。同學可以想想:你手機裡那個能量測桌子長度的 AR 尺,背後就是這個等級的技術。

穿戴式相機與運動攝影

運動攝影機(如 GoPro)配戴在頭盔或胸前,在騎車、跑步、攀岩時錄影。這類影片對 SLAM 極具挑戰:劇烈震動、快速轉向、動態模糊、強烈的光線變化(進出樹蔭、隧

道)。

純視覺方法在這類序列上幾乎必然跟丟,而視覺慣性 SLAM 因為有 IMU 在視覺失效時撐住數秒(第 7.6 節的第三項貢獻),得以維持連續軌跡。這類應用的典型用途包括:運動路線的三維重建、極限運動的軌跡分析、以及第五篇提過的背包式空間測繪。

無人機

無人機是視覺慣性 SLAM 最重要的應用之一,也是第 25 章的主題。它的特殊需求包括:

- 248. 重量與功耗極度受限:相機加 IMU 是唯一可行的組合(第 9.5 節)。
- 249. 六自由度快速運動:純視覺難以應付,IMU 不可或缺。
- 250. GPS 拒止環境:室內、橋下、隧道中,視覺慣性是唯一的定位手段(第 5 章案例 5-3)。
- 251. 即時性要求嚴苛:定位延遲直接影響飛行穩定與安全。

第 7 章案例 7-3 提過的 EuRoC 資料集,正是以微型飛行器錄製,而 ORB-SLAM3 在其上的優異表現(第 7.7 節的 3.5 公分精度),說明了它在無人機應用上的實用性。

機器人與大學場域

對本書讀者而言,幾個具體的專題方向值得考慮:

- 252. 視覺導航機器人:以雙目慣性 ORB-SLAM3 建立校園地圖,再結合第 23 章的視覺導航與第 31 章的語意理解。
- 253. 多階段建圖:利用 Atlas 的多階段能力,分多次建立整棟系館的地圖,驗證地圖合併的效果。
- 254. 跟丟恢復實驗:設計一個「刻意讓系統跟丟」的實驗流程,量化 Atlas 相對 ORB-SLAM2 在恢復能力上的優勢——這是一個很好的比較型研究題目。

255. 與光達 SLAM 對照:在同一場域分別執行 ORB-SLAM3(視覺慣性)與第 18 章的 LIO 系統(光達慣性),比較兩者在不同環境(明亮走廊、昏暗機房、玻璃帷幕大廳)的表現。這個對照能讓同學深刻體會第 8、9 章反覆強調的感測器互補性。

至此,第六篇的核心系統已完整介紹。下一章我們把視野從「定位建圖」擴展到「導航與應用」——視覺導航機器人如何利用這些能力執行任務?線追蹤、視覺伺服、AprilTag 導航、YOLO 語意導航、乃至於 VLM 輔助導航,將把本書的技術與真實的機器人任務串接起來。

公式或流程

一、視覺慣性 SLAM 的狀態向量

視覺慣性模式下,每個關鍵幀 i 需估計的狀態為:

$$S_i = \{R_i, p_i, v_i, b_g, b_a\}$$

其中 R_i 為旋轉、 p_i 為位置(合計 6 自由度)、 v_i 為速度(3)、 b_g 與 b_a 為陀螺儀與加速度計零偏(各 3),共 15 個自由度——相較純視覺的 6 個自由度多出速度與零偏。單目慣性模式在初始化階段還額外要估尺度 s 與重力方向 R_{wg} 。

二、緊耦合的聯合最佳化目標

ORB-SLAM3 的視覺慣性束調整,同時最小化視覺與慣性兩類誤差:

$$\min \Sigma \rho(\|e_{\text{視覺}}\|^2_{\Sigma}) + \Sigma \|e_{\text{慣性}}\|^2_{\Sigma} + \|e_{\text{先驗}}\|^2$$

256. 視覺誤差 $e_{\text{視覺}}$:重投影誤差(第 20、21 章),要求地圖點投影位置與實際觀測相符。

257. 慣性誤差 $e_{\text{慣性}}$:IMU 預積分殘差(第 7.5 節),要求相鄰關鍵幀的位姿、速度變化與 IMU 量測一致。

258. 先驗誤差 $e_{\text{先驗}}$: 保留先前最佳化累積的資訊。

ρ 為穩健核函數。這條式子正是第 7.6 節「緊耦合」的具體形式,也是第 12 章最小平方框架的又一次應用——同學至此應能看出,整本書的數學是高度統一的。

三、IMU 初始化的三階段

圖 22-2 IMU 初始化流程

① 純視覺 MAP 估測

只用相機建立短時間的地圖與軌跡(單目尺度未知)

↓ 提供運動基準

② 純慣性 MAP 估測(固定視覺軌跡)

估計: 尺度 s 、重力方向、陀螺儀零偏、加速度計零偏、各幀速度

↓ 取得慣性參數初值

③ 聯合視覺慣性束調整

所有變數(位姿、速度、零偏、尺度、重力、地圖點)一起精修

↓

完成: 2 秒內尺度誤差約 5%, 精修後約 1%

【前提】必須有足夠的運動激勵, 否則尺度與零偏不可觀測

四、閉環與地圖合併的統一流程

圖 22-3 地點辨識觸發的兩種結果

建圖執行緒建立新關鍵幀

↓

地點辨識:向 DBoW2 資料庫查詢 Atlas 中的相似關鍵幀(高召回率)

↓

幾何驗證:多步驟驗證 + Sim(3) / SE(3) 對齊(RANSAC + Horn 方法)

↓ (追求 100% 精確率)

匹配幀屬於哪張地圖?

└─ 活躍地圖 → 【閉環】位姿圖優化 + 全域 BA(第 21 章)

└─ 其他地圖 → 【地圖合併】焊接視窗融合 + 位姿圖傳播修正

案例

案例 22-1 跟丟不再是終點:Atlas 的實測體驗

這是對比 ORB-SLAM2 與 ORB-SLAM3 最有戲劇性的實驗,強烈建議在課堂上並排演示。

實驗設計很簡單:讓兩套系統執行同一段影片,而這段影片中間有一段嚴重的視覺失效——例如相機被完全遮住十秒、或快速甩過一大片白牆(第 9 章的白牆詛咒)。

ORB-SLAM2 的表現:跟丟後嘗試重定位,但因為離開原區域太遠而失敗,系統就此停擺。畫面凍結,之後的影片內容完全沒有被利用——即使後面有精彩的場景,也全部浪費。

ORB-SLAM3 的表現:跟丟後短暫嘗試重定位,失敗後隨即開啟一張新地圖,繼續正

常建圖與追蹤,使用者幾乎察覺不到中斷。而當相機日後繞回舊區域,地點辨識偵測到跨地圖關聯,兩張地圖無縫合併——螢幕上會看到「兩段原本分離的軌跡忽然接在一起」的震撼畫面。

這個對比讓同學直觀理解「穩健度」在真實系統中的價值。實驗室裡的乾淨資料集上,兩者精度差距或許只是數字;但在真實世界的挑戰性場景中,差別是「能不能用」。這也呼應第 15 章 Cartographer 的觀察:產品化的系統,穩健度往往比極致精度更重要。

案例 22-2 兩秒鐘的價值:初始化速度為何是關鍵

IMU 初始化從十五秒縮短到兩秒,聽起來只是「快了一點」。但這十三秒的差距,在實務上是決定性的。

想像一位使用者打開手機的 AR 應用。若需要拿著手機晃動十五秒才能開始使用,絕大多數人會在第五秒就關掉 App——這個產品根本無法上市。而兩秒鐘的初始化,使用者通常還在調整握姿,幾乎無感。

對機器人也一樣。一台無人機起飛前若要在地上晃十五秒,不僅浪費時間,在某些場景(例如緊急救援)根本不可行;而兩秒的初始化,可以在起飛滑行的過程中順帶完成。

更深一層看,快速初始化還帶來一個間接好處:當系統跟丟並需要在新地圖上重新初始化時(22.5 節),快速初始化意味著中斷時間極短。若初始化要十五秒,Atlas 的「無縫切換」就名不副實了。這說明了系統設計中各項技術的相互支撐——初始化速度、多地圖架構、資料關聯,三者共同構成了 ORB-SLAM3 的穩健度。

對同學的啟示是:評估一項技術改進時,不要只看單一指標的數字變化,要問「這個改變讓什麼原本不可能的事變成可能?」——這才是技術價值的真正衡量。

案例 22-3 從 PTAM 到 ORB-SLAM3:一條血脈的二十年

ORB-SLAM3 不是憑空出現的,它是特徵法視覺 SLAM 這條技術血脈二十年演進的結晶。它是自 PTAM 與 ORB-SLAM 開始的特徵法 SLAM 血脈的集大成之作,發布時是當時最完整、最精確的開源 SLAM 函式庫。

259. PTAM(2007):首創「追蹤與建圖分離為兩個執行緒」的架構,讓即時視覺 SLAM 成為可能。這個雙執行緒思想,是後續所有系統的基礎。

260. ORB-SLAM(2015):以 ORB 特徵統一追蹤、建圖與地點辨識三項任務(第 21.1 節),加入閉環執行緒,並引入共視圖與本質圖(第 21.6 節)。

261. ORB-SLAM2(2017):擴展到雙目與 RGB-D,成為最廣泛使用的開源視覺 SLAM(第 21 章)。

262. ORB-SLAM3(2021):加入緊耦合視覺慣性與 Atlas 多地圖,並全面運用四種資料關聯。

觀察這條血脈,同學能學到一個重要的研究方法論:每一代都不是推翻前代,而是保留有效的架構、針對明確的弱點做突破。PTAM 的雙執行緒、ORB-SLAM 的共視圖與詞袋、ORB-SLAM2 的多相機支援——這些設計在 ORB-SLAM3 中全都還在。

這也給有志研究的同學一個務實的建議:與其追求「發明全新的東西」,不如深入理解一個成熟系統,找出它明確的弱點,提出針對性的改良。ORB-SLAM3 之所以能提出 Atlas,正是因為作者深知 ORB-SLAM2「跟丟即結束」的痛點。理解限制,才能突破限制。

實作活動

實作 22-1 ORB-SLAM3 視覺慣性 SLAM 實作

目的:執行 ORB-SLAM3 的多種模式,親身體驗 IMU 帶來的差異、Atlas 的跟丟恢復能力,並完成與 ORB-SLAM2 的對照實驗。這是第六篇的總結實作,也是第 29、30 章的重

要基礎。

第一部分:環境建置與基本執行(全體)

205. 編譯 ORB-SLAM3(相依套件與第 21 章類似:OpenCV、Eigen3、Pangolin; 注意版本相容性問題)。
206. 下載 EuRoC 資料集的一個序列(建議從 MH_01_easy 開始),先以純雙目模式執行,記錄軌跡與地圖。
207. 同一序列改以雙目慣性模式執行,比較兩者的軌跡、地圖點數量與系統穩定度。
208. 觀察初始化:單目慣性模式下,記錄從啟動到 IMU 初始化完成所需的時間與影像幀數,對照 22.4 節的「兩秒」資料。

第二部分:IMU 的價值(全體或大學部)

209. 挑戰序列:選一段快速運動的困難序列(如 EuRoC 的 V1_03_difficult 或 MH_05_difficult),分別以純視覺與視覺慣性模式執行,比較是否跟丟、跟丟幾次。
210. 尺度驗證:單目模式與單目慣性模式各執行一次,用 evo 評測時,前者需做尺度對齊、後者不需要——親身驗證 IMU 提供絕對尺度的價值(第 19.5 節)。
211. 運動激勵實驗:若使用自己的相機與 IMU,分別嘗試「靜止不動」「等速直線移動」「帶加減速與旋轉的移動」三種啟動方式,觀察哪一種能成功初始化,驗證 22.4 節的論述。

第三部分:Atlas 多地圖實驗(核心,大學部)

212. 人為製造跟丟:執行資料集時,以程式暫時中斷影像輸入數秒(或選用含嚴重遮擋的序列),觀察系統是否開啟新地圖。

213. 觀察 Atlas 狀態:在終端輸出或視覺化中,確認當前有幾張地圖、哪一張是活躍地圖。
214. 地圖合併:讓相機回到舊地圖涵蓋的區域,觀察合併發生的瞬間,並錄影記錄——這是本章最值得展示的畫面。
215. 對照實驗:對同一段含嚴重跟丟的序列,分別執行 ORB-SLAM2 與 ORB-SLAM3,比較最終軌跡的完整度,重現案例 22-1。
216. 多階段實驗:利用 EuRoC 同一場景的多個序列(如 MH_01 至 MH_05),依序執行並讓系統合併,觀察多階段作業如何逐步擴充地圖。

討論問題

104. 純雙目與雙目慣性的精度(ATE)差多少?在哪一種序列上差距最大?為什麼?
105. 困難序列中,純視覺跟丟了幾次?加入 IMU 後改善多少?對照第 7.6 節,IMU 的哪一項貢獻在此發揮了作用?
106. IMU 初始化在你的實驗中花了多久?若初始化時載體靜止不動,會發生什麼?這印證了 22.4 節的哪一項論述?
107. 地圖合併發生時,軌跡與地圖有什麼變化?這與第 21 章的閉環修正有何異同?(提示:22.6 節的統一流程)
108. (大學部)依 22.2 節的四種資料關聯,說明 ORB-SLAM3 相對 VINS-Mono 精度優勢的主要來源是哪一種關聯?為什麼它如此重要?

延伸挑戰(大學部)

其一:大學場域實測——用自備的雙目慣性相機(或含 IMU 的深度相機)錄製系館走廊與大廳的資料,執行 ORB-SLAM3 建圖。刻意設計一條「經過昏暗機房造成跟丟、再回到明亮走廊」的路線,測試 Atlas 的合併能力,並以第 13.7 節的方法評估地圖品質。其二:跨模態比較——在同一場域同時錄製視覺慣性與光達慣性資料,分別執行 ORB-SLAM3

與第 18 章的 LIO 系統,在明亮走廊、昏暗機房、玻璃帷幕大廳三種環境下比較兩者表現,量化驗證第 8、9 章的感測器互補論述。這個題目兼具教學與研究價值,適合作為專題主軸。

延伸閱讀

146. C. Campos, R. Elvira, J. J. G. Rodríguez, J. M. M. Montiel, and J. D. Tardós, "ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM," IEEE T-RO, 2021.(本章主要依據)
147. R. Elvira, J. D. Tardós, and J. M. M. Montiel, "ORB-SLAM-Atlas: A Robust and Accurate Multi-Map System," IROS, 2019.(Atlas 多地圖架構的前導工作)
148. C. Campos, J. M. M. Montiel, and J. D. Tardós, "Inertial-Only Optimization for Visual-Inertial Initialization," ICRA, 2020.(IMU 初始化三階段流程的理論來源)
149. C. Forster et al., "On-Manifold Preintegration for Real-Time Visual-Inertial Odometry," IEEE T-RO, 2017.(IMU 預積分,第 7.5 節的理論基礎)
150. ORB-SLAM3 開源專案(GitHub: UZ-SLAMLab/ORB_SLAM3).(原始程式碼、編譯說明與各模式執行範例)
151. T. Qin, P. Li, and S. Shen, "VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator," IEEE T-RO, 2018.(另一套經典視覺慣性系統,可與本章對照)
152. EuRoC MAV 與 TUM-VI 資料集官方網站.(本章實驗與論文評測所用的標準基準)

小結

119. ORB-SLAM3 有三項核心突破:以最大後驗估測為基礎的緊耦合視覺慣性

SLAM、Atlas 多地圖系統、以及在所有演算法階段全面運用四種資料關聯;支援單目 / 雙目 / RGB-D 與針孔 / 魚眼鏡頭的多種組態。

120. 四種資料關聯為短期(最近數秒)、中期(附近有輕微漂移的元素)、長期(以地點

辨識匹配先前造訪區域)與多地圖(跨建圖階段);中期關聯是精度優勢的關鍵來源,使其在 EuRoC 單目慣性單階段運作中精度達 VINS-Mono 的 2.6 倍。

121. 視覺慣性模式下狀態向量從純視覺的 6 自由度擴充為 15 自由度(加上速度與

兩組零偏);IMU 帶來絕對尺度、重力方向、視覺失效時的支撐與高頻輸出,而視覺回報以不漂移的觀測校正 IMU 零偏。

122. IMU 初始化採三階段:純視覺 MAP 估測 → 固定視覺軌跡的純慣性 MAP 估測

→ 聯合視覺慣性 BA;2 秒內達 5% 尺度誤差、精修後約 1%,但必須有足夠的運動激勵,否則尺度與零偏不可觀測。

123. Atlas 由一張活躍地圖與多張非活躍地圖組成,共用以詞袋為基礎的關鍵幀資

料庫;跟丟時開啟新地圖繼續運作,日後偵測到跨地圖關聯時執行無縫合併——不同於其他方案把子地圖保留為分離實體。

124. 地點辨識對每個新關鍵幀查詢 Atlas 中的相似幀以求高召回率,再以多步驟幾

何驗證與 $\text{Sim}(3)$ / $\text{SE}(3)$ 對齊確保百分之百精確率;匹配屬活躍地圖則閉環,否則合併。合併分焊接視窗融合與位姿圖傳播兩階段,以免阻塞追蹤。

125. 視覺慣性 SLAM 廣泛應用於手機 AR、穿戴式相機與無人機;在類似 AR / VR

的 TUM-VI 房間序列中,雙目慣性平均 ATE 為 0.009 公尺、單目慣性為 0.011 公尺,達到毫米級精度。

第六篇的核心系統至此完整介紹。但 SLAM 本身不是目的——機器人建了地圖、知道自己在哪之後,要拿它來做什麼?下一章我們把視野從「定位建圖」擴展到「導航

與任務」:視覺導航與 SLAM 有何不同?線追蹤、視覺伺服、AprilTag 導航如何運作?YOLO 與 VLM 如何讓機器人理解「請去教務處」這樣的指令?機器狗與校園巡檢機器人如何整合這一切?第 23 章將把本書的技術與真實的機器人任務串接起來。

台灣自編大學教科書

@prof

《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第 23 章

視覺導航機器人

編者：李世淵 (機器人叫獸)

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

學習目標

前四章我們建立了完整的視覺 SLAM 能力:機器人能知道自己在哪、環境長什麼樣。但 SLAM 本身不是目的——建了地圖之後,要拿它來做什麼?本章是第六篇的最後一章,把技術與真實的機器人任務串接起來,從最樸素的線追蹤,一路走到能聽懂人話的 VLM 導航。學完本章之後,同學應能夠達成下列目標:

145. 說明視覺導航與視覺 SLAM 的差異與互補關係。
146. 比較線追蹤、地標導航與視覺伺服三種基本視覺導航策略。
147. 說明 AprilTag / ArUco 在機器人導航中的四種典型用途。
148. 說明 YOLO 物件辨識如何支援語意導航,並理解其在機器人視角下的限制。
149. 說明 VLM 視覺語言模型如何讓機器人理解自然語言導航指令。
150. 說明機器狗視覺導航的特殊挑戰與對策。
151. 規劃一個校園巡檢或資源回收辨識機器人的系統架構。

核心概念

中文名詞	英文名詞	概念說明
視覺導航	Visual Navigation	以視覺資訊引導機器人移動到目標的技術。
線追蹤	Line Following	沿地面標線行進的最基本視覺導航方式。
視覺伺服	Visual Servoing	以影像回授直接控制機器人運動的方法。
影像式視覺伺服	IBVS	直接以影像特徵誤差作為控制訊號。
位置式視覺伺服	PBVS	先解出三維位姿誤差再控制。
地標導航	Landmark Navigation	依據可辨識的地標決定行進方向。

中文名詞	英文名詞	概念說明
語意導航	Semantic Navigation	依物體與場景意義而非幾何座標導航。
物件目標導航	Object-Goal Navigation	接受「去找某個物體」的任務指令。
視覺語言導航	VLN	依自然語言指令在環境中導航。
視覺語言模型	VLM	能同時理解影像與語言的多模態 AI 模型。
視覺語言動作模型	VLA	直接由影像與語言產生動作指令的模型。
邊界探索	Frontier Exploration	朝已知與未知交界處前進以探索環境。
零樣本	Zero-Shot	未經任務特定訓練即可執行新任務的能力。

23.1 視覺導航與 SLAM 的差異

這是本章第一個必須釐清的觀念。許多同學把「視覺 SLAM」與「視覺導航」混為一談,但兩者回答的是不同的問題。

兩個不同的問題

- 視覺 SLAM 回答的是:「我在哪裡?環境長什麼樣?」它產出的是位姿與地圖——這是第 19 至 22 章的主題。
- 視覺導航回答的是:「我該往哪走?」它產出的是運動指令——這是本章的主題。

用第 3 章的導航三問來對照:SLAM 專門解決第一問(我在哪裡),而視覺導航則涵蓋第二、三問(我要去哪裡、我該怎麼去)。

視覺導航不一定需要 SLAM

這是一個重要但常被忽略的事實。有些視覺導航方法完全不建地圖:

- 線追蹤機器人只看地上的線,不知道自己在哪,也不需要知道(23.2 節)。
- 視覺伺服直接把「影像中目標的偏移」轉成馬達指令,中間不經過任何地圖。
- 依循視覺地標的走廊機器人,只需認得「看到滅火器就右轉」,不需全域座標。

這些方法屬於第 3 章介紹的反應式架構——感知直接驅動行為,不建模、不規劃。它們簡單、快速、穩健,在受控環境中非常實用。

SLAM 讓導航能力升級

但反應式導航有明顯的天花板:無法規劃全域路徑、無法處理「去一個沒有標線通往的地方」、無法在中途改變目標。加上 SLAM 之後,機器人才具備:

263. 全域路徑規劃:在地圖上算出最佳路線(第 3、28 章)。

264. 任意目標導航:能前往地圖上任何座標,而非只能沿預設路線。
265. 斷點恢復:跟丟後能重定位並繼續任務(第 21、22 章)。
266. 語意任務:結合語意地圖,能理解「去教務處」這種指令(23.5 節)。

比較項目	視覺 SLAM	視覺導航
核心問題	我在哪?環境長什麼樣?	我該往哪走?
輸出	位姿與地圖	運動指令(速度、方向)
是否需地圖	產出地圖	不一定需要
架構類型	審議式(需建模)	可反應式,也可審議式
對應導航三問	第一問	第二、三問
本書章節	第 19–22 章	本章、第 28 章

表 23-1 視覺 SLAM 與視覺導航的對照

實務系統多半兩者並用:以 SLAM 提供全域定位與規劃,以反應式視覺導航處理即時避障與精確對接。這正是第 3 章「混合式架構」的體現。

23.2 線追蹤、地標導航、視覺伺服

本節介紹三種不需要完整 SLAM 的基本視覺導航策略,由簡到繁。

一、線追蹤(Line Following)

這是最樸素、也是無數同學第一個做出來的機器人:在地面貼一條黑色膠帶,機器人以向下的相機或紅外線感測器偵測線的位置,若線偏左就左轉、偏右就右轉。

- 原理:把影像轉為灰階或二值化(影像處理的基本功),找出線的中心與畫面中心的偏差,轉為轉向指令。

- 優點:成本極低、程式極簡、極為穩健、不需地圖與定位。
- 限制:只能走預設路線、無法繞開障礙、環境必須改造(貼線)。
- 實務應用:工廠 AGV 的磁條或色帶導引至今仍廣泛使用——在可控環境中,它便宜可靠得無可取代(呼應第 4 章亞馬遜倉儲案例的觀念:環境越可控,技術選型越能簡化)。

二、地標導航(Landmark Navigation)

比線追蹤進階一階:不鋪連續的線,而是在關鍵位置放置可辨識的地標,機器人「看到地標 A 就右轉、看到地標 B 就停」。

地標可以是人工的(第 4.6 節的 AprilTag、QR Code、色塊)或自然的(門、滅火器、消防栓)。前者可靠但需改造環境,後者不需改造但辨識較難——這正是 YOLO 等 AI 辨識技術的用武之地(23.4 節)。

地標導航本質上建立了一張拓樸地圖(第 3 章):節點是地標,邊是「從這個地標到下一個地標的行進動作」。它不需要精確的座標,卻能完成相當複雜的路線任務。

三、視覺伺服(Visual Servoing)

視覺伺服是三者中最精密的,它把相機直接放進控制迴路:持續比較「當前影像」與「目標影像」的差異,並產生運動指令以縮小差異,直到兩者一致。

它有兩大流派:

- 影像式視覺伺服(IBVS):直接以影像上的特徵誤差(例如目標點應該在畫面中央,現在偏左 30 像素)作為控制訊號。優點是不需重建三維資訊、對相機校正誤差不敏感;缺點是機器人的實際運動軌跡可能不直觀。

- 位置式視覺伺服(PBVS):先由影像解出「相機相對目標的三維位姿誤差」(用第 20 章的 PnP),再據此規劃運動。優點是運動軌跡在三維空間中合理;缺點是需要準確的相機校正與目標模型。

視覺伺服的典型應用是精確對接:機器人手臂抓取物件、掃地機器人回充電座對位、無人機精準降落。它能達到公釐級的相對定位精度——遠高於 SLAM 的全域定位精度,因為它只關心「相對於眼前目標」的位置。

方法	需要環境改造	精度	彈性	典型應用
線追蹤	需要(貼線)	中	低(固定路線)	工廠 AGV、教學機器人
地標導航	視地標而定	中	中(拓樸路線)	走廊巡邏、展場導覽
視覺伺服	不需要	極高(公釐級)	低(僅對目標)	精確對接、抓取、降落

表 23-2 三種基本視覺導航策略比較

23.3 AprilTag / ArUco 導航

第 4.6 節介紹過視覺基準標記的原理——由四個角點與相機內參解出完整 6-DoF 位姿(第 20 章的 PnP 問題)。本節聚焦它們在導航任務中的實際用法。

四種典型用途

267. 絕對位姿校正:在走廊、電梯口、實驗室門口貼標記。機器人只要看到任一標記,就能取得不漂移的絕對位姿,一舉消除 SLAM 的累積誤差。這是最常見也最實用的用法(第 4.7 節的融合思想)。
268. 精確對接:充電站、取放料點、電梯按鈕前貼標記,配合視覺伺服(23.2 節)完成公釐級的最後對位。SLAM 的定位精度通常在數公分,不足以應付需要精密對接的動作。

269. 樓層與區域識別:第 5.7 節提過,光達 SLAM 無法分辨長得一樣的不同樓層。在每層樓的電梯口貼上不同 ID 的標記,問題迎刃而解。

270. SLAM 精度評估:在場域中密集布置已知座標的標記,可作為近似的真值軌跡,用來評測 SLAM 系統的精度(第 21.7 節的 ATE 評測需要真值)。這對沒有動作捕捉系統的教學實驗室極為實用。

實作要點

- 標記尺寸與偵測距離:偵測距離大致與標記邊長成正比。走廊用的標記若太小,機器人要走到很近才看得到,失去校正的意義。實務上會依需求製作不同尺寸。
- 安裝高度與角度:標記應貼在機器人相機視野的合理高度,並盡量正對走道。斜視角度過大時,四個角點的偵測精度下降,解出的位姿誤差變大。
- 光線與反光:標記應避免直接反光(護貝的標記在燈光下可能完全無法辨識)。建議用消光材質列印。
- 唯一性管理:每個標記的 ID 與其世界座標必須建檔管理。這份「標記地圖」本身就是一種簡易的語意地圖(第 3 章)。

與 SLAM 的融合層次

第 4.7 節提過三種融合層次,在此具體化:

- 鬆耦合:SLAM 正常運作,只在看到標記時把位姿「拉」到標記給出的值。實作最簡單,適合教學。
- 緊耦合:把標記觀測當作約束加入 SLAM 的後端最佳化(第 12 章的一條邊)。精度最佳,是研究級做法。
- 輔助初始化與重定位:最務實——SLAM 平時自主運作,只在啟動時或跟丟時用標記提供絕對參考。以極低成本解決最頭痛的問題。

23.4 YOLO 物件辨識與語意導航

前三節的導航,機器人都不「理解」它看到的東西——線就是線、標記就是編號。加入物件辨識之後,機器人才開始理解場景的意義。

從幾何到語意

第 1、3 章提過語意地圖的概念:在幾何地圖之上,標註「這是門、那是樓梯、那邊是滅火器」。YOLO 這類即時物件偵測模型,正是產生這些標註的工具。

有了語意,導航任務的表達方式就徹底改變了:

- 幾何導航:「前往座標(12.5, 8.3)」——使用者必須知道座標。
- 語意導航:「前往最近的飲水機」——使用者只需說出物體名稱。

後者才是人類自然的溝通方式,也是服務機器人真正可用的前提。

YOLO 在導航中的三種角色

271. 語意標註:偵測到的物體連同其三維位置(由 RGB-D 或雙目提供深度)一起寫入地圖,形成語意地圖。
272. 動態物體處理:辨識出「人」「車」等會移動的物體,在 SLAM 中把這些區域的特徵剔除,避免動態物體干擾位姿估計(第 9.5 節提過的視覺 SLAM 弱點)。這是一個極為實用的技巧。
273. 任務目標偵測:巡檢機器人辨識儀表、裂縫、垃圾;資源回收機器人辨識瓶罐種類——這是任務本身的核心功能。

機器人視角的挑戰

這裡有一個容易被忽略、但在實務上極為關鍵的問題:大多數視覺模型是用「人類視角」的照片訓練的,而機器人的視角往往截然不同。

研究指出,四足機器人的自主導航需要穩健的語意感知,然而由於這類平台在低於 50 公分的高度運作,以人類中心視角訓練的視覺模型往往難以良好運作。

這個問題對掃地機器人、機器狗、小型輪型機器人都成立:從 30 公分高往上看的椅子、從地面看到的門,與訓練資料中的照片差異很大,辨識率因此下降。

- 對策一:蒐集機器人視角的資料重新訓練或微調模型——這正是第 31 章實作 31-1「校園物件辨識資料集」的用意。
- 對策二:使用具備開放詞彙能力的較新模型,它們的泛化能力較強。
- 對策三:調整相機安裝高度與俯仰角,盡量接近訓練資料的視角。

這個案例再次印證本書反覆強調的觀念:技術的效能高度依賴部署情境,不能只看論文上的數字。

23.5 VLM 視覺語言模型輔助導航

YOLO 讓機器人認得物體,但它仍然只能回答「畫面中有什麼」。VLM(視覺語言模型)則更進一步——它能理解場景、推理常識、並接受自然語言指令。這是機器人導航正在發生的典範轉移。

VLM 帶來什麼

VLM 在大規模影像與文字語料上預訓練,展現出統合視覺與語言模態的強大能力,對需要豐富語意感知的導航任務日益重要。早期模型把影像與文字特徵對齊到共同的嵌入空間,而 CLIP 引入對比學習實現開放詞彙的視覺辨識,大幅提升多模態表徵的泛化能力;近期的 VLM 更進一步提供視覺語言推理與決策支援的統一介面。

對導航而言,VLM 的關鍵價值在於常識推理。舉例來說,若指令是「去找洗手台」,機器人不需事先標註過洗手台的位置——VLM 知道洗手台通常在廁所或廚房,因此會

優先探索這類區域。這種能力稱為語意先驗。

三種應用模式

274. 視覺語言導航(VLN):接受自然語言指令,如「走到走廊盡頭,右轉後在第二扇門停下」。系統需理解指令、對應到當前視野、逐步執行。

275. 物件目標導航:接受「去找一個垃圾桶」這類目標,系統需在未知環境中探索並找到目標。VLM 的常識推理讓探索更有效率。

276. 社交感知導航:理解場景中的人與情境,決定合宜的行為(不從人群中硬穿、在教室外放慢速度)。

零樣本能力:不需任務特定訓練

VLM 導航最具突破性的特質,是零樣本能力——不需要為每個新環境或新任務重新訓練。

相關研究以 VLM 作為全域規劃器,把多個機器人不同視角的子地圖聚合為統一的全域地圖,編碼機器人狀態與邊界區域;VLM 據此在機器人之間分配邊界,促成協調而高效的探索。實驗顯示這類方法在成功率與導航效率上優於既有基準,且不需要任務特定的訓練;消融實驗進一步證實了 VLM 語意先驗的重要性。

這與傳統的學習式導航形成鮮明對比:後者往往需要在目標環境中蒐集大量資料訓練,換個場域就要重來。

前沿方向:視覺語言動作模型(VLA)

更新的發展是 VLA(Vision-Language-Action)模型——不只理解,而是直接由影像與語言產生動作指令。

近期研究提出能實現全模態目標條件化的訓練框架:以高容量的視覺語言動作骨幹,同時以 2D 位姿、第一人稱影像與自然語言三種目標模態(以及它們的組合)進行訓練。

這呼應了人類的能力——我們能靈活地解讀並組合不同形式的目標指定,無論是語言指令、空間座標還是視覺參照。

另有研究展示同一套代理核心能部署到四種異質的真實機器人上——雙臂輪式平台、四足機器人、人形機器人與自建的四旋翼無人機——只需更換底層控制器。這種跨平台的通用性,是機器人基礎模型(Robotics Foundation Model)願景的具體展現。

實務限制:別忘了工程現實

VLM 導航令人興奮,但同學務必認識它的限制:

- 運算負擔:大型 VLM 推論耗時,難在機器人上即時運行。因此出現了各種效率最佳化研究,例如以注意力為基礎的權杖重要性評估與查詢引導的時空過濾,在不重新訓練或修改預訓練模型的前提下保留導航相關資訊,並在真實的四足機器人上驗證了可靠且低延遲的指令跟隨導航。
- 幻覺風險:VLM 可能「看到」不存在的東西或做出錯誤推理。若機器人完全信任它,可能撞牆或走錯路。實務系統必須保留幾何層的安全檢查(第 28 章的避障)。
- 延遲與安全:VLM 適合做高階決策(去哪裡),不適合做即時控制(閃避突然出現的障礙)。分層架構是必然選擇——這正是第 3 章混合式架構的現代版本。

這些限制正是第 31 章要深入討論的主題。同學此刻只需建立一個核心認知:VLM 讓機器人「聽得懂、想得通」,但「走得穩、不撞人」仍然要靠前面二十二章的幾何與控制技術。兩者缺一不可。

23.6 機器狗視覺導航

四足機器人(機器狗)近年成為視覺導航研究與應用的熱門平台。本節說明它的特殊挑戰與對策。

為什麼選機器狗

- 地形適應力:能上下樓梯、跨越門檻與碎石、走過不平地面——這是輪型機器人做不到的(第 3.6 節)。
- 校園與工廠場域適用:大學這類有樓梯、斜坡、戶外通道的場域,機器狗的通行能力明顯優於輪型。
- 平台成熟:商用四足機器人已相當普及,並提供開放的開發介面,適合作為研究與教學平台。

四項特殊挑戰

277. 步態震動:四足行走時機身持續上下起伏與左右擺動,造成影像模糊與劇烈的位姿變化。這對視覺 SLAM 是嚴峻考驗——這正是第 22 章 IMU 緊耦合不可或缺的原因,也呼應第 18 章機器狗需要光達慣性緊耦合的論述。
278. 低視角問題:如 23.4 節所述,機器狗在低於 50 公分的高度運作,以人類中心視角訓練的視覺模型往往難以良好運作。語意感知的準確度因此下降。
279. 三維地形理解:輪型機器人只需知道「前方能不能走」,機器狗還需要知道「這個台階多高、踏得上去嗎、落腳點穩不穩」。導航從 2D 升級為 2.5D 甚至完整 3D(第 3.6 節)。
280. 續航與運算限制:四足行走的能耗遠高於輪型,可搭載的運算平台也受限。這使得輕量化的演算法與邊緣運算格外重要(第七篇的主題)。

對策與系統設計

- 感測器組合:3D 光達或深度相機(地形) + IMU(姿態與震動補償) + RGB 相機(語意)。多感測器融合在此不是選項而是必需。
- 緊耦合估測:以第 18 章的 LIO 或第 22 章的視覺慣性 SLAM 為骨幹,IMU 是穩定度的關鍵。

- 分層控制:上層做語意決策與路徑規劃(可用 VLM),中層做地形評估與落腳點規劃,底層做步態控制。各層時間尺度不同,這是第 3 章混合式架構的典型實現。
- 語意模型的在地化:蒐集機器狗視角的資料微調模型,是提升辨識率最直接的方法。

研究前沿

機器狗已成為驗證前沿導航技術的標準平台。近年的視覺語言導航研究,普遍會在真實的四足機器人上驗證方法的實用性——因為它同時具備真實世界的複雜度與可控的實驗條件。已有研究把專為腿式機器人設計的視覺語言動作導航模型作為代表性工作提出,顯示這個方向的活躍程度。

23.7 校園巡檢與資源回收辨識機器人

本節把本章技術整合成兩個具體的專題規劃,作為第六篇的收尾,也為第 30 章的專題預作準備。

專題一:校園巡檢機器人

這是第 5.7 節、第 18.7 節都提過的主軸,本節從視覺導航的角度完整規劃。

圖 23-1 校園巡檢機器人的系統架構

【感知層】RGB-D 或雙目相機 + IMU + (可選)光達、GNSS

↓

【定位建圖】ORB-SLAM3 視覺慣性 SLAM(第 22 章)

+ AprilTag 絕對位姿校正(23.3 節)

↓

【語意理解】YOLO 物件偵測 → 語意地圖(23.4 節)

↓

【任務決策】巡檢點清單 / VLM 自然語言任務(23.5 節)

↓

【路徑規劃】全域規劃(地圖) + 局部避障(即時感知)

↓

【執行與紀錄】移動至巡檢點 → 拍照 / 量測 → 標上三維座標存檔

巡檢任務的視覺重點

- 儀表判讀:偵測並讀取壓力表、電表、溫度計的數值。這需要精確對位(視覺伺服,23.2 節)加上數字辨識。
- 異常偵測:裂縫、漏水痕跡、鏽蝕、雜物堆放——這些多半只有視覺看得出來(第 8 章提過光達給不了語意)。
- 變化偵測:比較不同時間的同一位置影像,找出「昨天沒有、今天出現」的異常。這需要精確的重複定位能力(第 22 章 Atlas 的多階段作業正好適用)。
- 門禁與安全:偵測門是否關好、消防通道是否堵塞、滅火器是否在位。

專題二:資源回收辨識機器人

這是一個兼具技術挑戰與教育意義的題目,特別適合高中與大學部合作。

- 任務定義:機器人在校園中巡行,辨識並記錄(或收集)可回收物,分類為寶特瓶、鋁罐、紙類、一般垃圾。
- 視覺挑戰:物體姿態多變(躺著、立著、壓扁的)、材質反光(鋁罐)、透明(寶特瓶,這對深度相機也是難題,呼應第 9 章的限制)、光線多變(戶外)。

- 資料集建置:這是專題的核心工作——在校園實地拍攝各種角度、光線、背景下的回收物,建立標註資料集,訓練 YOLO 模型。這正是第 31 章實作 31-1 的方法。
- 系統整合:SLAM 提供位置(知道在哪裡發現的)、YOLO 提供辨識、導航讓機器人自主巡行、資料庫記錄成果。

教學層次的規劃

階段	內容	適合對象
一	線追蹤與 AprilTag 導航,理解基本視覺導航	高中
二	訓練 YOLO 辨識校園物件,建立資料集	高中進階 / 大學部
三	SLAM 建圖 + 語意標註,產生語意地圖	大學部
四	自主巡檢:規劃路線、執行任務、產出報告	大學部專題
五	VLM 自然語言任務、跨階段變化偵測	專題 / 研究生

表 23-3 視覺導航專題的五個教學階段

這個層次規劃的優點是:每個階段都有可展示的成果,學生不必等到最後才看到東西動起來;而且各階段技術逐步疊加,前面的成果不會白費。

至此,第六篇「Visual SLAM 與視覺導航機器人」全部完成。我們從第 19 章的特徵基礎出發,經過第 20 章的幾何、第 21 章的 ORB-SLAM2、第 22 章的 ORB-SLAM3,到本章把技術落實為真實任務。下一篇,我們把視野轉向極端受限的平台——微型機器人與無人機。當運算資源只有毫瓦等級、當載重只有數十公克時,SLAM 該如何實現? 第七篇「微型機器人、無人機與邊緣 AI SLAM」將帶來截然不同的思考。

公式或流程

一、線追蹤的比例控制

設影像寬度為 W 、偵測到的線中心像素為 x_line 、畫面中心為 $x_center = W / 2$ ，則橫向偏差為：

$$e = x_line - x_center$$

最基本的比例控制器直接把偏差轉為角速度指令：

$$\omega = -K_p \cdot e$$

其中 K_p 為比例增益。負號表示線偏右(e 為正)時應右轉。實務上常加入微分項以抑制震盪(PD 控制)，或依偏差大小調整線速度——偏差大時放慢、走直線時加速。這是最簡單的視覺回授控制，也是同學理解視覺伺服的最佳起點。

二、影像式視覺伺服的控制律

IBVS 的目標是讓當前影像特徵 s 逼近目標特徵 s^* 。定義特徵誤差：

$$e = s - s^*$$

控制律為(以指數方式收斂誤差)：

$$v = -\lambda \cdot L^+ \cdot e$$

其中 v 是相機的六自由度速度指令、 λ 是收斂速率、 L 是影像雅可比矩陣(描述「相機動一單位，影像特徵會移動多少」)、 L^+ 為其偽反矩陣。這條式子的精神與第 11、12 章的最佳化一脈相承：定義誤差、求出誤差對控制量的敏感度、沿誤差下降方向移動。

三、語意導航的目標轉換

語意導航的核心是把「語意目標」轉為「幾何目標」：

圖 23-2 從語言指令到運動指令的完整流程

使用者指令:「請去最近的飲水機」

↓

① 語言理解(VLM / LLM):解析出目標物件類別 = 飲水機

↓

② 語意地圖查詢:地圖中是否已標註飲水機?

└ 有 → 取得其三維座標,直接進入 ④

└ 無 → 進入 ③ 探索

↓

③ 語意探索:以 VLM 常識推理選擇有希望的探索方向

(飲水機通常在走廊、茶水間附近)→ 邊界探索 + YOLO 偵測

↓

④ 幾何路徑規劃:在地圖上算出到目標的路徑(第 3、28 章)

↓

⑤ 局部避障與運動控制:即時感知,輸出速度指令

↓

⑥ 抵達後精確對接(視覺伺服,如需要)

這條流程清楚呈現了本書的整體架構:上層由 AI 理解語意,中層由 SLAM 與規劃提供幾何,底層由控制執行動作。三層缺一不可——這也是 23.5 節末尾強調的核心認知。

四、AprilTag 位姿校正的融合

當機器人偵測到已知世界座標的標記時,可由標記推算自身位姿:

$$T_{world \leftarrow robot} = T_{world \leftarrow tag} \cdot T_{tag \leftarrow camera} \cdot T_{camera \leftarrow robot}$$

其中 $T_{world \leftarrow tag}$ 由標記地圖查得(已知)、 $T_{tag \leftarrow camera}$ 由 PnP 解出(第 20 章)、 $T_{camera \leftarrow robot}$ 為外參(第 10 章的相機安裝位置)。三個變換相乘即得機器人在世界座標中的位姿——這正是第 10 章「變換組合就是矩陣連乘」的直接應用。

案例

案例 23-1 從貼一條線到聽懂人話:視覺導航的四十年

把本章的技術依時間排列,會看到一條清晰的演進脈絡,而且每一步都對應著「機器人理解世界的深度」在提升。

- 1980 年代——線追蹤:機器人只認得亮度差異,環境必須為它改造(貼線)。它不知道自己在哪、也不知道要去哪,只是不斷修正偏差。
- 2000 年代——地標與視覺伺服:機器人開始能辨識特定圖案,並精確地相對定位。環境改造需求降低(只需貼標記),但仍需改造。
- 2010 年代——視覺 SLAM:機器人能自行建立地圖並定位,不再需要改造環境。但它理解的是「幾何」,不是「意義」。
- 2020 年代——語意與 VLM 導航:機器人開始理解「那是門、那是飲水機」,並能接受自然語言指令。它終於能用人類的方式溝通。

觀察這條脈絡,同學會發現一個有趣的規律:每一代技術降低的,都是「人類為機器人做的準備工作」。從改造環境、到標註地標、到只需下達指令——機器人越來越能適應人類的世界,而非要求人類遷就它。

但這裡有一個重要提醒:新技術並未淘汰舊技術。今天的工廠 AGV 仍大量使用磁

條導引,因為在可控環境中它最可靠、最便宜;精確對接仍需視覺伺服,因為 SLAM 的精度不夠。工程師的任務不是永遠用最新的技術,而是為每個問題選擇最合適的技術——這是本書從第 8 章開始就反覆強調的判斷力。

案例 23-2 低視角的困境:當模型不認識機器人眼中的世界

這是一個看似技術細節、實則影響深遠的問題,也是同學做專題時最容易踩到的坑。

狀況是這樣的:同學下載了一個在標準資料集上表現優異的物件偵測模型,興沖沖裝到機器狗或掃地機器人上,結果辨識率慘不忍睹——明明是椅子,模型卻認不出來;明明前方是門,模型標成了牆。

原因如研究所指出:四足機器人在低於 50 公分的高度運作,而以人類中心視角訓練的視覺模型往往難以良好運作。訓練資料中的椅子照片,多半是站著的人以約 150 公分高度拍攝的;從 30 公分高往上看,椅子的形狀、透視、遮蔽關係都完全不同。

這個問題還有幾個變形:掃地機器人看到的是家具的下半部與地板紋理;無人機看到的是物體的頂視圖;水下機器人面對的是色偏嚴重的影像。共同點是:部署情境與訓練情境的分布不一致。

解法沒有捷徑——蒐集目標視角的資料重新訓練或微調。這聽起來麻煩,但正是第 31 章實作的核心,也是專題最有價值的部分:一份「大學機器人視角的校園物件資料集」,本身就是可發表、可延續的研究成果。

這個案例給同學的更深啟示是:AI 模型不是萬用的黑盒子。理解它的訓練資料分布與部署情境的差異,是負責任地使用 AI 的基本功。

案例 23-3 多機器人協作探索:VLM 作為全域規劃器

本章 23.5 節提到的一項研究,值得單獨展開,因為它展示了 VLM 導航的完整潛力。

該研究提出的框架把視覺語言模型整合為全域規劃器,實現具備常識推理的多機器人視覺目標導航。它把多個機器人不同視角的子地圖聚合為統一的全域地圖,編碼機器人狀態與邊界區域;VLM 據此在機器人之間分配邊界,促成協調而高效的探索。

這個設計精彩之處在於:VLM 不是用來做低階控制,而是扮演「指揮官」的角色。它看著全域地圖,運用常識判斷「這幾個未探索方向中,哪一個最可能找到目標」,再把任務分配給不同的機器人。低階的移動與避障,仍由傳統的規劃與控制負責。

實驗結果顯示,這個方法在成功率與導航效率上都優於既有基準,且不需要任務特定的訓練;消融實驗進一步證實了 VLM 語意先驗的重要性。研究團隊也在真實場景中以四足機器人驗證了這個框架。

對大學場域的啟發是:這正是「多台機器狗校園巡檢」專題(第 26、30 章)的技術路徑。多台機器人分頭探索、VLM 統籌分配、各自以 SLAM 定位建圖、最後合併成完整的校園語意地圖——這是一個層次豐富、可分工進行、且與國際前沿接軌的專題題目。

實作活動

實作 23-1 從線追蹤到語意導航的階梯式實作

目的:依照 23.7 節的五階段規劃,逐步建立視覺導航能力。本實作可依班級程度與資源選擇進行到哪一階段,每一階段都有可展示的成果。

階段一:線追蹤(高中適用,不需 SLAM)

217. 準備一台附相機的教學小車(或在模擬環境中),在地面貼出一條含直線與轉彎的路線。
218. 以 OpenCV 讀取影像、轉灰階、二值化,找出線的中心位置。
219. 依公式一實作比例控制器,把橫向偏差轉為轉向指令,讓小車自動循線。

220. 參數實驗:調整比例增益 K_p ,觀察太小(反應遲鈍、出彎道)與太大(左右震盪)的現象,找出合適值。

221. 挑戰:在路線中加入十字交叉或分岔,設計規則讓小車做出選擇。

階段二:AprilTag 導航與位姿校正(高中進階 / 大學部)

222. 在教室四周貼上數個 AprilTag,量測並記錄每個標記的世界座標(建立標記地圖)。

223. 用第 9 章校正過的相機偵測標記,以 PnP 解出相機相對標記的位姿(第 20 章)。

224. 依公式四,把三個變換相乘,算出機器人在世界座標中的位姿,並即時顯示。

225. 與 SLAM 對照:同時執行視覺 SLAM,把兩條軌跡畫在一起——觀察 SLAM 軌跡平滑但漸漸偏離、標記軌跡跳動但始終正確(重現第 4 章案例 4-3)。

226. 簡易融合:實作鬆耦合校正——平時用 SLAM,每當看到標記就把位姿拉回標記給出的值,觀察漂移是否被抑制。

階段三:YOLO 語意標註(大學部)

227. 蒐集資料:用機器人視角的相機(注意安裝高度)在校園拍攝目標物件——門、飲水機、滅火器、垃圾桶、樓梯口,每類至少 200 張,涵蓋不同角度與光線。

228. 標註並訓練 YOLO 模型,在驗證集上評估準確率。

229. 視角對照實驗:另外拍一組「人類視角」(約 150 公分高)的測試影像,比較模型在兩種視角下的辨識率,量化重現案例 23-2 的現象。

230. 語意標註:把 YOLO 偵測結果與 RGB-D 深度結合,計算物件的三維位置,標註到 SLAM 建好的地圖上,產生語意地圖。

階段四:語意導航任務(大學部專題)

231. 建立語意地圖後,實作一個簡單的語意導航:輸入物件名稱(如「飲水機」),系統查詢地圖中該物件的座標,規劃路徑並導航前往。
232. 加入動態物體處理:用 YOLO 偵測「人」,在 SLAM 中剔除人身上的特徵點,觀察定位穩定度是否改善(23.4 節的第二種角色)。
233. 實作巡檢任務:設定數個巡檢點,讓機器人依序前往、拍照記錄、標上三維座標,最後產出巡檢報告。

階段五:VLM 輔助(專題 / 研究生)

234. 把機器人當前影像送入 VLM,以自然語言提問:「前方是否可通行?」「畫面中有樓梯嗎?」「哪裡可能是門?」記錄回答的正確率。
235. 比較 VLM 與 YOLO:在同一批影像上,比較兩者對「開放詞彙物件」(訓練時沒見過的類別)的辨識能力。
236. 延遲量測:記錄 VLM 推論的耗時,評估它適合放在系統的哪一層(高階決策?即時控制?),對照 23.5 節的限制討論。

討論問題

109. 階段一中,比例增益太大與太小各造成什麼現象?這與控制理論的哪些概念相關?
110. 階段二中,SLAM 軌跡與 AprilTag 軌跡的差異隨時間如何變化?加入校正後改善多少?
111. 階段三的視角對照實驗中,兩種視角的辨識率差多少?這對「直接套用網路上的預訓練模型」有什麼警示?
112. 階段四中,剔除人身上的特徵點後,定位穩定度改善了嗎?若場景中人很多(如下課時間走廊),系統還能運作嗎?

113. (專題)VLM 的推論延遲是多少?若機器人以每秒 0.5 公尺移動,這個延遲代表機器人在「思考」期間移動了多遠?這對安全設計有什麼意涵?

延伸挑戰(大學部)

其一:資源回收辨識機器人——依 23.7 節專題二的規劃,建立校園回收物資料集(特別注意透明寶特瓶與反光鋁罐的拍攝),訓練辨識模型,並整合 SLAM 讓機器人記錄「在哪裡發現了什麼」。這個題目同時涵蓋 AI、SLAM 與環境教育,展示效果佳。其二:多機器人協作探索的簡化版——用兩台機器人(或兩個模擬實例)分頭探索同一場域,各自建圖後合併(可用第 22 章 Atlas 的概念),並比較「協作探索」與「單機探索」完成覆蓋所需的時間,呼應案例 23-3 的研究方向。

延伸閱讀

153. F. Chaumette and S. Hutchinson, "Visual Servo Control, Part I: Basic Approaches" 與 "Part II: Advanced Approaches," IEEE Robotics & Automation Magazine, 2006–2007.(視覺伺服的權威教程,IBVS 與 PBVS 的完整說明)
154. E. Olson, "AprilTag: A Robust and Flexible Visual Fiducial System," ICRA, 2011.(第 4、23 章共同的基準標記理論來源)
155. A. Radford et al., "Learning Transferable Visual Models From Natural Language Supervision (CLIP)," ICML, 2021.(開放詞彙視覺辨識的里程碑,VLM 導航的基礎)
156. B. Yu et al., "Co-NavGPT: Multi-Robot Cooperative Visual Semantic Navigation Using Vision Language Models," arXiv, 2023(2025 修訂).(案例 23-3 的來源,多機器人 VLM 協作探索)

157. N. Yokoyama et al., "VLFM: Vision-Language Frontier Maps for Zero-Shot Semantic Navigation," ICRA, 2024.(零樣本語意導航的代表作)
158. Ultralytics YOLO 官方文件(docs.ultralytics.com).(物件偵測模型的訓練與部署,第 31 章實作的主要工具)
159. OpenCV 官方文件之 ArUco 模組與影像處理教學(docs.opencv.org).(線追蹤與標記偵測的實作基礎)

小結

126. 視覺 SLAM 回答「我在哪、環境長什麼樣」,視覺導航回答「我該往哪走」; 視覺導航不一定需要 SLAM(線追蹤、視覺伺服屬反應式架構),但加上 SLAM 後才具備全域規劃、任意目標導航、斷點恢復與語意任務的能力。
127. 三種基本策略:線追蹤成本極低但只能走預設路線;地標導航建立拓樸地圖、彈性較高;視覺伺服以影像回授直接控制,能達公釐級的相對定位精度,分 IBVS 與 PBVS 兩流派。
128. AprilTag / ArUco 在導航中有四種用途:絕對位姿校正、精確對接、樓層與區域識別、以及作為 SLAM 精度評估的近似真值;與 SLAM 的融合可分鬆耦合、緊耦合與輔助初始化三個層次。
129. YOLO 在導航中扮演語意標註、動態物體剔除與任務目標偵測三種角色;但因四足與小型機器人在低於 50 公分的高度運作,以人類中心視角訓練的模型往往難以良好運作,必須蒐集機器人視角資料微調。
130. VLM 在大規模影像文字語料上預訓練,提供語意先驗與常識推理,使機器人能接受自然語言指令並零樣本執行新任務;但受限於運算負擔、幻覺風險與延遲,適合做高階決策而非即時控制。

131. 機器狗視覺導航面臨四項特殊挑戰:步態震動、低視角語意問題、三維地形理解與續航運算限制;對策是多感測器緊耦合估測與分層控制架構。

132. 校園巡檢與資源回收辨識是整合本篇技術的理想專題,可依線追蹤、AprilTag、YOLO 語意、自主巡檢、VLM 任務五階段規劃,兼顧高中到研究生的不同層次。

第六篇「Visual SLAM 與視覺導航機器人」至此全部完成。我們從特徵與幾何出發,建立了完整的視覺 SLAM 系統,最後把它落實為真實的機器人任務。但前面所有的討論,都預設機器人有足夠的運算資源與電力。下一篇,我們走向另一個極端:當機器人只有 44 公克、運算功耗只有毫瓦等級時,SLAM 還可能嗎?第七篇「微型機器人、無人機與邊緣 AI SLAM」將展示這個領域最令人驚嘆的工程成就。

台灣自編大學教科書

@prof

《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第七篇 微型機器人、無人機與邊緣 AI SLAM

第 24 章

NanoSLAM 與低功耗 SLAM

編者：李世淵 (機器人叫獸)

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

學習目標

前六篇的所有系統,都預設機器人有足夠的運算資源與電力——ORB-SLAM3 需要一顆完整的處理器、光達 SLAM 需要數十瓦的運算平台。但如果機器人只有 44 公克重、掌心大小、電力預算只有毫瓦等級呢?本章展示這個領域最令人驚嘆的工程成就,也開啟第七篇的極限挑戰。學完本章之後,同學應能夠達成下列目標:

- 152. 說明微型機器人執行 SLAM 的三大限制與傳統解法的問題。
- 153. 說明 NanoSLAM 的設計目標與五項核心貢獻。
- 154. 說明 RISC-V 與平行超低功耗運算典範如何支撐毫瓦級 SLAM。
- 155. 說明毫瓦級 SLAM 的系統設計取舍與各模組的功耗分配。
- 156. 舉例說明微型無人機、群體機器人與 AR 眼鏡的應用場景。
- 157. 說明邊緣 AI 與 TinyML 對 SLAM 發展的影響。
- 158. 規劃一個高中程度的微型無人機地圖建構模擬專題。

核心概念

中文名詞	英文名詞	概念說明
微型無人機	Nano-UAV / Nano-drone	掌心大小、數十公克重的超小型飛行器。
機載運算	Onboard Computation	所有運算都在機器人自身完成,不依賴外部設施。
運算卸載	Computation Offload	把運算送到外部電腦執行,受限於通訊頻寬與延遲。
平行超低功耗	PULP	以多核心平行處理換取能效的運算典範。
系統單晶片	SoC	把處理器、記憶體與周邊整合在單一晶片

中文名詞	英文名詞	概念說明
		上。
開放指令集架構	RISC-V	開放且可擴充的處理器指令集架構。
飛時測距矩陣	ToF Matrix Sensor	一次量測多個方向距離的小型深度感測器。
掃描匹配	Scan Matching	比對前後量測以估計相對運動(第 13、17 章)。
位姿圖優化	Pose Graph Optimization	以圖優化修正累積誤差(第 12 章)。
動態頻率調節	Dynamic Frequency Scaling	依負載即時調整時脈與電壓以省電。
深度睡眠	Deep Sleep	閒置時把功耗降到極低的省電狀態。
邊緣人工智慧	Edge AI	在裝置端而非雲端執行 AI 推論。
微型機器學習	TinyML	在微控制器等極受限硬體上執行機器學習。

24.1 為什麼微型機器人很難跑 SLAM

第七篇的第一個問題是:為什麼把 SLAM 裝進微型機器人這麼困難?答案不只是「晶片比較慢」這麼簡單,而是三重限制的疊加。

三大限制

- 運算能力:SLAM 的前端(特徵提取、掃描匹配)與後端(圖優化、BA)都極耗運算。第 21、22 章的 ORB-SLAM 系列需要完整的多核心處理器,微控制器等級的晶片根本跑不動。
- 記憶體:第 13 章算過,一張 20×20 公尺、5 公分解析度的 2D 地圖就有 16 萬格;第 14 章的 Gmapping 還要乘上粒子數。而微型平台的記憶體可能只有數百 KB 至數 MB。
- 電力:這是最嚴苛的限制。微型無人機的電池能量極少,而飛行本身就消耗絕大部分電力。若運算再耗掉數瓦,飛行時間會從數分鐘掉到數十秒——系統形同無用。

傳統解法:運算卸載,以及它的問題

由於執行 SLAM 演算法所需的龐大運算負載與記憶體需求,目前完全機載的建圖只能在能搭載高瓦數處理器的機器人平台上實現;正因如此,口袋大小、硬體受限的機器人只能把 SLAM 的執行卸載到外部設施。

這個做法有明顯的代價:

- 依賴基礎設施:必須有無線網路與一台外部電腦,機器人無法真正自主。這在災害現場、地下坑道、野外環境根本不可行。
- 通訊延遲:影像或感測資料傳出去、指令傳回來,延遲可能達數十至數百毫秒。對高速飛行的無人機而言,這個延遲可能導致撞牆。

- 頻寬與功耗:持續傳輸大量資料本身也耗電,而且多台機器人同時運作時頻寬會成為瓶頸(第 26 章的多機器人議題)。
- 可靠度:網路一斷,機器人就失去導航能力。

本章的核心問題

因此本章要回答的問題是:能不能讓 SLAM 完全在機載完成,而且功耗低到微型無人機負擔得起?

答案是肯定的,而且已經被實現了。這需要三個層面的同步突破:演算法必須輕量化、硬體必須高能效、感測器必須低功耗。接下來的幾節,就是這三者如何協同的故事。

24.2 NanoSLAM 的設計目標

NanoSLAM 由蘇黎世聯邦理工學院(ETH Zürich)與波隆那大學的團隊於 2023 年提出,是第 7、24 章反覆提及的代表性工作。它的目標明確而大膽:讓微型無人機完全靠自己執行 SLAM。

核心宣言

NanoSLAM 是一套輕量化且最佳化的端到端 SLAM 方法,專為在僅 87.9 毫瓦的功耗預算下,於公分尺寸的機器人上運作而設計。研究團隊在真實場景中展示其建圖能力,並把 NanoSLAM 部署在一台重 44 公克、搭載名為 GAP9 的新型商用 RISC-V 低功耗平行處理器的微型無人機上。

讓同學對這些數字有感:87.9 毫瓦大約是一顆 LED 指示燈的功耗等級;44 公克比一顆雞蛋還輕。在這樣的條件下執行完整的 SLAM——這是本章最令人驚嘆之處。

五項核心貢獻

NanoSLAM 論文明確列出五項貢獻,它們構成了本章的技術骨架:

281. 平行化的最佳化實作:針對演算法進行最佳化的平行實作,並評估其執行時間與記憶體需求。
282. 機載 ICP:對掃描匹配的標竿演算法 ICP(第 13、17 章)進行平行實作與評估,能在機載以 55 毫秒完成。
283. 感測擴充板:為商用的 Crazyflie 2.1 微型無人機設計客製化的外掛伴隨板,以 4 顆 ToF 矩陣感測器擴充無人機的感測能力,使其能執行掃描匹配與自主導航。
284. 通訊協定:設計一套通訊協定,協調無人機原廠微控制器與 GAP9 系統單晶片之間的整合與資料交換,規範如何儲存圖結構、交換圖的位姿、新增邊,以及執行圖優化。
285. 實地驗證:廣泛的實地實驗評估,證明系統的閉環建圖功能——運用 NanoSLAM 達成軌跡誤差最多降低 67%,以及 4.5 公分的建圖精度。

關鍵成果資料

這個演算法設計上善用 RISC-V 處理核心的平行能力,能以 4.5 公分的精度建立一般環境的地圖,端到端執行時間小於 250 毫秒。

把這些數字與前面章節對照,更能體會其價值:第 22 章 ORB-SLAM3 在 EuRoC 上的 3.5 公分精度,是在完整的電腦上跑出來的;NanoSLAM 的 4.5 公分,則是在毫瓦級的微型平台上達成。研究團隊指出,藉由結合新型低功耗深度感測器與 GAP9 的平行能力,系統達成了與更大、運算能力更強的無人機(通常稱為微型飛行器或標準尺寸無人機)相當的室內建圖精度。

24.3 RISC-V GAP9 與低功耗平行運算

演算法再輕量,也需要合適的硬體才能實現。本節介紹讓毫瓦級 SLAM 成為可能的關鍵硬體。

微控制器設計的新趨勢

近年微控制器設計的趨勢強調平行處理、硬體加速器與能源效率。GreenWaves Technologies 的 GAP9 系統單晶片正是這些趨勢的體現,適用於微型無人機的專門應用;它以 RISC-V 核心提供先進的平行能力、電源最佳化與感測器整合,賦予微型無人機即時邊緣運算、更長的飛行時間與更強的資料處理能力。

PULP:平行超低功耗運算典範

GAP9 建立在平行超低功耗(Parallel Ultra-Low-Power,PULP)運算典範之上,體積小,功耗低於 180 毫瓦。

PULP 的核心思想值得同學理解,因為它與直覺相反:與其用一顆跑得很快的核心,不如用多顆跑得慢的核心平行工作。原因在於晶片的功耗大致與電壓平方及頻率成正比——把頻率降低一半、用兩顆核心平行處理,總運算量相同,但功耗可能降到原本的一半以下。這是「以平行換能效」的工程智慧。

晶片規格對照

把微型無人機常見的幾種晶片放在一起比較,能清楚看出 GAP9 的定位:

晶片(架構)	CPU 核心數	最高頻率	功耗	定位
STM32(ARM)	1	168 MHz	116 mW	飛控與狀態估計
ESP32(Xtensa)	2	240 MHz	300 mW	通訊與輕量運算
GAP8(RISC-V)	8 + 1	175 MHz	96 mW	前一代 AI 加速
GAP9(RISC-V)	9 + 1	370 MHz	66 mW	本章的主角

表 24-1 微型無人機常見系統單晶片比較

這張表格說明了三件事:第一,GAP9 的核心數遠多於傳統微控制器,體現 PULP 的平行思想;第二,即使核心多、頻率高,它的功耗反而更低,能效優勢明顯;第三,飛控用的 STM32 與運算用的 GAP9 是分工的關係——這正是 24.2 節第四項貢獻(通訊協定)存在的原因。

為什麼是 RISC-V

RISC-V 是一套開放的指令集架構,任何人都能自由使用與擴充,不需授權費。這對研究與新創極為有利:

- 可客製化:能為特定運算(如 SLAM 中的矩陣運算)加入專用指令,大幅提升能效。
- 無授權成本:對學術研究與小量產品的成本結構影響顯著。
- 生態成長快:近年 RISC-V 在邊緣 AI 與物聯網領域快速崛起,工具鏈日趨成熟。

對本書讀者而言,認識 RISC-V 的價值在於:它代表了「運算硬體不再由少數廠商壟斷」的趨勢,也是台灣半導體與機器人產業值得關注的方向。

進階功能:動態調節與深度睡眠

GAP9 支援執行時期調整時脈頻率(最高 370 MHz)與電壓,以及自動時脈閘控;閒置功耗可降至 45 微瓦。

這意味著系統能「該快則快、該省則省」:執行 SLAM 時全速運轉,飛行平穩無需建圖時降頻甚至進入深度睡眠。45 微瓦的閒置功耗幾乎可以忽略不計——這種細緻的電源管理,正是延長飛行時間的關鍵。

24.4 毫瓦級 SLAM 的系統設計

本節說明 NanoSLAM 如何把第 13、17 章的技術,壓縮進毫瓦級的預算。

感測器選擇:ToF 矩陣而非光達或相機

第八章介紹的旋轉式 2D 光達雖然好用,但對微型無人機而言太重、太耗電。相機則需要大量運算才能提取深度(第 9 章)。NanoSLAM 的選擇是 ToF 矩陣感測器——一種小型的飛時測距晶片,一次能量測多個方向的距離。

- 優點:體積僅數公釐、功耗極低、直接輸出距離(不需運算)、無移動部件。
- 配置:如 24.2 節所述,以 4 顆 ToF 矩陣感測器分別朝向四個方向,合起來提供近似 360 度的距離量測——這相當於一個極簡版的 2D 光達。
- 限制:解析度與量測範圍遠低於真正的光達,因此演算法必須能在稀疏的量測下運作。

演算法管線

NanoSLAM 的整體流程,是同學已經熟悉的架構,只是每個模組都被極致輕量化:

圖 24-1 NanoSLAM 的系統管線

【感測】4 顆 ToF 矩陣感測器 → 近似 360 度的稀疏距離量測

↓

【前端】平行化 ICP 掃描匹配(第 13、17 章)

估出相鄰時刻的相對位姿 → 機載 55 毫秒完成

↓

【圖建構】把位姿存為節點、掃描匹配結果存為邊(第 12 章的位姿圖)

↓

【閉環偵測】辨識重訪位置 → 新增閉環邊

↓

【後端】平行化位姿圖優化(第 12 章)

修正累積漂移 → 軌跡誤差最多降低 67%

↓

【輸出】機載完成的地圖與軌跡 端到端 < 250 毫秒,87.9 毫瓦

三個關鍵的輕量化策略

286. 平行化而非簡化:NanoSLAM 沒有把 ICP 或圖優化「改成簡單的近似版」,而是把標準演算法平行化到多核心上。這保住了精度,又善用了 PULP 硬體的特性。

287. 稀疏而非稠密:用 ToF 矩陣提供的稀疏量測取代稠密點雲,大幅減少資料量與運算量——這與第 18 章 LOAM「不需要用上所有的點」的思想一脈相承。

288. 分工而非全包:飛控與狀態估計由原廠的 STM32 負責,SLAM 運算由 GAP9 負責,兩者以專門的通訊協定協調。各司其職,避免單一晶片超載。

功耗分配的啟示

把相關資料放在一起,能看出系統設計的取舍:

任務	功耗 / 延遲	說明
NanoSLAM 建圖	約 87.9 mW,延遲 < 250 ms	本章主角
蒙地卡羅定位	平均 23 mW	已知地圖時的定位(第 11 章)
YOLO 物件偵測	低於 100 mW,最短 17 ms	邊緣 AI 推論(第 23、31 章)
深度睡眠	45 μ W	閒置時的極低功耗

表 24-2 微型平台上各項任務的功耗表現

系統維持在低於 100 毫瓦的功耗範圍——尤其是對運算密集的任務而言——對延長飛行時間與支援複雜的即時任務負載至關重要。

值得注意的是:定位(23 mW)遠比建圖(87.9 mW)省電。這印證了第 14 章案例 14-3 的觀察——首次建圖與後續定位是不同的任務。對微型機器人而言,這意味著一個實用策略:先建好地圖(可以慢一點、耗電一點),之後的任務只需低功耗的定位即可。

24.5 微型無人機、Swarm Robots、AR 眼鏡應用

毫瓦級 SLAM 開啟了哪些原本不可能的應用?本節說明三個代表性方向。

微型無人機:安全與可及性

微型無人機能放進手掌,適合室內環境,且與人互動時很安全;然而它們常因缺乏足夠的感測與運算硬體,而無法在複雜任務中提供所需的效能。

「安全」這一點特別值得強調。一台數公斤的標準無人機若失控撞到人,可能造成嚴重傷害;而 44 公克的微型無人機幾乎不具危險性。這使得它們能在教室、辦公室、醫院等有人的空間中運作——這是大型無人機做不到的。

- 室內探索與測繪:在人無法或不宜進入的空間(狹窄管道、災後建築、天花板夾層)進行探索與建圖。
- 農業巡檢:已有研究以多台自主微型無人機視覺偵測害蟲,再由單一較慢的重型載具前往偵測到的爆發點施藥;為因應微型無人機的極端限制(如低解析度感測器與低於 100 毫瓦的運算功耗預算),團隊設計並最佳化了一個微型的影像式卷積神經網路用於害蟲偵測。
- 教育與研究:微型無人機價格相對親民、安全性高,是極佳的教學平台(24.7 節)。

群體機器人(Swarm Robots)

當單台機器人的成本與體積都降低,「用很多台」就成為可行的策略。群體機器人的價值在於:

- 覆蓋效率:多台分頭探索,完成大範圍任務的時間大幅縮短(第 23 章案例 23-3 的多機器人協作探索)。
- 容錯性:單台故障不影響整體任務——這是單一昂貴機器人無法比擬的優勢。
- 分散式感知:從多個視角同時觀測,能建立更完整的環境模型。

但群體機器人有一個關鍵前提:每台機器人必須能自主定位。若全部依賴外部設施或中央電腦,通訊頻寬會隨機器人數量成長而崩潰(24.1 節提過)。機載 SLAM 因此是群體機器人可規模化的必要條件——這正是 NanoSLAM 這類技術的戰略價值。第 26 章會深入多機器人 SLAM 的協作議題。

AR 眼鏡與穿戴裝置

第 19、22 章談過 AR 需要視覺慣性 SLAM,但 AR 眼鏡的限制比手機更嚴苛:必須輕到能長時間配戴、電池必須小、且不能發熱燙臉。

這與微型無人機的限制驚人地相似——都是「極小的功耗預算下要跑 SLAM」。因此本章的技術路線(專用低功耗晶片、平行超低功耗運算、輕量化演算法)也直接適用於穿戴裝置。

這個跨領域的共通性,是本章一個重要的觀念:低功耗 SLAM 不只是微型無人機的專門技術,它是所有「受限平台」的共同需求。掌握這條路線,同學就掌握了通往 AR 眼鏡、智慧手錶、微型感測節點等眾多應用的鑰匙。

24.6 Edge AI 與 TinyML 對 SLAM 的影響

本節把視野從 SLAM 擴大到整個邊緣智慧的趨勢,並說明它與本書後續章節的連結。

邊緣 AI 的意義

邊緣 AI 指的是在裝置端而非雲端執行 AI 推論。它的價值與 24.1 節討論機載 SLAM 的理由完全一致:不依賴網路、低延遲、隱私保護、以及可規模化。

TinyML 則是邊緣 AI 的極端版本——在微控制器等記憶體只有數百 KB 的硬體上執行機器學習。這聽起來不可思議,但透過模型量化、剪枝、知識蒸餾等技術,已經能在極小的硬體上執行實用的推論。

與 SLAM 的三種結合

289. 感知增強:在微型平台上執行物件偵測。相關評測顯示,推論任務如 YOLO 物件偵測能在低於 100 毫瓦下完成,最短推論時間為 17 毫秒、約 1.59 毫焦耳。這讓微型機器人也能做語意感知(第 23 章)。
290. 學習取代運算:用小型神經網路直接估計某些量(如光流、深度、甚至位姿),取代傳統的幾何運算。若模型夠小,推論可能比傳統演算法更省電。
291. 線上學習:更前沿的方向是讓機器人在飛行中自我學習調適。已有研究探討在極低功耗預算下於微型無人機上進行裝置端自監督學習——這代表機器人不只是執行預先訓練好的模型,還能持續適應新環境。

硬體與演算法的共同設計

本章最重要的方法論啟示是:在極限受限的條件下,硬體與演算法不能分開設計。

NanoSLAM 之所以成功,不是因為演算法特別聰明,也不是因為晶片特別快,而是因為兩者互相配合:演算法為 RISC-V 多核心的平行架構量身打造,硬體則提供了演算法所需的能效與整合度。這種「硬體軟體共同設計」(Hardware-Software Co-design)的思維,是邊緣運算領域的核心能力。

對大學智慧機器人學程的同學而言,這是一個重要的提醒:除了學會使用現成的演算法,理解底層硬體的特性(平行度、記憶體階層、功耗行為)同樣重要。這兩者的交會處,正是台灣半導體優勢與機器人應用能夠結合的地方。

與後續章節的連結

- 第 25 章:微型無人機的視覺慣性導航,延續本章的平台限制討論。
- 第 26 章:多機器人與雲端邊緣協作,呼應 24.5 節的群體機器人。
- 第 31 章:AI 與 VLM 輔助導航,以及邊緣 AI 部署(Jetson、NPU),是本章邊緣運算討論的延伸。

24.7 高中專題:微型無人機地圖建構模擬

本章的技術聽起來遙不可及,但它其實非常適合作為高中專題——因為核心觀念可以用模擬完整體驗,不需昂貴硬體。

為什麼適合高中

- 觀念直觀:「用四個方向的距離感測器建地圖」比視覺 SLAM 的特徵與幾何容易理解得多。
- 可完全模擬:用 Python 就能寫出一個微型無人機的 2D 模擬,不需飛行器。
- 連結有感:微型無人機、群體機器人、AR 眼鏡都是學生感興趣的主題。
- 層次分明:從最簡單的「感測與畫圖」,到「加入漂移」,再到「閉環修正」,難度可調。

專題設計:四個階段

292. 階段一——建立模擬環境:用 Python 畫出一個 2D 房間(牆與障礙),放入一台虛擬的微型無人機,實作四個方向的距離感測(從無人機位置往四個方向找到最近的牆)。

293. 階段二——理想建圖:讓無人機沿預定路線飛行,每一步用四個方向的距離量測,把偵測到的牆點畫在地圖上。此階段假設位置完全準確,學生會看到一張漂亮的房間地圖逐漸浮現。
294. 階段三——加入漂移:在無人機的位置估計中加入小小的隨機誤差(模擬里程計漂移,第 1、7 章),重新建圖。學生會親眼看到地圖開始扭曲、牆線重影——這是第 13.7 節地圖病徵的具體重現。
295. 階段四——閉環修正:讓無人機繞一圈回到起點,加入「終點應該等於起點」的約束,用簡化的方式把誤差均攤回整條軌跡(第 12 章實作 12-1 的紙上方法可直接套用),觀察地圖被「拉正」的效果。

延伸方向

- 功耗預算計算:給定電池容量與各模組的功耗(參考表 24-2),計算飛行時間,並實驗「若把建圖頻率降低一半,能飛多久?」——這是很好的工程權衡練習。
- 群體模擬:加入多台虛擬無人機分頭探索,比較完成全區覆蓋所需的時間(呼應 24.5 節與第 26 章)。
- 感測器數量實驗:比較用 4 個、8 個、16 個方向的距離感測,對地圖品質的影響——這對應真實系統中「感測器數量與功耗、重量」的取舍。

教學建議

這個專題的教學價值,在於它讓學生在最簡化的情境中,完整走過 SLAM 的核心邏輯:感測 → 建圖 → 發現漂移 → 閉環修正。這正是本書第一篇到第四篇的縮影。

建議教師先讓學生做出階段二的「完美地圖」建立成就感,再刻意加入漂移製造「破壞」——那個「原本漂亮的地圖忽然歪掉」的瞬間,會讓學生真切理解為什麼 SLAM 是一個困難的問題。這種先建立、再打破、最後修復的教學節奏,比直接講解理論有效得多。

至此,同學已經看到 SLAM 在極限受限平台上的實現。下一章我們聚焦無人機本身的導航挑戰:GPS 拒止環境如何飛行?高速運動與震動如何處理?視覺慣性里程計在無人機上如何運作?第 25 章將把第七篇的討論帶向飛行的核心。

公式或流程

一、功耗與飛行時間的關係

微型無人機的飛行時間,由電池能量與總功耗決定:

$$\text{飛行時間} = \text{電池能量} / (\text{飛行功耗} + \text{運算功耗} + \text{感測功耗})$$

以 Crazyflie 2.1 為例,搭配 380 mAh 電池時單次飛行可達約 7 分鐘。假設飛行本身的功耗遠大於運算,則:

$$\text{若運算功耗佔總功耗比例為 } r, \text{ 則飛行時間下降約 } r$$

這條簡單的關係式解釋了為什麼「毫瓦」如此關鍵。若 SLAM 耗電 5 瓦,對一台總功耗約 10 瓦的微型無人機而言,飛行時間會直接砍半;而 87.9 毫瓦僅佔約 1%,幾乎不影響續航。這就是 NanoSLAM 把功耗壓到毫瓦級的工程意義。

二、平行加速的效益上限

把演算法平行化到 N 個核心上,理論加速比受限於「無法平行的部分」:

$$\text{加速比 } S(N) = 1 / ((1 - p) + p / N)$$

其中 p 為可平行化的比例。這稱為阿姆達爾定律(Amdahl's Law)。例如 $p = 0.9$ 、 $N = 8$ 時,加速比約為 4.7 倍,而非 8 倍。

這條式子對 NanoSLAM 的設計有直接意涵:ICP 的最近點搜尋高度可平行(每個點獨立搜尋),因此 p 很高、平行效益顯著;而圖優化中的稀疏矩陣分解則較難完全平行。理解哪些部分值得平行化,是善用 PULP 架構的關鍵。

三、能效指標

比較不同平台的運算能力時,單看速度不夠,要看「每焦耳能做多少運算」:

$$\text{能效} = \text{運算量} / \text{能量消耗} \quad (\text{常用單位: } GOPS / W \text{ 或 } Ops / J)$$

這正是 PULP 典範追求的目標(24.3 節)。以本章的資料為例,YOLO 物件偵測最短推論時間 17 毫秒、約 1.59 毫焦耳——用「每次推論消耗多少能量」來衡量,遠比「每秒能跑幾次」更能反映受限平台的實際可用性。

四、低功耗 SLAM 的設計決策流程

圖 24-2 受限平台的 SLAM 設計決策流程

① 功耗預算是多少?(電池容量 ÷ 目標運行時間 - 移動所需功耗)

↓

② 感測器選擇:在預算內能負擔什麼?

- 數瓦 → 3D 光達 / 深度相機
- 數百毫瓦 → 2D 光達 / 單目相機
- 數十毫瓦 → ToF 矩陣感測器(本章方案)

↓

③ 演算法選擇:資料量與運算量能否負擔?

- 稠密方法 → 需要大量記憶體與運算,受限平台不可行
- 稀疏 / 特徵法 → 大幅減量(第 18、19 章的共同思想)

↓

④ 平行化評估:哪些部分可平行?(依阿姆達爾定律估算效益)

↓

⑤ 電源管理:動態調頻、深度睡眠、依任務切換模式

↓

⑥ 分工設計:飛控與運算是否分離到不同晶片?通訊如何協調?

案例

案例 24-1 一顆 LED 的電力:重新理解「運算成本」

本章最值得同學停下來思考的,是那個 87.9 毫瓦的數字。

讓我們做幾個對照:一顆手機 LED 手電筒約消耗數百毫瓦;一台筆記型電腦執行 ORB-SLAM3 時,處理器功耗可能達到數十瓦——是 NanoSLAM 的數百倍。而 NanoSLAM 在這樣的預算下,達成了 4.5 公分的建圖精度,與更大、運算能力更強的無人機相當。

這個對比顛覆了一個常見的直覺:我們往往認為「更好的結果需要更多的運算」。但 NanoSLAM 證明,透過演算法與硬體的共同設計,可以在功耗降低兩三個數量級的同時,維持相當的精度。

這給同學兩個啟示。第一,效能瓶頸往往不在「算力不夠」,而在「用錯方法」。第 18 章 LOAM「不需要用上所有的點」、第 19 章 ORB「以部分精度換速度」、本章「以平行換能效」——都是同一種思維:找到問題的本質,只做必要的計算。

第二,在資源受限的條件下創新,往往比在資源充裕時更能激發真正的工程智慧。這對台灣的機器人與半導體產業特別有啟發意義:我們或許無法在算力上與大廠競爭,但在「用更少資源做更多事」這條路上,有充分的發揮空間。

案例 24-2 誰負責飛、誰負責想:雙晶片的分工哲學

NanoSLAM 系統中一個容易被忽略、卻極具工程智慧的設計,是它的雙晶片架構:原廠的 STM32 負責飛控與狀態估計,新增的 GAP9 負責 SLAM 運算,兩者以專門設計的通訊協定協調——包括如何儲存圖結構、交換圖的位姿、新增邊,以及執行圖優化。

為什麼不用一顆晶片全包?原因有三:

- 即時性要求不同:飛控迴路必須以極高頻率、極低延遲執行,任何延遲都可能導致墜機;SLAM 則可以慢一點(250 毫秒也能接受)。把兩者放在同一顆晶片上,SLAM 的運算尖峰可能拖累飛控。
- 可靠度隔離:即使 SLAM 當掉,飛控仍能維持飛行安全——這是安全設計的基本原則。
- 各取所長:STM32 適合即時控制,GAP9 適合平行運算。用對的晶片做對的事,整體能效最佳。

同學應該注意到:這個「快速反應層 + 慢速思考層」的分工,與第 3 章的混合式架構、第 15 章 Cartographer 的局部與全域子系統、第 18 章 LOAM 的高頻里程計與低頻建圖,是完全相同的設計模式。

這說明了一件事:好的系統設計原則具有跨層次的普遍性。無論是在演算法層、軟體架構層還是硬體層,「依時間尺度分層、各層各司其職」都是有效的策略。掌握這種模式思維,比記住任何單一系統的細節都更有價值。

案例 24-3 微型無人機的田野應用:從實驗室到農田

微型無人機的價值,在一個具體的農業應用中展現得淋漓盡致。

有研究團隊提出這樣的構想:高效的作物生產需要及早偵測害蟲爆發並及時處理,他們考慮以一支由多台自主微型無人機組成的機隊視覺偵測害蟲,再由單一台較慢的重型

載具前往偵測到的爆發點施藥。

這個「輕騎兵偵察、重裝部隊處理」的分工設計極為聰明:微型無人機便宜、安全、可大量部署,適合大範圍快速搜尋;重型載具雖然慢,但能攜帶藥劑執行實際處理。兩者結合,效率遠高於單獨使用任一種。

技術上的挑戰同樣值得注意:為因應微型無人機的極端限制(如低解析度感測器與低於 100 毫瓦的運算功耗預算),團隊設計、微調並最佳化了一個微型的影像式卷積神經網路用於害蟲偵測。這個網路每次推論僅需 0.58 GOps,在他們的資料集上偵測有害昆蟲的平均精度均值達 0.79——相較文獻中表現最佳的網路,平均精度低了 14%,但運算量少了 32 倍。

這個「精度降低 14%、運算量減少 32 倍」的取捨,正是受限平台工程的縮影。對大學的同學而言,這個案例也提示了一個在地化的專題方向:台灣的農業場域(果園、稻田、溫室)同樣面臨病蟲害偵測的需求,而微型無人機的安全性與低成本,使它比大型無人機更適合在密集種植的環境中作業。

實作活動

實作 24-1 微型無人機地圖建構模擬

目的:以純模擬的方式完整體驗低功耗 SLAM 的核心邏輯——從稀疏感測建圖、觀察漂移、到閉環修正,並進行功耗與感測器數量的工程權衡分析。本實作只需 Python,適合高中到大學部各層次。

第一部分:建立模擬環境與理想建圖(全體)

237. 用 Python 定義一個 2D 房間:以線段表示牆與障礙物,並在其中放置一台虛擬微型無人機(以位置與朝向表示)。

238. 實作四方向距離感測:從無人機位置沿前、後、左、右四個方向射出射線,計算與最近牆面的交點距離。這模擬了 24.4 節的 4 顆 ToF 矩陣感測器。
239. 讓無人機沿一條預定路線(例如繞房間一圈)移動,每一步記錄四個距離值,並依第 10 章的座標轉換把偵測到的牆點畫在地圖上。
240. 用 matplotlib 顯示逐步成長的地圖,觀察房間輪廓如何浮現。此階段假設位置完全準確,地圖應該相當漂亮。

第二部分:漂移與閉環(全體)

241. 加入里程計漂移:在無人機每一步的位置與角度估計中加入小的隨機誤差(例如位置誤差標準差 2 公分、角度誤差 0.5 度),重新建圖。
242. 觀察病徵:地圖出現什麼問題?牆線是否重影?繞完一圈後,終點與起點是否對不上?量測這個落差有多大。對照第 13.7 節的三種地圖病徵。
243. 實作簡化的閉環修正:當無人機回到起點附近時,加入「終點應等於起點」的約束,把累積誤差平均分攤到整條軌跡的每一步(第 12 章實作 12-1 的方法),重新繪製地圖。
244. 量化改善:比較修正前後的「終點與起點落差」以及地圖的目視品質,計算誤差降低的百分比,與 NanoSLAM 論文的 67% 對照。

第三部分:工程權衡分析(大學部,本章特色)

245. 感測器數量實驗:把四方向感測改為 8、16、32 個方向,比較地圖品質(牆面完整度、細節解析度)。找出「品質提升趨緩」的轉折點。
246. 功耗估算:假設每個 ToF 感測器耗電 20 毫瓦、每次掃描匹配耗電依感測點數線性成長,計算不同感測器數量下的總功耗。
247. 飛行時間試算:依公式一,假設電池能量固定、飛行本身耗電 5 瓦,計算各種配置下的飛行時間,畫出「感測器數量 vs 地圖品質 vs 飛行時間」的三方權衡圖。

248. 決策練習:若任務要求「至少飛行 5 分鐘且地圖誤差小於 10 公分」,你會選擇哪一種配置?說明理由。

第四部分:延伸探索(進階)

249. 建圖頻率實驗:把 SLAM 執行頻率從每步一次改為每三步、每五步一次,觀察地圖品質下降多少、功耗節省多少。這對應真實系統的動態調頻策略(24.3 節)。

250. 群體模擬:加入第二台、第三台虛擬無人機分頭探索,各自建圖後合併,比較「協作探索」與「單機探索」完成全區覆蓋所需的步數。這呼應 24.5 節與第 26 章。

251. 定位模式:先用一台無人機建好地圖,再讓第二台在已知地圖中只做定位(不建圖),比較兩者的運算量。對照表 24-2 中「定位 23 mW、建圖 87.9 mW」的差異。

討論問題

114. 加入漂移後,地圖出現了哪些病徵?繞一圈的終點落差有多大?這與第 13.7 節的描述是否吻合?

115. 閉環修正把誤差「均攤」到整條軌跡,而不是集中修正最後一步。為什麼均攤比較好?(提示:第 12 章案例)

116. 感測器從 4 個增加到 32 個,地圖品質提升了多少?功耗增加多少?你認為最合適的數量是多少?為什麼?

117. 若你是微型無人機的系統設計者,在「更好的地圖」與「更長的飛行時間」之間,你會如何取舍?這個取舍是否依任務而異?

118. (大學部)依阿姆達爾定律(公式二),若 ICP 有 95% 可平行化,用 GAP9 的 9 個核心能加速多少倍?這與理想的 9 倍差多少?差距從何而來?

延伸挑戰(大學部)

其一:實作一個簡化的位姿圖優化——把模擬中每一步的位姿當作節點、相鄰步之間的運動當作邊、閉環當作額外的邊,用 `scipy` 的最小平方求解器最佳化整條軌跡(第 12 章實作 12-1 的程式版本),比較它與「平均分攤」的簡易方法在精度上的差異。其二:若學校有 Crazyflie 或類似的微型無人機平台,嘗試讀取其感測器資料並在電腦端執行你的 SLAM 程式,體驗真實資料與模擬的差異——真實的感測雜訊、時間延遲與飛行擾動,會讓你對本章的工程挑戰有更深的體會。

延伸閱讀

160. V. Niculescu et al., "NanoSLAM: Enabling Fully Onboard SLAM for Tiny Robots," IEEE Internet of Things Journal, 2023(arXiv:2309.12008).(本章主要依據,程式碼公開於 GitHub: ETH-PBL/NanoSLAM)
161. H. Müller et al., "GAP9Shield: A 150GOPS AI-capable Ultra-low Power Module for Vision and Ranging Applications on Nano-drones," arXiv:2407.13706, 2024.(微型無人機的感測與運算模組,含功耗實測資料)
162. D. Rossi et al., PULP 平台相關論文與 pulp-platform.org 官方網站.(平行超低功耗運算典範的技術來源)
163. Bitcraze Crazyflie 官方文件(bitcraze.io).(本章實驗平台的硬體規格、開發環境與社群資源)
164. GreenWaves Technologies GAP9 產品文件.(RISC-V 低功耗處理器的規格與開發工具)
165. P. Warden and D. Situnayake, TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers, O'Reilly, 2019.(TinyML 的入門經典)

166. L. Crupi et al., "Training on the Fly: On-device Self-supervised Learning aboard Nano-drones within 20 mW," arXiv:2408.03168, 2024.(微型無人機上的裝置端學習,24.6 節的前沿方向)

小結

133. 微型機器人執行 SLAM 面臨運算、記憶體與電力三重限制;傳統做法是把運算卸載到外部設施,但這帶來依賴基礎設施、通訊延遲、頻寬功耗與可靠度的問題,使機器人無法真正自主。
134. NanoSLAM 是輕量化且最佳化的端到端 SLAM 方法,專為在僅 87.9 毫瓦的功耗預算下於公分尺寸機器人上運作而設計;它被部署在重 44 公克、搭載 RISC-V 低功耗平行處理器 GAP9 的微型無人機上,達成 4.5 公分建圖精度與小於 250 毫秒的端到端執行時間。
135. GAP9 建立在平行超低功耗(PULP)運算典範上,以多核心平行換取能效,功耗低於 180 毫瓦;它支援動態調整時脈與電壓、自動時脈閘控,閒置功耗可降至 45 微瓦。RISC-V 的開放性使其可客製化且無授權成本。
136. NanoSLAM 以 4 顆 ToF 矩陣感測器取代光達、以平行化 ICP 做掃描匹配(機載 55 毫秒)、以位姿圖優化修正漂移(軌跡誤差最多降低 67%);三個輕量化策略為平行化而非簡化、稀疏而非稠密、分工而非全包。
137. 微型無人機能放進手掌、適合室內且與人互動安全;機載 SLAM 是群體機器人可規模化的必要條件;而 AR 眼鏡的限制與微型無人機驚人地相似,使低功耗 SLAM 成為所有受限平台的共同需求。
138. 邊緣 AI 與 TinyML 與 SLAM 有三種結合:感知增強(YOLO 可在低於 100 毫瓦、最短 17 毫秒完成推論)、以學習取代運算、以及線上自我學習;核心方法論是硬體與演算法的共同設計。

139. 微型無人機地圖建構模擬極適合高中專題:觀念直觀、可完全模擬、層次分明;建議的教學節奏是先建立完美地圖、再刻意加入漂移製造破壞、最後以閉環修復。

本章展示了 SLAM 在極限受限平台上的實現,也帶出一個貫穿全書的觀念:效能瓶頸往往不在算力不足,而在方法未經最佳化。下一章我們聚焦無人機導航本身的挑戰——GPS 拒止環境如何飛行?高速運動、震動與影像模糊如何處理?光流、VIO 與視覺 SLAM 在無人機上如何分工?第 25 章將把第七篇的討論帶向飛行的核心。

台灣自編大學教科書

@prof

《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第 25 章

無人機 SLAM 與視覺慣性導航

編者：李世淵 (機器人叫獸)

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

學習目標

上一章我們看到 SLAM 如何被壓縮進毫瓦級的預算。本章聚焦無人機導航本身的挑戰：在天空中，機器人失去了地面的支撐、擁有完整的六自由度、速度快得多、而且任何定位失誤都可能導致墜機。學完本章之後，同學應能夠達成下列目標：

- 159. 說明無人機定位相對地面機器人的四項特殊困難。
- 160. 說明 GPS 拒止環境的成因與典型場景，並列舉可用的定位手段。
- 161. 比較光流、VIO 與視覺 SLAM 三種技術的能力層次與適用場合。
- 162. 說明高速運動、震動與影像模糊如何影響視覺定位，並提出對策。
- 163. 規劃一次無人機室內導航實驗的流程與安全措施。
- 164. 說明 EuRoC 與 TUM-VI 兩大視覺慣性資料集的特性與用途。
- 165. 說明從微型無人機到視覺導航無人機的技術演進脈絡。

核心概念

中文名詞	英文名詞	概念說明
微型飛行器	MAV	小型無人飛行器，是視覺慣性導航研究的主要平台。
GPS 拒止環境	GPS-denied	衛星訊號被遮蔽或干擾而無法使用的環境。
光流	Optical Flow	影像中亮度模式隨時間移動的向量場。
光流定位	Optical Flow Positioning	以向下相機的光流估計水平速度以維持懸停。
視覺慣性里程計	VIO	緊耦合相機與 IMU 估計運動，會累積漂移。
視覺慣性 SLAM	VI-SLAM	VIO 加上建圖與閉環，可消除長期漂移。
動態模糊	Motion Blur	曝光期間相機移動造成的影像模糊。

中文名詞	英文名詞	概念說明
特徵軌跡縮短	Shortened Feature Tracks	高速下特徵快速離開視野,追蹤時間變短。
滾動快門	Rolling Shutter	感測器逐列曝光,高速時造成影像變形。
全域快門	Global Shutter	全畫面同時曝光,適合高速運動。
積極機動	Aggressive Maneuver	大角度、高加速度的飛行動作。
光度校正	Photometric Calibration	校正相機的亮度響應與暗角,對直接法重要。
動作捕捉系統	Motion Capture	以外部相機提供毫米級真值位姿的實驗設備。

25.1 無人機定位的特殊困難

第 3.6 節曾比較過五種平台的導航需求,本節深入無人機為何是其中最嚴苛的一種。

四項特殊困難

- 完整的六自由度運動:地面機器人在平面上運動,只有三個自由度(第 10.3 節的 x 、 y 、 θ);無人機能在三維空間中任意平移與旋轉。狀態變多、運動更複雜,估測難度直接上升。
- 沒有里程計:這是最容易被忽略卻極為關鍵的一點。輪型機器人有輪速編碼器提供里程計(第 3、13 章),即使不完美也是穩定的運動參考;無人機在空中沒有任何接觸,完全沒有等效的感測器。它只能靠 IMU 積分——而第 7.3 節告訴我們,IMU 單獨使用會迅速漂移。
- 速度快、動態劇烈:無人機的飛行速度遠高於室內地面機器人,而且姿態變化劇烈。這直接導致影像模糊與特徵追蹤困難(25.4 節)。
- 失誤的後果嚴重:輪型機器人定位錯誤最多是撞牆停下;無人機定位錯誤可能直接墜機,造成設備損壞甚至人員傷害。這使得系統的穩健度要求遠高於精度要求。

載重與電力的雙重限制

第 24 章已詳述受限平台的問題,對無人機而言更為尖銳:每增加一公克的感測器重量,就要消耗額外的推力與電力;而飛行本身已經消耗了絕大部分電量。這使得無人機的感測器選擇極度受限——這也是第 9.5 節指出「相機加 IMU 是無人機最理想組合」的根本原因。

即時性的嚴苛要求

地面機器人的定位若延遲 0.5 秒,它只是慢半拍地停下;無人機的定位若延遲 0.5 秒,在飛行過程中可能已經移動了數公尺,控制迴路會失去穩定。

因此無人機系統對延遲的容忍度極低。這解釋了為什麼第 24 章提到 NanoSLAM 要把端到端執行時間壓在 250 毫秒以內,也解釋了為什麼高階飛控的姿態控制迴路要以數百赫茲執行——這正是第 7 章 IMU 高頻輸出價值的體現。

25.2 GPS-denied 環境導航

第 5 章詳細討論過 GNSS 的原理與限制,本節聚焦無人機在衛星訊號不可用時的處境與對策。

典型的 GPS 拒止場景

- 296. 室內:建材完全遮蔽衛星訊號(第 5.5 節)。工廠、倉庫、商場、體育館的巡檢與盤點任務都屬此類。
- 297. 橋樑與結構物下方:橋樑檢測是無人機的重要應用,但橋面本身就是遮蔽物。
- 298. 隧道與地下空間:礦坑、下水道、地下停車場——完全沒有衛星訊號。
- 299. 都市峽谷:第 5.5 節提過的高樓夾道,不只遮蔽,還有嚴重的多路徑造成錯誤定位——這比完全沒訊號更危險,因為系統可能不知道自己錯了。
- 300. 森林與植被下:樹冠遮蔽使訊號微弱且不穩定。
- 301. 人為干擾:第 5.5 節提過的 Jamming 與 Spoofing,對無人機是嚴肅的資安議題。

可用的定位手段

在 GPS 拒止環境中,無人機能倚賴的定位手段依可靠度排序如下:

手段	提供什麼	限制	本書章節
IMU	高頻姿態與運動	單獨使用快速漂移	第 7 章

手段	提供什麼	限制	本書章節
氣壓計	相對高度	受氣流與溫度影響	本章
光流(向下相機)	水平速度	需地面紋理,不提供絕對位置	25.3 節
測距感測器(向下)	離地高度	地面起伏會誤判	第 8 章
VIO(相機 + IMU)	完整 6-DoF 位姿	會漂移,需紋理與光線	第 22 章
VI-SLAM	位姿 + 地圖 + 閉環	運算量較大	第 22 章
UWB / 視覺標記	絕對位置校正	需事先部署基礎設施	第 4 章

表 25-1 GPS 拒止環境中無人機的定位手段

分層策略

實務系統採用分層設計,把這些手段組合起來:

- 底層(數百 Hz):IMU 提供姿態,維持飛行穩定。這一層不能有任何中斷,否則直接失控。
- 中層(數十 Hz):光流與測距提供速度與高度,支撐定點懸停。即使沒有地圖,無人機也能「待在原地不飄走」。
- 上層(10 至 30 Hz):VIO 或 VI-SLAM 提供完整位姿與地圖,支撐路徑規劃與自主導航。
- 校正層(不定期):UWB、視覺標記或回到 GPS 可用區域時,提供絕對位置校正(第 4、5 章的融合思想)。

這個分層的重要性在於容錯:若上層的 VIO 失效,無人機仍能靠中層的光流維持懸停,不至於墜機;若中層也失效,底層的 IMU 至少能維持姿態穩定,讓操作者接手。安全設計

的核心原則,就是讓失效逐級降級而非瞬間崩潰。

25.3 Optical Flow、VIO、Visual SLAM

本節釐清三種視覺定位技術的能力層次——它們常被混用,但提供的資訊差異很大。

光流:最基本的一層

第 9.4 節介紹過光流是「影像中亮度模式隨時間的移動」。在無人機上,它有一個極為實用的應用:向下光流定位。

做法是在機腹裝一顆朝下的相機,持續計算地面紋理在畫面中的移動速度。配合測距感測器提供的離地高度,就能換算出無人機的水平移動速度。

302. 提供什麼:水平速度(不是位置)。

303. 能做什麼:定點懸停。若光流顯示地面在往後移,表示無人機在往前飄,飛控就往後修正。這讓無人機能在沒有 GPS 的室內穩定停在原地。

304. 限制:只給速度,積分後才是位置,因此仍會漂移;需要地面有足夠紋理(在純白地板或水面上會失效);高度量測不準會直接造成速度換算錯誤。

光流定位是消費級無人機室內懸停的標準配備,運算量極小,是「用最少資源解決最關鍵問題」的好例子。

VIO:完整的六自由度里程計

VIO 緊耦合相機與 IMU(第 7.6、22.3 節),輸出完整的 6-DoF 位姿與速度。相較光流,它提供的資訊完整得多,能支撐路徑追蹤與自主飛行。

但 VIO 本質上仍是里程計——它會累積漂移(第 19.1 節提過視覺里程計的特性)。飛行時間一長,估測位置就會偏離真實位置。

視覺慣性 SLAM:加上地圖與閉環

VI-SLAM 在 VIO 之上加入建圖與閉環偵測(第 22 章),能在重訪同一地點時修正累積漂移。代價是運算量與記憶體需求上升。

技術	輸出	會漂移嗎	運算需求	典型用途
光流	水平速度	積分後會	極低	定點懸停
VIO	6-DoF 位姿 + 速度	會累積	中	路徑追蹤、自主飛行
VI-SLAM	位姿 + 地圖 + 閉環修正	閉環可修正	較高	探索建圖、長時間任務

表 25-2 三種視覺定位技術的能力比較

如何選擇

- 305. 任務是「懸停拍攝」:光流就夠了,省電省運算。
- 306. 任務是「飛一段預定航線」:需要 VIO 提供位置追蹤。
- 307. 任務是「探索未知空間並建圖」:需要 VI-SLAM。
- 308. 任務是「長時間反覆巡檢同一場域」:VI-SLAM 加上地圖重用(第 22 章 Atlas 的多階段作業)最為合適。

這個層次選擇的思維,呼應了本書反覆強調的原則:不要用超過任務需求的技術,因為每一分運算與電力,在無人機上都是飛行時間的代價。

25.4 高速運動、震動與影像模糊

本節是無人機視覺定位最核心的技術挑戰。

三重打擊

研究指出,現有的 VIO 系統在高速下會因為動態模糊、特徵軌跡縮短與控制迴路延遲增加而效能劣化。這三者恰好構成一個惡性循環:

- 動態模糊:曝光期間相機移動,影像變糊。在如此的速度下,新的問題會浮現,既有的困難也被加劇——動態模糊變得更為明顯,損害視覺特徵追蹤。模糊的影像中,角點變成一片糊,特徵提取直接失敗(第 19 章的特徵需要清晰的角點)。
- 特徵軌跡縮短:高速飛行時,同一個特徵點快速掃過視野就消失了。第 22.2 節強調的「中期資料關聯」因此難以建立,系統只能倚賴短期關聯,精度下降。
- 控制迴路延遲:視覺處理本身需要時間,高速下這個延遲對應的位移更大,控制品質惡化,可能引發震盪。

資料集反映的現實

這些挑戰在標準評測資料集上有清楚的呈現。有研究觀察到,由於無人機的高速移動,影像會有些模糊,在困難序列如 MH_04、MH_05、V1_03 與 V2_03 中尤其明顯。

更有趣的是一個細節:EuRoC 的機房序列在開頭都有快速運動以協助 IMU 初始化(第 22.4 節),但由此產生的動態模糊對視覺追蹤是一大挑戰;因此 DSO 的作者裁掉開頭,只使用無人機在空中的部分,但對 VIO 系統而言 IMU 初始化不可或缺,無法捨棄開頭。

這個細節同時說明了兩件事:第一,IMU 初始化需要運動激勵(第 22.4 節反覆強調),但運動激勵本身會造成模糊,兩者互相拉扯;第二,不同方法在同一資料集上的評測條件可能不同,同學閱讀論文比較資料時要留意這類細節。

速度的量級

讓同學對「高速」有具體概念:EuRoC 微型飛行器資料集的速度很少超過每秒 5 公尺;競速無人機資料集可達每秒 28 公尺,但通常是室內結構化場地或短程戶外賽道的 RGB 相機。有研究以雙目 RGB 的 VIO 支援到每秒 17.5 公尺,但它依賴雙目視覺並假設有利的光線條件。

對照之下,室內地面機器人的速度通常在每秒 0.5 公尺以下——無人機的速度是它

的十倍甚至數十倍。這就是為什麼無人機的視覺定位困難得多。

五種對策

- 提高快門速度:縮短曝光時間直接減少模糊,代價是影像變暗,需要更好的感光元件或更亮的環境。
- 使用全域快門:滾動快門逐列曝光,高速旋轉時會造成影像歪斜;全域快門全畫面同時曝光,是高速應用的必要選擇。
- 倚重 IMU:有研究整合以 IMU 為基礎的運動預測模組來改善跨幀的特徵追蹤,特別是在高速運動期間。IMU 能預測特徵「應該移動到哪裡」,大幅縮小搜尋範圍,提升追蹤成功率。這是第 7.6 節「IMU 為視覺提供初值」的具體實現。
- 影像增強:對模糊或低對比的影像做預處理。有研究結合對比度受限的自適應直方圖等化(CLAHE)、梯度濾波與描述子驗證,以在動態模糊下維持穩定的特徵軌跡。
- 自適應權重:依影像品質動態調整視覺與慣性的信任比例。有研究在 EuRoC 與 TUM VI 資料集以及實地測試中驗證了自適應共變異數方法,在有動態模糊的情境中顯著改善無人機的導航精度。

震動的問題

除了飛行速度,螺旋槳與馬達造成的高頻震動也是無人機特有的問題:

309. 對 IMU:震動會污染加速度計讀值,若頻率接近取樣率還可能造成混疊。實務上需要機械減震(軟膠墊)與濾波處理。
310. 對相機:高頻震動造成的微小模糊會降低特徵品質,通常以減震座與較快的快門緩解。
311. 對安裝:感測器與機身的相對位置(外參,第 10 章)若因震動而鬆動改變,整個系統的精度都會受影響。

25.5 無人機室內導航實驗

本節說明如何規劃一次安全而有教學價值的無人機室內導航實驗,這也是專題實作的基礎。

安全第一:必須先講的事

無人機實驗與地面機器人最大的差異在於風險。以下是不可妥協的原則:

- 312. 防護網或防護籠:室內實驗應在有防護網的空間進行,或使用附有槳葉保護框的機型。
- 313. 選用微型平台:第 24 章介紹的微型無人機重量僅數十公克,即使失控撞到人也幾乎不會受傷。教學實驗強烈建議從微型平台開始。
- 314. 緊急停止:操作者必須隨時能一鍵切斷動力,且此功能不依賴需要除錯的自主系統。
- 315. 分階段測試:先在模擬中驗證、再繫繩測試、最後才自由飛行。跳過階段是意外的主因。
- 316. 人員位置:實驗時所有非必要人員應在防護網外,操作者站在無人機後方而非前方。

實驗場地的準備

- 317. 紋理:地面與牆面需有足夠的視覺紋理(第 9、19 章的白牆詛咒在無人機上同樣成立)。必要時可在地面鋪設有圖案的墊子。
- 318. 光線:均勻且充足,避免逆光與強烈明暗對比。窗戶可能造成過曝,建議拉上窗簾以維持穩定的照明。
- 319. 氣流:避免冷氣直吹或開窗形成的氣流,它會嚴重干擾微型無人機的穩定。

320. 真值系統(選用):若有動作捕捉系統,能提供毫米級的真值位姿,用於量化評測(25.6 節)。沒有的話,可用第 4 章的 AprilTag 提供近似真值(第 23.3 節的第四種用途)。

實驗流程

- 系統檢查:確認 IMU 校正、相機校正(第 9.2 節)、感測器外參(第 10 章)、以及通訊連線正常。
- 懸停測試:最基本的能力。啟動光流定位,讓無人機在原地懸停 30 秒,量測位置漂移。若懸停都不穩,後續的自主飛行不必嘗試。
- VIO 初始化:依第 22.4 節,以帶加減速與旋轉的運動完成 IMU 初始化,確認尺度估計正確(可用已知距離驗證)。
- 簡單航線:設定一條簡單的方形或直線航線,讓無人機自主飛行,記錄軌跡。
- 閉環測試:讓無人機繞一圈回到起點,量測終點與起點的落差,評估累積漂移(第 13.7 節的評估方法)。
- 挑戰測試:逐步增加難度——提高速度、加入急轉、暫時遮蔽相機——觀察系統在什麼條件下失效。這是最有教學價值的環節。

資料記錄

每次飛行都應完整記錄:影像、IMU 原始資料、飛控狀態、估測軌跡、以及(若有)真值軌跡。這些資料能離線反覆分析,也能作為自製資料集——對專題研究而言,一份自己場域的資料集本身就是有價值的成果。

25.6 視覺慣性資料集:EuRoC、TUM-VI

沒有無人機硬體,同樣能做無人機 SLAM 的研究——這正是公開資料集的價值。本節介紹兩大標準資料集。

EuRoC MAV 資料集

第 7 章案例 7-3 已介紹過 EuRoC,此處補充細節。它包含十一段以微型飛行器錄製的序列,提供雙目相機影像、IMU 量測與來自動作捕捉系統的真值位姿。

每段序列涵蓋 30 至 60 秒的飛行,包含來自外部動作捕捉的高保真 6-DoF 真值位姿,以及運作環境的高解析度點雲掃描,使軌跡估計與稠密三維地圖重建都能以幾何一致的參考資料進行評估。典型的評測流程是以 20 Hz 的單目影像與 200 Hz 的 IMU 資料,交由特徵法 VIO(例如關閉閉環的 ORB-SLAM3)處理,產出真實尺度的相機軌跡與稀疏三維地標。

321. 序列分類:機房(MH)序列在工業廠房錄製,房間(V1、V2)序列在有動作捕捉的實驗室錄製。

322. 難度分級:每組序列都分 easy、medium、difficult 三級,難度來自速度、光線與運動劇烈程度。這讓同學能循序漸進地測試演算法。

323. 使用建議:從 MH_01_easy 開始建立成功經驗,再挑戰 V1_03_difficult 等困難序列。

TUM-VI 資料集

TUM-VI 是另一套廣泛使用的基準。它提供 1024×1024 解析度、20 Hz 的相機影像,具備高動態範圍與光度校正;IMU 以 200 Hz 量測三軸加速度與角速度,相機與 IMU 在硬體層級時間同步。

相較之下,EuRoC MAV 的序列較短、變化較少,只包含一個機房與一個實驗室的錄製;此外 EuRoC MAV 不含光度校正,而這對評測直接法(第 19.2 節)相當重要。

324. 特色:涵蓋室內外多種場景、序列較長、有光度校正、時間同步在硬體層級完成。

325. AR / VR 相關:第 22.7 節引述過 ORB-SLAM3 在 TUM-VI 房間序列的毫米級精度,那正是這個資料集的房間場景。

特性	EuRoC MAV	TUM-VI
平台	微型飛行器	手持感測器組
影像	雙目,20 Hz	雙目 1024×1024,20 Hz
IMU	200 Hz	200 Hz,硬體時間同步
真值	動作捕捉 / 雷射追蹤	動作捕捉(部分序列)
光度校正	無	有
場景多樣性	一個機房 + 一個實驗室	室內外多種場景
額外資料	環境點雲掃描	高動態範圍影像

表 25-3 EuRoC 與 TUM-VI 資料集比較

資料集的使用建議

326. 演算法比較:在同一資料集上比較不同系統(第 22 章實作 22-1 的做法),資料才有可比性。

327. 留意評測條件:如 25.4 節提過的「是否裁掉開頭」,不同論文的評測條件可能不同,直接比較數字要謹慎。

328. 自製資料集:公開資料集無法涵蓋你的特定場域。用第 25.5 節的方法錄製大學場域的資料,是專題極有價值的貢獻。

329. 新興資料集:近年也出現速度更快、感測模態更多的資料集,如競速無人機資料集(速度可達每秒 28 公尺)與含熱像的資料集,適合進階研究。

25.7 從 nanoSLAM 到視覺導航無人機

本節把第 24、25 兩章連起來,梳理無人機自主導航的技術演進與未來方向。

三個層次的無人機自主

層次	能力	關鍵技術	代表平台
穩定	不飄走、能懸停	IMU + 光流 + 測距	消費級無人機
自主	能沿航線飛、能避障	VIO + 路徑規劃	專業無人機
智慧	能探索、建圖、理解任務	VI-SLAM + 語意 + VLM	研究平台、未來產品

表 25-4 無人機自主能力的三個層次

這個層次表能幫同學定位自己的專題目標。多數教學專題應從「穩定」層次確認基礎,再挑戰「自主」層次;「智慧」層次適合研究所或長期專題。

技術演進的三條主線

- 運算的下沉:從外部電腦運算(卸載)、到機載高效能運算(如 Jetson)、再到第 24 章的毫瓦級專用晶片。這條線讓自主能力得以進入更小、更便宜的平台。
- 感測的輕量化:從笨重的光達、到相機加 IMU、再到第 24 章的 ToF 矩陣感測器。這條線降低了重量與功耗的門檻。
- 智慧的上升:從幾何導航、到語意理解(第 23 章)、再到 VLM 驅動的任務級自主。這條線提升了機器人能執行的任務複雜度。

值得注意的是,這三條線正在交會:第 24 章的微型平台已經能執行 YOLO 物件偵測(低於 100 毫瓦、最短 17 毫秒),意味著「小、省電、且有語意理解能力」的無人機正在成為現實。

大學場域的專題方向

330. 室內測繪:以微型無人機掃描系館內部(如挑高中庭、樓梯間),產生三維模型。這些是地面機器人難以到達的空間,能展現無人機的獨特價值。
331. 設施巡檢:屋頂太陽能板、外牆、高處管線的檢查——無人機能安全地取代人員登高作業。
332. 農業應用:呼應第 24 章案例 24-3,台灣的果園與溫室是微型無人機的理想場域。
333. 教學資料集:錄製並公開一份大學場域的視覺慣性資料集,供後續學生與研究者使用。這是相對容易入手、卻能長期產生價值的貢獻。
334. 地面與空中協作:讓機器狗(第 18、23 章)與無人機協同探索——地面機器人續航長、承載大,無人機視野廣、能到高處。這是第 26 章多機器人協作的具體應用。

前沿方向

最後提幾個值得關注的研究方向,供有志深入的同學參考:

335. 高速飛行的感知:如 25.4 節所述,現有 VIO 在高速下會劣化。以熱像儀等非傳統感測器突破速度限制,是活躍的研究方向。
336. 學習與幾何的結合:第 19.7 節提過深度學習特徵正進入 SLAM。有研究整合深度特徵提取器與稀疏特徵匹配器,並結合傳統的手工關鍵點偵測與光流追蹤,以 TensorRT 加速推論,在快速相機運動與光線變化下改善前端追蹤效能。
337. 學習式慣性里程計:純 IMU 的學習方法在特定情境下已能接近 VIO 的表現——這對視覺失效的場景(黑暗、煙霧)是重要的備援。
338. 群體協作:多台微型無人機協同建圖與探索,是第 26 章的主題,也是本篇的最後一塊拼圖。

至此,無人機導航的核心挑戰與技術路線已完整呈現。下一章我們把視野從單機擴

大到群體:多台機器人如何協同建圖?各自的地圖如何融合?雲端與邊緣如何分工?通訊、同步與安全又有哪些課題?第 26 章將完成第七篇的最後一塊拼圖。

公式或流程

一、光流速度換算

向下相機量到的光流是「影像上每秒移動幾個像素」,要換算成真實的水平速度,需要離地高度與相機焦距:

$$v = (flow_pixels / f) \times h$$

其中 flow_pixels 是每秒的像素位移、f 是焦距(像素單位,第 9.1 節的內參)、h 是離地高度(由測距感測器提供)。

這條式子透露一個關鍵風險:高度 h 出現在分子。若測距感測器誤判高度(例如飛過桌面時高度突然變小),換算出的速度就會錯誤,飛控可能因此做出錯誤修正而發生漂移甚至撞擊。這是光流定位最常見的失效原因,同學在實驗時務必注意場地的地面起伏。

二、動態模糊的量化

曝光期間影像上的模糊長度,約為:

$$blur_pixels \approx (v / h) \times f \times t_exposure$$

其中 v 為飛行速度、h 為到場景的距離、t_exposure 為曝光時間。

代入實際數字感受一下:飛行速度 5 m/s、距離場景 5 公尺、焦距 400 像素、曝光 10 毫秒,則模糊約 4 像素——已足以嚴重影響角點偵測(第 19 章的特徵需要清晰的局部結構)。若速度提高到 20 m/s,模糊達 16 像素,特徵提取幾乎完全失效。

這條式子也直接指出對策:縮短曝光時間 t_exposure 能線性減少模糊——這正是 25.4 節第一項對策的數學依據。

三、VIO 的狀態與誤差項

無人機 VIO 的狀態向量與第 22.3 節相同,每個關鍵幀為:

$$S = \{R, p, v, b_g, b_a\} \quad (\text{共 } 15 \text{ 自由度})$$

緊耦合最佳化同時最小化視覺與慣性誤差:

$$\min \Sigma \rho(\|e_{\text{視覺}}\|^2) + \Sigma \|e_{\text{慣性}}\|^2$$

對無人機而言,一個實務重點是兩者權重的動態調整:當偵測到影像模糊或特徵數不足時,應降低視覺項的權重、提高慣性項的比重——這正是 25.4 節第五項對策(自適應權重)的實現方式,也呼應第 11 章卡爾曼增益「按可信度加權」的核心思想。

四、無人機 GPS 拒止導航的分層架構

圖 25-1 無人機導航的分層架構與失效降級

【校正層】不定期:UWB / 視覺標記 / 重回 GPS 區 → 絕對位置校正

↓ 失效則降級

【上層】10–30 Hz:VI-SLAM / VIO → 完整 6-DoF 位姿 + 地圖

支撐:路徑規劃、自主導航、探索建圖

↓ 失效則降級

【中層】數十 Hz:光流 + 測距 → 水平速度 + 離地高度

支撐:定點懸停(至少不飄走)

↓ 失效則降級

【底層】數百 Hz:IMU → 姿態

支撐:維持飛行穩定,讓操作者接手

【設計原則】讓失效逐級降級,而非瞬間崩潰

案例

案例 25-1 沒有輪子的代價:無人機為何比地面機器人難

同學若同時做過地面機器人與無人機的 SLAM,會強烈感受到後者困難得多。原因不只是「飛比較快」,而是一個常被忽略的結構性差異:無人機沒有里程計。

回顧本書的脈絡:第 13.4 節說明 2D SLAM 靠「里程計預測、光達校正」的循環運作;第 14.5 節指出 Gmapping 的三個前置條件中,里程計是必要的;第 18.3 節的 LOAM 也需要運動估計作為配準初值。里程計雖然會漂移,但它提供了穩定、連續、免費的運動參考。

無人機完全沒有這個奢侈品。它唯一的內部運動來源是 IMU,而第 7.3 節已經證明:消費級 IMU 單獨積分,數秒內就會漂移數公尺。這意味著無人機的視覺系統必須「獨力承擔」原本由里程計分擔的責任——一旦視覺失效,系統就沒有可靠的後備。

這解釋了三件事:為什麼無人機領域如此重視 VIO 的緊耦合(第 22 章)、為什麼光流定位是消費級無人機的標準配備(25.3 節)、以及為什麼 25.2 節的分層降級架構如此關鍵。

對同學的啟示是:理解一個平台的「缺什麼」,往往比理解它「有什麼」更能解釋系統設計的取舍。這種從限制反推設計的思維,是工程判斷力的核心。

案例 25-2 初始化的兩難:動得不夠不行,動太多也不行

這是一個極具教學價值的矛盾,而且同學在實作時必定會遇到。

一方面,第 22.4 節反覆強調:IMU 初始化需要足夠的運動激勵,否則尺度與零偏不可觀測。靜止或等速直線移動,系統就是初始化不了。

另一方面,25.4 節提到的實例顯示:EuRoC 的機房序列在開頭都有快速運動以協助 IMU 初始化,但由此產生的動態模糊對視覺追蹤是一大挑戰。快速運動雖然滿足了慣性系統的需求,卻破壞了視覺系統的輸入品質。

這個矛盾的存在,恰恰說明了視覺慣性融合的本質:兩種感測器的需求並不一致,甚至互相拉扯。IMU 喜歡有加速度的運動,相機喜歡平穩清晰的畫面。

實務上的解法是折衷:做「中等幅度、帶加減速與旋轉、但不過快」的運動。具體而言,拿著裝置或操控無人機做小幅度的前後左右移動並轉動數秒,速度以「畫面不明顯模糊」為上限。

這個案例也提醒同學:當論文中的方法在你手上跑不起來時,問題未必在程式,而可能在「你的資料不符合方法的隱含假設」。理解方法對輸入的要求,是成功實作的前提。

案例 25-3 速度的邊界:VIO 能飛多快

「無人機視覺定位能支援多快的飛行?」這個問題的答案,近年正被不斷推進,也清楚呈現了技術的演進。

先看基準:EuRoC 微型飛行器資料集的速度很少超過每秒 5 公尺——這是多數 VIO 論文的評測條件。以此為基準,同學看到的多數精度資料,都是在相對溫和的速度下取得的。

往上推進:有研究以雙目 RGB 的 VIO 支援到每秒 17.5 公尺,但它依賴雙目視覺並假設有利的光線條件。競速無人機資料集則可達每秒 28 公尺,但通常是室內結構化場地

或短程戶外賽道的 RGB 相機。

突破限制:更前沿的研究開始改用非傳統感測器。有研究以單目熱像慣性里程計,在低空飛行中達成每秒 30 公尺的即時運作。熱像的優勢在於不受光線影響、且對某些模糊來源較不敏感。

但這條路也有代價。研究指出,雖然以深度學習為基礎的方法在熱像上展現了不錯的效能提升,但它們的推論時間仍與無人機機載硬體嚴格的延遲與功耗限制不相容——某些方法在筆電等級的 GPU 上每幀就需要超過 30 毫秒。因此手工設計的管線仍是嵌入式高速閉環飛行最實際的解法。

這個案例濃縮了本書一再出現的主題:前沿技術的精彩資料,往往建立在特定條件之上;而真正能部署的系統,必須同時滿足精度、速度、功耗與可靠度的多重約束。學會閱讀論文時同時看「達成了什麼」與「在什麼條件下達成」,是研究能力的重要一環。

實作活動

實作 25-1 無人機視覺慣性導航實驗

目的:透過公開資料集與(可選的)實體飛行,理解無人機視覺定位的挑戰與對策。第一、二部分不需硬體,適合全體;第三部分需要微型無人機平台。

第一部分:資料集分析(全體,不需硬體)

252. 下載 EuRoC 資料集,選取 MH_01_easy 與 V1_03_difficult 兩段序列,分別以第 22 章的 ORB-SLAM3 雙目慣性模式執行,用 evo 計算 ATE。

253. 比較兩者的精度差異,並播放影像觀察 difficult 序列在哪些片段出現明顯的動態模糊,對照 25.4 節的討論。

254. 模糊量化:寫程式計算每一幀影像的清晰度(可用拉普拉斯運算子的變異數),畫出「清晰度隨時間變化」的曲線,標出最模糊的片段。
255. 關聯分析:把清晰度曲線與軌跡誤差曲線畫在一起,觀察兩者是否相關——影像最模糊的時刻,是否也是誤差最大的時刻?
256. 初始化觀察:留意序列開頭的快速運動段,記錄系統完成 IMU 初始化所需的時間,並觀察此段的影像模糊程度,親身驗證案例 25-2 的兩難。

第二部分:光流與模糊模擬(全體)

257. 用 OpenCV 的 Lucas-Kanade 光流,對一段向下拍攝的影片(可用手機在桌面上方平移拍攝)計算光流,依公式一換算水平速度。
258. 高度敏感度實驗:故意把高度值設錯(例如實際 1 公尺卻填 0.5 公尺),觀察換算速度的誤差,理解 25.5 節提到的失效風險。
259. 模糊模擬:對清晰影像人為加入動態模糊(用 OpenCV 的運動模糊核),模糊長度依公式二計算,測試 ORB 特徵偵測數量隨模糊程度如何下降。
260. 繪製「模糊像素數 vs 可偵測特徵數」的曲線,找出特徵數急遽下降的臨界點。這個臨界值就是你的系統能容忍的速度上限依據。

第三部分:實體飛行(有微型無人機平台的班級)

務必先閱讀 25.5 節的安全原則,並在防護網內、以微型平台進行。

261. 懸停測試:啟動光流定位,讓無人機在原地懸停 30 秒,以地面標記量測水平漂移範圍。
262. 地面紋理實驗:分別在有圖案的地毯、純色地板、與反光地面上懸停,比較漂移程度,驗證光流對紋理的依賴。
263. 高度實驗:在不同離地高度(0.5、1.0、1.5 公尺)懸停,觀察穩定度差異,思考公式一中高度誤差的影響。

264. 簡單航線:設定一條方形航線自主飛行,回到起點後量測落差,評估累積漂移。

265. 挑戰測試:在安全前提下逐步提高飛行速度,記錄系統開始不穩定的速度,對照案例 25-3 的討論。

討論問題

119. easy 與 difficult 序列的 ATE 差多少倍?影像清晰度與軌跡誤差的相關性如何?

120. 依公式二,若你的相機曝光 5 毫秒、焦距 400 像素、距離場景 3 公尺,飛行速度多快時模糊會達到 5 像素?這對速度上限有什麼意涵?

121. 光流換算實驗中,高度誤差 50% 造成速度誤差多少?若無人機依此錯誤速度做修正,會發生什麼?

122. (有飛行者)在哪種地面上懸停最穩定?最不穩定的原因是什麼?這印證了 25.3 節的哪一項限制?

123. 依圖 25-1 的分層架構,若你的無人機在飛行中 VIO 突然失效,系統應如何反應?請設計一套降級流程。

延伸挑戰(大學部)

其一:自適應權重實作——在 VIO 系統中加入影像品質評估(如第二部分的清晰度指標),當影像模糊時自動降低視覺項權重、提高慣性項比重,在 EuRoC difficult 序列上測試是否改善精度。這對應 25.4 節的第五項對策,也是一個完整的小型研究題目。其二:大學場域資料集——依 25.5 節的方法,以微型無人機或手持感測器組錄製系館內部的視覺慣性資料(含影像、IMU、以及用 AprilTag 提供的近似真值),整理成標準格式並公開,供後續學生使用。這是一份能長期產生價值的貢獻,也是專題成果的良好呈現形式。

延伸閱讀

167. M. Burri et al., "The EuRoC Micro Aerial Vehicle Datasets," International Journal of Robotics Research, 2016.(EuRoC 資料集的官方說明,本章 25.6 節依據)
168. D. Schubert et al., "The TUM VI Benchmark for Evaluating Visual-Inertial Odometry," IROS, 2018.(TUM-VI 資料集,含光度校正與硬體時間同步的設計說明)
169. C. Campos et al., "ORB-SLAM3," IEEE T-RO, 2021.(第 22 章的系統,本章實作的主要工具)
170. T. Qin, P. Li, and S. Shen, "VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator," IEEE T-RO, 2018.(無人機視覺慣性系統的另一代表作)
171. D. Scaramuzza and Z. Zhang, "Visual-Inertial Odometry of Aerial Robots," Encyclopedia of Robotics, Springer, 2019.(空中機器人視覺慣性導航的權威綜述,UZH 研究群,呼應第 2 章)
172. K. Sun et al., "Robust Stereo Visual Inertial Odometry for Fast Autonomous Flight," IEEE RA-L, 2018.(高速飛行 VIO 的代表性工作,案例 25-3 提及)
173. PX4 與 ArduPilot 開源飛控之視覺定位整合文件.(把 VIO 輸出接入飛控的實務指引,實體飛行必讀)

小結

140. 無人機定位有四項特殊困難:完整六自由度運動、沒有里程計(只能靠會漂移的 IMU)、速度快且動態劇烈、以及失誤後果嚴重;加上載重電力限制與嚴苛的即時性要求,使它成為最難的 SLAM 平台之一。

141. GPS 拒止場景包含室內、橋下、隧道、都市峽谷、植被下與人為干擾;可用手段依序為 IMU、氣壓計、光流、測距、VIO、VI-SLAM 與 UWB / 視覺標記,實務採分層架構並讓失效逐級降級而非瞬間崩潰。
142. 三種視覺定位技術的層次:光流只給水平速度但運算極省,支撐定點懸停;VIO 給完整 6-DoF 位姿但會漂移;VI-SLAM 加上地圖與閉環可修正長期漂移。應依任務需求選擇,不用超過需求的技術。
143. 高速飛行造成三重打擊:動態模糊損害特徵追蹤、特徵軌跡縮短使中期關聯難以建立、控制迴路延遲惡化;對策包含提高快門速度、使用全域快門、以 IMU 預測改善追蹤、影像增強、以及自適應調整視覺與慣性的權重。
144. 室內飛行實驗的安全原則不可妥協:防護網、微型平台、緊急停止、分階段測試;場地需注意紋理、光線與氣流;實驗流程應由懸停測試逐步進展到挑戰測試。
145. EuRoC 提供十一段微型飛行器序列、雙目影像與動作捕捉真值,速度很少超過每秒 5 公尺;TUM-VI 則提供 1024×1024 影像、光度校正與硬體時間同步,場景多樣性較高。比較資料時須留意各研究的評測條件。
146. 無人機自主可分穩定、自主、智慧三個層次;技術演進沿運算下沉、感測輕量化、智慧上升三條主線推進,而三者正在交會——小、省電且具語意理解能力的無人機正在成為現實。

本章完成了單機無人機導航的討論。但無論多先進,單一機器人的視野、續航與運算能力終究有限。下一章我們把視野擴大到群體:多台機器人如何協同建圖?各自的地圖如何融合成一致的全域地圖?雲端與邊緣如何分工?通訊頻寬、時間同步與資料安全又有哪些課題?第 26 章將完成第七篇的最後一塊拼圖,也為第八篇的實作與第九篇的 AI 語意導航鋪路。

台灣自編大學教科書

@prof

《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第 26 章

多機器人 SLAM 與雲端 / 邊緣協作

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

學習目標

前面二十五章討論的都是「一台機器人」。但單機的視野、續航與運算終究有限:一台機器狗要花數小時才能巡完整棟系館,而十台可能只需二十分鐘。本章把視野擴大到群體,也完成第七篇的最後一塊拼圖。學完本章之後,同學應能夠達成下列目標:

- 166. 說明多機器人 SLAM 相對單機的優勢與新增的三大挑戰。
- 167. 說明地圖融合的流程與跨機器人閉環偵測的關鍵。
- 168. 比較集中式、分散式與去中心化三種協作架構的優缺點。
- 169. 說明雲端 SLAM 與邊緣 SLAM 的分工原則與適用場合。
- 170. 規劃一個多台機器狗校園巡檢的系統架構與任務分配。
- 171. 說明 SLAM 與 AI Agent 如何協作,以及 VLM 在多機協調中的角色。
- 172. 說明多機器人系統在隊形、通訊、資料同步與安全上的課題。

核心概念

中文名詞	英文名詞	概念說明
協作式 SLAM	Collaborative SLAM	多台機器人共同建立一致地圖的技術。
集中式架構	Centralized	所有資料匯集到中央伺服器處理。
分散式架構	Distributed	運算分散在多個節點,彼此交換資訊。
去中心化	Decentralized	無中央節點,機器人直接彼此通訊並各自融合。
地圖融合	Map Fusion / Merging	把多張地圖對齊並合併為一致的全域地圖。
跨機器人閉環	Inter-robot Loop Closure	不同機器人觀測到同一地點所建立的約束。
相對變換	Relative Transform	兩台機器人座標系之間的旋轉與平移關係。

中文名詞	英文名詞	概念說明
會合	Rendezvous	兩台機器人相遇並交換資訊的事件。
感知混淆	Perceptual Aliasing	不同地點看起來相同而造成的錯誤關聯。
離群點剔除	Outlier Rejection	排除錯誤閉環以免破壞全域地圖。
邊緣運算	Edge Computing	在靠近機器人的本地節點進行運算。
雲端運算	Cloud Computing	在遠端資料中心進行運算與儲存。
異質團隊	Heterogeneous Team	由不同類型機器人(地面、空中)組成的團隊。

26.1 單機 SLAM 與多機 SLAM

本節說明為什麼要用多台機器人,以及這個選擇帶來什麼新問題。

多機器人的四項優勢

- 速度:多台分頭探索,完成大範圍任務的時間大幅縮短。這是最直接的動機——第 23 章案例 23-3 的多機器人協作探索正是這個思路。
- 容錯:單台故障不影響整體任務。相較之下,單一昂貴機器人一旦故障,整個任務就中止。
- 視角互補:從多個位置同時觀測,能建立更完整的環境模型,也能減少遮蔽造成的盲區。
- 能力互補:異質團隊能結合不同平台的長處——地面機器人續航長、承載大;無人機視野廣、能到高處(第 25.7 節提過的地空協作)。

三大新挑戰

但多機器人不是「單機乘以 N」這麼簡單,它帶來三個單機不存在的問題:

339. 座標系統一:每台機器人開機時各自以自己的起點為原點(第 10 章)。要把它們的地圖合在一起,必須先知道彼此座標系之間的相對變換——而這個變換一開始是未知的。
340. 資料關聯的擴大:單機 SLAM 只需判斷「我現在看到的,是不是我以前看過的」(第 22 章的長期關聯);多機器人還要判斷「我看到的,是不是別人看過的」。搜尋空間變大,錯誤關聯的風險也隨之上升。
341. 通訊限制:機器人之間的頻寬有限、連線可能中斷、延遲不固定。所有協作機制都必須在這個現實下運作——這是本章反覆出現的主軸。

與單機技術的關係

好消息是:多機器人 SLAM 的核心技術,同學都已經學過了。

342. 每台機器人的自身定位建圖:第 13 至 25 章的各種單機 SLAM。

343. 跨機器人閉環偵測:第 19、21 章的地點辨識(詞袋模型)與幾何驗證。

344. 地圖融合的最佳化:第 12 章的位姿圖優化。

345. 錯誤閉環的處理:第 12、15 章的穩健核函數與離群點剔除。

換句話說,多機器人 SLAM 是把單機的技術「擴充到多個座標系與多個資料來源」。

理解這一點,同學就不會覺得本章的內容陌生——它是前面所有章节的自然延伸。

26.2 Map Fusion 地圖融合

地圖融合是多機器人 SLAM 的核心操作:把兩張(或多張)各自建立的地圖,對齊並合併成一致的全域地圖。

與單機閉環的異同

同學應該立刻聯想到第 22.6 節的 Atlas 地圖合併。兩者的數學本質相同——找出兩張地圖之間的相對變換,再以位姿圖優化把修正傳播出去。差別在於:

346. 單機的多地圖:兩張地圖來自同一台機器人的不同時段,感測器與參數完全相同。

347. 多機器人的地圖融合:兩張地圖來自不同機器人,可能使用不同的感測器、不同的視角、甚至不同的地圖表示。異質團隊的融合尤其困難。

融合的三步驟

- 偵測相遇:判斷兩台機器人是否觀測到同一個地點。這可以是實體的會合(兩台機器人互相看見),也可以是「地點辨識的重疊」——A 機器人現在看到的場景,B 機器人稍早也看過。

- 估計相對變換:確認重疊後,由共同觀測到的特徵或掃描,解出兩台機器人座標系之間的旋轉與平移(第 20 章的幾何求解,或第 17 章的點雲配準)。
- 融合與最佳化:把兩張位姿圖用這條「跨機器人的邊」連起來,執行位姿圖優化(第 12 章),使整個系統達成全域一致。

關鍵風險:錯誤的跨機器人閉環

第 12、15、21 章都強調過:一條錯誤的閉環會把地圖硬拉到錯誤位置。在多機器人系統中,這個風險更高,因為:

- 348. 候選空間更大:要比對的不只是自己的歷史,還有所有其他機器人的觀測。
- 349. 感知混淆更嚴重:不同機器人可能在不同的相似地點(例如兩間格局相同的教室),產生看似正確的錯誤匹配。
- 350. 後果更嚴重:一條錯誤的跨機器人邊,會同時破壞多台機器人的地圖。

因此穩健的離群點剔除是多機器人 SLAM 的必備元件。相關系統會整合穩健的離群點剔除機制來強化這個做法,並使其適應零星的機器人間通訊。另有系統採用分散式的漸進非凸化演算法,並把語意資料納入產出的地圖中。

通訊效率的考量

地圖融合需要交換資料,而頻寬有限。因此如何「用最少的資料達成最好的融合」,是這個領域的核心課題。有系統引入基於代數連通度最大化的新穎跨機器人閉環優先排序技術,以減少通訊並加速收斂;它也採用去中心化的鄰居管理與位姿圖優化方式,適合零星的機器人間通訊,並支援不同感測器類型的描述子——影像用 CosPlace 與 NetVLAD、光達掃描用 Scan Context。

這裡的核心洞見是:不是所有閉環候選都同等重要。有些閉環對改善全域一致性貢獻很大,有些則影響甚微。優先傳輸高價值的候選,能在有限頻寬下獲得最大效益——

這是一個極具工程智慧的思路。

26.3 Collaborative SLAM 的三種架構

多機器人 SLAM 的系統架構,是本章最重要的設計決策。多機器人 SLAM 架構的光譜,從集中式一路到完全去中心化。

一、集中式架構

所有機器人把資料傳給一台中央伺服器,由伺服器統一處理與最佳化,再把結果分發回去。

像 CCM-SLAM 這類集中式系統把資料匯集到伺服器進行聯合最佳化;這簡化了地圖融合,但引入了通訊瓶頸與單點故障。

351. 優點:地圖融合簡單(所有資料都在同一處)、全域最佳化容易、機器人端運算負擔輕。

352. 缺點:通訊瓶頸(所有資料都要傳)、單點故障(伺服器一掛全系統停擺)、擴展性差。

353. 代表系統:CCM-SLAM、COVINS(針對視覺慣性感測、以移除冗餘關鍵幀來稀疏化)、LAMP 2.0(光達系統,以圖神經網路預測每個跨機器人閉環候選的優先順序)。

二、分散式與去中心化架構

在分散式系統中,處理分散在多個節點或代理上;機器人蒐集資料並執行自己的定位與建圖,同時透過持續通訊彼此分享資訊,以建立完整的全域地圖。

去中心化的做法中,機器人直接彼此交換資訊,並在機載完成所有資料融合,不依賴中央實體來協調系統。

354. 優點:無單點故障、擴展性佳、對通訊中斷較有韌性。

355. 缺點:同步複雜度高、需要分散式最佳化演算法、每台機器人的運算負擔較重。

356. 代表系統:DOOR-SLAM 與 Kimera-Multi 執行點對點的位姿圖優化並具離群點剔除,以增加擴展性為代價換來更高的同步複雜度;Swarm-SLAM 提供了支援多種感測模態的 ROS 2 去中心化框架;LAMP 與 LAMP 2.0 則處理 DARPA 地下挑戰賽中的地下多機器人 SLAM(第 1 章案例 1-2)。

架構選擇的關鍵:通訊條件

集中式架構的缺點在於,它們需要與共同後端保持良好的通訊,而若該節點失效,整個系統就會失效;此外它們無法擴展到大型機器人團隊。

因此,只依賴機器人之間零星、間歇通訊的去中心化解法,更適合大範圍的場景。

架構	運算位置	優點	缺點	適用場景
集中式	中央伺服器	融合簡單、機器人端輕量	通訊瓶頸、單點故障	通訊良好的小型團隊
分散式	多節點協同	無單點故障、擴展性佳	同步複雜	中大型團隊
去中心化	各機器人機載	對通訊中斷有韌性	機載運算負擔重	通訊零星的大範圍任務

表 26-1 三種協作 SLAM 架構的比較

實務上的混合

真實系統往往不是純粹的某一種。例如:機器人在機載執行自己的 SLAM(去中心化的部分),但當通訊良好時把關鍵幀傳給伺服器做全域最佳化(集中式的部分)。這種混合

設計能兼顧兩者的優點,也是本書反覆強調的工程思維——不必在教科書式的分類中二選一。

26.4 Cloud SLAM 與 Edge SLAM

上一節談的是「機器人之間」如何分工,本節談的是「機器人與外部運算資源」如何分工。

三層運算的分工

層級	位置	延遲	運算能力	適合的任務
機載	機器人本身	極低	受限(第 24 章)	即時控制、姿態估計、避障
邊緣	本地伺服器 / 基地台	低(數毫秒至數十毫秒)	中至高	SLAM 後端、地圖融合、AI 推論
雲端	遠端資料中心	高(數十至數百毫秒)	極高	大規模地圖儲存、模型訓練、跨場域分析

表 26-2 機載、邊緣、雲端三層運算的分工

分工的基本原則

決定「什麼該在哪一層做」,有三個判準:

- 即時性要求:控制迴路必須在機載(第 25.1 節的無人機案例最為明顯);可以容忍數十毫秒延遲的任務才適合送到邊緣。
- 資料量:傳輸大量原始資料成本高。若能在機載先做壓縮或特徵提取,再傳輸精簡後的資料,效率會好得多(這正是 26.2 節通訊效率的思路)。

- 安全與隱私:涉及個人隱私的影像資料,是否適合傳上雲端?這在醫院、學校等場域是必須考慮的問題。

邊緣輔助的協作 SLAM

邊緣運算特別適合多機器人 SLAM,因為它同時解決了兩個問題:機器人端運算不足、以及集中式架構的延遲問題。

有系統以卸載繁重運算來最佳化伺服器端,同時以機載的視覺里程計維持代理的自主性。這是一個典型的分工:機器人自己做即時的前端(不依賴網路),把耗時的後端最佳化與地圖融合交給邊緣伺服器。

通訊技術的進步也在推動這個方向。有研究運用 5G 通訊以處理來自多台機器人的即時稠密體積資料,顯示高吞吐與低延遲如何讓大型感測器串流的卸載成為可能。第 5.4 節提過 5G 在機器人定位的應用,此處則是它在協作運算上的價值。

通訊品質作為規劃資源

一個近年重要的觀念轉變值得同學注意。有研究的核心想法是把通訊品質視為一種規劃資源,並結合多代理協作,使「導航斷崖」成為可管理的降級事件,而非硬性的運作極限。

這個思路極具啟發性:與其被動接受「這裡沒訊號」,不如主動把通訊品質納入路徑規劃——例如讓一台機器人停在訊號良好處作為中繼、或安排機器人定期回到通訊區交換資料。這與第 25.2 節「讓失效逐級降級」的設計哲學一脈相承。

26.5 多台機器狗校園巡檢

本節把前面的技術整合成一個具體的系統規劃,這也是第 30 章專題四的基礎。

為什麼需要多台

以大學的校園規模估算:一台機器狗以每秒 0.5 公尺的巡檢速度,要走遍一棟系館的所有走廊與樓層,可能需要數小時,而電池續航往往不足一小時。多台機器狗分頭進行,才能在單次充電內完成任務。

系統架構

圖 26-1 多機器狗校園巡檢的系統架構

【機載層】每台機器狗

- 光達慣性 SLAM(第 18 章)或視覺慣性 SLAM(第 22 章)
- 即時避障與步態控制
- 語意感知:YOLO 偵測巡檢目標(第 23 章)

↓ 傳送:關鍵幀描述子、位姿圖、巡檢發現

【邊緣層】校園內的本地伺服器

- 跨機器狗閉環偵測與地圖融合(26.2 節)
- 全域位姿圖優化(第 12 章)
- 任務分配與路線協調

↓ 傳送:精簡後的地圖、巡檢報告

【雲端層】(選用)

- 長期地圖儲存與跨時間比對(變化偵測)
- AI 模型訓練與更新
- 管理介面與報表

任務分配策略

357. 依區域劃分:最簡單——每台機器狗負責一個樓層或一個區域。優點是互不干擾、實作容易;缺點是無法動態調整,若某區提早完成,該機器狗就閒置。
358. 依邊界動態分配:第 23 章案例 23-3 提過的做法——把已探索與未探索的交界(邊界)動態分配給最合適的機器人。效率較高,但需要即時的協調機制。
359. 依能力分配:異質團隊的做法——機器狗負責樓梯與室內,無人機負責中庭與屋頂,輪型機器人負責平坦的長廊(續航最佳)。

實務課題

- 起始座標對齊:讓所有機器狗從同一個充電站出發,或在起點放置 AprilTag(第 23.3 節),即可直接建立共同的座標系,大幅簡化地圖融合。這是一個「用簡單方法避開困難問題」的好例子。
- 通訊覆蓋:校園的 Wi-Fi 未必處處覆蓋(地下室、樓梯間)。系統必須能容忍斷線,並在恢復連線時補傳資料——這正是 26.3 節去中心化架構的價值。
- 安全與互相避讓:多台機器狗可能在走廊相遇,需要協調通行。此外還要考慮與行人的互動(第 23 章的社交感知導航)。
- 電池管理:機器狗續航短,系統需規劃充電輪替——例如兩台巡檢、一台充電,輪流交替以維持持續覆蓋。

26.6 SLAM 與 AI Agent 協作

第 23.5 節介紹過 VLM 輔助導航,本節聚焦 AI 在多機器人協調中的角色。

VLM 作為全域規劃器

第 23 章案例 23-3 已詳述過這個方向:VLM 不做低階控制,而是扮演「指揮官」——看著全域地圖,運用常識推理判斷哪些方向最可能找到目標,再把任務分配給不同的機器人。

這個分工的優雅之處在於:VLM 的優勢是語意理解與常識推理(慢但聰明),機器人的優勢是即時感知與控制(快但不理解意義)。讓各自做擅長的事,是最合理的架構。

AI Agent 在多機系統中的三種角色

- 任務規劃者:把高階目標(「巡檢三樓所有教室」)分解為具體的子任務,並分配給各機器人。這需要理解場域的語意結構(語意地圖,第 3、23 章)。
- 異常判讀者:巡檢機器人回傳大量影像,由人工逐一檢視不切實際。AI Agent 能篩選出「需要人類注意」的異常(裂縫、漏水、堵塞的通道),大幅減輕人力負擔。
- 協調仲裁者:當多台機器人的需求衝突(都想去同一區域、都需要充電),Agent 依全域目標做出決策。

語意地圖的共享

多機器人協作的一個進階方向,是共享語意而非只共享幾何。有系統把語意資料納入產出的地圖中,能在僅依賴局部通訊的情況下,即時建立全域一致的度量語意三維網格模型,並對錯誤的閉環具有韌性。

共享語意的價值在於:機器人 A 辨識出「這是電梯」之後,機器人 B 不需要重新辨識,直接沿用即可。這不只省運算,也讓整個團隊對環境的理解保持一致。

務實的提醒

第 23.5 節強調過的限制,在多機器人系統中同樣適用甚至更嚴重:

360. 延遲:VLM 推論耗時,若每個決策都要等它,系統反應會過慢。因此 AI 應處理「秒級以上」的決策,即時避讓仍由傳統方法負責。
361. 幻覺:AI 判斷錯誤可能導致整個團隊往錯誤方向探索。系統應保留幾何層的驗證與人類的監督介面。

362. 通訊:AI 決策通常在邊緣或雲端執行,這意味著它依賴通訊。斷線時,機器人必須能退回預設的自主行為(26.4 節的降級思維)。

26.7 隊形、通訊、資料同步與安全

本節收攏多機器人系統的四項工程課題,它們雖不像演算法那樣光鮮,卻往往決定系統能否真正運作。

一、隊形與空間協調

363. 避免碰撞:多台機器人在同一空間移動,必須避免互撞。做法包括共享即時位置、劃分行進區域、以及在狹窄處(走廊、門口)設定通行規則。

364. 維持通訊鏈:若團隊需要保持連線,隊形必須考慮通訊範圍。有時會刻意讓一台機器人停在中繼位置(26.4 節提過的把通訊視為規劃資源)。

365. 覆蓋效率:隊形也影響探索效率。分散能加快覆蓋,但過度分散會失去互相支援與通訊的能力。

二、通訊

366. 頻寬限制:原始影像與點雲的資料量極大,不可能全部傳輸。實務上傳輸的是精簡的表示——關鍵幀描述子、位姿圖、稀疏特徵(26.2 節的效率考量)。

367. 間歇連線:實際場域中,連線會斷。系統設計必須假設「隨時可能斷線」,並在恢復時能補傳與重新同步。

368. 通訊拓樸:是所有機器人都能互相通訊,還是只有相鄰的能?這直接影響架構選擇(26.3 節)。

三、資料同步

369. 時間同步:這是多機器人系統最容易被低估的難題。每台機器人的時鐘略有差異,若不校準,融合時會出現時間錯位。第 5、10 章反覆強調過時間同步的重要

性,在多機系統中影響更大。常用做法是網路時間協定或以 GNSS 授時(第 5.1 節提過 GNSS 同時提供精確授時)。

370. 座標同步:即 26.1 節提過的座標系統一問題。除了靠地圖融合估計,也可用共同的起始點或環境中的已知標記(26.5 節的實務做法)。

371. 版本一致:當地圖被更新(例如閉環修正後),所有機器人都需要拿到最新版本。若某台機器人用著舊地圖行動,可能導致衝突。

四、安全

372. 通訊安全:機器人之間的通訊若被竊聽或竄改,攻擊者可能注入錯誤的地圖資料,誤導整個團隊。加密與身分驗證是必要的。

373. 資料隱私:巡檢機器人拍攝的影像可能包含人臉與私人空間。在校園、醫院等場域,必須明確規範資料的蒐集、儲存與存取權限。這不只是技術問題,也是倫理與法規問題。

374. 實體安全:多台機器人同時運作,對周圍人員的風險增加。系統需有全體緊急停止機制,且每台機器人都應有獨立的安全底線(即使失去通訊也不會失控)。

375. 故障隔離:一台機器人故障或行為異常時,系統應能將其隔離,避免它的錯誤資料污染全域地圖——這正是 26.2 節離群點剔除在系統層級的延伸。

第七篇的收束

第七篇從第 24 章的毫瓦級微型平台出發,經過第 25 章的無人機導航,到本章的群體協作,呈現了一條清晰的線索:當單一機器人的能力受限時,答案不是把它做得更大更強,而是讓它更小、更省電、然後用更多台協同工作。

這個思路與自然界的策略不謀而合——螞蟻與蜜蜂個體能力有限,群體卻能完成驚人的任務。本篇介紹的技術,正是讓機器人團隊具備這種集體智慧的基礎。

下一篇,我們回到最實際的層面:如何把這一切真正跑起來。第八篇「ROS / Python / C++ 實作」將帶同學建置開發環境、實作 2D 光達建圖與導航、執行視覺 SLAM,並完成完整的專題。理論走到這裡,是時候讓機器人真的動起來了。

公式或流程

一、跨機器人的座標變換

設機器人 A 與 B 各自的世界座標系為 W_A 、 W_B 。地圖融合的核心,是求出兩者之間的相對變換:

$$T_{\{W_A \leftarrow W_B\}}$$

取得這個變換後,B 的任一位姿即可轉換到 A 的座標系:

$$p_{\{W_A\}} = T_{\{W_A \leftarrow W_B\}} \cdot p_{\{W_B\}}$$

這正是第 10 章「變換組合就是矩陣連乘」在多機器人上的應用。若團隊有 N 台機器人,原則上只需 N - 1 個相對變換就能把所有地圖统一到同一座標系(構成一棵生成樹,呼應第 21.6 節的概念)。

二、多機器人位姿圖的目標函數

把所有機器人的位姿圖合併後,整體最佳化的目標為:

$$\min \sum_{\{\text{機器人內部邊}\}} \|e\|^2_{\Sigma} + \sum_{\{\text{跨機器人邊}\}} \rho(\|e\|^2_{\Sigma})$$

第一項是各機器人自己的里程計與閉環約束,第二項是跨機器人的閉環約束。

請特別注意第二項外面的穩健核函數 ρ ——跨機器人邊的錯誤風險遠高於機器人內部的邊(26.2 節),因此必須用穩健核函數保護。這個細節體現了「依可信度差異化處理」的思想,與第 11 章卡爾曼增益、第 12 章資訊矩陣一脈相承。

三、通訊量的估算

多機器人系統的通訊需求,可粗估為:

$$\text{總通訊量} \approx N \times (N - 1) \times \text{每對機器人交換的資料量} \times \text{頻率}$$

注意 N 的平方成長——這正是集中式與全連通架構難以擴展的數學原因。以 10 台機器人為例,兩兩通訊的配對數是 45 對;若增加到 20 台,配對數躍升至 190 對。

這條式子解釋了 26.2 節「閉環候選優先排序」與 26.3 節「去中心化架構」的必要性:當 N 變大,必須設法讓實際通訊量遠低於這個上限——只在需要時通訊、只傳最有價值的資料。

四、多機器人 SLAM 的完整流程

圖 26-2 多機器人協作 SLAM 的處理流程

【各機器人機載】

每台獨立執行單機 SLAM(第 13–25 章)→ 各自的位姿圖與地圖

↓ 提取精簡表示(關鍵幀描述子、稀疏特徵)

【通訊】

依優先順序交換候選資訊(頻寬有限,只傳高價值的)

↓

【跨機器人閉環偵測】

地點辨識比對 → 幾何驗證 → 離群點剔除

↓ 取得跨機器人的相對變換

【地圖融合】

建立跨機器人邊 → 合併位姿圖

↓

【全域最佳化】

位姿圖優化(含穩健核函數)→ 全域一致的地圖

↓

【分發】把修正後的結果回傳各機器人

案例

案例 26-1 地下挑戰賽:多機器人 SLAM 的極限考場

第 1 章案例 1-2 介紹過 DARPA 地下挑戰賽,此處從機器人的角度重看這場比賽。

比賽要求機器人團隊在人造隧道、地下城市與天然洞穴中自主探索、建圖並尋找目標物。這個場域幾乎集合了本章所有挑戰的最惡劣版本:

376. 通訊極差:地下環境無線電訊號衰減嚴重,機器人往往在深入後就完全失聯——這正是 26.3 節「零星、間歇通訊」的極端情況。

377. 環境退化:隧道的幾何高度重複(第 17.3 節的結構退化),感知混淆風險極高,錯誤閉環的機率大增。

378. 異質團隊:參賽隊伍普遍組合輪型機器人、四足機器狗與無人機,各自適應不同的地形(26.1 節的能力互補)。

這場比賽直接催生了多個重要的系統。LAMP 與 LAMP 2.0 正是為了處理 DARPA 地下挑戰賽中的地下多機器人 SLAM 而發展;LAMP 2.0 引入以圖神經網路為基礎的集中式優先排序機制,預測每個跨機器人閉環候選經過位姿圖優化後的結果。

這個「用 AI 預測哪些閉環候選值得處理」的做法極具啟發性:當資源(通訊、運算)有限時,與其平均對待所有候選,不如用學習方法預測價值再分配資源。這與 26.2 節的優先排序思想相同,只是用了不同的技術手段。

對同學的啟示是:極端場景往往是技術突破的溫床。地下挑戰賽的參賽系統,如今已成為多機器人 SLAM 的重要基礎——困難的問題逼出了好的答案。

案例 26-2 通訊不可靠時,系統會壞到什麼程度

多機器人 SLAM 的論文,多半在通訊良好的條件下評測。但真實場域中,通訊往往時斷時續——這對系統的影響有多大?

有研究專門量化了這個問題,結果相當驚人:相對於可靠通訊的基準,Swarm-SLAM 的誤差增加為 5.08 倍、Kimera-Multi 為 6.17 倍。

換句話說,同一套系統在通訊不可靠時,誤差可能惡化五到六倍。這個數字提醒我們:實驗室的漂亮資料,未必能在真實場域重現。

這也解釋了為何近年的研究越來越重視通訊條件。第 26.4 節提過的「把通訊品質視為規劃資源」正是一個回應:與其被動接受通訊不良的後果,不如主動規劃路線以維持連線,或設計能在斷線時優雅降級的架構。

對做專題的同學而言,這是一個重要提醒:評測你的多機器人系統時,務必加入「通訊不良」的情境測試。只在理想條件下測試,會嚴重高估系統的實際能力。這與第 14.6 節「常見失敗案例」、第 25.5 節「挑戰測試」的精神一致——主動尋找失效邊界,是負責任的工程實踐。

案例 26-3 從模擬到現場:異質團隊的開發工具

多機器人系統的開發有一個現實困難:硬體成本。十台機器狗的價格對多數實驗室

都是沉重負擔,而且調度十台機器人做實驗的人力與場地成本同樣可觀。

模擬因此成為多機器人研究的必要工具。近年已有專為異質多機器人設計的開源模擬環境出現。例如有開源模擬器與資料蒐集管線,建立在虛幻引擎 5 的模擬器之上,解決了先前框架的關鍵架構限制,以支援在大規模、照片級真實、動態環境中同時運作無人空中與地面載具;它引入了新的路徑點追蹤地面載具控制器以對應既有的無人機控制介面,並為異質平台提供共享的導航堆疊,涵蓋建圖、可通行性分析、規劃與控制。

這類工具對教學特別有價值:同學能在模擬中建立十台、二十台機器人的團隊,測試任務分配、地圖融合與通訊策略,完全不需要對應數量的實體硬體。等到演算法在模擬中驗證可行,再用兩三台實體機器人驗證關鍵環節即可。

這正是本書從第 8 章就建議的學習路徑:先模擬、再便宜硬體、後追求規模。多機器人系統把這個原則的價值放大到極致——沒有模擬,多機器人研究幾乎寸步難行。

實作活動

實作 26-1 多機器人協作建圖模擬

目的:透過模擬完整體驗多機器人 SLAM 的核心議題——地圖融合、通訊限制、任務分配與錯誤閉環的風險。本實作以模擬為主,不需多台實體機器人。

第一部分:兩機器人地圖融合(全體)

266. 沿用第 24 章實作 24-1 的模擬環境(或第 13 章的 2D 建圖模擬),建立兩台虛擬機器人,各自從房間的不同角落出發,以不同路線探索並建立自己的地圖。

267. 觀察問題:把兩張地圖直接疊在一起顯示,會發現它們完全對不上——因為各自以自己的起點為原點(26.1 節的座標系統一問題)。

268. 人工融合:假設你「知道」兩台機器人起點的相對位置,依公式一做座標變換,把 B 的地圖轉到 A 的座標系,再疊合顯示。觀察融合後的地圖是否完整。
269. 自動估計變換:讓兩台機器人的探索路線有部分重疊,用第 17 章的 ICP 對兩張地圖(或重疊區域的點)做配準,自動估出相對變換。與真值比較誤差。
270. 加入漂移:讓兩台機器人的位姿估計都帶有累積誤差,重新融合,觀察地圖如何錯位,並用位姿圖優化(第 12 章)修正。

第二部分:通訊限制實驗(全體或大學部)

271. 設定通訊範圍:規定兩台機器人只有在距離小於某個值時才能交換資料。模擬它們各自探索、偶爾相遇的過程。
272. 比較三種通訊條件:全時連線、只在相遇時通訊、以及隨機斷線(例如每步有 30% 機率無法通訊),比較最終地圖的品質差異。
273. 量化影響:計算三種條件下的地圖誤差,與案例 26-2 提到的「誤差增加五到六倍」對照,看看你的模擬是否呈現類似趨勢。
274. 資料量統計:記錄三種條件下的總通訊量,思考「通訊量」與「地圖品質」的取捨。

第三部分:任務分配與錯誤閉環(大學部)

275. 探索效率比較:讓 1 台、2 台、4 台機器人分別探索同一個房間直到全區覆蓋,記錄各自所需的步數。畫出「機器人數量 vs 完成時間」的曲線,觀察效益是否線性成長(提示:通常不會,因為會有重複探索)。
276. 任務分配策略:比較「隨機探索」與「依區域劃分」兩種策略的效率差異,對照 26.5 節的討論。
277. 錯誤閉環實驗:人為加入一條錯誤的跨機器人閉環邊(例如把 A 的某位姿錯誤地連到 B 的不相關位姿),執行位姿圖優化,觀察地圖如何被破壞。

278. 穩健化:在最佳化中加入穩健核函數(可用 scipy 的 `soft_l1` 等損失函數),重跑一次,觀察錯誤閉環的破壞是否被抑制。這直接驗證公式二中 ρ 的作用。

討論問題

- 124. 未做座標變換時,兩張地圖疊起來是什麼樣子?這說明了多機器人 SLAM 相對單機的什麼新問題?
- 125. ICP 自動估計的相對變換誤差有多大?若兩台機器人的探索路線幾乎沒有重疊,會發生什麼?
- 126. 通訊條件從「全時連線」變成「只在相遇時通訊」,地圖誤差增加了幾倍?與案例 26-2 的資料相比如何?
- 127. 機器人從 1 台增加到 4 台,完成時間縮短為幾分之一?為什麼不是四分之一?
- 128. (大學部)加入錯誤閉環後,地圖的哪些部分被破壞得最嚴重?穩健核函數把破壞抑制了多少?這對真實系統的設計有什麼意涵?

延伸挑戰(大學部)

其一:實作簡易的閉環優先排序——當通訊量有限,只能傳輸部分閉環候選時,設計一個評分機制(例如依據「這條閉環能連接多遠的兩段軌跡」或「涉及的機器人是否已有其他連結」),優先傳輸高分候選,比較它與隨機選擇的效果差異。這對應 26.2 節的核心思想。其二:異質團隊模擬——加入一台「速度快但感測範圍小」與一台「速度慢但感測範圍大」的機器人,設計任務分配策略以發揮各自優勢,並與同質團隊比較效率。這呼應 26.1 節的能力互補與第 25.7 節的地空協作。

延伸閱讀

174. Y. Tian et al., "Kimera-Multi: Robust, Distributed, Dense Metric-Semantic SLAM for Multi-Robot Systems," IEEE T-RO, 2022.(分散式多機器人語意 SLAM 的代表作,程式碼公開)
175. P.-Y. Lajoie and G. Beltrame, "Swarm-SLAM: Sparse Decentralized Collaborative SLAM Framework for Multi-Robot Systems," IEEE RA-L, 2024.(ROS 2 去中心化框架,含閉環優先排序技術)
176. Y. Chang et al., "LAMP 2.0: A Robust Multi-Robot SLAM System for Operation in Challenging Large-Scale Underground Environments," IEEE RA-L, 2022.(DARPA 地下挑戰賽的代表系統)
177. P. Schmuck and M. Chli, "CCM-SLAM: Robust and Efficient Centralized Collaborative Monocular SLAM," Journal of Field Robotics, 2019.(集中式協作 SLAM 的經典)
178. P.-Y. Lajoie et al., "DOOR-SLAM: Distributed, Online, and Outlier Resilient SLAM for Robotic Teams," IEEE RA-L, 2020.(分散式架構與離群點剔除)
179. awesome-Collaborative-SLAM 開源清單(GitHub).(協作 SLAM 論文與程式碼的完整整理,研究入門的好起點)
180. S. Macenski et al., "Robot Operating System 2: Design, Architecture, and Uses in the Wild," Science Robotics, 2022.(ROS 2 的多機器人支援,第八篇實作的基礎)

小結

147. 多機器人 SLAM 的四項優勢為速度、容錯、視角互補與能力互補;三大新挑戰為座標系統一、資料關聯範圍擴大、以及通訊限制。其核心技術是單機技術

(SLAM、地點辨識、位姿圖優化、離群點剔除)在多座標系與多資料來源上的擴充。

148. 地圖融合分三步驟:偵測相遇、估計相對變換、融合與最佳化;最大風險是錯誤的跨機器人閉環——候選空間更大、感知混淆更嚴重、後果更嚴重,因此穩健的離群點剔除是必備元件,而閉環候選的優先排序能在有限頻寬下獲得最大效益。

149. 三種架構各有取舍:集中式融合簡單但有通訊瓶頸與單點故障、無法擴展到大型團隊;分散式與去中心化執行點對點位姿圖優化,擴展性佳且對通訊中斷有韌性,代價是同步複雜度與機載運算負擔;只依賴零星間歇通訊的去中心化解法更適合大範圍場景。

150. 機載、邊緣、雲端三層依即時性、資料量與隱私分工;邊緣輔助的做法是以機載視覺里程計維持自主性、把繁重運算卸載到伺服器;近年更把通訊品質視為規劃資源,使導航斷崖成為可管理的降級事件而非硬性極限。

151. 多機器狗校園巡檢採機載、邊緣、雲端三層架構;任務分配可依區域、依邊界動態或依能力;實務課題包含起始座標對齊(可用共同充電站或 AprilTag 簡化)、通訊覆蓋、互相避讓與電池輪替。

152. AI Agent 在多機系統中扮演任務規劃者、異常判讀者與協調仲裁者;共享語意而非只共享幾何能讓團隊對環境的理解保持一致,但須留意延遲、幻覺與通訊依賴三項限制。

153. 四項工程課題:隊形需兼顧避撞、通訊鏈與覆蓋效率;通訊須面對頻寬限制與間歇連線;資料同步含時間、座標與版本一致;安全涵蓋通訊加密、資料隱私、實體安全與故障隔離。

第七篇到此完成。從第 24 章的毫瓦級微型平台、第 25 章的無人機導航,到本章的群體協作,呈現了一條清晰的線索:當單一機器人的能力受限時,答案不是把它做得更大

更強,而是讓它更小、更省電,然後用更多台協同工作——這與螞蟻和蜜蜂的策略不謀而合。

下一篇,我們回到最實際的層面:如何把這一切真正跑起來。第八篇「ROS / Python / C++ 實作」將帶同學建置開發環境、實作 2D 光達建圖與導航、執行視覺 SLAM,並完成完整的專題與評量。理論走到這裡,是時候讓機器人真的動起來了。

台灣自編大學教科書

[@prof](#)

《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第八篇 ROS / Python / C++ 實作

第 27 章

SLAM 程式設計環境

編者：李世淵 (機器人叫獸)

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

學習目標

前七篇建立了完整的理論與系統知識。從本篇開始,我們回到最實際的層面:如何把這一切真正跑起來。本章是第八篇的入口,負責把開發環境備妥、把工具鏈打通——這是所有後續實作的地基。學完本章之後,同學應能夠達成下列目標:

- 173. 說明 Ubuntu 與 ROS 2 的版本對應關係,並完成環境安裝。
- 174. 說明 Python 與 C++ 在 ROS 2 中的分工,並理解 CMake 與建置系統的角色。
- 175. 說明話題、服務、動作三種通訊機制的差異與適用場合。
- 176. 使用 rosbag2 錄製與重播感測資料,並理解 MCAP 格式的意義。
- 177. 使用 RViz2、Gazebo 與 Foxglove 進行視覺化、模擬與除錯。
- 178. 說明閱讀 GitHub 開源專案的系統性方法。
- 179. 規劃從高中積木式實作到大學原始碼實作的學習路徑。

核心概念

中文名詞	英文名詞	概念說明
長期支援版	LTS	有較長維護期限的穩定版本,適合教學與產品。
工作空間	Workspace	存放 ROS 套件原始碼與建置產物的目錄結構。
套件	Package	ROS 中程式碼組織的基本單位。
建置工具	colcon	ROS 2 用來編譯工作空間中所有套件的工具。
節點	Node	執行特定功能的獨立程式(第 3 章)。
話題	Topic	節點間以發布訂閱模式交換資料的通道。
服務	Service	一問一答式的同步通訊機制。

中文名詞	英文名詞	概念說明
動作	Action	可回報進度、可取消的長時間任務通訊機制。
啟動檔	Launch File	一次啟動多個節點並設定參數的腳本。
資料錄製	rosbag2	錄製與重播 ROS 話題資料的工具。
訊息擷取格式	MCAP	自我描述、可快速隨機存取的記錄檔格式。
視覺化工具	RViz2	ROS 2 內建的三維視覺化工具。
模擬器	Gazebo	提供物理模擬與感測器模擬的環境。

27.1 Ubuntu 與 ROS / ROS 2

第 3.5 節介紹過 ROS 的角色與核心概念,本節聚焦環境建置的實務。

為什麼是 Ubuntu

ROS 2 支援多種作業系統,但 Ubuntu 是所謂的第一級平台(Tier 1)——也就是官方主要開發與測試的平台。實務上,絕大多數開源 SLAM 專案的說明文件、相依套件與社群支援都以 Ubuntu 為前提。因此本書的實作一律以 Ubuntu 為基準。

版本對應:最容易踩的第一個坑

ROS 2 與 Ubuntu 的版本是綁定的——每個 ROS 2 發行版只支援特定的 Ubuntu 版本。裝錯版本會導致一連串難以排查的相依性錯誤,這是初學者最常見的挫折來源。

ROS 2 每年 5 月發布新版本,偶數年為長期支援版(LTS)。截至本書撰寫時,主要的版本對應如下:

ROS 2 發行版	發布年	Ubuntu 版本	類型	備註
Humble Hawksbill	2022	22.04	LTS	支援至 2027 年 5 月,最成熟穩定
Jazzy Jalisco	2024	24.04	LTS	rosbag2 預設改用 MCAP 格式
Kilted Kaiju	2025	24.04	一般版	rosbag2 效能改善
Lyrical Luth	2026	26.04	LTS	最新的長期支援版

表 27-1 ROS 2 發行版與 Ubuntu 版本對應

版本選擇建議

- 教學與初學:建議選擇成熟的 LTS 版本。Humble 是目前教學資源最豐富的選擇——網路上絕大多數教學、開源專案與問答都以它為基準,遇到問題最容易找到答案。
- 新專案:若要長期維護,可選最新的 LTS 版本以獲得較長的支援期。
- 沿用既有專案:直接對齊該專案指定的版本,不要嘗試「順便升級」——這往往會浪費大量時間在相依性問題上。
- 重要提醒:全班應統一版本。不同版本之間的錄製資料未必能互通,教學時版本不一致會造成極大困擾。

安裝方式的三種選擇

- 379. 原生安裝:直接在實體電腦上安裝 Ubuntu 與 ROS 2。效能最好、硬體支援最完整,是連接實體機器人時的必要選擇。缺點是需要重灌或雙系統。
- 380. 虛擬機:在 Windows 或 macOS 上以虛擬機執行 Ubuntu。安裝容易、不影響原系統,適合純軟體的模擬練習。缺點是 3D 加速效能較差(Gazebo 可能會卡)、USB 裝置連接較麻煩。
- 381. 容器(Docker):以容器封裝完整環境。優點是環境一致、可快速重建、不污染主系統——這對教學特別有價值,教師可以提供統一的容器映像,免去全班各自安裝的混亂。缺點是需要額外設定才能使用圖形介面與 USB 裝置。

教學上的實務建議

依多年教學經驗,建議的策略是:高中與大學部入門階段使用容器或虛擬機,讓所有同學在一致的環境中操作模擬;進入實體機器人實作階段,再改用原生安裝。這能把「環境問題」的干擾降到最低,讓課堂時間花在 SLAM 本身而非除錯安裝。

27.2 Python、C++、CMake

ROS 2 同時支援 Python 與 C++, 兩者在 SLAM 開發中有清楚的分工。

兩種語言的分工

面向	Python(rclpy)	C++(rclcpp)
開發速度	快,適合快速原型	較慢,需編譯
執行效能	較慢	快,適合高頻運算
適合的任務	控制邏輯、資料分析、視覺化	SLAM 核心、影像處理、即時控制
學習門檻	低	較高(記憶體管理、模板)
本書用途	實作練習、資料處理	閱讀開源 SLAM 原始碼

表 27-2 Python 與 C++ 在 ROS 2 中的分工

一個實用的原則是:先用 Python 把邏輯跑通,若效能不足再用 C++ 重寫關鍵部分。同學會發現主流的 SLAM 系統(ORB-SLAM3、Cartographer、LOAM 系列)幾乎都是 C++ 寫的——因為它們需要處理高頻的影像與點雲資料,Python 的效能無法勝任。

CMake 與建置系統

C++ 專案需要編譯,而管理編譯過程的工具就是 CMake。它負責:找出相依的函式庫在哪裡、決定編譯順序、產生實際的編譯指令。

對 SLAM 學習者而言,CMake 最常出現的場合是「編譯開源專案時出錯」。第 21.7 節提過 ORB-SLAM2 在新系統上可能編不過,問題往往就出在 CMakeLists.txt 中指定的函式庫版本與系統上的版本不符。

- 必備技能:看懂 CMakeLists.txt 中的 find_package(尋找相依套件)與 target_link_libraries(連結函式庫)兩個指令,就能解決大部分編譯問題。

- 常見錯誤:找不到套件(需安裝或指定路徑)、版本不符(需修改版本要求或安裝對應版本)、C++ 標準不符(需調整編譯選項)。

工作空間與 colcon

ROS 2 的程式碼組織成「工作空間」,其中放置多個「套件」。建置工具 colcon 會自動分析套件之間的相依關係,依正確順序編譯全部套件。

典型的工作流程是:建立工作空間目錄 → 把套件原始碼放進 src 子目錄 → 執行 colcon 建置 → 載入環境設定(source)→ 執行節點。這個流程同學會在整個第八篇反覆使用。

環境設定的常見陷阱

- 忘記 source:建置完成後必須載入環境設定,系統才找得到新編譯的套件。這是最常見的「明明編譯成功卻找不到節點」原因。
- 多個工作空間疊加:若同時有多個工作空間,載入順序會影響哪個版本的套件被使用。出現詭異行為時,先檢查環境變數。
- 修改程式後未重新建置:Python 節點在某些設定下可以直接生效,C++ 則一定要重新編譯。這個差異常讓初學者困惑。

27.3 ROS Topic、Service、Action

第 3.5 節簡介過這三種通訊機制,本節說明它們的差異與選用時機——這是設計 ROS 系統的核心判斷。

話題(Topic):持續的資料流

話題採發布訂閱模式:發布者持續送出資料,訂閱者持續接收。雙方不需要知道彼此存在,只要約定好話題名稱與訊息格式。

- 特性:非同步、一對多、持續流動、不保證接收方收到。
- 適合:感測器資料(光達掃描、影像、IMU)、狀態廣播(位姿、地圖)、控制指令(速度)。
- SLAM 中的例子:光達驅動發布 /scan、SLAM 節點訂閱它並發布地圖(第 14.5 節的實作流程)。

服務(Service):一問一答

服務是同步的請求回應模式:用戶端送出請求,等待伺服器回覆。

- 特性:同步、一對一、有明確的回覆、呼叫端會等待。
- 適合:短時間就能完成的查詢或設定,例如「取得目前地圖」「重設里程計」「切換模式」。
- 注意:因為呼叫端會等待,不適合用在耗時的操作——會阻塞整個節點。

動作(Action):可監控的長任務

動作專為「需要花時間、且過程中想知道進度」的任務設計。它結合了三個元素:目標(要做什麼)、回饋(進行到哪了)、結果(最後如何)。

- 特性:非同步、可回報進度、可中途取消。
- 適合:導航到某個目標點、執行一段軌跡、完成一次掃描任務。
- SLAM 中的例子:第 28 章的 Nav2 導航就是以動作介面提供——你送出「去這個座標」的目標,系統持續回報剩餘距離,你也可以隨時取消。

機制	通訊模式	是否等待	可否取消	典型用途
話題	發布訂閱,一對多	不等待	不適用	感測器資料、狀態、指令
服務	請求回應,一對一	等待回覆	不可	快速查詢與設定

機制	通訊模式	是否等待	可否取消	典型用途
動作	目標回饋結果	不等待,可監控	可以	導航、長時間任務

表 27-3 三種通訊機制的比較

選用的判斷流程

遇到「這個功能該用哪種機制」時,依序問三個問題:

382. 這是持續流動的資料嗎?是 → 話題。

383. 是不是很快就能回答的一次性請求?是 → 服務。

384. 是不是耗時且需要監控進度的任務?是 → 動作。

這個判斷看似簡單,卻是 ROS 系統設計品質的關鍵。用錯機制(例如用服務執行耗時的導航)會導致系統阻塞、難以除錯——這是實作時值得留意的細節。

27.4 rosbag 資料錄製與重播

rosbag2 是 SLAM 開發最重要的工具之一,它讓「錄一次、分析無數次」成為可能。

為什麼 rosbag 如此重要

- 可重現性:同一段資料反覆重播,能公平比較不同演算法或參數——第 15 章實作 15-1 的「Cartographer vs Gmapping 對照實驗」正需要這個能力。
- 離線開發:資料錄好後,不需要機器人也能開發演算法。這對硬體有限的教學環境極為實用。
- 除錯:出問題時把資料錄下來,可以反覆重播找出原因,不必在現場一次次重跑實驗。
- 資料集建立:第 25.5 節建議的「自製場域資料集」,正是以 rosbag 錄製的。

MCAP:新一代的記錄格式

rosbag2 的儲存格式近年有重要演進。MCAP(Message Capture)是來自 Foxglove 的記錄容器格式,自 ROS 2 Jazzy 起成為 rosbag2 的預設儲存格式。

它的特性包含:經過索引以支援快速隨機存取、內嵌訊息定義因而具自我描述性,並支援 ROS、protobuf、JSON、flatbuffers 等多種格式。

對同學而言,「自我描述」這一點特別重要:記錄檔本身就包含了訊息格式的定義,因此即使在沒有安裝對應訊息套件的電腦上,也能正確讀取——這大幅改善了資料交換與長期保存的可行性。

錄製的實務要點

385. 錄什麼:SLAM 開發至少要錄感測器原始資料(/scan 或影像)、IMU、里程計、以及 TF 座標轉換。TF 常被遺漏,但缺少它重播時什麼都對不上。

386. 不要錄什麼:輸出結果(地圖、估測軌跡)通常不需要錄,因為重播時會重新產生。錄了反而佔空間並可能造成混淆。

387. 時間標記:確認所有資料的時間戳記正確——這是第 5、10、26 章反覆強調的工程細節,在重播時尤其關鍵。

388. 檔案大小:影像與點雲資料量龐大,一分鐘可能就數 GB。錄製前先估算並確認儲存空間充足。

重播的常見問題

實務上常見的三個問題值得同學留意:ROS 版本不符——在某個版本錄製的資料可能無法在另一版本順暢重播;缺少 TF 轉換——許多資料集缺少轉換資訊,導致 RViz 視覺化失效;以及話題名稱不符——若你的演算法預期 /scan 但資料中是 /lidar_scan,就必須做話題重新對應。

第三點特別實用:重播時可以「改名」,讓錄好的資料餵給預期不同話題名稱的演算法,不必修改程式碼。這是整合他人資料集時的常用技巧。

27.5 RViz、Gazebo、Foxglove

本節介紹三個貫穿第八篇的工具:視覺化、模擬與除錯分析。

RViz2:看見機器人的世界

RViz2 是 ROS 2 內建的三維視覺化工具,也是本書從第 10 章開始就反覆提到的除錯利器。它能同時顯示:

- 感測器資料:光達掃描點、點雲、影像。
- 座標框架:TF 樹的各個座標軸(第 10 章「地圖歪了先看 TF」的檢查方法)。
- SLAM 輸出:佔據格地圖、估測軌跡、粒子雲(第 11 章案例 11-2)、關鍵幀與共視圖(第 21 章)。
- 導航資訊:規劃路徑、代價地圖、目標點(第 28 章)。

近年 RViz2 也持續改進,例如加入基於正規表示式的 TF 過濾器讓你能依名稱顯示或隱藏座標轉換、話題頻率會自動顯示在狀態列、以及可訂閱 point_cloud_transport 話題等功能。

Gazebo:沒有機器人也能實驗

Gazebo 提供物理模擬與感測器模擬,讓同學能在虛擬環境中完整驗證 SLAM 演算法。本書從第 8 章就一再建議「先模擬、再便宜硬體、後追求精度」,Gazebo 正是這條路徑的第一站。

- 模擬什麼:機器人的運動學與動力學、光達與相機的量測(含雜訊)、環境的幾何與碰撞。

- 教學價值:每位同學都能有一台「自己的機器人」,不受硬體數量限制;可以任意設計場景(長走廊、玻璃隔間)來重現第 13、14 章討論的各種困難情境;也能安全地做危險實驗。
- 版本搭配:ROS 2 從 Jazzy 起,官方會為每個 ROS 版本推薦特定的 Gazebo 版本,並提供 Gazebo 供應商套件讓使用者能快速安裝上手。安裝時務必依官方建議的搭配,以免相容性問題。

Foxglove:更強大的除錯與分析

Foxglove Studio 是由前自駕車公司工程師於 2021 年創辦的公司所開發的多協定視覺化工具,同時支援桌面、網頁與雲端,並支援 ROS 1、ROS 2、MCAP、MAVLink、MQTT 與原始 WebSocket,提供 3D、繪圖、影像、狀態轉換、地圖等多種面板。

它之所以受到重視,是因為 RViz2 雖然功能足夠但有其限制:它依賴 ROS,難以檢視非 ROS 的系統;記錄檔分析能力較弱;難以協作或分享(只能靠截圖與錄影);且沒有網頁或平板的支援。

- 對教學的價值:網頁版讓同學不必安裝就能檢視資料;能分享連結給老師或同學,方便討論與批改。
- 對研究的價值:強大的繪圖與時間序列分析,適合做第 21.7 節的軌跡誤差分析。
- 建議:RViz2 用於即時的三維視覺化(執行時看機器人在做什麼),Foxglove 用於事後的資料分析(找出哪裡出了問題)。兩者互補而非取代。

27.6 GitHub 開源專案閱讀方法

SLAM 領域的一大幸運,是幾乎所有重要系統都開源。但面對一個有數萬行程式碼的專案,同學常不知從何下手。本節提供一套系統性的閱讀方法。

五個步驟

389. 先讀論文,再讀程式:這是最重要的建議。ORB-SLAM3、Cartographer、LOAM 都有對應的論文,先理解演算法架構(如本書第 15、18、21、22 章所做的),再看程式碼會事半功倍。直接跳行程式碼往往會迷失在細節中。
390. 讀 README 與文件:確認相依套件、支援的平台版本、以及執行範例的指令。特別注意硬體需求——第 18 章案例 18-3 提過,許多同學卡關的原因是硬體規格不符,而這在文件中往往寫得清清楚楚。
391. 先跑起來,再讀程式:用官方提供的資料集把系統跑通,建立成功經驗。能跑之後再讀程式碼,你會有具體的參照——「剛剛畫面上那條軌跡,是這段程式產生的」。
392. 從進入點往下追:找到 main 函式或 ROS 節點的進入點,順著資料流往下讀。不要企圖一次讀懂全部,而是追蹤「一幀資料進來後,依序經過哪些函式」。
393. 加輸出訊息驗證理解:在關鍵位置加上輸出訊息,執行後觀察它們的觸發時機與頻率。第 21 章實作 21-1 的延伸挑戰正是這個方法——追蹤 ORB-SLAM2 在什麼情況下切換三種追蹤策略。

閱讀時的重點對照

讀 SLAM 專案時,建議依本書的架構去找對應的模組:

- 前端在哪:特徵提取或掃描匹配的程式(第 13、19 章)。
- 後端在哪:呼叫最佳化函式庫的程式(第 12 章的 g2o、Ceres、GTSAM)。
- 閉環在哪:地點辨識與驗證的程式(第 21、22 章)。
- 資料結構:關鍵幀、地圖點、位姿圖如何定義(第 21.6 節)。

把程式碼的模組對應到本書的章節,你會發現理論與實作的對應關係非常清楚——這也是本書刻意以「架構」而非「數學細節」為主軸的原因。

善用社群資源

- Issues:專案的 issue 區是寶庫。你遇到的問題,很可能別人已經問過並得到解答。搜尋錯誤訊息往往能直接找到答案。
- Fork 與分支:原始專案可能已停止維護,但社群常有維護中的分支(第 21.7 節提過的建議)。留意 star 數較多、近期有更新的 fork。
- 貢獻回饋:若你解決了一個未被記錄的問題,把解法寫成 issue 或 pull request 回饋社群。這是參與開源生態最直接的方式,也是很好的履歷素材。

27.7 從高中積木式實作到大學原始碼實作

本章最後,規劃一條從高中到大學、乃至研究所的完整學習路徑。這也是全書教學設計的總綱。

階段	目標	主要工具	本書章節
一:觀察體驗	理解 SLAM 在做什麼	模擬器、掃地機器人觀察	第 1-3、16 章
二:積木式操作	會用現成系統建圖導航	Gazebo、啟動檔、參數調整	第 14、15、28 章
三:資料與分析	會錄資料、評估品質	rosbag2、Python、evo	第 13.7、21.7 節
四:讀原始碼	理解系統如何實現	GitHub、除錯工具	第 21、22 章
五:修改與創新	能改良或整合系統	C++、CMake、演算法設計	第 29、30 章

表 27-4 SLAM 學習的五個階段

各階段的教學建議

- 階段一(高中適用):不碰程式碼,重點是建立直覺。用模擬器操作、觀察家中掃地機器人(第 16.7 節)、做紙上活動(第 12、16、24 章的實作)。目標是讓同學「知道 SLAM 在解什麼問題、為什麼困難」。
- 階段二(高中進階/大學部):使用現成系統,學會啟動、設定參數、觀察結果。重點是理解參數的意義與影響,而非撰寫程式。第 14、15 章的實作正是設計在這個層次。
- 階段三(大學部):開始寫程式——但寫的是分析與工具程式,而非 SLAM 核心。錄製資料、計算誤差、繪製圖表。這個階段培養的是「用資料說話」的工程素養。
- 階段四(大學部後期/專題):閱讀開源系統的原始碼,理解理論如何落實為程式。此時前面三階段的經驗會成為理解的支架。
- 階段五(專題/研究生):修改既有系統或整合多個系統以解決特定問題。第 30 章的專題與第 31 章的 AI 整合,都屬於這個層次。

三個教學心法

- 394. 先成功、再深入:讓同學在每個階段都有可展示的成果。太早要求讀原始碼,容易造成挫折而放棄。
- 395. 刻意示範失敗:本書每一章都討論「常見失敗」,是有意的設計。看到系統在什麼情況下壞掉,比看它順利運作學到更多——第 14.6 節、第 25.5 節的挑戰測試都是這個精神。
- 396. 環境問題與學習內容分開:如 27.1 節建議,用容器或統一環境把安裝問題降到最低。課堂時間應該花在 SLAM,而不是排除相依性錯誤。

環境與工具至此備齊。下一章,我們把第四篇的 2D SLAM 理論真正跑起來:從硬體組裝、TF 設定、里程計發布,到 Gmapping 與 Cartographer 建圖、AMCL 定位、以

及完整的導航堆疊。第 28 章將是本書第一個從頭到尾的完整系統實作。

公式或流程

一、環境建置的完整流程

圖 27-1 從零到能跑 SLAM 的環境建置流程

① 確認版本對應:選定 ROS 2 發行版 → 查出對應的 Ubuntu 版本(表 27-1)

↓

② 安裝 Ubuntu:原生 / 虛擬機 / 容器(依用途選擇,27.1 節)

↓

③ 安裝 ROS 2:依官方文件加入軟體來源並安裝桌面版(含 RViz2)

↓

④ 安裝模擬器:依該 ROS 版本官方推薦的 Gazebo 版本

↓

⑤ 建立工作空間:建立目錄 → 放入套件原始碼 → colcon 建置

↓

⑥ 載入環境設定(source)→ 驗證:啟動範例節點,確認能通訊

↓

⑦ 安裝 SLAM 套件:slam_toolbox / Cartographer / Nav2 等

↓

⑧ 冒煙測試:啟動模擬器與 SLAM,在 RViz2 中看到地圖成長

第八步「冒煙測試」是關鍵的檢查點:若能看到地圖逐步長出來,表示整條工具鏈都通了,可以放心進入第 28 章的實作。若卡住,依 27.1 至 27.2 節的常見陷阱逐項排查。

二、通訊機制的選用判斷

圖 27-2 話題、服務、動作的選用決策

這個功能需要什麼樣的通訊?

↓

① 是持續流動的資料嗎?(感測器、狀態、控制指令)

└ 是 → 【話題】發布訂閱,非同步,一對多

└ 否 → 進入 ②

↓

② 是很快就能回答的一次性請求嗎?(查詢、設定)

└ 是 → 【服務】請求回應,呼叫端等待

└ 否 → 進入 ③

↓

③ 是耗時且需監控進度的任務嗎?(導航、掃描任務)

└ 是 → 【動作】可回報進度、可取消

三、rosvag 錄製與重播的完整流程

圖 27-3 資料錄製到分析的工作流程

【錄製階段】

- ① 確認要錄的話題:感測器資料 + IMU + 里程計 + TF(勿漏 TF)
- ② 估算檔案大小與儲存空間
- ③ 執行錄製 → 產生 MCAP 格式的記錄檔

**【重播階段】**

- ④ 檢查話題名稱是否與演算法預期相符,必要時做重新對應
- ⑤ 重播資料 → 同時啟動 SLAM 節點
- ⑥ 在 RViz2 中即時觀察,或以 Foxglove 事後分析

**【分析階段】**

- ⑦ 輸出估測軌跡 → 與真值比較 → 計算 ATE / RPE(第 21.7 節)
- ⑧ 調整參數 → 回到 ⑤ 重播同一份資料 → 公平比較

注意第八步:因為重播的是同一份資料,不同參數的比較才是公平的。這正是 rosbag 對研究與教學最大的價值——把「感測資料的隨機性」這個變因固定下來。

案例

案例 27-1 半堂課的代價:版本不符的連鎖反應

這是幾乎每個 SLAM 課程都會發生的場景,值得同學引以為戒。

狀況是這樣的:同學下載了某個開源 SLAM 專案,依 README 執行編譯,結果出現一長串錯誤訊息。上網搜尋、逐一嘗試各種解法、安裝了幾個看起來相關的套件——兩

個小時過去,問題不但沒解決,系統還被裝壞了。

根源往往極其單純:專案是為某個 ROS 2 版本寫的,而同學裝的是另一個版本。相依的函式庫版本不同,連鎖反應下產生數十個看似無關的錯誤。

這個案例的教訓有三:

- 第一,先確認版本對應。開始編譯前,先花五分鐘確認專案支援的 ROS 版本與你的環境是否相符——這五分鐘能省下兩小時。
- 第二,錯誤訊息要從第一個看起。編譯錯誤常有連鎖效應,後面數十個錯誤可能都源於第一個。從最後往回看是常見的誤區。
- 第三,容器是好朋友。用容器封裝環境,裝壞了就重建,不會污染主系統,也能確保全班環境一致(27.1 節的建議)。

更深一層的啟示是:工程實作中,「環境問題」佔用的時間往往遠超預期。這不是同學能力不足,而是這個領域的常態。有經驗的工程師與初學者的差別,不在於不會遇到問題,而在於能快速判斷問題的類別並找到對應的解法。

案例 27-2 從 rosbag 到論文:一份資料的多重價值

rosbag 的價值,在一個真實的研究流程中展現得最清楚。

假設一位同學要做「不同 SLAM 系統在大學走廊的表現比較」的專題。若沒有 rosbag,他必須:為每個系統各跑一次實體實驗——但每次實驗的路線、速度、光線、人流都不完全相同,比較結果毫無說服力;而且每次都要借用機器人、佈置場地,耗時費力。

有了 rosbag,流程完全不同:錄製一次高品質的資料(含影像、IMU、光達、TF),之後所有系統都用同一份資料測試。這帶來四個好處:

- 公平比較:所有系統面對完全相同的輸入,差異純粹來自演算法。
- 可重現:別人拿到你的資料,能重現你的結果——這是科學研究的基本要求。
- 效率:參數調整可以反覆進行,不需重跑實驗。一個下午能測試數十組參數。
- 長期價值:這份資料明年、後年的學生仍能使用。第 25.7 節建議的「錄製並公開大學場域資料集」,正是這個思路。

對大學的具體建議是:每一屆的專題生都貢獻一份自己場域的資料,累積下來就是一個有價值的在地資料集。這對後續研究、教學示範、甚至與其他學校交流,都是實質的資產。

案例 27-3 工具的演進:從 rosbag 到 MCAP、從 RViz 到 Foxglove

本章介紹的工具鏈,近年有顯著的演進,值得同學留意這個趨勢。

記錄格式方面,MCAP 已成為事實上的新標準,取代了原本的 rosbag 格式,並自 ROS 2 Jazzy 起成為 rosbag2 的預設儲存格式。它的關鍵優勢是自我描述——記錄檔內嵌訊息定義,不需額外的套件就能解讀。

視覺化方面,Foxglove 補足了 RViz2 的限制:RViz2 依賴 ROS、難以檢視非 ROS 系統、記錄檔分析能力弱、難以協作分享、且無網頁或平板支援。Foxglove 則同時支援桌面、網頁與雲端,並能處理 ROS 1、ROS 2、MCAP、MAVLink、MQTT 等多種協定。

這個演進背後有一個值得注意的模式:推動改變的往往不是 ROS 核心團隊,而是產業界的實務需求。Foxglove 由前自駕車公司的工程師創辦,而他們正是把自駕車開發中的痛點——大量資料的分析、跨團隊協作、非 ROS 系統的整合——轉化為工具。

對同學的啟示是:工具會不斷演進,不必執著於某個特定工具。重要的是理解「這個工具解決什麼問題」——當你清楚問題的本質,就能快速上手新工具,也能判斷什麼時候

候該換工具。這與本書一貫的主張相同:理解原理比記憶操作重要。

實作活動

實作 27-1 建置 SLAM 開發環境並完成冒煙測試

目的:完整建置可用的 ROS 2 開發環境,熟悉三種通訊機制與 rosbag 工具,並完成第一次模擬建圖。本實作是第 28、29 章的必要前置作業。

第一部分:環境建置(全體)

279. 依表 27-1 選定 ROS 2 版本與對應的 Ubuntu 版本,記錄你的選擇與理由。
280. 依 27.1 節選擇安裝方式(原生 / 虛擬機 / 容器),完成 Ubuntu 與 ROS 2 桌面版的安裝。
281. 驗證安裝:啟動 ROS 2 內建的範例節點(如 talker 與 listener),確認兩者能正常通訊。
282. 建立工作空間:建立目錄結構,用 colcon 建置一個空的工作空間,並載入環境設定。
283. 安裝模擬器與 TurtleBot3 套件,啟動模擬環境,確認能在畫面中看到虛擬機器人。

第二部分:通訊機制探索(全體)

284. 列出目前系統中所有活躍的話題,找出 /scan、/odom、/tf 等 SLAM 相關的話題。
285. 檢視 /scan 話題的訊息內容,對照第 8.3 節的 LaserScan 格式,找出 angle_min、angle_increment、ranges 等欄位。
286. 觀察話題的發布頻率,記錄光達、里程計、TF 各自的頻率,思考為什麼它們不同。

- 287. 列出可用的服務與動作,找出與導航相關的動作介面(第 28 章會用到)。
- 288. 用 Python 撰寫一個最簡單的節點:訂閱 `/scan`,計算並印出「當前最近障礙物的距離與方向」。這是第 8 章實作 8-1 延伸挑戰的 ROS 版本。

第三部分:rosvbag 錄製與重播(全體)

- 289. 遙控模擬機器人在環境中移動,同時用 rosvbag2 錄製 `/scan`、`/odom`、`/tf` 與 `/imu`(若有),觀察產生的檔案格式與大小。
- 290. 停止模擬器,單獨重播錄好的資料,在 RViz2 中顯示光達掃描與 TF,確認能正常重現。
- 291. 話題重新對應練習:重播時把 `/scan` 改名為 `/laser_scan`,觀察原本訂閱 `/scan` 的節點是否還能運作,理解 27.4 節提到的重新對應技巧。
- 292. 刻意不錄 TF 再重播一次,觀察 RViz2 中會發生什麼——親身驗證 27.4 節「勿漏 TF」的提醒。

第四部分:冒煙測試——第一次建圖(全體)

- 293. 啟動模擬器與 SLAM 套件(建議用 slam_toolbox 或 Cartographer),在 RViz2 中顯示地圖、光達與 TF。
- 294. 緩慢遙控機器人繞行環境一圈,觀察佔據格地圖如何逐步長出來(第 13 章的理論在此具象化)。
- 295. 儲存地圖,並用第 13.7 節的方法評估品質:牆是否筆直?迴路是否閉合?
- 296. 把整個流程寫成一個啟動檔,讓下次能一鍵啟動模擬器、SLAM 與 RViz2。

討論問題

- 129. 你選擇了哪個 ROS 2 版本?依 27.1 節的建議,這個選擇對「找教學資源」與「長期維護」各有什麼利弊?

130. 光達、里程計、TF 三者的發布頻率各是多少?為什麼它們的頻率不同?(提示:第 7、13 章的感測器特性)
131. 不錄 TF 的資料重播時發生了什麼?這印證了第 10 章的哪個觀念?
132. 依圖 27-2 的決策流程,若要實作「讓機器人執行一次完整的房間掃描任務,過程中可查詢進度也可中止」,應該用哪種通訊機制?為什麼?
133. 第一次建圖的結果如何?若不理想,依第 14.6 節的失敗模式,你認為最可能的原因是什麼?

延伸挑戰(大學部)

其一:容器化你的開發環境——撰寫一個容器設定檔,封裝完整的 ROS 2 環境與所需的 SLAM 套件,讓同學能一鍵取得一致的環境。這對教學助教與專題團隊極為實用,也是現代軟體工程的必備技能。其二:開源專案閱讀練習——選一個本書介紹過的開源 SLAM 專案(建議從 `slam_toolbox` 開始,程式規模適中),依 27.6 節的五個步驟閱讀,並繪製一張「資料如何從輸入流到輸出」的流程圖。把你的流程圖與本書對應章節的架構圖比較,看看理論與實作的對應關係。

延伸閱讀

181. ROS 2 官方文件(docs.ros.org).(安裝指引、教學與 API 文件,實作的第一手參考)
182. REP-2000: ROS 2 Releases and Target Platforms(ros.org/reps).(版本與平台對應的官方規範,27.1 節的依據)
183. S. Macenski et al., "Robot Operating System 2: Design, Architecture, and Uses in the Wild," Science Robotics, 2022.(ROS 2 設計理念的權威介紹)

- 184. Gazebo 官方文件之 Installing Gazebo with ROS(gazebo-sim.org). (ROS 與 Gazebo 版本搭配的官方建議)
- 185. MCAP 官方文件(mcap.dev)與 Foxglove 文件(docs.foxglove.dev). (新一代記錄格式與視覺化工具)
- 186. slam_toolbox 開源專案(GitHub). (ROS 2 主流的 2D SLAM, 程式規模適中, 適合作為原始碼閱讀的入門)
- 187. ROBOTIS TurtleBot3 e-Manual(emanual.robotis.com). (教學平台的完整實作指引, 含模擬與實體機器人)

小結

- 154. ROS 2 與 Ubuntu 的版本是綁定的, 裝錯會造成連鎖的相依性錯誤; ROS 2 每年 5 月發版、偶數年為 LTS, 教學建議選用資源最豐富的成熟 LTS 版本, 且全班應統一版本。安裝方式有原生、虛擬機與容器三種, 教學階段建議用容器以確保環境一致。
- 155. Python 適合快速原型與資料分析、C++ 適合 SLAM 核心與高頻運算; CMake 管理 C++ 的編譯, 看懂 find_package 與 target_link_libraries 就能解決多數編譯問題; colcon 依相依關係建置工作空間, 建置後務必載入環境設定。
- 156. 三種通訊機制各有適用: 話題用於持續流動的資料(感測器、狀態、指令)、服務用於快速的一次性請求、動作用於耗時且需監控進度的任務(如導航); 用錯機制會導致系統阻塞與除錯困難。
- 157. rosbag2 讓「錄一次、分析無數次」成為可能, 支撐可重現性、離線開發、除錯與資料集建立; MCAP 自 ROS 2 Jazzy 起成為預設格式, 具索引與自我描述特性; 錄製時勿漏 TF, 重播時留意版本、TF 與話題名稱三個常見問題。

158. RViz2 用於即時三維視覺化、Gazebo 提供無需硬體的模擬(版本須依官方推薦搭配)、Foxglove 補足 RViz2 在記錄檔分析、協作分享與非 ROS 系統上的限制;三者互補而非取代。
159. 閱讀開源專案的五步驟:先讀論文再讀程式、讀 README 確認相依與硬體需求、先跑起來再讀程式、從進入點順資料流往下追、加輸出訊息驗證理解;並善用 issue 區與社群維護的分支。
160. 學習路徑分五階段:觀察體驗、積木式操作、資料與分析、讀原始碼、修改與創新;教學心法為先成功再深入、刻意示範失敗、以及把環境問題與學習內容分開。

環境與工具至此備齊,同學已經有能力把前面所有理論真正跑起來。下一章,我們把第四篇的 2D SLAM 從頭到尾實作一次:硬體組裝與 TF 設定、里程計發布、Gmapping 與 Cartographer 建圖、AMCL 定位,最後接上完整的導航堆疊,讓機器人真正自己走到目標點。這將是本書第一個完整的端到端系統實作。

台灣自編大學教科書

@prof

《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第 28 章

2D 光達小車建圖實作

編者：李世淵 (機器人叫獸)

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

學習目標

前二十七章的理論與工具至此全部備齊。本章是本書第一個從頭到尾的完整系統實作：從硬體與座標設定開始，經過建圖、定位，最後讓機器人自己走到目標點。學完本章之後，同學應能夠達成下列目標：

- 180. 說明 2D SLAM 小車的硬體組成與各元件的角色。
- 181. 撰寫 URDF 機器人描述檔並正確設定 TF 座標樹。
- 182. 由輪速編碼器計算並發布里程計資料。
- 183. 執行 2D SLAM 建圖並儲存地圖檔。
- 184. 說明 AMCL 定位的設定要點與常見問題。
- 185. 說明 Nav2 導航堆疊的架構與一次導航任務的完整資料流。
- 186. 依系統性的順序排除導航系統的故障。

核心概念

中文名詞	英文名詞	概念說明
機器人描述檔	URDF	以 XML 描述機器人的連桿、關節與感測器安裝位置。
連桿與關節	Link / Joint	URDF 的基本元素：剛體與連接它們的關節。
狀態發布器	robot_state_publisher	依 URDF 發布各連桿之間 TF 變換的節點。
差速驅動	Differential Drive	以左右兩輪速度差控制方向的驅動方式。
里程計	Odometry	由輪速積分推算的位姿，提供 odom→base_link 變換。
代價地圖	Costmap	在佔據格上加註通行成本的導航用地圖。
膨脹層	Inflation Layer	把障礙物向外擴張安全距離的代價地圖層。

中文名詞	英文名詞	概念說明
全域規劃器	Global Planner	在靜態地圖上規劃起點到目標的完整路徑。
控制器	Controller	把路徑轉為速度指令的局部規劃元件。
行為樹	Behavior Tree	Nav2 用來組織導航任務流程的架構。
復原行為	Recovery Behavior	導航失敗時嘗試脫困的動作,如原地旋轉、後退。
生命週期節點	Lifecycle Node	有明確啟動狀態轉換的 ROS 2 節點類型。
速度指令	cmd_vel	送給底盤的線速度與角速度指令。

28.1 硬體:小車底盤、馬達、編碼器、2D 光達

本節說明一台能跑 SLAM 的 2D 小車需要哪些元件,以及選型時的考量。

五大元件

- 底盤與馬達:最常見的是差速驅動——左右兩個驅動輪加一到兩個萬向輪。它結構簡單、控制直觀(第 10.3 節的 2D 位姿正好對應),是教學平台的標準選擇。
- 編碼器:裝在馬達軸或輪軸上,計算輪子轉了多少。這是里程計的資料來源(28.3 節),也是第 13.4 節「里程計預測、光達校正」中不可或缺的一半。
- 2D 光達:第 8 章介紹過的旋轉式雷射掃描器,提供 /scan 話題。教學用的入門級型號量測範圍約 8 至 12 公尺,對室內建圖已足夠。
- 微控制器:負責馬達驅動與編碼器讀取,並與上層電腦通訊。它處理的是毫秒級的即時控制,不適合放在單板電腦上做。
- 單板電腦:執行 ROS 2 與 SLAM 演算法。這個「微控制器負責即時控制、單板電腦負責運算」的分工,正是第 24 章案例 24-2 討論過的雙晶片架構在教學平台上的體現。

選型的三個考量

397. 光達安裝高度與遮蔽:光達必須有清晰的 360 度視野。若機身或其他元件擋住部分角度,那些方向就永遠是盲區。安裝時務必實際檢查掃描是否被遮擋。
398. 編碼器解析度:解析度太低,里程計的精度就差。一般教學平台的編碼器解析度足夠,但若使用自製底盤,需確認每圈的脈衝數足以分辨小幅度的移動。
399. 運算能力:2D SLAM 對運算的需求不高(相較第五、六篇的 3D 與視覺 SLAM),常見的單板電腦即可勝任。若要同時跑 Nav2 導航,則需留意記憶體是否充足。

三種平台選擇

- 現成教學平台:整合完整、文件齊全、有社群支援,能讓同學快速進入 SLAM 本身而非硬體除錯。適合課程時間有限的情況。
- 自組平台:成本較低、學習深度更高——同學能親手處理每一個環節,對系統的理解遠比使用現成平台深刻。適合專題與有充裕時間的課程。
- 純模擬:完全不需硬體,但能完整體驗 28.2 至 28.7 節的所有流程。第 27 章建議的「先模擬、再硬體」路徑,在此章尤其適用——先在模擬中把整條管線跑通,再搬到實體車上,能省下大量除錯時間。

28.2 TF 設定與 URDF

第 10 章強調過「地圖歪了先看 TF」,本節說明如何正確建立座標樹。這是整個實作最容易出錯、也最需要謹慎的環節。

URDF:機器人的骨架描述

URDF(統一機器人描述格式)以 XML 描述機器人的物理結構:有哪些剛體(連桿)、它們如何連接(關節)、以及各自的尺寸與相對位置。

對 SLAM 而言,URDF 最重要的功能是描述感測器的安裝位置——也就是第 5、10 章反覆提到的外參。若 URDF 中寫的光達位置與實際安裝差了幾公分或幾度,建出的地圖就會模糊或錯位。

典型的座標樹結構

第 10.7 節介紹過 ROS 的標準框架階層,在實體小車上的具體形式為:

圖 28-1 2D SLAM 小車的完整 TF 樹

map(地圖座標系) 全域固定、不漂移、會跳變

↓ 由 SLAM 或 AMCL 發布

odom(里程計座標系) 連續平滑、長期漂移

↓ 由里程計節點發布(28.3 節)

base_footprint(機器人在地面的投影,選用)

↓

base_link(機器人本體中心)

└─→ laser(光達) ↖

└─→ imu_link(IMU) 由 URDF 與 robot_state_publisher

└─→ left_wheel 依固定的安裝關係發布

└─→ right_wheel ↙

誰負責發布哪一段

這是初學者最常混淆的地方,務必弄清楚:

- map → odom:由 SLAM 系統(建圖時)或 AMCL(導航時)發布。官方文件明確指出,這個變換是 Nav2 系統的主要需求之一。
- odom → base_link:由里程計系統發布。里程計系統的角色就是提供這個變換,而其來源可以是光達、雷達、輪速編碼器、視覺慣性里程計或 IMU;目標是提供一個基於機器人運動的平滑連續局部座標系。
- base_link → 各感測器:由 robot_state_publisher 依據 URDF 提供,它會發布 base_link 到各感測器的變換。

常見的 TF 錯誤

400. 座標樹斷裂:某一段沒有節點發布,導致 map 到 laser 的完整鏈路斷開。RViz 中會顯示「找不到變換」的錯誤。
401. 外參填錯:光達的安裝位置或朝向填錯。症狀是掃描點與實際環境錯位——把機器人放在牆邊,若 RViz 中的掃描點與牆的位置對不上,就是這個問題。
402. 重複發布:兩個節點同時發布同一段變換,造成衝突與跳動。這在整合多個套件時容易發生。
403. 時間戳記問題:變換的時間戳記與感測資料不同步,導致高速移動時明顯錯位(第 10.6 節提過時間戳記的重要性)。

驗證的方法

- 視覺化檢查:在 RViz 中同時顯示 TF 座標軸與光達掃描,確認掃描點與機器人的相對位置合理。
- 產生 TF 樹圖:用工具產生座標樹的圖形,確認結構完整、沒有斷裂或重複。
- 靜態驗證:把機器人靜置在已知位置(例如緊靠牆角),檢查掃描點是否落在正確的距離上。

28.3 里程計資料發布

里程計是 2D SLAM 的另一半基石。本節說明它如何從編碼器讀值變成 ROS 的里程計訊息。

從編碼器到位姿

差速驅動小車的里程計計算,是第 3 章航位推算與第 10.3 節座標轉換的直接應用:

404. 讀取編碼器:取得左右輪在這個時間間隔內各轉了多少。
405. 換算輪子位移:轉動圈數乘上輪子周長,得到左右輪各自走了多遠。

406. 計算機體運動:由左右輪位移的平均值得到前進距離、由兩者的差除以輪距得到轉角(公式見本章公式一)。
407. 更新位姿:依第 10.3 節的 2D 位姿複合公式,把相對運動累加到當前位姿,並記得處理角度環繞。
408. 發布:發布里程計訊息(含位姿與速度),同時發布 `odom` $\rightarrow$ `base_link` 的 TF 變換。

校正:精度的關鍵

第 14.6 節提過「輪子打滑與里程計失準」是建圖失敗的首要原因,而其中最常被忽略的是參數校正。兩個關鍵參數是:

- 輪徑:若設定值與實際不符,直線行走的距離就會系統性偏差,導致整張地圖等比例放大或縮小。校正方法是讓小車直走一段固定距離(例如 3 公尺),比較里程計讀值與捲尺實測值。
- 輪距:兩個驅動輪的中心距離。若設錯,轉彎角度就會偏差,這對建圖的影響比輪徑更嚴重——因為角度誤差會透過後續的移動被放大(第 7.3 節討論過姿態誤差的放大效應)。校正方法是讓小車原地轉 10 圈,比較里程計累計角度與 3600 度的差異。

這兩項校正應該在第一次建圖之前完成。許多同學花數小時調 SLAM 參數,卻沒發現問題出在未校正的里程計——這是實作中最常見的冤枉路。

里程計的品質要求

里程計的目標是提供一個平滑而連續的局部座標系,而全域定位系統會更新相對於全域座標系的變換以補償里程計的漂移。

請注意「平滑而連續」這個要求:里程計不需要絕對準確(漂移會被 SLAM 修正),但

必須平滑不跳變。若里程計因為程式錯誤或通訊丟包而出現跳躍,SLAM 的運動預測就會失效,建圖必然失敗。

進階:融合 IMU

純輪速里程計的角度估計容易因打滑而失準。實務上常以濾波器融合輪速與 IMU 的角速度(第 7、11 章),得到更穩健的里程計。ROS 2 生態中可用 robot_localization 套件的 EKF 節點來提供 odom → base_link 變換,這在地面不平或容易打滑的場域特別有幫助。

28.4 2D SLAM 建圖

前置作業完成後,終於可以建圖了。本節說明流程與工具選擇。

工具選擇

第 14、15 章介紹過 Gmapping 與 Cartographer,而在現代 ROS 2 環境中,還有第三個常用選擇。Nav2 官方把 SLAM Toolbox 列為預設的 SLAM 演算法,用於定位並產生靜態地圖。

工具	理論基礎	特點	適用
slam_toolbox	圖優化(第 12 章)	ROS 2 預設、參數簡潔、支援續建地圖	教學首選
Cartographer	圖優化 + 子地圖	閉環強、支援 3D、參數較多	大場域、需閉環
Gmapping	粒子濾波(第 11 章)	概念直觀、ROS 1 時代主流	理解粒子濾波

表 28-1 三種 2D SLAM 工具的比較

教學建議:先用 `slam_toolbox` 建立成功經驗(參數少、容易上手),再用 `Cartographer` 體驗閉環的威力(第 15 章實作 15-1),最後可用 `Gmapping` 對照理解粒子濾波與圖優化的差異。

建圖流程

409. 啟動硬體:機器人驅動、光達、里程計節點全部啟動,確認 `/scan` 與 `/odom` 都有資料。
410. 檢查 TF:確認座標樹完整(28.2 節的驗證方法)。這一步不可跳過。
411. 啟動 SLAM:啟動選定的 SLAM 節點。要確認它有提供 `map` → `odom` 變換並發布 `/map` 話題。
412. 啟動 RViz:顯示地圖、掃描、TF 與機器人模型,以便即時觀察。
413. 緩慢遙控建圖:第 14.6 節強調過,「開慢一點」是新手成功率最大的變數。轉彎時尤其要慢。
414. 評估品質:依第 13.7 節檢查牆是否筆直、迴路是否閉合、有無重影。
415. 儲存地圖:用 `nav2_map_server` 儲存,會產生 `.yaml` 與 `.pgm` 兩個檔案來表示地圖。

地圖檔的兩個檔案

- `.pgm`:灰階影像檔,就是佔據格地圖的視覺化——黑色為障礙、白色為自由空間、灰色為未知(第 13.1 節的三態)。可以用一般看圖軟體開啟檢視。
- `.yaml`:描述檔,記錄影像檔名、解析度(每像素幾公尺)、原點座標、以及判定佔據與自由的機率門檻。

這兩個檔案就是第 14.5 節提過的地圖產物。日後導航時由 `map_server` 載入,供 AMCL 定位與路徑規劃使用。

28.5 AMCL 定位

地圖建好之後,日常運行不需要再建圖,只需在已知地圖中定位——這就是第 3、11 章介紹過的 AMCL。

AMCL 是什麼

AMCL 是一套機率式的 2D 移動機器人定位系統,實作了自適應(或 KLD 取樣)的蒙地卡羅定位方法,以粒子濾波追蹤機器人在已知地圖上的位置。它已不再是 SLAM 演算法,因為它不建立環境地圖。

演算法的每次迭代,會比較地圖的樣貌與當前的雷射掃描,只保留那些讓資料與地圖最吻合的位置。這正是第 11 章粒子濾波「預測—秤重—重採樣」循環的實現。

AMCL 與 SLAM 的分工

項目	SLAM(建圖階段)	AMCL(導航階段)
是否建圖	是	否,使用既有地圖
地圖來源	即時建立	由 map_server 載入
運算負擔	較高	較低
提供的 TF	map → odom	map → odom
使用時機	首次進入新環境	日常運行

表 28-2 SLAM 與 AMCL 的分工

注意兩者都提供 map → odom 變換——這正是第 10.7 節設計的巧妙之處:對下游的導航系統而言,不論是 SLAM 還是 AMCL 在提供定位,介面完全相同,因此可以無縫切換。

關鍵參數

- 粒子數:與第 14.3 節的討論相同,多則穩健但耗運算。AMCL 的「自適應」特性能依據不確定性動態調整粒子數——不確定時多用粒子,收斂後減少。
- 運動雜訊模型:描述里程計有多不準的參數。其中控制旋轉對機器人旋轉估計影響的參數,若調高會讓系統對旋轉運動更謹慎、假設更大的不確定性;調低則對旋轉估計更有信心,但若感測器不夠準確會有風險。
- 更新門檻:機器人移動或轉動多少才執行一次濾波更新。設定較小的角度門檻(例如 0.05 弧度而非預設的 0.2,約 11.5 度)能讓機器人在較少的旋轉後就更新,適合緩慢或精確的移動;調高則減少更新次數以節省運算。
- 變換容許時間:發布的變換被視為有效的時間長度,預設 1 秒是合理值;調高有助於處理通訊延遲但可能使用過時的位置,調低則確保資料較新但可能造成抖動。

初始位姿:最常見的問題

AMCL 啟動時需要一個初始位姿的猜測。若沒有給,粒子會撒滿整張地圖,可能需要機器人移動一段距離才能收斂,甚至收斂到錯誤位置(第 11 章的多峰問題)。

- 手動設定:在 RViz 中用「2D Pose Estimate」工具點選機器人的大致位置與朝向。這是最常用的做法。
- 固定起點:讓機器人總是從同一個位置(例如充電站)啟動,並在設定中寫死初始位姿。這是第 26.5 節提過的實務技巧。
- 視覺標記輔助:用 AprilTag 提供絕對位姿(第 23.3 節),自動完成初始化。

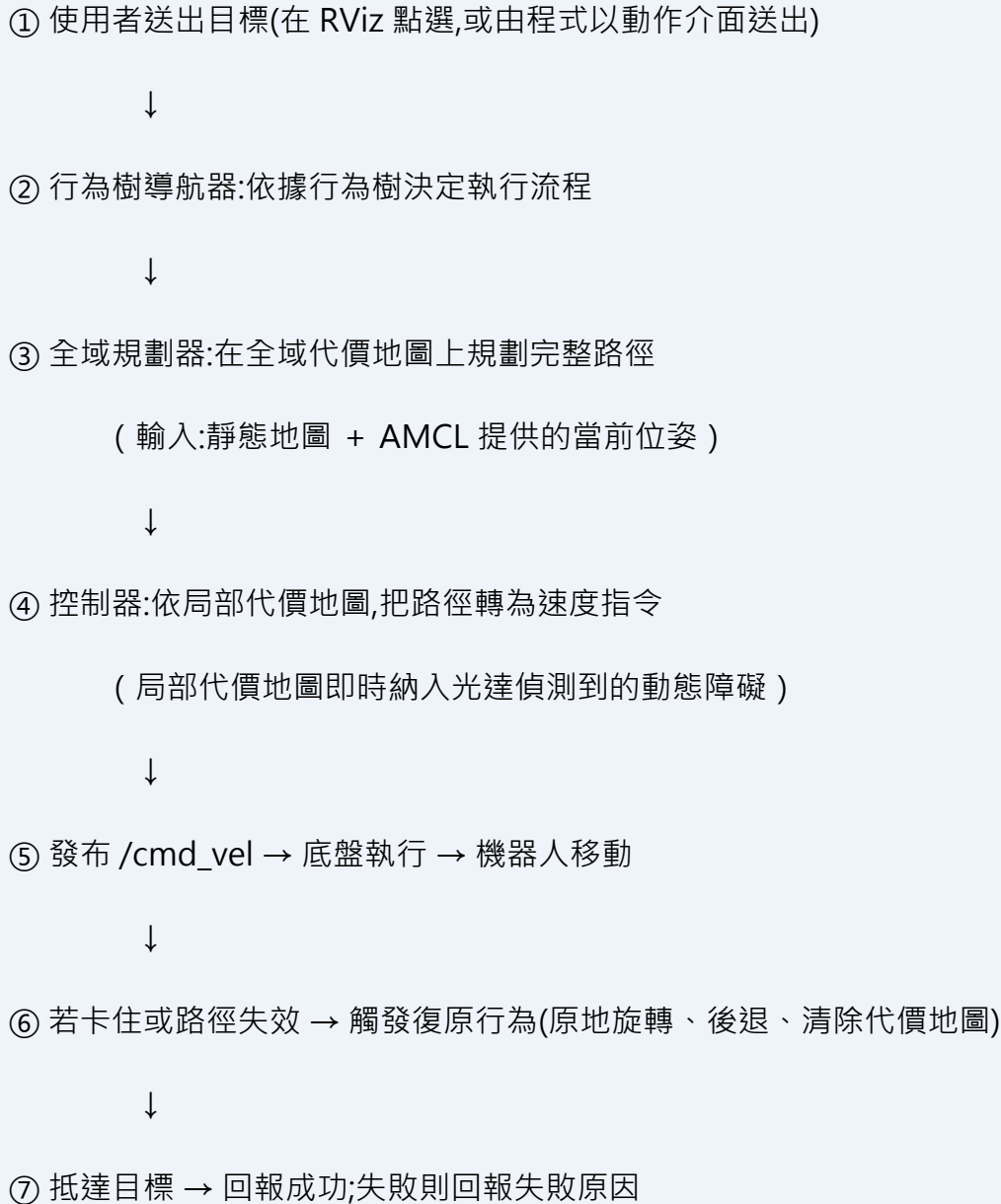
28.6 Navigation Stack 導航

最後一步:讓機器人自己走到目標點。這是第 3.4 節導航系統架構的實際落地。

Nav2 的整體架構

Nav2 是 ROS 2 的導航堆疊,由多個模組協同運作。一個導航目標從送出到執行,會流經行為樹導航器、規劃器、控制器、代價地圖、復原行為、速度指令,最後到達底盤。

圖 28-2 一次導航任務的完整資料流



代價地圖:導航的核心資料結構

代價地圖套件運用感測器資訊,以佔據格的形式提供機器人環境的表示;格子儲存 0

到 254 之間的成本值,代表通過該區域的代價。成本 0 表示格子是自由的,254 表示致命佔據,而介於兩者之間的值則被導航演算法用來像位能場一樣引導機器人遠離障礙物。

代價地圖以多個圖層實作,每個圖層有特定功能,共同貢獻於格子的整體成本。常見的圖層包括:

- 靜態層:載入建好的地圖(28.4 節的產物)。
- 障礙層:即時納入光達偵測到的障礙,包含地圖上沒有的動態物體。
- 膨脹層:把障礙物向外擴張一圈安全距離。這一層極為重要——沒有它,規劃出的路徑會緊貼牆面,機器人必定擦撞。膨脹半徑應設為機器人半徑加上安全餘裕。

全域與局部的分工

- 全域代價地圖:涵蓋整張地圖,以靜態地圖為主,供全域規劃器算出完整路徑。更新頻率低。
- 局部代價地圖:只涵蓋機器人周圍數公尺,以即時感測為主,供控制器閃避動態障礙。更新頻率高。

這個「全域慢而完整、局部快而即時」的分工,同學應該覺得熟悉——它與第 15 章 Cartographer 的雙子系統、第 18 章 LOAM 的雙層架構、第 25 章無人機的分層設計,是同一個系統設計模式。

28.7 系統性除錯:當機器人不動的時候

本章最實用的一節。導航系統元件眾多,出問題時若沒有系統性的排查順序,很容易亂槍打鳥浪費時間。

排查的正確順序

建議的除錯順序是:依序檢查生命週期狀態、TF、地圖伺服器、AMCL、代價地圖、規劃器輸出、控制器輸出,最後才是底盤運動。

更重要的是一個常被忽略的原則:規劃器與控制器的調參,應該等到生命週期、TF、定位、地圖與代價地圖都健康之後才進行。

這個順序背後的邏輯是:下游的問題往往源於上游。若 TF 有問題,再怎麼調規劃器參數都沒用。同學務必養成「由上游往下游排查」的習慣。

圖 28-3 導航系統的除錯檢查表

① 生命週期狀態:所有 Nav2 節點是否都已啟動並進入 active 狀態?

↓ 通過

② TF:map → odom → base_link → laser 的鏈路是否完整?

↓ 通過

③ 地圖:map_server 是否成功載入?RViz 中看得到地圖嗎?

↓ 通過

④ 定位:AMCL 是否收斂?粒子雲是否聚集在正確位置?

↓ 通過

⑤ 代價地圖:全域與局部代價地圖是否正確顯示?障礙是否有膨脹?

↓ 通過

⑥ 規劃器:送出目標後,是否有規劃出路徑?

↓ 通過

⑦ 控制器:是否有發布 /cmd_vel?數值是否合理?

↓ 通過

⑧ 底盤:收到 /cmd_vel 後輪子是否轉動?方向是否正確?

常見症狀與對應的檢查點

症狀	最可能的原因	檢查順序
完全無反應	節點未啟動或未進入 active	①
RViz 顯示變換錯誤	TF 斷裂或外參錯誤	②
機器人位置亂跳	AMCL 未收斂或初始位姿錯誤	④
規劃不出路徑	目標在障礙上、代價地圖膨脹過大	⑤⑥
規劃出路徑但不動	控制器問題、/cmd_vel 未送達底盤	⑦⑧
一直撞牆	膨脹半徑太小、光達外參錯誤	②⑤
原地打轉	角度控制參數不當、里程計角度失準	28.3 節校正

表 28-3 常見故障症狀與檢查順序

每個階段都要留下證據

一個好的實作習慣是:每個檢查點都應該產出可驗證的證據——一段指令輸出、RViz 中一個看得見的座標框架、一個地圖檔、一台移動中的模擬機器人、一份乾淨的程式碼倉庫,或一段簡短的展示錄影。

這個習慣有三重價值:除錯時能快速定位問題在哪一段;報告與專題時有具體的成果

可展示;而且累積下來就是一份完整的學習歷程。建議同學從本章開始就養成這個習慣——第 30 章的專題評量正需要這些證據。

至此,同學已經完成第一個從感測器到自主導航的完整系統。下一章我們把感測器換成相機,實作 RGB-D 與雙目的視覺 SLAM:RealSense 相機設定、相機校正、點雲產生、RTAB-Map 與 ORB-SLAM3 實作,並比較視覺 SLAM 與光達 SLAM 在同一場域的表現差異。

公式或流程

一、差速驅動的里程計計算

設在一個取樣週期內,左右輪各走了 d_L 與 d_R (由編碼器脈衝數乘上每脈衝距離得到),輪距為 b ,則機體的運動為:

$$\Delta s = (d_R + d_L) / 2 \quad (\text{前進距離})$$

$$\Delta \theta = (d_R - d_L) / b \quad (\text{轉角})$$

再依第 10.3 節的 2D 位姿複合公式更新位姿:

$$x \leftarrow x + \Delta s \cdot \cos(\theta + \Delta \theta / 2)$$

$$y \leftarrow y + \Delta s \cdot \sin(\theta + \Delta \theta / 2)$$

$$\theta \leftarrow \text{normalize}(\theta + \Delta \theta)$$

式中取 $\theta + \Delta \theta / 2$ 是為了用「這段運動的平均朝向」來計算位移,比直接用起始朝向精確。`normalize` 則是第 10.3 節強調的角度環繞處理,不可省略。

注意 $\Delta \theta$ 的分母是輪距 b ——這解釋了為什麼 28.3 節說「輪距設錯對建圖的影響比輪徑更嚴重」:輪距誤差直接造成角度誤差,而角度誤差會被後續移動放大。

二、每脈衝距離的換算

編碼器輸出的是脈衝數,換算為距離需要:

$$\text{每脈衝距離} = (\pi \times D) / (N \times G)$$

其中 D 為輪徑、 N 為編碼器每圈脈衝數、 G 為減速比(若編碼器裝在馬達軸上)。

這個常數是里程計精度的基礎,務必以實測校正而非只依規格書。

三、輪徑與輪距的校正

圖 28-4 里程計參數校正流程

【輪徑校正】

- ① 讓小車沿直線行走一段固定距離(建議 3 公尺以上)
- ② 以捲尺量測實際距離 L_{real} ,讀取里程計距離 L_{odom}
- ③ 修正:新輪徑 = 舊輪徑 $\times (L_{\text{real}} / L_{\text{odom}})$
- ④ 重複驗證,直到誤差小於 1%

【輪距校正】

- ① 讓小車原地旋轉 10 圈(順時針與逆時針各做一次)
- ② 讀取里程計累計角度 θ_{odom} ,理論值為 3600 度
- ③ 修正:新輪距 = 舊輪距 $\times (\theta_{\text{odom}} / 3600^\circ)$
- ④ 重複驗證,直到誤差小於 2%

【順序】先校輪徑、再校輪距;兩者都完成後才開始建圖

四、代價地圖的膨脹半徑

膨脹層的關鍵參數是膨脹半徑,它決定障礙物向外擴張多遠:

$$\text{膨脹半徑} \geq \text{機器人外接圓半徑} + \text{安全餘裕}$$

設定過小,規劃出的路徑會緊貼牆面導致擦撞;設定過大,狹窄通道會被完全封死而規劃不出路徑(表 28-3 的常見症狀)。實務上先設為機器人半徑的 1.2 至 1.5 倍,再依場域的通道寬度微調。

五、從零到自主導航的完整流程

圖 28-5 2D SLAM 小車的完整實作流程

【階段一:硬體與座標】

① 組裝硬體 → ② 撰寫 URDF → ③ 驗證 TF 樹完整

↓

【階段二:里程計】

④ 實作編碼器讀取與里程計計算 → ⑤ 校正輪徑與輪距(圖 28-4)

↓

【階段三:建圖】

⑥ 啟動 SLAM → ⑦ 緩慢遙控建圖 → ⑧ 評估品質 → ⑨ 儲存地圖

↓

【階段四:定位】

⑩ 載入地圖 → ⑪ 啟動 AMCL → ⑫ 設定初始位姿 → ⑬ 確認收斂

↓

【階段五:導航】

⑭ 啟動 Nav2 → ⑮ 調整代價地圖 → ⑯ 送出目標 → ⑰ 觀察執行

↓

【階段六:除錯與調校】 依圖 28-3 的順序排查與調校

案例

案例 28-1 三公分的代價:一個未校正的輪徑

這是大學課堂上真實發生過、也極具代表性的案例。

一組同學的建圖結果總是「怪怪的」:地圖看起來完整,牆線也算筆直,但把地圖上的教室尺寸與實際量測比較,發現地圖上的教室比實際大了約 8%。更奇怪的是,他們花了整個下午調整 SLAM 的各種參數,結果毫無改善。

問題出在輪徑。他們使用的輪子標稱直徑 65 公釐,但實際裝上後因為橡膠輪胎受壓變形,有效直徑只有約 60 公釐。里程計以 65 公釐計算,因此「以為」自己走得比實際遠,建出的地圖就被等比例放大。

這個案例的教訓有三層:

- 第一,規格書不等於實際值。輪胎變形、製造公差、安裝誤差都會造成差異,必須實測校正(28.3 節與公式三)。
- 第二,系統性誤差不會被 SLAM 修正。SLAM 能修正隨機的漂移(靠掃描匹配與閉環),但對「每一步都固定放大 8%」的系統性誤差無能為力——因為所有資料都一致地錯,系統無從察覺。

- 第三,調參之前先檢查基礎。這正是 28.7 節強調的排查順序:上游的問題,調下游的參數永遠解決不了。

建議教師在課程一開始就要求同學完成里程計校正並記錄資料——這半小時能省下後續數小時的困惑。

案例 28-2 從模擬到實體:預期之外的落差

第 27 章建議「先模擬、再實體」,但同學常會遇到一個挫折:在 Gazebo 中跑得完美的系統,搬到實體小車上卻問題百出。這個落差來自哪裡?

主要有四個來源:

- 感測器雜訊:模擬中的光達雖然可加雜訊,但真實光達還有第 8 章談過的各種問題——玻璃穿透、黑色物體反射微弱、量測距離的隨機跳動。這些在模擬中往往未被完整建模。
- 里程計品質:模擬中的輪速通常是理想的;真實世界有打滑、地面材質變化、輪胎變形(案例 28-1)、以及通訊丟包造成的資料斷續。
- 時間同步:模擬中所有節點共用同一個模擬時鐘,時間戳記完美一致;真實系統中,微控制器、單板電腦、感測器各有時鐘,同步問題會浮現(第 5、10、26 章反覆強調的工程細節)。
- 環境複雜度:模擬場景通常乾淨簡單;真實教室有桌椅腳(第 8 章的角解析度問題)、玻璃窗、走動的人。

但這不表示模擬沒有價值——恰恰相反。模擬的價值在於:它讓你先確認「邏輯與架構是對的」,把 TF 設定、話題連接、參數配置等問題先解決掉。搬到實體時,剩下的就只是「真實世界的物理問題」,問題範圍大幅縮小。

務實的建議是:模擬中通過的系統,實體上仍要預留一到兩倍的除錯時間。這不是失

敗,而是工程的常態。

案例 28-3 為什麼機器人規劃出路徑卻不動

這是導航實作中最令人困惑的症狀之一:RViz 中明明看到一條漂亮的綠色路徑,機器人卻紋風不動。

依 28.7 節的排查順序,問題必然在第七或第八個檢查點——控制器或底盤。常見的原因包括:

416. 控制器沒有發布速度指令:可能是控制器節點未啟動、或參數設定錯誤導致它認為「無法執行這條路徑」。檢查方法是直接監看 `/cmd_vel` 話題有無資料。

417. 速度指令發布了但底盤沒收到:話題名稱不符(第 27.4 節提過的重新對應問題)、或底盤驅動節點未訂閱。

418. 底盤收到了但不動:速度指令的數值太小,低於馬達的啟動門檻;或馬達驅動器未啟用、電源未開。

419. 速度限制設得太保守:Nav2 的最大速度參數設得過小,或加速度限制過嚴,使機器人「動得太慢看起來像不動」。

這個案例的價值在於它示範了系統性除錯的威力:同樣的症狀,若隨機猜測可能花數小時;依循「監看 `/cmd_vel` → 判斷是控制器還是底盤 → 再往下細查」的順序,通常十分鐘內就能定位。

這也呼應第 27.6 節閱讀開源專案的方法——順著資料流追蹤。無論是讀程式碼還是除錯系統,「follow the data」都是最可靠的策略。

實作活動

實作 28-1 從建圖到自主導航的完整實作

目的:完成本書第一個端到端的系統實作。依圖 28-5 的六個階段循序進行,每階段都產出可驗證的證據。本實作可在模擬中完成,有硬體者建議先模擬再實體。

階段一:座標系統設定(全體)

297. 若使用現成平台,檢視其 URDF,找出 base_link 與 laser 之間的變換,並與實際安裝位置核對。若自組平台,則撰寫自己的 URDF。
298. 啟動系統並產生 TF 樹圖,確認 map → odom → base_link → laser 鏈路完整無斷裂。
299. 外參驗證:把機器人靜置在距離牆面已知距離處(例如 1 公尺),在 RViz 中檢查掃描點是否落在正確位置。若有偏差,回頭修正 URDF。
300. 刻意破壞實驗:故意把 URDF 中光達的位置改錯 20 公分,觀察 RViz 中掃描點如何錯位——親身理解外參的重要性。改回正確值後繼續。

階段二:里程計校正(有硬體者必做,模擬者可略)

301. 依圖 28-4 執行輪徑校正:直線行走 3 公尺,比較里程計與捲尺實測值,修正參數後重驗。記錄修正前後的誤差百分比。
302. 依圖 28-4 執行輪距校正:原地旋轉 10 圈,比較累計角度與 3600 度,修正輪距後重驗。
303. 平滑度檢查:遙控機器人移動,觀察里程計輸出是否連續平滑、有無跳變(28.3 節的品質要求)。

階段三:建圖(全體)

304. 啟動 slam_toolbox 建圖,緩慢遙控機器人繞行場域一圈,在 RViz 中觀察地圖成長。

305. 依第 13.7 節評估地圖品質:牆是否筆直?迴路是否閉合?量測地圖上的關鍵尺寸與實際比較,計算誤差百分比。
306. 速度對照實驗:分別以「很慢」與「較快」的速度各建一次圖,比較品質差異,驗證第 14.6 節「開慢一點」的建議。
307. 儲存地圖,檢視產生的 .yaml 與 .pgm 兩個檔案,用看圖軟體開啟 .pgm 觀察佔據格的三態表示。

階段四:定位(全體)

308. 啟動 map_server 載入剛才儲存的地圖,確認 RViz 中能看到地圖。
309. 啟動 AMCL,在 RViz 中顯示粒子雲。觀察未設定初始位姿時,粒子如何散布全圖。
310. 用 RViz 的位姿估計工具設定初始位姿,遙控機器人移動數步,觀察粒子雲如何收斂——這是第 11 章案例 11-2 的親身重現。
311. 綁架測試:把機器人(或模擬中的機器人)瞬移到地圖上的另一個位置,觀察 AMCL 是否能重新收斂,以及花了多久。

階段五:自主導航(全體)

312. 啟動 Nav2,在 RViz 中顯示全域與局部代價地圖,觀察障礙物的膨脹效果。
313. 用 RViz 的目標點工具送出一個導航目標,觀察規劃出的路徑,以及機器人如何沿路徑移動。
314. 動態避障測試:在機器人行進途中放置一個障礙物(模擬中可加入物件),觀察局部代價地圖如何更新、機器人如何繞行。
315. 膨脹半徑實驗:分別把膨脹半徑設為機器人半徑的 0.8 倍、1.2 倍、2.5 倍,觀察路徑的變化與是否出現「規劃不出路徑」的情況(公式四的討論)。

316. 復原行為觀察:把機器人困在死角或送出無法抵達的目標,觀察 Nav2 觸發哪些復原行為。

階段六:除錯練習(大學部)

317. 人為製造故障:依表 28-3 選三種症狀,分別人為製造(例如關閉某個節點、改錯話題名稱、把膨脹半徑設到極大),再依圖 28-3 的順序排查,記錄從發現症狀到定位問題所花的時間。
318. 建立你自己的檢查表:依本次實作的經驗,整理一份適合你的平台的除錯檢查表,供日後與同學使用。

討論問題

134. 外參改錯 20 公分後,RViz 中的掃描點如何錯位?若不修正就建圖,地圖會出現什麼問題?
135. (有硬體者)輪徑校正前後的誤差各是多少?這個誤差若不修正,對地圖尺寸的影響有多大?(對照案例 28-1)
136. 慢速與快速建圖的品質差多少?從第 13.3 節掃描匹配的原理解釋這個差異。
137. 綁架測試中 AMCL 花了多久收斂?若地圖中有兩個外觀相似的區域,你預期會發生什麼?(提示:第 11 章的多峰分布)
138. 膨脹半徑過大時,發生了什麼?過小時呢?你的場域最合適的值是多少?
139. (大學部)在除錯練習中,依系統性順序排查比隨機猜測快多少?這對於你日後的工程習慣有什麼啟示?

延伸挑戰(大學部)

其一:完整的大學場域導航系統——在系館的一個樓層完成建圖,設定數個具名的目標點(如「實驗室門口」「電梯口」「茶水間」),並撰寫一個 Python 節點,讓使用者輸入

地點名稱後機器人自動導航前往。這實際上是第 23 章語意導航的簡化版,也是第 30 章專題的良好起點。其二:里程計融合實驗——若平台有 IMU,設定 robot_localization 的 EKF 節點融合輪速與 IMU,比較「純輪速里程計」與「融合後里程計」在快速轉彎與地面打滑情況下的差異,量化 IMU 帶來的改善。這是第 7、11 章理論的直接應用。

延伸閱讀

188. Nav2 官方文件(docs.nav2.org).(架構說明、設定指引與教學,本章主要依據)
189. Nav2 官方文件之 Setup Guides:Setting Up Sensors、Mapping and Localization.(從零建立機器人的完整步驟)
190. slam_toolbox 開源專案(GitHub: SteveMacenski/slam_toolbox).(ROS 2 預設的 2D SLAM,程式規模適中適合閱讀)
191. S. Macenski et al., "The Marathon 2: A Navigation System," IROS, 2020.(Nav2 的官方論文,說明設計理念)
192. ROS 2 官方文件之 URDF 教學(docs.ros.org).(機器人描述檔的撰寫方法)
193. ROBOTIS TurtleBot3 e-Manual 之 SLAM 與 Navigation 章節.(教學平台的完整實作指引)
194. robot_localization 套件官方文件.(以 EKF 融合多感測器提供里程計,28.3 節的進階內容)

小結

161. 2D SLAM 小車由底盤馬達、編碼器、2D 光達、微控制器與單板電腦組成;微控制器負責毫秒級即時控制、單板電腦負責 SLAM 運算,這個分工與第 24 章的雙晶片架構思想一致。

162. URDF 描述機器人結構與感測器安裝位置(外參);TF 樹的分工是:SLAM 或 AMCL 發布 map→odom、里程計系統發布 odom→base_link、robot_state_publisher 依 URDF 發布 base_link 到各感測器。常見錯誤為斷裂、外參填錯、重複發布與時間戳記不同步。
163. 里程計由編碼器讀值經差速驅動公式計算並發布;輪徑與輪距必須實測校正,輪距誤差因直接造成角度誤差而影響更嚴重;里程計不需絕對準確但必須平滑連續,可用 EKF 融合 IMU 提升穩健度。
164. slam_toolbox 是 Nav2 的預設 SLAM 演算法、參數簡潔適合教學;建圖流程須先確認 TF 完整,緩慢遙控並持續評估品質;地圖以 .yaml 與 .pgm 兩檔儲存。
165. AMCL 以粒子濾波在已知地圖中定位、不建立地圖,與 SLAM 同樣提供 map→odom 變換因而可無縫切換;關鍵參數包含粒子數、運動雜訊模型、更新門檻與變換容許時間;初始位姿必須設定,否則可能收斂緩慢或錯誤。
166. Nav2 的導航目標流經行為樹導航器、規劃器、控制器、代價地圖、復原行為與速度指令;代價地圖以 0 至 254 的成本值表示,0 為自由、254 為致命佔據,並以靜態層、障礙層、膨脹層等多層疊加。
167. 除錯必須依系統性順序:生命週期、TF、地圖、定位、代價地圖、規劃器、控制器、底盤;規劃器與控制器的調參應等到上游都健康之後才進行;每個檢查點都應留下可驗證的證據。

本章完成了第一個從感測器到自主導航的端到端系統。同學應該能感受到:前面二十七章的理論,在這一章全部匯流成一台真正會動的機器人——第 8 章的光達、第 10 章的座標轉換、第 11 章的粒子濾波、第 12 章的圖優化、第 13 章的佔據格、第 15 章的 SLAM 系統、第 27 章的工具鏈,缺一不可。

下一章,我們把感測器換成相機。RealSense 與 ZED 等相機的設定、相機校正、RGB-D 點雲產生、RTAB-Map 與 ORB-SLAM3 的實作,以及最重要的——視覺 SLAM 與光達 SLAM 在同一場域的對照比較。第 29 章將讓同學親身驗證第 8、9 章反覆討論的感測器互補性。

台灣自編大學教科書

@prof

《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第 29 章

RGB-D / 雙目相機 SLAM 實作

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

學習目標

上一章我們用光達完成了完整的建圖與導航。本章把感測器換成相機,實作視覺 SLAM——並在最後做一件本書醞釀已久的事:在同一個場域,讓光達與視覺兩套系統正面對決。學完本章之後,同學應能夠達成下列目標:

187. 比較 RealSense、ZED、OAK-D 三類深度相機的原理與適用場合。

188. 完成相機校正並驗證內參與畸變係數的正確性。

189. 由 RGB 與深度影像產生彩色點雲,並理解對齊的重要性。

190. 執行 RTAB-Map 建圖並儲存三維地圖。

191. 執行 ORB-SLAM3 並完成軌跡評測。

192. 規劃室內場景的測試流程,並辨識視覺 SLAM 的典型失效情境。

193. 在同一場域比較視覺 SLAM 與光達 SLAM 的表現差異。

核心概念

中文名詞	英文名詞	概念說明
深度影像	Depth Image	每個像素記錄該點距離的影像。
深度對齊	Depth Alignment	把深度影像對齊到彩色影像的座標系。
彩色點雲	Colored Point Cloud	每個三維點帶有 RGB 顏色的點雲。
相機資訊	CameraInfo	ROS 中攜帶內參與畸變係數的訊息。
視覺里程計	Visual Odometry	由影像估計相機運動,提供 odom→base_link。
RGB-D 里程計	RGBD Odometry	RTAB-Map 提供的以 RGB-D 影像估計位姿的模組。
外觀式建圖	Appearance-based	以影像外觀相似度偵測閉環的建圖方法。

中文名詞	英文名詞	概念說明
	Mapping	
記憶體管理	Memory Management	RTAB-Map 控制地圖規模以維持即時性的機制。
資料庫檔	Database File	RTAB-Map 儲存地圖與影像的檔案格式。
定位模式	Localization Mode	載入既有地圖只做定位、不再擴充地圖。
全域快門	Global Shutter	全畫面同時曝光,適合運動中的視覺。
主動紅外投射	IR Projector	投射紋理圖案以協助立體匹配的裝置。
近似同步	Approximate Sync	容許不同話題時間戳記略有差異的同步方式。

29.1 RealSense / ZED / OAK-D 相機介紹

第 9.3 節介紹過三種取得深度的方式,本節聚焦實際可用的產品與選型考量。

三大類深度相機

- RealSense 系列:採主動紅外雙目(第 9.3 節)。它以改良的硬體與軟體達成室內外皆可用的十公尺以上量測範圍,並具備全域快門與寬視野,對機器人與無人機應用相當有用。部分型號內建 IMU,可直接支援視覺慣性 SLAM(第 22 章)。
- ZED 系列:採被動雙目,不投射紅外線,因此在戶外強光下仍可運作(這正好補上 RealSense 的弱點)。它一般能提供比 RealSense 更佳的深度精度與更寬的視野,但代價是需要 GPU 運算——ZED 的深度計算倚賴 CUDA,必須搭配 NVIDIA 顯示卡。
- OAK-D 系列:雙目加上內建的 AI 推論晶片。它能在相機端直接執行物件偵測等神經網路,把運算從主機卸載出去——這與第 24 章邊緣運算的思想一致,對運算資源有限的平台特別有價值。

選型的四個考量

420. 使用環境:室內為主可用 RealSense;需要戶外強光下運作則考慮 ZED 或被動雙目。第 9.3 節提過主動紅外投射會被陽光淹沒。
421. 運算資源:ZED 需要 NVIDIA GPU;RealSense 的深度由相機內部處理,對主機需求較低;OAK-D 甚至能把 AI 推論也搬到相機上。
422. 是否需要 IMU:若要做視覺慣性 SLAM(第 22 章),選內建 IMU 的型號能省去外部 IMU 的同步與外參校正麻煩。
423. 有效範圍:以 RealSense D435 為例,它的量測範圍達 10 公尺,但在 5 公尺範圍內精度較佳——實務上常把有效範圍設為 5 公尺。這呼應第 9.6 節「深度誤差隨距離平方成長」的討論。

類型	深度原理	強光下	運算需求	特色
RealSense	主動紅外雙目	較弱	低(相機內處理)	部分型號內建 IMU
ZED	被動雙目	可用	需 NVIDIA GPU	精度與視野較佳
OAK-D	雙目 + AI 晶片	可用	極低(相機端推論)	可在相機上跑神經網路

表 29-1 三類深度相機的比較

教學上的建議

對大學與高中的教學而言,建議優先考慮內建 IMU 的 RealSense 型號:它價格相對親民、ROS 2 驅動成熟、社群資源豐富、且不需額外的 GPU。等到專題階段需要戶外運作或更高精度時,再考慮 ZED。

29.2 相機校正

第 9.2 節說明過相機校正的原理,本節聚焦實作要點——這是視覺 SLAM 的第一道關卡,也是精度的基礎。

出廠參數 vs 自行校正

現代深度相機出廠時已經過校正,並會透過 ROS 的 CameraInfo 訊息提供內參與畸變係數。對多數應用而言,直接使用出廠參數即可。

但在三種情況下,建議自行校正:

- 相機受過碰撞或鏡頭被調整過。
- 使用一般的網路攝影機或自組的雙目系統(沒有出廠參數)。
- 對精度要求極高的研究應用,想確認出廠參數的準確性。

校正流程回顧

依第 9.2 節的張氏棋盤格法:

424. 列印棋盤格並貼在硬板上(務必保持平整,彎曲會嚴重影響校正精度)。
425. 從不同角度、距離拍攝 15 至 20 張,涵蓋畫面各區域——尤其別忘了邊角,徑向畸變在邊角最嚴重。
426. 執行校正程式,取得內參矩陣 K 與畸變係數。
427. 檢查重投影誤差,應小於 1 像素;若偏大,增加影像數量或重拍。
428. 驗證:對含直線的場景做去畸變,確認彎曲的直線被拉直。

ROS 中的校正工具

ROS 提供 `camera_calibration` 套件,能以互動方式完成校正:對著棋盤格移動相機,工具會即時提示涵蓋度(左右、上下、遠近、傾斜),達標後即可計算並儲存參數。這比自己寫程式方便得多,是實作首選。

雙目的額外校正

雙目相機除了各自的內參,還需要兩相機之間的相對位姿(外參)——這決定了基線長度,直接影響深度計算(第 9.6 節的公式 $Z = f \cdot b / d$)。雙目校正同樣可用 `camera_calibration` 套件完成,但需要同時拍攝左右影像。

常見的校正陷阱

- 棋盤格不平整:貼在紙板或曲面上會導致校正結果錯誤。務必用硬質平板。
- 影像涵蓋不足:只在畫面中央拍攝,邊角的畸變係數就估不準。務必讓棋盤格出現在畫面的各個角落。
- 解析度不一致:校正時與實際使用時的影像解析度必須相同,否則內參的像素單位會不匹配。

- 忘記更新:校正完成後,務必確認 SLAM 系統實際讀取的是新的參數檔,而非舊的或預設值。

29.3 RGB-D 點雲產生

有了 RGB 影像、深度影像與相機內參,就能產生彩色點雲——這是視覺 SLAM 的核心資料形式,也是第 17 章點雲處理技術的來源之一。

從深度影像到三維點

原理是第 9.1 節針孔投影模型的逆運算。已知某像素的座標與該點的深度,即可反推出它的三維座標(公式見本章公式一)。對整張深度影像的每個像素執行這個運算,就得到一片點雲。

再把對應像素的 RGB 顏色賦予每個點,就成為彩色點雲——這正是第 17.6 節點雲顏色化的簡化版:因為 RGB 與深度來自同一台相機,不需要處理跨感測器的外參與時間同步問題。

關鍵前提:深度對齊

這裡有一個初學者常忽略的細節:彩色相機與深度感測器在物理上位於不同位置,因此同一個像素座標在兩張影像中對應的並非同一個空間點。

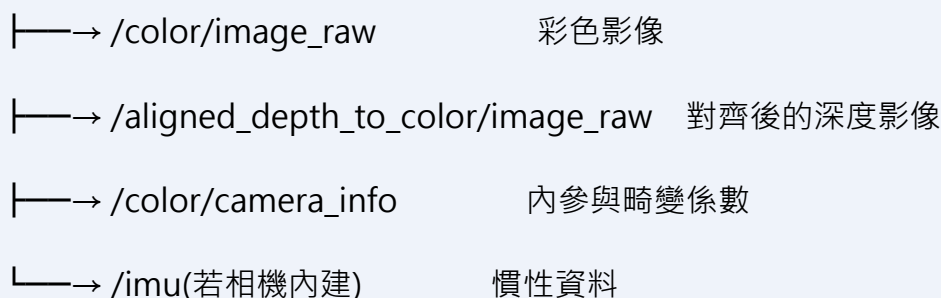
解決方法是深度對齊——把深度影像投影到彩色相機的視角,使兩者的像素一一對應。實務上這對應到相機驅動中的對齊設定,產生的話題會是「對齊到彩色影像的深度影像」;RTAB-Map 也正是訂閱這個對齊後的深度話題。

若忘了開啟對齊,產生的點雲會出現顏色與幾何錯位——牆的顏色跑到窗戶上,這與第 17.6 節提過的外參錯誤症狀相同。

ROS 中的資料流

圖 29-1 RGB-D 相機到點雲的資料流

相機驅動節點



↓

點雲產生節點(或 SLAM 系統內部)

依公式一把每個像素反投影為三維點,並賦予顏色

↓

└─→ /points PointCloud2 格式的彩色點雲(第 8.3 節)

資料量的現實

一張 640×480 的深度影像會產生約 30 萬個點,以 30 幀每秒計算,每秒就是 900 萬個點——這正是第 17.2 節必須降採樣的原因。實務上會先做體素降採樣,再交給後續處理。

另一個實務考量是有效範圍的裁切:如 29.1 節提過,深度相機在較遠處的精度快速下降,因此常設定一個最大有效距離(例如 5 公尺),超過的點直接捨棄——保留不可靠的遠點,反而會拉低整體精度。

29.4 RTAB-Map 實作

RTAB-Map 是 RGB-D SLAM 最常用的開源系統,也是本章的實作主角。

RTAB-Map 是什麼

RTAB-Map(Real-Time Appearance-Based Mapping)是一套基於增量式外觀閉環偵測器的 RGB-D 圖優化 SLAM 方法,設計為可通用於室內與室外環境。

它使用相機接收到的深度影像執行圖優化 SLAM,產生稠密的彩色點雲與相機里程計;它也能以詞袋與特徵偵測器即時完成閉環。它是一套支援 RGB-D 相機的圖優化 SLAM 框架,同時運用視覺外觀與深度資訊進行三維建圖與閉環偵測。

對照本書的架構,同學應該能立刻定位它:圖優化是第 12 章、詞袋閉環偵測是第 19、21 章、稠密點雲是第 17 章——RTAB-Map 是這些技術的整合實現。

三項實用特性

- 可用外部里程計:它允許你傳入自己的里程計話題,而非由它自行產生——若你有更精確的里程計(例如第 28 章的輪速里程計或融合 IMU 的結果),可直接使用。這對輪型機器人特別有價值。
- 可融合雷射:你也能使用自己的 2D 或 3D 雷射掃描來改善地圖品質。這意味著 RTAB-Map 能同時吃視覺與光達資料——正是第 8、9 章反覆強調的感測器互補在實作上的體現。
- 多種地圖格式:它相容於八叉樹與體素等資料結構,這些對三維規劃與多地圖網格化的效率都優於原始點雲。這呼應第 17.5 節三種地圖表示的討論。

RGB-D 里程計

RTAB-Map 套件也提供名為 RGBD 里程計的視覺里程計,用來提供位姿資訊與 `odom→base_link` 變換,在室內運作已展現可靠的結果。

這一點很重要:即使機器人沒有輪速編碼器(例如手持相機掃描),RTAB-Map 也能自

已產生里程計,完整運作。這讓它成為「一台相機就能做 SLAM」的實用方案。

執行流程

429. 啟動相機驅動,務必開啟深度對齊(29.3 節)。
430. 確認三個必要話題有資料:彩色影像、對齊後的深度影像、相機資訊。
431. 啟動 RTAB-Map,指定上述話題名稱與座標框架。首次執行建議加上「啟動時刪除資料庫」的參數,避免載入舊地圖造成混淆。
432. 在 RViz 或 RTAB-Map 自己的視覺化介面中觀察:即時點雲、相機軌跡、以及閉環偵測的發生。
433. 緩慢移動相機掃描整個空間,注意保持特徵豐富的視野。
434. 完成後儲存地圖(RTAB-Map 以資料庫檔案儲存,內含影像、特徵與位姿圖)。
435. 日後可切換到定位模式,載入既有地圖只做定位、不再擴充——這與第 28.5 節 AMCL 的角色相同。

關鍵參數

- 同步方式:若彩色與深度影像的時間戳記完全一致,可用精確同步;若來自不同來源或有微小差異,則需啟用近似同步,否則系統會因等不到配對而完全不動。這是實作中常見的卡關點。
- 最大深度:設定有效範圍上限(29.3 節),超過的點捨棄。
- 記憶體管理:RTAB-Map 有機制控制地圖規模以維持即時性——當地圖過大時,較舊的部分會被移到「長期記憶」中,只在需要時取回。這讓它能處理大範圍場景。

29.5 ORB-SLAM3 實作

第 22 章完整剖析過 ORB-SLAM3 的原理,本節聚焦實作要點。

與 RTAB-Map 的定位差異

面向	RTAB-Map	ORB-SLAM3
主要輸出	稠密彩色點雲地圖	稀疏特徵點地圖
ROS 整合	原生支援,開箱即用	需社群封裝或自行整合
適合任務	三維重建、導航地圖	高精度定位、軌跡評測
視覺化	點雲模型直觀	特徵點與關鍵幀
本書章節	本章	第 22 章原理、本章實作

表 29-2 RTAB-Map 與 ORB-SLAM3 的定位比較

簡言之:要「看得見的三維模型」用 RTAB-Map;要「精確的軌跡與定位」用 ORB-SLAM3。兩者不是替代關係,而是不同任務的工具。

實作要點

436. 編譯:相依套件與第 21.7 節相同(OpenCV、Eigen3、Pangolin)。注意版本相容性,建議參考社群維護的分支或使用容器(第 27.1 節的建議)。
437. 組態檔:準備 YAML 檔,填入 29.2 節校正得到的內參與畸變係數、影像尺寸、以及 ORB 特徵參數(每幀特徵數、金字塔層數)。
438. 模式選擇:依相機能力選擇單目、雙目、RGB-D,以及是否啟用慣性模式(需相機內建 IMU)。
439. IMU 初始化:若使用慣性模式,啟動後務必做帶加減速與旋轉的運動,否則初始化不會成功(第 22.4 節反覆強調的運動激勵)。
440. 觀察:在 Pangolin 視窗中觀察特徵點、關鍵幀、共視圖與閉環——第 21.6 節的資料結構在此具象化。

441. 軌跡輸出與評測:輸出估測軌跡,以 evo 工具計算 ATE 與 RPE(第 21.7 節)。單目模式記得先做尺度對齊。

三種模式的實測比較

同學可在同一場域依序測試三種模式,親身驗證第 19.6 節表 19-3 的理論比較:

- 單目:初始化最困難(需足夠視差,第 20.5 節)、軌跡無絕對尺度、但硬體最簡單。
- RGB-D:初始化最容易(第一幀就有深度)、有絕對尺度、地圖最稠密,但受限於深度相機的有效範圍與強光。
- 雙目慣性:最穩健、精度最佳(第 22.7 節提過的毫米級成績),但需要相機支援與較多的設定。

29.6 室內場景測試

本節說明如何設計有教學價值的測試,特別是「刻意尋找失效邊界」的實驗。

基本測試流程

442. 環境準備:選擇紋理豐富、光線均勻的空間開始(例如有書架、海報的教室),先建立成功經驗。
443. 緩慢掃描:手持相機或裝在小車上,以緩慢平穩的速度移動,涵蓋整個空間。
444. 閉環測試:繞行一圈回到起點,觀察閉環是否觸發、地圖是否被拉正(第 12、15 章的震撼畫面在此重現)。
445. 品質評估:量測地圖上的關鍵尺寸與實際比較;檢查牆面是否「變厚」(第 17 章案例 17-1 的三維品質指標)。

五種典型失效情境

這是本節最有價值的部分——刻意製造失敗,理解系統的邊界。

446. 白牆與弱紋理:對著大面積白牆掃描,觀察特徵點如何歸零、系統如何跟丟。這是第 9、19 章反覆提過的「白牆詛咒」,在此親身驗證。
447. 快速運動:這是視覺 SLAM 最常見的失效原因。移動或轉動太快會中斷成功的點雲匹配序列,導致系統跟丟。同學可逐步提高移動速度,找出系統開始失敗的臨界點。
448. 光線變化:從明亮處走進昏暗處,或正對窗戶造成過曝,觀察追蹤是否中斷。這是第 9.5 節「依賴光線」的具體呈現。
449. 玻璃與鏡面:對著落地窗或鏡子掃描,觀察深度資料出現什麼異常。這與第 8 章光達的玻璃問題是同一類困難。
450. 動態物體:讓人在畫面中走動,觀察是否影響定位。第 23.4 節提過可用物件偵測剔除動態物體的特徵,此處先觀察不處理時的後果。

一個實務上的重要提醒

關於資料錄製,有一個值得注意的經驗:相較於錄製資料再離線處理,線上即時建圖有一個優勢——RTAB-Map 在追蹤失敗時,能藉由把相機移回上次成功追蹤的位置來恢復里程計。

這個「人在迴路中」的即時修正能力,是離線處理錄好的資料所沒有的。因此教學上建議:先以線上方式建立成功經驗與手感,理解「相機要怎麼移動系統才不會跟丟」;等到需要做演算法比較時,再錄製資料以確保公平性(第 27.4 節 rosbag 的價值)。

掃描的手感

視覺 SLAM 有一個光達 SLAM 沒有的特點:操作者的手法會顯著影響結果。幾個實用建議:

- 轉彎時放慢:直線移動時特徵變化平緩,轉彎時視野快速改變,最容易跟丟。

- 保持特徵在視野中:不要突然把相機轉向空白牆面,而應沿著有紋理的方向移動。
- 避免只旋轉不平移:純旋轉無法三角測量(第 20.5 節、案例 20-1),單目模式尤其致命。
- 回頭確認:繞行時偶爾回頭看一下走過的區域,有助於觸發閉環並鞏固地圖。

29.7 視覺 SLAM 與光達 SLAM 比較

這是本書醞釀已久的一節:第 8、9 章比較過感測器特性,第 28 章實作了光達 SLAM,本章實作了視覺 SLAM——現在讓兩者在同一場域正面對決。

實測比較的設計

公平比較需要控制變因:

- 同一場域、同一條路線:最好用同一台機器人同時搭載光達與相機,一次移動同時錄下兩種資料。
- 相同的評估指標:軌跡誤差(ATE)、地圖品質(第 13.7 節的 2D 指標與第 17 章的牆厚度)、以及系統穩健度(跟丟次數)。
- 多種環境:這是關鍵——單一環境的比較會誤導,必須涵蓋各自的優勢與劣勢場景。

四種環境的預期表現

環境	光達 SLAM	視覺 SLAM	原因
明亮有紋理的教室	良好	良好	兩者都在舒適區
昏暗的機房	良好	失效	光達自帶光源、相機需環境光
大面積白牆走廊	尚可	失效	光達看幾何、相機找不到特徵

環境	光達 SLAM	視覺 SLAM	原因
玻璃帷幕大廳	失效	尚可	雷射穿透玻璃、相機看得到反光與框

表 29-3 兩種 SLAM 在四種環境的預期表現

請特別注意最後一列——這是唯一視覺勝過光達的情境,也最能說明「互補」的真義:兩者的弱點不重疊,因此組合起來能涵蓋更多場景。

其他維度的比較

比較維度	光達 SLAM	視覺 SLAM
成本	較高	較低
地圖資訊	精確幾何,無語意	含顏色與語意線索
運算需求	2D 較低、3D 較高	較高(影像處理)
深度精度	公分級且穩定	隨距離平方衰減
對操作手法	較不敏感	敏感(29.6 節的手感)
後續應用	導航、避障	語意理解、AR、三維重建

表 29-4 兩種 SLAM 的其他維度比較

融合:最務實的答案

比較的結論不是「哪個比較好」,而是「如何組合」。29.4 節提過 RTAB-Map 支援同時使用視覺與雷射資料,這正是融合的實作途徑。

- 光達提供精確幾何與穩定的里程計,在昏暗與弱紋理處撐住系統。
- 相機提供顏色、語意與閉環辨識能力,在玻璃環境與需要理解場景時發揮作用。

- IMU(第 7、22 章)在兩者都短暫失效時提供支撐。

這正是本書從第二篇就反覆強調的核心信念:沒有全能的感測器,只有恰當的組合。走過二十九章,同學應該能親手驗證這句話——而不只是接受它。

第八篇的技術實作至此完成。下一章,我們把所有能力整合成完整的專題:從高中的教室 2D 建圖,到大學部的三維掃描、掃地機器人路徑分析、機器狗巡檢、視覺導航無人機、以及多感測器融合定位,並提出一套完整的評分標準。第 30 章將是本書實作的總驗收。

公式或流程

一、由深度影像反投影為三維點

這是第 9.1 節針孔投影模型的逆運算。設某像素座標為(u, v)、該像素的深度值為 d 、相機內參為 f_x 、 f_y 、 c_x 、 c_y ,則對應的三維點座標為:

$$\begin{aligned} Z &= d / scale \\ X &= (u - c_x) \times Z / f_x \\ Y &= (v - c_y) \times Z / f_y \end{aligned}$$

其中 $scale$ 是深度縮放因子——深度影像常以毫米為單位的整數儲存,而點雲以公尺為單位,因此需要除以 1000。這個縮放因子填錯是初學者常見的錯誤,症狀是「點雲的尺度完全不對」(例如整個房間變成一公釐大)。

把這個運算套用到深度影像的每個像素,再賦予對應的 RGB 顏色,就得到彩色點雲(29.3 節)。

二、深度品質與有效範圍

第 9.6 節推導過雙目深度誤差隨距離平方成長:

$$\Delta Z \approx (Z^2 / (f \cdot b)) \times \Delta d$$

這解釋了 29.1 節提到的實務做法——雖然相機標稱十公尺量測範圍,但常把有效範圍設為五公尺。以此式估算:距離加倍,誤差變四倍。若在 2 公尺處誤差 1 公分,到 10 公尺處誤差就可能達 25 公分,已不足以支撐精確建圖。

實務建議:依你的精度需求,用這條式子反推可用的最大距離,並在系統中設定為深度裁切門檻。

三、視覺 SLAM 實作的完整流程

圖 29-2 RGB-D 視覺 SLAM 的完整實作流程

【階段一:相機準備】

- ① 安裝相機驅動 → ② 確認內參(出廠或自行校正,29.2 節)
- ③ 開啟深度對齊 → ④ 驗證彩色與深度影像的像素對應正確

↓

【階段二:資料驗證】

- ⑤ 檢查三個必要話題:彩色影像、對齊深度、相機資訊
- ⑥ 在 RViz 中顯示點雲,確認幾何與顏色都正確

↓

【階段三:SLAM 執行】

- ⑦ 啟動 RTAB-Map 或 ORB-SLAM3,注意同步方式設定
- ⑧ 緩慢掃描,觀察軌跡、地圖與閉環

↓

【階段四:評估與比較】

- ⑨ 儲存地圖 → ⑩ 量測誤差(尺寸比對、牆厚度、ATE)
- ⑪ 與光達 SLAM 在同場域比較(29.7 節)

四、視覺 SLAM 的故障排除順序

與第 28.7 節的導航除錯相同,視覺 SLAM 也應依系統性順序排查:

圖 29-3 視覺 SLAM 的除錯檢查表

- ① 相機驅動:三個必要話題都有資料嗎?頻率正常嗎?
↓
- ② 內參:CameraInfo 的數值合理嗎?(fx、fy 應相近;cx、cy 約為影像寬高的一半)
↓
- ③ 深度對齊:彩色與深度是否對齊?點雲的顏色有沒有錯位?
↓
- ④ 深度縮放:點雲的尺度正確嗎?量一個已知物體驗證(公式一的 scale)
↓
- ⑤ 同步:若系統完全不動,檢查是否需要啟用近似同步
↓
- ⑥ 特徵:當前畫面偵測到多少特徵點?太少表示環境紋理不足
↓

⑦ 運動:移動是否過快?是否只旋轉未平移?(29.6 節的手感)



⑧ 參數:確認上游都健康後,才開始調 SLAM 參數

案例

案例 29-1 動得快一點就跟丟:視覺 SLAM 最常見的挫折

這是同學實作視覺 SLAM 時幾乎必定遇到的狀況,值得完整說明。

典型情境是這樣的:同學拿著深度相機在實驗室慢慢移動,系統運作良好,點雲逐步建立。但只要稍微加快移動或轉動速度,畫面上的座標軸就開始亂飄,點雲停止記錄,終端顯示追蹤品質為零。

這個現象在社群中被反覆討論。有使用者描述:若非常緩慢地移動或旋轉相機,地圖能建立但尺寸不對;若稍微加快一點,相機的 TF 就開始隨機亂飄,點雲停止記錄。官方文件也明確警告:移動或轉動太快會中斷成功的點雲匹配序列,導致系統跟丟。

原因是第 25.4 節討論過的三重打擊在此重現:快速運動造成動態模糊使特徵提取失敗、特徵軌跡縮短使匹配困難、而深度相機的幀率限制也使前後幀差異過大,超出配準的收斂範圍(第 17.3 節 ICP 需要好初值)。

對策有三個層次:

- 操作層面:放慢速度,尤其轉彎時(29.6 節的手感建議)。這是最直接有效的一招。
- 設定層面:社群提出的一個解法是分別使用相機的兩個紅外線影像,以紅外線組態而非深度組態執行 SLAM——這相當於改用雙目模式,能提供更穩定的特徵追蹤。

- 系統層面:加入 IMU 融合(第 22 章)。IMU 能在視覺短暫失效時提供運動預測,大幅提升快速運動下的穩健度——這正是第 7.6 節「IMU 填補視覺失效空隙」的價值。

這個案例的教學價值在於:它讓同學親身體會「論文上的精度數字,是在特定條件下取得的」。同一套系統,慢速掃描能達公分級精度,快速移動則直接失效——理解這個落差,是負責任地使用技術的前提。

案例 29-2 三公尺長的房間變成三公釐:深度縮放的陷阱

這是一個看似荒謬、卻極為常見的錯誤,幾乎每個第一次自己寫點雲產生程式的同學都會遇到。

症狀是:點雲產生出來了,形狀看起來也對,但整個場景小得離譜——在 RViz 中要把視角拉到極近才看得見,而量測的尺寸完全不合理。

原因就是公式一中的深度縮放因子。深度影像為了節省空間,通常以 16 位元整數儲存,單位是毫米;而 ROS 的點雲以公尺為單位。若程式中忘了除以 1000,整個場景就會縮小一千倍——三公尺長的房間變成三公釐。

這個錯誤的診斷方法極為簡單:量測點雲中一個已知尺寸的物體(例如一張 A4 紙或桌子的寬度),若數值差了三個數量級,就是縮放因子的問題。

更廣泛的啟示是:單位錯誤是工程中最常見也最容易避免的錯誤之一。第 10.3 節提過弧度與角度的混用、本章的毫米與公尺、第 5 章的經緯度與公尺——養成「每個數值都標註單位」的習慣,能省下無數除錯時間。這個習慣在寫程式、寫報告、看規格書時都同樣受用。

案例 29-3 同一場域的對決:當光達與相機各自失效

第 29.7 節提出的對照實驗,在大學課堂上有極高的教學價值。這裡描述一次典型的實測結果。

實驗設計:同一台小車同時搭載 2D 光達與 RGB-D 相機,沿相同路線走過四個區域——明亮的教室、昏暗的機房、貼滿白色隔音板的走廊、以及玻璃帷幕的中庭。

結果相當戲劇性:

- 教室:兩者都表現良好,地圖清晰、軌跡準確。同學此時可能會覺得「兩者差不多」。
- 昏暗機房:視覺 SLAM 在進入後數秒內跟丟,特徵數歸零;光達則完全不受影響,因為它自帶雷射光源(第 8 章)。
- 白牆走廊:視覺 SLAM 再次失效(白牆詛咒);光達雖然也面臨走廊的結構退化問題(第 13.6 節),但至少能持續運作。
- 玻璃中庭:局勢逆轉——光達的雷射穿透玻璃,在地圖上畫出根本不存在的空間;相機反而能看到玻璃的反光、窗框與框外的景物,建出較合理的地圖。

這個實驗把第 8、9 章的理論從「表格上的比較」變成「親眼看到的失效」。同學普遍反映,做完這個實驗後,對感測器選型的理解遠比讀十遍表格深刻。

更重要的是後續討論:既然兩者的弱點不重疊,那麼組合起來會如何?這自然引導到 RTAB-Map 同時使用視覺與雷射的融合實作,也呼應第 26 章多感測器與多機器人協作的思想。建議教師務必安排這個實驗,它是第八篇最有說服力的一堂課。

實作活動

實作 29-1 視覺 SLAM 實作與跨感測器比較

目的:完成 RGB-D 視覺 SLAM 的完整實作,並在同一場域與第 28 章的光達 SLAM 進行

對照比較。這是第八篇的高潮實驗,也是第 30 章專題的重要基礎。

第一部分:相機設定與點雲產生(全體)

- 319. 安裝深度相機驅動,確認彩色影像、深度影像與相機資訊三個話題都有資料,並記錄各自的發布頻率。
- 320. 檢視 CameraInfo 中的內參數值,依圖 29-3 的第二步驗證合理性(f_x 與 f_y 是否相近? c_x 、 c_y 是否約為影像寬高的一半?)。
- 321. 開啟深度對齊,在 RViz 中顯示彩色點雲,檢查顏色與幾何是否正確對應。刻意關閉對齊再看一次,觀察錯位現象。
- 322. 尺度驗證:在點雲中量測一個已知尺寸的物體(A4 紙、桌面寬度),確認數值正確——親身避開案例 29-2 的陷阱。
- 323. (進階)自己寫程式:依公式一,從深度影像與內參產生點雲,不使用現成節點。故意把 scale 設錯,重現案例 29-2 的現象。

第二部分:RTAB-Map 建圖(全體)

- 324. 啟動 RTAB-Map,指定正確的話題名稱。首次執行加上刪除舊資料庫的參數。
- 325. 在教室中緩慢移動相機掃描,觀察即時建立的彩色點雲與相機軌跡。
- 326. 閉環測試:繞行一圈回到起點,觀察閉環偵測是否觸發、地圖是否被拉正。
- 327. 速度實驗:逐步加快移動速度,找出系統開始跟丟的臨界速度,重現案例 29-1。記錄這個速度值。
- 328. 儲存地圖,並切換到定位模式重新啟動,驗證系統能在既有地圖中定位(對照第 28.5 節的 AMCL)。

第三部分:失效情境探索(全體,本章特色)

依 29.6 節的五種情境,逐一測試並記錄系統行為:

- 329. 白牆測試:把相機對準大面積白牆,記錄特徵點數量的變化,以及系統跟丟前能撐多久。
- 330. 光線測試:從明亮處走入昏暗處,記錄追蹤中斷的時機;再對著窗戶造成過曝,觀察結果。
- 331. 玻璃測試:掃描玻璃窗或鏡子,觀察深度資料出現什麼異常(缺失?錯誤的深度值?)。
- 332. 動態物體測試:讓一位同學在畫面中走動,觀察是否影響定位精度與地圖品質。
- 333. 純旋轉測試:原地旋轉相機(不平移),觀察系統的反應,驗證第 20.5 節的三角測量限制。

第四部分:跨感測器比較(核心實驗,大學部)

- 334. 場域選擇:選出四個特性不同的區域——明亮有紋理處、昏暗處、大面積白牆處、玻璃處(參考表 29-3)。
- 335. 資料採集:以同一台小車同時搭載光達與相機,沿相同路線移動,同時錄製兩種資料(第 27.4 節的 rosbag)。
- 336. 分別執行:用同一份資料,分別執行第 28 章的光達 SLAM 與本章的視覺 SLAM。
- 337. 量化比較:記錄四個區域中各自的跟丟次數、地圖品質、以及(若有真值)軌跡誤差,填入你自己的比較表。
- 338. 融合嘗試:設定 RTAB-Map 同時使用視覺與雷射資料(29.4 節),觀察是否能涵蓋兩者各自失效的區域。

討論問題

- 140. 關閉深度對齊後,點雲的顏色如何錯位?錯位的方向與相機的物理結構有什麼關係?

141. 你的系統在多快的移動速度下開始跟丟?這個速度對「裝在移動機器人上」意味著什麼?
142. 五種失效情境中,哪一種最容易發生?哪一種後果最嚴重?為什麼?
143. (核心問題)在四個區域的比較中,光達與視覺各在哪些區域勝出?這是否符合表 29-3 的預期?有沒有出乎意料的結果?
144. 融合視覺與雷射後,系統的表現如何?它是否真的涵蓋了兩者各自的弱點?融合帶來什麼新的代價(運算、設定複雜度)?

延伸挑戰(大學部)

其一:大學場域三維數位分身——用 RTAB-Map 掃描一間實驗室或一段走廊,產生完整的彩色三維模型,並依第 17.5 節的方法轉為八叉樹或網格格式輸出。這份成果可用於系所展示、空間管理,也是第 30 章專題二的直接產物。其二:視覺與光達融合的量化研究——設計一套嚴謹的實驗,在多種環境中比較「純光達」「純視覺」「兩者融合」三種配置的表現,以 ATE 與跟丟次數量化,並撰寫成一份完整的研究報告。這個題目層次完整、成果具體,是極佳的專題主軸,也與第八篇強調的「用資料說話」相呼應。

延伸閱讀

195. M. Labbé and F. Michaud, "RTAB-Map as an Open-Source Lidar and Visual SLAM Library for Large-Scale and Long-Term Online Operation," Journal of Field Robotics, 2019.(RTAB-Map 的權威論文,含視覺與雷射融合的說明)
196. RTAB-Map 官方網站與 ROS 2 套件文件(introlab.github.io/rtabmap).(參數說明、教學與各種相機的設定範例)

197. C. Campos et al., "ORB-SLAM3," IEEE T-RO, 2021.(第 22 章的系統,本章的實作對象之一)
198. Intel RealSense ROS 2 套件文件與 SLAM 教學 Wiki(GitHub: IntelRealSense/realsense-ros).(相機設定、深度對齊與 IMU 融合的官方指引)
199. ROS 2 camera_calibration 套件文件.(相機校正的互動式工具,29.2 節的實作基礎)
200. L. Keselman et al., "Intel RealSense Stereoscopic Depth Cameras," CVPR Workshops, 2017.(深度相機的原理與規格,第 9、29 章共同參考)
201. evo 軌跡評測工具(GitHub: MichaelGrupp/evo).(ATE 與 RPE 計算,跨感測器比較的量化工具)

小結

168. 三類深度相機各有定位:RealSense 採主動紅外雙目、室內表現佳且部分型號內建 IMU;ZED 採被動雙目、戶外可用且精度較佳但需 GPU;OAK-D 內建 AI 晶片可在相機端推論。選型應考量環境、運算資源、是否需 IMU 與有效範圍。
169. 現代深度相機出廠已校正並透過 CameraInfo 提供參數,但在碰撞後、使用自組系統或高精度需求時應自行校正;校正的常見陷阱包含棋盤格不平整、影像涵蓋不足、解析度不一致與忘記更新參數。
170. 由深度影像反投影產生點雲是針孔模型的逆運算,關鍵前提是深度對齊——彩色與深度感測器物理位置不同,未對齊會造成顏色與幾何錯位;深度縮放因子填錯會使場景尺度差三個數量級。
171. RTAB-Map 是基於增量式外觀閉環偵測的 RGB-D 圖優化 SLAM,產生稠密彩色點雲與 RGB-D 里程計;它能使用外部里程計、融合 2D 或 3D 雷射掃描、並支援八叉樹與體素等結構,是視覺與光達融合的實作途徑。

172. RTAB-Map 與 ORB-SLAM3 定位不同:前者輸出稠密彩色地圖適合三維重建與導航,後者輸出稀疏特徵地圖適合高精度定位與軌跡評測;三種模式(單目、RGB-D、雙目慣性)的難易與精度差異可實測驗證。
173. 視覺 SLAM 的五種典型失效為白牆弱紋理、快速運動、光線變化、玻璃鏡面與動態物體;移動或轉動太快會中斷點雲匹配序列導致跟丟,而線上建圖相較離線處理的優勢是能靠人為把相機移回上次成功位置來恢復追蹤。
174. 光達與視覺的弱點不重疊:昏暗與白牆處光達勝出、玻璃處視覺勝出,這正是互補的真義;融合(如 RTAB-Map 同時使用視覺與雷射)是最務實的答案,印證了本書「沒有全能的感測器,只有恰當的組合」的核心信念。

第八篇的技術實作至此完成。同學已經親手建立了兩套完整的 SLAM 系統,並在同一場域驗證了它們的優劣互補——這比讀任何比較表格都更深刻。

下一章,我們把所有能力整合成完整的專題:從高中的教室 2D 建圖、大學部的三維空間掃描、掃地機器人路徑分析、機器狗校園巡檢、視覺導航無人機,到多感測器融合定位,並提出涵蓋地圖品質、定位誤差、系統穩定性、報告表達與程式碼品質的完整評分標準。第 30 章將是本書實作的總驗收。

台灣自編大學教科書

@prof

《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第 30 章

SLAM 專題與評量

編者：李世淵 (機器人叫獸)

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

學習目標

走過二十九章的理論與實作,同學已經具備完整的 SLAM 能力。本章是本書實作的總驗收:六個難度遞增的專題,以及一套完整的評分標準。它既是課程的結業設計,也是同學規劃自己研究方向的地圖。學完本章之後,同學應能夠達成下列目標:

194. 依自身的能力與資源,選擇合適難度的專題並規劃執行流程。

195. 完成教室 2D 地圖建構專題,並量化評估地圖品質。

196. 完成三維空間掃描專題,產出可展示的數位模型。

197. 設計並執行掃地機器人路徑分析與覆蓋率評估。

198. 規劃機器狗校園巡檢與視覺導航無人機的系統架構。

199. 設計多感測器融合定位的實驗與比較方法。

200. 依五向度評分標準自我檢核,並準備專題報告與展示。

核心概念

中文名詞	英文名詞	概念說明
專題導向學習	Project-Based Learning	以完成真實任務為核心的學習方式。
評分標準	Rubric	以明確向度與等級描述評分依據的工具。
絕對軌跡誤差	ATE	估計軌跡與真值對齊後的位置誤差(第 21.7 節)。
相對位姿誤差	RPE	固定間隔內相對運動的誤差,反映局部漂移。
覆蓋率	Coverage Rate	實際走過的面積佔可行區域的比例(第 16 章)。
重複率	Redundancy Rate	實際行走距離與理論最短距離的比值。

中文名詞	英文名詞	概念說明
地圖品質指標	Map Quality Metric	牆面厚度、尺寸誤差、閉環一致性等評估項目。
系統穩定性	Robustness	長時間運行不跟丟、能從失敗恢復的能力。
可重現性	Reproducibility	他人能依你的紀錄重現相同結果的程度。
成果證據	Evidence	可驗證的產出:地圖檔、軌跡圖、錄影、程式碼。
技術債	Technical Debt	為求快速而累積的、日後需償還的程式品質問題。
專題里程碑	Milestone	把長期專題切分成可檢核的階段性目標。

專題總覽:如何選擇

本章提供六個專題,難度由淺入深。它們不是彼此獨立的選項,而是一條可以循序漸進的路徑——後面的專題往往以前面的成果為基礎。

專題	主題	難度	適合對象	主要依據章節
一	教室 2D 地圖建構	★	高中、大學部入門	第 13–15、28 章
二	三維空間掃描	★★	大學部	第 17、29 章
三	掃地機器人路徑分析	★★	高中進階、大學部	第 16 章
四	機器狗校園巡檢	★★★★	專題生	第 18、23、26 章
五	視覺導航無人機	★★★★	專題生	第 22、25 章
六	多感測器融合定位	★★★★★	專題生、研究生	第 5–7、11、29 章

表 30-1 六個專題的難度與適用對象

選擇專題的三個判準

- 資源:有哪些硬體?有多少時間?若只有電腦,專題一與三可完全以模擬完成;專題四、五需要實體平台。
- 能力:對照第 27.7 節的五階段學習路徑,誠實評估自己目前在哪一階。挑戰高一階的專題最能成長,跳兩階則容易挫折。
- 興趣與延續性:專題最好能延續成長期的研究方向。若你對 AI 有興趣,專題四、五能自然銜接第 31 章的語意導航。

共通的執行原則

451. 切分里程碑:把專題拆成三到五個可檢核的階段,每階段都有可展示的成果。這能避免「做到最後才發現方向錯了」。
452. 留下證據:第 28.7 節強調過,每個階段都應產出可驗證的證據——地圖檔、誤差資料、錄影、程式碼倉庫。這既是除錯的依據,也是報告的素材。
453. 先模擬再實體:第 27、28 章反覆建議的路徑。在模擬中把流程與參數調通,能省下大量實體除錯時間。
454. 記錄失敗:失敗的嘗試與其原因,往往比成功的結果更有價值。專題報告中若能誠實呈現「我試過什麼、為什麼不行、最後怎麼解決」,遠比只報告成功結果更有說服力。

30.1 專題一:教室 2D 地圖建構

這是最基礎也最重要的專題。它涵蓋 SLAM 的完整循環,而且成果直觀可見——適合作為所有同學的第一個專題。

專題目標

以 2D 光達建立一間教室(或一段走廊)的佔據格地圖,並量化評估其品質。進階目標是完成建圖後的自主導航。

三個階段

- 階段一——模擬建圖:在模擬環境中完成建圖流程,熟悉工具鏈與參數(第 27、28 章)。產出:模擬地圖檔與操作紀錄。
- 階段二——實體建圖:在真實教室以小車建圖。務必先完成里程計校正(第 28.3 節)。產出:地圖檔、建圖過程錄影、品質評估報告。
- 階段三——自主導航:載入地圖,以 AMCL 定位並用 Nav2 導航到指定目標點(第 28.5、28.6 節)。產出:導航錄影、成功率統計。

必做的量化評估

- 455. 尺寸誤差:量測地圖上教室的長寬,與捲尺實測比較,計算誤差百分比。目標是小於 3%。
- 456. 閉環一致性:繞行一圈回到起點,量測地圖上起點與終點的落差。
- 457. 目視檢查:依第 13.7 節檢查牆是否筆直、有無重影與斷裂。
- 458. 導航成功率(若做階段三):設定五個目標點,各嘗試三次,統計成功抵達的比例與平均耗時。

進階挑戰

- 459. 演算法比較:用同一份錄製資料(rosbag),分別執行 slam_toolbox、Cartographer 與 Gmapping,比較地圖品質與運算負擔(第 15 章實作 15-1)。
- 460. 困難場域:挑戰長走廊或空曠大廳,重現第 13.6 節的困境,並嘗試提出對策(增加特徵、調整參數、融合 IMU)。
- 461. 參數研究:系統性地改變粒子數、解析度、更新門檻等參數,量化其對品質與效能的影響,繪製成圖表。

評量重點

這個專題的評量重點不在「有沒有建出地圖」,而在「能不能說清楚地圖為什麼好或不好」。同學應能指出誤差的來源(里程計?掃描匹配?環境結構?),並提出改善的依據。

30.2 專題二:智慧機器人教室 3D 點雲掃描

這個專題把維度提升到三維,產出的成果視覺效果極佳,適合作為系所展示與招生素材。

專題目標

以 RGB-D 相機或 3D 光達掃描一間實驗室或一段走廊,產出完整的彩色三維點雲模

型,並評估其精度。

三個階段

- 階段一——設備準備與校正:相機校正、深度對齊驗證、點雲尺度確認(第 29.2、29.3 節)。產出:校正參數檔與驗證紀錄。
- 階段二——掃描建模:以 RTAB-Map 掃描目標空間。注意規劃路線確保覆蓋完整、重疊充足(第 17.5 節)。產出:三維模型檔與掃描過程錄影。
- 階段三——後處理與輸出:降採樣、離群點剔除、轉為八叉樹或網格格式,產出可分享的模型(第 17.2、17.5 節)。

必做的量化評估

- 462. 牆面厚度:第 17 章案例 17-1 的核心指標。量測一段牆面點雲在垂直方向的分布厚度,理想值應接近感測器精度(數公分)。
- 463. 尺寸誤差:量測模型中的關鍵尺寸(門寬、房間長度)與實測比較。
- 464. 覆蓋完整性:檢查是否有明顯的空洞,並說明成因(遮蔽?玻璃?掃描路線不足?)。
- 465. 資料規模:記錄原始點數、降採樣後點數、以及各階段的處理時間。

進階挑戰

- 466. 多次掃描合併:分次掃描不同區域再合併,體驗第 22 章 Atlas 多階段作業或第 26 章地圖融合的概念。
- 467. 跨感測器比較:同一空間分別以 RGB-D 相機與 3D 光達掃描,比較模型的精度、密度與完整性(第 29.7 節的三維版本)。
- 468. 應用延伸:把模型匯入模擬環境,建立大學教室的「數位分身」,供後續學生在虛擬環境中測試演算法(第 17.7 節)。

成果的長期價值

這個專題的產出有實際用途:系所空間管理、無障礙路線規劃、招生展示、以及作為模擬環境的基礎。建議同學把成果整理成可分享的格式並附上說明文件——這比一份只有自己看得懂的檔案有價值得多。

30.3 專題三:掃地機器人路徑分析

這個專題不需要昂貴硬體,卻能深入覆蓋路徑規劃的核心議題,是高中與大學部入門的理想選擇。

專題目標

分析並改善覆蓋路徑規劃演算法,以覆蓋率與重複率量化評估,並(若可能)與真實掃地機器人的行為比較。

三個階段

- 階段一——紙上與模擬分析:依第 16.7 節的紙上活動,再以 Python 實作弓字形覆蓋規劃,計算覆蓋率與重複率。產出:程式碼與量化資料。
- 階段二——策略比較:實作並比較至少三種策略——隨機碰撞、弓字形、螺旋式,在不同房間形狀下比較效率。產出:比較圖表與分析。
- 階段三——真實觀察(選用):觀察並記錄真實掃地機器人的路線與 App 地圖,與你的模擬結果比較,推測它使用的策略。產出:觀察紀錄與對照分析。

必做的量化評估

- 469. 覆蓋率:實際走過的格數佔可行格數的比例,目標接近 100%。
- 470. 重複率:總行走距離除以理論最短覆蓋距離(第 16 章公式一)。
- 471. 轉彎次數:轉彎耗時且耗電,是實務上的重要指標。
- 472. 對障礙的適應性:在障礙物數量與配置不同的房間中測試,觀察策略的穩健度。

進階挑戰

473. 房間分割:實作第 16.4 節的距離變換法,把地圖自動切分成房間,再依房間逐一覆蓋,比較與「整體覆蓋」的效率差異。
474. 斷點續掃:實作中斷後記錄已完成區域、重啟後只掃剩餘部分的功能,並驗證覆蓋率仍達 100%。
475. 與實體整合:把你的覆蓋規劃器接上第 28 章建好的地圖與 Nav2,讓實體小車真的執行覆蓋清掃。

教學上的價值

這個專題的獨特之處是「人人都有直覺、一動手就發現不簡單」。同學可能自認「不就是一排一排掃」,但一遇到不規則房間、障礙物、多房間連通,就會發現問題的複雜度。這種從直覺到嚴謹的落差,是很好的演算法思維訓練。

30.4 專題四:機器狗校園巡檢 SLAM

這是難度較高的整合型專題,涵蓋第 18、23、26 章的技術,適合專題生投入一到兩個學期。

專題目標

以四足或輪型機器人在校園執行自主巡檢,建立三維地圖、辨識巡檢目標、並產出巡檢報告。

五個階段

- 階段一——資料採集:以手推車或機器人載具搭載光達與 IMU,在走廊、樓梯間、實驗室錄製資料集(第 18.7 節)。這一步不需演算法能力,適合起步。
- 階段二——離線建圖比較:以錄好的資料執行 LeGO-LOAM、LIO-SAM、Fast-LIO,比較軌跡精度、地圖品質與運算負擔(第 18 章)。

- 階段三——退化場景研究:專門針對長走廊,比較純光達與光達加 IMU 緊耦合的差異,量化 IMU 的貢獻。這是最有研究價值的一段。
- 階段四——語意感知:訓練 YOLO 模型辨識巡檢目標(消防設備、門、異常),注意蒐集機器人視角的資料(第 23.4 節的低視角問題)。
- 階段五——自主巡檢與報告:整合定位、導航與辨識,設定巡檢點自動執行,產出含三維位置標註的巡檢報告。

必做的量化評估

- 476. 軌跡精度:以閉環一致性或 AprilTag 提供的近似真值(第 23.3 節)評估。
- 477. 地圖品質:牆面厚度、樓層結構是否正確、跨樓層是否對齊。
- 478. 辨識準確率:YOLO 模型在機器人視角測試集上的精度,並與人類視角資料集的表現對照。
- 479. 任務完成率:設定十個巡檢點,統計自主抵達並成功採集資料的比例。

進階挑戰

- 480. 多機器人協作:兩台以上機器人分頭巡檢再融合地圖(第 26 章),比較與單機的效率差異。
- 481. 跨時間變化偵測:間隔數週重複巡檢同一路線,自動比對找出環境的變化(第 22 章 Atlas 的多階段作業)。
- 482. VLM 任務介面:讓使用者以自然語言指定巡檢任務(第 23.5 節),銜接第 31 章的內容。

安全與倫理提醒

校園巡檢涉及公共空間與人員,執行前務必注意:取得場域使用許可、避開人流尖峰、機器人明顯標示、緊急停止機制、以及影像資料的隱私規範(第 26.7 節)。這些不是形

式,而是專業工程師的基本素養。

30.5 專題五:視覺導航無人機

這個專題結合第 22、25 章的視覺慣性導航,難度高但成果吸引力極強。

專題目標

讓無人機在 GPS 拒止的室內環境自主定位與飛行,並完成指定的航線或探索任務。

四個階段

- 階段一——資料集分析:先以 EuRoC 或 TUM-VI 資料集執行 ORB-SLAM3,熟悉系統行為與評測方法(第 25.6 節)。不需硬體即可完成。
- 階段二——懸停穩定:以微型無人機在防護網內測試光流定位懸停,量測漂移範圍(第 25.5 節的安全原則)。
- 階段三——VIO 整合:讓 VIO 輸出的位姿接入飛控,實現以視覺定位為基礎的位置保持與簡單航線飛行。
- 階段四——自主任務:執行指定航線、簡單避障、或探索建圖任務。

必做的量化評估

- 483. 懸停精度:定點懸停 60 秒的水平漂移範圍。
- 484. 軌跡誤差:以 AprilTag 或動作捕捉提供真值,計算 ATE。
- 485. 初始化時間:從啟動到 IMU 初始化完成所需的時間與運動量(第 22.4 節)。
- 486. 失效邊界:逐步提高飛行速度,記錄系統開始不穩定的臨界速度(第 25.4 節)。

進階挑戰

- 487. 影像品質自適應:實作依影像清晰度動態調整視覺與慣性權重的機制(第 25.4 節第五項對策)。

488. 地空協作:與地面機器人協同探索,比較各自的覆蓋能力(第 25.7、26.1 節)。

489. 自製資料集:錄製大學場域的視覺慣性資料集並公開,供後續學生使用。

安全是第一優先

第 25.5 節的安全原則在此必須嚴格遵守:防護網或防護籠、微型平台優先、隨時可用的緊急停止、分階段測試、人員位置管理。

特別提醒:任何自主飛行的測試,都應先在繫繩或手動可隨時接管的狀態下驗證。跳過這個步驟是意外的主因。專題的成功不值得用任何人的安全去換。

30.6 專題六:地磁 + IMU + 視覺融合定位

這是難度最高、也最能展現整合能力的專題,涵蓋第 5 至 7 章的感測器與第 11 章的融合理論。

專題目標

設計並實作一套多感測器融合的定位系統,量化各感測器的貢獻,並在多種環境中驗證融合的價值。

四個階段

- 階段一——單感測器基準:分別以地磁、IMU、視覺(或光達)各自定位,量測各自的誤差特性與失效情境。這一步建立比較的基準。
- 階段二——兩兩融合:實作 IMU 加地磁(第 6.6 節的 AHRS)、IMU 加視覺(第 22 章的 VIO),量化融合帶來的改善。
- 階段三——完整融合:以 EKF 或圖優化整合三者(第 11、12 章),並加入依情境調整權重的機制(第 5.6 節的信任度切換)。

- 階段四——多環境驗證:在明亮教室、昏暗機房、鋼構旁、戶外等環境測試,驗證融合系統是否在各環境都優於單一感測器。

必做的量化評估

- 490. 各配置的軌跡誤差:單一感測器與各種融合組合,在各環境的 ATE 比較表。
- 491. 失效次數:各配置在各環境的跟丟或發散次數。
- 492. 權重變化:記錄融合系統在不同環境中,各感測器權重的動態變化,驗證機制是否如預期運作。
- 493. 運算成本:各配置的運算負擔,評估融合的代價。

這個專題的研究價值

本專題最有價值之處,在於它能產出一張「感測器在什麼環境該相信誰」的實測地圖。這不只是課堂練習——它正是第 4.7、5.6、7.6、29.7 節反覆討論的融合原則,在你自己的場域中的具體答案。

建議把成果整理成完整的研究報告:實驗設計、資料、圖表、分析與結論。這樣的報告已具備投稿學生研討會或作為研究所申請素材的水準。

與第 31 章的銜接

完成這個專題的同學,已經具備進入第九篇的所有基礎。第 31 章要討論的 AI 與 VLM 語意導航,正是在這套穩固的幾何定位之上,再疊加語意理解的能力——而幾何層若不穩,語意層就是空中樓閣。

30.7 評分 Rubric:五個向度

本節提供一套完整的評分標準。它既是教師評量的依據,也是同學自我檢核的清單——建議在專題開始時就閱讀,而非最後才看。

五個評分向度

向度	權重建議	評什麼
地圖品質	25%	尺寸誤差、牆面清晰度、閉環一致性、完整性
定位誤差	25%	軌跡精度、重複性、失效次數
系統穩定性	20%	長時間運行、從失敗恢復、對環境變化的耐受
報告表達	20%	問題定義、方法說明、資料呈現、分析深度
程式碼品質	10%	可讀性、模組化、文件、可重現性

表 30-2 專題評分在五個向度

各向度的等級描述

以下以四級描述各向度,同學可據此自我檢核:

一、地圖品質(25%)

494. 優秀:尺寸誤差小於 2%、牆線清晰無重影、迴路完美閉合、覆蓋完整無異常空洞,並能說明殘餘誤差的來源。
495. 良好:尺寸誤差小於 5%、牆線大致清晰、迴路閉合、僅有少量可解釋的空洞。
496. 及格:地圖可辨識環境結構,但有明顯變形或重影,且能指出問題所在。
497. 待改進:地圖嚴重扭曲或不完整,且無法說明原因。

二、定位誤差(25%)

498. 優秀:有真值比對的量化資料(ATE / RPE),誤差在合理範圍,並分析誤差隨時間與環境的變化。

499. 良好:有量化評估(如閉環落差、尺寸比對),資料合理。

500. 及格:有定性評估(目視軌跡合理),但缺乏量化。

501. 待改進:未做任何定位評估。

三、系統穩定性(20%)

502. 優秀:能長時間穩定運行、跟丟後能自動恢復、並主動測試過失效邊界(如第 29.6 節的五種情境)。

503. 良好:能完成完整任務,偶有失效但能人工恢復。

504. 及格:能在理想條件下完成任務。

505. 待改進:僅能片段運作,無法完成完整流程。

四、報告表達(20%)

506. 優秀:清楚定義問題、說明方法選擇的理由、以圖表呈現資料、深入分析結果、並誠實討論限制與失敗的嘗試。

507. 良好:結構完整、資料清楚、有基本分析。

508. 及格:能說明做了什麼,但缺乏分析與反思。

509. 待改進:僅有操作紀錄,無法看出理解程度。

五、程式碼品質(10%)

510. 優秀:結構清晰模組化、有註解與說明文件、版本控制良好、他人能依文件重現結果。

511. 良好:程式可執行、有基本註解與說明。

512. 及格:程式可執行,但難以閱讀或缺乏說明。

513. 待改進:程式無法執行或無法取得。

特別強調:誠實報告失敗

評量中一個常被忽略卻極重要的原則:誠實報告失敗的嘗試,應該加分而非扣分。

一份寫著「我們嘗試了 A 方法,因為某某原因失敗;分析後改用 B 方法,並驗證了改善」的報告,展現的理解深度遠高於只寫「我們用 B 方法,得到很好的結果」。前者展現了工程判斷,後者可能只是照抄教學。

本書從第一章就反覆討論「常見失敗」,正是這個精神。教師在評量時應明確傳達這個標準,鼓勵同學誠實面對困難。

展示的建議

514. 三分鐘能講完核心:先說「解決什麼問題、怎麼做、結果如何」,細節放後面。

515. 用影片而非現場演示:現場演示風險高(硬體故障、環境不同),事先錄好的影片更可靠,也能剪出重點。

516. 資料要有對照:「誤差 5 公分」沒有意義,「誤差 5 公分,而未校正前是 30 公分」才有說服力。

517. 準備一個「如果重來」:能說出「若有更多時間,我會如何改進」,展現的是持續思考的能力。

給教師的課程規劃建議

本節提供三種課程長度的專題安排建議,供教師參考。

課程類型	時數	建議專題	重點調整
高中選修	18 週 × 1-2 小時	專題一(模擬)或三	以模擬與紙上活動為主,不碰原始碼
大學部	18 週 × 3 小時	專題一 + 二 或 三	完整走過建圖、定位、導

課程類型	時數	建議專題	重點調整
			航、評估
專題 / 研究	一至兩學期	專題四、五或六	強調研究方法、量化比較與報告撰寫

表 30-3 三種課程的專題安排建議

時程規劃的原則

- 前三分之一講理論與工具:讓同學備妥環境與基礎知識,此時可穿插小型的模擬練習維持動機。
- 中間三分之一做基礎實作:完成第 28、29 章的引導式實作,建立成功經驗。
- 最後三分之一做專題:此時同學已有能力自主探索,教師轉為顧問角色。
- 預留緩衝:實作課程的意外極多(硬體故障、環境問題、相依性錯誤)。建議預留至少兩週的緩衝時間。

分組的建議

518. 三到四人一組較理想:太少則工作量過重,太多則容易有人搭便車。
519. 明確分工但輪替:例如分為建圖、導航、評估、報告四個角色,但中途輪替,確保每人都接觸到完整流程。
520. 個別檢核:除了團隊成果,應有個別的檢核機制(如口試或個人反思報告),避免評量失真。

資源有限時的替代方案

521. 硬體不足:專題一與三可完全以模擬完成;第 27.1 節建議的容器環境能讓全班在自己的筆電上操作。

522. 沒有真值系統:用 AprilTag 提供近似真值(第 23.3 節),或以閉環落差與尺寸比對作為評估依據。

523. 時間不足:專題一的階段一與二(模擬與實體建圖)即可構成一個完整的小型專題,不必勉強做到導航。

至此,第八篇「ROS / Python / C++ 實作」全部完成。同學已經從理論走到工具、從工具走到實作、再從實作走到完整的專題。

最後一篇,我們把視野投向未來:當機器人不只要知道「我在哪、環境長什麼樣」,還要理解「我看到了什麼、這個地方代表什麼、下一步該怎麼做」時,SLAM 會如何演變?第 31 章「訓練 AI 與 VLM 輔助 SLAM 導航」將帶同學走進 SLAM 與人工智慧交會的前沿。

公式或流程

一、絕對軌跡誤差(ATE)

把估計軌跡與真值軌跡對齊後,計算對應時刻位置差的均方根:

$$ATE = \sqrt{(1/N) \sum \|p_{est_i} - p_{gt_i}\|^2}$$

其中 p_{est} 為估計位置、 p_{gt} 為真值位置、 N 為取樣點數。單目模式評測時,必須先做尺度對齊(第 21.7 節),否則誤差毫無意義。

ATE 反映的是整體的軌跡偏差,是最常用的總體精度指標。

二、相對位姿誤差(RPE)

計算固定時間間隔 Δ 內相對運動的誤差:

$$RPE = \sqrt{(1/M) \sum \| (T_{est_i}^{-1} T_{est_{i+\Delta}}) \ominus (T_{gt_i}^{-1} T_{gt_{i+\Delta}}) \|^2)}$$

RPE 反映的是局部漂移程度。它與 ATE 的差別在於:一條整體偏移但形狀正確的軌

跡, ATE 大而 RPE 小; 一條整體位置對但局部抖動的軌跡則相反。兩者並看, 才能完整評估系統。

三、地圖尺寸誤差

最簡單也最實用的品質指標:

$$\text{尺寸誤差}(\%) = |L_{\text{map}} - L_{\text{real}}| / L_{\text{real}} \times 100\%$$

其中 L_{map} 為地圖上量測的長度、 L_{real} 為實測長度。建議量測至少三處(長邊、短邊、對角)取平均, 以避免單一量測的偶然性。

四、覆蓋率與重複率

第 16 章的指標, 用於專題三:

$$\text{覆蓋率} = \text{已走過的格數} / \text{可行格數} \times 100\%$$

$$\text{重複率} = \text{實際行走距離} / \text{理論最短覆蓋距離}$$

五、專題執行的完整流程

圖 30-1 專題從規劃到展示的完整流程

【規劃階段】

① 依表 30-1 選定專題 → ② 評估資源與能力 → ③ 切分三至五個里程碑

↓

【執行階段】

④ 模擬驗證(先把流程與參數調通)

⑤ 實體實作(預留兩倍除錯時間)

⑥ 每個里程碑產出可驗證的證據(第 28.7 節)

↓

【評估階段】

⑦ 依 30.7 節的五向度自我檢核

⑧ 量化評估:ATE / RPE / 尺寸誤差 / 覆蓋率(依專題而定)

⑨ 誠實記錄失敗的嘗試與原因

↓

【展示階段】

⑩ 撰寫報告:問題、方法、資料、分析、限制

⑪ 準備三分鐘核心說明 + 展示影片

⑫ 準備「如果重來會怎麼做」的反思

六、專題自我檢核表

圖 30-2 繳交前的自我檢核清單

- ☐ 地圖品質:有量化的尺寸誤差資料嗎?能說明誤差來源嗎?
- ☐ 定位精度:有 ATE 或閉環落差的資料嗎?
- ☐ 系統穩定:測試過失效邊界嗎?知道系統在什麼條件下會壞嗎?
- ☐ 對照組:有與其他方法或參數設定比較嗎?
- ☐ 失敗紀錄:誠實記錄了失敗的嘗試與解決過程嗎?
- ☐ 證據完整:地圖檔、軌跡圖、錄影、程式碼都齊全嗎?

- 可重現:他人依你的文件能重現結果嗎?
- 反思:能說出「如果重來會怎麼做」嗎?

案例

案例 30-1 一份好報告與一份普通報告的差別

兩組同學做了同一個專題——教室 2D 建圖。最終成果的地圖看起來差不多,但報告的品質天差地別。

普通的那一份寫著:「我們使用 slam_toolbox 進行建圖,設定解析度為 0.05 公尺,成功建立了教室地圖。地圖品質良好。」附上一張地圖截圖,結束。

好的那一一份寫著:「初次建圖時發現地圖比實際大 8%,經檢查為輪徑未校正所致(標稱 65 公釐、實測有效直徑 60 公釐)。校正後尺寸誤差降至 1.8%。我們另外比較了三種解析度:0.02 公尺細節最佳但記憶體用量為 0.05 的六倍;0.10 公尺則無法分辨桌腳。最終選用 0.05 公尺。此外,在走廊段落曾出現地圖變形,推測為第 13.6 節所述的縱向約束不足,我們以放慢速度與提高里程計權重改善,但未能完全消除——這是本專題的已知限制。」

兩份報告的差別不在文字量,而在思考的深度:第二份展現了發現問題、分析原因、驗證改善、誠實面對限制的完整歷程。

這正是 30.7 節評分向度「報告表達」的核心。同學要記住:專題評量的是你的理解與判斷,而不是機器人跑得多順。一份誠實呈現困難與思考的報告,永遠勝過一份只有漂亮結果的報告。

案例 30-2 從課堂專題到研討會論文

本章的專題四、五、六,規模與嚴謹度已接近研究水準。若同學有意進一步發展,以下是一條務實的路徑。

第一步是把專題做深而非做廣。與其淺嘗多個題目,不如把一個题目的比較做到嚴謹:多種方法、多種環境、足夠的重複次數、完整的量化資料。專題六「多感測器融合定位」正是這種結構。

第二步是建立可比較的基準。研究的價值來自比較——你的方法比什麼好?好多少?在什麼條件下?本書反覆強調的「同一場域對照實驗」(第 15、29 章),正是這個思維的訓練。

第三步是誠實面對限制。真正的研究報告都會有「限制與未來工作」一節。能清楚說出「我的方法在什麼情況下不適用」,反而是專業的表現。

第四步是資料與程式碼的公開。第 25.7、27.4 節都建議過「錄製並公開大學場域的資料集」——這類貢獻雖不炫目,卻能長期產生價值,也是進入學術社群最實在的方式。

許多國內外的學生研討會與大專生專題競賽,正適合這類成果發表。對有意升學或投入研究的同學而言,一份紮實的專題報告,遠比履歷上的一長串修課紀錄更有說服力。

案例 30-3 專題失敗的三種典型模式

教學經驗中,專題未能完成的原因高度集中在三種模式。提前認識它們,能大幅提高成功率。

524. 模式一——目標過大:「我要做一台能自主巡邏整個校園、辨識異常、並用語音回報的機器人。」這個目標在一學期內幾乎不可能完成。解法是切分里程碑

(30.0 節),先確保最小可行的成果,再往上加。一台能在一間教室穩定建圖並導航的機器人,遠勝一台什麼都做一半的機器人。

525. 模式二——環境問題吃掉時間:第 27.1 節案例 27-1 描述過的版本相依性地獄。許多組別在環境建置上耗掉一半的時間,最後只能草草收尾。解法是使用容器化的統一環境,並在專題開始前就完成環境驗證(第 27 章實作 27-1 的冒煙測試)。

526. 模式三——沒有量化評估:做出了東西,但說不清楚它好不好。到了評量階段才發現沒有資料可以呈現,只能給一段影片。解法是在專題一開始就依 30.7 節的向度規劃評估方法,並在每個里程碑收集資料——而非最後才補。

這三種模式的共同解方是:在專題開始時就想清楚「最後要交出什麼」。把 30.7 節的評分標準與圖 30-2 的檢核表放在眼前,逆推每個階段該做什麼——這是專案管理的基本功,也是同學進入業界或研究所後最實用的能力之一。

實作活動

實作 30-1 專題規劃與提案

目的:在正式投入專題之前,完成一份完整的規劃提案。這個練習本身就是重要的訓練——多數專題的成敗,在規劃階段就已決定。

第一部分:選題與可行性評估

339. 依表 30-1 選出兩到三個候選專題,對每一個評估:所需硬體是否可取得?所需能力是否具備(對照第 27.7 節的五階段)?時間是否充足?

340. 對最有興趣的專題,列出「最小可行成果」與「理想完整成果」。前者是必定要達成的底線,後者是行有餘力的目標。

341. 風險評估:列出三個最可能失敗的環節,並為每個準備備案。例如「若實體小車無法運作,改以模擬完成階段二」。

第二部分:里程碑規劃

342. 把專題切分成三到五個里程碑,每個里程碑明確定義:要完成什麼、產出什麼證據、預計花多少時間。
343. 在時程表上標出檢核點,並預留至少兩成的緩衝時間(案例 30-3 的模式二)。
344. 對每個里程碑,列出「若這一步卡住超過一週,我會怎麼辦」的應對方案。

第三部分:評估方法設計(關鍵)

345. 依 30.7 節的五個向度,為你的專題設計具體的評估方法。例如地圖品質要量什麼、定位誤差用什麼當真值。
346. 設計對照組:你的成果要與什麼比較?(不同方法?不同參數?有無某個元件?)沒有對照,資料就沒有意義(案例 30-2)。
347. 規劃資料收集:每個里程碑要收集哪些資料?用什麼格式記錄?這能避免最後才發現資料不足(案例 30-3 的模式三)。

第四部分:提案發表

348. 把上述內容整理成一份兩頁的提案,並準備五分鐘的口頭報告。
349. 向同學與教師報告,重點在說服他們:這個題目有價值、你有能力完成、而且你知道會遇到什麼困難。
350. 接受提問並修正提案。他人的質疑往往能點出你的盲點——這比自己埋頭做三週才發現問題有效率得多。

討論問題

145. 你的「最小可行成果」是什麼?若時間只剩一半,你會捨棄哪些部分?

146. 你設計的評估方法中,真值(ground truth)從何而來?若沒有真值,你如何論證成果的品質?
147. 你的對照組是什麼?為什麼這個對照能凸顯你的成果價值?
148. 依案例 30-3 的三種失敗模式,你的專題最可能踩到哪一種?你的預防措施是什麼?

延伸挑戰

把你的提案寫成正式的研究計畫格式:研究背景與動機、文獻回顧(可直接引用本書各章的延伸閱讀)、研究方法、預期成果、時程規劃、參考文獻。這份文件不只是專題的起點,也是日後撰寫研究所申請資料、科技部大專生研究計畫、或競賽企劃書的基礎。撰寫這份文件的過程,本身就會逼你把想法釐清——這是本活動最大的價值。

延伸閱讀

202. J. Sturm et al., "A Benchmark for the Evaluation of RGB-D SLAM Systems," IROS, 2012.(ATE 與 RPE 評測方法的權威來源,TUM RGB-D 資料集的說明論文)
203. evo 軌跡評測工具(GitHub: MichaelGrupp/evo).(專題評估的必備工具,支援多種資料集格式與圖表輸出)
204. Z. Zhang and D. Scaramuzza, "A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry," IROS, 2018.(軌跡評測的完整教程,含尺度對齊等細節)
205. C. Cadena et al., "Past, Present, and Future of SLAM," IEEE T-RO, 2016.(第 1 章即引用的權威回顧,適合作為專題文獻回顧的起點)

206. 本書第 13.7 節(地圖品質評估)、第 17 章案例 17-1(牆面厚度指標)、第 21.7 節(軌跡評測).(專題評估方法的內部參照)
207. S. Macenski et al., "The Marathon 2: A Navigation System," IROS, 2020.(Nav2 論文,含系統效能評估的方法論可供參考)
208. 各校大專學生研究計畫與專題競賽的優秀作品集.(觀摩他人如何組織一份完整的專題報告)

小結

175. 六個專題難度遞增:教室 2D 建圖(★)、三維空間掃描(★★)、掃地機器人路徑分析(★★)、機器狗校園巡檢(★★★★)、視覺導航無人機(★★★★)、多感測器融合定位(★★★★★);選擇時應評估資源、能力與延續性。
176. 專題一涵蓋 SLAM 完整循環且成果直觀,評量重點不在「有沒有建出地圖」,而在「能不能說清楚地圖為什麼好或不好」;專題二的產出對系所空間管理與模擬環境有長期價值。
177. 專題三不需昂貴硬體卻能深入覆蓋規劃的核心,其價值在於「人人有直覺、一動手就不簡單」的思維訓練;專題四、五、六則為整合型專題,規模已接近研究水準。
178. 評分採五個向度:地圖品質(25%)、定位誤差(25%)、系統穩定性(20%)、報告表達(20%)、程式碼品質(10%);建議在專題開始時就閱讀標準並逆推規劃。
179. 誠實報告失敗的嘗試應該加分而非扣分——展現發現問題、分析原因、驗證改善的完整歷程,遠比只呈現漂亮結果更能說明理解深度。
180. 專題失敗的三種典型模式為目標過大、環境問題吃掉時間、以及沒有量化評估;共同解方是在開始時就想清楚「最後要交出什麼」,並依評分標準逆推每階段的工作。

181. 展示的建議:三分鐘能講完核心、用預錄影片而非現場演示、資料要有對照組、並準備一個「如果重來會怎麼做」的反思。

第八篇「ROS / Python / C++ 實作」至此全部完成。從第 27 章的環境建置、第 28 章的光達建圖導航、第 29 章的視覺 SLAM 與跨感測器比較,到本章的專題整合與評量,同學已經走完了從理論到實作的完整路徑。

最後一篇,我們把視野投向未來。傳統 SLAM 解決的是「我在哪裡、地圖長什麼樣」;而 AI 與 VLM 要解決的是「我看到了什麼、這個地方代表什麼、下一步該怎麼做」。當幾何地圖升級為語意地圖、當機器人能聽懂「請去教務處」這樣的指令時,SLAM 會走向何方?第 31 章「訓練 AI 與 VLM 輔助 SLAM 導航」將是本書的最後一章,也是通往下一個十年的入口。

台灣自編大學教科書

@prof

《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第九篇 AI、VLM 與語意導航 SLAM

第 31 章

訓練 AI 與 VLM 輔助 SLAM 導航

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

學習目標

走過三十章,同學已經能讓機器人回答「我在哪裡、地圖長什麼樣」。但這還不夠——真正實用的機器人,還必須回答「我看到了什麼、這個地方代表什麼、下一步該怎麼做」。本章是全書的最後一章,也是通往下一個十年的入口。學完本章之後,同學應能夠達成下列目標:

- 201. 說明傳統 SLAM 的限制,以及 AI 為何成為必要的補充。
- 202. 說明 YOLO、SAM、Depth Anything、DINO、CLIP 在機器人感知中的分工。
- 203. 說明從 Visual SLAM 到 Semantic SLAM 再到具身 AI 導航的演進脈絡。
- 204. 規劃機器人訓練資料的蒐集與標註流程,並訓練校園場景辨識模型。
- 205. 說明 VLM 如何理解自然語言導航指令,以及 RAG 如何提供校園空間知識。
- 206. 說明邊緣 AI 部署的關鍵技術:量化、蒸餾與記憶體管理。
- 207. 分析 AI 輔助 SLAM 的風險,並展望 SLAM 與基礎模型結合的未來。

核心概念

中文名詞	英文名詞	概念說明
語意 SLAM	Semantic SLAM	在幾何地圖上加註物件與場景意義的 SLAM。
具身智慧	Embodied AI	透過與實體環境互動來學習與行動的人工智慧。
開放詞彙	Open-Vocabulary	不受預設類別限制,能辨識訓練時未見過的物件。
物件目標導航	ObjectNav	接受「去找某個物體」指令的導航任務。

中文名詞	英文名詞	概念說明
場景圖	Scene Graph	以節點與關係描述場景中物件及其空間關聯的結構。
視覺語言模型	VLM	能同時理解影像與語言的多模態模型。
視覺語言動作模型	VLA	直接由影像與語言產生動作指令的模型。
檢索增強生成	RAG	先檢索相關資料再生成回答,以提升正確性。
模型蒸餾	Distillation	把大模型的能力轉移到小模型上的訓練方法。
量化	Quantization	以較少位元表示模型參數以降低記憶體與運算。
邊緣部署	Edge Deployment	在機器人本體的運算平台上執行 AI 推論。
幻覺	Hallucination	AI 產生看似合理但實際錯誤內容的現象。
世界模型	World Model	學習環境動態、能預測未來狀態的模型。

31.1 為什麼傳統 SLAM 還需要 AI

本書前三十章建立的是一套極為紮實的幾何系統。它能精確地告訴你機器人在哪、環境的形狀如何——但它有一個根本的盲點。

傳統 SLAM 知道形狀,不知道意義

一台跑著 Cartographer 的機器人,面對一扇門時,它「知道」的是:前方 1.2 公尺處有一段寬 0.9 公尺的空隙,兩側是連續的平面。它不知道那是「門」——不知道門可以通過、門可能會關上、門後面通常是另一個房間。

這個差別在簡單任務中無關緊要(避開障礙不需要知道障礙是什麼),但在真實的服務任務中卻致命:

- 使用者說「去教務處」,而地圖上只有座標,沒有名字。
- 地上有一攤水,幾何上它是平坦的地面,語意上機器人應該繞開。
- 前方是玻璃門,光達看不到(第 8 章),但視覺能辨識出門框與把手。
- 走廊上站著一個人,幾何上是障礙物,語意上機器人應該減速並禮讓,而非把他當成牆壁繞過去。

三個層次的落差

層次	回答的問題	技術	本書章節
幾何	我在哪?環境形狀如何?	SLAM	第 1–30 章
語意	我看到了什麼?這裡是什麼地方?	物件偵測、分割、VLM	本章
任務	下一步該怎麼做?	LLM Agent、VLA	本章

表 31-1 機器人智慧的三個層次

這三層是疊加而非取代的關係——這是本章最重要的觀念。語意層若沒有穩固的幾何層支撐,機器人會「知道那是門,卻走不到門口」;而幾何層若沒有語意層,機器人只能執行座標指令,無法與人自然互動。

AI 帶來什麼

527. 感知的擴充:辨識物件、分割場景、估計深度——把像素轉成有意義的資訊。

528. 語言的介面:讓使用者以自然語言下達任務,而非輸入座標。

529. 常識的推理:「洗手台通常在廁所或茶水間」——這類知識無法從幾何地圖推導,卻能大幅提升探索效率(第 23.5 節)。

530. 泛化的能力:面對訓練時未見過的環境與物件,仍能做出合理判斷。

31.2 AI 視覺模型在 SLAM 中的角色

第 23.4、23.5 節介紹過 YOLO 與 VLM,本節系統性地整理各類視覺模型的分工——這是規劃 AI 感知系統的地圖。

五類模型的分工

近年的整理相當清楚地指出各家族的專長與弱點:影像文字對比模型(如 CLIP、SigLIP)是視覺語言模型的預設前端,擅長語意檢索但在稠密預測上較弱;自監督視覺變換器(如 DINOv2、DINOv3)因訓練目標保留了細緻的空間結構,在分割、深度、對應等稠密預測上表現優異,但其特徵未與語言對齊;分割基礎模型(如 SAM、SAM 2)是物件提議與追蹤的專家;稠密預測專家(如 Depth Anything、MiDaS)則處理深度。

模型類型	代表	擅長	在 SLAM 中的角色
物件偵測	YOLO	即時框出物件類別	語意標註、動態物體剔除
影像文字對比	CLIP、SigLIP	開放詞彙的語意檢索	地點辨識、語言查詢
自監督視覺	DINOv2 / v3	稠密預測、空間結構	特徵提取、分割
分割基礎模型	SAM、SAM 2	物件提議與追蹤	精確的物件邊界、動態追蹤
深度估計	Depth Anything	單張影像估深度	單目補足深度、稠密重建

表 31-2 五類 AI 視覺模型在 SLAM 中的角色

一個關鍵的工程限制

這裡有一個對機器人部署極為重要的現實:在穿戴裝置、機器人或智慧相機上的實用視訊系統,不可能為每一種能力各裝一個骨幹網路;既不能在能力上妥協,也無法承受記憶體與延遲的代價。

換句話說,雖然每個任務都有最佳的專用模型,但機器人的運算資源不允許同時跑五個大模型。實務上必須取捨——選一到兩個關鍵模型,或使用能兼顧多任務的統一骨幹。這正是第 24 章「硬體與演算法共同設計」思想在 AI 時代的延續。

對本書讀者的建議

- 入門:從 YOLO 開始。它成熟、輕量、資源豐富,能立即產生可見的成果(31.4 節的實作)。
- 進階:加入深度估計模型補足單目相機的深度(第 9.4 節),或用分割模型取得精確的物件邊界。

- 研究:探索開放詞彙模型,讓機器人能辨識訓練時未定義的物件——這是語意導航能否實用化的關鍵。

31.3 從 Visual SLAM 到 Semantic SLAM 到具身 AI 導航

本節梳理這條演進脈絡,讓同學看清自己所學的位置與前方的方向。

三個階段

- 531. 視覺 SLAM(第 19 至 22 章):輸出相機軌跡與稀疏或稠密的幾何地圖。地圖中的每個點只有座標,沒有意義。
- 532. 語意 SLAM:在建圖的同時進行物件辨識與分割,把語意標籤附加到地圖元素上。地圖中的點不再只是「有東西」,而是「這是椅子的一部分」。
- 533. 具身 AI 導航:機器人不只有語意地圖,還能理解任務、推理常識、規劃行動。它能接受「去把會議室的燈關掉」這樣的指令並自主完成。

語意 SLAM 的兩種做法

- 後處理式:先用傳統 SLAM 建好幾何地圖,再把物件偵測的結果投影上去。實作簡單、模組解耦,是教學與專題的首選(31.4 節的實作路徑)。
- 緊耦合式:把語意資訊納入 SLAM 的估測過程——例如用物件層級的關聯取代點層級的關聯,或利用「牆是平面、地板是水平」等語意先驗來約束最佳化。精度更高但實作複雜。

前沿:開放詞彙與場景圖

近年的研究正朝兩個方向推進:

- 開放詞彙線上語意建圖:讓 SLAM 系統不受預定義類別限制,能在建圖過程中辨識並標註任意物件——這對真實環境的複雜多樣至關重要。

- 場景圖表示:不只標註「這裡有一張椅子」,還記錄「椅子在桌子旁邊、桌子在會議室裡」的關係結構。近期研究已展示以多模態三維場景圖支撐零樣本的具身導航。

場景圖的價值在於它能支撐推理:當使用者說「去拿桌上的杯子」,機器人需要理解「桌上」這個空間關係——這在純幾何地圖中無法表達。

學術與產業的匯流

值得注意的是,這條演進線上,學術界與產業界正在快速匯流。相關研究已把 SLAM 與語意、三維高斯、鳥瞰視角、導航、大型語言模型、多階段作業等主題整合為同一個研究社群關注的範疇。對同學而言,這意味著:本書所學的幾何基礎,正是進入這個熱門領域的門票。

31.4 資料蒐集、標註與模型訓練

這是本章最實作導向的一節,也是大學與高中專題最容易入手的部分。

為什麼要自己蒐集資料

第 23.4 節案例 23-2 討論過的低視角問題,是核心理由:公開資料集多半以人類視角拍攝,而機器人的相機可能只有 30 至 50 公分高。訓練情境與部署情境不一致,辨識率就會大幅下降。

更廣泛地說,任何「部署環境與訓練資料分布不同」的情況都會造成效能落差。大學的走廊、教室、實驗室有其獨特的外觀與物件,一份在地的資料集因此有實質價值。

資料蒐集的規劃

534. 定義類別:先想清楚機器人需要辨識什麼。校園場景的典型類別包括:門、樓梯、電梯、飲水機、滅火器、垃圾桶、消防栓、路障、桌椅、人。類別不宜過多,先聚焦五到十類。

535. 視角一致:用實際會部署的相機、在實際的安裝高度與角度拍攝。這是最關鍵的一點。

536. 涵蓋多樣性:同一物件要拍不同角度、距離、光線(白天、夜晚、開燈、關燈)、以及部分遮蔽的情況。每類建議至少 200 至 500 張。

537. 包含困難樣本:刻意拍攝容易混淆的情況——反光的玻璃門、逆光的樓梯、堆放雜物的走廊。這些樣本對模型的穩健度貢獻最大。

538. 記錄後設資料:拍攝地點、時間、相機設定。這讓資料集日後能被正確使用與擴充。

標註的實務

- 標註工具:選用支援團隊協作的開源標註工具,能大幅提升效率並確保格式一致。
- 標註規範:團隊必須先訂好規則——半掩的門算不算「門」?框要框到哪裡?不一致的標註會讓模型學到矛盾的訊號。
- 品質抽查:標註完成後,由另一人抽查至少 10%,確認一致性。
- 分割資料:訓練、驗證、測試集應依「拍攝地點」而非隨機分割——否則同一地點的相似影像會同時出現在訓練與測試集,造成虛高的評估結果。這是初學者常犯的錯誤。

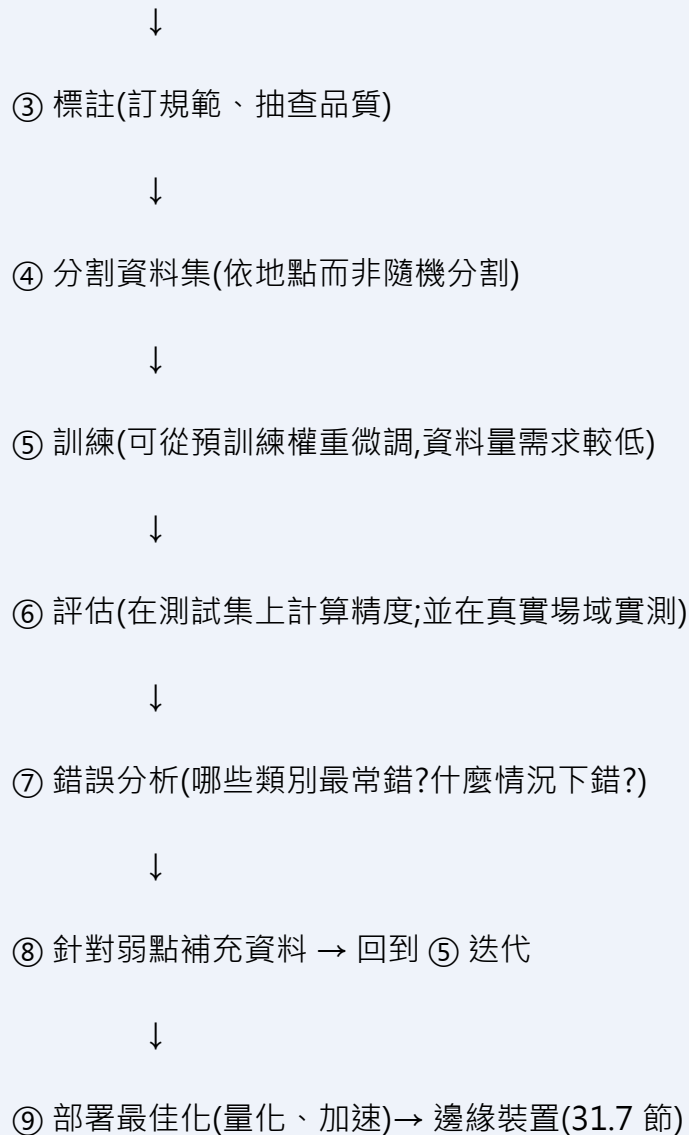
訓練與評估流程

圖 31-1 AI 模型從資料到部署的完整流程

① 定義類別與蒐集規劃



② 資料蒐集(機器人視角、多樣光線、含困難樣本)



請特別注意第七與第八步:錯誤分析與針對性補資料,往往比盲目增加訓練輪數更有效。這是實務上最重要的迭代循環。

31.5 VLM 語言導航與 RAG 校園知識庫

第 23.5 節介紹過 VLM 導航的概念,本節聚焦兩個實作面向:如何讓機器人聽懂指令,以及如何讓它知道校園的空間知識。

從語言到導航目標

當使用者說「請去教務處」,系統需要完成一連串轉換(第 23 章圖 23-2):理解指令的目標、在語意地圖中查詢該地點、若找不到則以常識推理選擇探索方向、最後轉為幾何座標交給導航系統。

VLM 在其中扮演兩個角色:語言理解(把口語轉為結構化的目標)與視覺推理(判斷眼前畫面是否符合目標,或該往哪個方向探索)。

RAG:讓機器人讀懂校園知識

這裡有一個實務問題:VLM 再聰明,也不知道「大學的教務處在哪一棟哪一樓」——這是特定機構的知識,不在模型的訓練資料中。

解法是檢索增強生成(RAG):建立一份校園空間知識庫,當機器人收到指令時,先從知識庫檢索相關資訊,再交給 VLM 推理。

圖 31-2 校園空間知識庫的範例結構

【樓層拓樸】

一樓:校門口 → 穿堂 → 教務處 → 學務處 → 樓梯 → 電梯

二樓:電梯 → 走廊 → 電腦教室 → 會議室

【地點屬性】

教務處:一樓、穿堂東側、開放時間 08:00–17:00、門為玻璃自動門

智慧機器人教室:三樓、入口 → 工作桌 → 機器狗測試區 → 光達建圖區 → 充電區

【通行規則】

機器人不可進入:實驗室內部、樓梯(除非為爬樓梯平台)

需減速區域:穿堂、教室門口、轉角

【常見查詢對應】

「辦公室」→ 教務處、學務處 「充電」→ 各樓層充電站

RAG 的三個優點

- 可維護:校園配置改變時,只需更新知識庫,不需重新訓練模型。
- 可驗證:機器人的回答可以追溯到知識庫的哪一條,便於除錯與問責。
- 低成本:相較於微調大模型,建立知識庫的成本低得多,也更適合教學專題。

三個實用的查詢範例

539. 「從入口到機器狗測試區怎麼走?」——RAG 檢索到智慧機器人教室的內部拓樸,VLM 據此生成路線描述,再轉為導航目標序列。
540. 「如果前方有學生坐在地上,機器人應該如何繞路?」——結合即時感知(YOLO 偵測到人)與知識庫的通行規則(需減速、禮讓),產生繞行策略。
541. 「掃地機器人如何規劃覆蓋整間教室?」——結合地圖(第 13 章)與覆蓋規劃(第 16 章),這是語意層與幾何層協作的典型例子。

務實的架構建議

對專題實作而言,建議的分工是:RAG 知識庫用結構化的文字檔或簡單資料庫即可(不必一開始就上向量資料庫);VLM 負責語言理解與視覺判斷;而所有涉及安全的決策(避障、速度限制)仍由傳統的幾何系統把關——這是下一節與 31.8 節反覆強調的原則。

31.6 LLM Agent 與 SLAM 系統整合

本節說明如何把 AI 與傳統 SLAM 系統組裝成一個完整的架構。

分層架構:各司其職

圖 31-3 AI 輔助 SLAM 導航的分層架構

【任務層】秒級以上 LLM / VLM Agent

- 理解自然語言指令、查詢 RAG 知識庫
- 分解任務、選擇探索方向、判讀異常

↓ 輸出:語意目標(去哪、找什麼)

【語意層】0.1–1 秒 視覺 AI 模型

- YOLO 物件偵測、分割、深度估計
- 語意地圖的建立與查詢

↓ 輸出:幾何目標座標、動態物體標註

【幾何層】10–30 Hz SLAM + 導航(第 1–30 章)

- 定位建圖、路徑規劃、代價地圖

↓ 輸出:速度指令

【控制層】100 Hz 以上 底盤 / 飛控

- 運動控制、即時避障、安全停止

【安全原則】上層可以出錯,下層必須守住安全底線

這個分層與第 3 章的混合式架構、第 15 章 Cartographer 的雙子系統、第 18 章 LOAM 的雙層設計、第 25 章無人機的降級架構,是同一個系統設計模式在 AI 時代的延伸——依時間尺度分層、各層各司其職。同學至此應該能深刻體會:好的架構原則具有跨時代的普遍性。

AI Agent 的三種角色

第 26.6 節提過的三種角色,在單機系統中同樣適用:

- 任務規劃者:把高階目標分解為可執行的子任務序列。
- 異常判讀者:從大量感知資料中篩選出需要人類注意的事件。
- 人機介面:以自然語言與使用者溝通,取代複雜的操作介面。

整合的實務要點

542. 非同步設計:AI 推論耗時,絕不能阻塞導航迴路。應以非同步方式呼叫,並在等待期間讓機器人維持安全行為(繼續執行上一個目標或停下)。
543. 明確的介面:AI 層輸出的應該是結構化的資料(目標座標、物件清單),而非自由文字。這讓下游系統能可靠地解析與驗證。
544. 可否決機制:幾何層應能否決 AI 的決策。若 AI 說「往前走」但代價地圖顯示前方是障礙,幾何層必須勝出。
545. 降級路徑:AI 服務不可用時(斷線、逾時、模型崩潰),系統應退回純幾何導航,而非停擺——這正是第 25.2 節「失效逐級降級」的原則。

31.7 邊緣 AI 部署:Jetson、量化與蒸餾

本節處理一個現實問題:大模型跑不動在機器人上,怎麼辦?

問題的規模

視覺語言模型的運算需求通常使它們只能在雲端執行,阻礙了在資源受限機器人上的本地低延遲推論。而雲端執行又帶來第 24.1 節討論過的所有問題:依賴網路、延遲、頻寬與可靠度。

三大關鍵技術

546. 量化:以較少的位元表示模型參數。目前邊緣端的視覺語言模型與視覺語言動作模型,預設採用 4 位元權重搭配 16 位元啟動值的組態;而 4 位元權重加 4 位元啟動值的格式則能解鎖新一代硬體平台,是支援時的產品級升級選擇;8 位元權重加 8 位元啟動值仍是成熟視覺與分割模型較安全的備選。

547. 蒸餾:把大模型的能力轉移到小模型。有研究提出蒸餾策略,把大型前沿模型的複雜空間語意推理轉移到輕量的 40 億參數本地模型上,以在嵌入式 GPU 裝置(如 Jetson Orin)上執行;實驗顯示,以約 500 條前沿模型的推理軌跡微調小模型,即能有效賦予導航能力,成功率達 34.5%,相較大型雲端模型的 39.7% 已相當接近。

548. 記憶體管理:這是常被忽略卻極關鍵的一環。有研究在 Jetson Orin Nano 上同時執行 SLAM 與 VLM,發現以幾何為基礎的地圖管理策略會發生記憶體不足的失敗與執行停滯;改採語意分區的關鍵幀載入卸載機制後,不僅消除了這兩個問題、維持了定位穩定性,還使吞吐量提升並降低了 21.7% 的延遲。

對專題的啟示

第三點特別值得同學注意:當你在 Jetson 上同時跑 SLAM 與 AI 模型時,瓶頸往往不是運算速度,而是記憶體。SLAM 的地圖會持續成長,AI 模型佔用大量記憶體,兩者相爭的結果就是系統崩潰。

- 對策一:限制地圖規模。依語意分區只載入當前需要的區域(這正是上述研究的做法,也呼應第 22 章 Atlas 的多地圖思想)。

- 對策二:選用量化後的小模型。第 31.2 節提過,不必為每個任務各裝一個大模型。
- 對策三:錯開執行。AI 推論不需要每幀都跑——每秒一到兩次通常已足夠支撐語意層的決策。

部署平台的選擇

- 入門:先在桌上型電腦或筆電上驗證整套流程,確認邏輯正確。
- 進階:移植到嵌入式 GPU 平台,此時開始面對記憶體與功耗的限制。
- 極限:第 24 章的毫瓦級平台。此時只能跑高度最佳化的小模型,但正如第 24 章所示,YOLO 等推論任務已能在百毫瓦以下完成。

31.8 風險、限制與未來發展

本書的最後一節,談三件事:AI 輔助 SLAM 的風險、當前的限制、以及未來的方向。

四大風險

549. 幻覺:AI 可能「看到」不存在的東西或做出錯誤推理,而且它通常不會表達不確定。若機器人完全信任 AI,可能撞牆或走錯路。對策是保留幾何層的驗證與否決權(31.6 節)。
550. 誤判的系統性:AI 的錯誤往往不是隨機的,而是有系統的偏差——例如在特定光線下總是把某類物件認錯。這比隨機錯誤更危險,因為它會持續發生。對策是充分的場域測試與錯誤分析(31.4 節)。
551. 延遲:大模型推論耗時,若放在即時控制迴路中會造成災難。對策是嚴格的分層架構——AI 只做秒級以上的決策(31.6 節)。

552. 安全性:包含資安(模型或知識庫被竄改)、隱私(機器人拍攝的影像可能含個人資訊)、以及可問責性(出事時能否追溯決策依據)。這些不只是技術問題,也是倫理與法規問題。

當前的限制

- 資料需求:訓練或微調模型需要標註資料,而標註成本高昂。這是專題與小型團隊的主要障礙——也是為何蒸餾與少量樣本微調如此受重視。
- 泛化的落差:模型在實驗室表現良好,換個場域就下降。第 23.4 節的低視角問題只是其中一例。
- 可解釋性:AI 為什麼做出這個判斷?當它出錯時,往往難以診斷。這與傳統幾何方法「每一步都可追溯」形成鮮明對比。
- 運算與能源:如 31.7 節所述,即使經過量化與蒸餾,AI 仍是機器人上最耗資源的元件之一。

未來的四個方向

553. SLAM 與基礎模型的深度整合:不再是「SLAM 加上 AI 模組」,而是統一的架構——由基礎模型同時處理感知、建圖與規劃。
554. 世界模型:讓機器人不只理解「現在是什麼」,還能預測「接下來會發生什麼」。這對動態環境中的規劃至關重要。
555. 機器人基礎模型:第 23.5 節提過的跨平台通用代理——同一套模型能部署到輪型、四足、人形、飛行等不同平台。這將大幅降低機器人開發的門檻。
556. 終身學習:機器人在部署後持續學習與適應,而非停留在出廠時的能力。第 24.6 節提過的裝置端自監督學習是這個方向的早期嘗試。

給同學的最後幾句話

本書從第 1 章的「機器人沒有地圖,也不知道自己在哪裡」開始,走過感測器、數學、

2D 與 3D 系統、視覺方法、微型與群體機器人、完整實作,最後來到 AI 語意導航。

如果要說這三十一章有一個貫穿的主線,那就是:理解限制,才能突破限制。

我們反覆討論每種感測器的弱點(第二篇)、每種演算法的失敗模式(第四至六篇)、每個平台的資源約束(第七篇)、以及每個系統的除錯順序(第八篇)。這不是悲觀,而是因為工程的本質就是在限制中求解。

AI 的加入沒有改變這個本質。VLM 讓機器人「聽得懂、想得通」,但「走得穩、不撞人」仍然要靠前面三十章的幾何與控制。兩者缺一不可——而且可預見的未來也將如此。

最後,本書的兩個場域——大學智慧機器人學程與高中 AI 機器人社團課程——代表了兩種學習階段,但共享同一個信念:SLAM 不是遙不可及的尖端科技。它的核心版本可以跑在一顆手機等級的處理器上,它的原理可以用紙筆推演,它的成果就在你家客廳的掃地機器人裡默默運轉。

願同學帶著這本書打下的基礎,去建造屬於自己的機器人——無論它是巡檢校園的機器狗、掃描古蹟的無人機,還是一台只會沿線走的小車。每一台能自己找到路的機器人,背後都是同一套人類花了四十年才建立起來的智慧。

現在,輪到你了。

公式或流程

一、物件偵測的評估指標

訓練 YOLO 等偵測模型時,最常用的指標是平均精度均值:

$$mAP = (1 / C) \sum_c AP_c$$

其中 C 為類別數、AP_c 為第 c 類的平均精度(精確率對召回率曲線下的面積)。常

見的變體是以交併比門檻 0.5 計算的 $mAP@0.5$, 以及在多個門檻上平均的 $mAP@0.5:0.95$ (較嚴格)。

對機器人應用而言,還應關注兩個實務指標:

- 漏檢率:沒偵測到的比例。對安全相關的物件(人、樓梯口),漏檢比誤檢危險得多。
- 每類別的表現:總體 mAP 高不代表每一類都好。若「樓梯」這一類的精度特別低,而它正是安全關鍵,那麼整體數字就有誤導性。

二、把偵測結果投影到三維地圖

語意標註的核心運算,是把影像上的偵測框轉為三維座標。以偵測框中心像素(u, v)與其深度 d 為例,依第 29 章公式一反投影:

$$P_{camera} = ((u - cx) \cdot Z / fx, (v - cy) \cdot Z / fy, Z) \quad , \quad Z = d / scale$$

再依第 10 章的座標轉換鏈,轉到地圖座標系:

$$P_{map} = T_{\{map \leftarrow odom\}} \cdot T_{\{odom \leftarrow base\}} \cdot T_{\{base \leftarrow camera\}} \cdot P_{camera}$$

這條式子把本章的 AI 與第 10 章的座標轉換連在一起——語意標註看似是 AI 的工作,但它能否正確落在地圖上,取決於外參與定位的品質。這再次印證 31.1 節的觀點:語意層必須建立在穩固的幾何層之上。

三、量化的記憶體節省估算

模型量化後的記憶體需求約為:

$$記憶體 \approx 參數量 \times 每參數位元數 / 8 \quad (位元組)$$

以 40 億參數的模型為例:16 位元精度需約 8 GB、8 位元約 4 GB、4 位元約 2 GB。

這解釋了為何 4 位元量化能讓大模型擠進嵌入式平台的記憶體——差距是四倍。

但要注意:實際記憶體用量還包含啟動值、中間結果與框架開銷,通常是上述估算的 1.2 至 2 倍。規劃時務必預留餘裕(31.7 節的記憶體管理討論)。

四、AI 輔助 SLAM 的完整資料流

圖 31-4 從感測到語意決策的端到端流程

① 感測器:相機、光達、IMU



② 幾何 SLAM:定位與建圖(第 13–22 章)→ 位姿 + 幾何地圖



③ AI 感知:YOLO 偵測 / 分割 / 深度估計(31.2 節)

↓ 依公式二把偵測結果投影到地圖座標

④ 語意地圖:幾何 + 物件標籤 + 空間關係



⑤ RAG 知識庫:校園拓樸、地點屬性、通行規則(31.5 節)



⑥ VLM / LLM Agent:理解指令、推理、選擇目標(31.6 節)

↓ 輸出結構化的幾何目標

⑦ 路徑規劃與控制(第 28 章)→ 速度指令



⑧ 執行 → 環境改變 → 回到 ①

【安全把關】步驟 ⑦ 有權否決 ⑥ 的決策

案例

案例 31-1 把大模型裝進機器人:蒸餾的威力

這是本章最能說明「工程智慧」的案例,也直接回應了同學可能有的疑問:大模型跑不動,是不是就只能放棄?

問題的背景很清楚:視覺語言模型在機器人領域已成為強大的工具,能以語言脈絡理解環境,催化了物件目標導航這類開放詞彙任務;但它們的運算量通常使其只能在雲端執行。

研究團隊的做法是蒸餾:先以大型前沿模型建立一套基於場景圖的高水準管線,在標準的開放詞彙物件導航基準上達到 39.7% 的成功率;接著把這套推理能力轉移到僅 40 億參數的本地模型上,以便在嵌入式 GPU 裝置上執行。

關鍵的結果是:僅以約 500 條前沿模型的推理軌跡微調小模型,就能有效賦予它導航能力,成功率達到 34.5%,大幅縮小了與大型雲端模型之間的差距。

五百條資料——這個數字值得同學細細品味。相較於從零訓練所需的數十萬筆標註,這是極小的成本。而換來的是「能在機器人上跑」這個質變:不需要網路、沒有雲端延遲、資料不外流。

這個案例的方法論啟示,與第 24 章 NanoSLAM 的案例 24-1 完全一致:效能瓶頸往往不在算力不足,而在方法未經最佳化。當你覺得「這個技術太重、跑不動」時,先問的不該是「要不要放棄」,而是「有沒有更聰明的做法」。

案例 31-2 記憶體才是真正的瓶頸

這是一個許多同學在專題後期才會撞上的問題,提前認識能省下大量挫折。

情境是這樣的:同學在 Jetson 上成功跑起了 SLAM,也成功跑起了 YOLO,但一旦兩者同時執行,系統就變得極度緩慢,甚至直接崩潰。檢查 CPU 與 GPU 使用率,卻發現都沒有滿載——這讓人困惑。

真正的瓶頸是記憶體。相關研究對這個現象有清楚的量化:在 Jetson Orin Nano 上同時執行 SLAM 與 VLM 時,以幾何為基礎的地圖管理策略會遭遇記憶體不足的失敗與執行停滯;而改採語意分區的關鍵幀載入卸載機制後,不僅消除了這兩個問題並維持定位穩定性,還提升了吞吐量並降低了 21.7% 的延遲。

問題的根源是:SLAM 的地圖會隨探索持續成長(第 13.5 節的記憶體估算、第 14 章公式五的粒子數乘地圖大小),而 AI 模型本身就佔用大量記憶體。兩者相爭,系統必然崩潰。

解法的巧妙之處在於「語意分區」:機器人不需要隨時記得整棟樓的細節,只需載入當前所在區域的地圖。這個思想與第 22 章 Atlas 的多地圖、第 15 章 Cartographer 的子地圖一脈相承——把大問題切成可管理的小塊。

對專題的具體建議:在規劃階段就估算記憶體需求(公式三),並優先考慮限制地圖規模的機制。等到系統崩潰才處理,往往需要大幅重構。

案例 31-3 從第 1 章到第 31 章:一台機器人的完整旅程

作為全書的最後一個案例,讓我們用一次完整的任務,把三十一章的內容串起來。

情境:大學的服務機器人接到指令——「請去三樓的智慧機器人教室,檢查充電區的設備狀態,並回報異常」。

- 它先理解指令:VLM 解析出目標地點與任務內容(31.5、31.6 節)。
- 查詢知識庫:RAG 檢索出「智慧機器人教室在三樓、內部拓樸為入口→工作桌→機器狗測試區→光達建圖區→充電區」(31.5 節)。
- 規劃路線:在既有地圖上規劃從當前位置到三樓的路徑,包含搭電梯(第 3、28 章)。
- 移動與定位:光達 SLAM 或視覺慣性 SLAM 持續估計位姿(第 13–22 章),AMCL 在既有地圖中定位(第 11、28 章),IMU 在轉彎與震動時提供支撐(第 7 章)。
- 通過電梯口:看到 AprilTag,取得絕對位姿校正並確認樓層(第 4、23 章)。
- 避開行人:YOLO 偵測到人,一方面剔除其特徵以免干擾定位,一方面觸發減速禮讓(第 23、31 章)。
- 抵達充電區:依語意地圖找到目標,以視覺伺服完成精確對位(第 23 章)。
- 檢查與判讀:拍攝設備影像,AI 判斷是否異常,並把發現標上三維座標(31.4、31.6 節)。
- 回報:以自然語言生成報告,附上位置與影像證據(31.6 節)。

這一趟任務中,本書的每一篇都有貢獻:第一篇的觀念、第二篇的感測器、第三篇的數學、第四至六篇的 SLAM 系統、第七篇的邊緣運算、第八篇的實作工具、第九篇的 AI 語意。

而最關鍵的是:這九篇缺一不可。少了幾何層,機器人會撞牆;少了語意層,它聽不懂指令;少了工具鏈,一切都無法運行。這正是本書從第一頁到最後一頁想傳達的訊息——SLAM 不是單一技術,而是一整套互相支撐的知識體系。

實作活動

實作 31-1 校園語意導航系統

目的:完成本書的最終實作——建立校園物件辨識模型、產生語意地圖、並實現自然語言導航。本實作分四部分,可依課程層次選擇進行到哪一階段。

第一部分:校園物件辨識資料集(高中適用,本章核心)

351. 定義類別:與同學討論並決定五到十個校園常見物件(建議:教室門、白板、講桌、椅子、樓梯、電梯、走廊、垃圾桶、消防栓、飲水機、路障)。
352. 視角規劃:決定拍攝高度與角度,務必模擬機器人的實際視角(30 至 50 公分高),而非站著拍——這是本實作最重要的細節(第 23.4 節案例 23-2)。
353. 分組蒐集:每組負責二至三類,每類拍攝至少 200 張,涵蓋不同角度、距離、光線與遮蔽情況,並刻意包含困難樣本。
354. 標註:先訂定標註規範(半掩的門算不算?框到哪裡?),再進行標註,並互相抽查至少 10%。
355. 分割資料集:依「拍攝地點」而非隨機分割,避免評估結果虛高(31.4 節的提醒)。
356. 訓練 YOLO 模型,在測試集上評估 mAP,並針對表現最差的類別做錯誤分析。

第二部分:VLM 讀圖導航判斷(全體)

357. 蒐集機器人視角的影像數十張,涵蓋各種情境(可通行的走廊、被堵住的通道、樓梯口、玻璃門)。
358. 對每張影像,向 VLM 提出四個問題:「前方是否可通行?」「畫面中有沒有樓梯?」「哪裡可能是門?」「機器人應該往左、往右,還是停止?」
359. 記錄回答並人工評分正確率,分析 VLM 在哪類情境最容易出錯。

360. 對照實驗:同樣的影像交給你訓練的 YOLO 模型,比較兩者的能力差異——

YOLO 對已定義類別更準,VLM 則能回答未定義的問題。

361. 延遲量測:記錄 VLM 的推論時間,依 31.6 節的分層架構判斷它適合放在哪一

層。

第三部分:SLAM 地圖 + 校園知識 RAG(大學部)

362. 依圖 31-2 的結構,建立一份大學場域的空間知識庫,至少涵蓋一個樓層的拓

樸、地點屬性與通行規則。

363. 把第 28 或 29 章建好的地圖與知識庫連結:為地圖上的關鍵位置標上名稱與屬

性。

364. 實作查詢:讓 LLM 依據知識庫回答三個問題——「從入口到機器狗測試區怎

麼走?」「如果前方有學生坐在地上,機器人應該如何繞路?」「掃地機器人如何

規劃覆蓋整間教室?」

365. 評估回答品質:答案是否正確?是否可追溯到知識庫的哪一條?若知識庫沒有相

關資訊,系統是否誠實表示不知道(而非產生幻覺)?

第四部分:端到端語意導航(專題)

366. 整合:把第一部分的 YOLO 模型接上第 28 或 29 章的 SLAM 系統,把偵測結果

依公式二投影到地圖,產生語意地圖。

367. 語言介面:實作一個節點,接受自然語言指令(如「去飲水機」),經 RAG 與 VLM

轉為幾何目標,交給 Nav2 執行。

368. 安全驗證:刻意讓 AI 產生錯誤的目標(例如指向牆內),驗證幾何層是否正確否決

(31.6 節的可否決機制)。

369. 降級測試:斷開 AI 服務,驗證系統是否退回純幾何導航而非停擺。

370. 完整任務展示:設計一個包含「理解指令 → 導航 → 辨識 → 回報」的完整任務並錄影,作為專題成果。

討論問題

149. 你的模型在機器人視角與人類視角的測試影像上,mAP 差多少?這印證了本章的哪一項論述?
150. 哪一類物件最難辨識?為什麼?(反光?尺寸多變?樣本不足?)你會如何改善?
151. VLM 與 YOLO 在你的測試中各有什麼優勢?若要設計一個實用系統,你會如何分工使用兩者?
152. RAG 系統在知識庫沒有相關資訊時,表現如何?這對「AI 幻覺」的風險(31.8 節)有什麼啟示?
153. (專題)在安全驗證中,幾何層成功否決了 AI 的錯誤決策嗎?若沒有,你會如何修改架構?

延伸挑戰

其一:開放資料集貢獻——把你們蒐集的「大學機器人視角校園物件資料集」整理成標準格式(含說明文件、類別定義、拍攝條件紀錄)並公開。這類在地資料集對後續學生、其他學校的研究者都有實質價值,也是本書第 25、27 章反覆建議的長期貢獻方式。其二:邊緣部署最佳化——把訓練好的模型量化後部署到 Jetson 等嵌入式平台,與 SLAM 同時執行,量測記憶體用量、延遲與吞吐量,並依 31.7 節的三種對策嘗試改善,重現案例 31-2 的實驗。這是一個兼具工程深度與研究價值的題目,也是本書技術路線的完美收尾。

延伸閱讀

209. N. Yokoyama et al., "VLFM: Vision-Language Frontier Maps for Zero-Shot Semantic Navigation," ICRA, 2024.(零樣本語意導航的代表作,第 23、31 章共同參考)
210. T. B. Martins, M. R. Oswald, and J. Civera, "Open-Vocabulary Online Semantic Mapping for SLAM," IEEE RA-L, 2025.(開放詞彙語意建圖,31.3 節的前沿方向)
211. D. Shah et al., "LM-Nav: Robotic Navigation with Large Pre-trained Models of Language, Vision, and Action," CoRL, 2022.(語言模型驅動導航的開創性工作)
212. Embodied AI with Foundation Models for Mobile Service Robots: A Systematic Review, arXiv, 2025.(具身智慧與基礎模型的完整綜述,適合作為研究起點)
213. Ultralytics YOLO 官方文件(docs.ultralytics.com).(物件偵測模型的訓練、評估與部署,本章實作的主要工具)
214. awesome-and-novel-works-in-slam 開源清單(GitHub).(語意 SLAM、三維高斯、導航、大型語言模型等新方向的持續更新整理)
215. NVIDIA Jetson 開發者文件與 TensorRT 最佳化指南.(邊緣 AI 部署的實務參考,31.7 節的技術基礎)

小結

182. 傳統 SLAM 知道形狀卻不知道意義;機器人智慧可分幾何、語意、任務三個層次,三者疊加而非取代——語意層若無穩固的幾何層支撐,機器人會「知道那是門卻走不到門口」。

183. 五類 AI 視覺模型各有專長:影像文字對比模型擅長語意檢索但稠密預測較弱、自監督視覺模型擅長稠密預測但未與語言對齊、分割基礎模型專精物件提議與追蹤、深度模型處理深度;但機器人不可能為每種能力各裝一個骨幹,必須取捨。
184. 演進脈絡為視覺 SLAM → 語意 SLAM → 具身 AI 導航;語意 SLAM 有後處理式與緊耦合式兩種做法,前沿則朝開放詞彙線上建圖與場景圖表示發展。
185. 自建資料集的關鍵是視角一致(用實際部署的相機與高度)、涵蓋多樣性與困難樣本、以及依拍攝地點而非隨機分割資料集;訓練後的錯誤分析與針對性補資料,比盲目增加訓練輪數更有效。
186. VLM 負責語言理解與視覺推理,RAG 則提供機構特定的空間知識;RAG 的三個優點是可維護、可驗證、低成本,特別適合教學專題。
187. AI 輔助 SLAM 應採分層架構:任務層(秒級,LLM/VLM)、語意層(0.1–1 秒,視覺 AI)、幾何層(10–30 Hz,SLAM 與導航)、控制層(100 Hz 以上);上層可以出錯,下層必須守住安全底線,且幾何層有權否決 AI 決策。
188. 邊緣部署的三大技術為量化(4 位元權重是目前邊緣 VLM 的預設)、蒸餾(以約 500 條推理軌跡微調 40 億參數模型即可達到接近雲端大模型的導航成功率)、以及記憶體管理(語意分區的關鍵幀載入卸載能消除記憶體不足並降低 21.7% 延遲)。
189. 四大風險為幻覺、系統性誤判、延遲與安全性;當前限制包含資料需求、泛化落差、可解釋性與運算能源;未來方向則是 SLAM 與基礎模型的深度整合、世界模型、機器人基礎模型與終身學習。

《地圖建構與同步定位》全書至此完成。從第 1 章「機器人沒有地圖,也不知道自己在哪裡」的核心問題出發,我們走過了感測器、數學、2D 與 3D 系統、視覺方法、微

型與群體機器人、完整實作,最後來到 AI 語意導航。

貫穿這三十一章的主線是:理解限制,才能突破限制。每一種感測器都有弱點、每一個演算法都有失敗模式、每一個平台都有資源約束——而工程的本質,正是在這些限制中求解。

AI 的加入沒有改變這個本質。VLM 讓機器人聽得懂、想得通,但走得穩、不撞人,仍然要靠前面三十章的幾何與控制。這是本書最後想留給同學的認識。

現在,輪到你了。